

DEEP LEARNING FOR 3D VISION

Algorithms and Applications



Edited by

Xiaoli Li

A*STAR, Singapore

Xulei Yang

A*STAR, Singapore

Hao Su

UC San Diego, USA

 **World Scientific**

NEW JERSEY • LONDON • SINGAPORE • BEIJING • SHANGHAI • HONG KONG • TAIPEI • CHENNAI • TOKYO

Published by

World Scientific Publishing Co. Pte. Ltd.

5 Toh Tuck Link, Singapore 596224

USA office: 27 Warren Street, Suite 401-402, Hackensack, NJ 07601

UK office: 57 Shelton Street, Covent Garden, London WC2H 9HE

Library of Congress Control Number: 2023053648

British Library Cataloguing-in-Publication Data

A catalogue record for this book is available from the British Library.

DEEP LEARNING FOR 3D VISION

Algorithms and Applications

Copyright © 2024 by World Scientific Publishing Co. Pte. Ltd.

All rights reserved. This book, or parts thereof, may not be reproduced in any form or by any means, electronic or mechanical, including photocopying, recording or any information storage and retrieval system now known or to be invented, without written permission from the publisher.

For photocopying of material in this volume, please pay a copying fee through the Copyright Clearance Center, Inc., 222 Rosewood Drive, Danvers, MA 01923, USA. In this case permission to photocopy is not required from the publisher.

ISBN 978-981-12-8648-3 (hardcover)

ISBN 978-981-12-8649-0 (ebook for institutions)

ISBN 978-981-12-8650-6 (ebook for individuals)

For any available supplementary material, please visit

<https://www.worldscientific.com/worldscibooks/10.1142/13683#t=suppl>

Desk Editors: Soundararajan Raghuraman/Amanda Yun

Typeset by Stallion Press

Email: enquiries@stallionpress.com

Printed in Singapore

Preface

3D deep learning is a rapidly developing field with tremendous research value and potential real-world applications. As we live in a 3D world, 3D deep learning is a natural way to perceive and understand our environment, enabling emerging and new industrial applications, such as autonomous driving, robot perception, medical imaging, and scientific simulations, among others.

Although deep learning has proven to be a powerful tool for building models for one-dimensional (language) and two-dimensional (image) understanding, it is still in its early stages for the study of 3D deep learning. Most of the current works focus on extending or adapting 2D technology into 3D, which can limit the performance of developed deep learning models as they are not tailored for 3D data.

3D data has its own unique characteristics, such as large-scale, sparse, irregular, unstructured, and containing rich geometry information. These characteristics present unique challenges for the research and adoption of 3D deep learning and therefore greatly limit its real-world applications. Although researchers have made significant strides in 3D deep learning, there is still much to explore in the future.

This book collates the most recent research advances in 3D deep learning, including algorithms and applications, with a focus on efficient methods to tackle the key technical challenges that occurred in current 3D deep learning research and adoption.

Session I: Sample Efficient 3D Deep Learning — this session includes Chapter 2, which investigates self-training for weakly supervised 3D scene understanding, and Chapter 3, which presents masked autoencoders for 3D point cloud self-supervised learning. These chapters explore ways to build 3D models with limited annotated samples.

Session II: Representation Efficient 3D Deep Learning — this session includes Chapter 4, which explores various methodologies for modelling and learning representations in dynamic 3D scenes, Chapter 5, which presents eDiGS (Extended Divergence guided shape implicit neural representation) for unoriented point clouds, and Chapter 6, which introduces a depth-aware monocular 3D object detector by synthetic images with virtual depth. These chapters explore efficient methods to handle heterogeneous 3D structures.

Session III: Robust 3D Deep Learning — this section includes Chapter 7, which proposes a novel bi-level optimisation framework for robust 3D point cloud classification, and Chapter 8, which introduces a test-time anchored clustering approach under a realistic sequential test-time training protocol. These chapters explore robust deep learning methodology for real-world applications with imperfect data, adversarial attacks, and complicated scenes.

Session IV: Resource Efficient 3D Deep Learning — this session includes Chapter 9, which explores algorithm-system-hardware co-design for efficient 3D deep learning, and Chapter 10, which presents sampling strategies for efficient segmentation and object detection of point clouds. These chapters explore ways to reduce the number of parameters and computation cost of 3D models.

Session V: Emerging 3D Deep Learning Applications — this session includes Chapter 11, which focuses on efficient 3D representation learning for medical image analysis, and Chapter 12, which proposes a semi-supervised learning for semiconductor buried structure detection and segmentation. These chapters demonstrate how 3D deep learning enables real-world applications.

3D deep learning is a rapidly evolving field that has the potential to transform various industries. This book aims to provide a comprehensive overview of the current state-of-the-art in 3D deep learning, covering a wide range of research topics and applications.

By presenting the latest research advances, this book can serve as a valuable resource for researchers and practitioners interested in exploring the potential of 3D deep learning. We hope that this book will not only contribute to the advancement of 3D deep learning but also inspire further research and create more real-world impact in this exciting field.

This page intentionally left blank

About the Editors



Li Xiaoli is currently the Head of the Machine Intellection Department (consisting of 130+ AI and data scientists, which is Singapore's largest AI and data science group) and a senior principal scientist at the Institute for Infocomm Research, A*STAR, Singapore. He also holds the adjunct professor position at Nanyang Technological University, Singapore. He is also serving as co-director of KPMG-I2R's joint lab. He has been a member of ITSC

(Information Technology Standards Committee) from Enterprise Singapore and Singapore's Infocomm Media Development Authority (IMDA) since 2020. His research interests include data mining, machine learning, artificial intelligence, and bioinformatics. He has been serving as the area chairs/senior PC members/workshop chairs/section chairs in leading AI/data mining/machine learning-related conferences (including KDD, ICDM, SDM, PKDD/ECML, ACML, PAKDD, WWW, IJCAI, AAAI, ACL, and CIKM). Dr. Li has edited over six book editions. His papers have more than 28K citations. Dr. Li has an h-index of 69.



Yang Xulei is a principal scientist from the Institute for Infocomm Research (I2R), A*STAR. He received his Ph.D. in Electrical and Electronics Engineering from Nanyang Technological University, Singapore. His research interests focus on computer vision, deep learning, machine learning, and video analysis. He is the principal investigator for various research and industrial projects involved in providing deep learning solutions for real-world computer vision and healthcare applications. He has published more than 100 scientific papers. Dr. Yang has rich industrial experiences and insights on the transformation of advances in AI into real-world products. He had worked in industry for nearly 10 years. He was previously the Research Head of the AI start-up giant YITU Technology Singapore, leading the R&D team developing AI solutions for practical applications in smart nation initiatives and public safety. He has filed 18 international and local patents. Dr. Yang is currently an IEEE Senior Member and Kaggle Competition Master.



Su Hao is an assistant professor of Computer Science and Engineering at the University of California San Diego, USA. He is interested in fundamental problems in broad disciplines related to artificial intelligence, including machine learning, computer vision, computer graphics, and robotics. His most recent work focuses on integrating the disciplines for building and training embodied AI that can interact with the physical world. In the past, his work on ShapeNet, PointNet series, and graph neural networks has significantly impacted the emergence and growth of a new field: 3D deep learning. He also participated in the development of ImageNet, a large-scale 2D image database. He has served as the Area Chair, Associate Editor, and other comparable positions in the program committee of CVPR, ICCV, ECCV, ICRA, Transactions on Graphics (TOG), and AAAI. His papers have more than 85K citations. Dr. Hao has an h-index of 58.

Contents

<i>Preface</i>	v
<i>About the Editors</i>	ix
1. Introduction to 3D Deep Learning <i>Xiaoli Li, Xulei Yang, and Hao Su</i>	1
2. Masked Autoencoders for 3D Point Cloud Self-Supervised Learning <i>Yatian Pang, Zhenghua Chen, and Li Yuan</i>	27
3. You Only Need One Thing One Click: Self-Training for Weakly Supervised 3D Scene Understanding <i>Zhengzhe Liu, Xiaojuan Qi, and Chi-Wing Fu</i>	57
4. Representation Learning for Dynamic 3D Scenes <i>Yunzhu Li and Jiajun Wu</i>	91
5. eDiGS: Extended Divergence-Guided Shape Implicit Neural Representation for Unoriented Point Clouds <i>Yizhak Ben-Shabat, Chamin Hewa Koneputugodage, and Stephen Gould</i>	159

6.	Improving Monocular 3D Object Detection by Synthetic Images with Virtual Depth	201
	<i>Chenhang He and Lei Zhang</i>	
7.	Robust Structured Declarative Classifiers for Point Clouds	227
	<i>Ziming Zhang, Kaidong Li, and Guanghui Wang</i>	
8.	Towards Inference Stage Robust 3D Point Cloud Recognition	257
	<i>Yongyi Su, Xun Xu, and Kui Jia</i>	
9.	Algorithm-System-Hardware Co-design for Efficient 3D Deep Learning	281
	<i>Zhijian Liu, Haotian Tang, Yujun Lin, and Song Han</i>	
10.	Sampling Strategies for Efficient Segmentation and Object Detection of 3D Point Clouds	357
	<i>Qingyong Hu</i>	
11.	Efficient 3D Representation Learning for Medical Image Analysis	399
	<i>Yucheng Tang, Jie Liu, Zongwei Zhou, Xin Yu, and Yuankai Huo</i>	
12.	AI-Based 3D Metrology and Defect Detection of HBMs in XRM Scans	439
	<i>Ramanpreet Singh Pahwa, Richard Chang, and Wang Jie</i>	
	<i>Index</i>	475

Chapter 1

Introduction to 3D Deep Learning

Xiaoli Li^{*,†}, Xulei Yang^{*,§}, and Hao Su^{†,¶}

**Institute for Infocomm Research (I²R),
A*STAR, Singapore*

*†Department of Computer Science and Engineering,
UC San Diego, USA*

‡xlli@i2r.a-star.edu.sg

§yangx@i2r.a-star.edu.sg

¶haosu@eng.ucsd.edu

3D deep learning, i.e., deep learning with 3D data, has been attracting increasing attention due to its tremendous research values and broad real-world applications. In this chapter, we first provide an overview of 3D data, its unique characteristics, and the challenges associated, as well as the wide range of applications of 3D data in various fields. We then introduce the fundamentals of 3D deep learning, including the basic concepts, the key technical challenges, the recent developments, and the emerging real-world applications of 3D deep learning. Finally, we present an overview of this book, highlighting its structure, contents, and primary contributions.

1. Introduction

For any AI-enabled agent to accomplish its task, visual understanding or perception is the first step towards interacting with the three-dimensional (3D) world. Due to its inherent limitations,

visual understanding techniques based solely on two-dimensional (2D) images may be inadequate for real-world applications. This calls for 3D deep learning techniques that operate on 3D data, which enables a direct visual understanding of the 3D world. In recent years, 3D deep learning has been attracting increasing attention. As we live in a 3D world, 3D deep learning is a natural way to perceive and understand our environment, enabling emerging and new industrial applications, such as autonomous driving, robot perception, medical imaging, and scientific simulations, and many more.

In this chapter, we provide a brief overview of the intersection between 3D data and deep learning, mainly referring to the comprehensive survey papers.^{1,2}

First, we begin with the definition of 3D data, introducing the common types of 3D data and listing the widely used 3D benchmarks. We then highlight the unique characteristics of 3D data, such as sparsity, irregularity, and high complexity. After that, we discuss the challenges involved in processing and analysing 3D data, i.e., data representation, feature extraction, and model training. Furthermore, we introduce various applications of 3D data across various industry sectors, such as manufacturing, automotive, aerospace, and healthcare.

Second, we provide an introduction to 3D deep learning, clarifying its definition and outlining its key technical challenges, which poses unique obstacles, such as limited annotated data, complex geometry representations, and computational complexity. Furthermore, we cover recent advancements in 3D deep learning techniques, such as point-based networks, convolution-based networks, graph-based networks, and transformer-based architectures. We demonstrate how these cutting-edge techniques address some of the challenges associated with 3D deep learning.

Finally, we provide an overview of this book's structure, contents, and main contributions. This book is designed to serve as a valuable reference and guide for researchers and postgraduate students in the field of 3D deep learning.

2. Introduction to 3D Data

Three-dimensional (3D) data refers to digital information that represents objects, surfaces, and environments in three dimensions, typically using the Cartesian coordinate system. Unlike two-dimensional

(2D) data, which only provides information on the x and y axes, 3D data also includes depth or z -axis information. This makes it possible to accurately represent the spatial relationships between objects and surfaces, as well as their relative positions in 3D space.

One of the key advantages of 3D data is its ability to provide a more realistic and immersive experience compared to 2D data. For example, in video games, architecture, product design, as well as virtual reality and augmented reality applications, 3D data can be used to create interactive environments that allow users to explore and interact with digital objects in a more natural and intuitive way.

Another key advantage of 3D data is its ability to enable accurate and detailed spatial analysis. With 3D data, it becomes possible to analyse and understand complex spatial relationships between objects or environments. This is particularly beneficial in fields, such as urban planning, geology, medical imaging, and engineering, where precise measurements and spatial insights are crucial. By leveraging 3D data, professionals can make more informed decisions, identify patterns or anomalies, and optimise various processes related to spatial analysis.

In recent years, advances in 3D modelling and printing technology have made it easier to create and manipulate 3D data. For example, 3D modelling can be used to create digital models of physical objects, while 3D printing can be used to create physical objects from digital models. These technologies have enabled new possibilities in fields, such as product design, manufacturing, medicine, and art. As technology continues to evolve, it is likely that 3D data will become even more pervasive and essential in many fields, driving innovation and enabling new forms of creativity.

2.1. Common Types of 3D Data

The common types of 3D data include point clouds, meshes, volumetric data, multi-view data, RGB-D scans, part assembly, and implicit functions, as shown in Figure 1.

Point clouds: Point clouds are sets of 3D points that represent the surfaces of objects or environments. They are often generated using LiDAR or photogrammetry techniques, which capture large numbers of points by bouncing laser beams or taking multiple photos



Fig. 1. Common types of 3D data representation. The picture is taken from 3D deep learning tutorial.³

from different angles. Point clouds can be used to create accurate 3D models of real-world objects, as well as for applications, such as mapping, surveying, and 3D scanning.

Meshes: Meshes are composed of interconnected polygons (usually triangles) that define the surfaces of 3D objects. They are often used in computer graphics and game development to create complex 3D models, as well as in engineering and architecture for simulations and virtual prototyping. Meshes can also be used for 3D printing, where they are converted into physical objects by additive manufacturing techniques.

Volumetric data: Volumetric data represents objects or environments as a series of voxels, which are 3D pixels that contain information on the density, colour, or other properties of the object at that point in space. Volumetric data is often used in medical imaging and scientific visualisation to represent internal organs, tissues, and other structures.

Multi-view data: Multi-view data is created by capturing multiple images or video streams from different viewpoints and then using

computer vision techniques to reconstruct a 3D model of the scene. This approach is often used in robotics, autonomous driving, and virtual reality applications, where it is important to create a realistic and accurate representation of the environment.

RGB-D scans: RGB-D scans combine colour (RGB) images with depth (D) information, typically obtained from depth sensors or depth cameras. These scans provide both visual appearance and geometric information of the scene or objects. RGB-D data is commonly used for applications, such as 3D mapping, scene understanding, and robotics.

Part assembly: Part assembly is a type of 3D modelling that involves creating individual parts or components that fit together to form a larger object. Each component is modelled as a separate 3D object and is designed to fit together with other components according to specific design constraints, such as alignment, size, and shape. Part assembly is commonly used in engineering and manufacturing to design and test complex assemblies, such as cars, aeroplanes, and machines.

Implicit functions: Implicit functions are mathematical representations of 3D objects that describe the boundary or surface of the object as a continuous function. Instead of defining the surface explicitly, as in the case of meshes or point clouds, implicit functions represent the surface as the zero level set of a function. For example, the implicit function for a sphere is defined as $f(x, y, z) = x^2 + y^2 + z^2 - r^2$, where r is the radius of the sphere.

Each of these types of 3D data has its own unique characteristics and applications, and understanding their strengths and limitations is important for working with 3D data effectively. Table 1 lists popular and widely used 3D benchmark datasets, categorised based on the data types (point clouds, meshes, multi-view data, volumetric data, part assembly, and RGB-D) and environment (indoor or outdoor). These datasets are widely used in various research areas, such as computer vision, robotics, 3D reconstruction, and scene understanding, and they serve as valuable resources for evaluating and benchmarking algorithms and models.

Table 1. Benchmark 3D datasets.

Dataset Name	Data Types	Environment
ModelNet ⁴	Point Clouds	Indoor
ShapeNet ⁵	Meshes	Indoor
ScanNet ⁶	RGB-D	Indoor
SUNCG ⁷	RGB-D	Indoor
NYU Depth V2 ⁸	RGB-D	Indoor
KITTI ⁹	Point Clouds	Outdoor
SemanticKITTI ¹⁰	Point Clouds	Outdoor
ApolloScape ¹¹	Point Clouds	Outdoor
3DMatch ¹²	RGB-D	Indoor
PartNet ¹³	Part Assembly	Indoor
FAUST ¹⁴	Meshes	Indoor
SceneNN ¹⁵	RGB-D	Indoor
Middlebury ¹⁶	RGB-D	Indoor/Outdoor
FlyingThings3D ¹⁷	Multi-view Data	Outdoor
3DShapeNet ¹⁸	Volumetric Data	Indoor
3D-R2N2 ¹⁹	Multi-view Data	Indoor
YCB-Video ²⁰	RGB-D scans	Indoor
Redwood ²¹	RGB-D scans	Indoor
ETH3D ²²	Multi-view Data	Outdoor
Aachen Day-Night ²³	Multi-view Data	Outdoor
S3DIS ²⁴	Point Clouds	Indoor
Waymo ²⁵	Point Clouds, Images	Outdoor

2.2. Unique Characteristics and Challenges

3D data contains information about objects in 3D space, presenting unique challenges and opportunities compared to 2D data. Though deep learning is proven to be a powerful tool to build models for language (one-dimensional) and image (2D) understanding, applying it to 3D data is still in its early stages, largely due to the following challenges:

Rich geometric information: 3D data contains rich geometric information about objects in 3D space, including their shape, size, and location. However, learning good representations that can leverage this geometry presents challenges, especially for deep learning algorithms that are designed for processing regular and ordered data.

Sparsity: 3D data is often sparse, with data points far apart from each other. This can be due to the nature of the data acquisition process, such as LIDAR scanning or the inherent sparsity of the data itself. This sparsity can make data acquisition and efficient computation a challenge and may require specialised hardware and software.

Unordered and irregular: 3D data can be unordered and irregular, meaning that the data points do not follow a regular pattern or sequence. This can be due to the nature of the data acquisition process, such as point clouds, or the inherent irregularity of the data itself. This makes it difficult to apply traditional deep learning algorithms that assume regular and ordered data and may require specialised techniques for processing these unordered and irregular data.

High dimensionality and complexity: 3D data is high-dimensional and complex, often requiring processing of millions or billions of data points. This makes it difficult to process and analyse the data and may require specialised hardware and software. Additionally, 3D data can be complex due to the many possible variations in shape, size, and location, requiring specialised techniques for handling this complexity.

These unique characteristics of 3D data present challenges for data acquisition, processing, and analysis. We list some challenges of processing and analysing 3D data, specifically related to data representation, feature extraction, and model training:

Data representation: Representing effective 3D data in a way that captures its rich geometric information and irregularity is a challenge. Traditional methods of representing data, such as grids or sequences, are not well suited for 3D data. Developing effective data representations that can be efficiently processed by deep learning algorithms is an active area of research.

Feature extraction: Extracting meaningful features from 3D data is a challenge, especially when the data is sparse, unordered, and irregular. Traditional feature extraction methods that work well for regular and ordered data may not be effective for 3D data. Developing specialised feature extraction techniques that can leverage the geometric information and irregularity of 3D data is an active area of research.

Model training: Training deep learning models on 3D data is a challenge due to the high dimensionality and complexity of the data. The large size of 3D datasets can also make model training computationally expensive and time-consuming. Developing efficient training algorithms and specialised hardware for processing 3D data is a crucial research area.

In addition to these challenges, there are also issues related to data quality, such as noise, outliers, and missing data, that can affect the performance of deep learning algorithms on 3D data. Addressing these challenges requires specialised techniques for data cleaning, pre-processing, and augmentation.

2.3. *Applications of 3D Data*

3D data has become increasingly important and valuable in various fields due to its ability to capture rich intricate geometric information about objects in 3D space, which makes it useful for numerous applications. Following are some examples of the emerging applications of 3D data across a wide range of industry sectors:

- **Healthcare:** 3D data is revolutionising medical imaging and diagnostics. It enables physicians to create intricate 3D models of patient anatomy, aiding in surgical planning, patient education, and personalised medical treatments. Additionally, 3D printing of medical devices and implants has become feasible with 3D data.
- **Manufacturing:** 3D data finds extensive use in manufacturing processes. It enables the creation of digital twins, virtual replicas of physical products or equipment, which can be used for simulation, optimisation, and predictive maintenance. 3D data also plays a crucial role in additive manufacturing (3D printing), allowing the production of complex and customised parts.
- **Automotive and aerospace:** The automotive and aerospace industries benefit from 3D data in multiple ways. It assists in designing aerodynamic structures, optimising vehicle performance, and simulating crash tests. Additionally, 3D data enables the creation of virtual prototypes, reducing time and costs associated with physical prototyping.

- **Architecture and construction:** Architects and engineers can use 3D data to create detailed models and visualisations of buildings, improving design accuracy and facilitating better communication with clients. Construction companies can leverage 3D data for precise planning, clash detection, and construction site optimisation.
- **Entertainment and gaming:** 3D data is at the core of modern entertainment and gaming experiences. It powers realistic computer-generated imagery (CGI) in movies, television shows, and video games, creating immersive virtual worlds and life-like characters.
- **Retail and e-commerce:** Retailers can leverage 3D data to enhance the online shopping experience. By creating 3D models of products, customers can visualise them from various angles and even place them in virtual environments to assess size and fit. This technology improves product visualisation, reduces returns, and enhances customer engagement.
- **Cultural heritage and tourism:** 3D data contributes to the preservation and presentation of cultural heritage sites and artefacts. It allows for the creation of accurate digital replicas, enabling virtual tours and immersive experiences that provide access to historical and cultural treasures remotely.
- **Energy and natural resources:** The energy sector benefits from 3D data in activities, such as oil and gas exploration, renewable energy site assessment, and infrastructure planning. 3D models help in visualising subsurface data, optimising resource extraction, and minimising environmental impact.
- **Education and training:** 3D data facilitates interactive and immersive learning experiences. It allows students to explore complex subjects through virtual simulations, interactive models, and augmented reality (AR) applications.

From the above applications, we could see innovative uses of 3D data. As technology continues to evolve and new applications are developed, we can expect 3D data to play an increasingly important role in many aspects of our lives.

3. Introduction to 3D Deep Learning

3.1. *Deep Learning*

Deep learning is a subfield of machine learning that utilises artificial neural networks to learn from large amounts of data. In deep learning, neural networks are composed of multiple layers of interconnected nodes, or neurons, that process and transform data, allowing the network to automatically learn complex features and patterns in the data.

One of the key strengths of deep learning is its ability to perform end-to-end learning, which means that the network can learn to directly map raw input data to output predictions, without the need for manual feature engineering. This makes deep learning particularly powerful in domains with large amounts of complex data, such as image recognition, natural language processing, and now 3D data.

Deep learning has seen significant advancements in recent years, driven by both the availability of large datasets and advances in computing power and hardware. This has led to the development of increasingly complex models, such as convolutional neural networks (CNNs), recurrent neural networks (RNNs), and more recently, transformer models, such as the Generative Pre-trained Transformer (GPT) family.

In the context of 3D deep learning, deep neural networks have been adapted and extended to work with 3D data, including point clouds, meshes, and volumetric data. This has led to significant progress in tasks, such as 3D object detection and segmentation, point cloud classification, and 3D reconstruction. Nevertheless, working with 3D data presents unique challenges compared to 2D data, such as sparsity, irregularity, and complexity of the geometric structure. Therefore, new methods and architectures are needed to tackle these challenges and unlock the potential of 3D deep learning for a wide range of applications. The following subsections provide an overview of technical challenges and latest development in 3D deep learning.

3.2. *3D Deep Learning*

Deep learning is a powerful tool to build models for language and image understanding. While the potential of deep learning for 3D

data is immense and holds great promise for real-world applications, its deployment is still in its early stages. Currently, most research focuses on extending or adapting existing 2D technology to handle 3D data. There is still plenty of subjects to explore in this area.

3.2.1. *Technical challenges of 3D deep learning*

As discussed in previous sections, 3D data has its own unique characteristics, which creates many technical challenges for the research and adoption of 3D deep learning techniques. Some of the key challenges in applying deep learning to 3D data are discussed in as follows:

Data and annotation challenges: The unique characteristics of 3D data, such as its rich geometry information, make data annotation considerably more complicated and expensive than for 2D data. Most existing 3D deep learning methods require large amounts of labelled 3D data to achieve a decent performance. However, in real-world scenarios, it is extremely challenging to acquire and annotate sufficient 3D data for model training. While some recent studies have attempted to address this challenge using weakly and semi-supervised methods, there remains a considerable performance gap compared to fully supervised methods.

Representation challenges: Realistic 3D sensors generate heterogeneous multi-modal 2D/3D data. The geometric structure of this data is rich and varied, which makes joint representation learning and transfer learning across multiple modalities challenging. The majority of current 3D deep learning methods focus on learning a single type of representation, which means that they fail to fully exploit the information contained within multiple representations of 3D data. Although there are some existing heterogeneous 3D methods that concentrate on fusing 2D images and 3D point clouds at the feature level, they are unable to learn detailed joint 3D representations.

Robustness challenges: In real-world applications, the acquired 3D data is frequently noisy, contains partial-view information, may be subject to intended attacks, and often faces challenges like object shifting or changes. Making the 3D deep learning robust that can effectively handle these difficulties remains a crucial issue. Most current 3D deep learning methods are designed to work on clean and perfectly aligned datasets, with only a few attempts at studying

their robustness through strategies like data augmentation and self-supervised learning, but performance in these scenarios often significantly degrades when compared to the use of ideally clean data.

Computation challenges: Compared to 2D deep neural networks, 3D deep neural networks are significantly more computationally intensive, with the 3D models being at least one order of magnitude larger than their 2D counterparts. As a result, deploying 3D deep models to real-world applications, particularly those requiring real-time responses under computational resource constraints, is challenging. Most current methods adapt 2D approaches for 3D data, focusing solely on the efficiency of the neural network, but ignoring hardware constraints, and failing to incorporate 3D data specifications.

In conclusion, applying deep learning to 3D data poses several challenges, related to the scalability of 3D deep learning models, the scarcity of annotated 3D data, the computational complexity of 3D models, the difficulty of modelling geometric structures and relationships, and the challenges in handling noise, partial views, and adversarial attacks. Addressing these challenges is critical for the successful deployment of deep learning models in various 3D data applications.

3.2.2. *Latest developments in 3D deep learning*

Over recent years, there has been significant progress in the development of 3D deep learning techniques aimed at addressing the challenges of processing and analysing 3D data.

Point-based networks: Point-based deep learning technology is a branch of 3D deep learning that focuses on processing and analysing 3D data in its raw form, without requiring voxelisation or pre-defined grids. This approach has gained significant attention due to its ability to handle point clouds, which are sets of unordered 3D points, and has led to advancements in various applications, such as 3D object recognition, segmentation, and scene understanding. Two prominent models in this field are PointNet²⁶ and PointNet++.²⁷

PointNet, introduced by Qi *et al.* in 2017, was one of the pioneering works in point-based deep learning. It tackles the challenge of directly processing point clouds by leveraging a symmetric

function to aggregate features from individual points. PointNet provides a global feature descriptor for the entire point cloud, which can be used for tasks like object classification and part segmentation. The model is permutation invariant, meaning it produces consistent results regardless of the order of the input points. PointNet revolutionised the field by introducing a simple yet effective framework for point cloud analysis. It demonstrated that deep learning models could process unstructured point cloud data directly, eliminating the need for time-consuming and memory-intensive voxelisation.

PointNet++ is an extension of PointNet that aims to address its limitations in capturing local context and hierarchical information. PointNet++ introduces a hierarchical neural network architecture that partitions the input point cloud into a nested set of local regions. The model applies the PointNet architecture recursively on these regions to capture both local and global information effectively. By hierarchically grouping points, PointNet++ can better capture fine-grained details while maintaining a global understanding of the entire point cloud. PointNet++ improves feature learning by exploiting the local and contextual relationships among points. It enhances the performance of PointNet on tasks like segmentation and shape classification, demonstrating the importance of hierarchical processing in point-based deep learning.

PointNet and PointNet++ have inspired numerous applications and extensions in the field of 3D deep learning. Researchers have developed variations of these models to address specific challenges or improve performance. For example, PointCNN²⁸ is a convolutional neural network designed explicitly for point cloud data, while Dynamic Graph CNN (DGCNN)²⁹ exploits the local geometric structure by constructing a k -nearest neighbour graph for each point. These applications and extensions demonstrate the versatility and ongoing development of point-based deep learning technology, as researchers continue to explore novel approaches for analysing 3D data.

CNN-based networks: 3D CNNs are a type of deep learning architecture that can be used for processing 3D data, such as volumetric medical images, 3D scans, and videos. 3D CNNs extend the concept of 2D convolutional layers to three dimensions, allowing them to learn spatial features and patterns from 3D data.

In a 3D CNN, the input data is typically a 3D volume represented as a stack of 2D image slices. The convolutional layers in a 3D CNN use 3D filters or kernels to convolve over the input volume, computing a dot product between the filter weights and the input values at each location in the volume. The output of each convolutional layer is a new 3D volume, which represents a learned feature map of the input data. The feature maps are then typically downsampled using pooling layers, which reduce the spatial resolution of the data while retaining the most important features. Finally, the feature maps are flattened and passed through one or more fully connected layers to produce the final output of the model.

The development in 3D CNN can be summarised in two branches,³⁰ dense representation and sparse representation. The first branch of 3D-convolutional neural networks utilises a rectangular grid and a dense representation.^{6,31} In this representation, empty space in 3D scans is either represented as 0 or using the signed distance function. This straightforward representation is intuitive and is supported by all major public neural network libraries. However, a significant limitation of this approach is its high memory consumption and slow computation due to the fact that most of the 3D space in typical scans is empty. To address this issue, OctNet³² proposed the use of the octree structure to represent 3D space and perform convolutions on it. By employing the octree structure, this branch can efficiently handle sparse 3D data, reducing memory usage and improving computation speed.

The second branch comprises sparse 3D-convolutional neural networks. Previous works employ quantisation methods for handling high dimensions: a rectangular grid and a permutohedral lattice. In Ref. [33], a permutohedral lattice is used, while Ref. [34] utilises a rectangular grid for 3D classification and semantic segmentation tasks. Recent development³⁰ proposes the generalised sparse convolutions for sparse tensors and published an open-source autodiff library for sparse tensors with comprehensive standard neural network functions. These techniques enable efficient processing of sparse 3D data, alleviating memory and computation challenges associated with such data.

Graph-based networks: Graph neural networks (GNNs) are a class of deep learning architectures that are designed for processing

data represented as graphs, which are collections of nodes and edges that represent the relationships between them. In the context of 3D deep learning, graph-based networks consider each point in a point cloud as a vertex of a graph and generate directed edges for the graph based on the neighbours of each point. GNNs can be categorised into two main types²: spectral and spatial. Spectral GNNs operate in the frequency domain and use graph Fourier transforms to compute convolutions. Spatial GNNs operate in the spatial domain and use message-passing algorithms to propagate information between neighbouring nodes.

In the spatial domain, Simonovsky *et al.*³⁵ pioneer the perspective of each point as a graph vertex, connecting them to neighbours. DGCNN²⁹ constructs dynamic feature space graphs, using MLPs for edge feature learning and symmetric aggregation for neighbouring points. LDGCNN³⁶ improves DGCNN by linking hierarchical features and removing transformation networks to enhance performance and reduce model size. Hassani and Haley³⁷ present an unsupervised multi-task autoencoder. Its encoder employs multi-scale graphs, while the decoder handles clustering, self-supervised classification, and reconstruction tasks. Liu *et al.*³⁸ streamline point agglomeration with DPAM, leveraging graph convolution for sampling, grouping, and pooling in a single step through matrix multiplication.

To exploit local geometric structures, KCNet³⁹ learns via kernel correlation, using a set of learnable points as kernels to represent specific geometric types. It calculates affinity between these kernels and the neighbourhood of a point. While ClusterNet⁴⁰ employs a rigorous rotation-invariant module to extract such features from the k -nearest neighbours of each point.

In the spectral domain, RGCNN⁴¹ establishes a graph linking all points within the point cloud, updating the graph Laplacian matrix in each layer. To encourage similarity among adjacent vertex features, a graph-signal smoothness prior is included in the loss function. Addressing the challenge of diverse graph topologies, AGCN's SGC-LL layer⁴² employs a learnable distance metric for vertex similarity. The adjacency matrix is normalised using Gaussian kernels and learned distances. HGNN⁴³ employs spectral convolution on a hypergraph for hyperedge convolutional layers.

To leverage local structural insights, Wang *et al.* introduced LocalSpecGCN,⁴⁴ an end-to-end spectral convolution network

operating on local graphs constructed from k -nearest neighbours. It avoids offline graph Laplacian computation and coarsening hierarchy. In PointGCN,⁴⁵ a graph is built based on k -nearest neighbours, weighted using Gaussian kernels. Convolutional filters are Chebyshev polynomials in the graph spectral domain. Pan *et al.*'s 3DTI-Net⁴⁶ applies spectral convolution on k -nearest neighbouring graphs, ensuring invariance to geometric transformations through relative Euclidean and direction distances.

Transformer-based networks: The transformer architecture is a type of neural network that was originally introduced for natural language processing tasks. It has since been applied to a wide range of other tasks, including 3D deep learning tasks, such as point cloud processing and volumetric data analysis. The key innovation of the transformer architecture is the self-attention mechanism, which allows the network to selectively attend to different parts of the input at each layer. This makes it particularly well suited for processing input data that has long-range dependencies, such as sequences or graphs.

In the context of 3D deep learning, transformers have been used for tasks, such as point cloud classification, segmentation, and generation.^{47–49} According to the operating scale, 3D transformers can be divided into two parts: global transformers and local transformers. The former denotes that transformer blocks are applied to all the input points for global feature extraction, while the latter denotes that transformer blocks are applied in a local patch for local feature extraction.

Point transformer⁵⁰ applies self-attention within each data point's local neighbourhood. This block comprises an attention layer, linear projections, and residual connection. Instead of using 3D point coordinates for position encoding, an encoding function with linear layers and ReLU nonlinearity is adopted. 3DCTN⁵¹ combines graph convolution layers with transformers, leveraging local and global learning. LFT-Net⁵² introduces self-attention for point cloud features and a trans-pooling layer to reduce feature size.

On a broader scale, ShapeContextNet⁵³ pioneers self-attention in point cloud recognition, using it to select, aggregate, and transform features for shape context learning. Adaptive wavelet transformer⁵⁴ employs multi-resolution analysis through neural networks, capturing geometric information through decomposition. TransPCNet⁵⁵

aggregates features, applies separable convolutions, and employs attention for defect detection in sewer point clouds.

Many approaches deploy transformer architecture to learn local and global information, adapting it for different information processing. Engel *et al.*⁵⁶ use local-global attention for geometric relations. CpT⁵⁷ employs a dynamic point cloud graph in transformer layers. Point-Voxel Transformer (PVT)⁵⁸ merges voxel-based and point-based transformers for feature extraction.

In conclusion, there are various 3D deep learning techniques that have been developed to address the unique challenges posed by 3D data. Each technique has its own strengths and limitations, and the optimal choice depends on the specific application and the type of 3D data involved. Careful consideration is therefore required when selecting the appropriate technique to achieve optimal performance and efficiency.

3.2.3. Applications of 3D deep learning

3D deep learning has a wide range of real-world applications in various fields, and it has already demonstrated significant breakthroughs in many areas. We list down some popular applications in the following:

Autonomous driving: 3D deep learning can analyse 3D point cloud data obtained from sensors such as LiDAR to detect and classify objects in the environment, such as cars, pedestrians, and road markings, enabling autonomous vehicles to navigate safely and efficiently.

Robot perception: 3D deep learning can enable robots to perceive and understand their environment in 3D, allowing them to perform tasks, such as object recognition and manipulation.

Augmented reality (AR): 3D deep learning can be used in AR applications to accurately map and track the user's surroundings, enabling realistic virtual objects to be placed and interacted with in the real world.

Metaverse: 3D deep learning can play a critical role in creating and rendering the 3D virtual worlds that are at the heart of the metaverse concept.

UAV inspection: 3D deep learning can help automate and improve the inspection of infrastructure, such as bridges and buildings, by analysing 3D point cloud data obtained from UAVs.

3D inspection and metrology: 3D deep learning can be used for the analysis and measurement of 3D objects, enabling precise and automated inspection and quality control.

Computer graphics: 3D deep learning can be used to generate realistic and high-quality 3D graphics, such as for video games and special effects in movies.

Medical imaging: 3D deep learning can enable accurate and automated analysis of medical images, such as CT and MRI scans, for applications, such as diagnosis and treatment planning.

Neuroscience: 3D deep learning can be used to analyse and understand the complex structure and function of the brain, enabling breakthroughs in areas, such as brain-computer interfaces and treatment of neurological disorders.

Scientific simulations: 3D deep learning can enable more accurate and efficient simulations of complex physical and biological systems, enabling new insights and discoveries in fields, such as physics, chemistry, and biology.

From the listed examples, 3D deep learning has already demonstrated significant breakthroughs in many applications, and its impact is expected to continue to grow. As researchers continue to develop new techniques and algorithms for processing and analysing 3D data, we can expect to see even more breakthroughs and innovations in the years to come.

3.2.4. *Future directions and challenges*

The potential future directions of 3D deep learning are vast, as this technology continues to develop and mature. One potential direction is the development of new architectures that can handle even more complex and diverse 3D data, such as those with a higher degree of irregularity, larger scale, or greater sparsity. This may require the use of novel deep learning techniques or the integration of 3D data with other types of data. The other potential direction is the improvement of the efficiency of 3D deep learning models, both in terms of

computation time and resource utilisation. This could involve the development of new algorithms, such as those that utilise parallel processing or distributed computing, as well as the optimisation of existing algorithms for more efficient use of hardware.

Another important direction for 3D deep learning is addressing the challenge of domain adaptation for 3D data. Currently, many 3D deep learning models are trained on specific datasets or in specific domains, making it difficult to generalise to other contexts or datasets. Developing models that can adapt to new domains or datasets with minimal training is an important area of research for the future. This could involve exploring techniques, such as transfer learning, where a pre-trained model on one dataset is fine-tuned on another, or domain adaptation methods that enable models to learn from multiple domains or adapt to new domains at test time. In addition, developing methods to quantify and visualise model uncertainty can help improve the reliability and robustness of 3D deep learning models in new and unfamiliar domains.

Exploring new applications for 3D deep learning is also a promising area for future research. While this technology is already being used in a wide variety of fields, there are likely many new applications that have yet to be discovered. For example, 3D deep learning could be applied to environmental monitoring, architectural design, or even space exploration. By continuing to investigate the potential of this technology in various domains, we can expand its impact and unlock new opportunities for innovation.

While 3D deep learning has made significant strides, there are still notable challenges and limitations that remain to be addressed. One of the most pressing challenges is the limited availability of annotated 3D data, particularly in specific or specialised fields. As 3D deep learning continues to expand into new areas, it will be important to develop new methods for efficiently acquiring and labelling large volumes of 3D data. Another challenge is the difficulty of modelling temporal dynamics in 3D data. While many 3D deep learning models can handle static data, such as 3D images or meshes, modelling dynamic 3D data, such as video or simulations, presents a significant challenge. Addressing this challenge requires the development of models that can efficiently learn and represent this type of data. Therefore, this area of research presents an important opportunity for future advancement of 3D deep learning.

Finally, as 3D deep learning models become increasingly complex, there is a growing need for improved interpretability and explainability. While these models can achieve impressive levels of accuracy and performance, they often operate as “black boxes”, making it difficult to understand how they arrive at their predictions or decisions. This lack of transparency poses significant ethical and safety concerns, particularly in applications, such as healthcare and autonomous vehicles. Developing methods for interpreting and explaining the inner workings of 3D deep learning models is a critical area of research that will be essential for ensuring the responsible and trustworthy deployment of these models in the future.

4. Overview of This Book

This book is organised into five sessions, each of which addresses different aspects of 3D deep learning:

Session I — Sample efficient 3D deep learning: This session focuses on developing efficient algorithms to build accurate 3D models with limited annotated samples. Chapter 2 presents a self-training approach with extremely sparse annotations, achieving superior results on 3D scene understanding tasks compared to weakly supervised methods and approaching fully supervised performance. While Chapter 3 proposes a novel scheme of masked autoencoders for 3D point cloud self-supervised learning, addressing challenges posed by point cloud data and achieving efficient pre-training and strong generalisation on various downstream tasks.

Session II — Representation efficient 3D deep learning: This session deals with the challenge of developing efficient representations for dynamic 3D scenes and multiple 3D modalities. Chapter 4 explores various methodologies for modelling and learning representations in dynamic 3D scenes, advancing embodied AI systems with diverse 3D representation methods and their applications. Chapter 5 proposes a new approach for learning shape implicit neural representations from point cloud data without requiring normal vectors as input, achieving smooth solutions and outperforming methods using normal vectors directly. While Chapter 6 introduces a depth-aware monocular 3D object detector, trained from augmented data

with virtual depths synthesised from reference images and depth maps, eliminating the need for an external depth estimator during deployment.

Session III — Robust 3D deep learning: This session presents methods for improving the robustness and reliability of deep learning models in real-world applications. Chapter 7 proposes a novel bi-level optimisation framework for robust 3D point cloud classification, achieving state-of-the-art performance against adversarial attacks on ModelNet40 and ScanNet datasets. While Chapter 8 introduces a test-time anchored clustering approach under a realistic sequential test-time training protocol, improving generalisation and outperforming state-of-the-art methods on variety of benchmark datasets.

Session IV — Resource efficient 3D deep learning: This session presents the topic of resource efficiency in 3D deep learning, covering algorithm-system-hardware co-design in Chapter 9 and sampling strategies in Chapter 10 for efficient segmentation and object detection of point clouds. It explores ways to reduce the computation cost of 3D models and improve their efficiency in resource-limited environments.

Session V — Emerging 3D deep learning applications: This session discusses exciting and emerging use cases of 3D deep learning in various fields, including medical image analysis in Chapter 11 and semiconductor defect detection in Chapter 12. It showcases how 3D deep learning is transforming industries and enabling new applications for healthcare and manufacturing.

Each chapter in this book provides an in-depth exploration of a specific topic in 3D deep learning, including the latest research developments, challenges, and future directions. This book aims to serve as a comprehensive reference for postgraduate students, researchers, and practitioners who are interested in exploring the potential of 3D deep learning.

References

1. E. Ahmed, A. Saint, A. E. R. Shabayek, K. Cherenkova, R. Das, G. Gusev, D. Aouada, and B. Ottersten. A survey on deep learning advances on different 3D data representations. *arXiv preprint arXiv:1808.01462* (2019).

2. Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun. Deep learning for 3D point clouds: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**(12), 4338–4364 (December, 2021). <https://doi.org/10.1109/TPAMI.2020.3005434>.
3. H. Su, J. Gu, and M. Liu. Tutorial on 3D deep learning, Online tutorial at https://cseweb.ucsd.edu/haosu/talks.html#_3d_deep_learning (2020).
4. Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3D ShapeNets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1912–1920 (2015).
5. A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su et al. ShapeNet: An information-rich 3D model repository. *arXiv preprint arXiv:1512.03012* (2015).
6. A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5828–5839 (2017).
7. S. Song, F. Yu, A. Zeng, A. X. Chang, M. Savva, and T. Funkhouser. Semantic scene completion from a single depth image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1746–1754 (2017).
8. P. K. Nathan Silberman, D. Hoiem and R. Fergus. Indoor segmentation and support inference from RGBD images. In *ECCV*, pp. 746–760 (2012).
9. A. Geiger, P. Lenz, and R. Urtasun. Are we ready for autonomous driving? The KITTI vision benchmark suite. In *2012 IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3354–3361 (2012).
10. J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall. Semantickitti: A dataset for semantic scene understanding of LiDAR sequences. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9297–9307 (2019).
11. X. Huang, P. Wang, X. Cheng, D. Zhou, Q. Geng, and R. Yang. The ApolloScape open dataset for autonomous driving and its application, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **42**(10), 2702–2719 (2019).
12. A. Zeng, S. Song, M. Nießner, M. Fisher, J. Xiao, and T. Funkhouser. 3DMatch: Learning local geometric descriptors from RGB-D reconstructions. In *CVPR*, pp. 199–208 (2017).

13. K. Mo, S. Zhu, A. X. Chang, L. Yi, S. Tripathi, and L. J. Guibas. Part-Net: A large-scale benchmark for fine-grained and hierarchical part-level 3D object understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 909–918 (2019).
14. F. Bogo, J. Romero, M. Loper, and M. J. Black. FAUST: Dataset and evaluation for 3D mesh registration. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3794–3801 (2014).
15. B.-S. Hua, Q.-H. Pham, D. T. Nguyen, M.-K. Tran, L.-F. Yu, and S.-K. Yeung. SceneNN: A scene meshes dataset with annotations. In *2016 4th International Conference on 3D Vision (3DV)*, pp. 92–101 (2016).
16. D. Scharstein, H. Hirschmüller, Y. Kitajima, G. Krathwohl, N. Nešić, X. Wang, and P. Westling. High-resolution stereo datasets with subpixel-accurate ground truth. In *Pattern Recognition: 36th German Conference, GCPR 2014, Münster, Germany, September 2–5, 2014, Proceedings 36*, pp. 31–42 (2014).
17. N. Mayer, E. Ilg, P. Hausser, P. Fischer, D. Cremers, A. Dosovitskiy, and T. Brox. A large dataset to train convolutional networks for disparity, optical flow, and scene flow estimation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4040–4048 (2016).
18. H. Wang, S. Sridharan, J.-B. Huang, and G. Hua. 3D ShapeNet: A deep representation for volumetric shapes, *ACM Transactions on Graphics (TOG)* **38**(4), 85 (2019).
19. C. B. Choy, D. Xu, J. Gwak, K. Chen, and S. Savarese. 3D-R2N2: A unified approach for single and multi-view 3D object reconstruction. In *Computer Vision–ECCV 2016: 14th European Conference, Amsterdam, The Netherlands, October 11–14, 2016, Proceedings, Part VIII 14*, pp. 628–644 (2016).
20. Y. Xiang, T. Schmidt, V. Narayanan, and D. Fox. PoseCNN: A convolutional neural network for 6D object pose estimation in cluttered scenes. *arXiv preprint arXiv:1711.00199* (2017).
21. S. Choi, Q.-Y. Zhou, S. Miller, and V. Koltun. A large dataset of object scans. *arXiv preprint arXiv:1602.02481* (2016).
22. T. Schops, J. L. Schonberger, S. Galliani, T. Sattler, K. Schindler, M. Pollefeys, and A. Geiger. A multi-view stereo benchmark with high-resolution images and multi-camera videos. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3260–3269 (2017).

23. T. Sattler, B. Leibe, and L. Kobbelt. Benchmarking 6DOF outdoor visual localization in changing conditions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8601–8610 (2018).
24. I. Armeni, O. Sener, A. R. Zamir, H. Jiang, I. Brilakis, M. Fischer, and S. Savarese. 3D semantic parsing of large-scale indoor spaces. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1534–1543 (2016).
25. P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine et al. Scalability in perception for autonomous driving: Waymo open dataset. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2446–2454 (2020).
26. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2017).
27. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems (NeurIPS)* (2017).
28. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on x-transformed points. In *Advances in Neural Information Processing Systems (NeurIPS)* (2018).
29. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph CNN for learning on point clouds, *ACM Transactions on Graphics (TOG)* **38**(5), 1–12 (2019).
30. C. B. Choy, J. Gwak, and S. Savarese. 4D spatio-temporal convnets: Minkowski convolutional neural networks. In *2019 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3070–3079 (2019). <https://api.semanticscholar.org/CorpusID:121123422>.
31. L. P. Tchapmi, C. B. Choy, I. Armeni, J. Gwak, and S. Savarese. SEGCloud: Semantic segmentation of 3D point clouds. In *International Conference on 3D Vision (3DV)* (2017).
32. G. Riegler, A. O. Ulusoy, and A. Geiger. OctNet: Learning deep 3D representations at high resolutions. In *Proceedings of the IEEE Computer Vision and Pattern Recognition (CVPR)* (2017).
33. H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz. SPLATNet: Sparse lattice networks for point cloud processing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).

34. B. Graham. Sparse 3D convolutional neural networks. In *British Machine Vision Conference* (2015).
35. M. Simonovsky and N. Komodakis. Dynamic edge-conditioned filters in convolutional neural networks on graphs. In *CVPR* (2017).
36. K. Zhang, M. Hao, J. Wang, C. W. de Silva, and C. Fu. Linked dynamic graph CNN: Learning on point cloud via linking hierarchical features. *arXiv preprint* arXiv:1904.10014 (2019).
37. K. Hassani and M. Haley. Unsupervised multi-task feature learning on point clouds. In *ICCV* (2019).
38. J. Liu, B. Ni, C. Li, J. Yang, and Q. Tian. Dynamic points agglomeration for hierarchical point sets learning. In *ICCV* (2019).
39. Y. Shen, C. Feng, Y. Yang, and D. Tian. Mining point cloud local structures by kernel correlation and graph pooling. In *CVPR* (2018).
40. C. Chen, G. Li, R. Xu, T. Chen, M. Wang, and L. Lin. Cluster-Net: Deep hierarchical cluster network with rigorously rotation-invariant representation for point cloud analysis. In *CVPR* (2019).
41. G. Te, W. Hu, A. Zheng, and Z. Guo. RGCNN: Regularized graph CNN for point cloud segmentation. In *ACM Multimedia (ACM MM)* (2018).
42. R. Li, S. Wang, F. Zhu, and J. Huang. Adaptive graph convolutional neural networks. In *AAAI Conference on Artificial Intelligence (AAAI)* (2018).
43. Y. Feng, H. You, Z. Zhang, R. Ji, and Y. Gao. Hypergraph neural networks. In *AAAI Conference on Artificial Intelligence (AAAI)* (2019).
44. C. Wang, B. Samari, and K. Siddiqi. Local spectral graph convolution for point set feature learning. In *European Conference on Computer Vision (ECCV)* (2018).
45. Y. Zhang and M. Rabbat. A graph-CNN for 3D point cloud classification. In *IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)* (2018).
46. G. Pan, J. Wang, R. Ying, and P. Liu. 3DTI-Net: Learn inner transform invariant 3D geometry features using dynamic GCN. *arXiv preprint* arXiv:1812.06254 (2018).
47. D. Lu, Q. Xie, M. Wei, K. Gao, L. Xu, and J. Li. Transformers in 3D point clouds: A survey. *arXiv preprint* arXiv:2205.07417 (2022).
48. J. Zeng, D. Wang, and P. Chen. A survey on transformers for point cloud processing: An updated overview, *IEEE Access* **10**, 86510–86527 (2022). doi:10.1109/ACCESS.2022.3198999.
49. J. Lahoud, J. Cao, F. S. Khan, H. Cholakkal, R. M. Anwer, S. Khan, and M.-H. Yang. 3D vision with transformers: A survey. *arXiv preprint* arXiv:2208.04309 (2022).

50. H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun. Point transformer. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 16259–16268 (2021).
51. D. Lu, Q. Xie, L. Xu, and J. Li. 3DCTN: 3D convolution-transformer network for point cloud classification. *arXiv preprint arXiv:2203.00828* (2022).
52. Y. Gao, X. Liu, J. Li, Z. Fang, X. Jiang, and K. M. S. Huq. LFT-Net: Local feature transformer network for point clouds analysis, *IEEE Transactions on Intelligent Transportation Systems (T-ITS)* **24**(2), 2158–2168 (2023). doi:10.1109/TITS.2022.3140355.
53. S. Xie, S. Liu, Z. Chen, and Z. Tu. Attentional shapecontextnet for point cloud recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4606–4615 (2018).
54. H. Huang and Y. Fang. Adaptive wavelet transformer network for 3D shape representation learning. In *International Conference on Learning Representations (ICLR)* (2021).
55. Y. Zhou, A. Ji, and L. Zhang. Sewer defect detection from 3D point clouds using a transformer-based deep learning model, *Automation in Construction* **136**, 104163 (2022).
56. N. Engel, V. Belagiannis, and K. Dietmayer. Point transformer, *IEEE Access* **9**, 134826–134840 (2021).
57. C. Kaul, J. Mitton, H. Dai, and R. Murray-Smith. CPT: Convolutional point transformer for 3D point cloud processing. *arXiv preprint arXiv:2111.10866* (2021).
58. C. Zhang, H. Wan, S. Liu, X. Shen, and Z. Wu. PVT: Point-voxel transformer for 3D deep learning. *arXiv preprint arXiv:2108.06076* (2021).

Chapter 2

Masked Autoencoders for 3D Point Cloud Self-Supervised Learning

Yatian Pang^{*,†,¶}, Zhenghua Chen^{‡,||}, and Li Yuan^{§,}**

[†]*National University of Singapore, Singapore*

[‡]*Institute for Infocomm Research, A*STAR, Singapore*

[§]*School of Electronic and Computer Engineering, Peking University,
Beijing, China*

[¶]*yatian_pang@u.nus.edu*

^{||}*chen0832@e.ntu.edu.sg*

^{**}*yuanli-ece@pku.edu.cn*

Self-supervised learning methods are able to learn latent features from unlabelled data samples by designing pre-text tasks and hence have attracted a great deal of interest in terms of sample efficient learning. As a promising scheme of self-supervised learning, masked autoencoding has significantly advanced natural language processing and computer vision but has not been introduced to point cloud yet. In this chapter, we propose a novel scheme of masked autoencoders for 3D point cloud self-supervised learning, addressing special challenges posed by point cloud, including leakage of location information and uneven information density. Concretely, we divide the input point cloud into irregular point patches and randomly mask them at a high ratio. Then, a standard transformer-based autoencoder, with an asymmetric design

*Work done as an intern at A*STAR.

and a shifting mask tokens operation, learns high-level latent features from unmasked point patches, aiming to reconstruct the masked point patches. Extensive experiments show that our approach is efficient during pre-training and generalises well on various downstream tasks. Apart from the proposed method, we also introduce potential directions for 3D point cloud self-supervised learning, including improvements for masked autoencoding and developments for point cloud scene understanding.

1. Introduction

3D point cloud represents a set of points in 3D space, providing geometry features. Since PointNet,¹ deep learning methods have been developed rapidly for point cloud representation learning, including shape classification, object detection, and part segmentation. Nevertheless, to some extent, the development is limited by relatively small point cloud datasets (usually less than 100k instances), which is mainly caused by high costs in data acquisition and manual annotating processes. To address this limitation, sample-efficient deep learning methods have been explored by the research community. As one of the most popular categories, self-supervised learning methods have been extensively studied.

Self-supervised learning learns latent features from unlabelled data instead of building representations based on human-defined annotations. It is usually done by designing a pre-text task to pre-train the model and then fine-tune downstream tasks. Relying less on labelled data, self-supervised learning has significantly advanced natural language processing (NLP)^{2–5} and computer vision.^{6–13} Among them, masked autoencoding,^{11–13} illustrated in Figure 1, is a promising scheme for both languages and images. It randomly masks a portion of input data and adopts an autoencoder to reconstruct explicit features (e.g., pixels) or implicit features (e.g., discrete tokens) corresponding to the original masked content. As masked parts do not provide data information, this reconstruction task enables the autoencoder to learn high-level latent features from unmasked parts. Besides, the powerful capability of masked autoencoding gives credit to its autoencoder’s backbone, which adopts transformer¹⁴ architecture. For example, BERT² in NLP and MAE¹² in computer vision both apply masked autoencoding and adopt a standard transformer architecture as autoencoder’s backbone to achieve state-of-the-art performance.

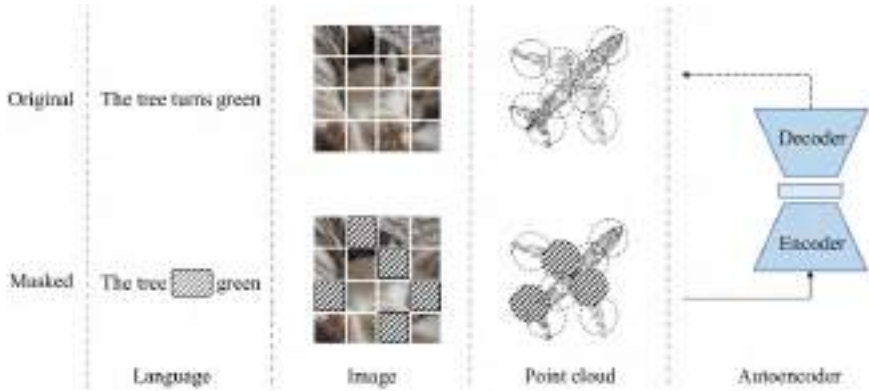


Fig. 1. Illustration of masked autoencoding. A portion of input data is masked, then an autoencoder is trained to recover the masked parts from original input data. The encoder in autoencoder is encouraged to learn high-level latent features from unmasked parts.

The idea of masked autoencoding is also applicable for point cloud self-supervised learning, as point cloud essentially shares a common property with both languages and images (see Figure 1). Specifically, the fundamental elements (i.e., points, vocabularies, and pixels) that carry information are not independent. Instead, neighbouring elements form a meaningful subset to present local features. Together with local features, the complete set of elements makes up global features. Therefore, after embedding point subsets into tokens, the point cloud can be processed similarly with languages and images. Furthermore, considering datasets for the point cloud are relatively small, masked autoencoding as a self-supervised learning method can naturally address the large data demand of transformer architecture, which is the autoencoder’s backbone. Indeed, a recent work Point-BERT¹⁵ attempts to design a scheme somewhat similar to masked autoencoding. It proposes a BERT-style pre-training strategy by masking input tokens of the point cloud and then adopts a transformer architecture to predict discrete tokens of the masked tokens. However, this method is relatively sophisticated as it is required to train a DGCNN-based¹⁶ discrete Variational AutoEncoder (dVAE)¹⁷ before pre-training and relies heavily on contrastive learning as well as data augmentation during pre-training. Moreover, the masked tokens from their inputs are processed from the input of transformer

during pre-training, leading to early leakage of location information and high consumption of computing resources. Different from their method, and more importantly, to introduce masked autoencoding to the point cloud, we aim to design a neat and efficient scheme of masked autoencoders. To this end, we first analyse the main challenges of introducing masked autoencoding for point cloud from the following aspects:

- (i) The first is the lack of a unified transformer architecture. Compared to transformer¹⁴ in NLP and Vision Transformer (ViT)¹⁸ in computer vision, transformer architectures for point cloud are less studied and relatively diverse, mainly because small datasets cannot meet the large data demand of transformer. Different from previous methods that use dedicated transformer or adopt extra non-transformer models to assist (such as Point-BERT¹⁵ uses an extra DGCNN¹⁶), we aim to build our autoencoder’s backbone entirely based on standard transformer, which can serve as a potential unified architecture for point cloud. This also enables further development for point cloud to join general multi-modality frameworks, such as Data2vec.¹¹
- (ii) Positional embeddings for mask tokens lead to leakage of location information. In masked autoencoders, each masked part is replaced by a share-weighted learnable mask token. All the mask tokens need to be provided with their location information in input data by positional embeddings. Then after processing by autoencoders, each mask token is used to reconstruct the corresponding masked part. Providing location information is not an issue for languages and images because they do not contain location information. While point cloud naturally has location information in the data, leakage of location information to mask tokens makes the reconstruction task less challenging, which is harmful for autoencoders learning latent features. We address this issue by shifting mask tokens from the input of the autoencoder’s encoder to the input of the autoencoder’s decoder. This delays the leakage of location information and enables the encoder to focus on learning features from unmasked parts.
- (iii) Point cloud carries information in a different density compared to languages and images. Languages contain high-density

information, while images contain heavy redundant information.¹² In the point cloud, information density distribution is relatively uneven. The points that make up key local features (e.g., sharp corners and edges) contain a much higher density of information than the points that make up less important local features (e.g., flat surfaces). In other words, if being masked, the points that contain high-density information are more difficult to be recovered in the reconstruction task. This can be directly observed in reconstruction examples, as shown in Figure 2. Taking the last row of Figure 2 for illustration, the masked desk surface (left) can be easily recovered, while the reconstruction of the masked motorcycle’s wheel (right) is much worse. Although the point cloud contains uneven density of information, we find that random masking at a high ratio (60–80%) works well, which is surprisingly the same as images. This indicates the point cloud is similar to images instead of languages, in terms of information density.

Driven by the analysis, we propose a novel self-supervised learning framework for **Point** cloud by designing a neat and efficient scheme of **Masked AutoEncoders**, termed as **Point-MAE**. As shown in Figure 3, our Point-MAE mainly consists of a point cloud masking

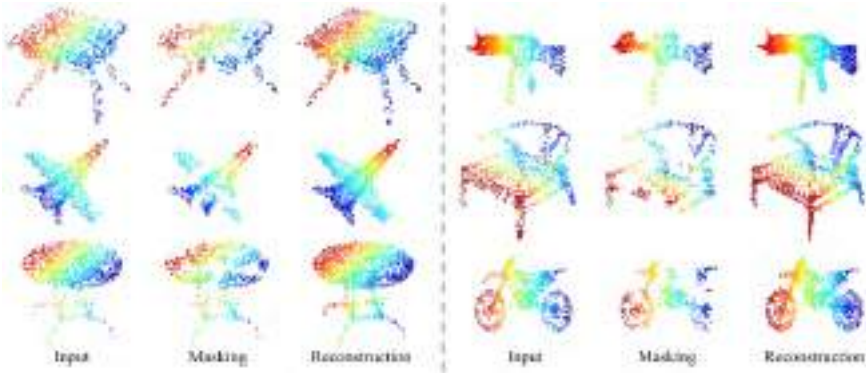


Fig. 2. Reconstruction examples on ShapeNet validation set. In each group, we show the original input (i.e., ground truth), masked point cloud, and reconstruction result from left to right. The masking ratio is 60%. It can be observed directly that reconstructions of key local features (such as sharp corners) are much worse than reconstructions of less important local features (such as flat surfaces).

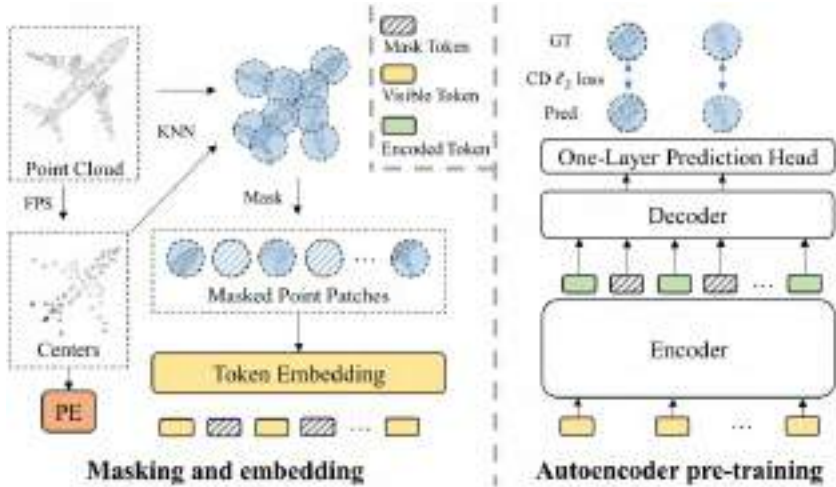


Fig. 3. Overall scheme of our Point-MAE. On the left, we show the masking and embedding process. The input cloud is divided into point patches, which are masked randomly and then embedded. Autoencoder pre-training is shown on the right. The encoder only processes visible tokens. Mask tokens are added to the input sequence of the decoder to reconstruct masked point patches.

and embedding module and an autoencoder. The input point cloud is divided into irregular point patches, which are randomly masked at a high ratio to reduce data redundancy. Then, the autoencoder learns high-level latent features from unmasked point patches, aiming to reconstruct masked point patches in coordinate space. Specifically, our autoencoder’s backbone is entirely built by standard transformer blocks and adopts an asymmetric encoder–decoder structure.¹² The encoder only processes unmasked point patches. Then taking both encoded tokens and mask tokens as input, the lightweight decoder with a simple prediction head reconstructs masked point patches. Compared to processing mask tokens from the input of the encoder, shifting mask tokens to the lightweight decoder results in significant computational savings, more importantly, avoiding early leakage of location information.

Our approach is effective, and pre-trained models generalise well on various downstream tasks. In object classification tasks, our Point-MAE achieves 85.18% accuracy on the hardest setting of real-world dataset ScanObjectNN and 94.04% accuracy on a clean object dataset ModelNet40, outperforming all the other

self-supervised learning methods. Meanwhile, Point-MAE surpasses all the dedicated transformer models from supervised learning. In the few-shot object classification, Point-MAE significantly advances state-of-the-art accuracies by 1.5–2.3% on different settings of ModelNet40. When generalised to the part segmentation task, Point-MAE largely improves the baseline by 1% mean IoU.

Our main contributions can be summarised as follows:

- (1) We propose a novel scheme of masked autoencoders for point cloud self-supervised learning, addressing key issues including backbone architecture, early leakage of location information, and information density of the point cloud. Our approach is neat and efficient, with high generalisation capability on various downstream tasks, outperforming all the other self-supervised learning methods.
- (2) We show with our approach that a simple architecture that is entirely based on standard transformer can surpass dedicated transformer models from supervised learning. This result suggests that standard transformer can serve as a potential unified architecture in the point cloud discipline.
- (3) From the perspective of multi-modal learning, our work inspires that unified architectures for languages, especially images, such as masked autoencoders, are also applicable for point cloud, when equipped with a modality-specific embedding module and a task-specific output head. We hope our field could be further advanced with the joint of other modality data.

2. Related Work

2.1. Self-Supervised Learning

In the machine learning field, Self-Supervised Learning (SSL) is defined as “the machine predicts any parts of its input for any observed part”.^a The main ideas can be summarised as follows: (a) supervision labels are generated from the data itself instead of human annotating and (b) the model predicts parts of the data from

^a<https://aaai.org/Conferences/AAAI-20/invited-speakers/>.

other parts.¹⁹ This process is usually done by designing a pre-text task, which relieves the high demand for manual labelling data.

2.1.1. *SSL for NLP and image*

In the NLP field, SSL has been well developed. Generative SSL methods such as BERT² gain huge success by designing pre-text tasks that mask input tokens and pre-train the model to predict original vocabularies. In computer vision for images, contrastive SSL methods^{8,9,20–22} aim to discriminate the degree of similarities between different augmented images. These methods have dominated until recent generative SSL methods^{12,13,23} result in more competitive performance. For example, MAE¹² randomly masks input patches and pre-trains the model to recover masked patches in pixel space.

2.1.2. *SSL for point cloud*

SSL has also been widely studied for point cloud representation learning.^{24–33} Pre-text tasks are relatively diverse. Among them, Depth-Contrast²⁵ sets an instance discrimination task for two augmented versions of an input point cloud. OcCo²⁴ attempts to recover the original point cloud from the occluded point cloud in camera views. IAE³³ adopts an autoencoder to reconstruct implicit features from augmented inputs. A recent work Point-BERT¹⁵ proposes a BERT-style pre-training strategy by masking input tokens and aims to predict discrete tokens of masked parts, with the assistance of dVAE.¹⁷ Different from previous methods, we attempt to design a neat scheme for point cloud self-supervised learning.

2.2. *Autoencoders*

Generally, an autoencoder consists of an encoder followed by a decoder. The encoder is responsible for encoding inputs to high-level latent features. Then the decoder decodes latent features, aiming to reconstruct the input. The optimisation goal is to make the reconstructed data as similar as possible to the original input, such as mean squared error loss in pixel space for images.

Specifically, our approach belongs to the class of denoising autoencoders. The main idea of denoising autoencoders is to enhance the robustness of the model by introducing input noise. Following the

same principle, masked autoencoders introduce input noise through a masking operation. For example, in NLP, BERT² adopts masked language modelling. It randomly masks tokens from the input and then applies an autoencoder to predict vocabularies corresponding to masked tokens. In computer vision, both MAE¹² and SimMIM¹³ propose a similar masked image modelling, which randomly masks input image patches. Then autoencoders are applied to predict the masked patches in pixel space. Inspired by the earlier ideas, our work aim to introduce masked autoencoders to point cloud.

2.3. *Transformer*

Transformer¹⁴ model global dependencies of input through the self-attention mechanism and have dominated in NLP.^{2-5,34} Since ViT,¹⁸ transformer architectures have been popular in computer vision.³⁵⁻⁴² It also shows strength in multi-modality learning.¹¹ However, as backbones for masked autoencoders, transformer architectures for point cloud representation learning are less developed. PCT⁴¹ designs a dedicated input embedding layer and modifies the self-attention mechanism in transformer layers. PointTransformer⁴⁰ also modifies the transformer layer and uses extra aggregating operations between transformer blocks. These modifications largely limit the joint of other modality data from the perspective of multi-modality learning. Set transformer⁴³ and the perceiver line of works⁴⁴⁻⁴⁶ mainly focus on supervised learning for multi-modality data. However, their experiment results for point cloud are less satisfactory. The recent work Point-BERT¹⁵ introduces a standard transformer architecture but requires DGCNN¹⁶ to assist pre-training. Different from previous works, our work presents an architecture that is entirely based on standard transformer.

3. Point-MAE

We aim to design a neat and efficient scheme of masked autoencoders for point cloud self-supervised learning. Figure 3 illustrates the overall scheme of our approach Point-MAE. The input point cloud is first processed by a masking and embedding module. Then a standard transformer-based autoencoder is adopted, including a simple prediction head, to reconstruct the masked parts of the input point cloud.

3.1. Point Cloud Masking and Embedding

Unlike images in computer vision that can be naturally divided into regular patches, point cloud consists of unordered points in 3D space. Based on its property, we process the input point cloud through three stages: point patches generation, masking, and embedding.

3.1.1. Point patches generation

Following Point-BERT,¹⁵ we divide input point cloud into irregular point patches (may overlap) via Farthest Point Sampling (FPS) and K -Nearest Neighbourhood (KNN) algorithm. Formally, given an input point cloud with p points $X^i \in \mathbb{R}^{p \times 3}$, FPS is applied to sample n points for centres CT in point patches. Based on centre points, KNN selects k -nearest points from input for corresponding point patches P :

$$CT = \text{FPS}(X^i), \quad CT \in \mathbb{R}^{n \times 3}, \quad (1)$$

$$P = \text{KNN}(X^i, CT), \quad P \in \mathbb{R}^{n \times k \times 3}. \quad (2)$$

Note that in point patches, each point is represented by normalised coordinates with respect to its centre point. This leads to better convergence.

3.1.2. Masking

Considering point patches may overlap, we mask them separately, in order to keep information complete in each point patch. With a masking ratio m , the set of masked patches is denoted as $P_{\text{gt}} \in \mathbb{R}^{mn \times k \times 3}$, which is used as ground truth in the computing of reconstruction loss. As for masking strategy, we find random masking at a high ratio (60–80%) works well for our approach (see Section 4.2.6).

3.1.3. Embedding

For the embedding of each masked point patch, we replace it with a share-weighted learnable mask token. We denote the full set of mask tokens as $T_m \in \mathbb{R}^{mn \times C}$, where C is the embedding dimension. For the unmasked (visible) point patches, a naive idea is to flatten and embed them with a trainable linear projection, similar to ViT.¹⁸ However, we argue that linear embedding fails to follow the principle

of permutation invariance.¹ A more reasonable embedding method should be adopted. To keep neat, we implement a lightweight PointNet,¹ which mainly consists of MLPs and max pooling layers. The visible point patches $P_v \in \mathbb{R}^{(1-m)n \times k \times 3}$ are hence embedded into visible tokens T_v :

$$T_v = \text{PointNet}(P_v), \quad T_v \in \mathbb{R}^{(1-m)n \times C}. \quad (3)$$

Considering point patches are represented in normalised coordinates, providing centres' position information to embedding tokens is essential. A simple method for Position Embedding (PE) is mapping coordinates of centres to embedding dimension with a learnable MLP, following previous works.^{15,40} Note that we use two separate PE for encoder and decoder respectively in our autoencoder, introduced in the following.

3.2. Autoencoder's Backbone

Our autoencoder's backbone is entirely based on standard transformer, with an asymmetric encoder-decoder design.¹² The last layer of the autoencoder adopts a simple prediction head to achieve the reconstruction target.

3.2.1. Encoder-decoder

Our encoder consists of standard transformer blocks and only encodes visible tokens T_v without mask tokens T_m . The encoded tokens are denoted as T_e . Furthermore, positional embeddings are added to every transformer block, providing location information.

Our decoder is similar to the encoder but contains fewer transformer blocks. It takes both encoded tokens T_e and masks tokens T_m as input. A full set of positional embeddings is added to every transformer block, providing location information to all the tokens. After processing, the decoder only outputs the decoded mask tokens H_m , which are fed to the following prediction head. The encoder-decoder structure is formulated as follows:

$$T_e = \text{Encoder}(T_v), \quad T_e \in \mathbb{R}^{(1-m)n \times C}, \quad (4)$$

$$H_m = \text{Decoder}(\text{concat}(T_e, T_m)), \quad H_m \in \mathbb{R}^{mn \times C}. \quad (5)$$

In our encoder–decoder structure, we shift the mask tokens to the lightweight decoder instead of processing them from the input of the encoder. This design is beneficial from two aspects. First, as we use high masking ratios, shifting mask tokens significantly reduces the number of input tokens for the encoder. Therefore, we can save computational resources due to the quadratic complexity of transformer. More importantly, shifting mask tokens to the decoder can avoid early leakage of location information to the encoder, making the encoder learn latent features better (see Section 4.2.6).

3.2.2. Prediction head

As the last layer of backbone, the prediction head aims to reconstruct masked point patches in coordinate space. We simply use a fully connected (FC) layer as our prediction head. Taking the output H_m from the decoder, the prediction head projects it to a vector, which has the same number of dimensions as the total number of coordinates in a point patch. Then followed by a reshape operation, predicted masked point patches P_{pre} are obtained:

$$P_{\text{pre}} = \text{Reshape}(\text{FC}(H_m)), \quad P_{\text{pre}} \in \mathbb{R}^{mn \times k \times 3}. \quad (6)$$

3.3. Reconstruction Target

Our reconstruction target is to recover the coordinates of the points in every masked point patch. Given the predicted point patches P_{pre} and ground truth P_{gt} , we compute the reconstruction loss by l_2 Chamfer Distance:⁴⁷

$$L = \frac{1}{|P_{\text{pre}}|} \sum_{a \in P_{\text{pre}}} \min_{b \in P_{\text{gt}}} \|a - b\|_2^2 + \frac{1}{|P_{\text{gt}}|} \sum_{b \in P_{\text{gt}}} \min_{a \in P_{\text{pre}}} \|a - b\|_2^2. \quad (7)$$

4. Experiments

We conduct the following experiments with our Point-MAE⁴⁸ and further extend with more experiments: (a) pre-training the model on ShapeNet⁶ training set. (b) evaluating the pre-trained model on various downstream tasks, including object classification, few-shot learning, and part segmentation. (c) We study different masking

strategies and show the effect of shifting mask tokens. Additionally, we provide studies on the effect of model size and pre-training datasets.

In our Point-MAE, for different resolutions of the input point cloud, we divide them into different numbers of patches with a linear scaling. A typical input with $p = 1024$ points is divided into $n = 64$ point patches. For the KNN algorithm, we set $k = 32$ to keep the number of points in each patch constant. In the autoencoder’s backbone, the encoder has 12 transformer blocks, while the decoder has 4 transformer blocks. Each transformer block has 384 hidden dimensions and 6 heads. MLP ratio in transformer blocks is set to 4. Note in downstream tasks, the decoder is discarded.

4.1. *Pre-training Setup*

ShapeNet consists of about 51,300 clean 3D models, covering 55 common object categories. We split the dataset into a training set and a validation set but only conduct pre-training on the training set. For each instance, we sample 1024 points via FPS as input point cloud. Note that we only apply standard random scaling and random translation for data augmentation during pre-training. For pre-training details, we use an AdamW optimiser⁴⁹ and cosine learning rate decay.⁵⁰ The initial learning rate is set to 0.001, with a weight decay of 0.05. We pre-train our model for 300 epochs, with a batch size of 128. To demonstrate the effectiveness of our method, we visualise reconstruction results on ShapeNet validation set in Figure 4. The model is pre-trained with a masking ratio of 60%, but it is able to reconstruct inputs with different masking ratios. This high generalisation capability can be expected, as our model learns high-level latent features well. Furthermore, our method speeds up pre-training by $1.7\times$ compared to Point-BERT.¹⁵

4.2. *Downstream Tasks*

4.2.1. *Object classification on real-world dataset*

In SSL for point cloud, one of the main concerns is that the commonly used dataset for pre-training, ShapeNet,⁵⁷ only contains clean object models, without any scene context, such as backgrounds. Motivated

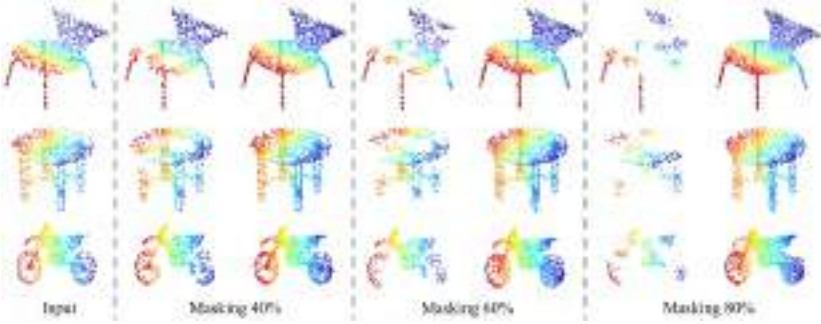


Fig. 4. Reconstruction results on ShapeNet validation set. The model is pre-trained with a masking ratio of 60% but can generalise well on inputs with different masking ratios. Inputs are shown in the leftmost column. In the following columns, we show the masked input (left) and reconstruction (right) with different masking ratios.

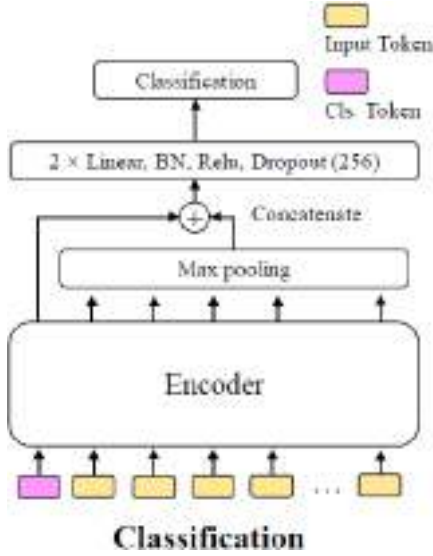


Fig. 5. Model structure for classification and few-shot learning.

by this, we evaluate our pre-trained model on a challenging real-world dataset, ScanObjectNN,⁵⁴ in which the objects are scanned from real-world indoor scene with backgrounds. As shown in Figure 5, for classification-related tasks, we add a classification token to the input tokens. Taking the processed input tokens, we adopt a max

Table 1. Object classification on real-world ScanObjectNN dataset. We evaluate our approach on three variants, among which PB-T50-RS is the hardest setting.

Methods	OBJ-BG(%)	OBJ-ONLY(%)	PB-T50-RS(%)
PointNet ¹	73.3	79.2	68.0
SpiderCNN ⁵¹	77.1	79.5	73.7
PointNet++ ⁵²	82.3	84.3	77.9
DGCNN ¹⁶	82.8	86.2	78.1
PointCNN ⁵³	86.1	85.5	78.5
BGA-DGCNN ⁵⁴	—	—	79.7
BGA-PN++ ⁵⁴	—	—	80.2
GBNet ⁵⁵	—	—	80.5
PRANet ⁵⁶	—	—	81.0
Transformer ¹⁵	79.86	80.55	77.24
Transformer-OcCo ¹⁵	84.85	85.54	78.79
Point-BERT ¹⁵	87.43	88.12	83.07
Point-MAE	90.02	88.29	85.18

pooling operation and concatenate the resulted feature with the processed classification token. Then, the concatenated feature is fed to an MLP to complete classification. BatchNorm, RELU activation, and Dropout with a ratio of 0.5 are adopted in each layer of MLP. In this downstream task, the model is fine-tuned for 300 epochs with a batch size of 32. We use an AdamW optimiser⁴⁹ with an initial learning rate of 0.0005 and a weight decay of 0.05. We also adopt a cosine scheduler⁵⁰ to optimise the learning rate for each epoch. We conduct experiments on three variants and show the results in Table 1. Our Point-MAE largely improves the baselines of standard transformer. On the hardest variant PB-T50-RS, our model achieves 85.18% accuracy, outperforming Point-BERT¹⁵ by 2.11%. Though being pre-trained on clean objects, Point-MAE generalises well on real-world data, presenting a strong generalisation capability.

4.2.2. Object classification on clean objects dataset

We evaluate the pre-trained model on ModelNet40⁶² for object classification. ModelNet40 consists of 12,311 clean 3D CAD models, covering 40 object categories. We follow standard protocols to split ModelNet40 into 9843 instances for the training set and 2468 for

the testing set. Standard random scaling and random translation are applied for data augmentation during training. The training settings are the same as in the earlier subsection. For fair comparisons, we also use the standard voting method⁵⁹ during testing.

Experiment results are presented in Table 2. All the reported methods are given 1024 points that only contain coordinate information without any normal information. Point-MAE achieves 93.8% accuracy, improving 2.4% accuracy compared to training from scratch (91.4%). Compared with other self-supervised learning methods, Point-MAE achieves state-of-the-art performance. Specifically, our approach with standard transformer backbone surpasses IAE³³ that uses a more powerful DGCNN¹⁶ as the backbone (in Table 2, DGCNN achieves a higher accuracy when training from scratch). Besides, Point-MAE outperforms sophisticated Point-BERT¹⁵ by 0.6% accuracy. Besides, our approach surpasses all the dedicated transformer models from supervised learning. Furthermore, given

Table 2. Object classification on ModelNet40. [T] represents the model is based on modified transformer. [ST] represents the standard transformer models.

Supervised Methods	Accuracy (%)
PointNet ¹	89.2
PointNet++ ⁵²	90.7
PointCNN ⁵³	92.5
KPConv ⁵⁸	92.9
DGCNN ⁵³	92.9
RS-CNN ⁵⁹	92.9
[T]PCT ⁴¹	93.2
[T]PVT ⁶⁰	93.6
[T]PointTransformer ⁴⁰	93.7
[ST]Transformer ¹⁵	91.4
Self-supervised methods	Accuracy (%)
OcCo ²⁴	93.0
STRL ⁶¹	93.1
IAE ³³	93.7
[ST]Transformer-OcCo ¹⁵	92.1
[ST]Point-BERT ¹⁵	93.2
[ST] Point-MAE	93.8

8,192 points as input, our Point-MAE achieves 94.04% accuracy. Note that this improvement is significant as ModelNet40 is a relatively small dataset.

4.2.3. Few-shot learning

We follow previous works^{15,24,63} to conduct few-shot learning experiments on ModelNet40,⁶² adopting n -way, m -shot setting, where n is the number of classes randomly selected from the dataset and m is the number of objects randomly sampled for each class. We use the above-mentioned $n \times m$ objects for training. During testing, we randomly sample 20 unseen objects from each of n classes for evaluation. The training settings also follow the classification tasks except training epochs are 150. Following the standard protocol, we conduct 10 independent experiments for each setting and report mean accuracy with standard deviation. The results with the setting of $n \in \{5, 10\}$ and $m \in \{10, 20\}$ are presented in Table 3. Our Point-MAE significantly advances state-of-the-art accuracies of four settings by 1.5–2.3%, with smaller deviations.

4.2.4. Part segmentation

We evaluate the representation learning capability of our Point-MAE on ShapeNetPart dataset,⁶⁴ which contains 16,881 objects covering 16 categories. We follow previous works^{1,15,52} to sample 2,048 points as input for each object, which results in 128 point patches. As shown in Figure 6, our segmentation head is simple and does

Table 3. Few-shot classification on ModelNet40. We conduct 10 independent experiments for each setting and report mean accuracy (%) with standard deviation.

Methods	5-way 10-shot	5-way 20-shot	10-way 10-shot	10-way 20-shot
DGCNN-rand ²⁴	31.6 $\pm$ 2.8	40.8 $\pm$ 4.6	19.9 $\pm$ 2.1	16.9 $\pm$ 1.5
DGCNN-OcCo ²⁴	90.6 $\pm$ 2.8	92.5 $\pm$ 1.9	82.9 $\pm$ 1.3	86.5 $\pm$ 2.2
Transformer-rand ¹⁵	87.8 $\pm$ 5.2	93.3 $\pm$ 4.3	84.6 $\pm$ 5.5	89.4 $\pm$ 6.3
Transformer-OcCo ¹⁵	94.0 $\pm$ 3.6	95.9 $\pm$ 2.3	89.4 $\pm$ 5.1	92.4 $\pm$ 4.6
Point-BERT ¹⁵	94.6 $\pm$ 3.1	96.3 $\pm$ 2.7	91.0 $\pm$ 5.4	92.7 $\pm$ 5.1
Point-MAE	96.3 $\pm$ 2.5	97.8 $\pm$ 1.8	92.6 $\pm$ 4.1	95.0 $\pm$ 3.0

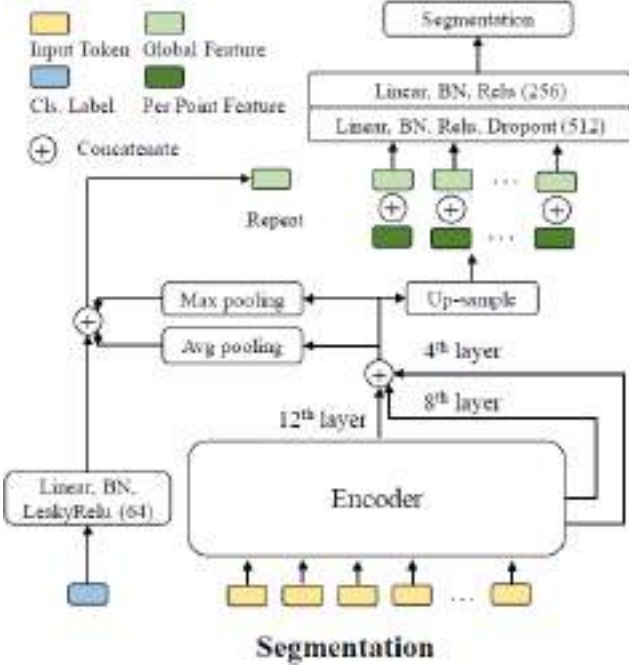


Fig. 6. Model structure for segmentation.

not use any propagating operation or DGCNN.¹⁶ For a fair comparison, it has a similar weight with Point-BERT.¹⁵ We take the features from 4th, 8th, and 12th transformer block and then concatenate them together. After doing max pooling and average pooling separately, we concatenate them together with the class feature, resulting in a global feature. The concatenated feature of three layers can be regarded as point features for the centre points of point patches. Therefore, we up-sample⁵² these sparse centre points' features to the features of every input point based on point coordinates. Finally, we concatenate global features and per-point features and then feed them to an MLP to predict the part label for each point, completing the part segmentation task. Note that no voting methods or data augmentation is used during testing. As shown in Table 4, we report mean IoU (mIoU) for all instances, with IoU for each category. Our Point-MAE achieves 86.1% mIoU, improving the baseline by 1% mIoU. Point-MAE with a simple segmentation head also outperforms Point-BERT,¹⁵ which uses DGCNN¹⁶ and propagation in their segmentation head.

Table 4. Part segmentation on ShapeNetPart dataset. We report mean IoU for all instances $mIoU_I$ (%), with IoU (%) for each category.

Methods	$mIoU_I$	Aero	Bag	Cap	Car
		Chair	e-phone	Guitar	Knife
		Lamp	Laptop	Motor	Mug
		Pistol	Rocket	s-board	Table
PointNet ¹	83.7	83.4	78.7	82.5	74.9
		89.6	73.0	91.5	85.9
		80.8	95.3	65.2	93.0
		81.2	57.9	72.8	80.6
PointNet++ ⁵²	85.1	82.4	79.0	87.7	77.3
		90.8	71.8	91.0	85.9
		83.7	95.3	71.6	94.1
		81.3	58.7	76.4	82.6
DGCNN ¹⁶	85.2	84.0	83.4	86.7	77.8
		90.6	74.7	91.2	87.5
		82.8	95.7	66.3	94.9
		81.1	63.5	74.5	82.6
Transformer ¹⁵	85.1	82.9	85.4	87.7	78.8
		90.5	80.8	91.1	87.7
		85.3	95.6	73.9	94.9
		83.5	61.2	74.9	80.6
Point-BERT ¹⁵	85.6	84.3	84.8	88.0	79.8
		91.0	81.7	91.6	87.9
		85.2	95.6	75.6	94.7
		84.3	63.4	76.3	81.5
Point-MAE	86.1	84.3	85.0	88.3	80.5
		91.3	78.5	92.1	87.4
		86.1	96.1	75.2	94.6
		84.7	63.5	77.1	82.4

4.2.5. 3D object detection

We further evaluate the performance of our method on 3D object detection tasks to verify the capability of understanding scene-level point clouds. We conduct a study on ScanNet. ScanNet contains 1513 scene-level samples covering 21 categories of objects. We follow the standard protocol to fine-tune the pre-trained model with 1201 samples and test on the other 312 scenes. The result is shown in Table 5. Our method outperforms other SSL methods consistently.

Table 5. 3D object detection on ScanNet.

Methods	AP25	AP50
PointContrast ⁶⁵	59.2	38.0
DepthContrast ²⁵	61.3	—
IAE ³³	61.5	39.8
Point-BERT ¹⁵	61.0	38.3
Point-MAE	63.0	42.4

Table 6. Semantic segmentation results on the S3DIS Area 5. We report mean accuracy (mAcc) and mean intersection-over-union (mIoU).

Methods	Input	mAcc	mIoU
PointNet ¹	xyz, rgb	49.0	41.1
PointNet++ ⁵²	xyz, rgb	67.1	53.5
PointCNN ⁵³	xyz, rgb	63.9	57.3
PCT ⁴¹	xyz, rgb	67.7	61.3
Transformer	xyz	68.6	60.0
Point-BERT ¹⁵	xyz	69.7	60.5
Point-MAE	xyz	69.9	60.8

4.2.6. 3D semantic segmentation

We also evaluate our method on 3D semantic segmentation tasks to show the ability of scene-level point cloud understanding. We conduct a study on S3DIS dataset, which contains 6 large indoor areas, comprising a total of 271 rooms and 13 semantic categories. Following common practice, we reserved Area 5 for testing and use other areas for fine-tuning the pre-trained model. The result is shown in Table 6. Our method outperforms other SSL methods and non-SSL methods, showing advanced ability in scene understanding.

4.3. Ablation Study

4.3.1. Masking strategy

To find a proper masking strategy for our method, we compare two masking types with different masking ratios. No voting method is

Table 7. Ablation study on masking strategy. We conduct experiments using two masking strategies with different masking ratios (%) and report pre-train loss (10^{-3}) as well as fine-tune accuracy (%).

Type	Ratio	Loss	Acc.
Block	40	2.83	92.67
Block	60	2.89	92.67
Block	80	2.98	92.50
Random	40	2.49	92.46
Random	50	2.54	92.43
Random	60	2.60	93.19
Random	70	2.68	93.11
Random	80	2.77	93.03
Random	90	2.89	92.63

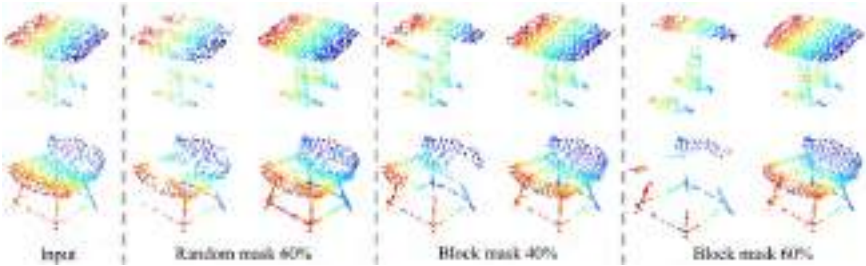


Fig. 7. Reconstructions with different masking strategies. We mainly show three different masking strategies for the same inputs (leftmost). In each column, masked inputs (left) and reconstructions (right) are shown. Instances are from ShapeNet validation set.

used during testing. The reconstruction loss and fine-tune accuracy on ModelNet40 are presented in Table 7. We also visualise reconstructions with different masking strategies in Figure 7.

The block masking^{7,15} type masks neighbouring point patches, resulting in masked blocks. Though this strategy is harder for reconstruction, adopting a medium masking ratio can also achieve good performance.

The random masking type masks random point patches and empirically results in the best performance with a high masking ratio (i.e., 60–80%). The performance degrades largely with low making ratios and also degrades slightly if the masking ratio is too high.

4.3.2. *Effect of shifting mask tokens*

Our Point-MAE shifts mask tokens from the input of the encoder to the lightweight decoder. To demonstrate the effectiveness of this design, we conduct an experiment in which the mask tokens are processed from the input of the encoder. For fair comparisons, the autoencoder’s backbone adopts the same encoder and prediction head as Point-MAE but without the decoder, resulting in the exact same model on fine-tune tasks. We use random masking at a ratio of 60% in this experiment. After pre-training, a smaller reconstruction loss is observed. However, for the fine-tune performance on ModelNet40, it achieves 92.14% accuracy, much lower than Point-MAE (93.19%). The result can be explained. At the input of the encoder, all tokens, including mask tokens, are provided with location information by positional embeddings. This causes early leakage of location information because mask tokens are processed for the reconstruction of point patches in coordinate space. The leakage of location information makes the reconstruction task less challenging, and the model cannot learn latent features well, leading to worse fine-tune performance.

4.3.3. *Model size*

We also conduct a simple study on model size. We use random masking at a ratio of 60% during pre-training. No voting method is used during testing in fine-tuning on ModelNet40. Other settings remain the same. As shown in Table 8, our base model has 12 transformer blocks for the encoder and 4 transformer blocks for the decoder, with a transformer dimension of 384, achieving the best accuracy 93.19%. When we only increase the dimension of transformer (Model B), the

Table 8. Additional study on model size. We report the pre-train loss (10^{-3}) and fine-tune accuracy (%) on ModelNet40 for different sizes of models.

Model	Trans. Dim	Enc. Depth	Dec. Depth	Loss	Acc.
A(ours)	384	12	4	2.60	93.19
B	768	12	4	2.59	92.50
C	384	24	8	2.55	92.75

fine-tune accuracy drops to 92.50%. When we only double the depth of the base model (Model C), the fine-tune accuracy drops to 92.75%. Although a larger model, in terms of depth or width, can achieve better reconstruction, it causes overfitting during fine-tuning, which is also observed in training curves.

4.3.4. Pre-training data

Considering in the real-world application, point clouds are always presented in a format that combines objects and background, we hence explore the effects of different pre-training datasets in this subsection. As shown in Table 9, we conduct additional pre-training and fine-tune the resulting model on the corresponding datasets. We find that our proposed method could consistently bring improvement on different datasets with different pre-training datasets compared to direct fine-tuning from a scratch model. We note that our proposed method is effective even when pre-training on noise data (objects with background). The performance gap between pre-training on noise data and clean data needs to be further explored, as ShapeNet is four times larger than ModelNet and ScanObjectNN. We leave this as future work.

Table 9. Ablation study on pre-training data. We report fine-tune accuracies (%). None means we fine-tune the model directly on the target datasets without pre-training.

Fine-tune Dataset	Pre-train Dataset	Ours
ModelNet40	None	91.4
ModelNet40	ModelNet40	92.5
ModelNet40	ScanObjNN(OBJ-BG)	92.3
ModelNet40	ShapeNet	93.2
ScanObjNN(OBJ-BG)	None	79.9
ScanObjNN(OBJ-BG)	ScanObjNN(OBJ-BG)	87.1
ScanObjNN(OBJ-BG)	ShapeNet	90.0
ScanObjNN(OBJ-ONLY)	None	80.6
ScanObjNN(OBJ-ONLY)	ScanObjNN(OBJ-BG)	87.3
ScanObjNN(OBJ-ONLY)	ShapeNet	88.3
ScanObjNN(PB-T50-RS)	None	77.2
ScanObjNN(PB-T50-RS)	ScanObjNN(OBJ-BG)	81.2
ScanObjNN(PB-T50-RS)	ShapeNet	85.2

4.4. Discussion

Here we attempt to analyse the reasons why our Point-MAE outperforms Point-BERT,¹⁵ which also adopts a similar mask and reconstruction framework. First, during pre-training, our reconstruction target is a part from the input data (i.e., coordinates), which is noise-free. While Point-BERT aims to reconstruct high-level latent representations, which are obtained from a frozen dVAE. If the dVAE is not well learned to provide meaningful latent representations, the reconstruction target might be noisy. Second, Point-BERT processes masked tokens from the input. This causes leakage of location information, which is harmful for learning latent features. We address it by shifting mask tokens. Third, we argue the point cloud is similar to images instead of languages, in terms of information density. Empirically, we adopt random masking at a ratio of 60–80%, which is largely different from Point-BERT’s 25–45% block masking.

5. Conclusions and Future Works

In this chapter, we present a novel scheme of masked autoencoders for point cloud self-supervised learning, termed as Point-MAE. Our Point-MAE is neat and efficient, with minimal modifications based on the properties of the point cloud. The effectiveness and high generalisation capability of our approach are verified on various tasks, including object classification, few-shot learning, and part segmentation. Specifically, Point-MAE outperforms all the other self-supervised learning methods. We also show with our approach, a simple architecture that is entirely based on standard transformer can surpass dedicated transformer models from supervised learning. Furthermore, our work inspires the feasibility of applying unified architectures from languages and images to the point cloud.

As for future works, we expect the following aspects could be further explored. First, as we mentioned in the ablation study, though our method could bring performance improvement regardless of the pre-training dataset, it is still unclear how noise data could affect the model performance. This is important from the angle of application, as the point cloud collected from the real-world naturally has noise. Second, as we apply standard transformer architecture, we hope point cloud could join the multi-modality learning frameworks,

which enables learning ability with the help of other modalities, such as languages or images. Third, our method takes the first step and has been verified on small-scale indoor datasets. Further works can also focus on developing our method for large-scale outdoor datasets, aiming to complete more complex downstream tasks, such as detection and segmentation for self-driving tasks.

Acknowledgements

This chapter is based on the paper “Masked autoencoders for point cloud self-supervised learning”, which was originally published in Computer Vision–ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II. We expand with more experimental results. The experiment details are also added, with more figures to show the downstream tasks’ networks. Moreover, we discuss the future directions for this line of work.

References

1. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 652–660 (2017).
2. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint* arXiv:1810.04805 (2018).
3. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell, *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33, pp. 1877–1901 (2020).
4. A. Radford, J. Wu, R. Child, D. Luan, D. Amodei, I. Sutskever, *et al.* Language models are unsupervised multitask learners, *OpenAI Blog* 1(8), 9 (2019).
5. C. Raffel, N. Shazeer, A. Roberts, K. Lee, S. Narang, M. Matena, Y. Zhou, W. Li, and P. J. Liu. Exploring the limits of transfer learning with a unified text-to-text transformer. *arXiv preprint* arXiv:1910.10683 (2019).
6. D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. In *Proceedings of*

- the *IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2536–2544 (2016).
7. H. Bao, L. Dong, and F. Wei. BEiT: BERT pre-training of image transformers. *arXiv preprint* arXiv:2106.08254 (2021).
 8. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning*, pp. 1597–1607 (2020).
 9. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9729–9738 (2020).
 10. M. Chen, A. Radford, R. Child, J. Wu, H. Jun, D. Luan, and I. Sutskever. Generative pretraining from pixels. In *International Conference on Machine Learning*, pp. 1691–1703 (2020).
 11. A. Baevski, W.-N. Hsu, Q. Xu, A. Babu, J. Gu, and M. Auli. Data2vec: A general framework for self-supervised learning in speech, vision and language, pp. 1–27 (2022). <https://ai.facebook.com/research/data2vec-general-framework-for-self-supervised-learning-in-speech-vision-and-language/>.
 12. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. *arXiv preprint* arXiv:2111.06377 (2021).
 13. Z. Xie, Z. Zhang, Y. Cao, Y. Lin, J. Bao, Z. Yao, Q. Dai, and H. Hu. SimMIM: A simple framework for masked image modeling. *arXiv preprint* arXiv:2111.09886 (2021).
 14. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, Vol. 30 (2017).
 15. X. Yu, L. Tang, Y. Rao, T. Huang, J. Zhou, and J. Lu. Point-BERT: Pre-training 3D point cloud transformers with masked point modeling. *arXiv preprint* arXiv:2111.14819 (2021).
 16. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph CNN for learning on point clouds, *ACM Transactions on Graphics (TOG)* **38**(5), 1–12 (2019).
 17. J. T. Rolfe. Discrete variational autoencoders. *arXiv preprint* arXiv:1609.02200 (2016).
 18. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly, et al. An image is worth 16×16 words: Transformers for image recognition at scale. *arXiv preprint* arXiv:2010.11929 (2020).

19. X. Liu, F. Zhang, Z. Hou, L. Mian, Z. Wang, J. Zhang, and J. Tang. Self-supervised learning: Generative or contrastive, *IEEE Transactions on Knowledge and Data Engineering* **35**(1), 857–876 (2023). doi:10.1109/TKDE.2021.3090866.
20. Z. Wu, Y. Xiong, S. X. Yu, and D. Lin. Unsupervised feature learning via non-parametric instance discrimination. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3733–3742 (2018).
21. J.-B. Grill, F. Strub, F. Altché, C. Tallec, P. Richemond, E. Buchatskaya, C. Doersch, B. Avila Pires, Z. Guo, M. Gheshlaghi Azar, *et al.* Bootstrap your own latent—a new approach to self-supervised learning. In *Advances in Neural Information Processing Systems*, Vol. 33, pp. 21271–21284 (2020).
22. X. Chen, H. Fan, R. Girshick, and K. He. Improved baselines with momentum contrastive learning. *arXiv preprint arXiv:2003.04297* (2020).
23. C. Wei, H. Fan, S. Xie, C.-Y. Wu, A. Yuille, and C. Feichtenhofer. Masked feature prediction for self-supervised visual pre-training. *arXiv preprint arXiv:2112.09133* (2021).
24. H. Wang, Q. Liu, X. Yue, J. Lasenby, and M. J. Kusner. Unsupervised point cloud pre-training via occlusion completion. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9782–9792 (2021).
25. Z. Zhang, R. Girdhar, A. Joulin, and I. Misra. Self-supervised pretraining of 3D features on any point-cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10252–10263 (2021).
26. S. Xie, J. Gu, D. Guo, C. R. Qi, L. Guibas, and O. Litany. Pointcontrast: Unsupervised pre-training for 3D point cloud understanding. In *European Conference on Computer Vision*, pp. 574–591 (2020).
27. J. Sauder and B. Sievers. Self-supervised deep learning on point clouds by reconstructing space. In *Advances in Neural Information Processing Systems*, Vol. 32 (2019).
28. P. Achlioptas, O. Diamanti, I. Mitliagkas, and L. Guibas. Learning representations and generative models for 3D point clouds. In *International Conference on Machine Learning*, pp. 40–49 (2018).
29. J. Li, B. M. Chen, and G. H. Lee. So-Net: Self-organizing network for point cloud analysis. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9397–9406 (2018).
30. Y. Yang, C. Feng, Y. Shen, and D. Tian. FoldingNet: Point cloud auto-encoder via deep grid deformation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 206–215 (2018).

31. Y. Rao, J. Lu, and J. Zhou. Global-local bidirectional reasoning for unsupervised representation learning of 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5376–5385 (2020).
32. B. Eckart, W. Yuan, C. Liu, and J. Kautz. Self-supervised learning on 3D point clouds by learning discrete generative models. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8248–8257 (2021).
33. S. Yan, Z. Yang, H. Li, L. Guan, H. Kang, G. Hua, and Q. Huang. Implicit autoencoder for point cloud self-supervised representation learning. *arXiv preprint* arXiv:2201.00785 (2022).
34. Y. Liu, M. Ott, N. Goyal, J. Du, M. Joshi, D. Chen, O. Levy, M. Lewis, L. Zettlemoyer, and V. Stoyanov. RoBERTa: A robustly optimized BERT pretraining approach. *arXiv preprint* arXiv:1907.11692 (2019).
35. L. Yuan, Y. Chen, T. Wang, W. Yu, Y. Shi, Z.-H. Jiang, F. E. Tay, J. Feng, and S. Yan. Tokens-to-token ViT: Training vision transformers from scratch on imageNet. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 558–567 (2021).
36. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10012–10022 (2021).
37. L. Yuan, Q. Hou, Z. Jiang, J. Feng, and S. Yan. VOLO: Vision outlooker for visual recognition. *arXiv preprint* arXiv:2106.13112 (2021).
38. W. Wang, L. Yao, L. Chen, B. Lin, D. Cai, X. He, and W. Liu. Cross-former: A versatile vision transformer hinging on cross-scale attention. *arXiv preprint* arXiv:2108.00154 (2021).
39. W. Wang, E. Xie, X. Li, D.-P. Fan, K. Song, D. Liang, T. Lu, P. Luo, and L. Shao. Pyramid vision transformer: A versatile backbone for dense prediction without convolutions. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 568–578 (2021).
40. H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16259–16268 (2021).
41. M.-H. Guo, J.-X. Cai, Z.-N. Liu, T.-J. Mu, R. R. Martin, and S.-M. Hu. PCT: Point cloud transformer, *Computational Visual Media* **7**(2), 187–199 (2021).
42. G. Chen, M. Wang, Y. Yue, Q. Zhang, and L. Yuan. Full transformer framework for robust point cloud registration with deep information interaction. *arXiv preprint* arXiv:2112.09385 (2021).

43. J. Lee, Y. Lee, J. Kim, A. Kosiorek, S. Choi, and Y. W. Teh. Set transformer: A framework for attention-based permutation-invariant neural networks. In *International Conference on Machine Learning*, pp. 3744–3753 (2019).
44. A. Jaegle, F. Gimeno, A. Brock, O. Vinyals, A. Zisserman, and J. Carreira. Perceiver: General perception with iterative attention. In *International Conference on Machine Learning*, pp. 4651–4664 (2021).
45. A. Jaegle, S. Borgeaud, J.-B. Alayrac, C. Doersch, C. Ionescu, D. Ding, S. Koppula, D. Zoran, A. Brock, E. Shelhamer *et al.* Perceiver IO: A general architecture for structured inputs & outputs. *arXiv preprint* arXiv:2107.14795 (2021).
46. J. Carreira, S. Koppula, D. Zoran, A. Recasens, C. Ionescu, O. Henaff, E. Shelhamer, R. Arandjelovic, M. Botvinick, O. Vinyals *et al.* Hierarchical perceiver. *arXiv preprint* arXiv:2202.10890 (2022).
47. H. Fan, H. Su, and L. J. Guibas. A point set generation network for 3D object reconstruction from a single image. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 605–613 (2017).
48. Y. Pang, W. Wang, F. E. Tay, W. Liu, Y. Tian, and L. Yuan. Masked autoencoders for point cloud self-supervised learning. In *Computer Vision—ECCV 2022: 17th European Conference*, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II, pp. 604–621 (2022).
49. I. Loshchilov and F. Hutter. Decoupled weight decay regularization. *arXiv preprint* arXiv:1711.05101 (2017).
50. I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. *arXiv preprint* arXiv:1608.03983 (2016).
51. Y. Xu, T. Fan, M. Xu, L. Zeng, and Y. Qiao. SpiderCNN: Deep learning on point sets with parameterized convolutional filters. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 87–102 (2018).
52. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems*, Vol. 30 (2017).
53. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on x-transformed points. In *Advances in Neural Information Processing Systems*, Vol. 31 (2018).
54. M. A. Uy, Q.-H. Pham, B.-S. Hua, T. Nguyen, and S.-K. Yeung. Revisiting point cloud classification: A new benchmark dataset and classification model on real-world data. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1588–1597 (2019).

55. S. Qiu, S. Anwar, and N. Barnes. Geometric back-projection network for point cloud classification, *IEEE Transactions on Multimedia* (2021).
56. S. Cheng, X. Chen, X. He, Z. Liu, and X. Bai. PRA-Net: Point relation-aware network for 3D point cloud analysis, *IEEE Transactions on Image Processing* **30**, 4436–4448 (2021).
57. A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su et al. ShapeNet: An information-rich 3D model repository. *arXiv preprint* arXiv:1512.03012 (2015).
58. H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6411–6420 (2019).
59. Y. Liu, B. Fan, S. Xiang, and C. Pan. Relation-shape convolutional neural network for point cloud analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8895–8904 (2019).
60. C. Zhang, H. Wan, S. Liu, X. Shen, and Z. Wu. PVT: Point-voxel transformer for 3D deep learning. *arXiv preprint* arXiv:2108.06076 (2021).
61. S. Huang, Y. Xie, S.-C. Zhu, and Y. Zhu. Spatio-temporal self-supervised representation learning for 3D point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6535–6545 (2021).
62. Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3D ShapeNets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1912–1920 (2015).
63. C. Sharma and M. Kaul. Self-supervised few-shot learning on point clouds. In *Advances in Neural Information Processing Systems*, Vol. 33, pp. 7212–7221 (2020).
64. L. Yi, V. G. Kim, D. Ceylan, I.-C. Shen, M. Yan, H. Su, C. Lu, Q. Huang, A. Sheffer, and L. Guibas. A scalable active framework for region annotation in 3D shape collections, *ACM Transactions on Graphics (TOG)* **35**(6), 1–12 (2016).
65. S. Xie, J. Gu, D. Guo, C. R. Qi, L. Guibas, and O. Litany. PointContrast: Unsupervised pre-training for 3D point cloud understanding. In *European Conference on Computer Vision*, pp. 574–591 (2020).

Chapter 3

You Only Need One Thing One Click: Self-Training for Weakly Supervised 3D Scene Understanding

Zhengzhe Liu^{*,†}, Xiaojuan Qi^{†,§}, and Chi-Wing Fu^{*,¶}

^{*}*The Chinese University of Hong Kong, Hong Kong*

[†]*The University of Hong Kong, Hong Kong*

[‡]*zzliu@cse.cuhk.edu.hk*

[§]*cwfu@cse.cuhk.edu.hk*

[¶]*xjq@eee.hku.edu.hk*

3D scene understanding, e.g., point cloud semantic and instance segmentation, often requires large-scale annotated training data, but clearly, point-wise labels are too tedious to prepare. While some recent methods propose to train a 3D network with small percentages of point labels, we take the approach to an extreme and propose “One Thing One Click,” meaning that the annotator only needs to label one point per object. To leverage these extremely sparse labels in network training, we design a novel self-training approach, in which we iteratively conduct the training and label propagation, facilitated by a graph propagation module. Also, we adopt a relation network to generate the per-category prototype to enhance the pseudo label quality and guide the iterative training. Besides, our model can be compatible to 3D instance segmentation equipped with a point-clustering strategy.

Experimental results on both ScanNet-v2 and S3DIS show that our self-training approach, with extremely sparse annotations, outperforms all existing weakly supervised methods for 3D semantic and instance

segmentation by a large margin, and our results are also comparable to those of the fully supervised counterparts. Codes and models are available at <https://github.com/liuzhengzhe/One-Thing-One-Click>.

1. Introduction

The success of 3D semantic segmentation benefits a lot from the large annotated training data. However, annotating a large amount of point cloud data is exhausting and costly. Taking ScanNet-v2¹ as an example, it takes 22.3 minutes to annotate one scene on average. It is a great burden to annotate the whole dataset, which includes 1,513 scenes, thus potentially restricting further applications that require larger scale data. Thus, efficient approaches to facilitate 3D data annotation are highly desirable.

Recently, some methods²⁻⁴ were proposed to reduce efforts to annotating 3D point clouds. Though they improve annotation efficiency, various issues remain. Scene-level annotation in Wei *et al.*² could impose negative effects on the model in the absence of localisation information, whereas subcloud annotation in Wei *et al.*² requires an extra burden to first divide the input into subclouds and then repeatedly annotate semantic categories in individual subclouds. The 2D image annotation approach³ requires extra labour to prepare a 2D image annotation, which is also a tedious task on its own. Xu *et al.*⁴ presume that the labelled points follow a uniform distribution. Such a requirement can be achieved by subsampling from a fully annotated dataset but is hard for the annotators to follow in practice. Very recent works⁵⁻⁸ further improve the performance with fewer annotations, yet, there is still a certain performance gap compared with the fully supervised approaches; see Figure 1.

In this work, we also aim to reduce the amount of necessary annotations on point clouds, but we take the approach to an extreme by proposing “One Thing One Click,” so the annotator only needs to label one single point per object. To further relieve the annotation burden, such a point can be randomly chosen, not necessarily at the centre of the object. On average, it takes less than 2 minutes to annotate a ScanNet-v2 scene with our “One Thing One Click” scheme (see an example annotation in Figure 2(b), which contains only 13 clicks), which is more than 10× faster compared with the original ScanNet-v2 annotation scheme.

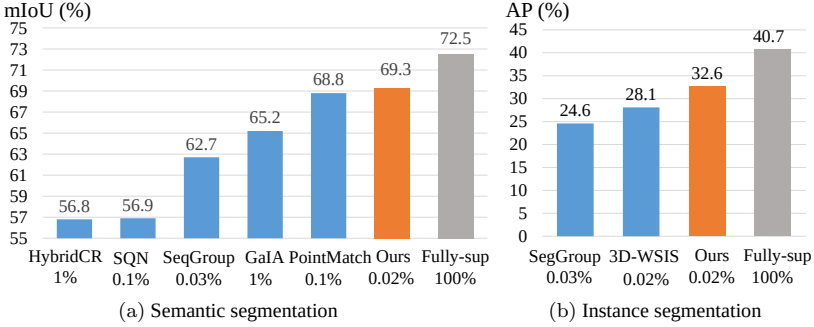


Fig. 1. Comparing our approach of “One Thing One Click++” and the fully supervised version of our method “Fully-sup” on 3D semantic and instance segmentation of ScanNet-v2. It is worth noting that all the works included in this comparison in the blue colour were proposed recently in either 2022 or 2023. Our approach outperformed these very recent works by *training on data with only one label per object*. Note the annotation percentages under each method in the charts.

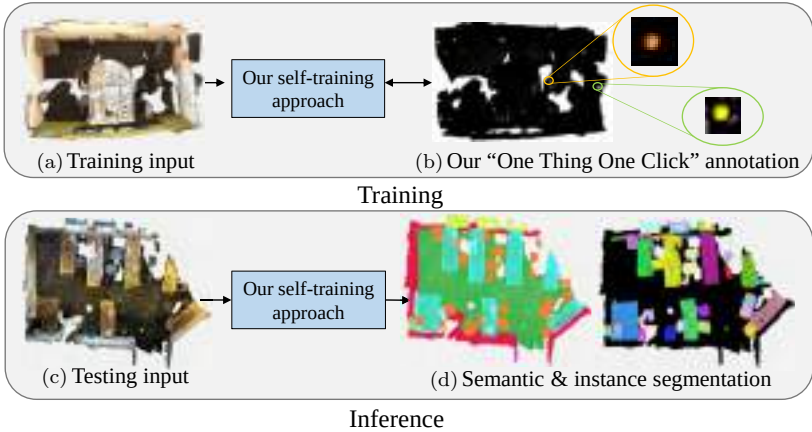


Fig. 2. We train our self-training approach using only our “One Thing One Click” annotations (top). Yet, it can produce plausible semantic and instance segmentation results.

However, directly training a network on extremely sparse labels from our annotation scheme (less than 0.02% in ScanNet-v2 and S3DIS) will easily make the network overfit the limited data and restrict its generalisation ability. Hence, it raises the following question: “can we achieve a performance comparable to a fully supervised baseline given the extremely sparse annotations?”

To meet such a challenge, we propose to design a self-training approach with a label-propagation mechanism for weakly supervised semantic segmentation. On the one hand, with the prediction result of the model, the pseudo labels can be expanded to unknown regions through our graph propagation module. On the other hand, with richer and higher quality labels being generated, the model performance can be further improved. Thus, we conduct the label propagation and network training iteratively, forming a closed loop to boost the performance of each other.

A core problem of label propagation is how to measure the similarity among nodes, especially on irregular 3D point clouds. Previous works on 2D image recognition^{9–11} build a graph model upon 2D pixels and measure the similarity with low-level image features, e.g., coordinates and colours. On the contrary, we propose a graph model building upon the 3D geometric coherent super-voxels, which have more complex geometric structures and a variable number of points in each group. Hence, existing hand-craft features cannot fully reveal the similarity among nodes in our case. To resolve this problem, we further propose a relation network to leverage 3D geometrical information for similarity learning among the graph nodes in 3D. The geometrical similarity and learned similarity are integrated together to facilitate label propagation. To effectively train the relation network with the extremely sparse and category-unbalanced data, we further propose to generate a category-wise prototype with a memory bank for better similarity measurement.

Further, our approach is ready for 3D instance segmentation with a point-clustering strategy. Leveraging the knowledge of the number and location of each instance provided by “One Thing One Click” annotation, point clustering aims to group the points of the same instance to generate instance-level pseudo label and enable the instance-level understanding.

Experiments conducted on two public datasets ScanNet-v2 and S3DIS manifest the effectiveness of the proposed method. With just around 0.02% point annotations, our approach achieves results that are comparable with a fully supervised counterpart; see Figure 1. These results manifest the high efficiency of our “One Thing One Click” scheme for 3D point cloud annotation and the effectiveness of our self-training approach for weakly supervised 3D scene understanding.

This work extends our research work presented in “One Thing One Click”,¹² which was presented at the 2021 IEEE Conference on Computer Vision and Pattern Recognition (CVPR). First, we expand the self-training framework for weakly supervised 3D instance segmentation. Besides, we provide an alternative design for the relation network, which achieves comparable performance to the original design in “One Thing One Click” but is more efficient. At last, we compare our approach with the latest methods on 3D semantic and instance segmentation, and the experimental results demonstrate the superiority of our new approach.

2. Related Work

Semantic segmentation for point cloud: Approaches for 3D semantic segmentation can be roughly divided into point-based methods and voxel-based methods.

Point-based networks take raw point clouds as input. Along this line of works, PointNet¹³ and PointNet++¹⁴ are the pioneering ones. Afterward, convolution-based methods^{15–18} were also proposed for 3D semantic segmentation on point clouds. Besides, Kundu *et al.*¹⁹ proposed to fuse features from multiple 2D views for 3D semantic segmentation. To aggregate together the geometrically homogeneous points, Landrieu *et al.*²⁰ modelled a point cloud as a super point graph. In addition, a number of recent research works^{21–24} are further proposed to improve the performance of point cloud semantic segmentation. Inspired by SPG,²⁰ we expand the sparse labels to geometrically homogeneous super-voxels to generate initial pseudo labels for the first-iteration network training.

Voxel-based networks take the regular voxel-grids as input instead of the raw data.^{25–29} The recently proposed methods SparseConv,³⁰ MinkowskiNet *et al.*,³¹ and OccuSeg *et al.*³² are among the representative works in this branch. In this chapter, we adopt the 3D-UNet architecture described in SparseConv³⁰ as the backbone architecture due to its high performance and applicability.

Weakly supervised 3D semantic segmentation: Although there have been significant advancements in 3D semantic segmentation, the arduous task of point-level annotation greatly limits its practicality. To overcome this challenge, several approaches have been

proposed.^{3,33,34} An early work² utilises the class activation map to generate pseudo point-wise labels from subcloud-level annotations. The performance is, however, limited by the lack of localisation information. In addition, Xu *et al.*⁴ achieve the performance close to fully supervised with less than 10% labels. However, they require the annotations to be uniformly distributed in the point cloud, which is practically very hard for the annotators to follow. Very recently, several approaches^{5-8,35-47} are proposed to further enhance the annotation efficiency and performance of weakly supervised 3D semantic segmentation.

In this work, we propose a new self-training approach with a label propagation module, in which the network training and label propagation are conducted iteratively. Our approach largely reduces the reliance on the quality of the initial annotation and achieves top performances, compared with existing weakly supervised methods, while using only extremely sparse annotations.

3D instance segmentation: 3D instance segmentation aims to derive instance-level understanding of a 3D scene, going beyond semantic segmentation. While existing works^{22,48-50} use point-level annotations for 3D instance segmentation, the labourious and tedious annotation process limits their practical applicability. Recent works^{35,38,51,52} focus on weakly supervised 3D instance segmentation to overcome this challenge. For example, Seg-Group⁵¹ proposes a segment grouping network to hierarchically group unlabelled segments into nearby labelled ones for 3D instance segmentation. Tang *et al.*³⁸ use semantic and spatial relations to adaptively learn inter-superpoint affinity. TWIST⁵² proposes a semi-supervised 3D instance segmentation approach that leverages object-level information to denoise pseudo labels. In this work, we extend our self-training approach to 3D instance segmentation using the One-Thing-One-Click annotation.

Self-training for semantic segmentation on 2D images: Self-training for weakly supervised 2D image understanding has been intensively explored. To reduce the annotation burden for 2D images, researchers proposed a variety of annotation approaches, e.g., image-level categories,⁵³⁻⁵⁶ points,^{57,58} extreme points,^{59,60} scribbles,⁶¹⁻⁶³ and bounding boxes.⁶⁴ With weak supervision, a self-training approach can learn to expand the limited annotations to unknown regions in the domain. Inspired by the previous works in

2D image understanding, we propose a novel self-training framework for weakly supervised 3D scene understanding.

3. Methodology

3.1. Overview

With “One Thing One Click,” we only need to annotate a point cloud with one point per object, as Figure 3(c) shows, and these points can be chosen at random to alleviate the annotation burden. Procedure-wise, given such sparse annotations, we first over-segment the point cloud $X = \{p_i\}$ into geometrically homogeneous super-voxels $V = \{v_j\}$, where $\cup_j v_j = X$ and $v_j \cap v_{j'} = \emptyset$ for $v_j \neq v_{j'}$. Note that throughout this chapter, we use i and j as the indices for points and super-voxels, respectively. Based on the super-voxel partition, we can produce initial pseudo labels of the point cloud by spreading each label to all the points locally in the super-voxel that contains the annotated point. However, as Figure 3(d) shows, the labels are still very sparse. More importantly, the propagated labels distribute

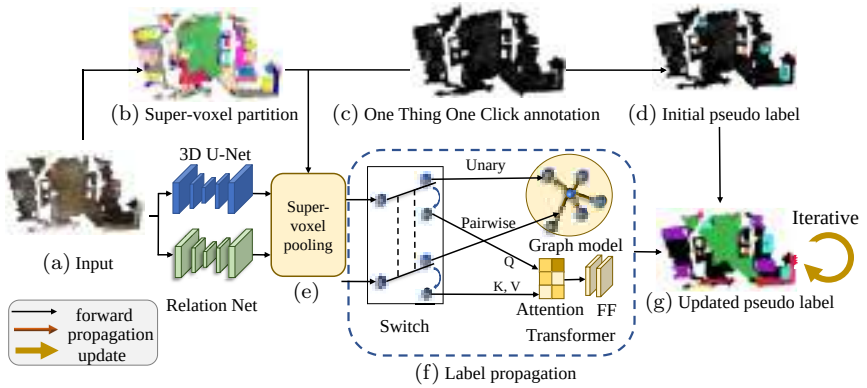


Fig. 3. Overview of our framework. Through a super-voxel partition (b), we expand our “One Thing One Click” annotations (c) to generate the initial pseudo labels (d) for guiding the update of the pseudo labels (g). On the other hand, we adopt the “3D U-Net” for semantic label prediction (blue region) and design the “Relation Net” for super-voxel-based similarity learning (green region). Then, we incorporate a super-voxel pooling (e) to aggregate features from the two networks. Afterward, we adopt either a graph model or a transformer (f) to propagate labels over the point cloud. Further, we iteratively update the predicted labels (g) and train the network.

mainly around the initially annotated points, which are far from the ideal uniform distribution for weakly semantic segmentation, as employed by Xu *et al.*⁴

An important insight in our approach is to iteratively propagate the sparse annotations to unknown regions in the point cloud, while training the network model to guide the propagation process. To achieve this, we adopt the 3D semantic segmentation network Θ (the blue regions in Figure 3) and to learn the label propagation via a feature propagation module (Figure 3(f)). Further, we design the relation network $\mathcal{R}$ (the green regions in Figure 3) to explicitly model the feature similarity. Afterward, predictions with high confidence are further employed as the updated pseudo labels for training the network in the following iteration (Figure 3(g)). This iterative self-training approach couples the label propagation and network training, enabling us to significantly enhance the segmentation quality, as revealed earlier in Figure 1.

3.2. 3D Semantic Segmentation Network

We adopt the 3D U-Net architecture³⁰ as the backbone, denoted as Θ . Its input is point cloud X of N points (Figure 3(a)). Each point has 3D coordinates p_i and colour c_i , where $i \in \{1, \dots, N\}$. The network predicts the probability of each semantic category $P(y_{i,\bar{c}}|p_i, c_i, \Theta)$ of each point p_i , where $\bar{c}$ is the ground-truth category of point p_i . The network is trained with the softmax cross-entropy loss in the following:

$$L_s = -\frac{1}{N} \sum_{i=1}^N \log P(y_{i,\bar{c}}|p_i, c_i, \Theta). \quad (1)$$

In the first iteration, the network is trained with the initial pseudo labels, as shown in Figure 3(d). In subsequent iterations, the network is trained with the updated pseudo labels, as shown in Figure 3(g), which is detailed in the following.

3.3. Pseudo Label Propagation

To facilitate the network training, we propose a label propagation mechanism to effectively propagate labels to unknown regions. Specifically, we provide two options to propagate the feature, i.e., graph

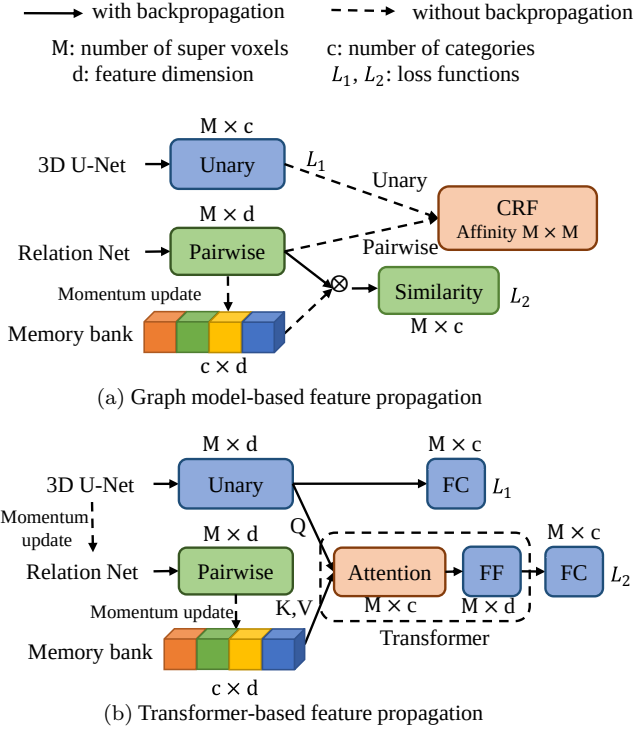


Fig. 4. The alternative approaches for label propagation.

model-based and transformer-based, as shown in Figure 4. We also propose the relation network to explicitly learn the similarity among the super-voxels to facilitate the label propagation process and complement the 3D U-Net.

Graph model-based feature propagation: First, we introduce our graph model-based feature propagation. To start, we leverage the 3D geometrically homogeneous super-voxels to build a graph. Compared with building on points, our graph has significantly fewer nodes to facilitate efficient label propagation.

To derive the prediction $P(y_{j,c}|v_j, \Theta)$ of the j th super-voxel, we apply a super-voxel pooling to aggregate the semantic prediction of the n_j points in v_j .

Graph-based label propagation: The architecture of graph-based label propagation is illustrated in Figure 4(a). To build the graph, we

treat each super-voxel as a graph node and compute the similarity between each pair of super-voxels $v_j, v_{j'}$, which is represented as an edge.

To propagate labels to unknown regions through the graph, we formulate it as an optimisation problem that considers both the network prediction and similarities among the super-voxels to achieve the global optimum with the following energy function similar to Conditional Random Field (CRF):

$$E(Y|V) = \sum_j \psi_u(y_j|V, \Theta) + \sum_{j < j'} \psi_p(y_j, y_{j'}|V, \mathcal{R}, \Theta), \quad (2)$$

where $\mathcal{R}$ is the relation network to be detailed later. The unary term $\psi_u(y_j|V, \Theta)$ represents the super-voxel pooled prediction of the 3D U-Net $P(y_j)$ on super-voxel v_j . Specifically, it denotes the minus log probability of predicting super-voxel v_j to have label y_j . We define it as follows:

$$\psi_u(y_j|V, \Theta) = -\log P(y_j|V, \Theta). \quad (3)$$

The pairwise term $\psi_p(j_k)$ in Equation (2) represents the similarity between super-voxels v_j and $v_{j'}$. We employ both the low-level features and learned features for measuring the similarity, as shown in Equation (4):

$$\begin{aligned} \psi_p(y_j, y_{j'}|V) = & \mathbb{K}(y_j, y_{j'}) \\ & \times \exp \left\{ -\lambda_c \frac{\|c_j - c_{j'}\|^2}{2\sigma_c^2} - \lambda_p \frac{\|p_j - p_{j'}\|^2}{2\sigma_p^2} \right. \\ & \left. - \lambda_u \frac{\|u_j - u_{j'}\|^2}{2\sigma_u^2} - \lambda_f \frac{\|f_j - f_{j'}\|^2}{2\sigma_f^2} \right\}, \end{aligned} \quad (4)$$

where $\mathbb{K}(y_j, y_{j'})$ is 1, if v_j and $v_{j'}$ have different predicted labels, and 0 otherwise. The pairwise term means that the cost will be higher if super-voxels with similar features are predicted to be different classes. Here, $c_j, c_{j'}, p_j, p_{j'}$, and $u_j, u_{j'}$ are the normalised mean colour, mean coordinates, and mean 3D U-Net feature, respectively, of super-voxels v_j and $v_{j'}$.

Relation network: Unlike existing works,⁹⁻¹¹ which build the graph on 2D image pixels, we build our graph on 3D super-voxels,

which have irregular and complex geometrical structures. Therefore, hand-crafted features $p_j, p_{j'}$ and $c_j, c_{j'}$ have the inferior capability to measure the similarity between super-voxels. To address this issue, we propose the *Relation Network* to better leverage the 3D geometrical information and explicitly learn the similarity among super-voxels.

The relation network $\mathcal{R}$ shares the same backbone architecture as the 3D U-Net Θ except for removing the last category-wise prediction layer. It aims to predict a category-related embedding f_j for each super-voxel v_j as the similarity measurement. f_j is the per super-voxel pooled feature in $\mathcal{R}$. In other words, the relation network groups the embeddings of the same category, while pushing those of different categories apart. To this end, we propose to learn a prototypical embedding for each category.

To assist the training of the relation network with sparse and unbalanced training data, we present a memory bank $K = \{k\}$ to generate one categorical prototype for each category, instead of simply regarding the average embedding as the prototype as in prototypical networks.⁶⁵

The embedding f_j generated by $\mathcal{R}$ serves as a “query,” and we compare it with the corresponding “key” k_c in the memory bank with a dot product. The two modules are optimised simultaneously with contrastive learning⁶⁶ as follows:

$$L_c = \frac{1}{M} \sum_j \left(-\log \frac{f_j \cdot k_{\bar{c}} / \tau}{\sum_c f_j \cdot k_c / \tau} \right), \quad (5)$$

where τ is a temperature hyper parameter⁶⁷ and $\bar{c}$ is the ground-truth category of v_j . The contrastive learning is equivalent to a c -way softmax classification task.

Following MOCO,⁶⁸ we update the key representations via a moving average with momentum as shown in the following:

$$k_{\bar{c}} \leftarrow mk_{\bar{c}} + (1 - m)f_j, \quad (6)$$

where m is a momentum coefficient to control the evolving speed of the memory bank.

Our relation net complements with 3D U-Net. It measures the relations between super-voxels using different training strategies and losses, while 3D U-Net aims to project the inputs into the latent

feature space for category assignment. The prediction of relation network is further combined with the prediction of 3D U-Net by multiplying the predicted possibilities of each category to boost the performance. In addition, the relation net offers a stronger measurement of the pairwise term in CRF vs. handcrafted features like colours and also complements with the 3D U-Net features.

Transformer-based label propagation: In the following, we introduce a transformer-based alternative to the graph model-based approach for label propagation, as illustrated in Figure 4(b). Unlike the graph model-based approach that learns the affinity among super-voxels, where the size of the affinity matrix $M \times M$ grows quadratically relative to the number of super-voxels M , the transformer-based label propagation aims to learn the correlation between a super-voxel v_j and a category prototype k_c . Therefore, the size of the attention map $M \times c$ grows proportionally to M , significantly improving efficiency in terms of memory and inference time. Additionally, transformer-based label propagation can be optimised end-to-end, further improving the performance of 3D semantic segmentation.

Specifically, the transformer-based label propagation can be formulated as follows:

$$\hat{f}_j = \Sigma_c \text{softmax} \left(\frac{Q(F_j)K(k_c)}{\sqrt{d_l}} \right) V(k_c), \quad (7)$$

where Q , K , and V represent MLP layers, while F_j represents the feature vector of the 3D U-Net. The transformer then aggregates the category prototype k_c based on the similarity between F_j and k_c . The resulting output feature $\hat{f}_j$ is then concatenated with F_j to make the final prediction for the semantic category.

Inspired by Mean Teacher,⁶⁹ we update the weights of the relation network in our transformer-based label propagation using the moving average of weights in the 3D U-Net with momentum, instead of using stochastic gradient descent (SGD). The weight update is formulated as follows:

$$\mathcal{R}_w \leftarrow m\mathcal{R}_w + (1 - m)\Theta_w, \quad (8)$$

where $\mathcal{R}_w$ and Θ_w represent the w th weight of the relation network and the 3D U-Net, respectively. By using the moving-average strategy, we accumulate the features F_j in the 3D U-Net over time, which

improves the quality and stability of the category prototypes K . Moreover, this approach helps reduce the computational complexity during training.

3.4. Self-Training

With the label propagation, we then propose a self-training approach to update networks Θ and $\mathcal{R}$ and also the pseudo labels Y iteratively. The self-training is started by the “One Thing One Click” annotations and the pre-constructed super-voxel graph. In each iteration, we fix network parameters $\Theta, \mathcal{R}$ and update label Y , and vice versa. There are two steps in each iteration:

- With Θ and $\mathcal{R}$ fixed, the label propagation is conducted to minimise the energy function in Equation (2). Then, the predictions with high confidence are taken as the updated pseudo labels for training the two networks in the following iteration. The confidence of super-voxel v_j , denoted as C_j , is the average of the minus log probability of all n_j points in v_j after the label propagation:

$$C_j = \frac{1}{n_j} \sum_i^{n_j} \log P(y_i | p_i, V, \Theta, \mathcal{R}, G), \quad \text{where } p_i \in v_j, \quad (9)$$

where G denotes the graph propagation.

- With pseudo labels Y , Θ and $\mathcal{R}$ are optimised, respectively.

3.5. 3D Instance Segmentation

To extend our self-training framework for 3D instance segmentation, we propose a point-clustering strategy to iteratively generate instance-level pseudo label and train the instance segmentation network.

In the first training iteration, we utilise the semantic segmentation network trained with the One Thing One Click annotation approach as described earlier, as shown in Figure 5(a). Next, we conduct k -means clustering using the annotated super-voxels $V_{\text{anno}} = \{v_k\}$ as initial centroids. Since our One Thing One Click annotation approach enables each annotated super-voxel to represent an instance, we can predict which instance k the super-voxel v_j belongs to based on the Euclidean distance $\|p_k - p_j\|_2^2$; see Figure 5(b). To enhance the

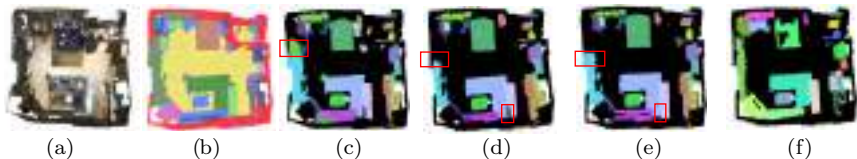


Fig. 5. Pseudo label generation for 3D instance segmentation. (a) Our semantic segmentation result. (b) Initial pseudo label by k -means clustering. (c) Unconnected-segment removal. (d) Iteration 1 of self-training. (e) Iteration 2 of self-training. (f) Ground truth.

robustness of the generated pseudo label, we then filter out small and unconnected semantic segments to obtain the initial instance-level pseudo label (Figure 5(c)).

Then we train the instance segmentation network leveraging the above pseudo label. To extend the 3D U-Net architecture for instance-level segmentation, we incorporate a multi-layer perceptron (MLP) prediction head on the top of 3D U-Net following Ref. [70] to predict the 3D offset o_j . With the predicted offset, we can move a super-voxel v_j towards the centroid coordinate C of the instance that v_j belongs to, such that the super-voxel v_j belonging to the same instance can be grouped together and the point-level clustering in the following iterations can be conducted based on the Euclidean distance $\|p_k - (p_j + o_j)\|_2^2$. In the first training iteration, we freeze the backbone network and only update the offset head. This helps the network maintain its ability to perform semantic segmentation.

In the subsequent iterations, we further update the pseudo instance-level label using k -means clustering based on v_j 's updated coordinate $p_j + o_j$. Then we fine-tune the entire network end-to-end instead of only updating the offset head like the first iteration, so we can further improve the instance segmentation performance.

We repeat the above process and network training iteratively to progressively improve the quality of the pseudo labels and enhance the performance of the model. The pseudo labels after the first and second iterations are illustrated in Figures 5(d) and 5(e), respectively, indicating that our self-training approach improves the instance segmentation performance with more training iterations. Empirically, more iterations of self-training cannot significantly improve the instance segmentation performance. Hence, we conduct self-training on the instance segmentation for two iterations.

During inference, we use the clustering method from PointGroup⁷⁰ to group super-voxels v_j into candidate clusters based on their predicted shifted coordinates $p_j + o_j$ (P branch in PointGroup⁷⁰) instead of their original coordinates p_j (Q branch in PointGroup⁷⁰). Additionally, for semantic prediction, we simplify the approach by averaging the predicted semantic scores of all points belonging to each instance, as proposed by Hou *et al.*,³⁵ instead of using the ScoreNet introduced in PointGroup.⁷⁰

4. Experiments

Datasets: Our experiments are conducted on two large 3D semantic segmentation datasets: ScanNet-v2¹ and S3DIS.⁷¹ **ScanNet-v2¹** contains 1513 3D scans of 20 semantic categories. We annotate the official training set with our “One Thing One Click” scheme and evaluate on the validation and test set. **S3DIS⁷¹** contains 3D scans of 271 rooms containing 13 categories. We follow the official train/validation split to annotate on Areas 1, 2, 3, 4, 6 and report the performance on Area 5.

“One thing one click” annotation details: In order to ensure the randomness of point selection in annotation, we simulate the annotation procedure by selecting a single point inside an object with the same probability for the following experiments. In ScanNet-v2, only 19.74 points per scene are annotated on average with “One Thing One Click” scheme, while this number in the original ScanNet-v2 is 108875.9. In S3DIS, only 36.15 points in each room are annotated on average using “One Thing One Click”, while the original S3DIS has 193797.1 points annotated in each room.

Implementation details: We implement all the modules of our self-training framework including the mean-field solver⁷² for label propagation with the PyTorch⁷³ framework based on the implementation of PointGroup.⁷⁰ Following PointGroup,⁷⁰ due to the GPU capacity, we randomly choose 250k points if the scene contains more points in training. In inference, the network takes the whole scene as input. We set the hyperparameters $D = 32$, $T = 0.9$, $s = 20$, $\tau = 0.07$, $m = 0.9$, $\sigma_c = \sigma_p = \sigma_u = \sigma_f = 1$, $\lambda_c = \lambda_p = \lambda_u = \lambda_f = 1$ with a small validation set. We found that the self-training converges after

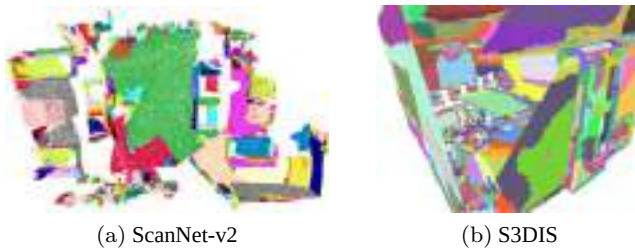


Fig. 6. Visualisation of super-voxel partition on (a) ScanNet-v2 and (b) S3DIS.

five iterations. After that, more iterations of training only brings very minor improvements.

Super-voxel partition: We use the mesh segment results¹ as super-voxels for ScanNet-v2 and the geometrical partition results described in SPG²⁰ for S3DIS super-voxel partition. Our super-voxel partition effectively groups points based on their geometric attributes, as shown in Figure 6.

4.1. *Semantic Segmentation on ScanNet-v2*

Comparing with existing methods: Table 1 reports the benchmark result on ScanNet-v2 test set. The baselines can be roughly divided into two branches: (i) fully supervised approaches with 100% supervision, including several representative works in 3D semantic segmentation, which are the upper bounds of weakly supervised ones; (ii) weakly and semi-supervised approaches.

With less than 0.02% annotated points, our result (69.3% mIoU) outperforms many existing works with full supervision. As for weakly and semi-supervised approaches, our approach outperforms all those very recent ones proposed from 2021 to 2023 with fewer annotations, demonstrating the superiority of our approach over the existing ones.

Results on ScanNet-v2 data-efficient benchmark: In Table 2, we show results on ScanNet-v2 “3D Semantic label with Limited Annotations” benchmark.³⁵ We report the results on the most challenging setting with only 20 points annotated for each scene in Table 3 “Data Efficient”. In this experiment, we use the officially

Table 1. Comparing with the existing methods on ScanNet-v2 test set for 3D semantic segmentation.

Method	Supervision	mIoU (%)
PointNet++ ¹⁴	100%	33.9
SPLATNet ²⁸	100%	39.3
TangentConv ⁷⁴	100%	43.8
PointCNN ¹⁵	100%	45.8
FPCov ⁷⁵	100%	63.9
DCM-Net ⁷⁶	100%	65.8
PointConv ¹⁷	100%	66.6
KPCov ¹⁶	100%	68.4
JSENet ⁷⁷	100%	69.9
SubSparseCNN ³⁰	100%	72.5
MinkowskiNet ³¹	100%	73.6
Virtual MVFusion ¹⁹	100%+2D	74.6
PointTransformer-v2 ²³	100%	75.2
Mix3D ⁷⁸	100%+2D	78.1
Our fully sup baseline	100%	72.5
Superpoint-guided ⁴⁶	10% scenes	52.4
TWIST ⁵²	10% scenes	61.1
2D Knowledge Transfer ⁴⁰	10% scenes	61.2
MPRM ²	scene-level	24.4
MPRM ²	subcloud-level	41.1
MPRM+CRF ²	subcloud-level	43.2
WyPR ⁴²	scene-level	24.0
Zhang <i>et al.</i> ⁴⁴	10.0%	52.0
PSD ⁴¹	1%	54.7
HybridCR ⁵	1%	56.8
SQN ⁶	0.1%	56.9
SegGroup ⁵¹ (MinkowskiNet)	0.028%	62.7
GaIA ⁷	1%	65.2
PointMatch ⁸	0.1%	68.8
One Thing One Click ¹²	0.02%	69.1
Ours	0.02%	69.3

provided 20 points instead of “One Thing One Click” and then employ our self-training approach for semantic segmentation. The results show that our approach is not limited to “One Thing One Click” and is applicable to other annotation schemes. In addition, our approach still outperforms existing works^{35,36,79} under this annotation scheme.

Table 2. Comparing with existing methods on ScanNet-v2 data efficient benchmark.³⁵

Method	Supervision	mIoU (%)
CSC_LA_SEM ³⁵	20 points/scene	53.1
PointContrast_LA_SEM ⁷⁹	20 points/scene	55.0
VIBUS ³⁶	20 points/scene	58.6
One Thing One Click	20 points/scene	59.4

Table 3. Our results and baselines on ScanNet-v2 val. set.

Setting	Annotation (%)	mIoU (%)
Our fully sup baseline	100	72.18
One Thing One Click ^{*12}	0.02	62.18
One Thing One Click ^{†12}	0.02	68.96
One Thing One Click ¹²	0.02	70.45

Notes: *means the baseline model trained with the initial pseudo labels shown in Figure 1(d).

†means disabling graph propagation and relation network during inference, but note that they are still used in training.

Comparing with our baselines: In this section, we first present three important baselines as shown in Table 3 on ScanNet-v2 validation set:

- Table 3 “Our fully sup baseline” is trained with the official 100% annotation provided by ScanNet-v2. It serves as the upper bound of our method.
- The model directly trained with the raw annotated points as Figure 3(c) cannot converge well due to the extreme sparsity of the training data.
- Table 3 “One Thing One Click^{*}”. The model trained with the initial pseudo labels as Figure 3(d) achieves 62.18% mIoU. It serves as the starting point of our self-training approach and is denoted as “our baseline” in the following.

Table 3 “One Thing One Click” manifests that our self-training approach surpasses the baseline by nearly 10% mIoU, attaining a 16% relative improvement. Compared with the fully supervised baseline

with the same network architecture, our performance is only 2% lower.

Table 3 “One Thing One Click[†]” refers to disabling the graph propagation and relation network in inference. Note that they are still being used in training for generating the pseudo labels. This brings no extra computational burden during the inference but helps improve nearly 7% mIoU, comparing with the baseline (68.96% vs. 62.18%).

Results with fewer annotations: To investigate the performance of our approach with even less annotated points, we further annotate ScanNet-v2 with a “Two Things One Click” scheme, where we annotate a single random point on half of the objects chosen randomly in the scene. In this way, only less than 0.01% points are annotated on ScanNet-v2. With even sparse annotations, we still achieve 60.62% mIoU as shown in Table 4. This experiment also demonstrates that our method can still achieve decent performance even though the annotator ignores several objects by mistake in “One Thing One Click” scheme. We further investigate the performance drop with a more challenging “Four Things One Click” scheme. However, the model cannot converge well in the very first iteration due to the insufficient label and the self-training fails in this case.

Qualitative results on ScanNet-v2: Then, we show prediction results on ScanNet-v2 in Figures 7. Through these results, we demonstrate that our approach can produce segmentation results that are comparable to the fully supervised baseline³⁰ with only

Table 4. Two Things One Click results and baselines on ScanNet-v2 val. set.

Method	Annotation (%)	mIoU (%)
Two Things One Click*	0.01	54.71
Two Things One Click [†]	0.01	59.56
Two Things One Click	0.01	60.62

Notes: *means the baseline model trained with the initial pseudo label shown in Figure 3(d).

[†]means disabling graph propagation and relation network during inference, but note that they are still used in training.

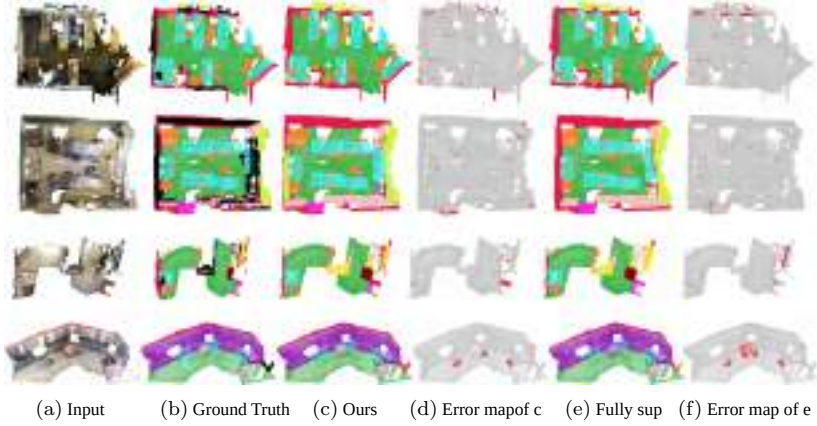


Fig. 7. Qualitative comparisons on ScanNet-v2. (c) Produced by our model trained only with “One Thing One Click” annotations. (e) Fully supervised results of Graham *et al.*³⁰ Red regions in (d) and (f) indicate the wrong predictions.

0.02% annotation. See the error maps shown in (d) and (f) for better visualisations.

Ablation studies: To further study the effectiveness of self-training, graph propagation, and relation network, we conduct ablation studies on these three modules on ScanNet-v2 validation set as shown in Table 5 with single view evaluation.

“3D U-Net” indicates that the labels are propagated only based on the confidence score of the 3D U-Net itself, i.e., the unary term in Equation (2). This ablation is designed to manifest the effectiveness of self-training. The “3D U-Net” column in Table 5 manifests that the performance is consistently improved with self-training strategy even without pairwise energy term in Equation (2) and super-voxel partition.

“3D U-Net+GP” refers to the label propagation with graph model, and the similarities among super-voxels are measured by the coordinates p_i and colours c_i without the learned feature f_i . This ablation study is to show the effectiveness of the graph model. The results in Table 5 indicate that the graph model benefits the label propagation and finally boosts the overall performance by 2% over “3D U-Net” (67.92% vs. 65.91%).

“3D U-Net+Rel+GP” utilises the relation network for similarity measurement based on “3D U-Net+GP”. In this setting, the

Table 5. Ablation studies. “GP” indicates the graph propagation, and “Rel” means the relation network. “3D U-Net ” refers to propagating labels only with the network prediction itself. “3D U-Net+GP” indicates label propagation with hand-crafted features. “3D U-Net+Rel+GP” indicates label propagation with our relation network. Evaluated on ScanNet-v2 val. set with single view testing.

Method	3D U-Net	3D U-Net+GP	3D U-Net+Rel+GP
Iter1	60.14	63.83	63.92
Iter2	62.39	64.74	66.97
Iter3	64.83	66.10	68.40
Iter4	65.81	67.78	70.01
Iter5	65.91	67.92	70.45



Fig. 8. Pseudo labels for each iteration on ScanNet-v2 training set.

similarity among super-voxels is measured with the averaged coordinates p_i , the colours c_i , the unary features u_i , and the relation network generated feature f_i , as shown in Equation (2). This experiment is to manifest that the relation network benefits the similarity measurement and pseudo label generation, compared with the hand-crafted feature, i.e., coordinates and colour. It outperforms the hand-crafted features especially in the later iterations since the network benefits from the richer pseudo labels. It finally achieves 2.5% improvement compared with “3D U-Net+GP” (70.45% vs. 67.92%). As shown in Figure 8, the generated pseudo labels for each iteration expand to unknown regions step by step and finally get close to the ground truth.

Analysis of relation network: Further, we study whether the learned embeddings of the relation network outperform the hand-crafted features for similarity measurement. We randomly sample 200 super-voxels for each category in ScanNet-v2 and conduct a t-SNE visualisation⁸³ on them. Figure 9 indicates that the relation network better groups the intra-class embeddings and distinguishes the inter-class embeddings compared with hand-crafted features.

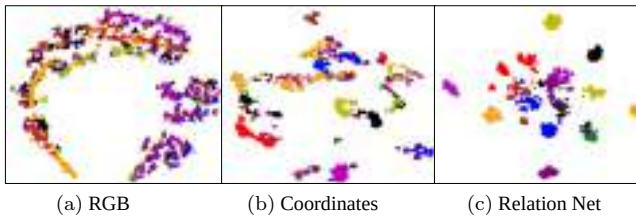


Fig. 9. The t-SNE visualisation of super-voxel features. Different colours and marks (point and plus) indicate different categories. The samples of the same category are better grouped together with our relation network (c), compared with hand-crafted features (a & b).

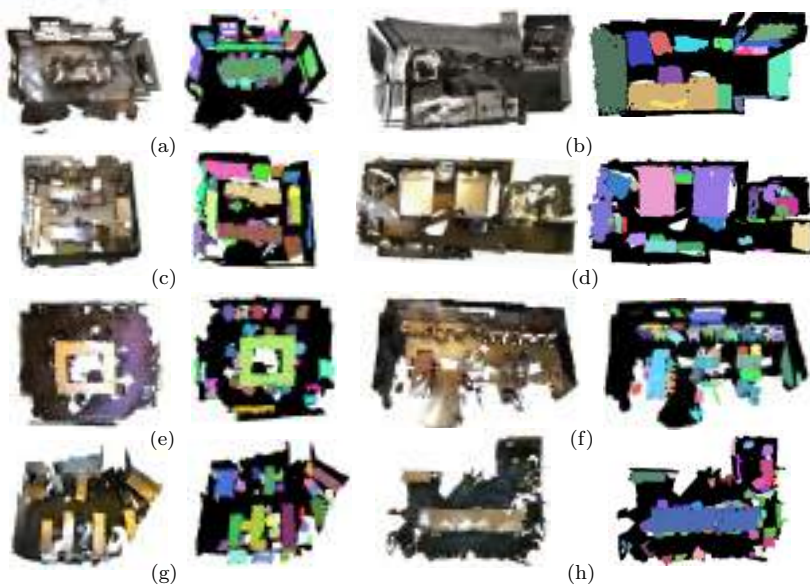


Fig. 10. Weakly supervised 3D instance segmentation results of our approach. (a)–(h) Quantitative results on various scenes: left is the input scene, while right is the instance segmentation result.

4.2. Instance Segmentation on ScanNet-v2

Figure 10 and Table 6 show the qualitative and quantitative evaluation results for instance segmentation. Figure 10 demonstrates the effectiveness of our approach in recognising nearby chairs as individual instances, as shown in (a), (e), (f), and (h). Also, our approach can accurately predict larger instances like tables and beds as a single

Table 6. Comparing with existing methods on ScanNet-v2 test set for 3D instance segmentation.

Method	Supervision	AP	AP50	AP25
3DSIS ⁴⁸	100%	16.1	38.2	55.8
3D-BoNet ⁴⁹	100%	25.3	48.8	68.7
RPGN ²²	100%	42.8	64.3	80.6
Mask3D ⁵⁰	100%	56.6	78.0	87.0
Our fully sup baseline (PointGroup ⁷⁰)	100%	40.7	63.6	77.8
TWIST ⁵²	10%	30.6	49.7	63.0
CSC-50 ³⁵	0.034%	22.9	41.4	62.0
SegGroup ⁵¹	0.028%	24.6	44.5	63.7
3D-WSIS ³⁸	0.02%	28.1	47.2	67.5
Ours	0.02%	32.6	52.9	67.5

entity; see (d), (e), and (h). Further, the results in Table 6 indicate that our approach outperforms all existing methods and even surpasses some fully supervised approaches, such as Hou *et al.*⁴⁸ and Yang *et al.*⁴⁹ for 3D instance segmentation.

4.3. Evaluations on S3DIS

We also evaluate our approach on the S3DIS dataset. Only less than 0.02% points in the dataset are annotated with our “One Thing One Click” scheme.

Comparing with existing works: We also compare with fully supervised approaches and weakly supervised approaches on S3DIS. As shown in Table 7, with the “One Thing One Click” scheme where less than 0.02% points are annotated, we achieve 56.6% mIoU, outperforming existing works by a considerable margin, including our previous version¹² (50.1%).

In addition, our approach achieves comparable results with several fully supervised methods, as shown in Table 7.

Qualitative results on S3DIS: Figure 11 illustrates our results on S3DIS. Again, with only 0.02% annotation, our approach can produce high-quality semantic predictions (c) that are comparable to the fully supervised approach (e). See the error maps (d, f) for better illustration.

Table 7. Comparison with existing methods and baselines on the S3DIS Area 5.

Method	Supervision (%)	mIoU(%)
PointNet ¹³	100%	41.1
SegCloud ²⁵	100%	48.9
TangentConv ⁷⁴	100%	52.8
3D RNN ⁸⁰	100%	53.4
PointCNN ¹⁵	100%	57.3
SuperpointGraph ²⁰	100%	58.0
MinkowskiNet32 ³¹	100%	65.4
Virtual MV-Fusion ¹⁹	100%+2D	65.4
Our fully sup baseline	100%	63.7
Superpoint-guided ⁴⁶	10% scenes	51.1
Π Model ⁸¹	0.2%	44.3
MT ⁶⁹	0.2%	44.4
Xu et al. ^{4*}	0.2%	44.0
Xu et al. ⁴	0.2%	44.5
Π Model ⁸¹	10%	46.3
MT ⁶⁹	10%	47.9
Xu et al. ^{4*}	10%	45.7
Xu et al. ⁴	10%	48.0
GPFN ³	16.7% 2D	50.8
GPFN ³	100% 2D	52.5
Zhang et al. ⁴⁴	0.03%	45.8
Joint 2D-3D ⁴³	Scene+Image	47.4
MulPro ⁸²	10%	49.0
MIL transformer ⁴⁵	0.02%	51.4
HybridCR ⁵	0.03%	51.5
GaIA ⁷	0.02%	53.7
DAT ⁴⁷	0.02%	54.6
One Thing One Click ¹²	0.02%	50.1
Ours	0.02%	56.6

5. Limitations

Despite of the good performance, there are still several limitations of our approach. For instance, our approach brings extra training burden compared with our fully supervised baseline.³⁰ In the model training, our approach requires additional training epochs and also optimises a relation network besides the 3D U-Net. Note that we

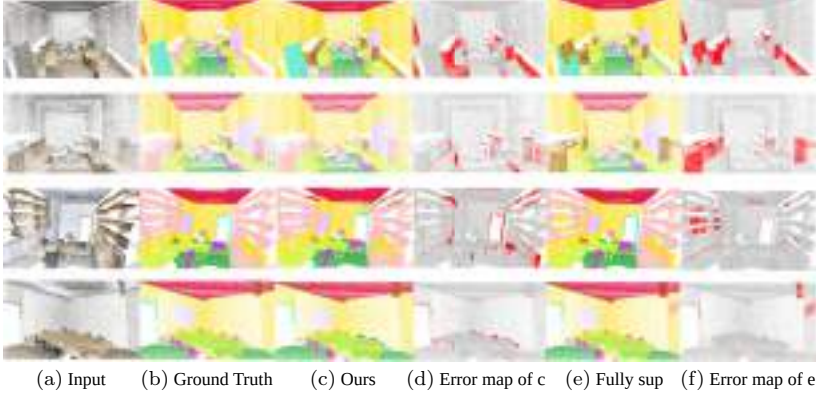


Fig. 11. Qualitative results on S3DIS. (c) Produced by our model trained only with “One Thing One Click” annotations. (e) Fully supervised results of Graham *et al.*³⁰ Red regions in (d) and (f) indicate the wrong predictions.

mainly consider the 3D U-Net and relation network since the computational burden of other modules can be neglected. Specifically, in the first training iteration, we train the network for 512 epochs following Refs. [30] and then fine-tune the network for 256 epochs for the remaining iterations; hence, we need three times of standard training time T of the baseline model, denoted as $T' = 3T$. In addition, optimising the relation network (taking the transformer-based option as an example) requires the same number of forward passes as 3D U-Net without backpropagation, and the training time is roughly $T'/2 = 3T/2$. Therefore, the overall training time of our approach is roughly $4.5T$. Besides, our approach doubles the number of parameters due to the relation network. Fortunately, in inference, the feedforward pass of 3D U-Net and relation network can be conducted in parallel to keep the inference time roughly equal to our fully supervised baseline.³⁰

6. Conclusion

We propose the “One Thing One Click” scheme to efficiently annotate point clouds for weakly supervised 3D semantic segmentation, requiring significantly fewer annotations than the previous approaches. To put this scheme into practice, we formulate a

self-training approach to make it feasible for the network to learn from such extremely sparse labels. Specifically, we execute the two key modules in our approach iteratively: expand labels through the label propagation module and train the network using the updated pseudo labels. Further, we adopt a relation network to explicitly learn the feature similarity. In addition, our approach is compatible to 3D instance segmentation using the One Thing One Click annotations. Experiments on two large 3D datasets ScanNet-v2 and S3DIS manifest that our approach, with only extremely sparse annotations, outperforms existing weakly supervised methods on 3D semantic segmentation and instance segmentation consistently. Moreover, our results are even comparable to those of the fully supervised counterparts.

References

1. A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of IEEE Computer Vision and Pattern Recognition (CVPR)* (2017).
2. J. Wei, G. Lin, K.-H. Yap, T.-Y. Hung, and L. Xie. Multi-path region mining for weakly supervised 3D semantic segmentation on point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4384–4393 (2020).
3. H. Wang, X. Rong, L. Yang, J. Feng, J. Xiao, and Y. Tian. Weakly supervised semantic segmentation in 3D graph-structured point clouds of wild scenes. *arXiv preprint arXiv:2004.12498* (2020).
4. X. Xu and G. H. Lee. Weakly supervised semantic point cloud segmentation: Towards 10x fewer labels. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 13706–13715 (2020).
5. M. Li, Y. Xie, Y. Shen, B. Ke, R. Qiao, B. Ren, S. Lin, and L. Ma. HybridCR: Weakly-supervised 3D point cloud semantic segmentation via hybrid contrastive regularization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 14930–14939 (2022).
6. Q. Hu, B. Yang, G. Fang, Y. Guo, A. Leonardis, N. Trigoni, and A. Markham. SQN: Weakly-supervised semantic segmentation of large-scale 3D point clouds. In *Computer Vision–ECCV 2022: 17th European Conference*, Tel Aviv, Israel, October 23–27, 2022, *Proceedings, Part XXVII*, pp. 600–619 (2022).

7. M. S. Lee, S. W. Yang, and S. W. Han. Gaia: Graphical information gain based attention network for weakly supervised point cloud semantic segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 582–591 (2023).
8. Y. Wu, Z. Yan, S. Cai, G. Li, Y. Yu, X. Han, and S. Cui, Pointmatch: A consistency training framework for weakly supervised semantic segmentation of 3D point clouds. *arXiv preprint* arXiv:2202.10705 (2022).
9. S. Zheng, S. Jayasumana, B. Romera-Paredes, V. Vineet, Z. Su, D. Du, C. Huang, and P. H. Torr. Conditional random fields as recurrent neural networks. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 1529–1537 (2015).
10. L.-C. Chen, G. Papandreou, I. Kokkinos, K. Murphy, and A. L. Yuille. DeepLab: Semantic image segmentation with deep convolutional nets, atrous convolution, and fully connected CRFs, *IEEE Transactions on Pattern Analysis and Machine Intelligence (T-PAMI)* **40**(4), 834–848 (2017).
11. H. Yuan and S. Ji. StructPool: Structured graph pooling via conditional random fields. In *International Conference on Learning Representations (ICLR)* (2019).
12. Z. Liu, X. Qi, and C.-W. Fu. One thing one click: A self-training approach for weakly supervised 3D semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1726–1736 (2021).
13. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 652–660 (2017).
14. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 5099–5108 (2017).
15. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on x-transformed points. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 820–830 (2018).
16. H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas. KpConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 6411–6420 (2019).
17. W. Wu, Z. Qi, and L. Fuxin. PointConv: Deep convolutional networks on 3D point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9621–9630 (2019).

18. A. Boulch, Convpoint: Continuous convolutions for point cloud processing, *Computers & Graphics* **88**, 24–34 (2020). ISSN 0097-8493. <https://doi.org/10.1016/j.cag.2020.02.005>. <https://www.sciencedirect.com/science/article/pii/S0097849320300224>.
19. A. Kundu, X. M. Yin, A. Fathi, D. A. Ross, B. Brewington, T. Funkhouser, and C. Pantofaru. Virtual multi-view fusion for 3D semantic segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)* (2020).
20. L. Landrieu and M. Simonovsky. Large-scale point cloud semantic segmentation with superpoint graphs. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4558–4567 (2018).
21. H. Zhao, L. Jiang, J. Jia, P. H. Torr, and V. Koltun. Point transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 16259–16268 (2021).
22. S. Dong, G. Lin, and T.-Y. Hung. Learning regional purity for instance segmentation on 3D point clouds. In *Computer Vision—ECCV 2022: 17th European Conference, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXX*, pp. 56–72 (2022).
23. X. Wu, Y. Lao, L. Jiang, X. Liu, and H. Zhao. Point transformer v2: Grouped vector attention and partition-based pooling. *arXiv preprint arXiv:2210.05666* (2022).
24. X. Lai, J. Liu, L. Jiang, L. Wang, H. Zhao, S. Liu, X. Qi, and J. Jia. Stratified transformer for 3D point cloud segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8500–8509 (2022).
25. L. Tchapmi, C. Choy, I. Armeni, J. Gwak, and S. Savarese. SEGCloud: Semantic segmentation of 3D point clouds. In *2017 International Conference on 3D Vision (3DV)*, pp. 537–547 (2017).
26. G. Riegler, A. Osman Ulusoy, and A. Geiger. OctNet: Learning deep 3D representations at high resolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3577–3586 (2017).
27. B. Graham, Sparse 3D convolutional neural networks. *arXiv preprint arXiv:1505.02890* (2015).
28. H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz. SplatNet: Sparse lattice networks for point cloud processing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2530–2539 (2018).
29. A. Dai and M. Nießner. 3DMV: Joint 3D-multi-view prediction for 3D semantic scene segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 452–468 (2018).

30. B. Graham, M. Engelcke, and L. van der Maaten. 3D semantic segmentation with submanifold sparse convolutional networks, *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
31. C. Choy, J. Gwak, and S. Savarese. 4D spatio-temporal ConvNets: Minkowski convolutional neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3075–3084 (2019).
32. L. Han, T. Zheng, L. Xu, and L. Fang. OccuSeg: Occupancy-aware 3D instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 2940–2949 (2020).
33. S. Guinard and L. Landrieu. Weakly supervised segmentation-aided classification of urban scenes from 3D lidar point clouds. In *ISPRS Workshop 2017* (2017).
34. J. Mei, B. Gao, D. Xu, W. Yao, X. Zhao, and H. Zhao. Semantic segmentation of 3D LiDAR data in dynamic scene using semi-supervised learning, *IEEE Transactions on Intelligent Transportation Systems* **21**(6), 2496–2509 (2019).
35. J. Hou, B. Graham, M. Nießner, and S. Xie. Exploring data-efficient 3D scene understanding with contrastive scene contexts. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 15587–15597 (2021).
36. B. Tian, L. Luo, H. Zhao, and G. Zhou, VIBUS: Data-efficient 3D scene parsing with viewpoint bottleneck and uncertainty-spectrum modeling, *ISPRS Journal of Photogrammetry and Remote Sensing* **194**, 302–318 (2022).
37. G. Liu, O. van Kaick, H. Huang, and R. Hu. Active self-training for weakly supervised 3D scene semantic segmentation. *arXiv preprint arXiv:2209.07069* (2022).
38. L. Tang, L. Hui, and J. Xie. Learning inter-superpoint affinity for weakly supervised 3D instance segmentation. In *Proceedings of the Asian Conference on Computer Vision*, pp. 1282–1297 (2022).
39. S. Dong, R. Li, J. Wei, F. Liu, and G. Lin. RWSeg: Cross-graph competing random walks for weakly supervised 3D instance segmentation. *arXiv preprint arXiv:2208.05110* (2022).
40. P.-C. Yu, C. Sun, and M. Sun. Data efficient 3D learner via knowledge transferred from 2d model. In *Computer Vision–ECCV 2022: 17th European Conference*, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXIX, pp. 182–198 (2022).
41. Y. Zhang, Y. Qu, Y. Xie, Z. Li, S. Zheng, and C. Li. Perturbed self-distillation: Weakly supervised large-scale point cloud semantic

- segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 15520–15528 (2021).
42. Z. Ren, I. Misra, A. G. Schwing, and R. Girdhar. 3D spatial recognition without spatially labeled 3D. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13204–13213 (2021).
 43. H. Kweon and K.-J. Yoon. Joint learning of 2D-3D weakly supervised semantic segmentation. In *Advances in Neural Information Processing Systems* (2022).
 44. Y. Zhang, Z. Li, Y. Xie, Y. Qu, C. Li, and T. Mei. Weakly supervised semantic segmentation for large-scale point cloud. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 35, pp. 3421–3429 (2021).
 45. C.-K. Yang, J.-J. Wu, K.-S. Chen, Y.-Y. Chuang, and Y.-Y. Lin. An mil-derived transformer for weakly supervised point cloud segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11830–11839 (2022).
 46. S. Deng, Q. Dong, B. Liu, and Z. Hu. Superpoint-guided semi-supervised semantic segmentation of 3D point clouds. In *2022 International Conference on Robotics and Automation (ICRA)*, pp. 9214–9220 (2022).
 47. Z. Wu, Y. Wu, G. Lin, J. Cai, and C. Qian. Dual adaptive transformations for weakly supervised point cloud segmentation. In *Computer Vision—ECCV 2022: 17th European Conference*, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part XXXI, pp. 78–96 (2022).
 48. J. Hou, A. Dai, and M. Nießner. 3D-SIS: 3D semantic instance segmentation of RGB-D scans. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4421–4430 (2019).
 49. B. Yang, J. Wang, R. Clark, Q. Hu, S. Wang, A. Markham, and N. Trigoni. Learning object bounding boxes for 3D instance segmentation on point clouds. In *Advances in Neural Information Processing Systems*, Vol. 32 (2019).
 50. J. Schult, F. Engelmann, A. Hermans, O. Litany, S. Tang, and B. Leibe. Mask3d: Mask transformer for 3D semantic instance segmentation. In *2023 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 8216–8223 (2023). doi:10.1109/ICRA48891.2023.10160590.
 51. A. Tao, Y. Duan, Y. Wei, J. Lu, and J. Zhou. SegGroup: Seg-level supervision for 3D instance and semantic segmentation, *IEEE Transactions on Image Processing* **31**, 4952–4965 (2022).

52. R. Chu, X. Ye, Z. Liu, X. Tan, X. Qi, C.-W. Fu, and J. Jia. TWIST: Two-way inter-label self-training for semi-supervised 3D instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1100–1109 (2022).
53. X. Qi, Z. Liu, J. Shi, H. Zhao, and J. Jia. Augmented feedback in semantic segmentation under image level supervision. In *European Conference on Computer Vision (ECCV)*, pp. 90–105 (2016).
54. S. J. Oh, R. Benenson, A. Khoreva, Z. Akata, M. Fritz, and B. Schiele. Exploiting saliency for object segmentation from image level labels. In *2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 5038–5047 (2017).
55. Y. Zhou, Y. Zhu, Q. Ye, Q. Qiu, and J. Jiao. Weakly supervised instance segmentation using class peak response. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3791–3800 (2018).
56. J. Ahn and S. Kwak. Learning pixel-level semantic affinity with image-level supervision for weakly supervised semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4981–4990 (2018).
57. A. Bearman, O. Russakovsky, V. Ferrari, and L. Fei-Fei. What’s the point: Semantic segmentation with point supervision. In *European Conference on Computer Vision (ECCV)*, pp. 549–565 (2016).
58. I. H. Laradji, N. Rostamzadeh, P. O. Pinheiro, D. Vazquez, and M. Schmidt. Where are the blobs: Counting by localization with point supervision. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 547–562 (2018).
59. K.-K. Maninis, S. Caelles, J. Pont-Tuset, and L. Van Gool. Deep extreme cut: From extreme points to object segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 616–625 (2018).
60. D. P. Papadopoulos, J. R. Uijlings, F. Keller, and V. Ferrari. Extreme clicking for efficient object annotation. In *Proceedings of the IEEE International Conference on Computer Vision (ICCV)*, pp. 4930–4939 (2017).
61. D. Lin, J. Dai, J. Jia, K. He, and J. Sun. ScribbleSup: Scribble-supervised convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3159–3167 (2016).
62. B. Wang, G. Qi, S. Tang, T. Zhang, Y. Wei, L. Li, and Y. Zhang. Boundary perception guidance: A scribble-supervised semantic segmentation approach. In *International Joint Conference on Artificial Intelligence (IJCAI)*, pp. 3663–3669 (2019).

63. J. Zhang, X. Yu, A. Li, P. Song, B. Liu, and Y. Dai. Weakly-supervised salient object detection via scribble annotations. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12546–12555 (2020).
64. J. Dai, K. He, and J. Sun. BoxSup: Exploiting bounding boxes to supervise convolutional networks for semantic segmentation. In *Proceedings of the IEEE International Conference on Computer Vision (CVPR)*, pp. 1635–1643 (2015).
65. J. Snell, K. Swersky, and R. Zemel. Prototypical networks for few-shot learning. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 4077–4087 (2017).
66. A. V. D. Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
67. Z. Wu, Y. Xiong, S. X. Yu, and D. Lin. Unsupervised feature learning via non-parametric instance discrimination. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3733–3742 (2018).
68. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick. Momentum contrast for unsupervised visual representation learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 9729–9738 (2020).
69. A. Tarvainen and H. Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1195–1204 (2017).
70. L. Jiang, H. Zhao, S. Shi, S. Liu, C.-W. Fu, and J. Jia. Pointgroup: Dual-set point grouping for 3D instance segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4867–4876 (2020).
71. I. Armeni, S. Sax, A. R. Zamir, and S. Savarese. Joint 2D-3D-semantic data for indoor scene understanding. *arXiv preprint arXiv:1702.01105* (2017).
72. D. Koller and N. Friedman. *Probabilistic Graphical Models: Principles and Techniques*. MIT Press: Cambridge, Massachusetts (2009).
73. A. Paszke, S. Gross, F. Massa, A. Lerer, J. Bradbury, G. Chanan, T. Killeen, Z. Lin, N. Gimelshein, L. Antiga, A. Desmaison, A. Kopf, E. Yang, Z. DeVito, M. Raison, A. Tejani, S. Chilamkurthy, B. Steiner, L. Fang, J. Bai, and S. Chintala. PyTorch: An imperative style, high-performance deep learning library. In H. Wallach, H. Larochelle,

- A. Beygelzimer, F. d' Alché-Buc, E. Fox, and R. Garnett (eds.) *Advances in Neural Information Processing Systems (NeurIPS)*, Vol. 32, pp. 8024–8035 (2019).
74. M. Tatarchenko, J. Park, V. Koltun, and Q.-Y. Zhou. Tangent convolutions for dense prediction in 3D. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3887–3896 (2018).
75. Y. Lin, Z. Yan, H. Huang, D. Du, L. Liu, S. Cui, and X. Han. FPCnv: Learning local flattening for point convolution. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 4293–4302 (2020).
76. J. Schult, F. Engelmann, T. Kontogianni, and B. Leibe. DualConvMesh-Net: Joint geodesic and euclidean convolutions on 3D meshes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 8612–8622 (2020).
77. Z. Hu, M. Zhen, X. Bai, H. Fu, and C.-l. Tai. JSENet: Joint semantic segmentation and edge detection network for 3D point clouds. *arXiv preprint arXiv:2007.06888* (2020).
78. A. Nekrasov, J. Schult, O. Litany, B. Leibe, and F. Engelmann. Mix3d: Out-of-context data augmentation for 3D scenes. In *2021 International Conference on 3D Vision (3DV)*, pp. 116–125 (2021).
79. S. Xie, J. Gu, D. Guo, C. R. Qi, L. Guibas, and O. Litany. Pointcontrast: Unsupervised pre-training for 3D point cloud understanding. In *European Conference on Computer Vision*, pp. 574–591 (2020).
80. X. Ye, J. Li, H. Huang, L. Du, and X. Zhang. 3D recurrent neural networks with context fusion for point cloud semantic segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 403–417 (2018).
81. S. Laine and T. Aila. Temporal ensembling for semi-supervised learning. *arXiv preprint arXiv:1610.02242* (2016).
82. Y. Su, X. Xu, and K. Jia. Weakly supervised 3D point cloud segmentation via multi-prototype learning. *arXiv preprint arXiv:2205.03137* (2022).
83. L. V. D. Maaten and G. Hinton. Visualizing data using t-SNE, *Journal of Machine Learning Research* **9**(November), 2579–2605 (2008).

This page intentionally left blank

Chapter 4

Representation Learning for Dynamic 3D Scenes

Yunzhu Li^{*,†} and Jiajun Wu^{†,§}

^{}University of Illinois Urbana-Champaign,
Champaign, USA*

[†]Stanford University, Stanford, USA

[‡]liyunzhu@stanford.edu

[§]jiajunwu@cs.stanford.edu

This chapter conducts an in-depth exploration of the various methodologies employed in modelling and learning representations for dynamic 3D scenes. The discussion draws upon the intrinsic human ability to interact with and predict changes within the 3D physical world, with the aim of advancing embodied AI systems to reach and exceed these capabilities.

The focus lies on the examination of diverse 3D representation methods, encompassing keypoints, particles, and neural fields among others. Their respective strengths, limitations, and applications within embodied AI tasks are thoroughly investigated. Highlights include the self-supervised learning of 3D object keypoints for model-based predictions, the use of particles in conjunction with graph neural networks (GNNs) to model complex dynamics, and the recent surge in the utilisation of 3D neural fields in world models, using inductive bias.

This chapter demonstrates how these representation learning methods have been instrumental in accomplishing complex control and planning tasks in the 3D world, thus significantly propelling AI capabilities in both simulated and real-world conditions. Concluding with a look

towards the future, this chapter outlines potential research directions to continue enhancing the performance and capabilities of embodied intelligence systems.

1. Introduction

Humans possess a strong intuitive understanding of the 3D physical world that surrounds us. We observe and engage with our environment, and during this process, our brains process sensory data, construct a mental model of the world, and predict how the environment would change if we executed a particular action, as illustrated in Fig. 1. This predictive ability does not depend on analytical physics equations, yet it applies to objects of various materials (e.g., rigid bodies, deformable objects, and fluids) in the 3D world. This enables us to exhibit a vast array of interactive skills that are far superior to those of current embodied AI systems.³⁶

Thus, it is essential to develop dynamic 3D representations for embodied intelligence systems to effectively model the scene dynamics. In these systems, agents need to learn dynamic 3D scene representations through physical interactions, with the aim of generalising across a broad spectrum of tasks. Recent advances in 3D deep



Fig. 1. Humans build a mental model of the 3D world through dynamic interactions with the environment. We can anticipate how a scene will change if we apply a specific action. We then use the learned predictive model to plan our actions.

learning have merged the adaptability of deep neural networks with structured inductive biases, yielding promising results for learning such representations. In this chapter, we aim to review methods that model dynamic 3D scenes using diverse 3D representations (such as keypoints, particles, neural fields, and more). These representations model the environment at different levels of abstraction. We discuss their respective advantages and limitations, explore how researchers have employed the learned dynamics models for various embodied AI tasks, and suggest potential future research directions.

For example, Manuelli *et al.*⁵³ proposed using **3D object keypoints**, which are learned in a self-supervised manner and tracked over time. These keypoints model the underlying dynamic 3D scenes through model-based predictions, and with concrete experimental evidence, they demonstrated that keypoints offer a range of appealing properties: (i) the output is interpretable and in 3D space, allowing analysis of the visual model separately from the predictive model, (ii) they can be applied to deformable objects, and (iii) they achieve category-level generalisation.

Keypoints are useful as a low-dimensional structural representation but cannot model objects with higher degrees of freedom, such as fluids. Li *et al.*,^{42,44} Shi *et al.*,⁷⁶ and Lin *et al.*⁴⁷ proposed to model the environment at a finer abstraction level by using **particles** as the representation. To model the dynamics, these researchers learn a graph-structured simulator using graph neural networks (GNNs) in 3D space. Huang *et al.*³³ took a step further by considering mesh-based objects. Combining GNNs with particle- or mesh-based systems works broadly across objects of different materials and injects a strong inductive bias for learning: particles or vertices of the same type are governed by the same dynamics. This structural prior embedded in the model enables more effective training and improved extrapolation generalisation performance in scenarios outside the training distribution. They demonstrated that the learned model could predict the dynamics of various objects, including rigid bodies and objects with more complex physical properties like plasticine, fluids, and clothes.

Recent developments in **3D neural fields** have also inspired a range of works in learning world models that incorporate inductive bias in the form of a learning-based differentiable renderer. For instance, Li *et al.*⁴¹ combined the differentiable volumetric renderer with an autoencoding framework and a latent dynamics model.

The resulting system significantly expands the model’s capability, allowing for (i) future prediction, (ii) out-of-distribution viewpoint generalisation, and (iii) novel view synthesis in dynamic scenes involving complex interactions between fluids and rigid objects. Driess *et al.*¹⁰ leveraged compositional neural implicit representations in combination with GNNs for dynamic modelling of multi-object interactions, while Tian *et al.*⁸⁵ proposed an extension using object-centric neural implicit scattering functions (OSFs) for inverse parameter estimation and generalisation to previously unseen and extreme lighting conditions.

Coupled with model-based planning or policy learning techniques, these methods have achieved complex planning and control tasks in the 3D world, such as manipulating multi-object interaction systems,^{10,53,85} shaping a deformable foam into target shapes,^{44,76} smoothing cloth,^{33,47} and handling a cup of water with floating ice cubes.⁴¹ Rooted in the advances of 3D deep learning, these works take embodied agents from manipulating rigid objects to handling a variety of dynamic and flexible objects, unlocking a fundamentally new set of embodied capabilities in both simulation and the real world.

Chapter structure: As we have discussed, 3D scene representations in various forms depict the environment at different levels of abstraction, implying distinct modelling and generalisation capabilities. Therefore, it is essential to understand their strengths and weaknesses to choose the most appropriate one for the task at hand. In this chapter, we explore three primary 3D representations: keypoints, particles, and neural fields, along with their corresponding model classes for dynamic modelling of the underlying 3D world. We demonstrate how a suitable choice of representation and structure can lead to improved generalisation and sample efficiency. Furthermore, we discuss how these models can be employed for downstream planning and control tasks.

In Section 2, we begin by discussing the problem formulation, which entails learning dynamic models of the environment. We focus on how to incorporate external actions as input, predict future system evolution, and formulate planning and control problems using the learned dynamic models. Section 3 then overviews the related work on representation learning for dynamic scenes.

Then, in Section 4, we discuss keypoint representation in modelling rigid body dynamics. Using a pusher-slider system as an

example, we demonstrate how employing 3D keypoints as representations can facilitate learning efficient models capable of performing closed-loop real-time model-predictive control, even in the presence of external disturbances. This section was previously published as Manuelli *et al.*⁵³ Next, Section 5 presents works on using particles as representations to learn dynamics at a finer level of granularity for objects of different materials (e.g., rigid bodies, deformable objects, and fluids). We also show how the learned models can facilitate model-based planning to manipulate these objects in both simulation and real-world settings. This section was previously published as Li *et al.*⁴⁴ Progressing further, for objects with extremely high degrees of freedom like fluids, estimating the particle set or mesh from raw visual observations is challenging, and keypoints may not capture the object’s variability. In Section 6, we discuss learning latent-space dynamics models augmented with differentiable volume rendering to model fluid dynamics purely from visual observations. This section primarily draws from Li *et al.*⁴¹

Finally, in Section 7, we discuss the limitations of existing methods and present a few pivotal future directions for representation learning in dynamic 3D scenes. Progression in this area will be of significant importance and substantially enhance the capabilities of the next generation of embodied AI systems.

2. Problem Statement

To provide a more concrete illustration of the problem and effectively showcase the advancements of the methods discussed in this chapter, we outline the formulation of the dynamics model representing the underlying 3D world. We also explain how to learn the model from data and how to formulate a planning problem using the dynamics model.

2.1. Dynamics Model Learning

The objective is to learn a dynamic model of the 3D environment that predicts how the environment will change when an agent applies a specific action. Suppose we have a dataset $\mathcal{D} = (y_t, u_t)$ collected through interactions with the environment. In this case, y_t represents the raw sensory information obtained at time t , essentially serving as

a (partial) observation of the environment. This sensory data, consisting of high-dimensional visual observations (e.g., RGB images and depth), is expected to include the necessary information for accomplishing downstream interaction tasks. u_t denotes the corresponding action applied at time t .

Next, a perception module $z_t = g(y_{t':t}, u_{t':t-1})$ maps a small sequence of observations $y_{t':t}$ from time t' to the current time t and the corresponding action $u_{t':t-1}$ into a latent representation of the environment z_t at time t . It is crucial to note again that the choice of z_t significantly impacts the overall system's structure. We must consider which specific forms z_t should take and at what levels of abstraction the objects and environment should be represented. The representation should contain sufficient information about the underlying environment to accomplish downstream planning and control tasks. In this chapter, we discuss various options, such as keypoints, particles, and neural fields.

Once we obtain the scene representation z_t , we learn the dynamics model f_θ , parameterised by θ , as a proxy for the real dynamics. This model takes the representation z_t and action u_t as inputs to predict the new representation as a result of the action at time $t + 1$:

$$z_{t+1} = f_\theta(z_t, u_t). \quad (1)$$

To optimise the model for accurate future predictions, we aim to find the parameter θ that minimises the following simulation error, which describes the long-term discrepancy between the prediction and the actual representation z^* :

$$\mathcal{L}(\theta) = \sum_t \|z_{t+1}^* - f_\theta(z_t, u_t)\|_2^2. \quad (2)$$

To achieve better generalisation, we should carefully consider how to characterise the structures of the underlying physical world and which model class we should use (e.g., linear models, convolutional neural networks, or graph neural networks), particularly when modelling dynamics for objects made of a diverse set of materials. In addition, to adapt the models to scenarios where the underlying structure and parameters are not directly observable, we also need to formulate an inference problem and perform continual structural and parameter estimation from data in an online fashion.

2.2. Model-Based Planning

To demonstrate the applicability of the learned dynamic models, we show how these models can be effectively utilised for model-based optimisation in downstream planning and control problems. Specifically, using the same notation, we aim to minimise an objective function $\sum_t c(y_t, u_t)$, which is based on the observation y_t and the action u_t over a specific time horizon executed in the underlying 3D environment:

$$\min_{\pi} \mathbb{E}_{\pi} \left[\sum_t c(y_t, u_t) \right], \quad (3)$$

$$z_t = g(y_{t':t}, u_{t':t-1}), \quad (4)$$

$$z_{t+1} = f_{\theta}(z_t, u_t), \quad (5)$$

$$u_t = \pi(z_t). \quad (6)$$

The problem definition includes three critical constraints: the perception module that maps the observations to a scene representation z_t , the dynamic model that predicts the underlying system’s evolution, and a control module that derives the control signal based on the current scene representation.

To solve the optimisation problem, we need to leverage the structure introduced by the representation and the learned dynamic model while considering the trade-off between effectiveness and efficiency. Additionally, we must explore how to close the perception-control loop and account for all practical constraints when deploying the system on real robots. Central to this problem is, again, the choice of representation $z_t = g(y_{t':t}, u_{t':t-1})$ and the dynamic model $z_{t+1} = f_{\theta}(z_t, u_t)$ that model the underlying 3D world’s changes. Progress in solving these problems requires careful consideration of the overall system design, detailed analysis of the role of different components, and a thorough understanding of the advantages and limitations of various modelling choices.

3. Related Work

3.1. 2D Representations for Dynamic Scenes

A significant body of literature on model learning from pixels for embodied interaction tasks can be broadly categorised into those

predicting the image-space dynamics of the entire image^{12,13,17,81,98} or those predicting the dynamics of a low-dimensional latent state from 2D observations.^{1,26,93,97} Although image-space dynamics approaches are general, they require large training datasets and can often result in blurry predictions for long-horizon futures. For latent-space dynamics approaches, a regularisation strategy is required for the latent state z_t to avoid a trivial solution where all observations map to a constant vector. Watter *et al.*⁹³ and Hafner *et al.*²⁶ employ an autoencoder architecture and regularise the latent state z_t using a reconstruction loss. Agrawal *et al.*¹ regularise the latent space by training both forward and inverse dynamics models simultaneously, while Yan *et al.*⁹⁷ employ a contrastive loss on the latent state.

Kulkarni *et al.*,³⁵ alternatively, considered a more structured latent space in the form of 2D keypoints. They demonstrated that freezing the visual model and using a keypoint-type representation as an input to model-free reinforcement learning (RL) algorithms enhances performance on Atari games.³ In contrast to these approaches, the methods we introduce in this chapter consider explicit 3D structures at different levels of abstraction. This allows for physically grounded and interpretable modelling of the underlying dynamics, which, we demonstrate, is more effective at modelling long-horizon scene evolution and accomplishing various downstream embodied interaction tasks.

3.2. 3D Representations for Dynamic Scenes

To endow models with a 3D structure, voxel grids can be utilised as neural scene representations,^{58,69,78,89,100} while others have attempted to predict particle sets or meshes from images^{42,92} or to embed an explicit 3D representation for inference from previously unseen viewpoints.⁷⁰ Prior work has also employed novel view synthesis from a single image for supervision, using autoencoder-like models to learn latent spaces as representations of the underlying 3D scene.^{82,95} Furthermore, Sitzmann *et al.*⁷⁹ proposed learning neural implicit representations of 3D shape and appearance, supervised with posed 2D images via a differentiable renderer.

To leverage the 3D representations for modelling the dynamic changes in the scene, researchers have explored object-level deformation through the lens of 3D keypoints³⁴ or meshes.²² Scene-level

deformation can be modelled by transporting input coordinates to neural implicit representations with an implicitly represented flow field or time-variant latent code.^{11,39,46,59,63,65,67,88,96} However, these works typically don't consider action inputs, often fitting only one trajectory, and failing to handle different initial conditions and external action inputs. This limitation restricts their utility in downstream planning and control tasks.

In the works presented in this chapter, we explore methods to incorporate external actions into learned dynamics models. This approach enables the handling of various initial conditions and input actions, making them more suitable for synthesising control signals for embodied AI agents.

3.3. *Physics-Based Dynamics Modelling*

For many decades, physics-based simulation methods have been steadily developed for the modelling of dynamic scenes. Models based on first principles or physics, such as those illustrated by Hogan and Rodriguez³⁰ and Zhou *et al.*,⁹⁹ depend on known object models and simulate their interactions. To use the models for downstream tasks, researchers have established techniques to differentiate through physics-based simulators, aiming to provide analytical gradients for inverse estimation and optimisation.^{9,14,84,86} Several studies have harnessed the analytical gradients from these simulators⁸ to solve control problems, such as tool manipulation and tool-use planning.⁸⁷ Despite these advancements, simulators capable of providing analytical gradients for deformable objects have been less studied. Recently, Schenck and Fox⁷¹ proposed SPNets to extract gradients from position-based fluid simulations,⁵⁰ and Hu *et al.*³¹ developed a new differentiable programming language specifically for building high-performance differentiable physical simulators for various deformable and dynamic scenes.

However, we are yet to develop a comprehensive, physics-based simulator capable of simulating all objects and interactions in our daily life. These models also face challenges in generalising to novel or unknown objects and require external state estimation systems. Furthermore, full-state estimation can prove challenging or even impossible in many complex real-world scenarios. These limitations hinder the utility of physics-based models. As a result, it is important

to advance learning methods that can learn models from visual observations, using data collected from agents’ interactions with their environment.

3.4. *Dynamics Learning for Embodied AI Applications*

People have been employing learned dynamics models for a variety of embodied AI tasks involving physical interactions, typically via a model-based planning framework. For instance, Hogan *et al.*²⁹ devised a dynamics model for a closed-loop-controlled planar pushing task. Nagabandi *et al.*⁵⁷ developed deep dynamics models for multiple simulated dexterous manipulation tasks, using ground-truth object states, as well as one task on actual hardware that utilised an external camera-based 3D object position tracker. Many recent studies have also investigated the use of latent-space dynamics models for policy learning and model-predictive control.^{16,24,38,56,61,77,80} While these works have demonstrated impressive results, their lack of accounting for explicit 3D structures and/or their dependence on ground-truth object states restricts their wider applicability in a variety of real-world embodied AI tasks.

Battaglia *et al.*² and Chang *et al.*⁶ have explored expanding model learning in compositional environments, and some studies have looked into the use of such models for planning and control, often learning a policy based on the models’ rollouts.^{27,66,68}

The approaches we introduce in this chapter leverage advancements in the application of learned models for planning and control across a variety of embodied AI tasks within the 3D world. Our emphasis lies on tasks that deal with complex physical properties. Specifically, we showcase examples that include the manipulation of rigid bodies, deformable objects, cloth, and fluids.

4. Keypoint Representation

In this section, we discuss dynamic 3D modelling and model-based planning using self-supervised keypoints. Specifically, we aim to develop models capable of predicting object 3D motion upon interaction and employing these models for planning, with

pusher-slider systems involving external disturbances as an example. We present results from highly efficient predictive models learned from dense visual correspondences that show significant performance improvements over alternative methods that do not incorporate explicit 3D structures (e.g., methods involving vision-based RL with autoencoder-type vision training). Through simulation experiments, we show that the learned models offer superior generalisation precision, especially in 3D scenes, scenes involving occlusion, and in category-generalisation scenarios. Moreover, we confirm the effective transfer of this method to real-world scenarios through hardware experiments. Video illustrations can be found on the project website: <https://sites.google.com/view/keypointsintothefuture>.

This section includes material previously published as Manuelli *et al.*⁵³ in the Conference on Robot Learning (CoRL) 2020, with co-authors Lucas Manuelli, Yunzhu Li, Pete Florence, and Russ Tedrake. Lucas Manuelli has made significant contributions to the content of this section.

4.1. Overview

In this section, we explore the use of object keypoints, tracked over time, as the latent state for learning dynamics. These keypoints anchor the model-based predictions and offer several advantages over alternative methods. First, they generate interpretable outputs, facilitating separate analysis of the visual and predictive model performance. Second, this representation is 3D and can naturally handle changing off-axis 3D camera positions. Lastly, as demonstrated in Florence *et al.*,^{19,20} the visual models we use, *Dense Object Nets*, perform reliably in various real-world scenarios and can generalise at the category level. In contrast, representations learned from autoencoder approaches or 6D object poses often struggle with deformable objects and category-level generalisation.

We show that this formulation enables reliable, sample-efficient learning that can perform precise visual-feedback-based manipulation in the real world. This learning is achieved using only a small amount of interaction data (10 minutes) and a single demonstration for goal specification. In the presented approach, keypoints are extracted as descriptors, tracked from a dense descriptor model. Although several methods could be used to acquire keypoints, the

method presented here can be entirely self-supervised. Unlike Florence *et al.*,¹⁹ which uses keypoints from a dense correspondence model in an imitation learning framework, employing keypoints as input to a model-based RL system presents unique challenges. Specifically, the keypoints must be both informative for the task and accurately tracked. We discuss these challenges in this section and propose solutions.

The primary contribution of the presented method is (i) a novel formulation of predictive model-learning, using learned self-supervised 3D keypoints as the state representation. The learned model can then be used to perform closed-loop visual feedback control via model-predictive control (MPC). (ii) Through simulated manipulation experiments, we show that the presented method outperforms a variety of baselines, and (iii) we validate the approach in real-world robot experiments.

4.2. Self-Supervised 3D Keypoint Dynamics

This section describes the approach to model-based RL using self-supervised 3D keypoint dynamics learned from visual observations. The goal is to learn a dynamics model that can then be used to perform online planning for closed-loop control.

4.2.1. Learning a visual representation

In this section, we consider a simplified perception model $z_t = g(y_t)$ that only takes the current observation as input, the objective of which is to generate a low-dimensional feature vector z_t at time t , which serves as a suitable latent state for learning the dynamics. For the types of tasks and environments considered in this section, spatial information about object locations is crucial. Pose estimation has been instrumental in classical manipulation pipelines, and it has also been used in dynamics learning approaches such as those by Hogan *et al.*²⁹ and Nagabandi *et al.*⁵⁷ In general, deriving pose information from high-dimensional observations (such as RGBD images) requires a dedicated perception system. While pose can be a powerful state representation for dealing with a single known object, it has several limitations in more general manipulation scenarios, as highlighted in Florence *et al.*,^{19,20} and Manuelli *et al.*⁵² Specifically,

it does not readily (i) extend to deformable objects, (ii) generalise to novel objects, or (iii) apply to category-level tasks.

The strategy presented in this section is to utilise visual-correspondence pre-training to track points on the object of interest. The locations of these tracked points can then serve as the latent state where we learn the dynamics. Following the approach in Florence *et al.*,^{19,20} the method employs visual-correspondence learning, trained in a completely self-supervised manner, to build a visual model that can be used to find correspondences across RGB images. We then discuss two approaches to produce a low-dimensional latent z_t in the form of 3D keypoints using the pre-trained dense-correspondence model.

First, we provide a brief background on dense correspondence models.^{18,20} Given an image observation $y_t \in \mathbb{R}^{W \times H \times C}$ (where C denotes the number of channels), the dense-correspondence model g^{dc} produces a full-resolution descriptor image $\mathcal{I}_D \in \mathbb{R}^{W \times H \times D}$. Since the aim is to learn a dynamics model on a low-dimensional state, we need a way to construct z_t from the descriptor image $\mathcal{I}_D$. The idea is for z_t to be a set of points on the object(s) that are localised in either image space or 3D space. These points are represented as a set $\{d^k\}_{k=1}^K$ of K descriptors, where each $d^k \in \mathbb{R}^D$ is a vector in the underlying descriptor space. A parameterless *correspondence function* $g^{\text{corr}}(\mathcal{I}_D, d^k)^a$ determines the location of the keypoint $z_t^k \in \mathbb{R}^B$ from the current observation. By combining our learned correspondences with the reference descriptors, we have a function that maps image observations y_t to keypoint locations $z_t^{\text{object}} = \{z_t^k\}_{k=1}^K$ on the object(s) of interest. We present two methods for constructing the latent state $z_t = g(y_t)$ from y_t .

Descriptor set (DS). In our most basic variant, the latent state z_t consists of keypoint locations z_t^k for a set of descriptor keypoints randomly sampled from the object. Specifically, we sample K descriptors $\{d^k\}_{k=1}^K$ (we use $K = 50$ in all experiments), which correspond to pixels from a masked reference descriptor image in our training set. Thus

$$z_t = z_t^{\text{object}} = \{z_t^k\}_{k=1}^K. \quad (7)$$

^aSee Appendix 4.5 for more details on the visual-correspondence model.

Spatial descriptor set (SDS). Unlike the **(DS)** method, which randomly samples descriptors, this method aims to select descriptors $\{d^k\}_{k=1}^K$ that possess specific properties. We particularly want the descriptors $\{d^k\}_{k=1}^K$ to be (i) *reliable* and (ii) *spatially separated*. By *reliable*, we mean that they can be accurately localised and don't become occluded under normal operating conditions, as illustrated in Figure 3. Being *spatially separated* implies that the chosen descriptors aren't all clustered around the same physical location on the object(s) of interest. Instead, they are sufficiently spread out (either in 3D space or pixel space) to provide meaningful information about both the object's position and orientation. The dense descriptor model used by the method can provide a confidence score linked with descriptors and their related correspondences. Figure 3 shows a clear example of high confidence for a valid match and low confidence when no valid correspondence exists due to an occlusion. We use this feature of the visual model to compute a confidence score c^k for each descriptor d^k according to what fraction of images in the training set $\mathcal{D}$ contain a high probability correspondence for d^k . The intuition is that descriptors d^k corresponding to points on the object that are easy to localise and remain unoccluded will have a high confidence score. As in the **DS** method, we initially select a large number of descriptors ($K = 100$) corresponding to points on the object. We then choose the K^* descriptors with the highest average confidence on the training set, which also meet a threshold on the minimum separation distance (typically 25 pixels in a 640×480 image). In the experiments, we use $K^* = 4$ or $K^* = 5$.

4.2.2. Learning the dynamics

We adopt a standard dynamics learning framework as we discussed in Section 2.1, where we keep the weights of the visual-correspondence network, g^{dc} , fixed and aim to predict the evolution of the self-supervised 3D keypoints z_t through $z_{t+1} = f_\theta(z_t, u_t)$ by minimising the prediction error in Equation (2).

4.2.3. Online planning for closed-loop control

After we've trained a dynamics model $z_{t+1} = f_\theta(z_t, u_t)$, we use Model Predictive Control (MPC) with online planning to select an action. Given a goal keypoint state z^* , our aim is to find a sequence of

actions $\{u_t\}_{t=0}^H$ over a look-ahead horizon of H that minimises the cost $\sum_{t=0}^H c(z_t, u_t)$ (here, the distance is measured directly in the keypoint space, rather than in the original observation space). Our approach to dynamics learning is independent of the optimiser used to resolve the MPC problem, focusing more on visual and dynamics learning than on MPC specifics. Various model-based RL methods^{13,17,26,57,98} employ a random-sampling-based planner (like the cross-entropy method) to tackle the underlying MPC problem. We conducted experiments with random shooting, gradient-based shooting, cross-entropy, and Model Predictive Path Integral (MPPI) planners and found MPPI to be the most effective in our scenarios.

We provide a brief overview of MPPI here but refer readers to Williams *et al.*⁹⁴ for a more detailed explanation. MPPI is a gradient-free optimiser that takes into account coordination between time steps when sampling action trajectories. The algorithm samples N trajectories, rolls them out using the trained model, calculates the reward/cost for each trajectory, and then re-weights the trajectories to sample a new set. A single trajectory in the MPC’s look-ahead horizon H consists of state-action pairs $\{(z_t, u_t)\}_{t=0}^H$. Let $R^{(k)} = \sum_{t=0}^H r(z_t^{(k)}, u_t^{(k)})$ represent the reward of the k th trajectory, which could be the negation of the cost $c(z_t^{(k)}, u_t^{(k)})$. Define

$$\mu_t = \frac{\sum_{k=1}^N \left(e^{\gamma R^{(k)}} \right) u_t^{(k)}}{\sum_{k=1}^N e^{\gamma R^{(k)}}}, \quad \forall t \in \{0, \dots, H\}.$$

A filtering technique is then employed to sample a *new* batch of trajectories from the previously computed mean μ_t , specifically

$$u_t^{(k)} = n_t^{(k)} + \mu_t, \quad (8)$$

where the noise $n_t^{(k)}$ is sampled as

$$\alpha_t^{(k)} \sim \mathcal{N}(0, \Sigma), \quad \forall k \in \{1, \dots, N\}, \quad \forall t \in \{0, \dots, H\}, \quad (9)$$

$$n_t^{(k)} = \beta \alpha_t^{(k)} + (1 - \beta) n_{t-1}^{(k)}, \quad \text{where } n_{t < 0} = 0. \quad (10)$$

This process is repeated for M iterations, after which the best action sequence is chosen. The key to the success of the method is leveraging GPUs for parallel sampling and evaluations.

4.3. Experiments

The goal of the experiments is to answer the following questions: (1) Can the proposed algorithm effectively learn self-supervised 3D keypoints as the latent state in a model-based RL system? (2) How do various design decisions affect the proposed algorithm? (3) How does visual-correspondence learning compare to several benchmark methods in terms of learning effective keypoint dynamics? (4) Can we implement this method on real hardware?

The experiments are conducted using four simulated tasks (illustrated in Figure 4) and one hardware task. All tasks require pushing an object to a specified goal state. The first three simulation tasks, labelled as *top-down*, *angled*, and *occlusions*, involve pushing a single object. The *top-down* task positions cameras in a top-down orientation, while the *angled* task tilts them at 45° . The *occlusions* task maintains the angled camera positions but positions the box on a different face, leading to significant self-occlusions. The *mugs* task requires pushing various objects from a category, specifically mugs of different sizes, shapes, and textures. The hardware task is essentially identical to the *angled* simulation task.

4.3.1. Self-supervised 3D keypoints from visual correspondence

Figures 2 and 3 show the performance of the self-supervised 3D keypoints learning procedure. Specifically, Figure 2 illustrates the localisation performance on real data throughout a trajectory, while Figure 3 presents an example of the confidence scores used in the **SDS** method. Figure 4 reveals the descriptors employed in the **SDS** method for each of our simulation tasks. In particular, Figures 4(d) and 4(e) demonstrate the descriptors’ capability to accurately find correspondences across different object instances within a category, despite variations in colour and shape.

4.3.2. Ablations on self-supervised 3D keypoints for dynamics learning

Ablation studies indicate that the decision of *what to track* can significantly affect model performance, particularly in scenarios involving partial occlusions. Table 1 details the quantitative results. In

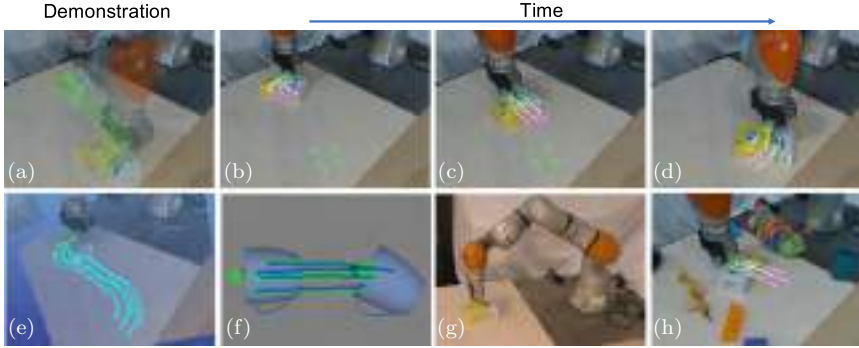


Fig. 2. Self-supervised 3D keypoint dynamics for model-predictive control. (a) Initial pose (blue keypoints) and goal pose (green keypoints), along with the demonstration trajectory. (b)–(d) Model-predictive control (MPC) at different stages throughout the episode. The current keypoints are represented in blue, while the white lines and purple keypoints depict the optimised trajectory from the MPC algorithm, using the learned dynamics model. The goal keypoints remain displayed in green. (e) Demonstration trajectory in a 3D visualiser. (f) Dynamics model on a category-level task. The actual keypoint trajectory is presented in green, and the predicted trajectory using the learned model is shown in blue. (g) Snapshot of the hardware setup. (h) Visual clutter used to evaluate the method’s robustness.



Fig. 3. Visualisation of the learned visual-correspondence model on a reference image (a) and target image (b). The coloured numbers in the target image represent the probability of the detected correspondence being valid. The green reticle indicates a valid correspondence with a high confidence score, while the red reticle indicates a scenario where no correspondence exists in the target image due to occlusion, resulting in a low confidence probability. The confidence heatmap is displayed in the bottom right corner.

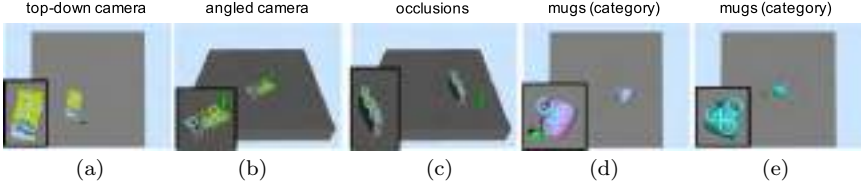


Fig. 4. This figure shows reference descriptors $\{d_i\}_{i=1}^K$ of the SDS variant for our different simulation environments. In particular, in (c), the sides of the object undergo occlusions as the box rotates about the vertical axis. (d)–(e) Two different mugs from the category-level *mugs (category)* task.

Table 1. Ablations and comparison with ground truth — quantitative results for various ablations of our method on four simulated tasks. Each method was evaluated on the same set of 200 different initial and goal states.

Task	Top-down camera		Angled camera		Occlusions		Mugs (category)	
	pos, cm	angle, °	pos, cm	angle, °	pos, cm	angle, °	pos, cm	angle, °
Avg.	11.32	32.05	11.32	32.05	10.36	31.81	11.28	56.25
GT 3D	1.14	4.01	1.14	4.01	1.29	5.45	—	—
DS	1.12	4.07	1.45	5.50	3.66	12.27	1.19	11.73
SDS	1.19	3.88	1.10	3.97	1.46	6.77	1.20	15.03

Notes: The *pos* and *angle* columns denote the translational (in cm) and rotational (in degrees) deviations of the object from the goal position, averaged across all trials for a specific method and task. *Avg.* denotes the average translation and rotation between the initial and goal poses for each task.

the *top-down* and *angled camera* tasks, all methods perform reasonably well, almost equalling the performance of the **GT 3D** baseline, which utilises ground-truth state information. This is likely because these settings involve minimal occlusions, enabling the reliable tracking of nearly all keypoints through dense visual correspondence. In contrast, the *occlusions* task introduces the potential for significant occlusions. Given the camera angle depicted in Figure 4(c) and the full 360° of yaw the object rotates through during the task, only the top face of the box remains visible, while the four side faces alternately become occluded and visible. This task highlights significant performance differences among our various ablations. Notably, **SDS** outperforms **DS**. We hypothesise this is because some of the descriptors $\{d^k\}_{k=1}^K$ tracked in **DS** correspond to object locations that

become occluded during an episode. The dense visual-correspondence model cannot track keypoints through occlusions, and when attempting to localise an occluded point, our dense-correspondence model maps it to the closest point in descriptor space, which is not the true correspondence location. Therefore, the keypoint locations that form the latent state z_t^{object} for **DS** experience reduced accuracy, resulting in a less accurate dynamics model and ultimately, lower performance when used for closed-loop MPC.

In the category-level *mugs* task, the camera is positioned top-down, mitigating occlusion issues. However, the task includes different objects from a category, introducing shape variations among the objects. Consequently, methods employing $K = 50$ keypoints, such as **DS**, outperform the sparser variants like **SDS**, which use only $K = 4, 5$ keypoints. We believe this is because having a greater number of keypoints better captures the shape variations across object instances, allowing for a more accurate dynamics model.

4.3.3. Comparison with baselines

In our comparisons against existing baselines, our model surpasses alternatives across all experimental tasks. The most noticeable performance differences emerge in the *occlusions* and *mugs* tasks, which deal with partial occlusions and category-level generalisation, respectively. The Transporter (Trans.) baseline utilises keypoint locations from Kulkarni *et al.*³⁵ as the latent state z , while the Autoencoder (AE) baseline jointly trains an autoencoder with a forward dynamics model. Table 2 details the quantitative results.

In the *top-down camera* task, the Trans. Kulkarni *et al.*³⁵ model achieved slightly lower performance than **SDS**, while AE lagged significantly behind. The feature transport approach of the Trans. model is ideally suited for this top-down setting. On the *angled camera* and *occlusions* tasks, which use angled camera positions in contrast to the top-down task, our methods maintained consistent performance, while Trans. fell short. This could be because the feature transport mechanism of Trans. doesn't adapt well to off-axis camera positions in 3D worlds. The performance of the AE baseline remained consistently worse than our approach across tasks without category-level generalisation.

Table 2. Comparisons with baselines — quantitative results of our method compared to various baselines on our four simulated tasks. Each method was evaluated on the same set of 200 different initial and goal states.

Task	Top-down camera		Angled camera		Occlusions		Mugs (category)	
	pos, cm	angle, °	pos, cm	angle, °	pos, cm	angle, °	pos, cm	angle, °
SDS (ours)	1.19	3.88	1.10	3.97	1.46	6.77	1.20	15.03
Trans. 3D	2.01	15.95	4.36	25.14	3.72	20.65	2.81	61.22
Trans. 2D	2.08	13.59	3.60	23.60	2.74	18.86	2.33	60.18
AE	2.18	14.79	2.85	13.79	3.08	14.20	9.05	56.84

Notes: The *pos* and *angle* denote the translational (in cm) and rotational (in degrees) deviations of the object from the goal position, averaged across all trials for a specific method and task.

The *mugs* task introduces a variety of mug shapes with differing visual appearances, thereby testing the category-level generalisation capabilities of both the perception and dynamics models. As discussed in Section 4.3.1, our dense-correspondence model can identify correspondences across these appearance variations using only self-supervision, enabling us to develop a dynamics model effective for completing the task. This task poses significantly more challenges than the others, not only due to the presence of novel objects but also because the goal states involve much larger rotations, as detailed in the first row of Table 1. Both Trans. and AE baselines perform poorly in this task. We hypothesise that this is due to the fact that there is much more variance in the visual appearance of the objects as compared to the other tasks, and thus the latent state z produced by these baselines is not amenable to dynamics learning.

4.3.4. Hardware evaluation

Experimental setup: Our hardware experiments utilised a Kuka IIWA LBR robot, equipped with a custom cylindrical pusher attached to its end-effector, as depicted in Figure 2. We relied on RGBD sensing from two offboard RealSense D415 cameras, which were rigidly mounted and calibrated to the robot’s coordinate frame. To ensure effective correspondence learning between views, it’s ideal to maintain *some* overlap between views for correspondence purposes, while keeping the views distinct enough. During testing, only

a single camera was used to localise the self-supervised 3D keypoints. The robot was operated by commanding end-effector velocity in the xy plane at a frequency of 5 Hz.

Hardware results: For visual learning, we collected a small dataset of the object positioned differently to provide a diverse set of views for training the dense correspondence and the self-supervised keypoint detection model. For dynamics learning, a dataset was gathered where the robot randomly pushed the object around. This comprised roughly 10 minutes of interaction time and was employed to train the dynamics model. All hardware experiments utilised the **SDS** method. To enable our MPC controller to execute long-horizon tasks, we supplied the controller with a reference trajectory for the object keypoints that came from a single demonstration, as shown in Figures 2(a)–(d). The MPC controller tracked this reference trajectory using a 2-second MPC horizon, which equates to $H = 10$ since we are issuing commands at 5 Hz. We collected four different reference trajectories^b and executed multiple rollouts for each trajectory, altering the initial condition of the object pose during each run to test our controller’s region of attraction. In every case, our controller demonstrated the capability to stabilise the system to the reference trajectory despite perturbations in the initial condition. Figure 5 provides detailed quantitative results. Specifically, the controller’s ability to

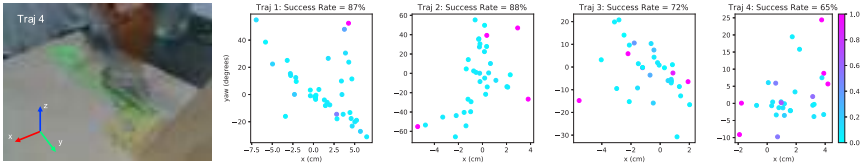


Fig. 5. Hardware results. The image on the left shows a demonstration trajectory. Scatter plots provide results of our method on the four distinct reference trajectory tracking tasks. The plot axes depict the deviation of the object’s starting pose from the initial pose in the demonstration. The colour represents the distance between the final and goal poses, with a lower cost indicating a better result. The various reference trajectories present differing degrees of difficulty, as indicated by the distinct attraction regions of the MPC controller.

Note: Further details are available in Appendix 4.6, and videos can be viewed at <https://sites.google.com/view/keypointsintothefuture>.

^bRefer to Appendix 4.6 for details on the reference trajectories.

stabilise the trajectory amidst disturbances in the initial condition. For trajectory (1), the controller can stabilise disturbances up to 60° , while trajectory (4) has a much narrower attraction region. As outlined in Appendix 4.6, trajectory (1) is relatively straightforward with minimal orientation change between start and goal, while trajectory (4) entails a challenging 180° orientation change and obliges the robot to operate at the edge of its kinematic workspace, limiting its control authority. Overall, the proposed system exhibits remarkable feedback capabilities and can accurately track a trajectory in the keypoint latent space, facilitating one-shot imitation learning. We invite readers to visit <https://sites.google.com/view/keypointsintothefuture> to see the system in action.

4.4. Discussion

This section introduced a method that utilises self-supervised visual-correspondence learning as an input for a 3D-keypoint-based predictive dynamics model. This approach generates interpretable latent states that outperform various competing baselines across a multitude of simulated manipulation tasks. Additionally, we demonstrated how the self-supervised keypoint dynamics model can generalise at a category level, leading to substantial performance enhancements over the baselines. Lastly, we highlighted the method’s application on a real hardware system, demonstrating its ability to stabilise complex long-horizon plans by tracking the 3D-keypoint trajectory derived from a single demonstration.

4.5. Appendix: Dense Correspondence for 3D Keypoints

Here we give a brief overview of the dense correspondence model formulation^{20,72} with spatial distribution losses.^{18,19} We briefly explain the loss functions and how the descriptors $\{d_k\}$ are localised.

Network architecture: We use an architecture that produces a full-resolution descriptor image. Namely, it maps

$$W \times H \times 3 \rightarrow W \times H \times D. \quad (11)$$

We use a fully convolutional network architecture (FCN)⁴⁹ with a ResNet-50 or ResNet-101 with the number of classes set to the descriptor dimension.

For all shown experiments, we use the spatially distributed loss formulation with a combination of heatmap and 3D spatial expectation losses.

Heatmap loss: Let p^* be the pixel space location of a ground-truth match. Then we can define the ground-truth heatmap as

$$H_{p^*}^*(p) = \exp\left(-\frac{\|p - p^*\|_2^2}{\sigma^2}\right), \quad (12)$$

where p represents a pixel location. A predicted heatmap can be obtained from the descriptor image $\mathcal{I}_D$ together with a reference descriptor d^* . Then the predicted heatmap is gotten by

$$\hat{H}(p; d^*, \mathcal{I}_D, \eta) = \exp\left(-\frac{\|\mathcal{I}_D(p) - d^*\|_2^2}{\eta^2}\right). \quad (13)$$

The heatmap can also be normalised to sum to one, in which case it represents a probability distribution over the image:

$$\tilde{H}(p) = \frac{\hat{H}(p)}{\sum_{p' \in \Omega} \hat{H}(p')}. \quad (14)$$

The heatmap loss is simply the MSE between H^* and $\hat{H}$ with mean reduction:

$$L_{\text{heatmap}} = \frac{1}{|\Omega|} \sum_{p \in \Omega} \|\hat{H}(p) - H^*(p)\|_2^2. \quad (15)$$

Spatial expectation loss: Given a descriptor d^* together with a descriptor image $\mathcal{I}_D$, we can compute the 2D spatial expectation as

$$J_{\text{pixel}}(d^*, \mathcal{I}_D, \eta) = \sum_{p \in \Omega} p \cdot \tilde{H}(p; d^*, \mathcal{I}_D, \eta). \quad (16)$$

If we also have a depth image $\mathbf{D}$, then we can define the spatial expectation over the depth channel as

$$J_{\text{depth}}(d^*, \mathcal{I}_D, \mathbf{D}, \eta) = \sum_{p \in \Omega} \mathbf{D}(p) \cdot \tilde{H}(p; d^*, \mathcal{I}_D, \eta). \quad (17)$$

The spatial expectation loss is simply the L1 loss between the ground truth and estimated correspondence using

$$L_{\text{spatial pixel}} = \|p^* - J_{\text{pixel}}(d^*)\|_1. \quad (18)$$

We can also use our 3D spatial expectation J_{depth} to compute a 3D spatial expectation loss. In particular, given a depth image $\mathbf{D}$, let the depth value corresponding to pixel p be denoted by $\mathbf{D}(p)$. The spatial expectation loss is simply

$$L_{\text{spatial depth}} = \|\mathbf{D}(p^*) - J_{\text{depth}}(d^*, \mathcal{I}_D, \mathbf{D}, \eta)\|_1. \quad (19)$$

Be careful to only take the expectation over pixels with valid depth values $\mathbf{D}(p)$.

Total loss: The total loss is the weighted sum of the heatmap loss and the spatial loss:

$$L = w_{\text{heatmap}} L_{\text{heatmap}} + w_{\text{spatial}} (L_{\text{spatial pixel}} + L_{\text{spatial depth}}), \quad (20)$$

where the w are weights.

Correspondence function: The correspondence function $g^{\text{corr}}(\mathcal{I}_D, d^k)$ in Section 4.2.1 is defined using the spatial expectations $J_{\text{pixel}}, J_{\text{depth}}$ defined earlier to localise the descriptor d^k in either pixel space or 3D space. If in 3D, we additionally use the known camera extrinsics to express the localised point in the world frame.

4.6. Appendix: Hardware Experiment Details

Hardware setup: We used a Kuka IIWA LBR robot with a custom cylindrical pusher attached to the end-effector to perform our hardware experiments; see Figure 6. RGBD sensing was provided by two RealSense D415 cameras rigidly mounted offboard the robot and calibrated to the robot’s coordinate frame. To enable effective correspondence learning between views, it is ideal to have views with *some*



Fig. 6. Overview of the experimental setup. It includes two external RealSense D415 cameras. Images from both cameras are used to train the dense-descriptor model, while only the right camera is used at runtime to localise the keypoints on the object.

overlap such that correspondences exist but still maintain different-enough viewpoints from each camera. At test time, only a single camera is used to localise the dense-descriptor keypoints. The robot is controlled by commanding end-effector velocity in the xy plane at 5 Hz. A high-rate Jacobian space controller consumes these 5 Hz end-effector velocity commands and closes the loop to command the robot’s joint positions at 200 Hz.

5. Particle Representation

In the preceding section, we concentrated on the use of keypoint representations for pusher-slider systems, particularly in manipulating rigid objects. However, real-world control tasks involve an array of materials, including soft bodies, granular substances, liquids, and

gases. Each of these materials exhibits unique physical behaviours and a high degree of freedom, making keypoint representations insufficient for capturing their variations. Physics-based simulators using dynamic 3D particles have been developed to model the dynamics of such complex scenarios. Still, due to a reliance on approximation techniques, their simulations often deviate from real-world physics, especially over longer periods. In this section, we discuss an alternative method: learning a particle-based simulator directly from data for complex embodied AI interaction tasks. This combination of learning with particle-based systems offers two primary advantages. First, the learned simulator, similar to other particle-based systems, can be applied to objects of various materials with high degrees of freedom. Second, the particle-based representation provides a strong inductive bias for learning because particles of the same type exhibit similar dynamics. This methodology allows the model to learn representations of intricate interactive systems from a relatively small dataset and generalise better than baseline models that do not exploit the underlying structure. We demonstrate how embodied AI agents can accomplish complex manipulation tasks using the learned simulator, such as manipulating fluids and deformable foam, through experiments in both simulated and real-world environments. Please visit the project page for video illustrations: <http://dpi.csail.mit.edu>.

This section was previously published as Li *et al.*⁴⁴ in the International Conference on Learning Representations (ICLR) 2019, with Yunzhu Li, Jiajun Wu, Russ Tedrake, Joshua B. Tenenbaum, and Antonio Torralba as co-authors.

5.1. Overview

Objects exhibit distinct dynamics. For instance, under the same push, a rigid box will slide, modelling clay will deform, and a cup filled with water will tip over, spilling its contents. The diverse behaviours of different objects pose challenges to traditional rigid-body simulators used in robotics.^{84,86} Although particle-based simulators strive to model the dynamics of these complex scenes,^{32,51} their dependence on approximation techniques for the sake of perceptual realism often results in deviations from real-world physics, particularly in the long term. Hence, developing generalisable and

accurate forward dynamics models directly from observational data is crucial for the manipulation of diverse real-life objects by robots.

In this section, we discuss a differentiable, particle-based simulator for complex control tasks, based on neural networks, drawing inspiration from recent developments in learning-based physical engines.^{2,6} In robotics, the use of simulators capable of producing gradients, coupled with continuous and symbolic optimisation algorithms, has enabled planning for increasingly complex whole-body motions with multi-contact and multi-object interactions.⁸⁷ Yet, most of the work in this direction has only tackled interactions within rigid body systems. We utilise 3D particles as the scene representation for the environment and develop dynamic particle interaction networks (DPI-Nets) for learning particle dynamics, focusing on capturing the dynamic, hierarchical, and long-range interactions of particles (Figures 7(a)–(c)). DPI-Nets can then be integrated with classic perception and gradient-based optimisation algorithms for manipulating deformable objects by robots (Figure 7(d)).

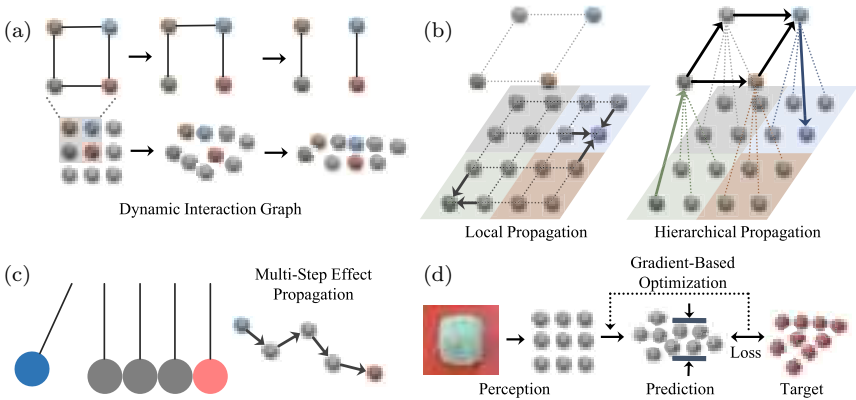


Fig. 7. Learning particle dynamics for control. (a) DPI-Nets dynamically construct the interaction graph over time while learning particle interactions. (b) A hierarchical graph is built to propagate effects across multiple scales. (c) Multi-step message passing is employed for managing instantaneous force propagation. (d) Perception and control with the learned model involves several steps. First, our system reconstructs the particle-based shape from visual observation. Then, it utilises gradient-based trajectory optimisation to search for the actions that yield the most desirable output.

As noted before, learning a particle-based simulator brings two significant benefits. First, the learned simulator, akin to other particle-based systems, operates broadly on objects of a range of different materials. DPI-Nets have successfully captured the complex behaviours of deformable objects, fluids, and rigid bodies. Using learned DPI-Nets, our robots have accomplished manipulation tasks that involve deformable objects with complex physical properties, such as molding plasticine into a target shape. Second, the particle-based representation offers a strong inductive bias for learning: particles of the same type exhibit the same dynamics. This facilitates the sharing of parameters in the neural modules that compute the interactions between particles. As a result, the model can learn from a relatively small dataset and extrapolate to testing environments that are either larger or smaller than the training distribution. Experiments suggest that DPI-Nets rapidly learn to characterise a novel object with unknown physical properties. The learned model also assists the robot in successfully manipulating deformable objects in the real world.

DPI-Nets combine three key features for effective particle-based simulation and control: multi-step spatial propagation, hierarchical particle structure, and dynamic interaction graphs. In particular, it employs dynamic interaction graphs, constructed on the fly throughout manipulation, to capture the meaningful interactions among particles of deformable objects and fluids. The use of dynamic graphs allows neural models to focus on learning meaningful interactions among particles and is crucial for obtaining accurate simulations and high success rates in manipulation. Since objects deform as robots interact with them, a fixed interaction graph over particles is insufficient for manipulating non-rigid objects by robots.

Experiments demonstrate that DPI-Nets significantly outperform interaction networks,² HRN,⁵⁵ and several other related baselines. More importantly, unlike previous studies that focused solely on forward simulation, this section also shows the effectiveness of the learned model in downstream control tasks. DPI-Nets enable complex manipulation tasks for deformable objects and fluids and adapt to scenarios with unknown physical parameters that need to be identified online. We have also performed real-world experiments to demonstrate our model’s generalisation ability.

5.2. Learning-Based 3D Particle Dynamics

5.2.1. Preliminaries

The learning-based 3D particle dynamics model is developed based on interaction networks (INs).² We first outline how INs represent the physical system and then extend them to accommodate particle-based dynamics.

We describe the scene representation that represents the interactions within a physical system as a directed graph, $z = (V, E)$, where vertices $V = \{v_i\}$ correspond to objects and edges $E = \{e_k\}$ denote relations. Specifically, $v_i = (x_i, r_i^v)$, where $x_i = (q_i, \dot{q}_i)$ signifies the state of object i , encompassing its position q_i and velocity $\dot{q}_i$. The term r_i^v designates its attributes (e.g., mass and radius). For the relation, we define $e_k = (a_k, b_k, r_k^e)$, $1 \leq a_k, b_k \leq |V|$, where a_k is the receiver, b_k is the sender, and both are integers. r_k^e represents the type and attributes of relation k (e.g., collision and spring connection).

As discussed in Section 2.1, the objective is to construct a learnable physical engine capable of capturing the underlying physical interactions using function approximators f_θ . The learned model can subsequently be used for system identification and predicting the future state from the current interaction graph as $z_{t+1} = f_\theta(z_t, u_t)$, where z_t in this section denotes the particle-based scene representation at time t .

Interaction networks: Battaglia *et al.*² proposed INs, a general-purpose, learnable physics engine that performs object- and relation-centric reasoning about physics. INs define an object function f_V and a relation function f_E to model objects and their relations in a compositional way. The future state at time $t + 1$ is predicted as

$$\hat{e}_{k,t} = f_E(v_{a_k,t}, v_{b_k,t}, r_k^e) \quad k = 1, \dots, |E|, \quad (21)$$

$$\hat{v}_{i,t+1} = f_V\left(v_{i,t}, \sum_{k \in \mathcal{N}_i} \hat{e}_{k,t}\right) \quad i = 1, \dots, |V|, \quad (22)$$

where $v_{i,t} = (x_{i,t}, r_i^v)$ denotes object i at time t , a_k and b_k are the receiver and sender of relation e_k , respectively, and $\mathcal{N}_i$ denotes the relations where object i is the receiver.

Propagation networks: A limitation of INs is that at every time step t , it only considers local information in the graph $z_t = (V_t, E_t)$ and cannot handle instantaneous changes in forces/accelerations in other bodies, which however is a common phenomenon in rigid-body dynamics. Li *et al.*⁴⁵ proposed propagation networks to approximate the instantaneous force changes by doing multi-step message passing. Specifically, they first employed the ideas on fast training of RNNs^{4,37} to encode the shared information beforehand and reuse them along the propagation steps. The encoders for objects are denoted as f_V^{enc} and the encoder for relations as f_E^{enc} , where we denote

$$c_{k,t}^e = f_E^{\text{enc}}(v_{a_k,t}, v_{b_k,t}, r_k^e) \quad k = 1, \dots, |E|, \quad (23)$$

$$c_{i,t}^v = f_V^{\text{enc}}(v_{i,t}), \quad i = 1, \dots, |V|. \quad (24)$$

At time t , denote the propagating influence from relation k at propagation step l as $\hat{e}_{k,t}^l$, and the propagating influence from object i as $\hat{v}_{i,t}^l$. For step $1 \leq l \leq L$, propagation can be described as

$$\text{Step 0:} \quad \hat{v}_{i,t}^0 = \mathbf{0}, \quad i = 1, \dots, |V|, \quad (25)$$

$$\text{Step } l = 1, \dots, L: \quad \hat{e}_{k,t}^l = f_E \left(c_{k,t}^e, \hat{v}_{a_k,t}^{l-1}, \hat{v}_{b_k,t}^{l-1} \right), \quad k = 1, \dots, |E|, \quad (26)$$

$$\hat{v}_{i,t}^l = f_V \left(c_{i,t}^v, \sum_{k \in \mathcal{N}_i} \hat{e}_{k,t}^l, \hat{v}_{i,t}^{l-1} \right), \quad i = 1, \dots, |V|, \quad (27)$$

$$\text{Output:} \quad \hat{v}_{i,t+1} = f_V^{\text{output}}(\hat{v}_{i,t}^L), \quad i = 1, \dots, |V|, \quad (28)$$

where f_V denotes the object propagator and f_E denotes the relation propagator.

5.2.2. Dynamic particle interaction networks

Particle-based system is widely used in physical simulation due to its flexibility in modelling various types of objects.⁵¹ We extend existing systems that model object-level interactions to allow particle-level deformation. Consider object set V containing N objects, where each object $v_i = \{v_i^k\}_{k=1 \dots |v_i|}$ is represented as a set of particles. We

now define the graph on the particles and the rules for influence propagation.

Dynamic graph building: The vertices of the graph are the union of particles for all objects $V = \{v_i^k\}_{i=1\dots N, k=1\dots |v_i|}$. The edges E between these vertices are dynamically generated over time to ensure efficiency and effectiveness. The construction of the relations is specific to the environment and task, which we elaborate on in Section 5.3. A common choice is to consider the neighbours within a pre-defined distance.

An alternative is to build a static, complete interaction graph, but it has two major drawbacks. First, it is not efficient. In many common physical systems, each particle is only interacting with a limited set of other particles (e.g., those within its neighbourhood). Second, a static interaction graph implies a universal, continuous neural function approximator; however, many physical interactions involve discontinuous functions (e.g., contact). In contrast, using dynamic graphs empowers the model to tackle such discontinuity.

Hierarchical modelling for long-range dependence: Propagation networks⁴⁵ require a large L to handle long-range dependence, which is both inefficient and hard to train. Hence, we add one level of hierarchy to efficiently propagate the long-range influence among particles.⁵⁵ For each object that requires modelling of the long-range dependence (e.g., rigid-body), we cluster the particles into several non-overlapping clusters. For each cluster, we add a new particle as the cluster’s root. Specifically, for each object v_i that requires hierarchical modelling, the corresponding roots are denoted as $\tilde{v}_i = \{\tilde{v}_i^k\}_{k=1\dots |\tilde{v}_i|}$, and the particle set containing all the roots is denoted as $\tilde{V} = \{\tilde{v}_i^k\}_{i=1\dots N, k=1\dots |\tilde{v}_i|}$. We then construct an edge set $E_{\text{LeafToRoot}}$ that contains directed edges from each particle to its root and an edge set $E_{\text{RootToLeaf}}$ containing directed edges from each root to its leaf particles. For each object that needs hierarchical modelling, we add pairwise directed edges between all its roots and denote this edge set as $E_{\text{RootToRoot}}$.

We employ a multi-stage propagation paradigm: first, propagation among leaf nodes, $f_{\text{LeafToLeaf}}(V, E)$; second, propagation from leaf nodes to root nodes, $f_{\text{LeafToRoot}}(V \cup \tilde{V}, E_{\text{LeafToRoot}})$; third, propagation between roots, $f_{\text{RootToRoot}}(\tilde{V}, E_{\text{RootToRoot}})$; fourth,

propagation from root to leaf, $f_{\text{RootToLeaf}}(V \cup \tilde{V}, E_{\text{RootToLeaf}})$. The signals on the leaves are used to do the final prediction.

Applying to objects of various materials: We define the interaction graph and the propagation rules on particles for different types of objects as follows:

- *Rigid bodies:* All the particles in a rigid body are globally coupled; hence for each rigid object, we define a hierarchical model to propagate the effects. After the multi-stage propagation, we average the signals on the particles to predict a rigid transformation (rotation and translation) for the object. The motion of each particle is calculated accordingly. For each particle, we also include its offset to the centre of mass to help determine the torque.
- *Elastic/Plastic objects:* For elastically deforming particles, only using the current position and velocity as the state is not sufficient, as it is not clear where the particle will be restored after the deformation. Hence, we include the particle state with the resting position to indicate the place where the particle should be restored. When coupled with plastic deformation, the resting position might change during an interaction. Thus, we also infer the motion of the resting position as a part of the state prediction. We use hierarchical modelling for this category but predict the next state for each particle individually.
- *Fluids:* For fluid simulation, one has to enforce density and incompressibility, which can be effectively achieved by only considering a small neighbourhood for each particle.⁵⁰ Therefore, we do not need hierarchical modelling for fluids. We build edges dynamically, connecting a fluid particle to its neighbouring particles. The strong inductive bias leveraged in the fluid particles allows good performance even when tested on data outside training distributions.

For the interaction between different materials, two directed edges are generated for any pairs of particles that are closer than a certain distance.

5.2.3. Closed-loop planning using the learned dynamics

Function approximators such as neural networks are naturally differentiable, and off-the-shelf toolboxes for neural networks have

emphasised the speed and ease of use of analytical gradients. We can thus rollout using the learned dynamics and optimise the control inputs by minimising a loss between the simulated results and a target configuration via gradient-based trajectory optimisation. In cases where certain physical parameters are unknown, we can perform online system identification by minimising the difference between the model’s prediction and reality.

Model predictive control using shooting methods: Let’s denote z_g as the goal and $u_{1:H}$ be the control inputs, where H is the time horizon. The control inputs are part of the interaction graph, such as the velocities or the initial positions of a particular set of particles. We denote the resulting trajectory as predicted by our dynamics model $z_{t+1} = f_\theta(z_t, u_t)$ after applying $u_{1:H}$ as $z = \{z_t\}_{t=1:H}$. The task here is to determine the control inputs to minimise the distance between the actual outcome and the specified goal $\mathcal{L}_{\text{goal}}(z, z_g)$. Since the neural networks are naturally differentiable, the loss $\mathcal{L}_{\text{goal}}$ can then be used to update the control inputs by doing stochastic gradient descent (SGD). This is known as the shooting method in trajectory optimisation.⁸³

The learned model might deviate from reality due to accumulated prediction errors. Similar to what we have discussed in Section 4.2.3, we use Model-Predictive Control (MPC)⁵ to account for the prediction error by doing forward simulation and updating the control inputs at every time step.

Online adaptation: In many real-world cases, without actually interacting with the environment, inherent attributes such as mass, stiffness, and viscosity are not directly observable. DPI-Nets can estimate these attributes on the fly with SGD updates by minimising the distance between the predicted future states and the actual future states.

5.3. Experiments

We evaluate our method on four different environments containing different types of objects and interactions. We first describe the environments and show simulation results. We then present how the learned dynamics help complete control tasks in both simulation and the real world.

5.3.1. *Environments*

FluidFall (Figure 8(a)): Two drops of fluids are falling down, colliding, and merging. We vary the initial position and viscosity for training and evaluation.

BoxBath (Figure 8(b)): A block of fluids is flushing a rigid cube. In this environment, we have to model two different materials and the interactions between them. We randomise the initial position of the fluids and the cube to test the model’s generalisation ability.

FluidShake (Figure 8(c)): We have a box of fluids, and the box is moving horizontally. The speed of the box is randomly selected at each time step. We vary the size of the box and volume of the fluids to test generalisation.

RiceGrip (Figure 8(d)): We manipulate an object with both elastic and plastic deformation (e.g., sticky rice). We use two cuboids to mimic the fingers of a parallel gripper, where the gripper is initialised at a random position and orientation. During the simulation of one grip, the fingers will move closer to each other and then restore to their original positions. The model has to learn the interactions between the gripper and the “sticky rice”, as well as the interactions within the “rice” itself.

We use all four environments in evaluating our model’s performance in simulation. We use the rollout MSE as our metric. We further use the latter two for control because they involve fully actuated external shapes that can be used for object manipulation. In FluidShake, the control task requires determining the speed of the box at each time step, in order to make the fluid match a target configuration within a time window; in RiceGrip, the control task corresponds to selecting a sequence of grip configurations (position, orientation, and closing distance) to manipulate the deformable object as to match a target shape. The metric for performance in control is the Chamfer distance between the manipulation results and the target configuration.

5.3.2. *Physical simulation*

Implementation details: (1) For FluidFall, we dynamically build the graph by connecting each particle to its neighbours within a

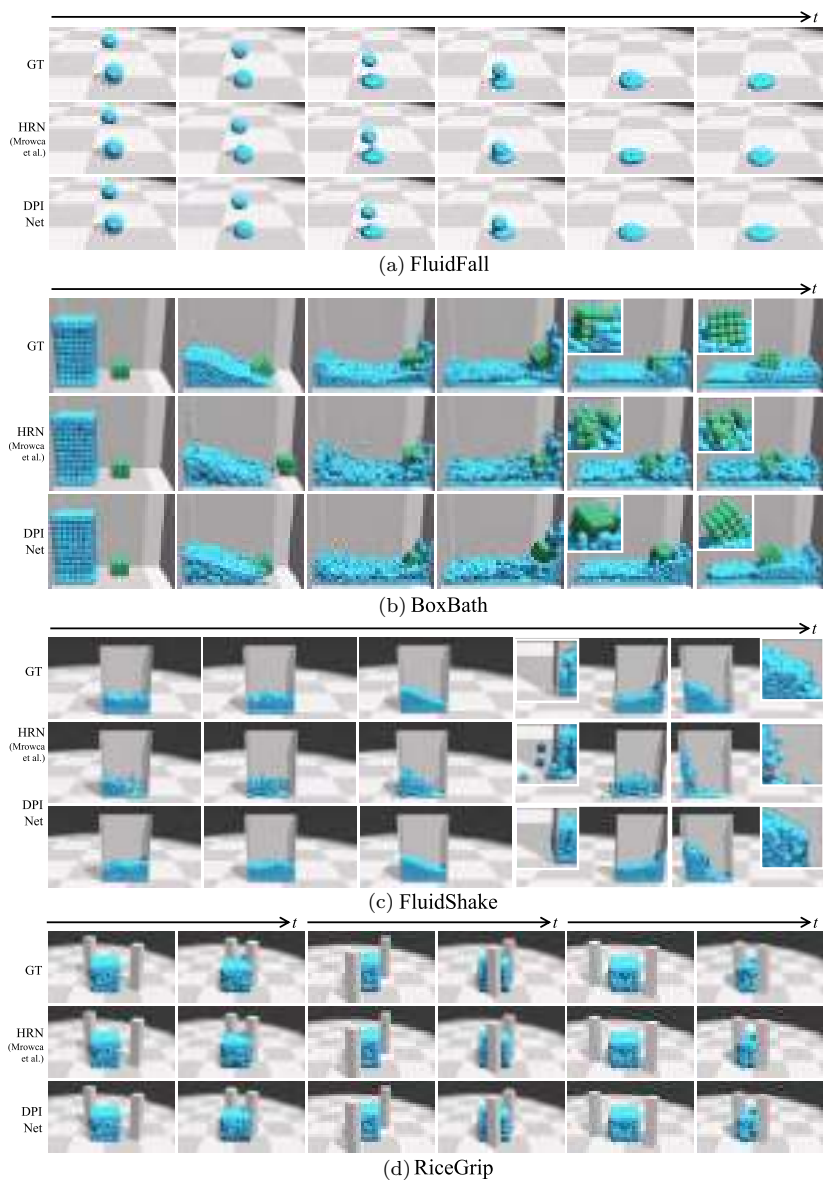


Fig. 8. Qualitative results on forward simulation. We compare the ground truth (GT) and the rollouts from HRN⁵⁵ and our model (DPI-Net) in four environments (FluidFall, BoxBath, FluidShake, and RiceGrip). The simulations from our DPI-Net are significantly better. We provide zoom-in views for a few frames to show details. Please see our video for more empirical results.

certain distance d . No hierarchical modelling is used. (2) For BoxBath, we model the rigid cube as in Section 5.2.2, using multi-stage hierarchical propagation. Two directed edges will be constructed between two fluid particles if the distance between them is smaller than d . Similarly, we also add two directed edges between rigid particles and fluid particles when their distance is smaller than d . (3) For FluidShake, we model fluid as in Section 5.2.2. We also add five external particles to represent the walls of the box. We add a directed edge from the wall particle to the fluid particle when they are closer than d . The model is a single propagation network, where the edges are dynamically constructed over time. (4) For RiceGrip, we build a hierarchical model for rice and use four propagation networks for multi-stage effect propagation (Section 5.2.2). The edges between the “rice” particles are dynamically generated if two particles are closer than d . Similar to FluidShake, we add two external particles to represent the two “fingers” and add an edge from the “finger” to the “rice” particle if they are closer than the distance d . As “rice” can deform both elastically and plastically, we maintain a resting position that helps the model restore a deformed particle. The output for each particle is a 6-dim vector for the velocity of the current observed position and the resting position.

Results: Qualitative and quantitative results are in Figure 8 and Table 3. We compare our method (DPI-Net) with three baselines: Interaction Networks,² HRN,⁵⁵ and DPI-Net without hierarchy.

Table 3. Quantitative results on forward simulation. MSE ($\times 10^{-2}$) between the ground truth and model rollouts. The hyperparameters used in our model are fixed for all four environments. FluidFall and FluidShake involve no hierarchy, so DPI-Net performs the same as the variant without hierarchy. DPI-Net significantly outperforms HRN⁵⁵ in modelling fluids (BoxBath and FluidShake) due to the use of dynamic graphs.

Methods	FluidFall	BoxBath	FluidShake	RiceGrip
IN ²	2.74 ± 0.56	N/A	N/A	N/A
HRN ⁵⁵	0.21 ± 0.04	3.62 ± 0.40	3.58 ± 0.77	0.17 ± 0.11
DPI-Net w/o hierarchy	0.15 ± 0.03	2.64 ± 0.69	1.89 ± 0.36	0.29 ± 0.13
DPI-Net	0.15 ± 0.03	2.03 ± 0.41	1.89 ± 0.36	0.13 ± 0.07

Note that we use the same set of hyperparameters in our model for all four testing environments. Specifically, Interaction Networks (INs) consider a complete graph of the particle system. Thus, it can only operate on small environments, such as FluidFall; it runs out of memory (12 GB) for the other three environments. IN does not perform well because its use of a complete graph makes training difficult and inefficient and because it ignores influence propagation and long-range dependence.

Without a dynamic graph, modelling fluids becomes hard because the neighbours of a fluid particle change constantly. Table 3 shows that for environments that involve fluids (BoxBath and FluidShake), DPI-Net performs better than those with a static interaction graph. Our model also performs better in scenarios that involve objects of multiple states (BoxBath, Figure 8(b)) because it uses state-specific modelling. Models such as HRN⁵⁵ aim to learn a universal dynamics model for all states of matter. It is therefore solving a harder problem and, for this particular scenario, expected to perform not as well. When augmented with state-specific modelling, HRN’s performance is likely to improve, too. Without hierarchy, it is hard to capture long-range dependence, leading to performance drop in environments that involve hierarchical object modelling (BoxBath and RiceGrip).

Ablation studies: We also test our model’s sensitivity to hyperparameters. We consider three of them: the number of roots for building hierarchy, the number of propagation steps L , and the size of the neighbourhood d . We test them in RiceGrip. As can be seen from the results shown in Figure 9(c), DPI-Nets can better capture the motion of the “rice” by using fewer roots, on which the information might be easier to propagate. Longer propagation steps do not necessarily lead to better performance, as they increase training difficulty. Using larger neighbourhoods achieves better results but makes computation slower. Using one TITAN Xp, each forward step in RiceGrip takes 30 ms for $d = 0.04$, 33 ms for $d = 0.08$, and 40 ms for $d = 0.12$.

We also perform experiments to justify our use of different motion predictors for objects of different states. Figure 9(d) shows the results of our model vs. a unified dynamics predictor for all objects in BoxBath. As there are only a few states of interest (solids, liquids, and soft bodies), and their physical behaviours are drastically different, it is not surprising that DPI-Nets, with state-specific motion predictors,

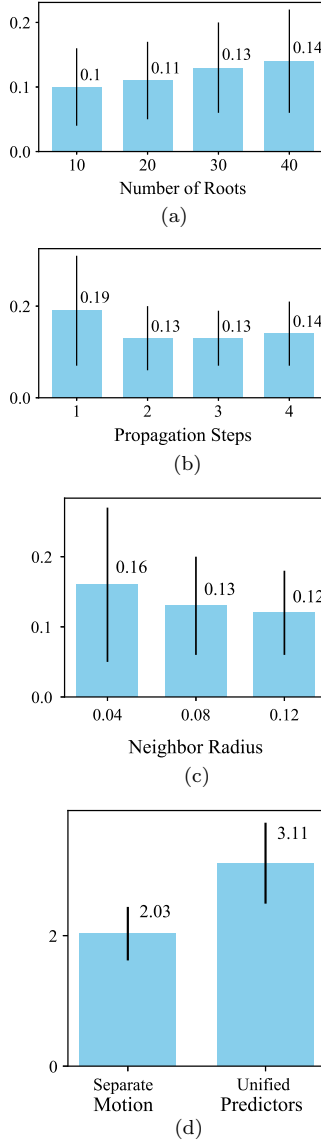


Fig. 9. Ablation studies. We perform ablation studies to test our model’s robustness to hyperparameters. The performance is evaluated by the mean squared distance ($\times 10^{-2}$) between the ground truth and model rollouts. (a–c) We vary the number of roots when building hierarchy, the propagation steps L during message passing, and the size of the neighbourhood d . (d) In BoxBath, DPI-Nets use separate motion predictors for fluids and rigid bodies. Here we compared with a unified motion predictor.

perform better and are equally efficient as the unified model (time difference smaller than 3 ms per forward step).

5.3.3. Control

We leverage dynamic particle interaction networks for control tasks in both simulation and the real world. Because trajectory optimisation using the shooting method can easily be stuck to a local minimum, we first randomly sample N_{sample} control sequences, similar to the MPPI algorithm described in Section 4.2.3, and select the best-performing one according to the rollouts of our learned model. We then optimise it via the shooting method using our model’s gradients. We also use online system identification to further improve the model’s performance. Figures 10 and 11 show qualitative and quantitative results, respectively.



Fig. 10. Qualitative results on control. (a) FluidShake — Manipulating a box of fluids to match a target shape. The Result and Target indicate the fluid shape when viewed from the cutaway view. (b) RiceGrip — Gripping a deformable object and moulding it to a target shape. (c) RiceGrip in Real World — Generalise the learned dynamics and the control algorithms to the real world by doing online adaptation. The last two columns indicate the final shape viewed from the top.

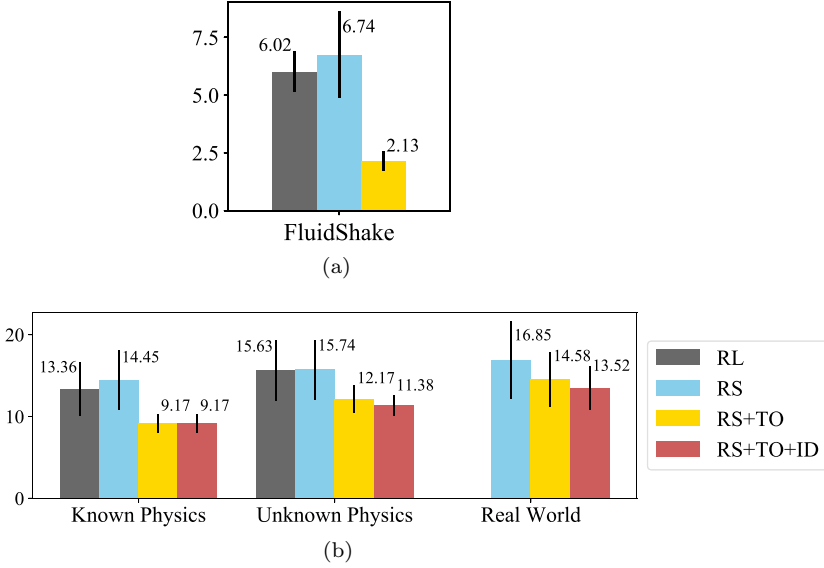


Fig. 11. Quantitative results on control. We show the results on control (as evaluated by the Chamfer distance ($\times 10^{-2}$) between the manipulated result and the target) for (a) FluidShake and (b) RiceGrip by comparing with four baselines. **RL**: model-free deep reinforcement learning optimised with PPO; **RS**: random search the actions from the learned model and select the best one to execute; **RS + TO**: trajectory optimisation augmented with model predictive control; **RS + TO + ID**: online system identification by estimating uncertain physical parameters during run time.

FluidShake: We aim to control the speed of the box to match the fluid particles to a target configuration. We compare our method (RS+TO) with random search (RS) using the learned model and Model-free Deep Reinforcement Learning (Actor-Critic method optimised with PPO,⁷⁴ RL). Figure 11(a) suggests that our model-based control algorithm outperforms both baselines by a large margin. Also, RL is not sample-efficient, requiring more than 10 million time steps to converge, while ours requires 600K time steps.

RiceGrip: We aim to select a sequence of gripping configurations (position, orientation, and closing distance) to mould the “sticky rice” to a target shape. We also consider cases where the stiffness of the rice is unknown and need to be identified. Figure 11(b) shows that our dynamic particle interaction network with system identification

performs the best and is much more efficient than RL (150K vs. 10M time steps).

RiceGrip in the real world: We generalise the learned model and control algorithm for RiceGrip to the real world. We first reconstruct object geometry using a depth camera mounted on our Kuka robot using TSDF volumetric fusion.⁷ We then randomly sampled N_{fill} particles within the object mesh as the initial configuration for manipulation. Figures 10(c) and 11(b) show that using DPI-Nets, the robot successfully adapts to the real-world environment of unknown physical parameters and manipulates a deformable foam into various target shapes. The learned policy in RiceGrip does not generalise to the real world due to domain discrepancy and outputs invalid gripping configurations.

5.4. Discussion

We have demonstrated that a learned particle dynamics model can approximate the interaction of diverse objects and can help solve complex manipulation tasks of deformable objects with high degrees of freedom. Our system requires standard open-source robotics and deep learning toolkits and can be potentially deployed in household and manufacturing environments. Robot learning of dynamic scenes with particle-based representations shows profound potential due to the generalisability and expressiveness of the representation. Our study helps lay the foundation for it.

6. Neural Field Representation

Now, we proceed to consider the modelling and manipulation of dynamical systems with extremely high degrees of freedom (DoF) using only visual observations, such as manipulating a container of liquid with ice cubes floating within it (i.e., fluid-body interactions). In such a scenario, accurately estimating the full state information of the particle set becomes challenging when the sole inputs are RGB images. Moreover, the usage of keypoints (typically a sparse set of points attached to semantically meaningful parts of an object) in this context is uncertain since the fluid, possessing an extremely high DoF, continuously alters its shape during interactions.

This section addresses this challenge, intending to learn models for highly dynamic 3D scenes solely from 2D visual observations. Our model merges Neural Radiance Fields (NeRF) and time contrastive learning with an autoencoding framework to learn viewpoint-invariant, 3D-aware scene representations (z_t in Equation (4)). We demonstrate that a dynamics model, built over the learned representation space, facilitates visuomotor control for complex manipulation tasks involving both rigid bodies and fluids. Here, the target can be specified from a viewpoint different from that the robot operates from. Furthermore, when combined with an auto-decoding framework, it can even support goal specification from camera viewpoints that are *outside the training distribution*. We further showcase the complexity of the learned 3D dynamics model by performing future predictions and novel view synthesis. Lastly, we provide comprehensive ablation studies related to various system designs and qualitative analysis of the learned representations. Video illustrations can be found on the project page: <https://3d-representation-learning.github.io/nerf-dy/>.

This section was originally published as Li *et al.*⁴¹ at the Conference on Robot Learning (CoRL) 2021 as an **Oral Presentation**. Significant contributions were made by Yunzhu Li and Shuang Li, along with co-authors Vincent Sitzmann, Pulkit Agrawal, and Antonio Torralba.

6.1. Overview

As discussed in the previous sections, the current model-based systems operating from vision usually treat images as 2D grids of pixels.^{12,25,73} However, our world is three dimensional. Modelling the environment in 3D enables amodal completion and allows agents to operate from various views. Therefore, acquiring robust 3D-aware representations of the environment from 2D observations is desirable to enhance task performance when precise 3D information inference is vital. This can further simplify task specification, learning from third-person videos, etc.

A central question in model learning for robotic manipulation is how to establish the state representation for learning the dynamics model. The ideal representation should readily capture the environmental dynamics, show a strong 3D understanding of the objects in

the scene, and be applicable to a wide range of object sets including rigid or deformable objects and fluids. As exemplified in Section 4, one approach has focused solely on predicting task-relevant features identified as keypoints.^{23,43,52,53,91} Such models exhibit good performance in terms of category-level generalisation, i.e., the same set of keypoints can represent different instances within the same category. However, they are not adequate for modelling objects with significant variations, such as fluids and granular materials. Section 5 addressed the use of particles. However, estimating the full-state information of an environment containing multiple objects of diverse materials can be challenging, especially when only visual observations are accessible. Issues like self-occlusion, 3D instance segmentation, and material estimation can all pose difficulties for our perception module.

Some methods, instead of estimating the state of the environment, learn dynamics in the latent space.^{1,25,26,73,93} However, the majority of these methods learn dynamics models using 2D convolutional neural networks and reconstruction loss, which is a similar issue to predicting dynamics in the image space, i.e., their learned representations lack equivariance to 3D transformations. Time contrastive networks⁷⁵ aim to learn viewpoint-invariant representations from multi-view inputs but do not necessitate detailed modelling of 3D contents. As a result, previously unseen scene configurations and camera poses are out-of-distribution samples for the state estimator. As we see, this leads to wrong state estimates and results in faulty control trajectories.

Meanwhile, recent advancements in computer vision have led to impressive progress in learning 3D-structured neural field representations. These methodologies allow inference of 3D structure and appearance, trained solely with 2D observations, either by overfitting a single scene^{48,54,78} or by generalising across scenes.^{79,89,90} Through their 3D inductive bias, the scene representations inferred by these models encode the scene contents more accurately and are invariant to changes in camera perspectives. It is desirable to extend these ideas further to gain a deeper understanding of how these methods, which directly reason in 3D, can introduce new characteristics and be beneficial for dynamics modelling and complicated embodied AI tasks.

In this section, we aim to leverage recently proposed 3D-structure-aware implicit neural scene representations for visuomotor control

tasks. Therefore, we propose embedding neural radiance fields⁵⁴ into an autoencoder framework, enabling tractable inference of the 3D-structure-aware scene state for dynamic environments. By also enforcing a time contrastive loss on the estimated states, we ensure that the learned state representations are viewpoint-invariant. We then train a dynamics model that predicts the evolution of the state space conditioned on the input action, enabling control in the learned state space.

Although the representation itself is grounded in the 3D implicit field, the convolutional encoder is not. At test time, we overcome this limitation by performing inference-via-optimisation,^{64,79} enabling accurate state estimation even for out-of-distribution camera poses, and hence, control of tasks where the goal view is specified in an entirely unseen camera perspective. These contributions enable us to perform model-based visuomotor control of complex scenes, modelling 3D dynamics of both rigid objects and fluids. Through comparison with various baselines, our model’s learned representation is more precise in describing the contents of 3D scenes, allowing it to accomplish control tasks involving rigid objects and fluids with significantly enhanced accuracy (Figure 12).

We summarise the contributions as follows: (i) We extend an autoencoding framework with a neural radiance field rendering module and time contrastive learning, enabling the learning of 3D-aware scene representations for dynamics modelling and control purely

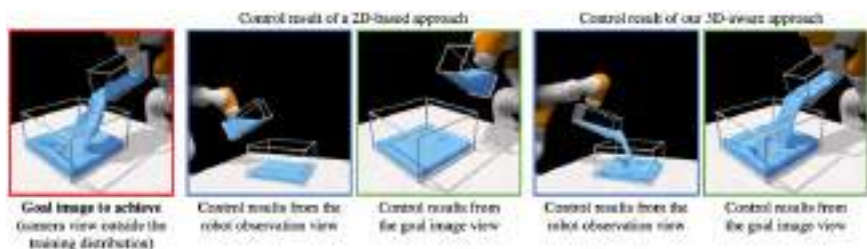


Fig. 12. Comparison of the control results between a 2D-based baseline and our 3D-aware approach. The task here is to achieve the configuration shown on the left, observed from a viewpoint that is outside the training distribution. The agent only takes a single-view visual observation as input (images with blue frames) from a viewpoint that is vastly different from the goal. Our method generalises well in this scenario and outperforms the 2D-based baseline, demonstrating the benefits of the learned 3D-aware scene representations.

from visual observations. (ii) By incorporating the autoencoder mechanism at test time, our framework can modify the learned representation and accomplish control tasks with the goal specified from camera viewpoints outside the training distribution. (iii) We are the first to augment neural radiance fields with a time-invariant dynamics model, supporting future prediction and novel view synthesis across a wide range of environments with different object types.

6.2. 3D Neural Fields for Dynamics Modelling

Inspired by Neural Radiance Fields (NeRF),⁵⁴ we propose a framework that learns a viewpoint-invariant model for dynamic environments. As shown in Figure 13, our framework has three parts: (1) an encoder that maps the input images into a latent state representation, (2) a decoder that generates an observation image under a certain viewpoint based on the state representation, and (3) a dynamics model that predicts the future state representations based on the current state and the input action.

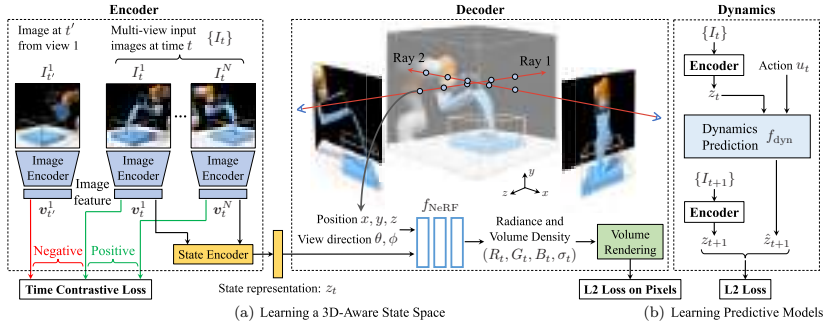


Fig. 13. Overview of the training procedure. Left: An encoder that maps the input images into a latent scene representation. The images are first sent to an image encoder to generate the image feature representations $\mathbf{v}$. Then we combine the image features from the same time step using a state encoder to obtain the state representation $\mathbf{z}_t$. A time contrastive loss is applied to enable our model to be invariant to camera viewpoints. Middle: A decoder that takes the scene representation as input and generates the visual observation conditioned on a given viewpoint. We use an L2 loss to ensure the reconstructed image to be similar to the ground-truth image. Right: A dynamics model that predicts the future scene representations $\hat{\mathbf{z}}_{t+1}$ by taking in the current representation $\mathbf{z}_t$ and action u_t . We use an L2 loss to enforce the predicted latent representation to be similar to the scene representation $\mathbf{z}_{t+1}$ extracted from the true visual observation I_{t+1} .

6.2.1. 3D-aware scene representation learning

Neural radiance field: Given a 3D point $\mathbf{x} \in \mathbb{R}^3$ in a scene and a viewing direction unit vector $\mathbf{d} \in \mathbb{R}^3$ from a camera, NeRF learns to predict a volumetric radiance field. This is represented using a differentiable rendering function f_{NeRF} that predicts the corresponding density σ and RGB colour $\mathbf{c}$ using $f_{\text{NeRF}}(\mathbf{x}, \mathbf{d}) = (\sigma, \mathbf{c})$. To render the colour of an image pixel, NeRF integrates the information along the camera ray using $\hat{\mathbf{C}}(\mathbf{r}) = \int_{h_{\text{near}}}^{h_{\text{far}}} T(h) \sigma(h) \mathbf{c}(h) dh$, where $\mathbf{r}(h) = \mathbf{o} + h\mathbf{d}$ is the camera ray with its origin $\mathbf{o} \in \mathbb{R}^3$ and unit direction vector $\mathbf{d} \in \mathbb{R}^3$, and $T(h) = \exp(-\int_{h_{\text{near}}}^h \sigma(s) ds)$ is the accumulated transparency between the pre-defined near depth h_{near} and far depth h_{far} along that camera ray. The mean squared error between the reconstructed colour $\hat{\mathbf{C}}$ and the ground truth $\mathbf{C}$ is

$$\mathcal{L}_{\text{rec}} = \sum_{\mathbf{r}} \|\hat{\mathbf{C}}(\mathbf{r}) - \mathbf{C}(\mathbf{r})\|_2^2. \quad (29)$$

Neural radiance field for dynamic scenes: One key limitation of NeRF is that it assumes the scene is static. For a dynamic scene, it must learn a separate radiance field f_{NeRF} for each time step. This severely limits the ability of NeRF to be used in planning and control, as it is unable to handle dynamic scenes with different initial configurations or input action sequences. While other models have shown generalisation across scenes,^{60,79} it's unclear how they can be used in visuomotor control. To enable f_{NeRF} to model dynamic scenes, we learn an encoding function f_{enc} that maps the visual observations to a feature representation z for each time step and learn the volumetric radiance field decoding function based on z . Let $\{I_t\}$ denote the set of 2D images that capture a 3D scene at time t from one or more camera viewpoints. The image taken from the i th viewpoint is represented as I_t^i . We use ResNet-18²⁸ to extract a feature vector for each image. We take the output of ResNet-18 before the pooling layer and send it to a fully connected layer, resulting in a 256 dimension image feature $\mathbf{v}_t^i$. This image feature is concatenated with the corresponding camera viewpoint information (a 16-D vector obtained by flattening the camera view matrix) and processed using a small multi-layer perceptron (MLP) to generate the final image feature. The scene representation z_t at time t is generated by first averaging the image features across multiple camera viewpoints and then being

encoded using another small MLP and normalised to have a unit L2 norm.

Given a 3D point $\mathbf{x}$, a viewing direction unit vector $\mathbf{d}$, and a scene representation z_t , we learn a function $f_{\text{dec}}(\mathbf{x}, \mathbf{d}, z_t) = (\sigma_t, \mathbf{c}_t)$ to predict the radiance field represented by the density σ_t and RGB colour $\mathbf{c}_t$. Similar to NeRF, we use the integrated information along the camera ray to render the colour of image pixels from an input viewpoint and then compute the image reconstruction loss using Equation (29). During each training iteration, we render two images from different viewpoints to calculate more accurate gradient updates. f_{dec} depends on the scene representation z_t , forcing it to encode the 3D contents of the scene to support rendering from different camera poses.

Time contrastive learning: To enable the image encoder to be viewpoint invariant, we regularise the feature representation of each image $\mathbf{v}_t^i$ using multi-view time contrastive loss (TCN)⁷⁵ (see Figure 13(a)). The TCN loss encourages features of images from different viewpoints at the same time step to be similar, while repulsing features of images from different time steps to be dissimilar. More specifically, given a time step t , we randomly select one image I_t^i as the anchor and extract its image feature $\mathbf{v}_t^i$ using the image encoder. Then we randomly select one positive image from the same time step but different camera viewpoint $I_t^{i'}$ and one negative image from a different time step but the same viewpoint $I_{t'}^i$. We use the same image encoder to extract their image features $\mathbf{v}_t^{i'}$ and $\mathbf{v}_{t'}^i$. Similar to Sermanet *et al.*,⁷⁵ we minimise the following time contrastive loss:

$$\mathcal{L}_{\text{tc}} = \max(\|\mathbf{v}_t^i - \mathbf{v}_t^{i'}\|_2^2 - \|\mathbf{v}_t^i - \mathbf{v}_{t'}^i\|_2^2 + \alpha, 0), \quad (30)$$

where α is a hyperparameter denoting the *margin* between the positive and negative pairs.

6.2.2. Learning the predictive model

After we have obtained the latent state representation z , we use supervised learning to estimate the forward dynamics model, $\hat{z}_{t+1} = f_{\text{dyn}}(z_t, u_t)$. Given z_t and a sequence of actions $\{u_t, u_{t+1}, \dots\}$, we predict H steps in the future by iteratively feeding in actions into the one-step forward model. We implement f_{dyn} as an MLP network

which is trained by optimising the following loss function:

$$\mathcal{L}_{\text{dyn}} = \sum_{h=1}^H \|\hat{z}_{t+h} - z_{t+h}\|_2^2, \quad \text{where } \hat{z}_{t+h} = f_{\text{dyn}}(\hat{z}_{t+h-1}, u_{t+h-1}),$$

$$\hat{z}_t = z_t. \quad (31)$$

We define the final loss as a combination of the image reconstruction loss, the time contrastive loss, and the dynamics prediction loss: $\mathcal{L} = \mathcal{L}_{\text{rec}} + \mathcal{L}_{\text{tc}} + \mathcal{L}_{\text{dyn}}$. We first train the encoder f_{enc} and decoder f_{dec} together using stochastic gradient descent (SGD) by minimising $\mathcal{L}_{\text{rec}}$ and $\mathcal{L}_{\text{tc}}$, which makes sure that the learned scene representation z encodes the 3D contents and is viewpoint-invariant. We then fix the encoder parameters and train the dynamics model f_{dyn} by minimising $\mathcal{L}_{\text{dyn}}$ using SGD.

6.3. Visuomotor Control

6.3.1. Online planning for closed-loop control

When given the goal image I^{goal} and its associated camera pose, we can feed them through the encoder f_{enc} to get the state representation for the goal configuration z^{goal} . We use the same method to compute the state representation for the current scene configuration z_1 . The goal of the online planning problem is to find an action sequence $u_1, \dots, u_{T-1}$ that minimises the distance between the predicted future representation and the goal representation at time T . As shown in Figure 14(a), given a sequence of actions, our model can iteratively predict a sequence of latent state representations. The latent-space dynamics model can then be used for downstream closed-loop control tasks via online planning with model-predictive control (MPC). We formally define the online planning problem as follows:

$$\min_{u_1, \dots, u_{T-1}} \|z^{\text{goal}} - \hat{z}_T\|_2^2, \quad \text{s.t.} \quad \hat{z}_1 = z_1, \hat{z}_{t+1} = f_{\text{dyn}}(\hat{z}_t, u_t). \quad (32)$$

Many existing off-the-shelf model-based RL methods can be used to solve the MPC problem.^{13,17,26,40,45,53,57} Similar to what we have discussed in Section 4.2.3, we experimented with random shooting, gradient-based trajectory optimisation, cross-entropy method, and

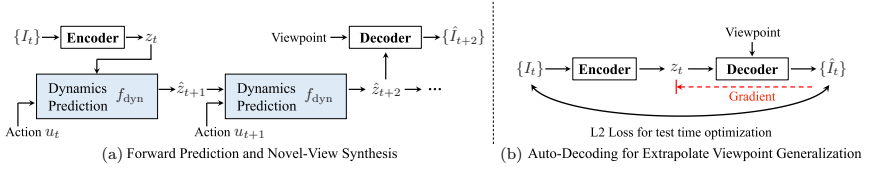


Fig. 14. Forward prediction and viewpoint extrapolation. (a) We first feed the input image(s) at time t to the encoder to derive the scene representation z_t . The dynamics model then takes z_t and the corresponding action sequence as input to iteratively predict the future. The decoder synthesises the visual observation conditioned on the predicted state representation and an input viewpoint. (b) We propose an autodecoding inference-via-optimisation framework to enable extrapolated viewpoint generalisation. Given an input image I_t taken from a viewpoint outside the training distribution, the encoder first predicts the scene representation z_t . Then the decoder reconstructs the observation $\hat{I}_t$ from z_t and the camera viewpoint from I_t . We calculate the L2 distance between I_t and $\hat{I}_t$ and back-propagate the gradient through the decoder to update z_t . The updating process is repeated for K iterations, resulting in a more accurate z_t of the underlying 3D scene.

model-predictive path integral (MPPI) planners⁹⁴ and found that MPPI performed the best. In our experiments, we specify the action space as the position and orientation of the arm’s end-effector. Then, the joint angle of the arm is calculated via inverse kinematics.

6.3.2. Autodecoder for viewpoint extrapolation

End-to-end visuomotor agents can undergo significant performance drop when the test-time visual observations are captured from camera poses outside the training distribution. The convolutional image encoder suffers from the same problem as it is not equivariant to changes in the camera pose, meaning it has a hard time generalising to out-of-distribution camera views. As shown in Figure 14(b), when encountered an image from a viewpoint outside training distribution, with a pixel distribution vastly different from what the model is trained on, passing it through the encoder f_{enc} will give us an amortised estimation of the scene representation z_t . It has a high chance that the decoded image is different from the ground truth as the viewpoint has never been encountered during training.

We fix this problem at test time by applying the inference-by-optimisation (also named as an autodecoding) framework that back-propagates through the volumetric renderer and the neural implicit

representation into the state estimate.^{64,79} This is inspired by the fact that the rendering function, $f_{\text{dec}}(\mathbf{x}, \mathbf{d}, z_t) = (\sigma_t, \mathbf{c}_t)$, is viewpoint equivariant, where the output only depends on the state representation z_t , the location $\mathbf{x}$, and the ray direction $\mathbf{d}$, meaning that the output is invariant to the camera position along the camera ray, i.e., even if we move the camera closer or farther away along the camera ray, f_{dec} still tends to generate the same results. We leverage this property and calculate the L2 distance between the input image and the reconstructed image $\mathcal{L}_{\text{ad}} = \|I_t - \hat{I}_t\|_2^2$ and then update the scene representation z_t using stochastic gradient descent. We repeat this updating process K times to derive the state representation of the underlying 3D scene. Note that this update only changes the scene representation while keeping the parameters in the decoder fixed. The resulting representation is used as z^{goal} in Equation (32) to solve the online planning problem.

6.4. Experiments

Environments: We consider the following four environments involving both fluid and rigid objects to evaluate the proposed model and baseline approaches. The environments are simulated using NVIDIA Flex.⁵¹ (1) FluidPour (Figure 18(a)): This environment contains a fully actuated cup that pours fluids into a container at the bottom. (2) FluidShake (Figure 18(b)): A fully actuated container moves on a 2D plane. Inside the container are fluids and a rigid cube floating on the surface. (3) RigidStack (Figure 18(c)): Three rigid cubes form a vertical stack and are released from a certain height but in different horizontal positions. They fall down and collide with each other and the ground. (4) RigidDrop (Figure 18(d)): A cube falls down from a certain height. There is a container fixed at a random position on the ground. The cube either falls into the container or bounces out. We use 20 camera views for all environments and generated 1,000 trajectories of 300 time steps for both FluidPour and FluidShake as an offline dataset to train the model, 800 trajectories of 80 time steps for RigidStack, and 1,000 trajectories of 50 time steps for RigidDrop.

Evaluation metrics: We use the first two environments, i.e., FluidPour and FluidShake, to measure the control performance, where we specify the target configuration of the control task using images from

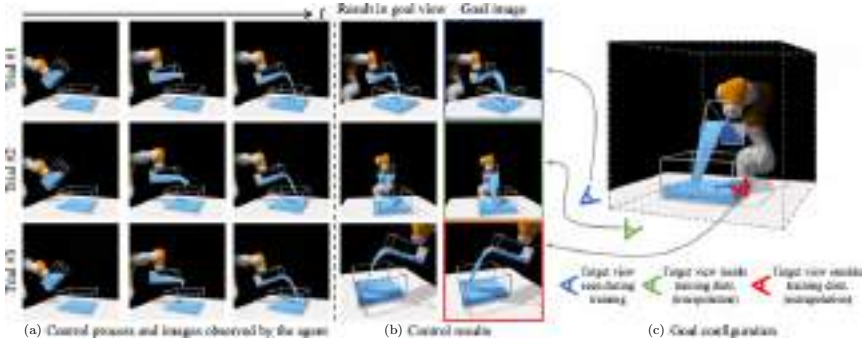


Fig. 15. Qualitative control results of our method on three types of testing scenarios. The image on the right shows the target configuration we aim to achieve. The left three columns show the control processes, which are also the input images to the agent. The fourth column is the control results from the same viewpoint as the goal image. Trial #1 specifies the goal using a different viewpoint from the agent’s but has been encountered during training. Trial #2 uses a goal view that is an interpolation of training viewpoints. Trial #3 uses an extrapolated viewpoint that is outside the training distribution. Our method performs well in all settings.

(1) one of the viewpoints encountered during training, (2) an interpolated viewpoint between training viewpoints, and (3) an extrapolated viewpoint outside the training distribution (Figure 15).

We provide quantitative evaluations on the control performance in FluidPour and FluidShake by extracting the particle set from the simulator and measuring the Chamfer distance between the result and the goal, which we denote as “Chamfer Dist.”. In FluidPour, we provide additional measurements on the L2 distance of the position/orientation of the cup towards the goal, denoting as “Position Error” and “Angle Error”, respectively. In FluidShake, we calculate the L2 distance of the container and cube’s position towards the goal and denote them as “Container Error” and “Cube Error”, respectively.

6.4.1. Baseline methods

For comparison, we consider the following three baselines — **TC**: Similar to TCN,⁷⁵ it only uses time contrastive loss for learning the image feature without having to reconstruct the scene. We learn a dynamics model directly on the image features for control. **TC+AE**:

Instead of using neural radiance fields to reconstruct the image, this method uses the standard convolutional decoder to reconstruct the target image when given a new viewpoint. This would then be similar to GQN¹⁵ augmented with a time contrastive loss. **NeRF**: This method is a direct adaptation from the original NeRF paper⁵⁴ and is the same as ours except that it does not include the time contrastive loss during training and the autodecoding test-time optimisation. We use the same dynamics model shown in Figure 13(b) and train the model for each baseline respectively for dynamic prediction. We use the same feedback control method, i.e., MPPI,⁹⁴ for our model and all the baselines.

6.4.2. Control results

Goal specification from novel viewpoints: Figure 15(c) shows the goal configuration, and we ask the learned model to perform three control trials where the goal is specified from different types of viewpoints. The left three columns show the MPC control process from the agent’s viewpoint. The fourth column visualises the final configuration the agent achieves from the same viewpoint as the goal image. Trial #1 specifies the goal using a different viewpoint from the agent’s but has been encountered during training. Trial #2 uses a goal view that is an interpolation of training viewpoints. Our agent can achieve the target configuration with decent accuracy. For trial #3, we specify the goal view by moving the camera closer, higher, and more downwards with respect to the container. Note this goal image view is outside the distribution of training viewpoints. With the help of test-time autodecoding optimisation introduced in Section 6.3.2, our method can successfully achieve the target configuration, as shown in the figure.

Baseline comparisons: We benchmark our model with the baselines by assessing their performance on the downstream control tasks. Figure 16 shows the qualitative comparison between our model (Ours), a variant of our model that does not perform the autodecoding test-time optimisation (Ours w/o AD), and the best-performing baseline (TC+AE) introduced in Section 6.4.1. We find that when the target view is outside the training distribution and vastly different from the agent view, our full method shows a much better

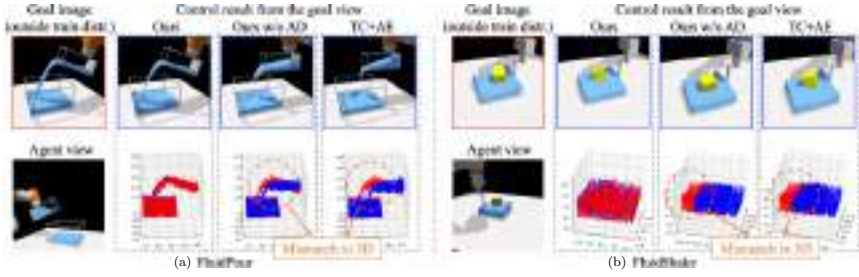


Fig. 16. Qualitative comparisons between our method and baseline approaches on control tasks. We show the closed-loop control results on the FluidPour and FluidShake environments. The goal image viewpoint (top-left image of each block) is outside the training distribution and is different from the viewpoint observed by the agent (bottom-left image of each block). Our final control results are much better than a variant that does not perform autodecoding test-time optimisation (Ours w/o AD) and the best-performing baseline (TC+AE), both of which fail to accomplish the task and their control results (blue points) exhibit an apparent deviation from the target configuration (red points) when measured in the 3D points space of the fluids and floating cube.

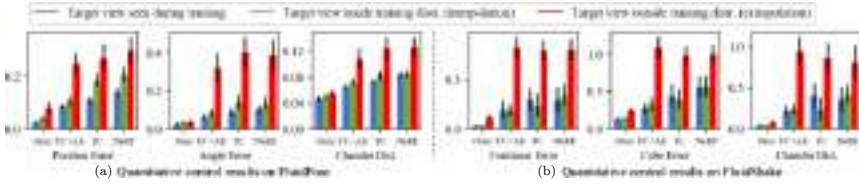


Fig. 17. Quantitative comparisons between our method and baselines on the FluidPour and FluidShake. In each environment, we compare the results using three different evaluation metrics under three settings, i.e., (1) the target image view seen during training, (2) the target image view is inside the training distribution but not seen during training (interpolation), and (3) the target image view is outside the training distribution (extrapolation). The height of the bars indicates the mean, and the error bar denotes the standard error. Our model significantly outperforms all baselines under all testing settings.

performance in achieving the target configuration. The variant without autodecoding optimisation and TC+AE fail to accomplish the task and exhibit an apparent deviation from the ground truth in the 3D points space of the fluids and floating cube. We also provide quantitative comparisons on the control results. Figure 17 shows the performance in the FluidPour and FluidShake environments. We find our full model significantly outperforms the baseline approaches in both

environments under all scenarios and evaluation metrics. The results effectively demonstrate the advantages of the learned 3D-aware scene representations, which contain a more precise encoding of the contents in the 3D environments and are capable of extrapolation view-point generalisation.

6.4.3. *Dynamic prediction and novel view synthesis*

Conditioned on a scene representation and an input action sequence, our dynamics model f_{dyn} can iteratively predict the evolution of the scene representations. Our decoder can then take the predicted state representation and reconstruct the corresponding visual observation from a query viewpoint. Figure 18 shows that our model can accurately predict the future and perform novel view synthesis on four environments involving both fluid and rigid objects, suggesting its usefulness in trajectory optimisation.

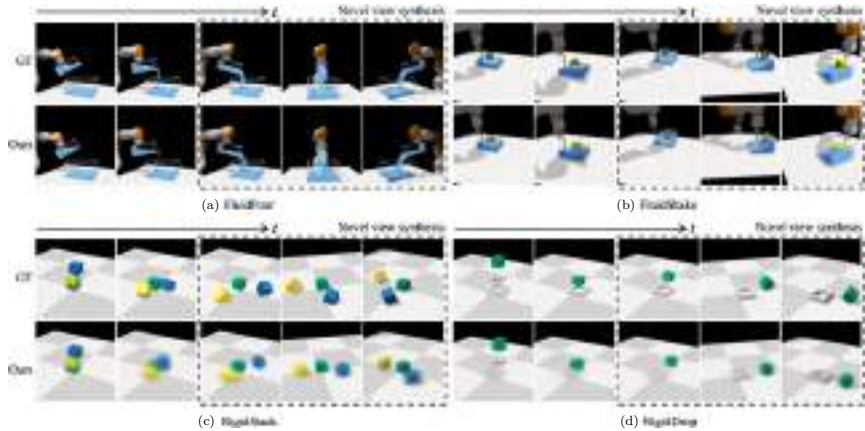


Fig. 18. Forward prediction and novel view synthesis on four environments. Given a scene representation and an input action sequence, our dynamics model predicts the subsequent latent scene representation, which is used as the input of our decoder model to reconstruct the corresponding visual observation based on different viewpoints. In each block, we render images based on the open-loop future dynamic prediction from left to right. Images in the dotted box compare our model’s novel view synthesis results in the last time step with the ground truth from three different viewpoints.

6.4.4. Autodecoder test-time optimisation for viewpoint extrapolation

Figure 19 shows the qualitative results on autodecoding test-time optimisation. When encountering an image from a viewpoint outside the training distribution, this mechanism can help us derive a better representation of the scene that holds a more accurate description of

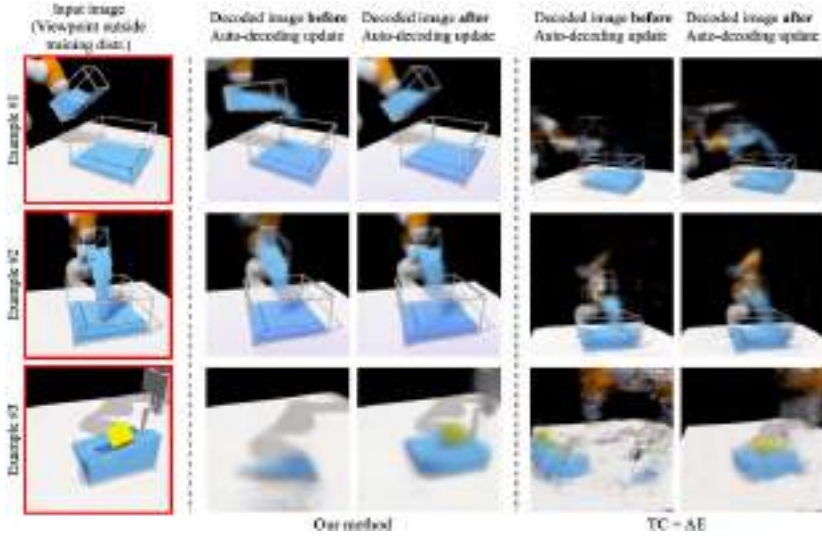


Fig. 19. Qualitative results on autodecoding test-time optimisation. Following from the pipeline illustrated in Figure 14(b), if the input image I_t is outside the training distribution as shown on the left column, the encoder won't be able to generate the most accurate state representation. When passing the predicted state embedding z_t and the same viewpoint as the input to the decoder, the generated image does not match the underlying scene as shown in the second column. We then calculate the L2 distance of the pixels between the generated image and true observation, backpropagating the gradient until the state representation and making updates to z_t using SGD. As discussed in Section 6.3.2, the translational equivariance nature of the decoder allows it to effectively optimise the latent representation to make it better reflect the 3D contents in the scene. After the optimisation, the generated visual observation is much closer to the ground truth, as shown in the third column. On the contrary, the vanilla autoencoder that uses a CNN-based decoder won't be able to capture the underlying scene even with test-time autodecoding optimisation, as shown on the right.

the 3D contents. The obtained representation after the optimisation can then be used as the goal embedding z^{goal} in Equation (32) that the agent needs to achieve.

6.4.5. *Validation of the dynamics prediction on real-world data*

We further evaluate our model’s dynamics prediction ability by conducting experiments on real-world data. As shown in Figure 20, we use four D415 RGBD cameras to record a human subject pouring water from one cup to another. The cameras are calibrated and synchronised with the frequency of 15 Hz. We recorded 50 pouring episodes of length 15 seconds, resulting in a dataset of 43,400 frames. We use the first 45 episodes for training and the remaining for testing.

To obtain the action, i.e., the movement of the cup that pours water out, we attached an AprilTag⁶² on the cup to obtain the six DoF pose of the cup at each time step. Our model can make open-loop future predictions on the testing trajectories in the representation space, i.e., given a latent scene representation of the current time step, the subsequent action sequence, our dynamic model can accurately predict the evolution of the latent scene representation

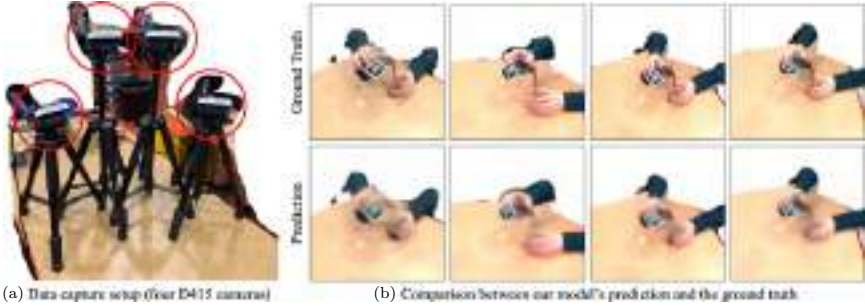


Fig. 20. Dynamics prediction using real-world data. (a) We build a data recording setup containing four D415 RGBD cameras to record a human subject pouring water from one cup to another and then use the recorded data to evaluate our model’s dynamics prediction ability. (b) We show a side-by-side comparison between the ground-truth data and our model’s prediction when predicting the future from the four camera views. Our model correctly identifies when the fluids pour out and start to fill the bottom container. Please see our supplementary video for better visualisations.

and render the corresponding frames that closely resemble the ground truth.

6.4.6. *Comparison with PID that only matches the robot’s state*

To show the necessity of modelling the fluid dynamics in our task, we include an additional baseline that uses PID control²¹ to reach the robot state in the target image without worrying about the fluid. Note that this baseline uses additional information, including the ground-truth state of the robot in both the current and the target image.

When the goal is to leave a small amount of fluid in the container at the end of the control episode, naively matching the container’s position and orientation will not work, as shown in the PID baseline that the top container did not pour any fluid into the bottom container. In contrast, our model first learns to pour a certain amount of fluid out and then tilt the container back to match the target configuration. We further use the Chamfer distance to measure the models’ performance in matching the fluid shape in the 3D point space, where our method significantly outperforms the PID baseline (Ours: 0.048 vs. PID: 0.086).

6.4.7. *Nearest neighbour search using the learned representations*

When the goal image is specified from a viewpoint different from the agent’s view, to ensure the planning problem defined in Equation (32) still work, it is essential that the distance in the learned feature space reflects the distance in the actual 3D space, i.e., scenes that are more similar in the real 3D space should be closer in the learned feature space, even if the visual observations are captured from different viewpoints. We visualise the nearest neighbour results in Figure 21. Given a query image, we search its nearest neighbours based on their state representation z (introduced in Section 6.2.1). Even if the images look quite different from each other when measured in pixel difference, the learned 3D-aware scene representations can retrieve reasonable neighbourhood images that share similar 3D contents, indicating that the learned 3D-aware scene representations



Fig. 21. Nearest neighbour (NN) results using our learned state representation. Given a query image (red boundary), we search its nearest neighbours based on their state representation. Our learned scene representations can retrieve reasonable neighbour images, indicating that our state representations retain a good estimation of the contents inside the 3D scene and are invariant to camera poses.

Table 4. Quantitative results on nearest neighbour (NN) search from different viewpoints. We calculate the accuracy of finding NN using the L1 distance between the ground truth and retrieved time indexes. Our method measures the distance in the learned representation space, which delivers the best performance, and the retrieved frame is, on average, around two time steps away from the ground truth.

Env	Ours	NN in pixel space	Random
FluidPour	1.772718	147.971993	99.903261
FluidShake	2.584928	132.467998	99.646617

hold a good understanding of the real 3D scene and are invariant to viewpoint variations.

We conducted additional experiments to quantitatively measure the accuracy in finding the nearest neighbours (NN) across viewpoints of our proposed method. The results are shown in Table 4. Specifically, for each query frame, the model is asked to find the closest frame from a randomly selected viewpoint from a trajectory with 300 frames. We measure the accuracy using the L1 distance between the time indexes. Randomly selecting the closest frame leads to an averaged distance of 100 times steps between the selected frame and the ground-truth frame. Selecting based on the pixel difference is even worse. Our method can accurately find the nearest neighbour. The average time step difference between the selected frame and the ground-truth frame is 1.77 in FluidPour and 2.58 in FluidShake.

6.5. Discussion

In this section, we proposed to learn viewpoint-invariant 3D-aware scene representations from visual observations using an autoencoding framework augmented with a neural radiance field rendering module and time contrastive learning. We show that the learned 3D representations perform well on the model-based visuomotor control tasks. When coupled with an autodecoding test-time optimisation mechanism, our method allows goal specification from a viewpoint outside the training distribution. We demonstrate the applicability of the proposed framework in a range of complicated physics environments involving rigid objects and fluids, which we hope can facilitate future works on visuomotor control for complex and dynamic 3D manipulation tasks.

Similar to many other data-driven methods, our model can deliver reasonable performance in regions well supported by the data. However, for cases that our model has never seen before, e.g., more containers than during training, we wouldn't expect our model to generalise. Potential solutions may include (1) adding the examples and increasing the diversity of the training set or (2) using a more structured/compositional representation space instead of a single vector as in our model, which we leave for future works.

7. Discussion and Future Work

In the future, embodied agents should possess the ability to operate in unstructured environments and carry out complex physical interactions as deftly and effectively as humans do. Over recent years, there have been significant strides in enhancing the capabilities of robots, yet their performance remains far from ideal for practical real-world deployments. Substantial performance gaps persist in nearly every aspect of the system between robots and humans, extending from perception and dynamics modelling to planning and control. Among these aspects, an appropriate 3D representation of the dynamic environment plays a crucial role and dictates the structure of the overall system.

The most suitable representation of the 3D world, however, may need to be dynamic, varying based on the objects involved, the tasks at hand, or even different stages of a single task. Despite significant progresses in recent years, the automatic selection and adaptation of structured scene representations from high-dimensional observational data remain unsolved challenges, especially when dealing with complex objects, such as sushi, salad, or cloth. These objects engage in intricate physical interactions and often experience severe occlusions. Humans, however, maintain a mental model of these objects that, although imperfect, significantly outperforms state-of-the-art computational models. For instance, when buttoning a shirt, we do not need complete knowledge of the cloth’s state but concentrate on local regions. Similarly, when grabbing a mug, our mental representations operate at varying levels of abstraction throughout the task. Replicating or even exceeding this capacity is not a trivial endeavour. It demands insights from cognitive science and neuroscience, and the development of new learning paradigms that incorporate structural priors into the optimisation procedure. This allows for automatic and adaptive representation selection for effective dynamics modelling — a problem we have coined as *online model order reduction*. This challenging issue is captivating and holds immense potential for broadening robotic capabilities.

Progress in representation learning for dynamic 3D scenes is vital for advancements in embodied AI. This progress demands interdisciplinary collaboration and systematic design across various research domains, including robotics, computer vision, machine learning, and control, among others. The methodologies and techniques developed during this process could enhance our understanding and widen the horizons of the respective subfields while also suggesting novel research topics.

References

1. P. Agrawal, A. V. Nair, P. Abbeel, J. Malik, and S. Levine. Learning to poke by poking: Experiential learning of intuitive physics. In *Advances in Neural Information Processing Systems*, pp. 5074–5082 (2016).

2. P. W. Battaglia, R. Pascanu, M. Lai, D. Rezende, and K. Kavukcuoglu. Interaction networks for learning about objects, relations and physics. In *NIPS* (2016).
3. M. G. Bellemare, Y. Naddaf, J. Veness, and M. Bowling. The arcade learning environment: An evaluation platform for general agents, *Journal of Artificial Intelligence Research* **47**, 253–279 (2013).
4. J. Bradbury, S. Merity, C. Xiong, and R. Socher. Quasi-recurrent neural networks. In *ICLR* (2017).
5. E. F. Camacho and C. B. Alba. *Model Predictive Control*. Springer Science & Business Media. Springer, London (2013).
6. M. B. Chang, T. Ullman, A. Torralba, and J. B. Tenenbaum. A compositional object-based approach to learning physical dynamics. In *ICLR* (2017).
7. B. Curless and M. Levoy. A volumetric method for building complex models from range images. In *SIGGRAPH* (1996).
8. F. de Avila Belbute-Peres, K. A. Smith, K. Allen, J. B. Tenenbaum, and J. Z. Kolter. End-to-end differentiable physics for learning and control. In *Neural Information Processing Systems* (2018).
9. J. Degraeve, M. Hermans, J. Dambre *et al.* A differentiable physics engine for deep learning in robotics, *Frontiers in Neurorobotics* **13**, PMID: PMC6416213 (2019).
10. D. Driess, Z. Huang, Y. Li, R. Tedrake, and M. Toussaint. Learning multi-object dynamics with compositional neural radiance fields. *arXiv preprint arXiv:2202.11855* (2022).
11. Y. Du, Y. Zhang, H.-X. Yu, J. B. Tenenbaum, and J. Wu. Neural radiance flow for 4d view synthesis and video processing. *arXiv e-prints arXiv-2012* (2020).
12. F. Ebert, C. Finn, S. Dasari, A. Xie, A. Lee, and S. Levine. Visual foresight: Model-based deep reinforcement learning for vision-based robotic control. *arXiv preprint arXiv:1812.00568* (2018). <https://arxiv.org/abs/1812.00568>.
13. F. Ebert, C. Finn, A. X. Lee, and S. Levine. Self-supervised visual planning with temporal skip connections. *arXiv preprint arXiv:1710.05268* (2017).
14. S. Ehrhardt, A. Monszpart, N. Mitra, and A. Vedaldi. Taking visual motion prediction to new heightfields. *arXiv preprint arXiv:1712.09448* (2017).
15. S. A. Eslami, D. J. Rezende, F. Besse, F. Viola, A. S. Morcos, M. Garnelo, A. Ruderman, A. A. Rusu, I. Danihelka, K. Gregor *et al.* Neural scene representation and rendering, *Science* **360**(6394), 1204–1210 (2018).

16. G. Farquhar, T. Rocktäschel, M. Igl, and S. Whiteson. TreeQN and ATreeC: Differentiable tree planning for deep reinforcement learning. In *ICLR* (2018).
17. C. Finn, I. Goodfellow, and S. Levine. Unsupervised learning for physical interaction through video prediction. In *Advances in Neural Information Processing Systems*, pp. 64–72 (2016).
18. P. Florence. Dense visual learning for robot manipulation. PhD Thesis (2019).
19. P. Florence, L. Manuelli, and R. Tedrake. Self-supervised correspondence in visuomotor policy learning, *IEEE Robotics and Automation Letters* **5**(2), 492–499 (2019). doi:10.1109/LRA.2019.2956365.
20. P. R. Florence, L. Manuelli, and R. Tedrake. Dense object nets: Learning dense visual object descriptors by and for robotic manipulation. In *Conference on Robot Learning (CoRL)* (2018).
21. G. F. Franklin, J. D. Powell, A. Emami-Naeini, and J. D. Powell. *Feedback Control of Dynamic Systems*, Vol. 4. Prentice Hall, Upper Saddle River, NJ (2002).
22. J. Gao, W. Chen, T. Xiang, A. Jacobson, M. McGuire, and S. Fidler. Learning deformable tetrahedral meshes for 3D reconstruction, *Advances in Neural Information Processing Systems*, Vol. 33, pp. 9936–9947 (2020).
23. W. Gao and R. Tedrake. kPAM 2.0: Feedback control for category-level robotic manipulation. *arXiv preprint* arXiv:2102.06279 (2021).
24. S. Gu, T. Lillicrap, I. Sutskever, and S. Levine. Continuous deep q-learning with model-based acceleration. In *ICML* (2016).
25. D. Hafner, T. Lillicrap, J. Ba, and M. Norouzi. Dream to control: Learning behaviors by latent imagination. *arXiv preprint* arXiv:1912.01603 (2019).
26. D. Hafner, T. Lillicrap, I. Fischer, R. Villegas, D. Ha, H. Lee, and J. Davidson. Learning latent dynamics for planning from pixels. In *International Conference on Machine Learning*, pp. 2555–2565 (2019).
27. J. B. Hamrick, A. J. Ballard, R. Pascanu, O. Vinyals, N. Heess, and P. W. Battaglia. Metacontrol for adaptive imagination-based optimization. In *ICLR* (2017).
28. K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).
29. F. R. Hogan, M. Bauza, and A. Rodriguez. A data-efficient approach to precise and controlled pushing. *arXiv preprint* arXiv:1807.09904 (2018).

30. F. R. Hogan and A. Rodriguez. Feedback control of the pusher-slider system: A story of hybrid and underactuated contact dynamics. *arXiv preprint* arXiv:1611.08268 (2016).
31. Y. Hu, L. Anderson, T.-M. Li, Q. Sun, N. Carr, J. Ragan-Kelley, and F. Durand. DiffTaichi: Differentiable programming for physical simulation. In *International Conference on Learning Representations (ICLR)* (2019).
32. Z. Huang, Y. Hu, T. Du, S. Zhou, H. Su, J. B. Tenenbaum, and C. Gan. Plasticineland: A soft-body manipulation benchmark with differentiable physics. In *International Conference on Learning Representations (ICLR)* (2020).
33. Z. Huang, X. Lin, and D. Held. Mesh-based dynamics model with occlusion reasoning for cloth manipulation. In *Robotics: Science and Systems (RSS)* (2022).
34. T. Jakab, R. Tucker, A. Makadia, J. Wu, N. Snavely, and A. Kanazawa. Keypointdeformer: Unsupervised 3D keypoint discovery for shape control. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12783–12792 (2021).
35. T. D. Kulkarni, A. Gupta, C. Ionescu, S. Borgeaud, M. Reynolds, A. Zisserman, and V. Mnih. Unsupervised learning of object keypoints for perception and control. In *Advances in Neural Information Processing Systems*, pp. 10723–10733 (2019).
36. B. M. Lake, T. D. Ullman, J. B. Tenenbaum, and S. J. Gershman. Building machines that learn and think like people, *Behavioral and Brain Sciences* **40**, (2017).
37. T. Lei and Y. Zhang. Training RNNs as fast as CNNs. *arXiv preprint* arXiv:1709.02755 (2017).
38. I. Lenz, R. A. Knepper, and A. Saxena. DeepMPC: Learning deep latent features for model predictive control. In *RSS* (2015).
39. T. Li, M. Slavcheva, M. Zollhoefer, S. Green, C. Lassner, C. Kim, T. Schmidt, S. Lovegrove, M. Goesele, and Z. Lv. Neural 3D video synthesis. *arXiv preprint* arXiv:2103.02597 (2021).
40. Y. Li, H. He, J. Wu, D. Katabi, and A. Torralba. Learning compositional koopman operators for model-based control. *arXiv preprint* arXiv:1910.08264 (2019).
41. Y. Li, S. Li, V. Sitzmann, P. Agrawal, and A. Torralba. 3D neural scene representations for visuomotor control. In *Conference on Robot Learning*, pp. 112–123 (2022).
42. Y. Li, T. Lin, K. Yi, D. Bear, D. Yamins, J. Wu, J. Tenenbaum, and A. Torralba. Visual grounding of learned physical models. In *International Conference on Machine Learning*, pp. 5927–5936 (2020).

43. Y. Li, A. Torralba, A. Anandkumar, D. Fox, and A. Garg. Causal discovery in physical systems from videos. In *Advances in Neural Information Processing Systems*, Vol. 33, pp. 9180–9192 (2020).
44. Y. Li, J. Wu, R. Tedrake, J. B. Tenenbaum, and A. Torralba. Learning particle dynamics for manipulating rigid bodies, deformable objects, and fluids. *arXiv preprint* arXiv:1810.01566 (2018).
45. Y. Li, J. Wu, J.-Y. Zhu, J. B. Tenenbaum, A. Torralba, and R. Tedrake. Propagation networks for model-based control under partial observation. In *2019 International Conference on Robotics and Automation (ICRA)*, pp. 1205–1211 (2019).
46. Z. Li, S. Niklaus, N. Snavely, and O. Wang. Neural scene flow fields for space-time view synthesis of dynamic scenes. *arXiv preprint* arXiv:2011.13084 (2020).
47. X. Lin, Y. Wang, Z. Huang, and D. Held. Learning visible connectivity dynamics for cloth smoothing. In *Conference on Robot Learning (CoRL)*, pp. 256–266 (2022).
48. S. Lombardi, T. Simon, J. Saragih, G. Schwartz, A. Lehrmann, and Y. Sheikh. Neural volumes: Learning dynamic renderable volumes from images. *arXiv preprint* arXiv:1906.07751 (2019).
49. J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3431–3440 (2015).
50. M. Macklin, and M. Müller. Position based fluids, *ACM TOG* **32**(4), 104 (2013).
51. M. Macklin, M. Müller, N. Chentanez, and T.-Y. Kim. Unified particle physics for real-time applications, *ACM Transactions on Graphics (TOG)* **33**(4), 1–12 (2014).
52. L. Manuelli, W. Gao, P. Florence, and R. Tedrake. kPAM: Keypoint affordances for category-level robotic manipulation. *arXiv preprint* arXiv:1903.06684 (2019).
53. L. Manuelli, Y. Li, P. Florence, and R. Tedrake. Keypoints into the future: Self-supervised correspondence in model-based reinforcement learning. In *Conference on Robot Learning (CoRL)*, pp. 693–710 (2021).
54. B. Mildenhall, P. P. Srinivasan, M. Tancik, J. T. Barron, R. Ramamoorthi, and R. Ng. NeRF: Representing scenes as neural radiance fields for view synthesis. In *European Conference on Computer Vision*, pp. 405–421 (2020).
55. D. Mrowca, C. Zhuang, E. Wang, N. Haber, L. Fei-Fei, J. B. Tenenbaum, and D. L. Yamins. Flexible neural representation for physics prediction. In *NIPS* (2018).

- 56. A. Nagabandi, G. Kahn, R. S. Fearing, and S. Levine. Neural network dynamics for model-based deep reinforcement learning with model-free fine-tuning. In *ICRA* (2018).
- 57. A. Nagabandi, K. Konolige, S. Levine, and V. Kumar. Deep dynamics models for learning dexterous manipulation. In *Conference on Robot Learning (CoRL)*, pp. 1101–1112 (2020).
- 58. T. Nguyen-Phuoc, C. Li, S. Balaban, and Y.-L. Yang. RenderNet: A deep convolutional network for differentiable rendering from 3D shapes. *arXiv preprint* arXiv:1806.06575 (2018).
- 59. M. Niemeyer, L. Mescheder, M. Oechsle, and A. Geiger. Occupancy flow: 4D reconstruction by learning particle dynamics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 5379–5389 (2019).
- 60. M. Niemeyer, L. Mescheder, M. Oechsle, and A. Geiger. Differentiable volumetric rendering: Learning implicit 3D representations without 3D supervision. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3504–3515 (2020).
- 61. J. Oh, S. Singh, and H. Lee. Value prediction network. In *NIPS* (2017).
- 62. E. Olson. AprilTag: A robust and flexible visual fiducial system. In *2011 IEEE International Conference on Robotics and Automation*, pp. 3400–3407 (2011).
- 63. J. Ost, F. Mannan, N. Thuerey, J. Knodt, and F. Heide. Neural scene graphs for dynamic scenes. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2856–2865 (2021).
- 64. J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 165–174 (2019).
- 65. K. Park, U. Sinha, J. T. Barron, S. Bouaziz, D. B. Goldman, S. M. Seitz, and R.-M. Brualdi. Deformable neural radiance fields. *arXiv preprint* arXiv:2011.12948 (2020).
- 66. R. Pascanu, Y. Li, O. Vinyals, N. Heess, L. Buesing, S. Racanière, D. Reichert, T. Weber, D. Wierstra, and P. Battaglia. Learning model-based planning from scratch. *arXiv preprint* arXiv:1707.06170 (2017).
- 67. A. Pumarola, E. Corona, G. Pons-Moll, and F. Moreno-Noguer. D-neRF: Neural radiance fields for dynamic scenes. *arXiv preprint* arXiv:2011.13961 (2020).

68. S. Racanière, T. Weber, D. Reichert, L. Buesing, A. Guez, D. J. Rezende, A. P. Badia, O. Vinyals, N. Heess, Y. Li, R. Pascanu, P. Battaglia, D. Silver, and D. Wierstra. Imagination-augmented agents for deep reinforcement learning. In *NIPS* (2017).
69. D. J. Rezende, S. Eslami, S. Mohamed, P. Battaglia, M. Jaderberg, and N. Heess. Unsupervised learning of 3D structure from images. *arXiv preprint* arXiv:1607.00662 (2016).
70. G. Saponaro, L. Jamone, A. Bernardino, and G. Salvi. Beyond the self: Using grounded affordances to interpret and describe others' actions, *IEEE Transactions on Cognitive and Developmental Systems* **12**(2), 209–221 (2019).
71. C. Schenck and D. Fox. SPNets: Differentiable fluid dynamics for deep neural networks. In *CoRL* (2018).
72. T. Schmidt, R. Newcombe, and D. Fox. Self-supervised visual descriptor learning for dense correspondence, *IEEE Robotics and Automation Letters* **2**(2), 420–427 (2017).
73. J. Schrittwieser, I. Antonoglou, T. Hubert, K. Simonyan, L. Sifre, S. Schmitt, A. Guez, E. Lockhart, D. Hassabis, T. Graepel *et al.* Mastering atari, go, chess and shogi by planning with a learned model, *Nature* **588**(7839), 604–609 (2020).
74. J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov. Proximal policy optimization algorithms. *arXiv preprint* arXiv:1707.06347 (2017).
75. P. Sermanet, C. Lynch, Y. Chebotar, J. Hsu, E. Jang, S. Schaal, S. Levine, and G. Brain. Time-contrastive networks: Self-supervised learning from video. In *2018 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 1134–1141 (2018).
76. H. Shi, H. Xu, Z. Huang, Y. Li, and J. Wu. RoboCraft: Learning to see, simulate, and shape elasto-plastic objects with graph networks. *arXiv preprint* arXiv:2205.02909 (2022).
77. D. Silver, H. van Hasselt, M. Hessel, T. Schaul, A. Guez, T. Harley, G. Dulac-Arnold, D. Reichert, N. Rabinowitz, A. Barreto, and T. Degris. The predictron: End-to-end learning and planning. In *ICML* (2017).
78. V. Sitzmann, J. Thies, F. Heide, M. Nießner, G. Wetzstein, and M. Zollhöfer. DeepVoxels: Learning persistent 3D feature embeddings. In *Proceedings of CVPR* (2019).
79. V. Sitzmann, M. Zollhöfer, and G. Wetzstein. Scene representation networks: Continuous 3D-structure-aware neural scene representations. *arXiv preprint* arXiv:1906.01618 (2019).
80. A. Srinivas, A. Jabri, P. Abbeel, S. Levine, and C. Finn. Universal planning networks. In *ICML* (2018).

81. H. Suh, and R. Tedrake. The surprising effectiveness of linear models for visual foresight in object pile manipulation. *arXiv preprint arXiv:2002.09093* (2020).
82. M. Tatarchenko, A. Dosovitskiy, and T. Brox. Single-view to multi-view: Reconstructing unseen views with a convolutional network. *1(2)*, 2 (2015). CoRR abs/1511.06702.
83. R. Tedrake. Underactuated robotics: Learning, planning, and control for efficient and agile machines course notes for mit 6.832 (2009).
84. R. Tedrake, and The Drake Development Team. Drake: Model-based design and verification for robotics (2019). <https://drake.mit.edu>.
85. S. Tian, Y. Cai, H.-X. Yu, S. Zakharov, K. Liu, A. Gaidon, Y. Li, and J. Wu. Multi-object manipulation via object-centric neural scattering functions. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9021–9031 (2023).
86. E. Todorov, T. Erez, and Y. Tassa. MuJoCo: A physics engine for model-based control. In *IROS*, pp. 5026–5033 (2012).
87. M. Toussaint, K. Allen, K. Smith, and J. Tenenbaum. Differentiable physics and stable modes for tool-use and manipulation planning. In *RSS* (2018).
88. E. Tretschk, A. Tewari, V. Golyanik, M. Zollhöfer, C. Lassner, and C. Theobalt. Non-rigid neural radiance fields: Reconstruction and novel view synthesis of a deforming scene from monocular video. *arXiv preprint arXiv:2012.12247* (2020).
89. H.-Y. F. Tung, R. Cheng, and K. Fragkiadaki. Learning spatial common sense with geometry-aware recurrent networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2595–2603 (2019).
90. H.-Y. F. Tung, Z. Xian, M. Prabhudesai, S. Lal, and K. Fragkiadaki. 3D-OES: Viewpoint-invariant object-factorized environment simulators. *arXiv preprint arXiv:2011.06464* (2020).
91. C. Wang, R. Martín-Martín, D. Xu, J. Lv, C. Lu, L. Fei-Fei, S. Savarese, and Y. Zhu. 6-pack: Category-level 6d pose tracker with anchor-based keypoints. In *2020 IEEE International Conference on Robotics and Automation (ICRA)*, pp. 10059–10066 (2020).
92. N. Wang, Y. Zhang, Z. Li, Y. Fu, W. Liu, and Y.-G. Jiang. Pixel2Mesh: Generating 3D mesh models from single RGB images. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 52–67 (2018).
93. M. Watter, J. Springenberg, J. Boedecker, and M. Riedmiller. Embed to control: A locally linear latent dynamics model for control from raw images. In *Advances in Neural Information Processing Systems*, pp. 2746–2754 (2015).

94. G. Williams, A. Aldrich, and E. Theodorou. Model predictive path integral control using covariance variable importance sampling. *arXiv preprint arXiv:1509.01149* (2015).
95. D. E. Worrall, S. J. Garbin, D. Turmukhambetov, and G. J. Brostow. Interpretable transformations with encoder-decoder networks. In *Proceedings of ICCV*, Vol. 4 (2017).
96. W. Xian, J.-B. Huang, J. Kopf, and C. Kim. Space-time neural irradiance fields for free-viewpoint video. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9421–9431 (2021).
97. W. Yan, A. Vangipuram, P. Abbeel, and L. Pinto. Learning predictive representations for deformable objects using contrastive estimation. In *Conference on Robot Learning (CoRL)* (2020).
98. L. Yen-Chen, M. Bauza, and P. Isola. Experience-embedded visual foresight. In *Conference on Robot Learning*, pp. 1015–1024 (2020).
99. J. Zhou, Y. Hou, and M. T. Mason. Pushing revisited: Differential flatness, trajectory planning, and stabilization, *The International Journal of Robotics Research* **38**(12–13), 1477–1489 (2019).
100. J.-Y. Zhu, Z. Zhang, C. Zhang, J. Wu, A. Torralba, J. Tenenbaum, and B. Freeman. Visual object networks: image generation with disentangled 3D representations. In *Proceedings of NIPS*, pp. 118–129 (2018).

Chapter 5

eDiGS: Extended Divergence-Guided Shape Implicit Neural Representation for Unoriented Point Clouds

**Yizhak Ben-Shabat^{*,‡}, Chamin Hewa Koneputugodage^{†,§},
and Stephen Gould^{†,¶}**

^{}Technion Israel Institute of Technology,
Haifa, Israel*

*[†]The Australian National University,
Canberra, Australia*

[‡]sitzikbs@gmail.com

[§]chamin.hewa@anu.edu.au

[¶]stephen.gould@anu.edu.au

In this chapter, we propose a new approach for learning shape implicit neural representations (INRs) from point cloud data that does not require normal vectors as input. We show that our method, which uses a soft constraint on the divergence of the distance function to the shape's surface, can produce smooth solutions that accurately orient gradients to match the unknown normal at each point, even outperforming methods that use normal vectors directly. This work extends the latest work on divergence-guided sinusoidal activation INRs¹ to Gaussian activation INRs and provides extended theoretical analysis and results. We evaluate our approach on tasks related to surface reconstruction and shape space learning.

1. Introduction

The problem of reconstructing surfaces from 3D point samples has been widely studied in computer vision and computer graphics. In recent years, researchers have used neural networks to learn an implicit neural representation (INR) that can be used to reconstruct the underlying surface.^{2–10} These methods typically require normal data to help train the INR, but raw 3D point clouds from scans are often unoriented and do not have normal vectors. As a result, pre-processing normal estimation is required. While some methods can handle absent normal supervision,^{9,10} their performance drops significantly without it. In this chapter, we address the challenges of using unoriented and noisy normal data in shape INR learning.

In this work, we extend the recently proposed divergence-guided shape implicit neural representation learning approach (DiGS INR)¹ in several ways. First, we extend the approach to INRs that use Gaussian activations, and second, we provide extended theoretical analysis and results. DiGS uses raw 3D point data without any pre-processing stages to train a sinusoidal activation INR. The approach is motivated by the observation, illustrated in Figure 1, that the gradient vector field of the signed distance function (SDF) (produced by the network) has low divergence nearly everywhere. This geometric prior is incorporated as a soft constraint in the loss function and annealed as training progresses. The basic requirement of such a prior is a network architecture that has continuous second derivatives, such

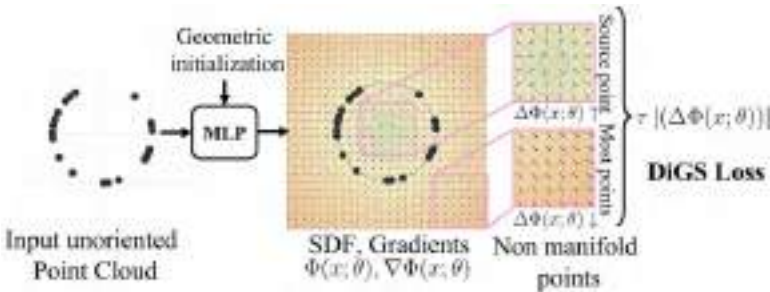


Fig. 1. DiGS intuition illustration. Given an unoriented input point cloud sampled on a shape, we train a shape implicit neural representation using a divergence penalty which arises from the observation that in most locations the divergence of the SDF should be low.

as SIRENs¹⁰ (which is used in DiGS¹), in contrast to many previous works that use ReLU multi-layer perceptrons (MLPs). In this work, we extend the divergence constraint to Gaussian activation network. The motivation for this extension is rooted in a recent work that showed the underlying connection between the choice of activation function and the Lipschitz constant.¹¹

We evaluate the performance of our proposed method, called eDiGS, on the tasks of surface reconstruction and shape space learning using datasets with challenging properties, such as topological complexity, nonuniform sampling, registration misalignment, missing data, and different levels of detail. These datasets include the Surface Reconstruction Benchmark (SRB),¹² DFAUST,¹³ and ShapeNet.¹⁴ Our results show that eDiGS performs well without the need for normal supervision and is on par with, or even outperforms, state-of-the-art methods that use normal data. Furthermore, eDiGS significantly outperforms other methods that work on unoriented point clouds.

The contributions of this paper include deriving two novel geometric initialisation methods for sinusoidal-based shape INR networks that lead to better representations. Additionally, we propose to learn INRs from raw point cloud data without ground-truth normal vector or distance to manifold information. Furthermore, we conduct extensive comparisons to state-of-the-art approaches with and without normal supervision and demonstrate the divergence constraint effectiveness on non-sinusoidal activation functions.

2. Related Work

2.1. Surface Reconstruction

Surface reconstruction from point clouds is a longstanding and challenging problem in computer vision and computer graphics. It involves overcoming several obstacles, such as non-uniform sampling, noise in point positions and normals, missing data due to occlusions, and more. Traditional approaches to surface reconstruction include triangulation methods,^{15,16} alpha shapes,¹⁷ and Voronoi diagrams.¹⁸ More recent approaches use neural networks to learn mappings from a parametric space to the surface,^{19,20} learn an implicit volumetric

function on regular grids,^{21,22} or learn an INR of the shape.²³ For a comprehensive survey of classical surface reconstruction methods, we refer the reader to Berger *et al.*²⁴

2.2. Shape Implicit Neural Representations (INRs)

Neural networks have proven effective at representing shapes as implicit functions^{2,3,6,7,9,10,25–29} and scenes.^{4,30–32} These networks are trained to output a SDF,^{2,6,7,9,10,25,27,28,30–32} an occupancy function,^{3,4,26} or a combination of the two.²⁹

Our method uses a SDF to represent the shape. The use of SDFs in shape INRs was popularised by DeepSDF.² However, this method requires ground-truth data for points that are outside the shape (i.e., the sign of the SDF), which is unrealistic. SAL⁶ addresses this limitation by using a sign-agnostic training method that does not require extra 3D supervision. SALD⁷ shows that using ground-truth normals for points on the surface greatly improves performance. IGR⁹ introduces an eikonal equation-based loss term to regularise the learned function towards being an SDF, which can be used with or without normals. NSP²⁷ frames the task as a kernel regression problem, where the kernel chosen is equivalent to the limit of infinitely wide, shallow ReLU networks. FFN²⁶ and SIREN¹⁰ demonstrate that INRs using conventional MLPs tend to bias towards low-frequency solutions and introduce high frequencies into their architecture to address this. FFN uses an ReLU MLP with a Fourier feature layer, while SIREN uses periodic (specifically, sine) activation functions. The benefit of the latter is that second derivatives (and in fact all orders) are defined and continuous, which allows for higher-order supervision, as we use in this paper. On the other hand, DeepSDF and SAL use ReLU for activation, so this is not possible with those methods. Recently, IDF²⁸ extended SIRENs to explicitly decompose learning the high and low frequencies of a shape and combine by having the local high frequency as a displacement in the low-frequency SIREN’s normal direction. PHASE²⁹ introduced a loss that learns a density function whose limit is an occupancy and whose log transform is an SDF. They also introduce a version without ground-truth normal supervision, PHASE+FF, which uses the Fourier features of FFN.²⁶

Ground-truth normal information is not typically available in real-world, raw scan data and must be estimated, which can be noisy. Note

that all of the above methods, except for SAL (and DeepSDF, which uses other ground-truth information), report results using ground-truth normal information. Technically, IGR and SIREN can operate without such information by dropping the corresponding loss term in their overall loss, but in our experiments, we observed that performance significantly drops in their absence.

2.3. Initialising Shape INRs

As finding good shape implicit representations is hard due to the ill-posed nature of the task,²⁴ many methods introduce initialisation or regularisation to bias towards favourable solutions.^{6,7,9} SAL⁶ introduce a geometric initialisation for ReLU MLPs, which carefully chooses the initial weights (or the distribution of the initial weights) in particular layers to make the initial function be approximately the SDF to an r -radius sphere. This initialisation has shown to have favourable properties for reconstruction, such as in-object and out-object sign consistency.

For SIRENs, to allow for training deeper networks for general INRs, Sitzmann *et al.*¹⁰ proposed an initialisation that preserves the distribution of activations through its layers. However, for shape INRs, this initialisation lacks a geometric grounding and sometimes causes unwanted ghost geometries. In this chapter, we propose two novel initialisations for SIRENs that extend the notion of geometric initialisation to periodic activations and show that initialising with high frequencies is important for capturing fine detail.

3. Outline of Divergence-Guided Shape INRs

We use a smooth-to-sharp approach that keeps the gradient vector field stay highly consistent during training (see Figure 2). In particular, it allows us to learn a good implicit representation without normal information. The proposed training procedure consists of the following four steps, which are detailed in subsequent sections:

- **Geometric initialisation:** Initialise to a sphere, biasing the function to start with an SDF that is positive away from the object and negative in the centre of the object’s bounding box while keeping the model’s ability to have high frequencies (in a controllable manner).

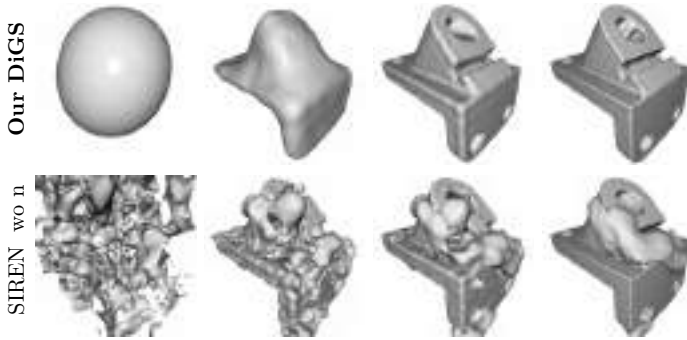


Fig. 2. Surface reconstruction training. Depicting results at four different training iterations (progressing from left to right) for DiGS (top) and SIREN wo n (bottom).

- **High divergence phase:** Guide the model towards a smooth reconstruction of the coarse shape. Importantly, this prevents the model from prematurely fitting to fine details.
- **Annealing divergence phase:** Slowly allow fine details to emerge while still learning a function that has smoothly changing normals.
- **Low divergence phase:** Allow very fine details such as sharp corners to emerge and for the function to interpolate the original data (point cloud samples) as much as possible (subject to also minimising the eikonal term).

A high weight on the divergence loss produces very smooth SDF functions, leading to oversmoothed reconstructions. However, learning such smooth representations can be done quickly and robustly.

We divide the total number of iterations in 50%, 25%, and 25% for the high, annealing, and low divergence phases, respectively.

In Figures 3 and 4, we depict the difference between the training of DiGS and SIREN without normals (henceforth, SIREN wo n), highlighting the four phases of DiGS. SIREN wo n fits to the shape very quickly but does not do so in a consistent manner and thus has multiple surfaces interpolating the surface points that have differing orientations for what is inside and out (best seen with the 2D contours). It then tries to refine and improve upon this but is stuck with the ghost geometry from its early fitting. DiGS on the other hand has a structured training procedure that prevents this: it slowly changes the SDF for the noisy sphere, allowing more and more details, which

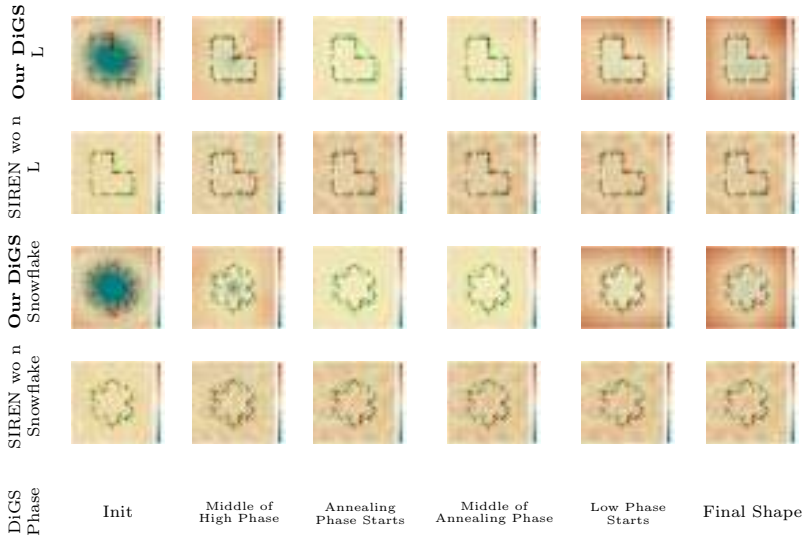


Fig. 3. Reconstruction training progress in 2D. Visualising DiGS and SIREN wo n at six time steps during training for 2D shapes. The six training time steps were selected according to the four phases of DiGS’ training. A contour plot of the learned function is shown: the black dots are surface points, the green arrows are ground-truth normals, and the red arrows are the current normals at those points.

reduces incorrect orientation and thus ghost geometry from occurring. As SIREN wo n does not have a zero level set at initialisation (see the contour plot for the 2D examples), there is no visible surface for its initialisation in 3D. Note that the initialisation for DC is the same noisy sphere as the initialisation for the gargoyle shape, however it may appear slightly different because it is rotated and cut due to the bounding box for the ground-truth DC shape.

4. Geometric Initialisation for SIRENs

We now detail our two geometrically motivated initialisations for SIRENs, shown in Figure 5 for 2D.

Initialisation to a sphere: A key component of our method is a geometrically meaningful initialisation for the parameters of SIRENs such that the initial SDF is approximately an r -radius sphere.

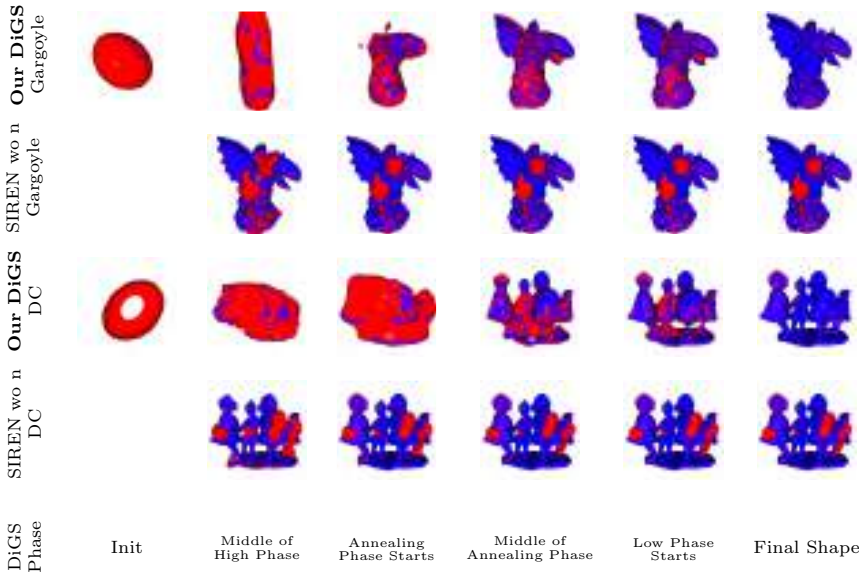


Fig. 4. Surface reconstruction training progress. Visualising DiGS and SIREN w/o n at six points during training on 3D shapes from SRB.¹² The six training time steps are labelled according to the four phases of DiGS’ training. The zero level set at each step is shown and coloured by the distance to the closest ground-truth point (thresholded at one). As SIREN w/o n does not have a zero level set at initialisation (see the contour plot for the 2D examples), there is no visualisation for its initialisation in 3D.

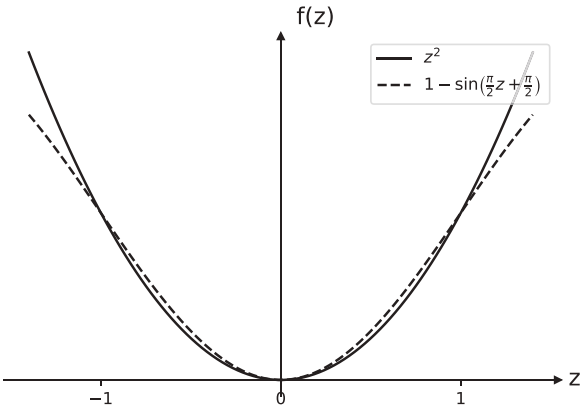


Fig. 5. Approximating z^2 using $1 - \sin\left(\frac{\pi}{2}z + \frac{\pi}{2}\right)$.

Let us consider a SIREN model specified by

$$\begin{aligned}\Phi(x; \theta) &= \mathbf{w}_n^T (\phi_{n-1} \circ \phi_{n-2} \circ \cdots \circ \phi_0)(x) + b_n, \\ \phi_i(x_i) &= \sin(\mathbf{W}_i x_i + \mathbf{b}_i),\end{aligned}\tag{1}$$

where $\phi_i : \mathbb{R}^{M_i} \rightarrow \mathbb{R}^{N_i}$ is the i^{th} layer of the network, with input $x_i \in \mathbb{R}^{M_i}$ (so $x_0 = x$) and parameters $\theta = \{\mathbf{w}_n, b_n, \mathbf{W}_{n-1}, \mathbf{b}_{n-1}, \dots, \mathbf{W}_1, \mathbf{b}_1\}$, where $\mathbf{W}_i \in \mathbb{R}^{N_i \times M_i}$, $\mathbf{b}_i \in \mathbb{R}^{N_i}$, $\mathbf{w}_n \in \mathbb{R}^{M_n}$, $b_n \in \mathbb{R}$. Rather than approximating a norm using smooth SIRENs, we instead approximate the more tractable signed squared norm³³ and apply the following function to the output of the SIREN:

$$\nu(d) = \mathbf{sign}(d) \sqrt{\|d\| + \varepsilon}.\tag{2}$$

Thus, we develop an initialisation of the network's parameters, $\theta = \theta_0$, such that $\nu(\Phi(x; \theta_0)) \approx \|x\|_2$ for x within the unit ball. Note translating and scaling to the unit ball is standard for point clouds, so we are only interested in the INR within this region. We then manually minus r from this to initialise to an r -radius sphere.

The following proposition and proofs show this for a single hidden layer SIREN, where we approximate $z \mapsto z^2$ using a translated sine wave.

Proposition 1. *Let Φ be a single hidden layer SIREN ($n = 1$ in Equation (1)) of dimension M_n and let x be a point within the unit ball. Set, $\mathbf{w}_n = -\mathbf{1}$, $\mathbf{W}_{n-1} = \frac{\pi}{2}I$, $\mathbf{b}_{n-1} = \frac{\pi}{2}\mathbf{1}$, and $b_n = M_n$. Then, $\nu(\Phi(x)) \approx \|x\|_2$.*

Proof. For 1D input $z \in [-1, 1]$, we can approximate z^2 by $1 - \sin\left(\frac{\pi}{2}z + \frac{\pi}{2}\right)$ (see Figure 6). Then

$$\Phi(x) = (-\mathbf{1})^T \sin\left(\left(\frac{\pi}{2}I\right)x + \left(\frac{\pi}{2}\mathbf{1}\right)\right) + M_n\tag{3}$$

$$= \sum_{i=1}^{M_n} 1 - \sin\left(\frac{\pi}{2}x_i + \frac{\pi}{2}\right)\tag{4}$$

$$\approx \sum_{i=1}^{M_n} x_i^2\tag{5}$$

$$= \|x\|_2^2\tag{6}$$

so $\nu(\Phi(x)) \approx \|x\|_2$. □

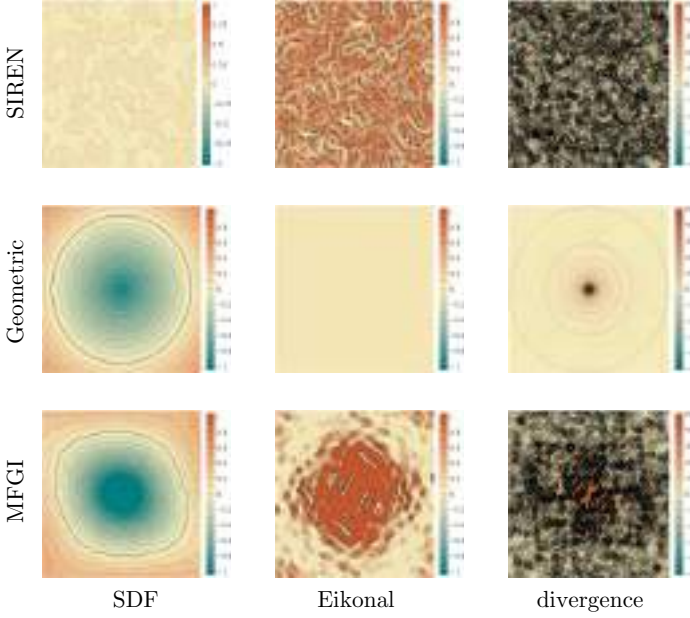


Fig. 6. Initialisation comparison in 2D. Visualisation of the proposed geometric initialisation and multi-frequency geometric initialisation for sinusoidal representation networks in 2D compared to Sitzmann *et al.*¹⁰ Depicting the sign distance function (left), eikonal (middle), and divergence (right).

To extend this to networks with an arbitrary number of layers, we design layers ϕ_i that preserve the norm on expectation w.r.t. the weights of each layer up to the penultimate layer, i.e., $\mathbb{E}[\|\phi_i(x)\|_2] = \|x\|_2$ for $i = 1, \dots, n - 2$. We first prove the following lemmas.

Lemma 1. *Let X, Y be two random d -dimensional vectors with elements X_i and Y_i sampled i.i.d. from $\mathcal{U}\left(-\sqrt{\frac{3}{d}}, \sqrt{\frac{3}{d}}\right)$. Then $\mathbf{E}[\|X\|] = \mathbf{E}[\|Y\|] = 1$ and $\mathbf{E}[\langle X, Y \rangle] = 0$.*

In words, random vectors generated i.i.d. from the above distribution are on average unit norm and orthogonal.

Proof. We have that $\mathbf{E}[X_i] = \mathbf{E}[Y_i] = 0$ and $\mathbf{var}(X_i) = \mathbf{var}(Y_i) = \frac{1}{12} \left(2\sqrt{\frac{3}{d}}\right)^2 = \frac{1}{d}$. Thus note that $\mathbf{E}[X_i^2] = \mathbf{E}[Y_i^2] = \mathbf{E}[(X_i - \mathbf{E}[X_i])^2] = \mathbf{var}(X_i) = \frac{1}{d}$. By the weak law of large numbers, we

have that for any $\varepsilon > 0$

$$\mathcal{P} \left(\left| \frac{X_1^2 + \dots + X_d^2}{d} - \mathbf{E} [X_i^2] \right| > \varepsilon \right) \leq \frac{\text{var}(X_i^2)}{d\varepsilon^2} \quad (7)$$

$$\therefore \Pr \left(\left| \frac{\|X\|_2^2}{d} - \frac{1}{d} \right| > \varepsilon \right) \leq \frac{\text{var}(X_i^2)}{d\varepsilon^2} \quad (8)$$

$$\therefore \Pr \left(\left| \frac{\|X\|_2^2}{d} - \frac{1}{d} \right| \leq \varepsilon \right) \geq 1 - \frac{\text{var}(X_i^2)}{d\varepsilon^2} \quad (9)$$

so $\|X\|_2^2 \approx 1$. It follows that $\mathbf{E} [\|X\|_2] = \mathbf{E} [\|Y\|_2] = 1$.

Furthermore, we have that

$$\mathbf{E} [\langle X, Y \rangle] = \mathbf{E} \left[\sum_{i=1}^d X_i Y_i \right] \quad (10)$$

$$= d \mathbf{E} [X_i Y_i] \quad (11)$$

$$= d \mathbf{E} [X_i] \mathbf{E} [Y_i] \quad (12)$$

$$= 0, \quad (13)$$

where Equation (12) follows from them being independent. $\square$

Lemma 2. Consider $A \in \mathbb{R}^{m \times p}$, $p \leq m$, such that $A_{ij} \sim \mathcal{U} \left(-\sqrt{\frac{3}{m}}, \sqrt{\frac{3}{m}} \right)$. Then for any $x \in \mathbb{R}^p$ we have that $\|Ax\|_2 \approx \|x\|_2$.

Proof. By Lemma 1, the p columns of A , which are of dimension m , are on average of unit norm and orthogonal to each other (note $p \leq m$). As a result, $A^T A \approx I_p$, so

$$\|Ax\|_2^2 = x^T A^T A x \approx x^T I x = \|x\|_2^2$$

implying that $\|Ax\|_2 \approx \|x\|_2$. $\square$

Lemma 3. Consider $A \in \mathbb{R}^{m \times p}$, $m \gg 10$, such that $A_{ij} \sim \mathcal{U} \left(-\sqrt{\frac{3}{m}}, \sqrt{\frac{3}{m}} \right)$. Then for $x \in \mathbb{R}^p$ s.t. $\|x\|_2 \leq 1$, we have that $\sin(Ax) \approx Ax$.

Proof. By Lemma 2, we have that $\|Ax\|_2 \approx \|x\|_2$. Furthermore, since A is generated uniform randomly, the values of $Ax \in \mathbb{R}^m$ should be randomly distributed, therefore as $\|Ax\|_2 \approx \|x\|_2 \leq 1$ and $m \gg 10$, with high probability $|(Ax)_i| < 0.2$. Thus, $\sin((Ax)_i) \approx (Ax)_i$ (as it is within the linear region of sine), so $\sin(Ax) \approx Ax$. $\square$

Proposition 2. *Let Φ be an n -hidden layer SIREN (Equation (1)) that maps from $\mathbb{R}^{M_0} \rightarrow \mathbb{R}$ and $\|x\|_2 \leq 1$. Set $\mathbf{W}_i \sim \mathcal{U}(-c_{wr}^i, c_{wr}^i)$, $c_{wr}^i = \sqrt{\frac{3}{M_{i+1}}}$, $\mathbf{b}_i = \mathbf{0}$ for $0 \leq i \leq n-2$ and $\mathbf{W}_{n-1} = \frac{\pi}{2}I$, $\mathbf{b}_{n-1} = \frac{\pi}{2}\mathbf{1}$, $\mathbf{w}_n = -\mathbf{1}$, and $b_n = M_n$. Then $\nu(\Phi(x)) \approx \|x\|_2$.*

Proof. We first prove that the input to the last hidden layer, x_{n-1} , has the property that $\|x_{n-1}\|_2 \approx \|x\|_2$. Proposition 1 then implies that $\nu(\Phi(x)) \approx \|x_{n-1}\|_2 \approx \|x\|_2$, as x_{n-1} is essentially the input to a one hidden layer network satisfying Proposition 1.

Now for $0 \leq i \leq n-2$, if $\|x_i\|_2 \leq 1$,

$$\|x_i\|_2 \approx \|\mathbf{W}_i x_i\|_2 \quad (14)$$

$$\approx \|\sin(\mathbf{W}_i x_i + \mathbf{b}_i)\|_2 \quad (15)$$

$$= \|x_{i+1}\|_2, \quad (16)$$

where Equation (14) holds from Lemma 2 and Equation (15) holds due to $\mathbf{b}_i = \mathbf{0}$ and Lemma 3.

Thus as $\|x_0\|_2 = \|x\|_2 \leq 1$, by induction, $\|x\| \approx \|x_{n-1}\|_2$. $\square$

However, as this initialisation keeps all activations within the first period of sin, in practice, SIRENs initialised this way will not generate high frequency output. Sitzmann *et al.*¹⁰ also note this problem and specifically scale the weight matrix of the first layer by $\omega_0 = 30$ to hit up to 30 periods, thus giving multiple frequencies through the network. To overcome this problem, we proposed the multi-frequency geometric initialisation (MFGI).

Multi-frequency geometric initialisation (MFGI): We initialise using the geometric initialisation of Proposition 2 and introduce high frequencies into the first layer in a controlled manner (see Figure 7 for an illustration of the method). Specifically, we keep the initialisation for the first k_r rows of the weight matrix $\mathbf{W}_0 = [w_{ij}^0] \in \mathbb{R}^{N_0 \times M_0}$, and for the last $N_0 - k_r$ rows, we scale

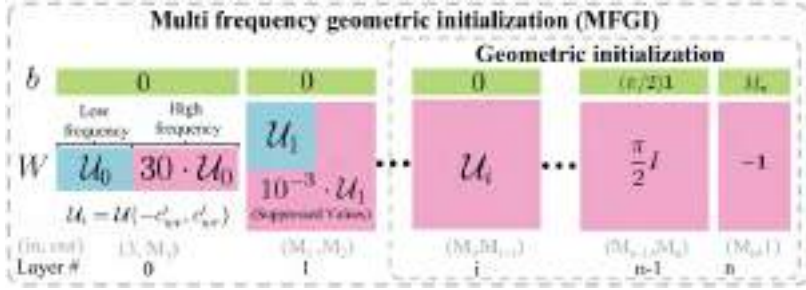


Fig. 7. The proposed initialisations illustration. Presenting the weight and bias distributions of our geometric initialisation and MFGI.

the initialised values by $n_p = 30$. This means that the output after the first layer, $x_1 \in \mathbb{R}^{N_0}$, has its first k_r elements hitting only one period and its remaining $N_0 - k_r$ elements hitting up to n_p periods. Then so that our geometric initialisation still works, we scale down any part of the next weight matrix, $\mathbf{W}_1 = [w_{ij}^1] \in \mathbb{R}^{N_1 \times M_1}$, that would multiply into the multi-period part of the vector by a factor $s = 10^{-3}$. This can be summarised as

$$w_{ij}^0 \sim \begin{cases} \mathcal{U}(-c_{wr}^0, c_{wr}^0) & 0 \leq j \leq k_r \\ \mathcal{U}(-n_p c_{wr}^0, n_p c_{wr}^0) & \text{otherwise} \end{cases} \quad (17)$$

$$w_{ij}^1 \sim \begin{cases} \mathcal{U}(-c_{wr}^1, c_{wr}^1) & 0 \leq j \leq k_r, 0 \leq i \leq k_r \\ \mathcal{U}(-s c_{wr}^1, s c_{wr}^1) & \text{otherwise} \end{cases}$$

In our experiments, we found that $k_r \approx \frac{1}{4}N_0$ is sufficient.

Visualisation of the initialisations: Figure 5 shows the two initialisations introduced compared to the initialisation proposed in SIREN¹⁰ for a 4-layer SIREN with 128 nodes in each layer. It depicts the SDF, Eikonal term, and divergence term. Notice that SIREN’s initialisation has both SDF and gradient vector norm of almost zero everywhere. On the other hand, our geometric initialisation has sphere-like level sets, which provide smooth and desirable eikonal and divergence terms. Our MFGI initialisation is a noisy version of our geometric initialisation, where the amount of noise can be tuned by n_p , k_r , and s .

5. DiGS Loss

5.1. Existing Loss Function Components and Setup

Prominent neural implicit representation methods^{2,6,7,9,10} train a neural network with parameters θ to output an SDF $\Phi(x; \theta)$ to the surface of an underlying (unknown) shape for every given point $x \in \mathbb{R}^3$. They require several loss functions when training to constrain the learned function in certain ways:

- *Manifold constraint* (L_A): Points on the surface manifold should be on the function's zero level set.^{2,6,7,9,10}
- *Normal constraint* (L_B): The gradients of points on the surface manifold should match the ground-truth normal if such supervision is available.^{7,9,10}
- *Non-manifold constraint* (L_{C1}): Points off the surface manifold should match their ground-truth SDF or SDF magnitude.^{2,6,7} This is only usable if such GT data is available.
- *Eikonal constraint* (L_{C2}): All points should have a unit gradient.^{9,10}
- *Non-manifold penalisation constraint* (L_D): Off surface points should not have zero SDF.¹⁰

Note that L_A and L_C are necessary, and L_B and L_D are used for improving results.

Our method uses the same network architecture and loss functions of SIREN,¹⁰ which we provide for completeness. Given domain Ω and surface manifold Ω_0 , they define the earlier loss functions as

$$L_A = \int_{\Omega_0} \|\Phi(x; \theta)\|_2 dx, \quad (18)$$

$$L_B = \int_{\Omega_0} (1 - \langle \nabla_x \Phi(x; \theta), n_{\text{GT}}(x) \rangle) dx, \quad (19)$$

$$L_{C2} = \int_{\Omega} \left| \|\nabla_x \Phi(x; \theta)\|_2 - 1 \right| dx, \quad (20)$$

$$L_D = \int_{\Omega \setminus \Omega_0} \exp(-\alpha |\Phi(x; \theta)|), \quad \alpha \gg 1. \quad (21)$$

In summary, the final loss for SIREN is given by a weighted sum of all of the earlier terms:

$$L_{\text{SIREN}} = \lambda_A L_A + \lambda_B L_B + \lambda_{C2} L_{C2} + \lambda_D L_D \quad (22)$$

with $(\lambda_A, \lambda_B, \lambda_{C2}, \lambda_D) = (3000, 100, 50, 100)$ as given by Sitzmann *et al.*¹⁰

Supervision for L_B and L_{C1} is either unlikely to have in practice or costly to approximate well. Gropp *et al.*⁹ showed however that L_{C1} can be replaced by L_{C2} , a popular constraint in PDE theory called the eikonal equation, which does not require extra data. In fact, given continuous (and sufficiently well behaved) boundary constraints L_A , solving for a viscous solution to the PDE defined by the eikonal equation will be unique (i.e., only L_A and L_{C2} are necessary). The difficulty in practice, however, is that constraint L_A is only defined at discrete points, resulting in infinitely many solutions, hence the requirement of other constraints to guide to favourable solutions (especially L_B).

We observe and deal with another problem (that we highlight in Section 5.3): our loss functions (and thus constraints) can only be evaluated at finite, discrete points within the domain. As a result, the quality of our solution depends highly on the sampling density of our method and how effectively our losses constrain the surrounding region. While we cannot increase the number of discrete points we get as supervision for L_A , L_B greatly constrains the function space by adding higher-order information (but requires extra supervision). We now do the same for L_C without extra supervision and show that it can even remove the need for L_B . Note that to benchmark against no normal supervision L_B , we can also define

$$L_{\text{SIREN wo n}} = \lambda_A L_A + \lambda_{C2} L_{C2} + \lambda_D L_D. \quad (23)$$

5.2. Second-Order Unsupervised Constraints

We turn to second-order information to further constrain the SDF around each sample on $\Omega \setminus \Omega_0$. Given the gradient vector field $\nabla\Phi$, we can compute its curl, $\nabla \times \nabla\Phi$, and divergence, $\Delta\Phi := \nabla \cdot \nabla\Phi$ (note that this is also known as the Laplacian of the underlying scalar

field Φ).³⁴ The curl of any gradient vector field is zero everywhere so it does not give us any information. However, we can observe that for a ground-truth SDF, the magnitude of the divergence is very low in most areas of the domain. We can thus impose a penalty on the magnitude of the divergence, i.e.,

$$L_{\text{div}} = \int_{\Omega \setminus \Omega_0} |\Delta \Phi(x; \theta)| dx = \int_{\Omega \setminus \Omega_0} |\nabla_x \cdot \nabla_x \Phi(x; \theta)| dx. \quad (24)$$

This leads to our proposed DiGS loss, given by

$$L_{\text{DiGS}} = L_{\text{SIREN}} + \tau \lambda_{\text{div}} L_{\text{div}}, \quad (25)$$

where we use $\lambda_{\text{div}} = 100$ and τ is an annealing factor we discuss in Section 3.

Note that our approach can only be used with architectures that have activation functions with non-zero second derivative, such as SIRENs. ReLU-based networks will not be affected by the divergence constraint.

Intuitively, the current losses enforce two properties of SDFs, the function should be zero on the surface, and the gradient of the norm should be one everywhere. In particular, most methods are looking at two scalar fields related to the SDF function: the function itself and the gradient norm field. On the other hand, we propose to also consider two more scalar fields: the divergence and curl of the gradient vector field of the SDF function. In Figure 8, we visualise the ground-truth value of these four fields for three 2D shapes. We can see straight away that the curl is zero everywhere, and the gradient norm is one everywhere. The divergence on the other hand has a very low magnitude in most areas, spiking sharply at points, such as centres, skeletons, or corners of the shape, and diffusing quickly from there.

The divergence theorem interpretation: The divergence theorem³⁵ states that integrating over the outward flow in a volume using a triple integral of the divergence is equivalent to a double integral of the flux through its encapsulating surface:

$$\iiint_V \Delta \Phi(x; \theta) dV = \iint_S \nabla \Phi(x; \theta) \cdot \hat{n} dS. \quad (26)$$

To intuitively understand what the divergence at a point means, consider the earlier equation with the volume V being a ball centred at

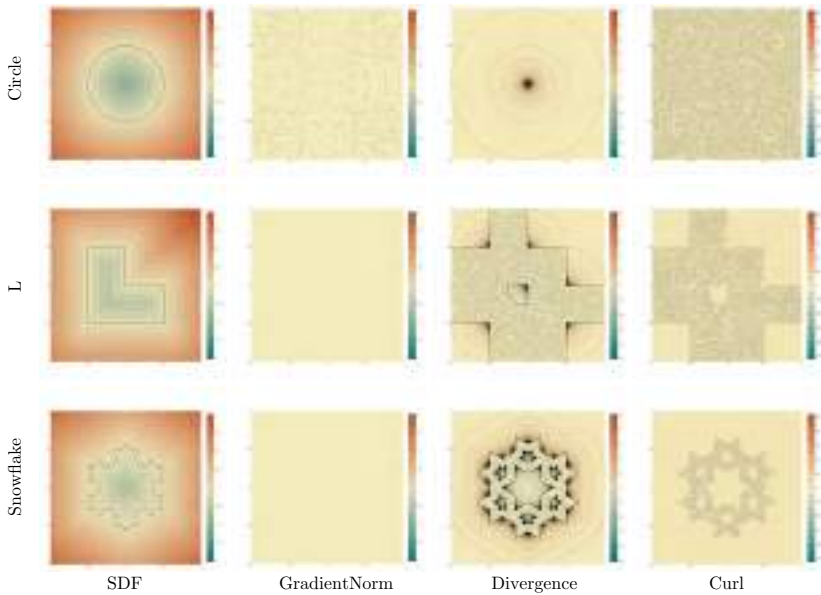


Fig. 8. Ground-truth scalar fields for 2D shapes. Depicting the SDF, gradient norm, divergence, and curl (from left to right). We can see that the divergence of the gradient vector field spikes sharply at points, such as centres, medial axes, skeletons, or corners of the shape, and diffuses quickly from there.

the point and taking the limit of the radius to 0. Then the divergence of a point is how much the vector field moves towards that point or away from that point from all directions, where mostly moving towards the point implies positive divergence (often called a sink), mostly moving away from that point implies negative divergence (often called a source), and it being balanced implies zero divergence. Thus a point having low divergence magnitude implies that the direction of the gradients is not changing much around that point. The theorem implies that divergence of a single point is heavily influenced by the surrounding region, so it incorporates a lot of local information of the (gradient) vector field.

5.2.1. Second-order supervision constraint

Normal vectors are often not available during training and are therefore estimated using some local approximation method.^{36–39} Some methods also provide an approximation of the principal

curvatures.^{36,38} The mean curvatures $\kappa_{\text{mean}} = \frac{1}{2}(\kappa_1 + \kappa_2)$ provide second-order information that can be utilised as supervision for learning shape representation. We propose a supervised variation to DiGS that penalises points on the surface for having a different mean curvature than the divergence of the vector field using the following constraint:

$$L_{\text{curv}} = \int_{\Omega_0} |\Delta\Phi(x; \theta)| - 2|\kappa_{\text{mean}}| dx. \quad (27)$$

The new supervised loss is given by

$$L_{\text{DiGS+curv}} = \lambda_A L_A + \lambda_B L_B + \lambda_{C2} L_{C2} + \lambda_D L_D + \lambda_{\text{curv}} L_{\text{curv}}. \quad (28)$$

Note that normal and curvature estimations are noisy and highly depend on local neighbouring points support size, therefore adding these supervisory signals does not guarantee improved performance.

5.3. Minimising Divergence as Regularisation

We now provide some justification of the loss in Equation (24) by showing that it is equivalent to regularising the learnt function. We can quantify the complexity of our learnt function by using the Dirichlet energy. The Dirichlet energy of a function Φ over a space Ω gives a notion for how smooth or variable the function is⁴⁰ (where lower implies smoother), defined by

$$E[\Phi] = \frac{1}{2} \int_{\Omega} \|\nabla\Phi(x)\|_2^2 dx. \quad (29)$$

Using Green's first identity we have that

$$E[\Phi] = \frac{1}{2} \int_{\partial\Omega} \langle \nabla\Phi(x), \mathbf{n}(x) \rangle \Phi(x) dx - \frac{1}{2} \int_{\Omega} \Delta\Phi(x) \Phi(x) dx \quad (30)$$

and its functional derivative is

$$DE[\Phi]\Psi = \int_{\partial\Omega} \langle \nabla\Phi(x), \mathbf{n}(x) \rangle \Psi(x) dx - \int_{\Omega} \Delta\Phi(x) \Psi(x) dx, \quad (31)$$

where $\mathbf{n}$ is the outward normal vector to the boundary $\partial\Omega$. Thus to minimise the Dirichlet energy, the functional derivative needs to be 0 for all Ψ . As Ψ is a infinitesimal displacement of Φ , it vanishes on $\partial\Omega$,

so we get $\Delta\Phi(x) = 0$. This is Laplace’s equation, whose solutions are harmonic functions (e.g., steady-state heat equation, which will have a unique solution under sufficiently regular boundary conditions).

To minimise this with respect to our constraints (A)–(D), it suffices to find the function satisfying our conditions whose magnitude of the divergence is as small as possible, i.e., minimising our divergence term L_{div} in Equation (24) since we are more restrictive in the functions Φ we want to minimise $E[\Phi]$ for. Specifically, we only consider Φ that interpolates our surface points within its zero level set (which can be considered as a boundary condition) and the eikonal equation must hold. As a result, we would not be able to solve for a harmonic function, as most SDFs are not harmonic (see Figure 8 for the ground-truth divergence field). However, as our functional is convex, to find a function that satisfies our conditions and has minimal Dirichlet energy, it suffices to find the function satisfying our conditions whose absolute value of the functional derivative is as small as possible, i.e., minimising our divergence term L_{div} (Equation (24)).

Note that this loss term is specifically defined over $\Omega \setminus \Omega_0$, compared to Equation (20) which is defined over Ω . The difference, Ω_0 , is a set of measure zero and mathematically does not change much, but it does computationally: to evaluate our losses, we have random samples over the space ($\Omega \approx \Omega \setminus \Omega_0$) and samples at ground-truth surface points (i.e., heavy sampling on the surface Ω_0). Thus Equation (20) is computed over both sets of samples, while the divergence loss term is only computed over the former. If we computed the divergence loss over the heavily sampled surface points, it would make our surface drastically smooth. In practice, we want the opposite of this: a surface with fine detail will have high variability in its SDF near the surface and low variability further away from the surface. Similarly, PHASE²⁹ also use the Dirichlet energy term for regularisation in their method but motivate it from the van der Waals–Cahn–Hilliard (WCH) theory for the physical phenomenon of phase transitions, and they do not anneal the loss.

Why not explicitly minimise the Dirichlet energy as a loss function? Doing so would be redundant, due to our eikonal term (Equation (20)): if the gradient is constrained to have unit norm on the sampled points, then the Dirichlet energy, as far as can be determined from at those sampled points, is already determined. However,

we argue that adding our divergence term does a much better job of reducing the variability of our learned function over the entire space, rather than just having the eikonal term. The reason for this is that it constrains the local region more due to its second-order nature. This can also be motivated by the divergence theorem.³⁵ We use the following toy problem to demonstrate this.

Toy example: Consider the problem of learning the SDF to the line $y = 0$ where below the line is negative, i.e., $\Phi((x, y)) = y$. We train a 2-layer SIREN with point constraints (A) and eikonal term (C2) and then add our divergence constraint for comparison. For point constraints, we sample 10 points on the lines $y = -1$, $y = 0$, and $y = 1$ (for $x \in \{0, 1\}$), and for eikonal and divergence constraints, we sample on an $n \times n$ grid. We repeat the experiment for $n \in \{20, 200\}$ and evaluate our learned functions on a finer $m \times m$ grid ($m = 1000$). We perform 20 repetitions of these 4 experiments with different random initialisations.

The results can be seen in Table 1, where we report the mean Dirichlet energy $\bar{E} = \overline{\|\nabla f\|_2^2}$, the mean gradient norm $\overline{\|\nabla f\|_2}$, and its standard deviation $\sigma(\|\nabla f\|_2)$. We also visualise the learnt SDFs in Figure 9 and a larger view for one of the repetitions in Figure 10. The results show that adding the divergence constraint both reduces the mean Dirichlet energy, indicating that the learned function is less variable, and makes the function more faithful to the eikonal constraint. This can also be seen in the visualisation, where the level set contours are less variable and have more consistent spacing.

Table 1. Toy example results. Comparing mean Dirichlet energy and mean gradient norm. The divergence term significantly improves performance.

Loss	Grid size	$\bar{E} = \overline{\ \nabla f\ _2^2}$	$\overline{\ \nabla f\ _2}$	$\sigma(\ \nabla f\ _2)$
A+C2	20×20	4.32 ± 3.07	1.63 ± 0.51	1.05 ± 0.56
A+C2 + Div	20×20	3.37 ± 3.89	1.46 ± 0.76	0.69 ± 0.43
A+C2	200×200	3.58 ± 3.55	1.51 ± 0.54	0.85 ± 0.52
A+C2 + Div	200×200	1.37 ± 1.35	1.04 ± 0.39	0.22 ± 0.33

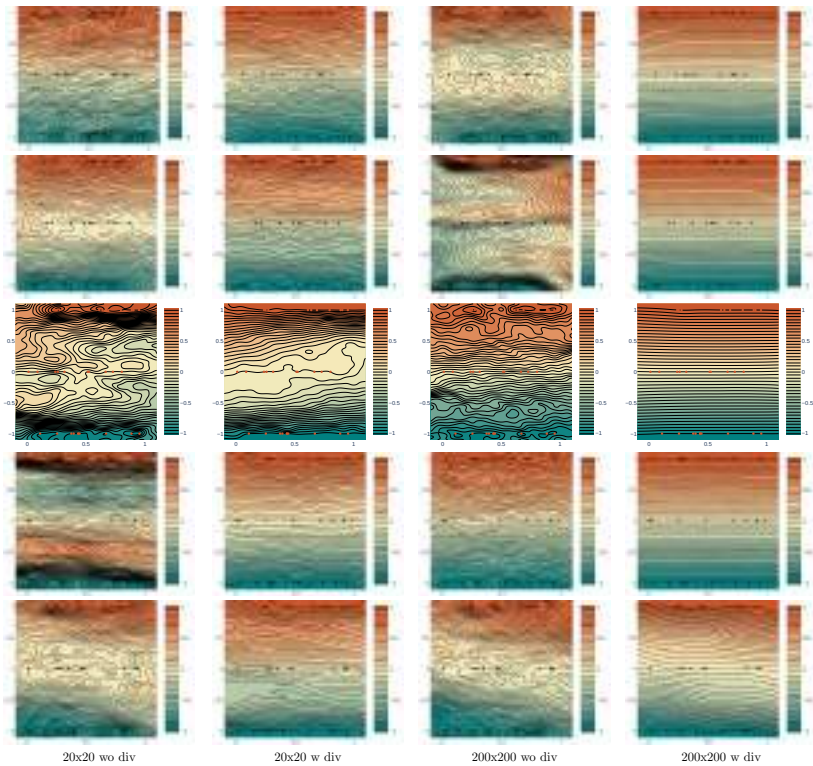


Fig. 9. Five repetitions of the toy example. Contour lines and colouring show the learned function. Black dots on the lines $y = -1$, $y = 0$, and $y = 1$ show the point constraints. When the divergence term is present, the contour lines are more smooth and the spacing is more uniform, as desired. When the divergence term is absent, sometimes the sign of the function is considerably incorrect, with negative above $y = 0$ and positive below. Note that when the sampling for the point constraints is not uniform, e.g., the last row where the constraints are almost clustered in a diagonal, the learned function is often biased.

6. Experiments

We evaluate our method on the tasks of surface reconstruction and shape space learning. We test the former on the SRB,¹² ShapeNet,¹⁴ and a scene from Sitzmann *et al.*¹⁰ and the latter on DFaust.¹³ In both tasks, we extract the mesh representing the zero-level set of the shape INR using the *marching cubes* algorithm.⁴¹ We follow the same mesh generation procedure as in IGR⁹ and use a grid whose shortest

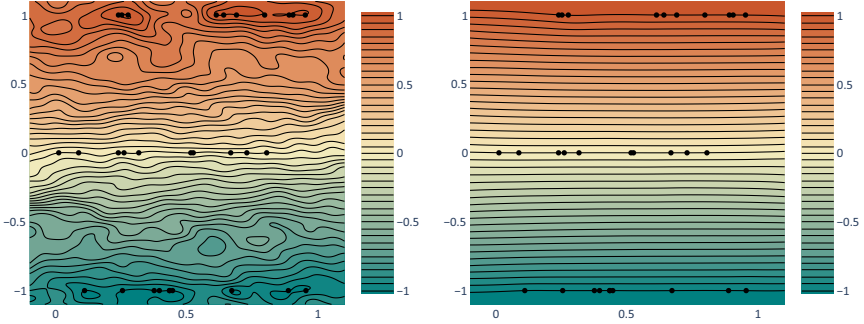


Fig. 10. Toy example SDF contour lines. Results are reported for a 200×200 grid. With (right) and without (left) the divergence loss term. It shows that the divergence term encourages the desired, smooth SDF properties.

axis is 512 elements, tightly fitted onto the shape (adapting the grid range to the input scan bounding box). Unless otherwise specified, we use the same architecture (5 layers, 256 units) and hyperparameters as SIREN.¹⁰

6.1. Evaluation Metrics

To compare between two point sets $\mathcal{X}_1, \mathcal{X}_2 \subset \mathbb{R}^3$, we use the Chamfer (d_C) and Hausdorff (d_H) distances from Williams *et al.*²⁰ For the ShapeNet dataset, instead of this Chamfer distance, we report the squared Chamfer to be consistent with previous works.^{2,27} Note that these metrics compare the accuracy of the predicted surface of the shape. To compare between the implicit function learnt and the ground-truth mesh, we use the volumetric Intersection over Union (IoU) of the interior of the shapes as per Mescheder *et al.*³ Note that this metric compares the underlying occupancy (interior) predicted by the method.

We define the

$$d_C(\mathcal{X}_1, \mathcal{X}_2) = \frac{1}{2}(d_{\vec{C}}(\mathcal{X}_1, \mathcal{X}_2) + d_{\vec{C}}(\mathcal{X}_2, \mathcal{X}_1)), \quad (32)$$

$$d_H(\mathcal{X}_1, \mathcal{X}_2) = \max(d_{\vec{H}}(\mathcal{X}_1, \mathcal{X}_2), d_{\vec{H}}(\mathcal{X}_2, \mathcal{X}_1)), \quad (33)$$

where

$$d_{\vec{C}}(\mathcal{X}_1, \mathcal{X}_2) = \frac{1}{|\mathcal{X}_1|} \sum_{x_1 \in \mathcal{X}_1} \min_{x_2 \in \mathcal{X}_2} \|x_1 - x_2\|_2, \quad (34)$$

$$d_{\vec{H}}(\mathcal{X}_1, \mathcal{X}_2) = \max_{x_1 \in \mathcal{X}_1} \min_{x_2 \in \mathcal{X}_2} \|x_1 - x_2\|_2 \quad (35)$$

are the one-directional Chamfer distance and one-directional Hausdorff distance, respectively.

We define the Chamfer distance used in Park *et al.*² as the squared Chamfer distance, given by

$$d_C^{sq}(\mathcal{X}_1, \mathcal{X}_2) = d_{\vec{C}}^{sq}(\mathcal{X}_1, \mathcal{X}_2) + d_{\vec{C}}^{sq}(\mathcal{X}_2, \mathcal{X}_1), \quad (36)$$

$$d_{\vec{C}}^{sq}(\mathcal{X}_1, \mathcal{X}_2) = \frac{1}{|\mathcal{X}_1|} \sum_{x_1 \in \mathcal{X}_1} \min_{x_2 \in \mathcal{X}_2} \|x_1 - x_2\|_2^2. \quad (37)$$

For the volumetric IoU, following Mescheder *et al.*,³ we obtain unbiased estimates of the occupied (interior) volume of the shapes by randomly sampling 100k points $\mathcal{X}$ in the space. Thus, given the known occupancy of the ground-truth mesh $O_{GT}(x) \in \{0, 1\}$ and the predicted SDF $\Phi(x) \in \mathbb{R}$ for a points $x \in \mathcal{X}$, the occupancy of the SDF is given by

$$O_\Phi(x) = \begin{cases} 1 & \Phi(x) < 0 \\ 0 & \text{otherwise} \end{cases} \quad (38)$$

and the IoU is given by

$$IoU_{\mathcal{X}}(O_{GT}, O_\Phi) = \frac{\sum_{x \in \mathcal{X}} O_{GT}(x) \text{ and } O_\Phi(x)}{\sum_{x \in \mathcal{X}} O_{GT}(x) \text{ or } O_\Phi(x)}. \quad (39)$$

In our experiments, we follow IGR⁹ and sample 1M points on the reconstruction and compare to the GT and scan point clouds. For ShapeNet, we follow NSP²⁷ and report the squared Chamfer and Intersection over Union (IoU).

6.2. Understanding the Divergence Constraint in 2D

We perform a qualitative analysis of the proposed approach on 2D simple shapes: circle, L-shaped polygon, and Koch’s snowflake polygon. In this experiment, we trained a network with 4 layers and 128 elements in each layer with sine activation functions for 10K epochs, sampling a new set of points in each epoch. We compare SIREN,¹⁰ SIREN without normal vectors, and the proposed DiGS approach with the proposed MFGI initialisation and without it. The results are shown in Figures 11 and 12 where a heatmap is used to visualise the learned distance function, the eikonal term, the divergence, the curl, and the difference between the unsigned predicted distance and the ground-truth distance. For the circle shape, incorporating the divergence constraint without decay yields the best result since the circle is a smooth closed shape without fine detail. In contrast, Koch’s snowflake is characterised by sharp edges, therefore starting with the divergence constraint and annealing it yields the best performance. In this case, the divergence term guides the learning process to a smoothed version of the snowflake, and the annealing allows it to fit the geometry more tightly. SIREN without the normal vectors exhibits ghost geometries (zero-level sets that should not appear), while DiGS does not.

The initialisation significantly affects the sign of the distance function as well as the model’s ability to properly reconstruct fine detail, particularly for the snowflake example.

6.3. Surface Reconstruction

The task of surface reconstruction of unoriented point clouds is specified as follows. Given an input point cloud $\mathcal{X} \subset \mathbb{R}^3$, find the surface $\mathcal{S}$ from which $\mathcal{X}$ was sampled. The point set is usually acquired by a 3D scanner which introduces various types of data corruptions, e.g., noise, occlusions, and non-uniform sampling. We evaluate the performance of our method on the SRB dataset¹² and on ShapeNet.¹⁴

We compare our results to recent prominent methods, most of which have shown to outperform classical methods, including SAL,⁶ IGR,⁹ SIREN,¹⁰ DGP,²⁰ and NSP.²⁷ Note that apart from SAL, these methods report results using normal vector supervision, therefore for a fair comparison, we evaluate SIREN and IGR without the normal vector loss term (note that the same cannot be done with DGP and

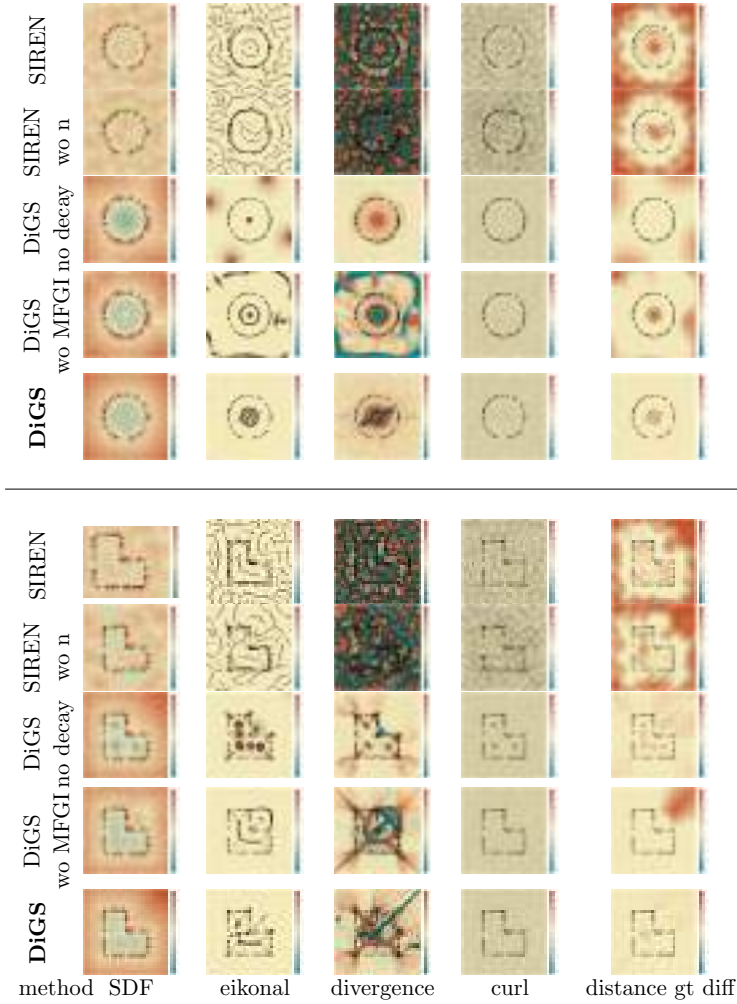


Fig. 11. Qualitative 2D results. Visualising informative quantities as heatmaps over the evaluation space: (left to right) sign distance function, divergence, eikonal term, curl, and distance difference between ground truth and inference for the Circle and L shapes.

NSP). Furthermore, we compare to results reported from the concurrent work PHASE,²⁹ which evaluates on SRB. While PHASE uses normal information, they also provide a version without normals that uses Fourier features, PHASE+FF, and a baseline for IGR without normal information, IGR+FF.

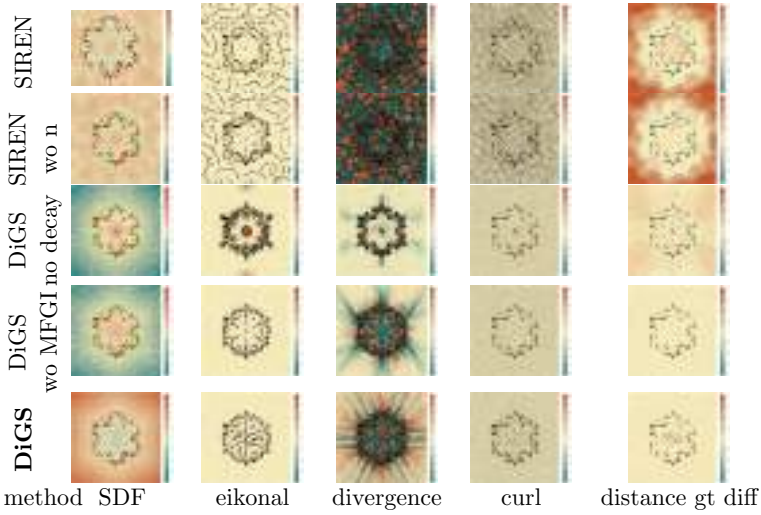


Fig. 12. Qualitative 2D results. Visualising informative quantities as heatmaps over the evaluation space: (left to right) sign distance function, divergence, eikonal term, curl, and distance difference between ground truth and inference for the Koch snowflake shape.

6.4. Surface Reconstruction Benchmark

We use the simulated scan and ground-truth data provided in Williams *et al.*²⁰ (freely available for academic use). The dataset contains five shapes, each with its own challenging traits, e.g., complex topology, high level of detail, missing data, and different feature sizes. In Table 2, we report the Chamfer and Hausdorff distances for each shape. It shows that our method is consistently better than SoTA methods on both metrics when normal information is not available. Additionally, we achieve improved performance when using normal vector supervision compared to a vanilla SIREN and show comparable results to other methods. Qualitative results for this dataset are shown in Figure 13. In the absence of normal information, SIREN and IGR struggle to converge to the correct zero-level set and produce undesired artefacts (ghost geometries). DiGS, on the other hand, is able to remove such artefacts. When compared to the version with normal supervision added, there is not much change other than DiGS being slightly smoother. In fact, DiGS manages to get similar results to the methods that use normal supervision. It is clear

Table 2. Surface reconstruction results on SRB. Reporting Chamfer d_C and Hausdorff d_H distances. We compare methods with normal supervision above the line and without normal supervision below the line. The *scans* column reports the one-sided distances ($d_{\bar{C}}, d_{\bar{H}}$) between the reconstruction and the simulated scans which give a measure of the reconstruction’s overfit to the noisy input.

Method	Mean		Anchor				Daratech				DC				Gargoyle				Lord Quas			
	GT		GT		Scans		GT		Scans		GT		Scans		GT		Scans		GT		Scans	
	d_C	d_H	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$
DGP	0.21	5.18	0.33	8.82	0.08	2.79	0.20	3.14	0.04	1.89	0.18	4.31	0.04	2.53	0.21	5.98	0.06	3.41	0.14	3.67	0.04	2.03
IGR	0.19	2.99	0.23	4.71	0.12	1.32	0.25	4.01	0.14	1.59	0.17	2.22	0.09	2.61	0.18	2.85	0.1	1.29	0.12	1.17	0.07	0.98
SIREN	0.19	3.86	0.31	7.32	0.11	1.23	0.21	4.74	0.09	1.85	0.15	2.37	0.07	2.71	0.17	4.26	0.09	0.82	0.12	0.62	0.08	0.81
NSP	0.17	2.85	0.22	4.65	0.11	1.11	0.21	4.35	0.08	1.14	0.14	1.35	0.06	2.75	0.16	3.20	0.08	2.75	0.12	0.69	0.05	0.62
PHASE	0.16	2.77	0.21	4.29	0.09	1.23	0.18	2.92	0.08	1.80	0.15	2.52	0.05	2.78	0.16	3.14	0.07	1.09	0.11	0.96	0.04	0.96
DiGS + n	0.18	3.55	0.28	5.71	0.11	1.14	0.21	5.02	0.09	1.75	0.15	2.13	0.06	2.74	0.16	3.81	0.09	0.90	0.12	1.1	0.06	0.77
IGR wo n	1.38	16.33	0.45	7.45	0.17	4.55	4.9	42.15	0.7	3.68	0.63	10.35	0.14	3.44	0.77	17.46	0.18	2.04	0.16	4.22	0.08	1.14
SIREN wo n	0.42	7.67	0.72	10.98	0.11	1.27	0.21	4.37	0.09	1.78	0.34	6.27	0.06	2.71	0.46	7.76	0.08	0.68	0.35	8.96	0.06	0.65
SAL	0.36	7.47	0.42	7.21	0.17	4.67	0.62	13.21	0.11	2.15	0.18	3.06	0.08	2.82	0.45	9.74	0.21	3.84	0.13	4.14	0.07	4.04
IGR+FF	0.96	11.06	0.72	9.48	0.24	8.89	2.48	19.6	0.74	4.23	0.86	10.3	0.28	3.98	0.26	5.24	0.18	2.93	0.49	10.7	0.14	3.71
PHASE+FF	0.22	4.96	0.29	7.43	0.09	1.49	0.35	7.24	0.08	1.21	0.19	4.65	0.05	2.78	0.17	4.79	0.07	1.58	0.11	0.71	0.05	0.74
DiGS	0.19	3.52	0.29	7.19	0.11	1.17	0.20	3.72	0.09	1.80	0.15	1.70	0.07	2.75	0.17	4.10	0.09	0.92	0.12	0.91	0.06	0.70

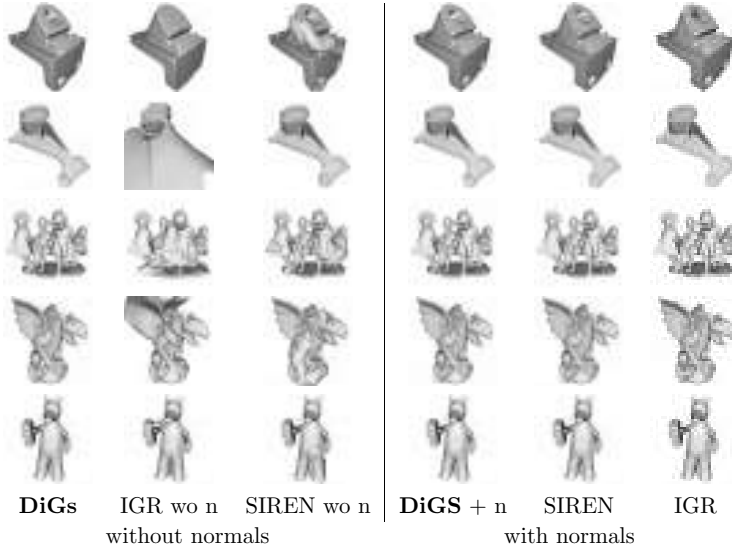


Fig. 13. Surface reconstruction qualitative results. Evaluated the anchor, daratec, dc, gargoyle, and lord quas shapes (top to bottom) from the Surface Reconstruction Benchmark.¹² Compared to state of the art approaches (IGR, SIREN) that use normal vectors as ground truth.

that whenever ground-truth normal vectors are not available, DiGS presents a significant improvement and yields comparable results to normal based methods.

Implementation details: The input point cloud is first centred to zero and scaled to have a maximum norm of one. Then a bounding box that is 1.1 times the size of the shape is selected. For each iteration, we sample 15,000 points from the original point cloud and sample 15,000 points uniformly randomly in a bounding box. We train for 10,000 iterations with a learning rate of 5×10^{-5} .

6.5. ShapeNet

The ShapeNet dataset contains 3D CAD models of a diverse range of objects. These shapes often have internal structures, inconsistent normals, and non-manifold meshes. The results are reported in Table 3 and we provide visualisations in Figure 14. When adding normal supervision to DiGS, our method has a much better

Table 3. Surface reconstruction results on ShapeNetm. Methods above the line use ground-truth normal information, and methods below do not. The mean, median, and standard deviation of the squared Chamfer distance and IoU of all 260 shapes are reported.

Method	Squared Chamfer ↓			IoU ↑		
	Mean	Median	Std	Mean	Median	Std
SPSR ⁴²	2.22e-4	1.70e-4	1.76e-4	0.6340	0.6728	0.1577
IGR ⁹	5.12e-4	1.13e-4	2.15e-3	0.8102	0.8480	0.1519
SIREN ¹⁰	1.03e-4	5.28e-5	1.93e-4	0.8268	0.9097	0.2329
FFN ²⁶	9.12e-5	8.65e-5	3.36e-5	0.8218	0.8396	0.0989
NSP ²⁷	5.36e-5	4.06e-5	3.64e-5	0.8973	0.9230	0.0871
DiGS + n	2.74e-4	2.32e-5	9.90e-4	0.9200	0.9774	0.1992
SIREN wo n	3.08e-4	2.58e-4	3.26e-4	0.3085	0.2952	0.2014
SAL ⁶	1.14e-3	2.11e-4	3.63e-3	0.4030	0.3944	0.2722
Our DiGS	1.32e-4	2.55e-5	4.73e-4	0.9390	0.9764	0.1262

median-squared Chamfer distance and median IoU among the 260 shapes. We attribute this to our divergence term smoothing out the space, which does well on shapes without much internal structure. However, for some shapes with internal structure (e.g., a loudspeaker with components inside or a sofa with structural beams), we get significant internal ghost geometry. As a result, our mean-squared Chamfer distance, while competitive with the other methods, is worse than methods, such as SIREN. Note, however, that this does not affect the IoU as much, our mean IoU is still better than other methods. When comparing without normals, DiGS has similar medians on both metrics to when normal supervision is added, however, it has better means. We attribute this to having fewer internal ghost geometries when not attempting to fit normal vectors at internal points. Note that DiGS outperforms other methods that do not use normal supervision on both metrics. The improvement in IoU is significant, which we attribute to other methods failing to contain ghost geometry and being inconsistent with what is inside/outside the shape. On the other hand, DiGS appropriately deals with both of these challenges by its structured training procedure.

Implementation details: We use the pre-processing and evaluation method from Williams *et al.*²⁷ who evaluate on 20 shapes in each

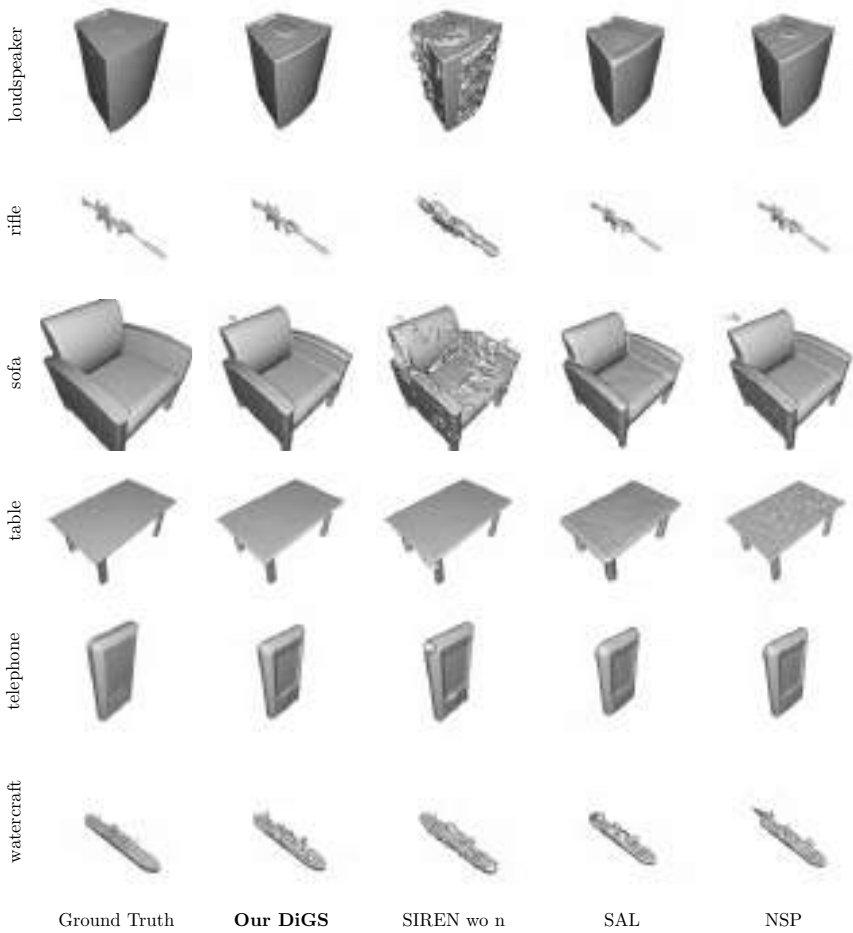


Fig. 14. ShapeNet qualitative results. One shape from each class is shown compared to other methods.

of 13 categories and pre-process to make the normals consistent and internal structure to be manifold meshes. They first pre-process using the method from Mescheder *et al.*³ and then report on the first 20 shapes of the test set for each shape class. The pre-processing extracts ground-truth surface points from the shapes of ShapeNet v1¹⁴ and extracts random samples within the space with their labelled occupancy values. The evaluation method uses the ground-truth points to calculate squared Chamfer distance and uses the labelled random

samples to calculate IoU. Note that the initial ShapeNet data has inconsistent normal orientation and many non-manifold surfaces due to its nature as CAD models, and this pre-processing helps orient the normals and remove most of the non-manifold surfaces.

6.6. Scene Reconstruction

Qualitative results for scene reconstruction are presented in Figure 15. Here, we use 8 layers with 512 units and train on the scene from Sitzmann *et al.*¹⁰ which includes 10M oriented points. We train SIREN with and without the normal constraint and the proposed DiGS method. In this experiment, we did not use a geometric initialisation because, unlike the shape reconstruction task, the target surface is vastly different than a sphere and instead only increase the frequencies. When training SIREN without normal supervision, we observe many ghost geometries (SIREN wo n). On the other hand, DiGS is able to reconstruct the scene without these, though it produces a very smooth result. This is desirable in most planar regions, e.g., ceiling, floor, and table, however, this trait has the drawback of smoothing out fine details, e.g., sofa legs and picture frame.

6.7. Shape Space Learning

Shape space experiments require training a single model to learn to represent multiple shapes from a class of related shapes. We use the DFAUST dataset¹³ which consists of $\sim 40k$ raw human scans of

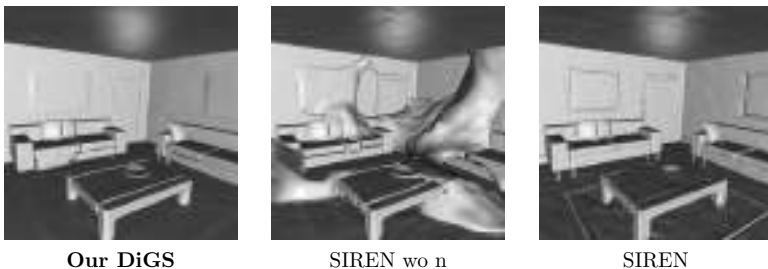


Fig. 15. Scene reconstruction qualitative results. SIREN (right) provides high level of detail, however, when normal vectors are not available (centre), the proposed divergence constraint (left) demonstrates a significant improvement.

10 humans at different time points during multiple types of activities. The scans are noisy and often incomplete (containing missing object surfaces). The dataset also provides ground-truth registrations for each scan. Following IGR’s⁹ setup, we use their 75%/25% train-test split, and we use DiGS as an autodecoder.² Thus at training time, multiple scans of the humans are learnt with a separate latent vector for each pose, and at test time, unseen poses of those same humans are reconstructed by jointly estimating a suitable latent vector. Note this is a much harder problem than surface reconstruction; the model has to learn to be able to fit multiple shapes given different (learnable) latent codes, stretching the test of the model’s capacity.

Table 4 shows the quantitative results of one-sided Chamfer distances. In particular, for registration and scan to reconstruction DiGS+n has a lower mean than IGR, showing it fits better to the ground truth and input surfaces and does not miss ground-truth regions. For reconstruction to registration and scan the mean distances increase for both IGR and DiGS+n, indicating that both models create ghost geometry, however DiGS+n creates much less. Both of these are demonstrated in Figure 16: despite IGR sometimes having more detail (e.g., the face), it has multiple ghost geometries and significant missing parts (e.g., forearms). DiGS+n also achieves slightly smaller median distances.

For the methods without normals supervision, DiGS clearly outperforms IGR w/o n; the latter is not able to converge. This can be seen in Figure 16, where the reconstructions do not even resemble the initial humans. DiGS on the other hand captures the human shapes but is oversmoothed and has large ghost geometries framing

Table 4. Quantitative results on DFaust. We compare the mean and median of the one-sided Chamfer distances (reported as $\times 10^2$) between the ground-truth registration meshes (reg), reconstructions (recon), and raw input scans (scan).

	$d_{\bar{C}}(\text{reg, recon})$		$d_{\bar{C}}(\text{recon, reg})$		$d_{\bar{C}}(\text{scan, recon})$		$d_{\bar{C}}(\text{recon, scan})$	
	Mean	Median	Mean	Median	Mean	Median	Mean	Median
IGR ⁹	1.053	0.509	4.916	0.540	1.054	0.509	4.916	0.540
DiGS + n	0.568	0.458	1.834	0.461	0.568	0.458	1.834	0.461
IGR w/o n	3.745	2.689	12.149	9.027	3.745	2.687	12.147	9.026
Our DiGS	0.856	0.707	12.318	9.202	0.856	0.707	12.319	9.204

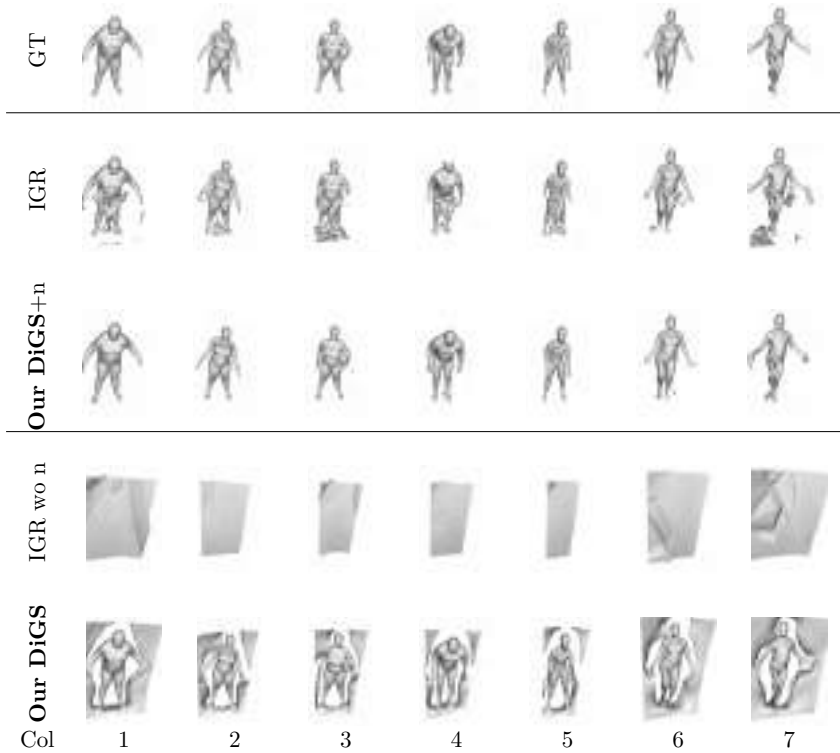


Fig. 16. Qualitative results for the shape space experiment on the DFaust dataset.¹³ Both methods with normals, IGR and DiGS+n, manage to capture the shape of the humans. IGR has more detail but has a lot of ghost geometries (all columns), sometimes changes orientation that is far away from the mean of the dataset (column 4) and often misses large surfaces, such as forearms (columns 3 and 5). DiGS+n captures the correct surface with minimal ghost geometry and few missing regions but oversmooths fine details (e.g., facial features). On the other hand, for methods without normals (DiGS and IGR wo n), only DiGS is able to learn multiple human shapes, whereas IGR wo n is not able to learn at all. DiGS also has large ghost geometries and oversmooths the human surfaces but manages to capture more key regions compared to IGR.

the humans. Looking at Table 4, the means and medians for registration and scan to reconstruction are still quite small, showing that despite oversmoothing, the detail DiGS learns to fit to the unseen test surfaces quite well. Due to the ghost geometry, for the reconstruction to registration and scan distances, DiGS has very large values, similar to IGR wo n.

Implementation details: For our network, we use an 8-layer SIREN with hidden layers of dimension 256. We also use 256 dimensional latent codes. At test time, we optimise for the latent vectors for 800 iterations using Adam with a learning rate of 10^{-3} . The latent code and the original input ((x, y, z) coordinates) are concatenated together for a 259-dimensional input, and the latent codes are initialised to zero. As is standard in an autoencoder, we use latent code regularisation,² with a scaling weight of 1.0. For IGR, we use their model (autoencoder with 8 layers, hidden dimension 512, and latent dimension 256) and use the same latent optimisation procedure. For the version with normals, we use their trained weights, and for the version without normals, we train their network using their provided training script.

Ablation study: We investigate the effects of several design choices made for DiGS and report the averages over all shapes in the SRB dataset in Table 5. First, we investigate the influence of the annealing function τ . We compare the case of the following:

- (1) No annealing:

$$\tau = 1. \quad (40)$$

Table 5. DiGS ablation study. Evaluated on the surface reconstruction benchmark.¹² We compare design choices such as annealing method (linear, step, and no annealing), divergence term penalties (L_1 vs. L_2), initialisation method (MFGI vs. SIREN), and curvature ground truth. We use the Chamfer d_C and Hausdorff d_H distance metrics.

Method	GT		Scans	
	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$
SIREN w/o n	0.42	7.67	0.08	1.42
DiGS L_1	0.20	4.47	0.07	1.77
DiGS L_2	0.25	5.18	0.07	1.67
DiGS no-decay	0.21	4.45	0.09	2.58
DiGS lin-decay	0.20	4.41	0.07	1.64
DiGS curv	0.30	4.83	0.10	1.57
DiGS L_1 MFGI	0.19	3.52	0.08	1.47

(2) Linear annealing:

$$\tau_{\text{Lin}}(t_0, t_1, \tau_1) = \begin{cases} 1 & t < t_0 \\ 1 + (\tau_1 - 1) \frac{(t-t_0)}{t_1-t_0} & t_0 \leq t \leq t_1 \\ \tau_1 & t > t_1 \end{cases}. \quad (41)$$

(3) Step annealing:

$$\tau_{\text{Step}}(t_0, \tau_1) = \begin{cases} 1 & t < t_0 \\ \tau_1 & t \geq t_0 \end{cases}. \quad (42)$$

Here, t_0 and t_1 are expressed as a fraction of training iterations. In our experiments, we used the following values: $(t_0, t_1, \tau_1) = (0.5, 0.75, 0)$. Results show that annealing improves performance with little difference between linear and step annealing.

We also investigate the choice of penalty function over the divergence term (with step decay) and compare between L_1 and L_2 . Here, L_1 allows for sparse spatial locations to have high divergence (which is desired because there may be some source or sink point in the gradient vector field) while L_2 provides an average low value over the volume. As expected, the results show an advantage for using L_1 penalty on the divergence.

Furthermore, we evaluate the second-order supervision approach presented in Section 5.2.1. For that, we estimate the mean curvature using DeepFit³⁸ (a recent state-of-the-art method for estimating normals and curvatures) and introduce the mean curvature as ground-truth information during training. The results show that, surprisingly, curvature supervision does not provide any benefit. This is due to the noisiness and local support dependency of curvature estimation which may have over smoothed or jittery values that make training inconsistent.

Finally, we investigate the effect of the multi-frequency geometric initialisation (with step decay and L_1 penalty) and show that it produces consistently better performance than all other ablations. This variant is denoted throughout this chapter as DiGS, i.e., L_1 penalty on the divergence term with step decay and MFGI (Section 4). Note that all ablations were trained without normal vector information.

6.8. *Use of DiGS Loss with Other Activation Functions*

Our divergence loss is general and can be applied to any network that has second-order derivatives defined everywhere. Note that this means it cannot be used with ReLU activations, though we can use smooth approximations, such as the SoftPlus activation function (which IGR⁹ uses).

However, our loss is targeted at high-frequency architectures, such as SIRENs. A drawback of such architectures is the trade-off between high fidelity detail and ghost geometries, especially without normal vector supervision. Our loss is particularly beneficial to alleviate this trade-off.

We experimented with using our loss paired with SoftPlus and found that it does not provide a boost in performance since the network is already biased towards low-frequency solutions (performance reduction of 0.99 d_C , and 6.34 d_H on SRB) which *SIREN* *wo n* outperforms even without the divergence loss performance boost. Furthermore, we experiment with Gaussian activations due to their advantages of high performance and simplicity suggested by Ramasinghe and Lucey¹¹ for images. This experiment was conducted on the SRB dataset where we trained an INR with normal supervision, with normal and divergence supervision and only with divergence supervision. Results are reported in Table 6 which show that the divergence term is beneficial for training Gaussian-activated INRs. In our experiments, we noticed that, unlike the results reported by Ramasinghe and Lucey,¹¹ these INRs are highly sensitive to initialization and activation parameter choices.

6.9. *Limitations, Experimental Setup, and Timing*

DiGS is mainly limited in two aspects: (1) capturing very thin structures, e.g., the left sofa’s legs in Figure 15, and (2) smoothing effects, e.g., the pictures on the walls in Figure 15. This is expected since these regions contain only very few points and without the normal vector information, uncovering the underlying surface is more challenging.

Table 6. Gaussian activation INR divergence effects. Results on the Surface Reconstruction Benchmark using Chamfer d_C and Hausdorff d_H distances. Comparing Gaussian activations with different types of supervision.

Loss	Mean		Anchor				Daratech					
	GT		GT		Scans		GT		Scans			
	d_C	d_H	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$		
n	0.278	5.399	0.333	7.754	0.136	2.418	0.338	7.240	0.111	1.770		
n + div	0.230	4.394	0.262	7.061	0.125	1.156	0.237	3.791	0.105	1.795		
div	0.3128	5.801	0.370	7.817	0.141	1.884	0.349	7.968	0.133	1.597		
Loss	DC				Gargoyle				Lord Quas			
	GT		Scans		GT		Scans		GT		Scans	
	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$	d_C	d_H	$d_{\bar{C}}$	$d_{\bar{H}}$
n	0.396	7.042	0.160	2.858	0.186	4.015	0.125	0.982	0.139	0.944	0.098	0.981
n + div	0.262	3.630	0.134	2.469	0.257	4.122	0.119	1.080	0.131	3.369	0.093	0.876
div	0.280	4.136	0.155	3.219	0.308	5.170	0.179	1.779	0.257	3.915	0.168	1.730

Table 7. Time performance results per iteration. Comparing standard SIREN (excluding the normal estimation stage time) to DiGS. Results are reported in milliseconds (ms).

Method	SIREN	IGR	DiGS
Time [ms]	5.2	17.5	12.0
# parameters	66.5 K	2.1 M	66.5 K

The surface reconstruction experiments on the SRB dataset¹² were implemented in PyTorch and trained on a single Nvidia RTX 2080 GPU. The architecture for 2D experiments of our method is a 4-layer MLP with sinusoidal activation (SIREN) with 128 nodes in each layer. Our method does not add parameters to the standard SIREN approach.

Note that due to computing the divergence term, there is an increase in computation time which is mainly attributed to computing (and backpropagating) the gradient of the gradient. At inference time, SIREN and DiGS have the same time and computation performance. Note that to train the standard SIREN, a pre-processing normal estimation stage is required. The number of parameters and timing performance are reported in Table 7 for the 2D reconstruction networks. It shows the increase in training time (per iteration) for DiGS compared to SIREN, however, DiGS is still faster and has fewer parameters compared to IGR.

7. Conclusion

We introduce eDiGS, an extended divergence-guided shape implicit neural representation approach for raw unoriented point clouds without any pre-processing. Additionally, we derive a geometrically motivated initialisation for sinusoidal representation networks while preserving high frequencies. Finally, we demonstrate that DiGS has the ability to reconstruct shapes of high fidelity with some limitations of smoothing and thin features reconstruction. We report state-of-the-art results compared to other methods that do not use normal supervision and show that our method is comparative to methods that do use such supervision.

All existing methods, including ours, struggle with missing data and thin features. Future work can explore extensions to use local self-similarity to deal better with these regions. Point cloud density is also an important factor in the representation power and additional work can be done to mitigate density effects. Lastly, shape representation networks are highly dependent on the initialisation and further work should be done to explore this direction.

References

1. Y. Ben-Shabat, C. H. Koneputugodage, and S. Gould. DiGS: Divergence guided shape implicit neural representation for unoriented point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 19323–19332 (2022).
2. J. J. Park, P. Florence, J. Straub, R. Newcombe, and S. Lovegrove. DeepSDF: Learning continuous signed distance functions for shape representation. In *CVPR*, pp. 165–174 (2019).
3. L. Mescheder, M. Oechsle, M. Niemeyer, S. Nowozin, and A. Geiger. Occupancy networks: Learning 3D reconstruction in function space. In *CVPR*, pp. 4460–4470 (2019).
4. S. Peng, M. Niemeyer, L. Mescheder, M. Pollefeys, and A. Geiger. Convolutional occupancy networks. In *ECCV* (2020).
5. Z. Chen and H. Zhang. Learning implicit fields for generative shape modeling. In *CVPR*, pp. 5939–5948 (2019).
6. M. Atzmon and Y. Lipman. SAL: Sign agnostic learning of shapes from raw data. In *CVPR*, pp. 2565–2574 (2020).
7. M. Atzmon and Y. Lipman. SALD: Sign agnostic learning with derivatives. In *ICLR* (2021).
8. D. Urbach, Y. Ben-Shabat, and M. Lindenbaum. DPDist: Comparing point clouds using deep point cloud distance. In *ECCV*, pp. 545–560 (2020).
9. A. Gropp, L. Yariv, N. Haim, M. Atzmon, and Y. Lipman. Implicit geometric regularization for learning shapes. In *ICML*, Vol. 119, pp. 3789–3799. PMLR (2020).
10. V. Sitzmann, J. Martel, A. Bergman, D. Lindell, and G. Wetzstein. Implicit neural representations with periodic activation functions. In *NeurIPS*, Vol. 33 (2020).
11. S. Ramasinghe and S. Lucey. Beyond periodicity: Towards a unifying framework for activations in coordinate-MLPs. In *European Conference on Computer Vision*, pp. 142–158 (2022).

12. M. Berger, J. A. Levine, L. G. Nonato, G. Taubin, and C. T. Silva. A benchmark for surface reconstruction, *ACM Transactions on Graphics* **32**, 1–17 (2013).
13. F. Bogo, J. Romero, G. Pons-Moll, and M. J. Black. Dynamic FAUST: Registering human bodies in motion. In *CVPR*, pp. 6233–6242 (2017).
14. A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su et al., ShapeNet: An information-rich 3D model repository. *arXiv preprint* arXiv:1512.03012 (2015).
15. F. Cazals and J. Giesen. Delaunay triangulation based surface reconstruction. In *Effective Computational Geometry for Curves and Surfaces*, pp. 231–276. Springer (2006).
16. R. Kolluri, J. R. Shewchuk, and J. F. O’Brien. Spectral surface reconstruction from noisy point clouds. In *Proceedings of the 2004 Eurographics/ACM SIGGRAPH symposium on Geometry Processing*, pp. 11–21 (2004).
17. F. Bernardini, J. Mittleman, H. Rushmeier, C. Silva, and G. Taubin. The ball-pivoting algorithm for surface reconstruction, *IEEE Transactions on Visualization and Computer Graphics* **5**, 349–359 (1999).
18. N. Amenta, S. Choi, and R. K. Kolluri. The power crust, unions of balls, and the medial axis transform, *Computational Geometry* **19**, 127–153 (2001).
19. T. Groueix, M. Fisher, V. G. Kim, B. C. Russell, and M. Aubry. A papier-mâché approach to learning 3D surface generation. In *CVPR*, pp. 216–224 (2018).
20. F. Williams, T. Schneider, C. Silva, D. Zorin, J. Bruna, and D. Panozzo. Deep Geometric Prior for surface reconstruction. In *CVPR*, pp. 10130–10139 (2019).
21. Y. Jiang, D. Ji, Z. Han, and M. Zwicker. SDFDiff: Differentiable rendering of signed distance fields for 3D shape optimization. In *CVPR*, pp. 1251–1261 (2020).
22. M. Tatarchenko, A. Dosovitskiy, and T. Brox. Octree generating networks: Efficient convolutional architectures for high-resolution 3D outputs. In *ICCV*, pp. 2088–2096 (2017).
23. S. Peng, C. Jiang, Y. Liao, M. Niemeyer, M. Pollefeys, and A. Geiger. Shape As points: A differentiable poisson solver. *arXiv preprint* arXiv:2106.03452 (2021).
24. M. Berger, A. Tagliasacchi, L. M. Seversky, P. Alliez, G. Guennebaud, J. A. Levine, A. Sharf, and C. T. Silva. A survey of surface reconstruction from point clouds. In *Computer Graphics Forum*, Vol. 36, pp. 301–329 (2017).

25. M. Michalkiewicz, J. K. Pontes, D. Jack, M. Baktashmotlagh, and A. Eriksson. Implicit surface representations as layers in neural networks. In *ICCV*, pp. 4743–4752 (2019).
26. M. Tancik, P. P. Srinivasan, B. Mildenhall, S. Fridovich-Keil, N. Raghavan, U. Singhal, R. Ramamoorthi, J. T. Barron, and R. Ng. Fourier features let networks learn high frequency functions in low dimensional domains. In *NeurIPS* (2020).
27. F. Williams, M. Trager, J. Bruna, and D. Zorin. Neural splines: Fitting 3D surfaces with infinitely-wide neural networks. In *CVPR*, pp. 9949–9958 (2021).
28. W. Yifan, L. Rahmann, and O. Sorkine-Hornung. Geometry-consistent neural shape representation with implicit displacement fields. *arXiv preprint* arXiv:2106.05187 (2021).
29. Y. Lipman. Phase transitions, distance functions, and implicit neural representations. *arXiv preprint* arXiv:2106.07689 (2021).
30. V. Sitzmann, M. Zollhöfer, and G. Wetzstein. Scene representation networks: Continuous 3D-structure-aware neural scene representations. In *NeurIPS* (2019).
31. C. Jiang, A. Sud, A. Makadia, J. Huang, M. Nießner, T. Funkhouser *et al.* Local implicit grid representations for 3D scenes. In *CVPR*, pp. 6001–6010 (2020).
32. D. Azinovic, R. Martin-Brualla, D. B. Goldman, M. Nießner, and J. Thies. Neural RGB-D surface reconstruction. *arXiv preprint* arXiv:2104.04532 (2021).
33. C. Williams and K. Duru. Provably stable full-spectrum dispersion relation preserving schemes. *arXiv preprint* arXiv:2110.04957 (2021).
34. J. E. Marsden and A. Tromba. *Vector Calculus*. 6th edn. W. H. Freeman. Macmillan (2003).
35. P. M. Morse and H. Feshbach. Methods of theoretical physics, *American Journal of Physics* **22**, 410–413 (1954).
36. F. Cazals and M. Pouget. Estimating differential quantities using polynomial fitting of osculating jets, *Computer Aided Geometric Design* **22**, 121–146 (2005).
37. P. Guerrero, Y. Kleiman, M. Ovsjanikov, and N. J. Mitra. PCPNet: Learning local shape properties from raw point clouds. In *Computer Graphics Forum*, Vol. 37, pp. 75–85 (2018).
38. Y. Ben-Shabat and S. Gould. DeepFit: 3D surface fitting via neural network weighted least squares. In *ECCV*, pp. 20–34 (2020).
39. Y. Ben-Shabat, M. Lindenbaum, and A. Fischer. Nesti-net: Normal estimation for unstructured 3D point clouds using convolutional neural networks. In *CVPR*, pp. 10112–10120 (2019).

40. M. M. Bronstein, J. Bruna, Y. LeCun, A. Szlam, and P. Vandergheynst. Geometric deep learning: Going beyond Euclidean data, *IEEE Signal Processing Magazine* **34**, 18–42 (2017).
41. W. E. Lorensen and H. E. Cline. Marching cubes: A high resolution 3D surface construction algorithm. In *SIGGRAPH*, Vol. 21, pp. 163–169. ACM, New York, USA (1987).
42. M. Kazhdan and H. Hoppe. Screened Poisson surface reconstruction, *ACM Transactions on Graphics* **32**, 1–13 (2013).

Chapter 6

Improving Monocular 3D Object Detection by Synthetic Images with Virtual Depth

Chenhang He* and Lei Zhang†

*Department of Computing,
The Hong Kong Polytechnic University, Hong Kong*

**csche@comp.polyu.edu.hk*

†cslzhang@comp.polyu.edu.hk

Exploiting geometric features is a common approach to enhance monocular 3D object detection. However, their performance is limited due to the absence of depth information. To address this limitation, an external depth estimator can be employed to predict depth, but this approach significantly reduces the efficiency and flexibility of the model. Instead of relying on a costly depth estimator, we propose a depth-aware monocular 3D object detector that is trained using augmented training data. Specifically, we utilise reference images and their corresponding depth maps to train an efficient rendering module, which synthesises a variety of photo-realistic images with different virtual depths. By learning from these images, the detector adapts its features to depth variations. Furthermore, we introduce an auxiliary module that guides the network to learn more informative representations from the depth images. Both modules are removed after training, resulting in no additional computational overhead during the final deployment.

1. Introduction

3D object detection plays a crucial role in autonomous driving as it enables accurate detection and localisation of objects in the 3D space, ensuring safe navigation over long distances for autonomous vehicles. While high-end LiDAR sensors are extensively utilised in autonomous vehicles due to their ability to directly perceive the depth of the surrounding visual environment, there are some limitations associated with LiDAR. LiDAR sensors can be expensive, susceptible to adverse weather conditions, and often generate sparse point clouds for small objects (Figure 1(a)). In contrast, cameras offer a more cost-effective solution and provide high-resolution images that are beneficial for perceiving through challenging weather conditions and offer better visibility for small objects (Figure 1(b)). This makes monocular detection a promising solution for self-driving systems.

While significant progress has been made in monocular 3D object detection research in recent years, there still exists a performance gap compared to LiDAR-based detection.^{1–8} This gap can be attributed to the inaccurate depth estimation of monocular images. Estimating object depth from a single image is inherently challenging. Current research on monocular 3D object detection can be categorised

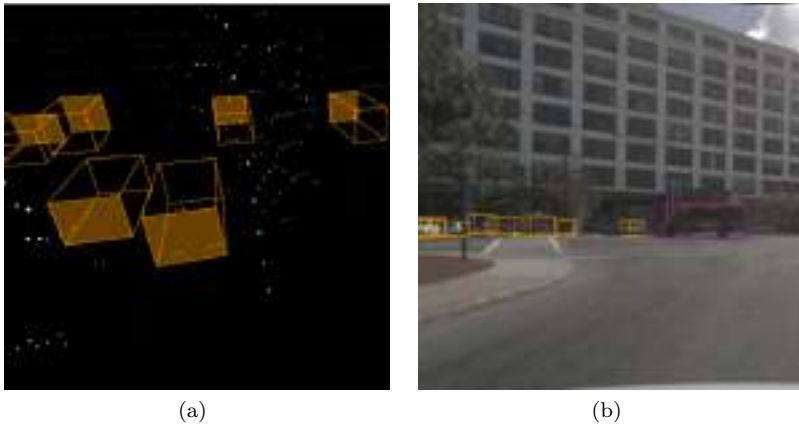


Fig. 1. (a) The point clouds produced by LiDAR often result in sparse representations, especially for objects at a distance, which can make it challenging to perceive those objects accurately. (b) In contrast, high-resolution images provide better perception quality for these small objects.

into two approaches: geometric constraint-based method^{9–22} and depth estimation-based method.^{23–32} The former aims to predict various geometric features based on perspective transform and detects objects by fitting these features within a perspective constraint. Effective geometric features establish a robust relationship between object depth and 2D image features. For instance, distant objects tend to appear small and located in the upper region of the image, while close objects tend to be larger and appear in the lower region. Features related to anchor boxes with different scales and locations in the image encode prior information to detect objects with varying depths and truncation levels. Figure 2(a) showcases the detection results obtained using M3D-RPN,¹⁰ a method that utilises a set of 2D–3D anchors to encode depth cues. This encoding enables the model to reliably estimate object depth, size, and orientation from 2D image features.

While geometric constraints can imply object depth information, they often lack depth information from background pixels. The depth of the background area provides scene geometry, which is valuable for inferring object depth. One line of approaches^{23–32} utilises a depth estimator to explicitly estimate depth for the detected image. Typically, the depth estimator is trained on an external image dataset that includes depth ground-truth. The dense depth predictions from the depth estimator provide the detection model with meaningful context

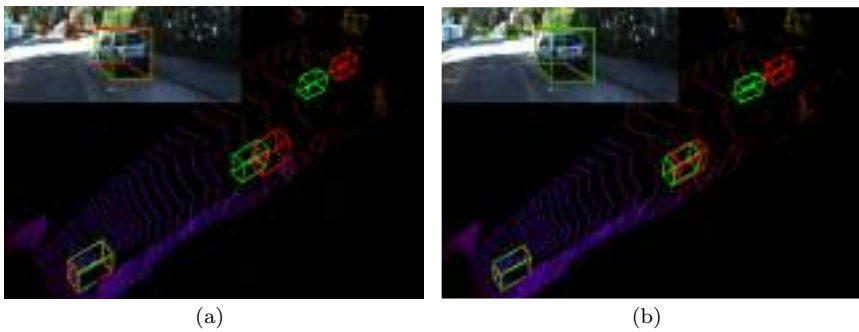


Fig. 2. (a) Bounding box predictions by *M3D-RPN*,¹⁰ where the model can accurately predict the size and orientation of the objects but fail to accurately localise the objects at far distances. (b) Bounding box predictions by *M3D-RPN*, learnt with synthetic virtual-depth images. The predicted and ground-truth boxes are shown in green and red, respectively.

to reason about more reliable bounding boxes. The depth predictions are then converted into pseudo LiDAR representations,^{26–31,33} fused with image features,²³ or served as a guidance to control the kernel size of the convolutional network.²⁵ However, integrating an external depth estimator into the detection pipeline at test time makes it challenging for depth estimation-based methods to achieve real-time speed. Additionally, depth estimators often have strong biases towards the external training data, and the domain gap between the detection dataset and the external dataset may result in unreliable depth estimation for the detection models.

In this study, we propose a novel learning framework to enhance 3D object detection using depth images. Instead of training a cumbersome depth estimation model, we utilise depth images to augment the input data by synthesising various images during the training stage. Unlike traditional 2D data augmentation techniques, we synthesise training images at different virtual depths (Figure 3). This approach enables the detection model to learn features that are robust to depth variations while avoiding the reliance on a depth estimator during the testing stage. As shown in Figure 2(b), the model trained with synthesised images can better estimate object depth and achieve more accurate bounding box detection.

To synthesise images with virtual depth, we drew inspiration from traditional novel-view synthesis methods^{34–37} and introduce an efficient rendering module. This module leverages the depth image from LiDAR to reconstruct the 3D scene of the reference image and generates novel images from a virtual camera that is shifted along

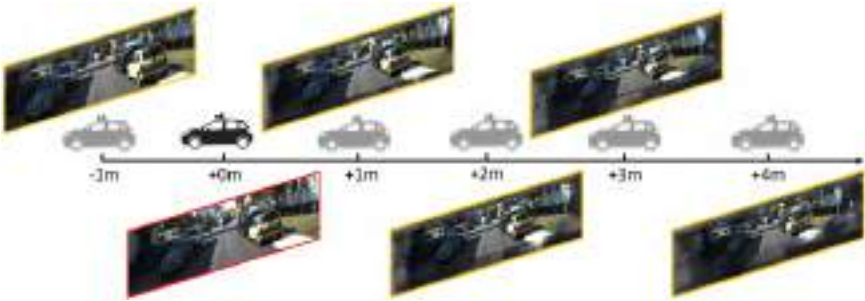


Fig. 3. Examples of reference image (in red) and its corresponding synthetic virtual-depth images (in yellow) with relative virtual depth -1 , $+1$, $+2$, $+3$, and $+4$ meters.

the depth axis. The ground-truth bounding boxes are also shifted based on the camera displacement, resulting in synthetic training pairs as augmented data. It is important to note that there are several challenges for a detection model to learn from these synthetic images. First, the depth images used are often very sparse, causing the synthetic images to contain numerous holes and limited meaningful semantics. Second, the depth images may not align perfectly with the camera images due to calibration errors. Unmatched depth pixels around object boundaries can lead to highly distorted rendered contents and ghost artefacts in the synthetic images. To address these issues, we propose a multi-stage rendering module that generates high-quality synthesised images in a coarse-to-fine manner.

Additionally, we introduce an auxiliary module that works in conjunction with the backbone of the detection model. This auxiliary module utilises the backbone features for depth estimation and jointly optimises them with a pixel-wise loss. By doing so, the detection model can gain a better understanding of scene geometry and achieve more reliable 3D bounding box predictions:

- A novel data augmentation strategy is proposed to improve the generalisation ability of 3D object detection models by generating synthetic images with virtual depth.
- An efficient rendering module is presented to synthesise photo-realistic images from coarse to fine.
- An auxiliary module is developed to jointly optimise the detection features through a pixel-level depth estimation task, leading to more accurate bounding-box reasoning.

Our proposed learning framework was applied to existing 3D object detectors and demonstrated consistent performance gains on the KITTI 3D/BEV detection benchmark.

2. Related Work

Geometric constraint-based monocular detection: The effective utilisation of geometric priors can greatly enhance the performance of monocular 3D object detection. Brazil *et al.*¹⁰ introduced a 2D–3D anchoring mechanism that utilises 2D image features for 3D bounding box detection. Mousavian *et al.*¹¹ and Naiden *et al.*¹²

employed the 2D–3D perspective constraint to stabilise 3D bounding box predictions. Liu *et al.*¹³ and Li *et al.*¹⁷ proposed estimating 3D bounding boxes based on a group of keypoints. Qin *et al.*¹⁴ and Zhou *et al.*¹⁵ decomposed the detection task into multiple subtasks and optimised them in a global context. Chen *et al.*¹⁶ utilised spatial constraints between objects and their occluded neighbours, incorporating a pair-distance function to regularise bounding box predictions. Roddick *et al.*⁹ converted 2D image features to a bird’s-eye view using orthographic transformation, providing a holistic view of image features in 3D space. Zhang *et al.*³⁸ proposed decoupling predictions for normal objects and truncated objects. Chen *et al.*¹⁹ presented a self-supervised framework for simultaneously learning dense correspondences and geometry. Peng *et al.*²¹ reformulated object depth as a combination of visual surface depth and instance attribute depth. Gu *et al.*²² introduced a homography loss to explore the mutual relationship of all objects in a scene. Li *et al.*²⁰ employed projection constraints on multiple depth estimation candidates and merged these candidates via graph matching for more accurate depth estimation.

In this chapter, we demonstrate that our proposed synthetic images can effectively complement multiple geometric constraint-based detectors^{10,13,17} and generally improve their detection performance.

Depth estimation-based monocular detection: Depth images can effectively compensate for the lack of 3D cues in 2D images. Pseudo LiDAR-based approaches^{26,27,33} explored converting depth images from monocular inputs into pseudo LiDAR representations, which better capture the 3D structure of objects using existing point cloud detectors. Qian *et al.*²⁸ devised a soft-quantisation technique to convert depth images into voxel inputs for point cloud detectors, enabling end-to-end learning. Ma *et al.*²⁹ conducted an in-depth investigation of pseudo LiDAR representation and proposed an efficient scheme based on 2D CNN with coordinate transformation. In addition to pseudo LiDAR conversion, Xu *et al.*²³ introduced a multi-level fusion algorithm that progressively aggregated features from RGB and depth images. Ding *et al.*²⁵ proposed a depth-aware model, where the convolutional receptive fields were dynamically adjusted by the depth image to process image contents at different depth levels.

Unlike the aforementioned methods that require depth estimation during inference, our proposed framework only utilises depth information during the training phase, resulting in a more efficient detection process.

Novel-view synthesis: Novel-view synthesis has been extensively explored in both the vision and graphics communities. Early approaches^{39–42} primarily focused on direct image-to-image transformations, allowing for end-to-end view manipulation. Subsequent approaches^{34–37,43,44} shifted their focus towards learning scene geometry, such as point clouds, multi-plane images, or implicit surfaces. These methods enable the generation of continuous novel views with high degrees of freedom using the estimated 3D representations.

In this study, considering the sensitivity of monocular detectors to depth changes, we synthesise images by fixing the target view along the z -axis, which corresponds to the depth direction. This approach allows us to avoid the need for extrapolating views and complementing unknown information from unobserved perspectives.

3. Methodology

3.1. Overall Learning Framework

Our proposed learning framework is depicted in Figure 4. It consists of three primary components: a detection model, a rendering module, and an auxiliary depth estimation module. The rendering module takes an RGB image and its corresponding sparse depth image as input, generating a set of synthetic images at various virtual depths. These synthetic images are utilised as augmented data to train the detector. On the other hand, the depth estimation module enhances the backbone features of the detection model by jointly optimising them with a pixel-wise loss through a depth prediction task. Both the rendering and auxiliary modules are removed after training, and only the learned detector is deployed. In this chapter, we demonstrate the adaptability of this learning framework to various existing monocular detectors.^{10,13,17,18} We generally refer to the detection model with our augmented design as AugMono3D. The subsequent subsections provide a detailed introduction to each component of AugMono3D and the training scheme.

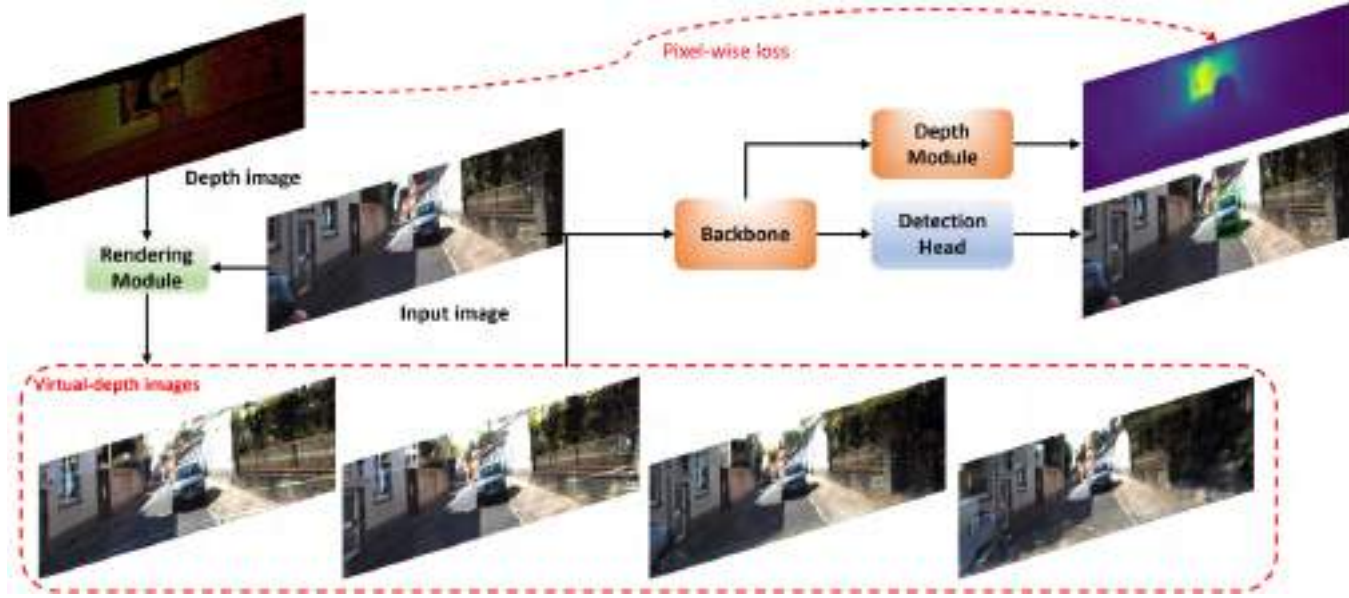


Fig. 4. The overview of the proposed learning framework. The framework contains a detection model, a rendering module to synthesise virtual-depth images, and an auxiliary depth module to transform the backbone feature for dense depth prediction.

3.2. *Synthesising Virtual-Depth Images for Data Augmentation*

To enhance the ability of the monocular detector in discriminating object depth, we augment the training data by generating synthetic images with virtual depth. The overall pipeline is illustrated in Figure 5, where we leverage depth information to perform virtual view synthesis and utilise an inpainting network to fill in the pixels with missing values.

We begin by mapping the reference image into a 3D point cloud based on the non-zero values of its corresponding depth image D . For each position (u, v) in the reference image, the 3D coordinates of the point cloud can be calculated as follows:

$$(x, y, z) = \left(\frac{u - c_x}{f_x}, \frac{v - c_y}{f_y}, 1 \right) \cdot \mathbb{1}[D_{(u,v)} > 0]. \quad (1)$$

Here, (f_x, f_y) and (c_x, c_y) represent the focal length and principal points of the reference camera, respectively. $\mathbb{1}$ represents the indicator function. The virtual-depth image is then synthesised by projecting the point cloud from the current camera coordinates to the image plane in the virtual view. Using the camera projection matrix $\mathbf{P}$, the position (u', v') of each point after projection can be calculated as

$$(z + \Delta z) \cdot [u' \ v' \ 1] = \mathbf{P} \cdot [x \ y \ z + \Delta z \ 1]. \quad (2)$$

The rendering process can be seen as transporting each coloured pixel from (u, v) to (u', v') .



Fig. 5. (a) The pipeline of rendering virtual-depth image. For background images, we first apply sparse rendering to generate virtual images with plausible semantics and then an in-painting network to fill-in the missing areas. For foreground areas, we warp the 2D viewport of 3D boxes based on the perspective transformation from two camera poses.

However, due to the extremely sparse nature of the depth image, the output image obtained through the sparse rendering method is predominantly composed of black pixels. The limited semantic information in these renderings poses a challenge for training the inpainting network. Moreover, some depth images may not align well with the reference images due to inevitable calibration errors, leading to unmatched pixels that distort the appearance of objects and introduce ghost artefacts at object boundaries.

To tackle these challenges, we incorporate a pre-rendering step to enhance the density of the renderings. Specifically, we “unfold” the reference image into a contextual image by stacking the 3×3 neighbouring pixels of each position into a single channel. If the original image has a shape of $H \times W \times 3$, the contextual image will have a shape of $H \times W \times 27$. We then apply the aforementioned rendering formula to extract a sparse version of the contextual image and subsequently “fold” the output back into the RGB space. As illustrated in Figure 6, the renderings obtained from the contextual image exhibit higher density and contain more meaningful content.

Regarding foreground objects, since they primarily consist of rigid bodies, we approximate their depth values using their bounding boxes. Specifically, we emit camera rays and check each ray for intersections with all bounding boxes. We transform the camera ray from camera coordinates to the local coordinates of the bounding box and perform an Axis-Aligned Bounding Box (AABB) intersection test, as proposed by Alexander *et al.*⁴⁵ This test calculates entrance and exit points for each ray-box pair. Next, we sort the points from the same ray based on their z -values and select the nearest points as the depth values for the corresponding objects. With these depth values, we can calculate the appearance flow for each foreground pixel using Equations (1) and (2) and remap them onto the synthetic image.



Fig. 6. The naive renderings (left) and renderings from contextual image (right).

As illustrated in Figure 7, the rendered foreground objects have clear boundaries and consistent aspect ratios.

Subsequently, we employ a standard U-Net architecture with partial convolutions⁴⁶ to complete the colour pixels in the missing areas. To train the inpainting network, we utilise the detection images as training targets and their pre-renderings with $\Delta z = 0$ as training inputs. We employ pixel-wise ℓ_1 loss, perceptual loss, and adversarial loss $\mathcal{L}_{\text{adv}}$ to supervise the inpainting network. For a reference image I_{ref} and its pre-renderings I_{pre} , we have

$$\mathcal{L} = \|I_{\text{ref}} - I_{\text{pre}}\|_1 + \|\phi(I_{\text{ref}}) - \phi(I_{\text{pre}})\|_2^2 + \mathcal{L}_{\text{adv}}(I_{\text{ref}}, I_{\text{pre}}). \quad (3)$$

Here, ϕ represents the hidden feature activation from a pre-trained VGG-16 classification network and $\mathcal{L}_{\text{adv}}$ denotes a relativistic adversarial loss from Yu *et al.*⁴⁷ where both a global and a local discriminator are employed for perception enhancement.

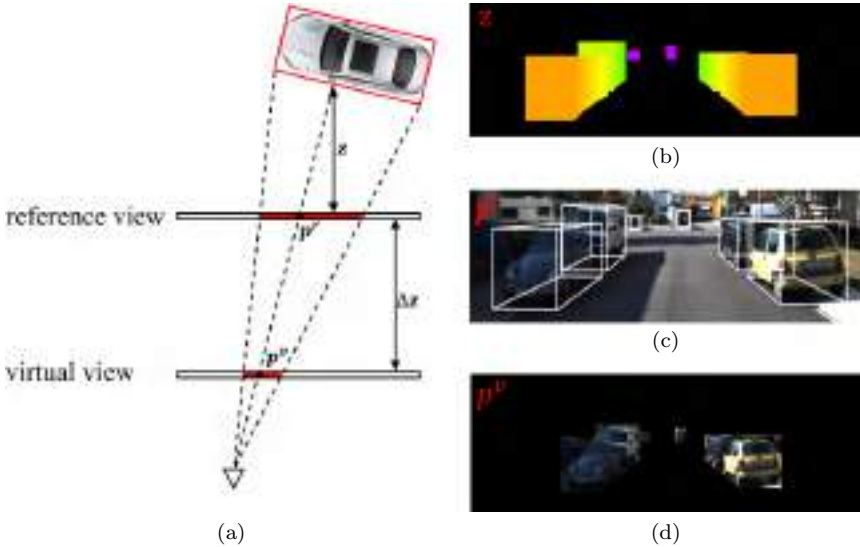


Fig. 7. (a) Ray-box intersection for calculating the object depth. (b) The foreground depth map by sorting the nearest points (with the smallest z values) from the ray-box intersections. (c) The projection of bounding box onto the image. (d) The foreground renderings on virtual image.

3.3. Joint Depth Estimation for Feature Augmentation

One of the core ideas of this work is utilising pixel-level supervision to improve the understanding of object-level task, which has been proven effective in many prior arts.^{6,48,49} Based on this, we propose a detachable auxiliary module to augment the backbone features by jointly optimising depth estimation and object detection tasks. In particular, we use a pyramidal architecture to deploy our auxiliary module so that it can aggregate multi-scale detection features into a full-resolution representation. For ResNet-101, we fuse the outputs from the first convolutional layer *Conv1* and four residual blocks *layer1*, *layer2*, *layer3*, and *layer4*.

As depicted in Figure 8, we upsample the feature from the bottom stage with bilinear interpolation. The features across stages are first concatenated and then undergo a 3×3 convolutional layer to generate features with 256 channels. In the final stage, one additional 1×1 convolution is applied to produce depth estimation $\tilde{D}$. Since the foreground components and background components have different depth distributions, we applied the following weighted loss to jointly optimise the detection and depth estimation tasks:

$$\mathcal{L} = \mathcal{L}_{\text{det}} + \lambda_{fg} \frac{1}{N_i} \mathcal{L}(\tilde{D}_i, D_i) + \lambda_{bg} \frac{1}{N_j} \mathcal{L}(\tilde{D}_j, D_j), \quad (4)$$

where $\mathcal{L}_{\text{det}}$ is the detection loss in Brazil and Liu,¹⁰ D is the ground-truth depth image, i and j respectively represent the non-zero foreground and background pixels in the ground-truth depth image.

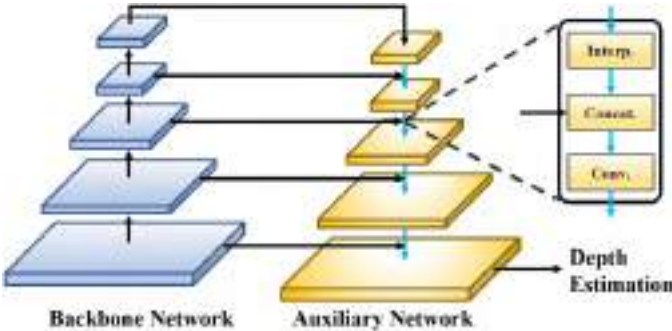


Fig. 8. The plug-and-play auxiliary network for depth estimation.

We found that by setting $\lambda_{fg} = 2.5$, $\lambda_{bg} = 1$, and $\mathcal{L}$ with Smooth- l_1 ⁵⁰ loss, our detection model can generally achieve the best performance.

3.4. 3D Object Reasoning with 2D–3D Detection Head

The detection head predicts both 2D and 3D geometries of the bounding boxes based on a set of pre-defined 2D–3D anchors and then associates these geometries according to the perspective relationship.

Specifically, we define a 2D–3D anchor by $(a_{2D}^w, a_{2D}^h) - (a_{3D}^w, a_{3D}^h, a_{3D}^l, a^z, a^\theta)$, where (a_{2D}^w, a_{2D}^h) denote its width and height in image scale and $(a_{3D}^w, a_{3D}^h, a_{3D}^l)$, a^z , and a^θ represent its 3D dimensions, depth, and observation angle in camera coordinates, respectively. Given 2D predictions $(p_{2D}^u, p_{2D}^v, p_{2D}^w, p_{2D}^h)$, the 2D bounding box $(b_{2D}^x, b_{2D}^y, b_{2D}^w, b_{2D}^h)$ at each anchor position (u, v) , can be written as

$$b_{2D}^x = u + a_{2D}^w p_{2D}^u, \quad b_{2D}^y = v + a_{2D}^h p_{2D}^v, \quad (5)$$

$$b_{2D}^w = e^{a_{2D}^w p_{2D}^w}, \quad b_{2D}^h = e^{a_{2D}^h p_{2D}^h}. \quad (6)$$

Given 3D predictions $(p_{3D}^w, p_{3D}^h, p_{3D}^l, p^{\Delta z}, p^\theta)$, the dimension of 3D bounding box $(b_{3D}^h, b_{3D}^w, b_{3D}^l, b^\theta)$, in terms of height, width, length, and observation angle, can be given as follows:

$$b^w = e^{a^{w3d} p^{w3d}}, \quad b^h = e^{a^{h3d} p^{h3d}}, \quad (7)$$

$$b^l = e^{a^{l3d} p^{l3d}}, \quad b^\theta = \Omega(a^\theta, p^\theta). \quad (8)$$

Its location $(b_{3D}^x, b_{3D}^y, b_{3D}^z)$ in camera coordinates therefore subjects to

$$(a^z + p^{\Delta z}) \cdot [b_{2D}^x, b_{2D}^y, 1]^T = \mathbf{P} \cdot [b_{3D}^x, b_{3D}^y, b_{3D}^z, 1]^T, \quad (9)$$

where $\mathbf{P} \in \mathbb{R}^{3 \times 4}$ is the camera projection and $\Omega(\cdot)$ serves to round the radian value within $[-\pi, \pi]$.

4. Experiments

We use KITTI dataset⁵¹ to evaluate our proposed AugMono3D. The dataset contains 7,481 training samples and 7,518 testing samples.

To evaluate the effectiveness of different modules, we use the splits in Chen *et al.*⁵² by dividing the training set into 3,712 training samples and 3,769 validation samples.

There are two detection tasks, known as 3D detection (3D) and birds-eye-view localisation (BEV) tasks. Each task has three levels of difficulties: *easy*, *moderate*, and *hard*, according to the object size, occlusion, and truncation level, and the ranking is based the average precision (AP) on the *moderate* level. The AP with 11 points⁵³ and 40 points⁵⁴ interpolation are respectively denoted by AP11 and AP40.

4.1. Implementation Details

We first extract the depth images by projecting LiDAR points onto image plane. Both depth images and RGB images are padded to 384×1248 and their mirrored versions are randomly sampled with a probability of 0.25 during training. For each ground-truth 3D box, we calculate its 2D box by projecting it onto the image plane. We ignore those objects which have 2D boxes less than 16 pixels or larger than 256 pixels in height and those with visibility less than 0.5.

To build the anchor classification target, we calculate the IoU score between the 2D anchors and 2D ground truths. An IoU threshold of 0.5 is used to label the anchors as positive or negative. The anchors that have IoU greater than 0.5 with the ignore region will be ignored. Besides, we apply online hard negative mining (OHNM) in single-shot detector⁵⁰ to re-balance the positive and negative anchor targets with a ratio of 1:3.

To create virtual-depth images, we ignore those objects that are truncated by camera or have visibility less than 0.6. By removing the depth values of these objects, our in-painting network will complete their area as background. For each training iteration, we sample two camera displacements from a uniform distribution $\mathcal{U}(0, 5)$ and one camera displacement from $\mathcal{U}(-1, 0)$. The later is to balance the distribution of objects that are truncated by camera.

Our AugMono3D detector is trained for 60 epochs by using the SGD optimiser. The batch size, learning rate, and weight decay are set to 4, 0.004, and 0.0005, respectively. The learning rate is decayed with a cosine annealing strategy.⁵⁹ In the inference phase, we select the predicted boxes with confidence larger than 0.75 and

apply non-maximum suppression (NMS) with an IoU threshold of 0.4 to remove the duplicated boxes.

4.2. Comparison with the State of the Art

We compare our AugMono3D detector with other state-of-the-art detectors by submitting the detection results to the KITTI server for evaluation. The evaluation is based on AP40 and the results are presented in Table 1. As can be seen, our method ranks higher than all previous methods in both 3D and BEV metric. We first evaluate the proposed AugMono3D with M3D-RPN¹⁰ as backend detector. The performance on easy level is comparable to Kinematic3D,⁵⁸ which is a multi-frame detector. As a geometric-constraint model, our backend model only runs at 80 ms per image, which is more than five times faster than the concurrent methods, such as AM3D (~ 400 ms), D4LCN (~ 600 ms), and PatchNet (~ 400 ms), as all these competitors are depth estimation-based detectors and they all rely on an expensive dense depth input. Comparing with other geometric constraint-based monocular 3D detectors, such as SMOKE¹³ and RTM3D,¹⁷ AugMono3D presents significant advantages. AugMono3D outperforms these methods by up to (3.1%, 2.7%, 1.0%) in 3D detection and (5.0%, 3.7%, 2.2%) in BEV detection. By employing GPU-Net¹⁸ as our backend model, AugMono3D still achieves leading performance on easy and hard level and achieves comparable performance to other state-of-the-art methods, such as HomoLoss²² and DID-M3D.²¹ AugMono3D is trained to be depth-aware without explicitly requiring depth estimation model in the testing stage, making it highly efficient and flexible for real-time processing.

The performance of AugMono3D with M3D-RPN backend on KITTI val set is also shown in Table 2. Again, our method can achieve leading performance on most detection metrics, which demonstrates the effective learning with synthetic images and auxiliary module.

4.3. Ablation Study

We first demonstrate our synthetic virtual-depth images can work well with other geometric constraint-based detectors. Here we choose three baseline detectors: M3D-RPN,¹⁰ SMOKE,¹³ and RTM3D.¹⁷

Table 1. Performance comparison with state-of-the-art methods on KITTI test set. BEV and 3D object detection metric are used, reported by AP40 (IoU>0.7). The top three performers are indicated by **red**, **blue**, and **green** colours, respectively. The runtime is reported from the KITTI leaderboard with slight variances in hardware.

Method	#Frames	3D			BEV			Runtime (s/img)
		Easy	Moderate	Hard	Easy	Moderate	Hard	
FQNet ⁵⁵	1	2.77	1.51	1.01	5.40	3.23	2.46	0.5
ROI-10D ⁵⁶	1	4.32	2.02	1.46	9.78	4.91	3.74	0.2
GS3D ⁵⁷	1	4.47	2.90	2.47	8.41	6.08	4.94	2.3
Shift R-CNN ¹²	1	6.88	3.87	2.83	11.84	6.82	5.27	0.25
MonoFENet ³⁰	1	8.35	5.14	4.10	17.03	11.03	9.05	0.15
MLF ²³	1	7.08	5.18	4.68	—	—	—	0.12
MonoGRNet ¹⁴	1	9.61	5.74	4.25	18.19	11.17	8.73	0.04
MonoPSR ³²	1	10.76	7.25	5.85	18.33	12.58	9.91	0.2
MonoPL ³³	1	10.76	7.50	6.10	21.27	13.92	11.25	—
MonoDIS ⁵⁴	1	10.37	7.94	6.40	17.23	13.19	11.12	—
M3D-RPN ¹⁰	1	14.76	9.71	7.42	21.02	13.67	10.23	0.16
SMOKE ¹³	1	14.03	9.76	7.84	20.83	14.49	12.75	0.03
MonoPair ¹⁶	1	13.04	9.99	8.65	19.28	14.83	12.89	0.06
AM3D ³¹	1	16.50	10.74	9.52	25.03	17.32	14.91	0.4
RTM3D ¹⁷	1	14.41	10.34	8.77	19.17	14.20	11.99	0.05
PatchNet ²⁹	1	15.68	11.12	10.17	22.97	16.86	14.97	0.4
D4LCN ²⁵	1	16.65	11.72	9.51	22.51	16.02	12.55	0.6 ²
Kinematic3D ⁵⁸	4	19.07	12.72	9.17	26.69	17.52	13.10	0.12
MonoRUN ¹⁹	1	19.65	12.30	10.58	27.94	17.34	15.24	0.07
MonoFlex ³⁸	1	19.94	13.89	12.07	28.23	19.75	16.89	0.03
GUPNet ¹⁸	1	22.26	15.02	13.12	30.29	21.19	18.20	—
HomoLoss ²²	1	21.75	14.94	13.07	29.60	20.68	17.81	0.04
DID-M3D ²¹	1	24.40	16.29	13.75	32.95	22.76	19.83	0.04
AugMono3D + M3D-RPN ¹⁰	1	17.82	12.99	9.78	26.00	17.89	14.18	0.08
AugMono3D + GPUNet ¹⁸	1	24.52	14.59	13.78	32.11	21.87	17.64	0.04

Table 2. 3D object detection performance on KITTI val set, reported by AP11. “—” means that the results are not available. The top 2 performers are indicated by red and blue colours, respectively.

Method	3D/BEV (IoU>0.7)		
	E	M	H
MonoPSR ³²	12.75/20.63	11.48/18.67	8.59/14.45
MonoDIS ⁵⁴	18.05/24.26	14.98/18.43	13.42/16.95
SMOKE ¹³	14.76/19.99	12.85/15.61	11.50/15.28
M3D-RPN ¹⁰	20.27/25.94	17.06/21.18	15.21/17.90
RTM3D ¹⁷	19.19/25.56	16.70/22.12	16.14/20.91
AM3D ³¹	32.23/—	21.09/—	17.26/—
D4LCN ²⁵	26.97/34.82	21.71/25.83	18.22/23.53
AugMono3D (ours)	28.53/36.27	22.61/28.76	18.34/23.70

All of them can achieve real-time efficiency and with neat architecture. By running their public codes, we present their performances on KITTI val set. As shown in Table 3, the model trained with virtual-depth images can consistently achieve around 2~3 point (in AP11) improvements on easy and moderate levels. The slight improvement on hard level is because most objects in this category have very few depth values due to high occlusion and long distance, which makes them hard to be rendered in the synthetic images.

Then we perform an in-depth analysis of how different components contribute to AugMono3D.

Effectiveness of synthetic virtual-depth images: As presented in Table 4, the model learning with synthetic images can achieve (3.3%, 3.0%, 1.7%) improvements by the baseline model (Exp. 2 vs. Exp. 1) and (3.4%, 3.2%, 1.0%) by the auxiliary-driven model (Exp. 6 vs. Exp. 5). On top of the auxiliary-driven model, we also evaluate how virtual-depth images can boost the overall depth estimation in pixel level. We calculate the absolute depth error (absError) within a reliable depth range (0m, 40m), the error rates (in meters) of each 10m interval are reported in Table 5. As can be seen, the model trained with synthetic images can exhibit more accurate depth estimation. Figure 9 presents a qualitative comparison by projecting the depth estimation into 3D space, in the form of pseudo LiDAR. As can be seen, the model trained with synthetic images obtains more

Table 3. Evaluation of 3D/BEV detection performance on KITTI val set, reported by AP11. The abbreviation “Syn.” indicates using synthetic images for training. “*” indicates the results are reproduced by the official public codes, which are slightly different from the results in the paper.

Method	3D (IoU>0.7)			BEV (IoU>0.7)			3D (IoU>0.5)			BEV (IoU>0.5)		
	Easy	Moderate	Hard	Easy	Moderate	Hard	Easy	Moderate	Hard	Easy	Moderate	Hard
M3D-RPN*	20.88	17.39	15.51	27.13	21.71	18.30	50.17	40.27	33.62	58.21	43.68	36.23
M3D-RPN + Syn.	24.41	20.52	17.13	31.86	26.12	20.12	54.56	43.72	35.25	63.68	47.99	36.79
Delta	3.53	3.13	1.62	4.73	4.41	1.82	4.39	3.45	1.63	5.47	4.31	0.56
SMOKE*	18.32	15.84	15.23	24.39	20.64	17.37	50.08	37.39	32.87	53.93	39.98	39.20
SMOKE + Syn.	21.46	18.31	16.60	28.11	24.12	18.99	54.23	41.02	35.22	56.98	44.55	40.47
Delta	3.14	2.47	1.37	3.72	3.48	1.62	4.15	3.63	2.35	3.05	4.57	1.27
RTM3D	19.19	16.70	16.14	25.96	21.88	18.88	54.97	42.68	36.95	60.98	45.74	42.93
RTM3D + Syn.	22.12	19.29	17.25	30.21	25.37	20.28	58.72	45.77	38.27	64.62	49.91	44.15
Delta	2.93	2.59	1.11	4.25	3.49	1.40	3.75	3.09	1.32	3.64	4.17	1.22

Table 4. Ablation experiments on the KITTI val set using AP11. “Syn.” means using synthetic images for training, and “Aux-fg” and “Aux-bg” refer to imposing auxiliary loss on the foreground and background image areas, respectively.

Exp.	Syn.	Aux-fg.	Aux-bg.	3D (IoU >0.7)		
				E	M	H
1				22.62	17.44	15.48
2	✓			25.89	20.45	17.22
3		✓		24.51	18.91	16.63
4			✓	23.14	17.98	16.25
5		✓	✓	25.16	19.48	17.31
6	✓	✓	✓	28.53	22.61	18.34

Table 5. The absolute depth error (in meters) in different depth intervals.

Method	0–10 m	10–20 m	20–30 m	30–40 m
w/o Syn.	2.05	3.29	5.07	9.41
w/Syn.	1.47	2.18	4.23	8.80
Relative Delta.	−28.3%	−33.7%	−16.6%	−6.5%

compact 3D points in the bounding-box boundary, which means that the depth prediction of the object is more accurate and consistent.

Effectiveness of auxiliary module: As shown in Table 4, the model guided by auxiliary module can earn additional (2.6%, 2.2%, 1.1%) points (Exp. 6 vs. Exp. 2). Actually, we can apply depth estimation loss separately on foreground/background regions of the input image. As shown in Table 4, applying loss on the foreground regions can result in (1.9%, 1.5%, 1.2%) performance gains, while applying loss on background regions can improve the performance by (0.5%, 0.5%, 0.8%). This reveals the fact that both image components are important to support the reasoning of object depth. By discriminating the depth of foreground object, the model can be aware of its appearance, therefore better inferring its orientation and local geometry. The background of images encode the geometry of the scene, which provides additional contextual cues to estimate the object depth.

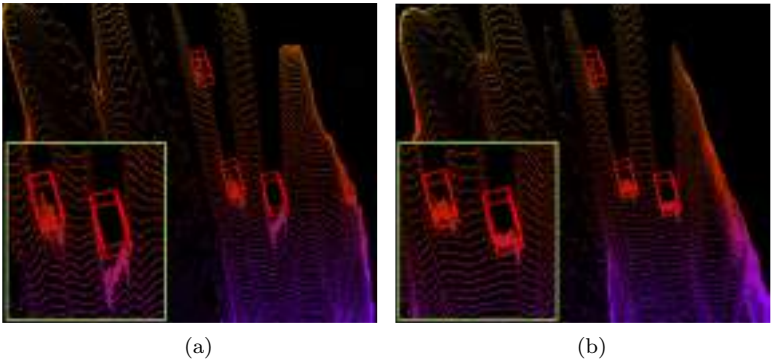
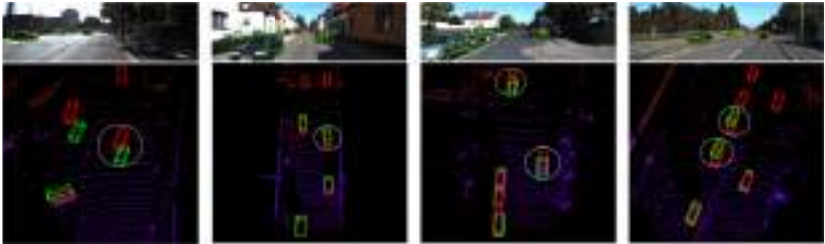
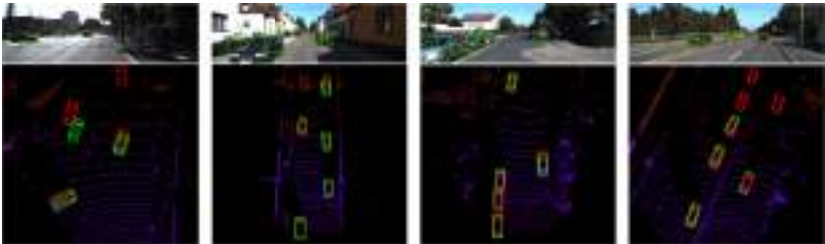


Fig. 9. The pseudo LiDAR generated by projecting the depth estimation from (a) Exp. 5 and (b) Exp. 6. The ground-truth bounding boxes are shown in red.



(a) The predictions by the baseline detection model.



(b) The predictions by the proposed AugMono3D model.

Fig. 10. Examples of detection results on KITTI val set. The predicted and ground-truth bounding boxes are shown in green and red colours, respectively. The predictions on 3D point cloud are plotted for better visualisation. Best viewed in colour.

Qualitative results: In Figure 10, we demonstrate some predictions from the baseline and full-setting models (Exp. 1 vs. Exp. 6). As can be observed, the detector resulted from our learning framework can predict more reliable 3D bounding boxes without having additional memory and runtime.

5. Conclusion

In this chapter, we propose an effective learning framework, which consists of a rendering module and an auxiliary module, to improve the monocular 3D object detection. The rendering module augments the training data by synthesising images with virtual depths, from which the detector can learn robust features that adapt to the depth changes of the objects. The auxiliary module can guide the detection model to better understand the scene geometry that is informative for object depth reasoning. Both modules significantly improve the detection accuracy and then are removed in the testing phase, adding no computational cost to the final deployment. Experiments on the KITTI 3D/BEV detection benchmark demonstrate the synthetic images can work well with existing monocular detectors and consistently improve their detection performance.

References

1. Y. Zhou and O. Tuzel. VoxelNet: End-to-end learning for point cloud based 3D object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4490–4499 (2018).
2. Y. Yan, Y. Mao, and B. Li. Second: Sparsely embedded convolutional detection, *Sensors* **18**(10), 3337 (2018).
3. S. Shi, X. Wang, and H. Li. PointRCNN: 3D object proposal generation and detection from point cloud. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–779 (2019).
4. A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom. PointPillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 12697–12705 (2019).
5. Z. Yang, Y. Sun, S. Liu, X. Shen, and J. Jia. STD: Sparse-to-dense 3D object detector for point cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2019).

6. C. He, H. Zeng, J. Huang, X.-S. Hua, and L. Zhang. Structure aware single-stage 3D object detection from point cloud. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2020).
7. S. Shi, C. Guo, L. Jiang, Z. Wang, J. Shi, X. Wang, and H. Li. PV-RCNN: Point-voxel feature set abstraction for 3D object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2020).
8. C. He, R. Li, S. Li, and L. Zhang. Voxel set transformer: A set-to-set approach to 3D object detection from point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2022).
9. T. Roddick, A. Kendall, and R. Cipolla. Orthographic feature transform for monocular 3D object detection. In *Proceedings of the British Machine Vision Conference* (2019).
10. G. Brazil and X. Liu. M3D-RPN: Monocular 3D region proposal network for object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 9287–9296 (2019).
11. A. Mousavian, D. Anguelov, J. Flynn, and J. Kosecka. 3D bounding box estimation using deep learning and geometry. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7074–7082 (2017).
12. A. Naiden, V. Paunescu, G. Kim, B. Jeon, and M. Leordeanu. Shift R-CNN: Deep monocular 3D object detection with closed-form geometric constraints. In *2019 IEEE International Conference on Image Processing*, pp. 61–65 (2019).
13. Z. Liu, Z. Wu, and R. Tóth. Smoke: Single-stage monocular 3D object detection via keypoint estimation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 996–997 (2020).
14. Z. Qin, J. Wang, and Y. Lu. MonoGRNet: A geometric reasoning network for monocular 3D object localization. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 33, pp. 8851–8858 (2019).
15. X. Zhou, Y. Peng, C. Long, F. Ren, and C. Shi. MoNet3D: Towards accurate monocular 3D object localization in real time. *arXiv abs/2006.16007* (2020).
16. Y. Chen, L. Tai, K. Sun, and M. Li. MonoPair: Monocular 3D object detection using pairwise spatial relationships. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12093–12102 (2020).

17. P. Li, H. Zhao, P. Liu, and F. Cao. RTM3D: Real-time monocular 3D detection from object keypoints for autonomous driving. *arXiv preprint arXiv:2001.03343* **2** (2020).
18. Y. Lu, X. Ma, L. Yang, T. Zhang, Y. Liu, Q. Chu, J. Yan, and W. Ouyang. Geometry uncertainty projection network for monocular 3D object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 3111–3121 (October, 2021).
19. H. Chen, Y. Huang, W. Tian, Z. Gao, and L. Xiong. MonoRun: Monocular 3D object detection by reconstruction and uncertainty propagation. In *IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2021).
20. Y. Li, Y. Chen, J. He, and Z. Zhang. Densely constrained depth estimator for monocular 3D object detection. In *European Conference on Computer Vision* (2022).
21. L. Peng, X. Wu, Z. Yang, H. Liu, and D. Cai. DID-M3D: Decoupling instance depth for monocular 3D object detection. In *European Conference on Computer Vision* (2022).
22. J. Gu, B. Wu, L. Fan, J. Huang, S. Cao, Z. Xiang, and X.-S. Hua. Homography loss for monocular 3D object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June, 2022).
23. B. Xu and Z. Chen. Multi-level fusion based 3D object detection from monocular images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2345–2353 (2018).
24. H. Chu, W.-C. Ma, K. Kundu, R. Urtasun, and S. Fidler. SurfConv: Bridging 3D and 2D convolution for rgb-d images. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3002–3011 (2018).
25. M. Ding, Y. Huo, H. Yi, Z. Wang, J. Shi, Z. Lu, and P. Luo. Learning depth-guided convolutions for monocular 3D object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (June, 2020).
26. Y. Wang, W.-L. Chao, D. Garg, B. Hariharan, M. Campbell, and K. Q. Weinberger. Pseudo-LiDAR from visual depth estimation: Bridging the gap in 3D object detection for autonomous driving. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 8445–8453 (2019).
27. Y. You, Y. Wang, W.-L. Chao, D. Garg, G. Pleiss, B. Hariharan, M. Campbell, and K. Q. Weinberger. Pseudo-LiDAR++: Accurate depth for 3D object detection in autonomous driving. *arXiv preprint arXiv:1906.06310* (2019).

28. R. Qian, D. Garg, Y. Wang, Y. You, S. Belongie, B. Hariharan, M. Campbell, K. Q. Weinberger, and W.-L. Chao. End-to-end pseudo-LiDAR for image-based 3D object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5881–5890 (2020).
29. X. Ma, S. Liu, Z. Xia, H. Zhang, X. Zeng, and W. Ouyang. Rethinking pseudo-LiDAR representation. In *Proceedings of the European Conference on Computer Vision* (2020).
30. W. Bao, B. Xu, and Z. Chen. MonoFENet: Monocular 3D object detection with feature enhancement networks, *IEEE Transactions on Image Processing* **29**, 2753–2765 (2019).
31. X. Ma, Z. Wang, H. Li, P. Zhang, W. Ouyang, and X. Fan. Accurate monocular 3D object detection via color-embedded 3D reconstruction for autonomous driving. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 6851–6860 (2019).
32. J. Ku, A. D. Pon, and S. L. Waslander. Monocular 3D object detection leveraging accurate proposals and shape reconstruction. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 11867–11876 (2019).
33. X. Weng and K. Kitani. Monocular 3D object detection with pseudo-LiDAR point cloud. In *IEEE International Conference on Computer Vision Workshops* (October, 2019).
34. O. Wiles, G. Gkioxari, R. Szeliski, and J. Johnson. SynSin: End-to-end view synthesis from a single image. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7467–7477 (2020).
35. X. Chen, J. Song, and O. Hilliges. Monocular neural image based rendering with continuous view control. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 4090–4100 (2019).
36. I. Choi, O. Gallo, A. Troccoli, M. H. Kim, and J. Kautz. Extreme view synthesis. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 7781–7790 (2019).
37. T. Zhou, R. Tucker, J. Flynn, G. Fyffe, and N. Snavely. Stereo magnification: Learning view synthesis using multiplane images. *arXiv preprint arXiv:1805.09817* (2018).
38. Y. Zhang, J. Lu, and J. Zhou. Objects are different: Flexible monocular 3D object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3289–3298 (June, 2021).
39. X. Chen, Y. Duan, R. Houthoofd, J. Schulman, I. Sutskever, and P. Abbeel. InfoGAN: Interpretable representation learning by information maximizing generative adversarial nets. In *Advances in Neural Information Processing Systems*, pp. 2172–2180 (2016).

40. T. D. Kulkarni, W. F. Whitney, P. Kohli, and J. Tenenbaum. Deep convolutional inverse graphics network. In *Advances in Neural Information Processing Systems*, pp. 2539–2547 (2015).
41. M. Tatarchenko, A. Dosovitskiy, and T. Brox. Multi-view 3D models from single images with a convolutional network. In *European Conference on Computer Vision*, pp. 322–337 (2016).
42. T. Zhou, S. Tulsiani, W. Sun, J. Malik, and A. A. Efros. View synthesis by appearance flow. In *European Conference on Computer Vision*, pp. 286–301 (2016).
43. D. E. Worrall, S. J. Garbin, D. Turmukhambetov, and G. J. Brostow. Interpretable transformations with encoder-decoder networks. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 5726–5735 (2017).
44. V. Sitzmann, M. Zollhöfer, and G. Wetzstein. Scene representation networks: Continuous 3D-structure-aware neural scene representations. In *Advances in Neural Information Processing Systems*, pp. 1121–1132 (2019).
45. M. Alexander, C. Cyril, S. Peter, and M. McGuire. A ray-box intersection algorithm and efficient dynamic voxel rendering, *Journal of Computer Graphics Techniques (JCGT)* **7**(3), 66–81 (September, 2018). ISSN 2331-7418. <http://jcgt.org/published/0007/03/04/>.
46. G. Liu, F. A. Reda, K. J. Shih, T.-C. Wang, A. Tao, and B. Catanzaro. Image inpainting for irregular holes using partial convolutions. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 85–100 (2018).
47. J. Yu, Z. Lin, J. Yang, X. Shen, X. Lu, and T. S. Huang. Generative image inpainting with contextual attention. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5505–5514 (2018).
48. M. Liang, B. Yang, Y. Chen, R. Hu, and R. Urtasun. Multi-task multi-sensor fusion for 3D object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (June, 2019).
49. T. Mordan, N. Thome, G. Henaff, and M. Cord. Revisiting multi-task learning with ROCK: A deep residual auxiliary block for visual detection. In *Advances in Neural Information Processing Systems (NeurIPS)*, pp. 1310–1322 (2018).
50. W. Liu, D. Anguelov, D. Erhan, C. Szegedy, S. Reed, C.-Y. Fu, and A. C. Berg. SSD: Single shot multibox detector. In *European Conference on Computer Vision*, pp. 21–37 (2016).
51. A. Geiger, P. Lenz, C. Stiller, and R. Urtasun. Vision meets robotics: The kitti dataset, *The International Journal of Robotics Research* **32**(11), 1231–1237 (2013).

52. X. Chen, H. Ma, J. Wan, B. Li, and T. Xia. Multi-view 3D object detection network for autonomous driving. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1907–1915 (2017).
53. X. Chen, K. Kundu, Y. Zhu, A. G. Berneshawi, H. Ma, S. Fidler, and R. Urtasun. 3D object proposals for accurate object class detection. In *Advances in Neural Information Processing Systems*, pp. 424–432 (2015).
54. A. Simonelli, S. R. Bulo, L. Porzi, M. López-Antequera, and P. Kotschieder. Disentangling monocular 3D object detection. In *Proceedings of the IEEE International Conference on Computer Vision*, pp. 1991–1999 (2019).
55. L. Liu, J. Lu, C. Xu, Q. Tian, and J. Zhou. Deep fitting degree scoring network for monocular 3D object detection. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1057–1066 (2019).
56. F. Manhardt, W. Kehl, and A. Gaidon. ROI-10D: Monocular lifting of 2D detection to 6d pose and metric shape. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2069–2078 (2019).
57. B. Li, W. Ouyang, L. Sheng, X. Zeng, and X. Wang. GS3D: An efficient 3D object detection framework for autonomous driving. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1019–1028 (2019).
58. G. Brazil, G. Pons-Moll, X. Liu, and B. Schiele. Kinematic 3D object detection in monocular video. In *European Conference on Computer Vision* (2020).
59. I. Loshchilov and F. Hutter. SGDR: Stochastic gradient descent with warm restarts. *arXiv preprint arXiv:1608.03983* (2016).

Chapter 7

Robust Structured Declarative Classifiers for Point Clouds

Ziming Zhang^{*,§}, Kaidong Li^{†,¶}, and Guanghui Wang^{‡,||}

^{*}*Department of ECE, Worcester Polytechnic Institute, MA, USA*

[†]*Department of EECS, University of Kansas, KS, USA*

[‡]*Department of CS, Toronto Metropolitan University,
Toronto ON, Canada*

[§]*zzhang15@wpi.edu*

[¶]*kaidong.li@ku.edu*

^{||}*wangcs@torontomu.ca*

Deep neural networks for 3D point cloud classification, such as PointNet, have been demonstrated to be vulnerable to adversarial attacks. Current adversarial defenders often learn to denoise the (attacked) point clouds by reconstruction and then feed them to the classifiers as input. In contrast to the literature, we propose a novel bilevel optimisation framework for robust point cloud classification, where the internal optimisation can effectively defend adversarial attacks as denoising and the external optimisation can learn classifiers accordingly. As a demonstration, we further propose an effective and efficient instantiation of our approach, namely *Lattice Point Classifier (LPC)*, based on structured sparse coding in the permutohedral lattice and 2D convolutional neural networks (CNNs) that integrate both internal and external optimisation problems into end-to-end trainable network architectures. We demonstrate state-of-the-art robust point cloud classification performance on ModelNet40 and ScanNet under seven different attackers. Our demo code is available at <https://github.com/Zhang-VISLab>.

1. Introduction

1.1. *Adversarial Attacks in 3D Point Clouds*

Point clouds are unstructured data widely used in real-world applications such as autonomous driving. To recognise them using deep neural networks, point clouds can be represented as points,¹ images,² voxels,³ or graphs.⁴ Recent works^{5–11} have demonstrated that such deep networks are vulnerable to (gradient-based) adversarial attacks, i.e., actions to fool classifiers by manipulating input data with noise.

To better understand adversarial attacks, let us take an example of 3D object detection using point clouds,¹² where we would like to detect objects of interest, such as cars in the (x, y, z) -coordinate system, i.e., physical world. To this end, we take a 3D object detector, PIXOR,¹³ and evaluate it on the KITTI¹⁴ dataset. The point cloud representation used in PIXOR is voxel, i.e., all the point locations are quantised into a 3D tensor where a voxel is labelled as 1 if there is at least one point inside it, otherwise 0. *In the sequel, we will consider adversarial attacks with limited visual perturbations.* In other words, we assume that the attacked point clouds should be visually similar to the original ones so that we can observe similar global structures. Such an assumption is widely used in image-based adversarial attacks.^{15,16}

To attack point clouds, we simply compute the gradients of the loss in PIXOR w.r.t. the input point cloud tensors and move the points to new locations within their 26 neighbours with the highest gradient values.¹² This process can maximise the loss while limiting the points moved. We also use a threshold to control the number of moved points whose maximum relative gradients w.r.t. the centres are higher than the threshold. This procedure is repeated over time until convergence.

We illustrate our findings in Figure 1. As we see in the plot, the detection performance deteriorates with the increase in noise level significantly, even though visually the attacked point clouds still preserve the global structures. With about 50% attacked points, the detection performance drops by about 30% relatively. This is totally unacceptable in real-world applications, such as autonomous driving, as the noise may occur at any time, such as hardware degradation or bad weather conditions. Therefore, defending such adversarial attacks in practice is very important for point cloud applications.

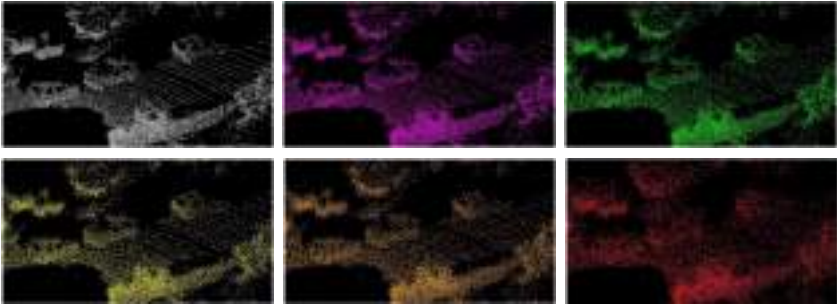
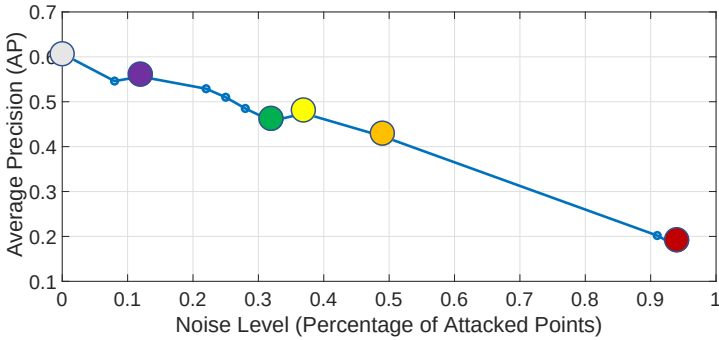


Fig. 1. Illustration of adversarial attacks in point clouds for 3D object detection. As a demonstration, the coloured dots in the plot correspond to the figures with the same coloured points from top-left to bottom-right.

1.2. Adversarial Defenders

Several adversarial defenders^{17–19} have been proposed for robust point cloud classification where the classifier is insensitive to noisy input data. The basic ideas of such defenders are often to *denoise* the (attacked) point clouds before feeding them into the classifiers as input to preserve their prediction accuracy.

One common way against adversarial attacks is to *break* the gradients over the input data in backpropagation (either inadvertently or intentionally, e.g., a defender is non-differentiable or prevents gradient signal from flowing through the network) so that the attackers fail to be optimised. Such scenarios are called *obfuscated gradients*²⁰ and have discussed the false sense of security in such defenders and proposed new methods, e.g., Backward Pass Differentiable Approximation (BPDA), to successfully attack some defenders with obfuscated gradients. Taking DUP-Net,¹⁷ for example, where a non-differentiable Statistical Outlier Removal (SOR)

defence strategy was proposed for 3D adversarial point cloud classification,⁷ proposed Joint Gradient Based Attack (JGBA) that can compute the gradient with a linear approximation (an instantiation of BPDA) of the SOR defence to attack DUP-Net successfully.

Another popular way against adversarial attacks is to use *implicit gradients* of differentiable defenders and classifiers, which has been significantly underestimated for robust point cloud classification in the literature. An implicit gradient, $\frac{\partial y}{\partial x}$, is defined by a differentiable function h that takes x, y as its input, i.e., $\frac{\partial y}{\partial x} = h(x, y)$. Such an equation can be also considered as a first-order ordinary differential equation (ODE), which is solvable (approximately) using Euler’s method.²¹ Here we assume that the gradients through the classifiers can be easily computed, which often holds empirically. As we see, implicit gradients require equations that contain both the input and output of defenders. One potential solution for this is to introduce optimisation problems as the defenders, where the first-order optimality conditions provide such equations. In the literature, there have been some works^{22–25} that proposed optimisation as network layers in deep neural networks.

A *declarative* network node²⁶ is introduced where the exact implementation of the forward processing function is not defined, rather the input-output relationship ($x \mapsto \tilde{x}$) is defined in terms of behaviour specified as the solution to an optimisation problem $\tilde{x} \in \arg \min_{z \in \mathcal{Z}} f(x, z; \theta)$. Here f is an objective function, θ denotes the node parameters, and $\mathcal{Z}$ is the feasible solution space. And implicit differentiation is used to perform backpropagation through declarative networks. The optimisation problems are introduced as the defenders, where the first-order optimality conditions provide such equations. Recently, Gould *et al.*²⁶ generalised these ideas and proposed a robust pooling layer as a declarative node using unconstrained minimisation with various penalty functions, such as Huber or Welsch, which can be efficiently solved using Newton’s method or gradient descent. The effectiveness of such pooling layers was demonstrated for point cloud classification.

1.3. Our Approach and Contributions

Motivated by the earlier methods, in this paper, we propose a novel robust structured declarative classifier for 3D point clouds

by embedding a declarative node into the networks. Different from robust pooling layers in Gould *et al.*²⁶ our declarative defender is designed to reconstruct each point cloud in a (learnable) structural space as a means of denoising. To this end, we borrow the idea from structured sparse coding^{27–29} by representing each point as a linear combination of atoms in a dictionary. Together with the backbone networks, the training of our robust classifiers can lead to a bilevel optimisation problem. Considering the inference efficiency, one plausible instantiation of our classifiers is to define the structural space using the permutohedral lattice,^{30–32} project each point cloud onto the lattice, generate a 2D image based on the barycentric weights, and feed the image to a 2D convolutional neural network (CNN) for classification. We call this instantiation *Lattice Point Classifier (LPC)*. As a result, we summarise our contributions in the following:

- We propose a family of novel robust structured declarative classifiers for 3D point clouds where the declarative nodes defend the adversarial attacks through implicit gradients.
- We propose a bilevel optimisation framework to learn the network parameters in an end-to-end fashion.
- We propose an effective and efficient instantiation of our robust classifiers based on the structured sparse coding in the permutohedral lattice and 2D CNNs.
- We demonstrate the superior performance of our approach by comparing it with the state-of-the-art adversarial defenders under the state-of-the-art adversarial attackers.

2. Related Works

Deep learning for 3D point clouds: Based on the point cloud representations, we simply group some typical deep networks into four categories. *Point-based networks*^{1,33–36} directly take each point cloud as input, extract point-wise features using multi-layer perceptrons (MLPs), and fuse them to generate a feature for the point cloud. *Image-based networks*^{2,37–41} often project a 3D point cloud onto a (or multiple) 2D plane to generate a (or multiple) 2D image for further process. *Voxel-based networks*^{3,42–45} usually voxelise each point cloud into a volumetric occupancy grid and further some classification

techniques such as 3D CNNs are used for the tasks. *Graph-based networks*^{4,46–49} often represent each point cloud as a graph such as KNN or adjacency graph which are fed to train graph convolutional networks (GCNs). A nice survey can be found in Guo *et al.*⁵⁰

Adversarial attacks on point clouds: Such attacks aim to modify the input point clouds in a way that is not noticeable but can fool a classifier. Attackers can either have a target or not. Targeted attackers try to fool the classifier to predict a specified wrong class, while untargeted ones do not care about the predicted class as long as it is wrong. Nice surveys on adversarial attacks can be found in Ozdag,⁵¹ Wang *et al.*,⁵² and Chakraborty *et al.*⁵³ In the following, we summarise some typical attackers for point clouds:

- *Point perturbation*^{7,9,54–56}: Inspired by Fast Gradient Sign Method (FGSM),¹⁵ they add on each point a small perturbation constrained by distance metrics (e.g., L_p norm,⁹ on the surface of an ϵ -ball⁵⁴).
- *Point addition*^{9,10,55}: Independent point attackers initially pick some points from target classes, add small perturbations, and finally append these points to the victim point clouds. Cluster attackers similarly pick most of the critical points and then find a specified number of small point clusters to append to the victim point clouds. Object attackers attach scaled and rotated foreign objects to the original point clouds.
- *Point dropping*: Adversarial attackers pick critical points from each input point cloud and drop them to fool the classifier. However, it is a non-differentiable operation. To address the issue in learning, Zheng *et al.*⁵⁷ proposed creating saliency maps by viewing dropping a point as moving it to the cloud centre. Wicker and Kwiatkowska¹⁰ designed an algorithm to iteratively find the points to drop by minimising a pre-defined objective function, similarly in Yang *et al.*⁵⁵
- *Others*: Hamdi *et al.*⁵⁸ proposed a transferable adversarial perturbation attacker based on an adversarial loss that can learn the data distribution. Zhao *et al.*⁵⁹ proposed a black-box (i.e., no access to the models) attacker with zero loss in isometry, as well as a white-box attacker (i.e., the attackers are assumed to have full access to both classifiers and defenders) based on the spectral norm. LG-GAN⁵ is based on generative adversarial network (GAN),

which learns and incorporates target features into victim point clouds. A backdoor attacker was proposed in Xiang *et al.*⁶ to trick the 3D models by inserting adversarial point patterns into the training set so that the victim models learn to recognise the adversarial patterns during inference.

Adversarial defence on point clouds: Adversarial defences aim to denoise the input point clouds to recover the ground-truth labels from the classifiers. Nice surveys on adversarial defences can be found in Ozdag,⁵¹ Wang *et al.*,⁵² and Chakraborty *et al.*⁵³ In the following, we summarise some typical defenders for point clouds:

- *Statistical outlier removal (SOR)*⁶⁰: SOR can be used to remove local roughness on the (smooth) surface as a means of defence. SOR is not differentiable, producing obfuscated gradients for defenders. DUP-Net¹⁷ uses SOR and an upsampling network to reconstruct higher-resolution point clouds. Similarly, IF-Defence¹⁸ utilises SOR, followed by a geometry-aware model to encourage evenly distributed points. However, such defenders have been demonstrated to be attackable in Tsai *et al.*⁶¹ and Ma *et al.*⁷ Dong *et al.*⁶² proposed replacing SOR with an attention mechanism.
- *Random sampling*: Yang *et al.*⁵⁵ suggested that 3D models with random sampling are robust to adversarial attacks. PointGuard¹⁹ proposed majority voting for point cloud classification by predicting multiple randomly subsampled point clouds.
- *Data augmentation*: Tramèr *et al.*⁶³ demonstrated that data augmentation can effectively account for adversarial attacks. Tu *et al.*⁶⁴ proposed generating physically realisable adversarial examples to train robust Lidar object detectors. Zhang *et al.*⁶⁵ proposed randomly permuting training data as a simple data augmentation strategy. PointCutMix⁶⁶ pairs two training clouds and swaps some points between the pair to generate new training data.

Permutohedral lattice: Permutohedral lattice is a powerful operation to project the coordinates from a high-dimensional space onto a hyperplane that defines the lattice. It has been widely used in high-dimensional filtering^{30,67} that consists of three components, i.e., splat, blur, and slice. In particular, we illustrate the splat in Figure 3, where each square represents a projection (i.e., projected point) from

a 3D coordinate. The splat first locates the enclosing lattice simplex for the 3D point and calculates the vertex coordinates of the simplex. Then each projection distributes its value to the vertices using barycentric interpolation with barycentric weights that are calculated as the normalised triangular areas between the projection and any pair of its corresponding lattice vertices. We refer the readers to Adams *et al.*³⁰ for more details. Recently, permutohedral lattice has been successfully explored in point cloud segmentation^{31,32,68} with remarkable performance.

3. Robust Structured Declarative Classifiers

3.1. Bilevel Optimisation Framework

Recall that adversarial defenders often aim to denoise the input point clouds by reconstructing them in certain ways. Therefore, we propose a bilevel optimisation framework, as illustrated in Figure 2, which involves two components, i.e., a declarative defender f and a network backbone g , that take the outputs of the defender as input and then make predictions. The declarative defender as a lower-level optimisation problem tries to learn good representations for point clouds and the network backbone as an upper-level optimisation problem tries to determine the classifier. Mathematically, we can formulate our learning objective as

$$\min_{\theta, \omega} \sum_{(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}} \ell(g(\tilde{\mathbf{x}}; \omega), y), \quad \text{s.t. } \tilde{\mathbf{x}} \in \arg \min_{\mathbf{z} \in \mathcal{Z}} f(\mathbf{x}, \mathbf{z}; \theta), \quad (1)$$

where $(\mathbf{x}, y) \in \mathcal{X} \times \mathcal{Y}$ denotes a training sample with point cloud $\mathbf{x}$ and label y , ℓ denotes a loss function, such as cross-entropy, and f, g denote a declarative defender and a neural network classifier parametrised by θ, ω , respectively. g corresponds to the learnt solution to the upper-level optimisation problem, while f is the solution of the lower-level problem.

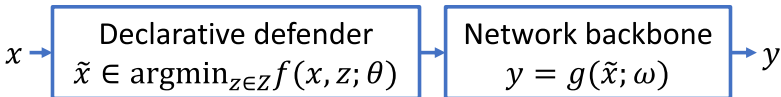


Fig. 2. Illustration of robust structured declarative classifiers, where our defender is optimised in a structural space.

Implicit gradients: In a gradient-based adversarial attack, the gradient for modifying an input point cloud $\mathbf{x}$ through backpropagation can be written as $\frac{\partial \ell}{\partial \mathbf{x}} = \frac{\partial \ell}{\partial g} \frac{\partial g}{\partial \tilde{\mathbf{x}}} \frac{\partial \tilde{\mathbf{x}}}{\partial \mathbf{x}}$. Note that $\tilde{\mathbf{x}}$ is essentially a function of $\mathbf{x}$ that may not be expressed explicitly, in general, especially within a constrained optimisation problem. Therefore, $\frac{\partial \tilde{\mathbf{x}}}{\partial \mathbf{x}}$ leads to an implicit gradient that prevents accurate gradients from being back-propagated to the adversarial examples for defence purposes.

Optimisation: Solving bilevel optimisation is challenging,⁶⁹ in general. However, one can consider the following two specific scenarios for computational efficiency: (1) there exists a close-form solution for the lower-level problem; (2) functions f, g, ℓ are continuous and differentiable w.r.t. $\mathbf{z}$, which can be potentially solved using approximate implicit differentiation (AID) or iterative differentiation (ITD) approaches with some convergence guarantee.⁷⁰ Then we can come up with an end-to-end network as a solver for the bilevel optimisation problem in Equation (1) that can be trained using (stochastic) gradient descent.

3.2. Sparse Coding as Declarative Defender

Sparse coding⁷¹ is one of the classic approaches for finding a sparse representation of the input data in the form of a linear combination of basic elements as well as those basic elements themselves. Due to its simplicity, we consider using sparse coding as a means to construct declarative nodes.

Specifically, in the 3D space given a (learnable) dictionary $\mathbf{B} \in \mathbb{R}^{3 \times N}$ with $N \gg 3$ atoms, (structured) sparse coding aims to solve the following optimisation problem for each point $\mathbf{x}_i \in \mathbb{R}^3$ in a point cloud $\mathbf{x} = \{\mathbf{x}_i\} \subseteq \mathbb{R}^3$:

$$\tilde{\mathbf{x}}_i \in \arg \min_{\mathbf{z} \in \mathcal{Z}} f(\mathbf{x}_i, \mathbf{z}; \mathbf{B}) = \frac{1}{2} \|\mathbf{x}_i - \mathbf{B}\mathbf{z}\|^2 + \phi(\mathbf{z}), \quad (2)$$

where ϕ denotes a regularisation term and $\mathcal{Z} \subseteq \mathbb{R}^N$ denotes a structural feasible solution space.

To see the connections between Equation (2) and implicit gradients, we can rewrite Equation (2) as $\tilde{\mathbf{x}}_i \in \arg \min_{\mathbf{z}} F(\mathbf{x}_i, \mathbf{z}) = f(\mathbf{x}_i, \mathbf{z}) + \phi(\mathbf{z}) + \delta(\mathbf{z})$, where $\delta(\mathbf{z})$ denotes the Dirac delta function returning 0 if $\mathbf{z} \in \mathcal{Z}$ holds, otherwise $+\infty$. Therefore, F will become non-differentiable when $\mathbf{z} \notin \mathcal{Z}$ or ϕ is non-differentiable over $\mathbf{z} \in \mathcal{Z}$,

leading to obfuscated gradients. Otherwise, based on the first-order optimality condition, we have $\mathbf{B}^T \mathbf{B} \tilde{\mathbf{x}}_i - \mathbf{B}^T \mathbf{x}_i + \phi'(\tilde{\mathbf{x}}_i) = \mathbf{0}$, where $(\cdot)^T$ denotes the matrix transpose operator and ϕ' denotes the first-order derivative of ϕ . By taking another derivative on both sides, we then have a linear system $(\mathbf{B}^T \mathbf{B} + \phi''(\tilde{\mathbf{x}}_i)) \cdot \frac{\partial \tilde{\mathbf{x}}_i}{\partial \mathbf{x}_i} = \mathbf{B}^T$, if the second-order derivative, ϕ'' , exists at $\mathbf{z} \in \mathcal{Z}$. Clearly, solving this linear system may be challenging because $\phi''(\tilde{\mathbf{x}}_i)$ may not be computable and the matrix $\mathbf{B}^T \mathbf{B} + \phi''(\tilde{\mathbf{x}}_i)$ may be rank-deficient. Such phenomena can easily lead to implicit gradients.

3.3. Permutohedral Lattice Coding: An Instantiation

In order to compute the representations of point clouds efficiently and effectively from the declarative node, we consider using the close-form solutions of certain optimisation problems and thus borrow the idea from permutohedral lattice for sparse coding. Specifically, barycentric coordinates ($\vec{A}$, $\vec{B}$, $\vec{C}$ in Figure 4) can be used to express the position of any point ($\vec{P}$) located on the entire triangle with three

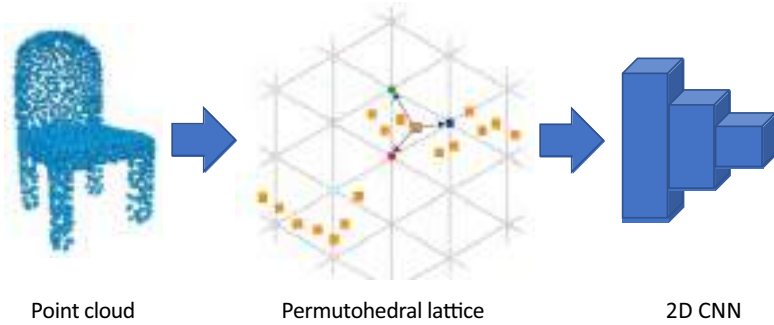


Fig. 3. Illustration of our Lattice Point Classifier (LPC).

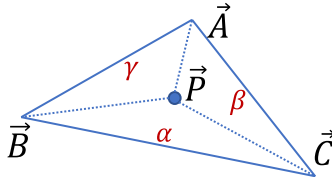


Fig. 4. Illustration of barycentric coordinates and weights.

scalar weights (α, β, γ) . To compute $\vec{P}$ using barycentric coordinates, we can always use the following equation:

$$\vec{P} = \alpha \vec{A} + \beta \vec{B} + \gamma \vec{C}, \quad \exists \alpha, \beta, \gamma \geq 0, \quad \alpha + \beta + \gamma = 1. \quad (3)$$

Note that this equation holds for an arbitrary dimensional space, including the 3D space. Now given $\vec{A}, \vec{B}, \vec{C}, \vec{P}$, we can compute α, β, γ using the normalised areas. Taking γ , for example, we can compute it as follows:

$$\gamma = \frac{\|\vec{AB} \times \vec{AP}\|}{\|\vec{AB} \times \vec{AC}\|} \propto \frac{\|\vec{AB} \times \vec{AP}\|}{\|\vec{AB} \times \vec{AC}\|}, \quad (4)$$

where $\vec{AB} = \vec{B} - \vec{A}$, $\vec{AP} = \vec{P} - \vec{A}$, $\vec{AC} = \vec{C} - \vec{A}$, $\times$ denotes the cross-product operator, and $\|\cdot\|$ denotes the ℓ_2 norm of a vector measuring its length. Clearly, the barycentric weights define a nonlinear mapping that can be computed efficiently using the splat operation for the permutohedral lattice.

By substituting Equation (3) into Equation (2), we manage to define a structured sparse coding problem, where the structural space $\mathcal{Z}$ and the regulariser ϕ can be constructed as follows:

$$\mathcal{Z} \stackrel{\text{def}}{=} \{\mathbf{z} \mid \mathbf{z}^T \mathbf{e} = 1, \mathbf{z} \succeq \mathbf{0}\}, \quad \phi(\mathbf{z}) \stackrel{\text{def}}{=} \lambda \sum_{n=1}^N \|\mathbf{B}\mathbf{z} - \mathbf{B}_n\| \cdot 1_{\{\mathbf{z}_n > 0\}}, \quad (5)$$

where $\mathbf{e}$ denotes a vector of 1's, $\succeq$ denotes the entry-wise operator of $\geq$, $\mathbf{B}_n \in \mathbb{R}^3$ denotes the n -th column in $\mathbf{B}$, $1_{\{\cdot\}}$ denotes a binary indicator returning 1 if the condition holds, otherwise 0 (i.e., the *binarisation* of barycentric weights), and $\lambda \geq 0$ is a small constant controlling the contribution of ϕ to the objective so that it will not be dominated by ϕ .

Prop 1. Supposing that $\mathbf{B}$ in Equation (2) represents the vertices in a permutohedral lattice that is large enough to cover all possible projections from points among the data, then there exists a solution to minimise the reconstruction loss using three vertices, at most, and the minimum loss is equal to the projection loss onto the lattice.

This is because Equation (3) defines a lossless representation using a linear combination of three vertices. The only loss occurs when projecting a point to the permutohedral lattice.

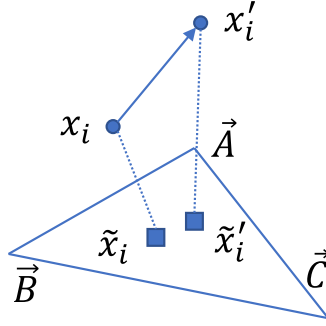


Fig. 5. Illustration of our defence mechanism.

Figure 5 illustrates how our declarative defender works with the permutohedral lattice, where the triangle, circles, and squares represent a lattice cell, 3D points, and their projections on the cell, respectively. In the adversarial point cloud, the attacker modifies a point from $\mathbf{x}_i$ to $\mathbf{x}'_i$. Then during the inference, our defender projects $\mathbf{x}'_i$ to $\tilde{\mathbf{x}}'_i$ on the lattice.

Prop 2. Supposing $\tilde{\mathbf{x}}_i = \alpha \vec{A} + \beta \vec{B} + \gamma \vec{C}$ where α, β, γ are the barycentric weights, then in order to guarantee that $\tilde{\mathbf{x}}_i, \tilde{\mathbf{x}}'_i$ lie in different cells, the distance between $\mathbf{x}_i$ and $\mathbf{x}'_i$ should be bigger than the shortest distance between $\tilde{\mathbf{x}}'_i$ and the boundary of the triangle, that is,

$$\|\mathbf{x}_i - \mathbf{x}'_i\| > \min \left\{ \frac{\alpha}{\|\vec{BC}\|}, \frac{\beta}{\|\vec{AC}\|}, \frac{\gamma}{\|\vec{AB}\|} \right\} \cdot s, \quad (6)$$

Algorithm 1 Inference Algorithm for Lattice Point Classifier

Input : A point cloud $\mathbf{x}$, a 2D CNN network g

Output: Class label y

Step 1: Compute the barycentric weights of each point in $\mathbf{x}$ using the splat for permutohedral lattice based on Equations (3) and (4);

Step 2: Generate an image for $\mathbf{x}$ by averaging the barycentric weights over all the points (after binarisation if applied) and aligning the lattice with the image representation (see Figure 6 for illustration);

Step 3: Classify the image using g to predict its class label y ;

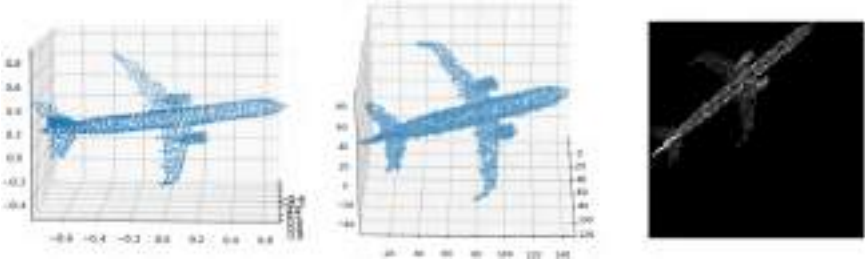


Fig. 6. Illustration of (left) a point cloud, (middle) its lattice representation, and (right) its image for classification.

where s denotes the area of the triangle. In particular, if the triangle is equilateral, then $\|\mathbf{x}_i - \mathbf{x}'_i\| > \frac{\sqrt{3}}{2}l \cdot \min\{\alpha, \beta, \gamma\}$, where l denotes the side length.

It will be more intuitive to understand this result if we binarise the barycentric weights before feeding them into the backbone network for classification because different projections lying in the same lattice cell will lead to the same representation. This could be an effective way to remove the adversarial noise in the data. Also, this result indicates that (1) the points whose projections are closer to the boundary are easier to change their sparse representations and (2) the movements that make such changes are proportional to the scale of the lattice cell. In general, larger cells will be more tolerant to the adversarial noise, but they may sacrifice the generalisation of the classifiers.

3.4. *Lattice Point Classifier*

So far we have explained the learning principles and defence philosophy in our approach. The key challenge now is how to design the structural space $\mathcal{Z}$ and the regulariser ϕ to achieve robust point cloud classifiers that can be trained and inferred effectively and efficiently. Therefore, based on permutohedral lattice coding, we propose our Lattice Point Classifier (LPC), as listed in Algorithm 1. The key challenge in our implementation of LPC is how to determine the projection matrix for the hyperplane and the scale of each permutohedral lattice cell.

- *Projection matrix:* By referring to Ref. [30], we initialise the projection matrix as $\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}$ and train the network in an end-to-end fashion. The lattice coordinates are losslessly projected to normal 2D space by ignoring z -axis because the lattice projection matrix has rank 2. However, we observe that a big update of this matrix will make the training crash, and to avoid this issue, the update per iteration has to be tiny, leading to an almost unnoticeable change eventually. The reason for this phenomenon is that this matrix has to satisfy certain requirements (see Ref. [30]), and thus the unconstrained update in backpropagation cannot work here. Therefore, in our experiments, we initialise and fix the projection matrix. We refer to Gu *et al.*³² for the lattice transformation.
- *Scale of lattice cell:* The parameter is simply not differentiable in backpropagation, and thus we tune it as a pre-defined hyperparameter using cross-validation with grid search, same for the other hyperparameters, such as learning rate. Such scales have a significant impact on the image resolution used in 2D CNNs for classification.

Specifically, we evaluate our LPC comprehensively based on three different CNNs, i.e., VGG16,⁷² ResNet50,⁷³ and EfficientNet-B5,⁷⁴ as the backbone network with random initialisation. By default, the image resolution for each backbone network is 512×512 , 128×128 , and 456×456 , respectively. On ModelNet40,³ we train the three models using a learning rate of 10^{-4} , but on ScanNet,⁷⁵ we only train our best model, i.e., EfficientNet-B5, using a learning rate of 5×10^{-5} . We use Adam⁷⁶ as our optimiser in all of our experiments with weight decay of 10^{-4} and learning rate decay of 0.7 for every 20 epochs. Dropout⁷⁷ and data augmentation are applied as well when needed.

4. Experiments

We conduct our experiments on ModelNet40³ and ScanNet.⁷⁵ ModelNet40 has a collection of 12,311 3D CAD objects from 40 common categories. It is split into 9,843 training and 2,468 test samples. Following Refs. [1,9], we uniformly sample 1,024 points from the surface of the original point cloud per object and scale them into a unit

ball. ScanNet contains 1,513 RGB-D scans from over 707 real indoor scenes with 2.5 million views. Following Ref. [34], we generate 12,445 training and 3,528 test point clouds from 17 categories, with 1,024 points for each point cloud as well.

We compare our Lattice Point Classifier (LPC) with different adversarial defenders for point clouds that work with PointNet,¹ including DUP-Net¹⁷ (with SOR and the upsampling network), IF-Defence¹⁸ (with ConvONet⁷⁸), and robust pooling layer (RPL).²⁶ We utilise public code⁷⁹ to train PointNet using the default setting and evaluate DUP-Net and IF-Defence based on the implementation in Wu *et al.*¹⁸ We report the best performance for each defender after fine-tuning. Specifically, we set $k = 2$ (the number of neighbour points) in KNN and $\alpha = 1.1$ (the percentage of outliers) for SOR, use 2 for the upsampling rate in DUP-Net, and choose the Welsch penalty function⁸⁰ for RPL²⁶ to replace the max pooling layer in PointNet. The model with RPL is trained with adversarial point clouds where 10% of input points are replaced by random outliers.

Eight attackers are utilised to evaluate the robustness of different point cloud classifiers, including untargeted attackers (FGSM¹⁵ and JGBA⁷), targeted attackers (perturbation, add, cluster, and object attackers⁹), isometry transformation-based attackers⁵⁹ (the untargeted black-box TSI attack and the targeted white-box CTRI attack), and GAN-based LG-GAN attacker.⁵

We apply both FGSM and JGBA to the full test set of both datasets. We slightly modify FGSM for attacking DUP-Net and IF-Defence under the white-box setting as both defenders are non-differentiable. To do so, during an attack, we simulate the SOR process and obtain the indices of the remaining points based on which the gradient is passed to the remaining points. By default, the parameter ϵ in FGSM is set to 0.1, and the perturbation norm constraint ϵ , number of iterations n , and step size α in JGBA are set to 0.1, 40, and 0.01, respectively. Such parameters work well in practice.

For the targeted attackers, by following Ref. [9] we pick 10 large classes from ModelNet40, where a batch of 6 point clouds per class is randomly selected and attacked using the other 9 classes as the targets, leading to $10 \times 9 \times 6 = 540$ victim-target pairs. Similarly, from ScanNet, we randomly select a batch of 6 point clouds as well from the 7 classes that contain more than 100 point clouds in test data. The learning rate of all the targeted attackers is set to 0.01.

For DUP-Net and IF-Defence, we first attack clean PointNet to obtain adversarial point clouds and then feed them into DUP-Net and IF-Defence for prediction. The perturbation attack first introduces small random perturbations on all original points and uses L_2 norm to constrain the adversarial shifts. Using the add attacker, we add 60 points to each cloud and use Chamfer distance as the metric. Cluster and object attackers generate the initial clusters with parameter $\epsilon = 0.11$ in DBSCAN.⁸¹ Using the cluster attacker, we add 3 clusters of 32 points to the original clouds. Using the object attacker, three 64-point adversarial objects are attached to each original point cloud.

For the TSI/CTRI attacker in Zhao *et al.*,⁵⁹ we evaluate the performance of LPC on ModelNet40 using EfficientNet-B5. By following Ref. [59], we use this attacker with the default settings to attack 2,000 randomly selected point clouds. The attacker applies the black-box TSI attacker first to trick the models and then the white-box CTRI attacker if TSI fails.

We implemented a vanilla version of LPC in TensorFlow with binarised weights, ResNet50 as the backbone, and 128×128 as the 2D image size. On ModelNet40, we trained a TensorFlow LPC model and integrated it into LG-GAN to generate adversarial samples. For the benchmark models, we trained a TensorFlow PointNet¹ model and generated adversarial samples with it since all other benchmark defences are PointNet based. We followed the setups in Zhou *et al.*⁵ and set weight factor $\alpha = 100$.

4.1. Ablation Study

In Table 1, we list three learning choices that we would like to evaluate for improving the performance of vanilla LPC, i.e., T-Net used

Table 1. Our learning choice comparison in terms of test accuracy (%) with learning rate 10^{-4} .

T-Net ¹		✓			✓	
Binarised weights			✓		✓	✓
Random rotation				✓		✓
ResNet-50 on ModelNet40	88.2	87.3	88.9	60.0	88.2	75.2
EfficientNet-B5 on ModelNet40	—	—	89.5	83.3	—	84.8
EfficientNet-B5 on ScanNet	80.5	—	80.0	82.6	—	—

in PointNet, binarised barycentric weights, and random rotation in data augmentation (together with point cloud random shifting and dropping¹). We can see the following: (1) T-Net seems to deteriorate the performance always. (2) Binarised weights work better on ModelNet40 than on ScanNet, but compared with using barycentric weights, the difference is $<1\%$. (3) Random rotation improves the performance on ScanNet but worsens it on ModelNet40. This phenomenon can be partially explained by the training loss curves, as shown in Figure 7, where on ScanNet the overfitting occurs clearly without random rotation, while on ModelNet40, random rotation makes the convergence slower and unstable.

Overall LPC achieves the fastest inference speed among all competing defences, as shown in Table 2. Our deepest model (17fps) is over 13 times faster than SOR-based defences. We can also improve the running time using shallower networks such as VGG16 (45fps) by slightly sacrificing the performance. We also tested the running time for each key component (declarative node and backbone 2D networks). With EfficientNet-B5 as the backbone, the declarative node

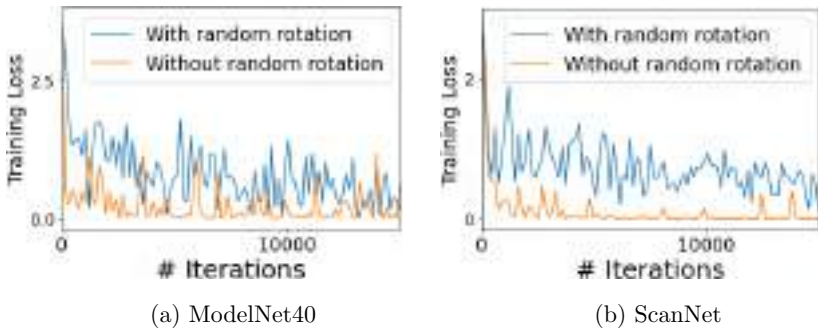


Fig. 7. Training loss comparison on both datasets using EfficientNet-B5 as the backbone.

Table 2. Inference Time (ms) on an NVIDIA V100S GPU with batch size 1 (clean PointNet¹ takes 8.6 ms).

DUP-Net ¹⁷	IF-Defence ¹⁸	RPL ²⁶	LPC+ VGG	LPC+ ResNet	LPC+ EfficientNet
808.1	1793.6	62.0	21.4	25.1	59.0

and 2D network take 17.1 and 41.9 ms respectively during inference and 15.5 and 165.3 ms during backpropagation. Clearly, the gradient passing through the declarative node takes relatively constant time in both feedforward and backpropagation. Considering the depth of EfficientNet-B5, the time spent on the declarative node is actually pretty long (recall that there is no learnable parameter inside), especially during inference. This partially validates our intuition of defending the adversarial attacks using implicit gradients.

4.2. *State-of-the-Art Performance Comparison*

We measure the robustness of classifiers using two metrics, i.e., classification accuracy and attack success rate. Classification accuracy under attacks is measured by feeding the entire adversarial attack test set to the victim models. It is only calculated for untargeted attackers. The attack success rate is the ratio between the number of successful attacks to the number of all attempted attacks. For untargeted attacks, tricking the model to predict a wrong class is considered a success, while for targeted attacks, it will have to trick the victim model into a specific class to be considered as successful. Untargeted attackers will only attack the point clouds that are correctly classified by the victim models, and targeted attackers will attack victim-target pairs.

4.2.1. *Classification accuracy*

We summarise our comparison in Table 3. On ScanNet, we only show the performance of LPC with EfficientNet-B5 because it achieves the best accuracy on ModelNet40. We can see the following: (1) On the clean test data with no attack, PointNet outperforms all the robust classifiers by small margins. (2) Using both attackers, our LPC variants work consistently and significantly better than the other defender-based robust classifiers as well as vanilla PointNet by large margins. For instance, compared with PointNet with IF-Defence under the JGBA attacks, the performance gaps are 84.14% and 69.71% on ModelNet40 and ScanNet, respectively. (3) For the backbone networks in LPC, it seems that the difference in performance is small among different CNNs. Though more evaluations are needed to confirm this, it also demonstrates the robustness of our approach.

Table 3. Test accuracy (%) comparison (higher is better) on the full test datasets for (middle column) ModelNet40 and (bottom column) ScanNet.

	PointNet	PointNet+ DUP-Net	PointNet+ IF-Defence	PointNet+ RPL	LPC+ VGG16	LPC+ ResNet50	LPC+ EfficientNet
No attack	90.15	89.30	87.60	84.76	88.65	88.90	89.51
FGSM ¹⁵	45.99	61.63	38.75	0.04	88.65	88.90	89.51
JGBA ⁷	0.00	1.14	5.37	0.00	88.65	88.90	89.51
No attack	84.61	83.62	80.19	76.02	—	—	83.16
FGSM ¹⁵	45.66	73.67	71.14	1.70	—	—	83.16
JGBA ⁷	0.00	7.77	13.45	0.00	—	—	83.16

Note: where “—” indicates no result.

Table 4. Attack success rate (%) comparison (lower is better) on ModelNet40 (middle column) and ScanNet (bottom column).

	PointNet	PointNet+ DUP-Net	PointNet+ IF-Defence	PointNet+ RPL	LPC+ VGG16	LPC+ ResNet50	LPC+ EfficientNet
FGSM	48.99	30.77	55.78	99.95	0.00	0.00	0.00
JGBA	100.00	98.73	93.85	100.00	0.00	0.00	0.00
Pert.	100.00	0.095	0.095	3.95	0.56	0.37	0.38
Add	99.72	0.095	0.095	3.33	0.56	0.19	0.19
Cluster	98.34	6.76	6.30	17.04	1.11	0.93	1.21
Object	98.43	1.02	1.11	74.07	0.93	1.11	0.75
FGSM	46.03	10.29	11.28	97.76	—	—	0.00
JGBA	100.00	90.55	83.21	100.00	—	—	0.00
Pert.	100.00	3.17	2.38	3.03	—	—	12.70
Add	100.00	1.98	2.38	18.65	—	—	2.78
Cluster	100.00	30.16	23.81	40.87	—	—	9.92
Object	100.00	7.14	7.54	85.71	—	—	5.95

Note: where “—” indicates no result and perturbation, add, cluster, and object attacks are from Xiang *et al.*,⁹ i.e., 4 specific CW attacks.⁸² The standard deviations of our LPC range from 0% to 0.28% in success rate on ModelNet40.

4.2.2. Attack success rate

We list our comparison in Tables 4 and 5. We can see the following: (1) Our LPC variants achieve perfect results under both FGSM and JGBA attacks. Note that compared with the other attackers, these two attackers aim to find adversarial examples fully based on gradients with no sampling. Their failure again strongly demonstrates the power of implicit gradients in defending gradient-based adversarial attacks. (2) The SOR defence mechanism seems to work better for the perturbation and add attackers but worse for the cluster and object attackers, compared with our LPC. However, overall LPC still achieves the best performance. (3) Our LPC also outperforms other

Table 5. Performance comparisons under LG-GAN⁵ attacks.

	PointNet ¹	DUP-Net ¹⁷	IF-Defence ¹⁸	RPL ²⁶	LPC w/ResNet
Classification accuracy (%)	0.6	31.6	37.2	56.8	76.7
Attack success rate (%)	97.0	13.9	10.4	2.6	0.8

defences under GAN-based attacks. Interestingly RPL²⁶ outperforms SOR defences under LG-GAN attack, while under gradient-based attacks, the SOR methods are better. It implies LG-GAN has better transferability. The two SOR-based defences are grey-box attacks (partial access to the model). Gradient-based adversarial samples suffer a significant success rate drop, as shown in Table 4. But LG-GAN transfers well to SOR-based defences, maintaining over 10% attack success rate.

Using the TSI/CTRI attacker,⁵⁹ our LPC with EfficientNet-B5 achieves the same success rate for both TSI and CTRI attackers on ModelNet40. Without random rotation, the result is 99.54%, but with random rotation, the performance decreases to 67.33% which is significantly better than PointNet (99.50% and 99.55% for TSI and CTRI, respectively). Such results also demonstrate that random rotation to point clouds as a means of data augmentation improves not only accuracy but also model robustness. We also implement expectation over transformation (EOT) attacks⁸³ on ScanNet. For each attack, we sample 10 randomised transformations to obtain the expected gradients. The attack success rate on our LPC with EfficientNet-B5 backbone is 0.

To attack each point cloud, the four targeted attackers in Xiang *et al.*⁹ conduct 10 random searches where 500 iterations are done to find the optimal adversarial samples. We notice that for PointNet, the best attacks could occur at any time within these 500 iterations, but for our LPC, most optimal attacks happen at the first few iterations, as shown in Figure 8 with different learning rates for the perturbation attacker. This behaviour indicates that, in order to attack LPC, random search (a sampling-based method) has contributed more to most of the successful attacks, rather than gradient-based iterations. With the increase in the learning rate, the gradients start to find

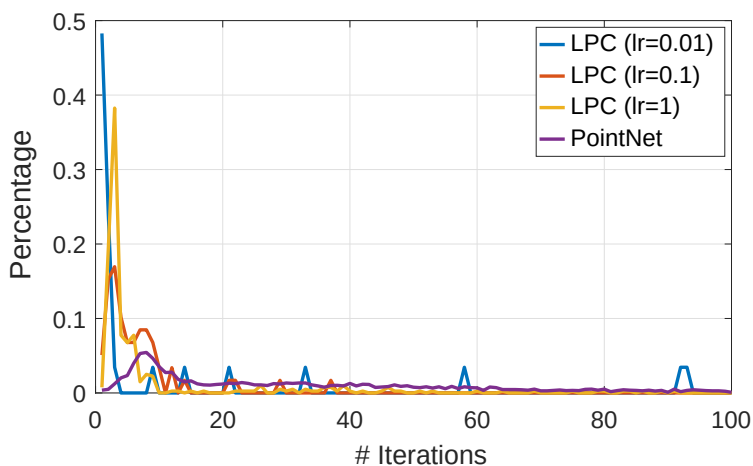


Fig. 8. Distribution comparison of successful perturbation attacks on ModelNet40. To avoid sparsity, LPC statistics are collected based on the cases from all three models.

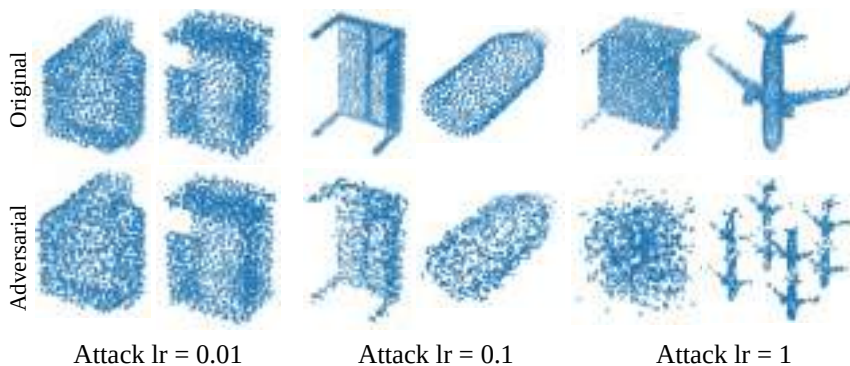


Fig. 9. Successful adversarial perturbation examples on ModelNet40 for LPC with EfficientNet-B5.

better adversarial samples. However, as we illustrate in Figure 9, with larger learning rates, the adversarial samples will look different from the original ones. For instance, with learning rate of 1, the point cloud of an aeroplane has been totally changed to a mixture of smaller aeroplane point clouds, which is not an adversarial attack anymore. To sum up, such analysis again demonstrates the great potential of implicit gradients against adversarial attacks.

5. Conclusion

In this chapter, we aim to address the problem of robust 3D point cloud classification by proposing a family of novel robust structured declarative classifiers, where a declarative node is defined by a constrained optimisation problem, such as the reconstruction of point clouds. The key insight in our approach is that the implicit gradients through the declarative node can help defend the adversarial attacks by leading them to wrong updating directions for inputs. We formulate the learning of our classifiers based on bilevel optimisation and further propose an effective and efficient instantiation, namely Lattice Point Classifier (LPC). The declarative node in LPC is defined as structured sparse coding in the permutohedral lattice, whose outputs, i.e., barycentric weights, are further transformed into images for classification using 2D CNNs. LPC is end-to-end trainable and achieves state-of-the-art performance on robust classification on ModelNet40 and ScanNet using seven different adversarial attackers.

Currently, the projection in the permutohedral lattice transformation in LPC is not learned but simply fixed. Also, more evaluations for demonstrating the robustness of our approach across different backbone networks and datasets are desirable. Therefore, in our future work, we will investigate more on how to properly learn the projection with its physical conditions and conduct more experiments to further demonstrate our robustness.

Acknowledgements

Z. Zhang was supported in part by NSF grant CCF-2006738. K. Li was supported in part by USDA NIFA under award no. 2019-67021-28996. G. Wang was partly supported by NSERC grant RGPIN-2021-04244.

References

1. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 652–660 (2017).

2. Y. Lyu, X. Huang, and Z. Zhang. Learning to segment 3D point clouds in 2d image space. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12255–12264 (2020).
3. Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3D ShapeNets: A deep representation for volumetric shapes. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 1912–1920 (2015).
4. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph CNN for learning on point clouds, *ACM Transactions on Graphics (TOG)* **38**(5), 1–12 (2019).
5. H. Zhou, D. Chen, J. Liao, K. Chen, X. Dong, K. Liu, W. Zhang, G. Hua, and N. Yu. LG-GAN: Label guided adversarial network for flexible targeted attack of point cloud based deep networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10356–10365 (2020).
6. Z. Xiang, D. J. Miller, S. Chen, X. Li, and G. Kesidis. A back-door attack against 3D point cloud classifiers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision (ICCV)*, pp. 7597–7607 (October, 2021).
7. C. Ma, W. Meng, B. Wu, S. Xu, and X. Zhang. Efficient joint gradient based attack against sor defense for 3D point cloud classification. In *Proceedings of the 28th ACM International Conference on Multimedia*, pp. 1819–1827 (2020).
8. C. Guo, J. Gardner, Y. You, A. G. Wilson, and K. Weinberger. Simple black-box adversarial attacks. In *International Conference on Machine Learning*, pp. 2484–2493 (2019).
9. C. Xiang, C. R. Qi, and B. Li. Generating 3D adversarial point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9136–9144 (2019).
10. M. Wicker and M. Kwiatkowska. Robustness of 3D deep learning in an adversarial setting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11767–11775 (2019).
11. D. Liu, R. Yu, and H. Su. Adversarial shape perturbations on 3D point clouds. In *European Conference on Computer Vision*, pp. 88–104 (2020).
12. R. Dollahite, K. Wang, K. Li, Y. Zhang, and Z. Zhang. Verifying adversarial robustness of 3D object detectors for autonomous vehicles. In *2022 IEEE MIT Undergraduate Research Technology Conference (URTC)*, pp. 1–4 (2022).
13. B. Yang, W. Luo, and R. Urtasun. PIXOR: Real-time 3D object detection from point clouds. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 7652–7660 (2018).

14. A. Geiger, P. Lenz, C. Stiller, and R. Urtasun. Vision meets robotics: The kitti dataset, *The International Journal of Robotics Research* **32**(11), 1231–1237 (2013).
15. I. J. Goodfellow, J. Shlens, and C. Szegedy. Explaining and harnessing adversarial examples. *arXiv preprint* arXiv:1412.6572 (2014).
16. N. Carlini, A. Athalye, N. Papernot, W. Brendel, J. Rauber, D. Tsipras, I. Goodfellow, A. Madry, and A. Kurakin. On evaluating adversarial robustness. *arXiv preprint* arXiv:1902.06705 (2019).
17. H. Zhou, K. Chen, W. Zhang, H. Fang, W. Zhou, and N. Yu. DUP-Net: Denoiser and upsampler network for 3D adversarial point clouds defense. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1961–1970 (2019).
18. Z. Wu, Y. Duan, H. Wang, Q. Fan, and L. J. Guibas. IF-Defense: 3D adversarial point cloud defense via implicit function based restoration. *arXiv preprint* arXiv:2010.05272 (2020).
19. H. Liu, J. Jia, and N. Z. Gong. PointGuard: Provably robust 3D point cloud classification. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 6186–6195 (2021).
20. A. Athalye, N. Carlini, and D. Wagner. Obfuscated gradients give a false sense of security: Circumventing defenses to adversarial examples. In *International Conference on Machine Learning*, pp. 274–283 (2018).
21. A. Kag, Z. Zhang, and V. Saligrama. RNNs incrementally evolving on an equilibrium manifold: A panacea for vanishing and exploding gradients? In *International Conference on Learning Representations* (2019).
22. B. Amos and J. Z. Kolter. OptNet: Differentiable optimization as a layer in neural networks. In *International Conference on Machine Learning*, pp. 136–145 (2017).
23. A. Agrawal, B. Amos, S. Barratt, S. Boyd, S. Diamond, and J. Z. Kolter. Differentiable convex optimization layers. In *Advances in Neural Information Processing Systems*, Vol. 32, pp. 9562–9574 (2019).
24. K. Lee, S. Maji, A. Ravichandran, and S. Soatto. Meta-learning with differentiable convex optimization. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10657–10665 (2019).
25. A. Rajeswaran, C. Finn, S. M. Kakade, and S. Levine. Meta-learning with implicit gradients. In *Advances in Neural Information Processing Systems*, Vol. 32, pp. 113–124 (2019).

26. S. Gould, R. Hartley, and D. J. Campbell. Deep declarative networks, *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2021).
27. A. Szlam, K. Gregor, and Y. LeCun. Fast approximations to structured sparse coding and applications to object classification. In *European Conference on Computer Vision*, pp. 200–213 (2012).
28. S. Karygianni and P. Frossard. Structured sparse coding for image denoising or pattern detection. In *2014 IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP)*, pp. 3533–3537 (2014).
29. W. Wei, L. Zhang, C. Tian, A. Plaza, and Y. Zhang. Structured sparse coding-based hyperspectral imagery denoising with intracluster filtering, *IEEE Transactions on Geoscience and Remote Sensing* **55**(12), 6860–6876 (2017).
30. A. Adams, J. Baek, and M. A. Davis. Fast high-dimensional filtering using the permutohedral lattice. In *Computer Graphics Forum*, Vol. 29, pp. 753–762 (2010).
31. H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz. SplatNet: Sparse lattice networks for point cloud processing. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 2530–2539 (2018).
32. X. Gu, Y. Wang, C. Wu, Y. J. Lee, and P. Wang. HPLFlowNet: Hierarchical permutohedral lattice flownet for scene flow estimation on large-scale point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3254–3263 (2019).
33. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. *arXiv preprint arXiv:1706.02413* (2017).
34. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on x-transformed points. In *Advances in Neural Information Processing Systems*, Vol. 31, pp. 820–830 (2018).
35. X. Yan, C. Zheng, Z. Li, S. Wang, and S. Cui. PointASNL: Robust point clouds processing using nonlocal neural networks with adaptive sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5589–5598 (2020).
36. Y. Wu, T. Marks, A. Cherian, S. Chen, C. Feng, G. Wang, and A. Sullivan. Unsupervised joint 3D object model learning and 6d pose estimation for depth-based instance segmentation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision Workshops*, pp. 1–10 (2019).
37. H. Su, S. Maji, E. Kalogerakis, and E. Learned-Miller. Multi-view convolutional neural networks for 3D shape recognition. In *Proceedings*

- of the *IEEE International Conference on Computer Vision*, pp. 945–953 (2015).
38. C. R. Qi, H. Su, M. Nießner, A. Dai, M. Yan, and L. J. Guibas. Volumetric and multi-view CNNs for object classification on 3D data. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 5648–5656 (2016).
 39. T. Yu, J. Meng, and J. Yuan. Multi-view harmonized bilinear network for 3D object recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 186–194 (2018).
 40. Z. Yang and L. Wang. Learning relationships for multi-view 3D object recognition. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 7505–7514 (2019).
 41. X. Mo, U. Sajid, and G. Wang. Stereo frustums: A siamese pipeline for 3D object detection, *Journal of Intelligent & Robotic Systems* **101**(1), 1–15 (2021).
 42. D. Maturana and S. Scherer. VoxNet: A 3D convolutional neural network for real-time object recognition. In *2015 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 922–928 (2015).
 43. G. Riegler, A. Osman Ulusoy, and A. Geiger. OctNet: Learning deep 3D representations at high resolutions. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 3577–3586 (2017).
 44. P.-S. Wang, Y. Liu, Y.-X. Guo, C.-Y. Sun, and X. Tong. O-CNN: Octree-based convolutional neural networks for 3D shape analysis, *ACM Transactions on Graphics (TOG)* **36**(4), 1–11 (2017).
 45. T. Le and Y. Duan. PointGrid: A deep network for 3D shape understanding. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 9204–9214 (2018).
 46. L. Landrieu and M. Boussaha. Point cloud oversegmentation with graph-structured deep metric learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7440–7449 (2019).
 47. W. Shi and R. Rajkumar. Point-GNN: Graph neural network for 3D object detection in a point cloud. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1711–1719 (2020).
 48. K. Fu, S. Liu, X. Luo, and M. Wang. Robust point cloud registration framework based on deep graph matching. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8893–8902 (2021).

49. G. Qian, A. Abualshour, G. Li, A. Thabet, and B. Ghanem. PU-GCN: Point cloud upsampling using graph convolutional networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11683–11692 (2021).
50. Y. Guo, H. Wang, Q. Hu, H. Liu, L. Liu, and M. Bennamoun. Deep learning for 3D point clouds: A survey, *IEEE Transactions on Pattern Analysis and Machine Intelligence* (2020).
51. M. Ozdag. Adversarial attacks and defenses against deep neural networks: a survey, *Procedia Computer Science* **140**, 152–161 (2018).
52. C. Wang, J. Wang, and Q. Lin. Adversarial attacks and defenses in deep learning: A survey. In *International Conference on Intelligent Computing*, pp. 450–461 (2021).
53. A. Chakraborty, M. Alam, V. Dey, A. Chattopadhyay, and D. Mukhopadhyay. A survey on adversarial attacks and defences, *CAAI Transactions on Intelligence Technology* **6**(1), 25–45 (2021).
54. D. Liu, R. Yu, and H. Su. Extending adversarial attacks and defenses to deep 3D point cloud classifiers. In *2019 IEEE International Conference on Image Processing (ICIP)*, pp. 2279–2283 (2019).
55. J. Yang, Q. Zhang, R. Fang, B. Ni, J. Liu, and Q. Tian. Adversarial attack and defense on point sets. *arXiv preprint arXiv:1902.10899* (2019).
56. J. Kim, B.-S. Hua, T. Nguyen, and S.-K. Yeung. Minimal adversarial examples for deep learning on 3D point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 7797–7806 (2021).
57. T. Zheng, C. Chen, J. Yuan, B. Li, and K. Ren. PointCloud saliency maps. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1598–1606 (2019).
58. A. Hamdi, S. Rojas, A. Thabet, and B. Ghanem. AdvPC: Transferable adversarial perturbations on 3D point clouds. In *European Conference on Computer Vision*, pp. 241–257 (2020).
59. Y. Zhao, Y. Wu, C. Chen, and A. Lim. On isometry robustness of deep 3D point cloud models under adversarial attacks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1201–1210 (2020).
60. R. B. Rusu, Z. C. Marton, N. Blodow, M. Dolha, and M. Beetz. Towards 3D point cloud based object maps for household environments, *Robotics and Autonomous Systems* **56**(11), 927–941 (2008).
61. T. Tsai, K. Yang, T.-Y. Ho, and Y. Jin. Robust adversarial objects against deep learning models. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34, pp. 954–962 (2020).

62. X. Dong, D. Chen, H. Zhou, G. Hua, W. Zhang, and N. Yu. Self-robust 3D point recognition via gather-vector guidance. In *2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 11513–11521 (2020).
63. F. Tramèr, A. Kurakin, N. Papernot, I. Goodfellow, D. Boneh, and P. McDaniel. Ensemble adversarial training: Attacks and defenses. *arXiv preprint arXiv:1705.07204* (2017).
64. J. Tu, M. Ren, S. Manivasagam, M. Liang, B. Yang, R. Du, F. Cheng, and R. Urtasun. Physically realizable adversarial examples for LiDAR object detection. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 13716–13725 (2020).
65. J. Zhang, B. Liu, L. Chen, B. Ouyang, J. Zhu, M. Kuang, H. Wang, and Y. Meng. The art of defense: Letting networks fool the attacker. *arXiv preprint arXiv:2104.02963* (2021).
66. J. Zhang, L. Chen, B. Ouyang, B. Liu, J. Zhu, Y. Chen, Y. Meng, and D. Wu. PointCutMix: Regularization strategy for point cloud classification. *arXiv preprint arXiv:2101.01461* (2021).
67. V. Jampani, M. Kiefel, and P. V. Gehler. Learning sparse high dimensional filters: Image filtering, dense crfs and bilateral neural networks. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 4452–4461 (2016).
68. R. A. Rosu, P. Schütt, J. Quenzel, and S. Behnke. LatticeNet: Fast spatio-temporal point cloud segmentation using permutohedral lattices, *Autonomous Robots* **46**(1), 1–16 (2021).
69. J. F. Bard. *Practical Bilevel Optimization: Algorithms and Applications*, Vol. 30. Springer Science & Business Media, New York, NY: Springer (2013).
70. K. Ji, J. Yang, and Y. Liang. Bilevel optimization: Convergence analysis and enhanced design. In *International Conference on Machine Learning*, pp. 4882–4892 (2021).
71. Z. Zhang, Y. Xu, J. Yang, X. Li, and D. Zhang. A survey of sparse representation: Algorithms and applications, *IEEE Access* **3**, 490–530 (2015).
72. K. Simonyan and A. Zisserman. Very deep convolutional networks for large-scale image recognition. *arXiv preprint arXiv:1409.1556* (2014).
73. K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 770–778 (2016).
74. M. Tan and Q. Le. EfficientNet: Rethinking model scaling for convolutional neural networks. In *International Conference on Machine Learning*, pp. 6105–6114 (2019).

75. A. Dai, A. X. Chang, M. Savva, M. Halber, T. Funkhouser, and M. Nießner. ScanNet: Richly-annotated 3D reconstructions of indoor scenes. In *Proceedings of Computer Vision and Pattern Recognition (CVPR)*. IEEE (2017).
76. D. P. Kingma and J. Ba. Adam: A method for stochastic optimization. *arXiv preprint* arXiv:1412.6980 (2014).
77. N. Srivastava, G. Hinton, A. Krizhevsky, I. Sutskever, and R. Salakhutdinov. Dropout: A simple way to prevent neural networks from overfitting, *The Journal of Machine Learning Research* **15**(1), 1929–1958 (2014).
78. S. Peng, M. Niemeyer, L. Mescheder, M. Pollefeys, and A. Geiger. Convolutional occupancy networks. In *Computer Vision–ECCV 2020: 16th European Conference*, Glasgow, UK, August 23–28, 2020, Proceedings, Part III 16, pp. 523–540 (2020).
79. X. Yan. PointNet/PointNet++ pytorch (2019). https://github.com/yanx27/Pointnet_Pointnet2_pytorch.
80. J. E. Dennis Jr and R. E. Welsch. Techniques for nonlinear least squares and robust regression, *Communications in Statistics-simulation and Computation* **7**(4), 345–359 (1978).
81. M. Ester, H.-P. Kriegel, J. Sander, X. Xu, *et al.* A density-based algorithm for discovering clusters in large spatial databases with noise. In *KDD*, Vol. 96, pp. 226–231 (1996).
82. N. Carlini and D. Wagner. Towards evaluating the robustness of neural networks. In *2017 IEEE Symposium on Security and Privacy (SP)*, pp. 39–57 (2017).
83. A. Athalye, L. Engstrom, A. Ilyas, and K. Kwok. Synthesizing robust adversarial examples. In *International Conference on Machine Learning*, pp. 284–293 (2018).

This page intentionally left blank

Chapter 8

Towards Inference Stage Robust 3D Point Cloud Recognition

Yongyi Su^{*,§}, Xun Xu^{†,¶}, and Kui Jia^{*,||}

^{*}*School of Electronic and Information Engineering, South China University of Technology, Guangzhou 510641, China*

[†]*Institute of Infocomm Research (I2R), Agency for Science, Technology and Research (A*STAR), 1 Fusionopolis Way, Connexis North, Singapore 138632*

[§]*eesuyongyi@mail.scut.edu.cn*

[¶]*xu_xun@i2r.a-star.edu.sg*

^{||}*kuijia@scut.edu.cn*

Deploying models on target domain data subject to distribution shift requires adaptation. Test-time training (TTT) emerges as a solution to this adaptation under a realistic scenario where access to full source domain data is not available and instant inference on target domain is required. Despite many efforts into TTT, there is a confusion over the experimental settings, thus leading to unfair comparisons. In this work, we first revisit TTT assumptions and categorise TTT protocols by two key factors. Among the multiple protocols, we adopt a realistic sequential test-time training (sTTT) protocol, under which we further develop a *test-time anchored clustering* (TTAC) approach to enable stronger test-time feature learning. TTAC discovers clusters in both source and target domain and match the target clusters to the source ones to improve generalisation. Pseudo label filtering and iterative updating are developed to improve the effectiveness and efficiency of anchored clustering. We demonstrate that under all TTT protocols TTAC consistently

outperforms the state-of-the-art methods on five TTT datasets. We hope this work will provide a fair benchmarking of TTT methods and future research should be compared within respective protocols.

1. Introduction

The recent success in deep learning is attributed to the availability of large labelled data^{1,2} and the assumption of i.i.d. between training and test datasets. Such assumptions could be violated when test data features a drastic difference from the training data, e.g., training on synthetic images and test on real ones, and this is often referred to as domain shift.^{3,4} To tackle this issue, domain adaptation (DA)⁵ emerges and the labelled training data and unlabelled testing data are often referred to as source and target data/domains respectively.

The existing DA works either require the access to both source and target domain data during training⁶ or training on multiple domains simultaneously.⁷ The former approach renders the methods restrictive to limited scenarios where source domain data is always available during adaptation while the latter ones are computationally more expensive. To alleviate the reliance on source domain data, which may be inaccessible due to privacy issues or storage overhead, source-free domain adaptation (SFDA) emerges which handles DA on target data without access to source data.^{8–12} SFDA is often achieved through self-training,⁸ self-supervised learning¹² or introducing prior knowledge⁸ and it requires multiple training epochs on the full target data to allow convergence. Despite easing the dependence on source data, SFDA has major drawbacks in a more realistic domain adaptation scenario where test data arrives in a stream and inference or prediction must be taken instantly, and this setting is often referred to as test-time training (TTT) or adaptation (TTA).^{12–15} Despite the attractive feature of adaption at time test, we notice a confusion of what defines a test-time training and as a result comparing apples and oranges happens frequently in the community. In this work, we first categorise TTT by two key factors after summarising various definitions made in existing works. First, under a realistic TTT setting, test samples are sequentially streamed and prediction must be made instantly upon the arrival of a new test sample. More specifically, the prediction of test sample X_T , arriving at time stamp T , should not be affected by any subsequent samples, $\{X_t\}_{t=T+1 \dots \infty}$. Throughout this work, We refer to the sequential streaming as **one-pass**

adaptation protocol and any other protocols violating this assumption are called **multi-pass adaptation** (model may be updated on all test data for multiple epochs before inference). Second, we notice some recent works must **modify source domain training loss**, e.g., by introducing additional self-supervised branch, to allow more effective TTT.^{12,13} This will introduce additional overhead in the deployment of TTT because re-training on some source dataset, e.g., ImageNet, is computationally expensive. In this work, we aim to tackle on the most realistic and challenging TTT protocol, i.e., one-pass test time training with no modifications to training objective. This setting is similar to TTA proposed in¹⁴ except for not restricting access to a light-weight information from the source domain. Given the objective of TTT being efficient adaptation at time-time, this assumption is computationally efficient and improves TTT performance substantially. We name this new TTT protocol as **sequential test time training (sTTT)**.

We propose two techniques to enable efficient and accurate sTTT. (i) We are inspired by the recent progresses in unsupervised domain adaptation¹⁶ that encourages testing samples to form clusters in the feature space. However, separately learning to cluster in the target domain without regularisation from source domain does not guarantee improved adaptation.¹⁶ To overcome this challenge, we identify clusters in both the source and target domains through a mixture of Gaussians with each component Gaussian corresponding to one category. Provided with the category-wise statistics from source domain as anchors, we match the target domain clusters to the anchors by minimising the KL-Divergence as the training objective for sTTT. Therefore, we name the proposed method *test-time anchored clustering* (TTAC). Since test samples are sequentially streamed, we develop an exponential moving averaging strategy to update the target domain cluster statistics to allow gradient-based optimisation. (ii) Each component Gaussian in the target domain is updated by the test sample features that are assigned to the corresponding category. Thus, incorrect assignments (pseudo labels) will harm the estimation of component Gaussian. To tackle this issue, we are inspired by the correlation between network’s stability and confidence and pseudo label accuracy,^{17,18} and propose to filter out potential incorrect pseudo labels. Component Gaussians are then updated by the samples that have passed the filtering. To exploit the filtered out samples, we incorporate a global feature alignment¹² objective.

We also demonstrate TTAC is compatible with existing TTT techniques, e.g., contrastive learning branch,¹² if source training loss is allowed to be modified. The contributions of this work are summarised as follows.

- In light of the confusions within TTT works, we provide a categorisation of TTT protocols by two key factors. Comparison of TTT methods is now fair within each category.
- We adopt a realistic TTT setting, namely sTTT. To improve test-time feature learning, we propose TTAC by matching the statistics of the target clusters to the source ones. The target statistics are updated through moving averaging with filtered pseudo labels.
- The proposed method is complementary to existing TTT method and is demonstrated on five TTT datasets, achieving the state-of-the-art performance under all categories of TTT protocols.

2. Related Work

Unsupervised domain adaptation. Domain adaptation aims to improve model generalisation when source and target data are not drawn i.i.d. When target data are unlabelled, UDA^{6,19} learns domain invariant feature representations on both source and target domains to improve generalisation. Follow-up works improve DA by minimising a divergence,^{20–22} adversarial training²³ or discovering cluster structures in the target data.¹⁶ Apart from formulating DA within a task-specific model, re-weighting has been adopted for domain adaptation by selectively up-weighting conducive samples in the source domain.^{24,25} Despite the efforts in UDA, it is inevitable to access the source domain data which may be not accessible due to privacy issues, storage overhead, etc. Deploying DA in more realistic scenarios has inspired research into SFDA and TTT/TTA.

Source-free domain adaptation. Without the access to source data, SFDA develops domain adaptation through self-training,^{8,9,15} self-supervised training¹² or clustering in the target domain.¹⁰ It has been demonstrated that SFDA performs well on seminal domain adaptation datasets even compared against UDA methods.¹⁶ Nevertheless, SFDA still requires access to all testing data and training must be carried out iteratively on the testing data. In a more realistic DA scenario where inference and adaptation must be implemented

simultaneously, SFDA will no longer be effective. Moreover, some statistical information on the source domain does not pose privacy issues and can be exploited to further improve adaptation on target data.

Test-time training. Collecting enough samples from target domain and adapt models in an offline manner restricts the application to adapting to static target domain. To allow fast and online adaptation, TTT^{13,26} or TTA¹⁴ emerges. Despite many recent works claiming to be TTT, we notice a severe confusion over the definition of TTT. In particular, whether training objective must be modified^{12,13} and whether sequential inference on target domain data is possible.^{14,15} Therefore, to reflect the key challenges in TTT, we define a setting called sequential test-time training (sTTT) which neither modifies the training objective nor violates sequential inference. Under the more clear definition, some existing works, e.g., TTT¹³ and TTT++¹² is more likely to be categorised into SFDA. Several existing works^{14,15} can be adapted to the sTTT protocol. Tent¹⁴ proposed to adjust affine parameters in the batchnorm layers to adapt to target domain data. The high TTA efficiency inevitably leads to limited performance gain on the target domain. T3A¹⁵ further proposed to update classifier prototype through pseudo labelling. Despite being efficient, updating classifier prototype alone does not affect feature representation for the target domain. Target feature may not form clusters at all when the distribution mismatch between source and target is large enough. In this work we propose to simultaneously cluster on the target domain and match target clusters to source domain classes, namely anchored clustering. To further constrain feature update, we introduce additional global feature alignment and pseudo label filtering. Through the introduced anchored clustering, we achieve TTT of more network parameters and achieve the state-of-the-art performance.

3. Methodology

In this section, we first introduce the anchored clustering objective for test-time training through pseudo labelling and then describe an efficient iterative updating strategy. An overview of the proposed pipeline is illustrated in Figure 1.

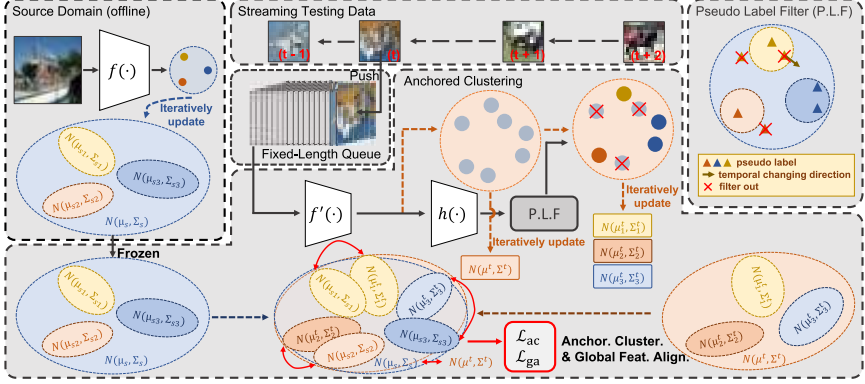


Fig. 1. Overview of TTAC pipeline. (i) In the source domain, we calculate category-wise and global statistics as anchors. (ii) In the testing stage, samples are sequentially streamed and pushed into a fixed-length queue. Clusters in target domain are identified through anchored clustering with pseudo label filtering. Target clusters are then matched to the anchors in source domain to achieve TTT.

3.1. Anchored Clustering for TTT

Discovering cluster structures in the target domain has been demonstrated effective for unsupervised domain adaptation¹⁶ and we develop an anchored clustering on the test data alone. We first use a mixture of Gaussians to model the clusters in the target domain, here each component Gaussian represents one discovered cluster. We further use the distributions of each category in the source domain as anchors for the target distribution to match against. In this way, test data features can simultaneously form clusters and the clusters are associated with source domain categories, resulting in improved generalisation to target domain. Formally, we first write the mixture of Gaussians in the source and target domains $p_s(x) = \sum_k \alpha_k \mathcal{N}(\mu_{sk}, \Sigma_{sk})$, $p_t(x) = \sum_k \beta_k \mathcal{N}(\mu_{tk}, \Sigma_{tk})$, where $\{\mu_k \in \mathbb{R}^d, \Sigma_k \in \mathbb{R}^{d \times d}\}$ represent one cluster in the source/target domain and d is the dimension of feature embedding.

Anchored clustering can be achieved by matching the above two distributions and one may directly minimise the KL-Divergence between the two distribution. Nevertheless, this is non-trivial because the KL-Divergence between two mixture of Gaussians has no closed-form solution which prohibits efficient gradient-based optimisation. Despite some approximations exist,²⁷ without knowing the semantic

labels for each Gaussian component, even a good match between two mixture of Gaussians does not guarantee target clusters are aligned to the correct source ones and this will severely harm the performance of TTT. In light of these challenges, we propose a category-wise alignment. Specifically, we allocate the same number of clusters in both source and target domains and each target cluster is assigned to one source cluster. We can then minimise the KL-Divergence between each pair of clusters as in Equation (1).

$$\begin{aligned}\mathcal{L}_{ac} &= \sum_k D_{KL}(\mathcal{N}(\mu_{sk}, \Sigma_{sk}) || \mathcal{N}(\mu_{tk}, \Sigma_{tk})) \\ &= \sum_k -H(\mathcal{N}(\mu_{sk}, \Sigma_{sk})) + H(\mathcal{N}(\mu_{sk}, \Sigma_{sk}), \mathcal{N}(\mu_{tk}, \Sigma_{tk})) \quad (1)\end{aligned}$$

The KL-Divergence can be further decomposed into the entropy $H(\mathcal{N}(\mu_{sk}, \Sigma_{sk}))$ and cross-entropy $H(\mathcal{N}(\mu_{sk}, \Sigma_{sk}), \mathcal{N}(\mu_{tk}, \Sigma_{tk}))$. It is commonly true that the source reference distribution $P_s(x)$ is fixed thus the entropy term is a constant C and only the cross-entropy term is to be optimised. Given the closed-form solution to the KL-Divergence between two Gaussian distributions, we now write the anchored clustering objective as,

$$\begin{aligned}\mathcal{L}_{ac} &= \sum_k \left\{ \log \sqrt{2\pi^d |\Sigma_{tk}|} + \frac{1}{2}(\mu_{tk} - \mu_{sk})^\top \right. \\ &\quad \left. \times \Sigma_{tk}^{-1}(\mu_{tk} - \mu_{sk}) + tr(\Sigma_{tk}^{-1} \Sigma_{sk}) \right\} + C \quad (2)\end{aligned}$$

The source cluster parameters can be estimated in an offline manner. These information will not cause any privacy leakage and only introduces a small computation and storage overheads. In the next section, we elaborate clustering in the target domain.

3.2. Clustering through Pseudo Labelling

In order to test-time train network with anchored clustering loss, one must obtain target cluster parameters $\{\mu_{tk}, \Sigma_{tk}\}$. For a minibatch of target test samples $\mathcal{B}^t = \{x_i\}_{i=1 \dots N_B}$ at timestamp t , we first denote the predicted posterior as $P^t = softmax(h(f(x_i))) \in [0, 1]^{B \times K}$ where $softmax(\cdot)$, $h(\cdot)$ and $f(\cdot)$ respectively denote a standard

softmax function, the classifier head and backbone network. The pseudo labels are obtained via $\hat{y}_i = \arg \max_k P_{ik}^t$. Given the predicted pseudo labels we could estimate the mean and covariance for each component Gaussian with the pseudo labelled testing samples. However, pseudo labels are always subject to model's discrimination ability. The error rate for pseudo labels is often high when the domain shift between source and target is large, directly updating the component Gaussian is subject to erroneous pseudo labels, a.k.a. confirmation bias.²⁸ To reduce the impact of incorrect pseudo labels, we first adopt a light-weight temporal consistency (TC) pseudo label filtering approach. Compared to co-teaching²⁹ or meta-learning³⁰ based methods, this light-weight method does not introduce additional computation overhead and is therefore more suitable for TTT. Specifically, to alleviate the impact from the noisy predictions, we calculate the temporal exponential moving averaging posteriors $\tilde{P}^t \in [0, 1]^{N \times K}$ as follows,

$$\tilde{P}_i^t = (1 - \xi) * \tilde{P}_i^{t-1} + \xi * P_i^t, \quad \text{s.t. } \tilde{P}_i^0 = P_i^0 \quad (3)$$

The temporal consistency filtering is realised as in Equation (4) where τ_{TC} is a threshold determining the maximally allowed difference in the most probable prediction over time. If the posterior deviate from historical value too much, it will be excluded from target domain clustering.

$$F_i^{\text{TC}} = \mathbb{1}((P_{ik}^t - \tilde{P}_{ik}^{t-1}) > \tau_{\text{TC}}), \quad \text{s.t. } \hat{k} = \arg \max_k (P_{ik}^t) \quad (4)$$

Due to the sequential inference, test samples without enough historical predictions may still pass the TC filtering. So, we further introduce an additional pseudo label filter directly based on the posterior probability as,

$$F_i^{\text{PP}} = \mathbb{1}(\tilde{P}_{ik}^t > \tau_{\text{PP}}) \quad (5)$$

By filtering out potential incorrect pseudo labels, we update the component Gaussian only with the leftover target samples as follows.

$$\begin{aligned} \mu_{tk} &= \frac{\sum_i F_i^{\text{TC}} F_i^{\text{PP}} \mathbb{1}(\hat{y}_i = k) f(x_i)}{\sum_i F_i^{\text{TC}} F_i^{\text{PP}} \mathbb{1}(\hat{y}_i = k)}, \\ \Sigma_{tk} &= \frac{\sum_i F_i^{\text{TC}} F_i^{\text{PP}} \mathbb{1}(\hat{y}_i = k) (f(x_i) - \mu_{tk})^\top (f(x_i) - \mu_{tk})}{\sum_i F_i^{\text{TC}} F_i^{\text{PP}} \mathbb{1}(\hat{y}_i = k)} \end{aligned} \quad (6)$$

3.3. Global Feature Alignment

As discussed above, test samples that do not pass the filtering will not contribute to the estimation of target clusters. Hence, anchored clustering may not reach its full potential without the filtered test samples. To exploit all available test samples, we propose to align global target data distribution to the source one. We define the global feature distribution of the source data as $\hat{p}_s(x) = \mathcal{N}(\mu_s, \Sigma_s)$ and the target data as $\hat{p}_t(x) = \mathcal{N}(\mu_t, \Sigma_t)$. To align two distributions, we again minimise the KL-Divergence as,

$$\mathcal{L}_{\text{ga}} = D_{\text{KL}}(\hat{p}_s(x) || \hat{p}_t(x)) \quad (7)$$

Similar idea has appeared in¹² which directly matches the moments between source and target domains²² by minimising the F -norm for the mean and covariance, i.e., $\|\mu_t - \mu_s\|_2^2 + \|\Sigma_t - \Sigma_s\|_F^2$. However, designed for matching complex distributions represented as drawn samples, central moment discrepancy²² requires summing infinite central moment discrepancies and the ratios between different order moments are hard to estimate. For matching two parametrised Gaussian distributions KL-Divergence is more convenient with good explanation from a probabilistic point of view. Finally, we add a small constant to the diagonal of Σ for both source and target domains to increase the condition number for better numerical stability.

3.4. Efficient Iterative Updating

Despite the distribution for source data can be trivially estimated from all available training data in a totally offline manner, estimating the distribution for target domain data is not equally trivial, in particular under the sTTT protocol. In a related research,¹² a dynamic queue of test data features are preserved to dynamically estimate the statistics, which will introduce additional memory footprint.¹² To alleviate the memory cost we propose to iteratively update the running statistics for Gaussian distribution. Formally, we define t th test minibatch as $\mathcal{B}^t = \{x_i\}_{i=1 \dots N_B}$. Denoting the running mean and covariance at step t as μ^t and Σ^t , we present the rules to update the mean and covariance in Equation (8). More detailed derivations and

update rules for per cluster statistics are deferred to the Appendix.

$$\begin{aligned}
\mu^t &= \mu^{t-1} + \delta^t, \\
\Sigma^t &= \Sigma^{t-1} + a^t \sum_{x_i \in \mathcal{B}} \{(f(x_i) - \mu^{t-1})^\top (f(x_i) - \mu^{t-1}) - \Sigma^{t-1}\} - \delta^{t\top} \delta^t \\
\delta^t &= a^t \sum_{x_i \in \mathcal{B}} (f(x_i) - \mu^{t-1}), \quad N^t = N^{t-1} + |\mathcal{B}^t|, \quad a^t = \frac{1}{N^t},
\end{aligned} \tag{8}$$

Additionally, N^t grows larger overtime. New test samples will have smaller contribution to the update of target domain statistics when N^t is large enough. As a result, the gradient calculated from current minibatch will vanish. To alleviate this issue, we impose a clip on the value of a^t as below. As such, the gradient can maintain a minimal scale even if N^t is very large.

$$a^t = \begin{cases} \frac{1}{N^t} & N^t < N_{\text{clip}} \\ \frac{1}{N_{\text{clip}}} & \text{others} \end{cases} \tag{9}$$

3.5. Self-Training for TTT

Self-training (ST) has been widely adopted in semi-supervised learning where predictions on unlabelled data are admitted as pseudo labels, and model is trained with the pseudo labels.^{18,31} In this work, we explore employing self-training for TTT. Blindly taking all pseudo labels for training has been demonstrated to deteriorate the performance as incorrect pseudo labels act as noisy labels and a high percentage of noisy label is harmful for model training. This phenomenon is also referred to as confirmation bias.²⁸ As demonstrated in the empirical evaluations, self-training alone is not guaranteed to outperform competing methods. The performance may even degrades after observing enough testing samples. To reduce the impact of confirmation bias, we first propose to employ anchored clustering as regularisation. As anchored clustering allows better alignment between source and target feature distributions, self-training is able to benefit from more accurate pseudo labels and the model is less likely to be harmed by the wrong pseudo labels. This can be achieved by simultaneously optimising anchored clustering losses and self-training loss.

In addition, we further take an approach similar to¹⁸ by filtering out less confident pseudo labels for self-training as in Equation (10).

$$\mathcal{L}_{\text{st}} = \frac{1}{|\mathcal{B}_t|} \sum_{x_i \in \mathcal{B}_t} \mathbb{1} \left(\max_k (q_k(\mathcal{W}(x_i))) \geq \tau_{\text{st}} \right) H(\hat{y}_i, q(\mathcal{A}(x_i))) \quad (10)$$

where $q(\cdot) = \sigma(h(f(\cdot)))$ denotes the probabilistic posterior, $\hat{y}_i = \arg \max_k (q(\mathcal{W}(x_i)))$ denotes the predicted pseudo label, $\mathcal{W}(\cdot)$ denotes a weak augmentation operation consisting of RandomHorizontalFlip and RandomResizedCrop, $\mathcal{A}(\cdot)$ denotes a strong augmentation operation implemented as RandAugment,³² and τ_{cr} denotes the confidence threshold.

Adaptive thresholding: The above self-training approach often adopts a fixed threshold to filter out less confident testing samples. Nevertheless, as the model could be biased and challenges of learning different classes are not equal. To further improve the robustness of self-training on diverse testing data distributions, we introduce an adaptive threshold for each class. In specific, the threshold for the confidence of the pseudo labels will be updated as the training processes. In each iteration, a self-adaptive global threshold is calculated according to the following rule, where C is the number of classes of the dataset, q_b is the softmax result of the model's prediction on the current batch of data and $\max(q_b)$ is the confidence of the predicted pseudo label. λ is the momentum decay of EMA.

$$\tau_t = \begin{cases} \frac{1}{C}, & \text{if } t = 0. \\ \lambda \tau_{t-1} + (1 - \lambda) \frac{1}{\mu B} \sum_{b=1}^{\mu B} \max(q_b), & \text{otherwise.} \end{cases} \quad (11)$$

The global threshold works for samples from all the predicted class. We further need local thresholds for each different predicted class of different samples by calculating the expectation of the model's predictions on each class to estimate the class-specific learning status as below, where k indicates class index and $q_b(k)$ is the posterior probability on class k .

$$\tilde{p}_t(k) = \begin{cases} \frac{1}{C}, & \text{if } t = 0. \\ \lambda \tilde{p}_{t-1} + (1 - \lambda) \frac{1}{\mu B} \sum_{b=1}^{\mu B} \max(q_b), & \text{otherwise.} \end{cases} \quad (12)$$

Finally, the self-adaptive threshold for each class in one mini-batch is given in Equation (13). The adaptive threshold is used for self-training following Equation (10).

$$\tau_t(k) = \frac{\tilde{p}_t(k)}{\max\{\tilde{p}_t(k) : k \in [K]\}} \cdot \tau_t \quad (13)$$

3.6. TTAC Training Algorithm

We summarise the training algorithm for the TTAC in Algorithm 1. For effective clustering in target domain, we allocate a fixed length memory space, denoted as $\mathcal{C} \in \mathcal{R}^{N_C \times N \times 3}$, to store the recent testing samples. In the sTTT protocol, we first make instant prediction on each testing sample, and only update the model when N_B testing samples are accumulated. TTAC can be efficiently implemented, e.g., with two devices, one is for continuous inference and another is for model updating.

Algorithm 1 TTAC++ Algorithm

input : A new testing sample batch $\mathcal{B}^t = \{x_i\}_{i=tN_B \dots (t+1)N_B}$.
 # Update the testing sample queue $\mathcal{C}$.
 $\mathcal{C}^t = \mathcal{C}^{t-1} \setminus \mathcal{B}^{t-N_C/N_B}$, $\mathcal{C}^t = \mathcal{C}^t \cup \mathcal{B}^t$
for 1 to N_{itr} **do**
 for minibatch $\{x_i^t\}_{i=1}^N$ in $\mathcal{C}^t$ **do**
 # Generate weak and strong augmented samples
 $\mathcal{W}(x_i^t)$, $\mathcal{A}(x_i^t)$
 # Obtain the predicted posterior and pseudo labels
 Obtain $\hat{y}_i$ by Equation (10).
 # Update the global and per-cluster running mean and covariance by Equation (8) with $\mathcal{W}(x_i^t)$
 μ^t , Σ^t , $\{\mu_k^t\}$, $\{\Sigma_k^t\}$
 # Calculate the anchored clustering losses according to Equations (2) and 7
 $\mathcal{L}_{\text{ac}} + \lambda_1 \mathcal{L}_{\text{ga}}$
 # Calculate self-training loss according to Equation (10)
 $\mathcal{L}_{\text{st}}$
 # One step gradient descent on the total loss
 $\mathcal{L}_{\text{ac}} + \lambda_1 \mathcal{L}_{\text{ga}} + \lambda_2 \mathcal{L}_{\text{st}}$

4. Experiment

In this section, we first compare various existing methods based on the two key factors. Evaluation is then carried out on five TTT datasets. We then ablate the components of TTAC. Further analysis on the cumulative performance, qualitative insights, etc. are provided at the end.

4.1. Datasets

We evaluate TTT on ModelNet40-C, which consists of 15 common and realistic corruptions of point cloud data, with 9,843 training samples and 2,468 test samples, and report the classification error rate (%) throughout the experiment section.

4.2. Experiment Settings

Hyperparameters. We use the DGCNN³³ on ModelNet40-C. We optimise the backbone network $f(\cdot)$ by SGD with momentum on all datasets. On ModelNet40-C we set BS = 64 and LR = 0.001.

TTT protocols. We categorise TTT based on two key factors. First, whether the training objective must be changed during training on the source domain, we use Y and N to indicate if training objective is allowed to be changed or not respectively. Second, whether testing data is sequentially streamed and predicted, we use O to indicate a sequential **One-pass** inference and M to indicate non-sequential inference, a.k.a. **Multi-pass** inference. With the above criteria, we summarise 4 test-time training protocols, namely N-O, Y-O, N-M and Y-M, and the strength of the assumption increases from the first to the last protocols. Ours sTTT setting makes the weakest assumption, i.e., N-O. Existing methods are categorised by the four TTT protocols, we notice that some methods can operate under multiple protocols.

Competing methods. We compare the following TTT methods. Direct testing (TEST) without adaptation simply do inference on target domain with source domain model. Test-time training (TTT-R)¹³ jointly trains the rotation-based self-supervised task and the classification task in the source domain, and then only train the

rotation-based self-supervised task in the streaming test samples and make the predictions instantly. The default method is classified into the Y-M protocol. Test-time normalisation (BN)³⁴ moving average updates the batch normalisation statistics by streamed data. The default method follows N-M protocol and can be adapted to N-O protocol. Test-time entropy minimisation (TENT)¹⁴ updates the parameters of all batch normalisation by minimising the entropy of the model predictions in the streaming data. By default, TENT follows the N-O protocol and can be adapted to N-M protocol. Test-time classifier adjustment (T3A)¹⁵ computes target prototype representation for each category using streamed data and make predictions with updated prototypes. T3A follows the N-O protocol by default. Source Hypothesis Transfer (SHOT)⁸ freezes the linear classification head and trains the target-specific feature extraction module by exploiting balanced category assumption and self-supervised pseudo-labelling in the target domain. SHOT by default follows the N-M protocol and we adapt it to N-O protocol. TTT++¹² aligns source domain feature distribution, whose statistics are calculated offline, and target domain feature distribution by minimising the F-norm between the mean covariance. TTT++ follows the Y-M protocol and we adapt it to N-O (removing contrastive learning branch) and Y-O protocols. We present our own approach without self-training, TTAC, which only requires a single pass on the target domain and does not have to modify the source training objective. We further modify TTAC for Y-O, N-M and Y-M protocols, for Y-O and Y-M we incorporate an additional contrastive learning branch.¹² We could further combine TTAC with additional diversity loss and entropy minimisation loss introduced in SHOT,⁸ denoted as TTAC+SHOT. Finally, we present the model combining anchored clustering and self-training, TTAC+ST, which simultaneously optimises the anchored clustering loss and self-training loss on testing data stream.

4.3. *TTT on Corrupted Target Domain*

We present the TTT results on ModelNet40C datasets in Table 1. We make the following observations from the results.

sTTT (N-O) protocol. We first analyse the results under the proposed sTTT (N-O) protocol. Our method outperforms all competing

Table 1. Comparison under different TTT protocols. Y/N indicates modifying source domain training objective or not. O/M indicate one pass or multiple passes TTT. C10-C, C100-C and MN40-C refer to CIFAR10-C, CIFAR100-C and ModelNet40-C datasets respectively. All numbers indicate error rate in percentage.

Method	TTT Protocol	Assum. Strength	C10-C	C100-C	MN40-C
TEST	—	—	29.15	60.34	34.62
BN ³⁴	N-O	Weak	15.49	43.38	26.53
TENT ¹⁴	N-O	Weak	14.27	40.72	26.38
T3A ¹⁵	N-O	Weak	15.44	42.72	24.57
SHOT ⁸	N-O	Weak	13.95	39.10	19.71
TTT++ ¹²	N-O	Weak	13.69	40.32	—
TTAC (Ours)	N-O	Weak	10.94	36.64	22.30
TTAC+SHOT (Ours)	N-O	Weak	10.99	36.39	19.21
TTT++ ¹²	Y-O	Medium	13.00	35.23	—
TTAC (Ours)	Y-O	Medium	10.69	34.82	—
BN ³⁴	N-M	Medium	15.70	43.30	26.49
TENT ¹⁴	N-M	Medium	12.60	36.30	21.23
SHOT ⁸	N-M	Medium	14.70	38.10	15.99
TTAC (Ours)	N-M	Medium	9.42	33.55	16.77
TTAC+SHOT (Ours)	N-M	Medium	9.54	32.89	15.04
TTT-R ¹³	Y-M	Strong	14.30	40.40	—
TTT++ ¹²	Y-M	Strong	9.80	34.10	—
TTAC (Ours)	Y-M	Strong	8.52	30.57	—

ones by a large margin, e.g., 3% improvement is observed on both CIFAR10-C and CIFAR100-C from the previous best (TTT++). We further combine TTAC with the class balance assumption made in SHOT (TTAC+SHOT). With the stronger assumptions our method can further improve upon TTAC alone, in particular on ModelNet40-C dataset. This result demonstrates TTAC’s compatibility with existing methods.

Alternative protocols. We further compare different methods under N-M, Y-O and Y-M protocols. Under the Y-O protocol, TTT++¹² modifies the source domain training objective by incorporating a contrastive learning branch.³⁵ To compare with TTT++, we also include the contrastive branch and observe a clear improvement

Table 2. TTT on CIFAR10.1.

TEST	BN	TTT-R	TENT	SHOT	TTT++	TTAC
12.1	14.1	11.0	13.4	11.1	9.5	9.2

on both CIFAR10-C and CIFAR100-C datasets. More TTT methods can be adapted to the N-M protocol which allows training on the whole target domain data multiple epochs. Specifically, we compared with BN, TENT and SHOT. With TTAC alone we observe substantial improvement on all three datasets and TTAC can be further combined with SHOT demonstrating additional improvement. Finally, under the Y-M protocol, we demonstrate very strong performance compared to TTT-R and TTT++. It is also worth noting that TTAC under the N-O protocol can already yield results close to TTT++ under the Y-M protocol, suggesting the strong TTT ability of TTAC even under the most challenging TTT protocol.

4.4. Additional Datasets

TTT on hard samples. CIFAR10.1 contains roughly 2,000 new test images that were re-sampled after the research on original CIFAR-10 dataset, which consists of some hard samples and reflects the normal domain shift in our life. The results in Table 2 demonstrate our method is better able to adapt to the normal domain shift.

TTT on synthetic to real adaptation. VisDA-C is a large-scale benchmark of synthetic-to-real object classification dataset. The setting of training on a synthetic dataset and testing on real data fits well with the real application scenario. On this dataset, we conduct experiments with our method under the N-O, Y-O and Y-M protocols and other methods under respective protocols, results are presented in Table 3. We make the following observations. First, our method (TTAC Y-O) outperforms all methods except TTT++ under the Y-M protocol. This suggests TTAC is able to be deployed in the realistic TTT protocol. Moreover, if training on the whole target data is allowed, TTAC (Y-M) further beats TTT++ by a large margin, suggesting the effectiveness of TTAC under a wide range of TTT protocols.

Table 3. TTT on VisDA. The numbers for competing methods are inherited from.¹²

Method	Plane	Bcycl	Bus	Car	Horse	Knife	Mcycl	Person	Plant	Sktbrd	Train	Truck	Per-class
TEST	56.52	88.71	62.77	30.56	81.88	99.03	17.53	95.85	51.66	77.86	20.44	99.51	65.19
BN (N-M) ³⁴	44.38	56.98	33.24	55.28	37.45	66.60	16.55	59.02	43.55	60.72	31.07	82.98	48.99
TENT (N-M) ¹⁴	13.43	77.98	20.17	48.15	21.72	82.45	12.37	35.78	21.06	76.41	34.11	98.93	45.21
SHOT (N-M) ⁸	5.73	13.64	23.33	42.69	7.93	86.99	19.17	19.97	11.63	11.09	15.06	43.26	25.04
TFA (N-M) ¹²	28.25	32.03	33.67	64.77	20.49	56.63	22.52	36.30	24.84	35.20	25.31	64.24	37.02
TTT++ (Y-M) ¹²	4.13	26.20	21.60	31.70	7.43	83.30	7.83	21.10	7.03	7.73	6.91	51.40	23.03
TTAC (N-O)	18.54	40.20	35.84	63.11	23.83	39.61	15.51	41.35	22.97	46.56	25.24	67.81	36.71
TTAC (Y-O)	7.19	29.99	22.52	56.58	8.14	18.41	8.25	22.28	10.18	23.98	13.55	67.02	24.01
TTAC (Y-M)	2.74	17.73	18.91	43.12	5.54	12.24	4.66	15.90	4.77	10.78	9.75	62.45	17.38

4.5. Ablation Study

We conduct ablation study on CIFAR10-C dataset for individual components, including anchored clustering, pseudo label filtering, global feature alignment and finally the compatibility with contrastive branch.¹² For anchored clustering alone, we use all testing samples to update cluster statistics. For pseudo label filtering alone, we implement as predicting pseudo labels followed by filtering, then pseudo labels are used for self-training. We make the following observations from Table 4. Under both N-O and N-M protocols, introducing anchored clustering or pseudo label filtering alone improves over the baseline, e.g., under N-O 29.15% $\rightarrow$ 14.32% for anchored clustering and 29.15% $\rightarrow$ 15.00% for pseudo label filtering. When anchored clustering is combined with pseudo label filtering, we observe a significant boost in performance. This is due to more accurate estimation of category-wise cluster in the target domain and this reflects matching directly in the feature space may be better than minimising cross-entropy with pseudo labels. We further evaluate aligning global features alone with KL-Divergence. This achieves relatively good performance and obviously outperforms the L2 distance alignment adopted in.¹² Finally, we combine all three components and the full model yields the best performance. When contrast learning branch is included, TTAC achieves even better results.

4.6. Additional Analysis

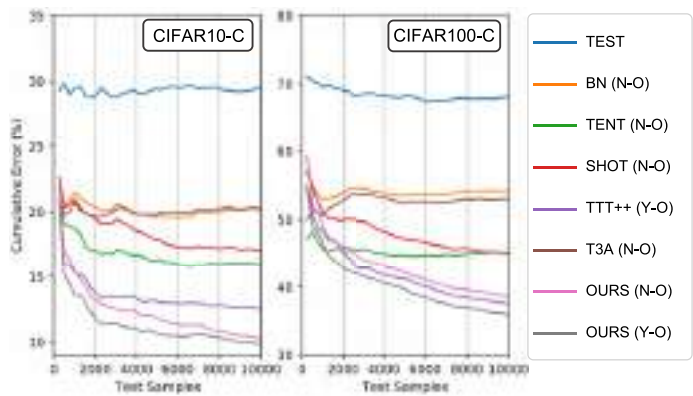
Cumulative performance under sTTT. We illustrate the cumulative error under the sTTT protocol in Figure 2(a). For both datasets TTAC outperforms competing methods from the early stage of TTT. The advantage is consistent throughout the TTT procedure.

TSNE visualisation of TTAC features. We provide qualitative results for TTT by visualising the adapted features through T-SNE.³⁶ In Figures 2(b) and 2(c), we compared the features learned by TTT++¹² and TTAC (Ours). We observe a better separation between classes by TTAC, implying an improved classification accuracy.

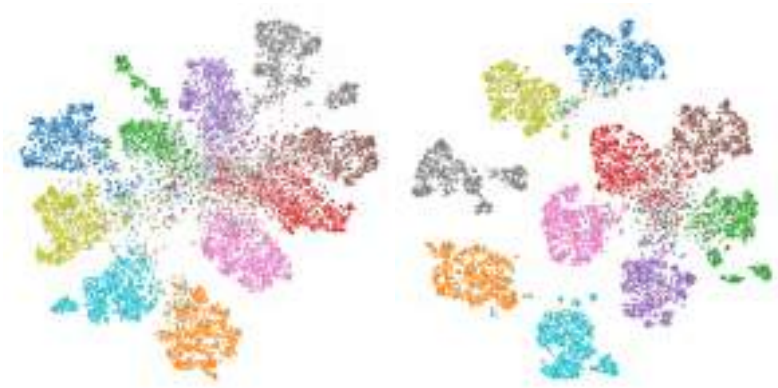
Source-free test-time training. TTT aims to adapt model to target domain data by doing simultaneous training and sequential

Table 4. Ablation study for individual components on CIFAR10-C dataset.

TTT Protocol	—		N-O			Y-O			N-M			Y-M	
Anchored Cluster.	—	✓	—	✓	—	✓	✓	✓	✓	—	—	✓	✓
Pseudo Label Filter.	—	—	✓	✓	—	✓	✓	—	✓	—	—	✓	✓
Global Feat. Align.	—	—	—	—	KLD	KLD	KLD	—	—	L2 Dist. ¹²	KLD	KLD	KLD
Contrast. Branch ¹²	—	—	—	—	—	—	✓	—	—	—	—	—	✓
Avg Acc	29.15	14.32	15.00	11.33	11.72	10.94	10.69	11.11	10.01	11.87	10.8	9.42	8.52



(a)



(b)

(c)

Fig. 2. (a) Comparison of test-time cumulative error under one-pass protocol. (b) T-SNE visualisation of TTT++ feature embedding. (c) T-SNE visualisation of TTAC feature embedding.

Table 5. Source-free sTTT on CIFAR10-C.

TEST	BN	TENT	T3A	SHOT	TTAC	TTAC+SHOT
29.15	15.49	14.27	15.44	13.95	13.74	13.35

inference. It has been demonstrated some light-weight information, e.g., statistics, from source domain will greatly improve the efficacy of TTT. Nevertheless, under a more strict scenario where source

domain information is strictly blind, TTAC can still exploit classifier prototypes to facilitate anchored clustering. Specifically, we normalise the category-wise weight vector with the norm of corresponding target domain cluster center as prototypes. Then, we build source domain mixture of Gaussians by taking prototypes as mean with a fixed covariance matrix. The results on CIFAR10-C are presented in Table 5. It is clear that even without any statistical information from source domain, TTAC still outperforms all competing methods.

5. Conclusion

TTT tackles the realistic challenges of deploying domain adaptation on-the-fly. In this work, we are first motivated by the confused evaluation protocols for TTT and propose two key criteria, namely modifying source training objective and sequential inference, to further categorise existing methods into four TTT protocols. Under the most realistic protocol, i.e., sTTT, we develop a TTAC approach to align target domain features to the source ones. Unlike batchnorm and classifier prototype updates, anchored clustering allows all network parameters to be trainable, thus demonstrating stronger TTT ability. We further propose pseudo label filtering and an iterative update method to improve anchored clustering and save memory footprint respectively. Experiments on five datasets verified the effectiveness of TTAC under sTTT as well as other TTT protocols.

References

1. A. Krizhevsky, I. Sutskever, and G. E. Hinton. Imagenet classification with deep convolutional neural networks. In *Advances in Neural Information Processing Systems* (2012).
2. Z.-H. Zhou, A brief introduction to weakly supervised learning, *National Science Review* **5**(1), 44–53 (08, 2018). ISSN 2095-5138.
3. J. Quiñero-Candela, M. Sugiyama, A. Schwaighofer, and N. D. Lawrence, *Dataset Shift in Machine Learning*. Mit Press (2008).
4. S. Ben-David, J. Blitzer, K. Crammer, A. Kulesza, F. Pereira, and J. W. Vaughan, A theory of learning from different domains, *Machine Learning* **79**(1-2), 151–175 (2010).

5. M. Wang and W. Deng, Deep visual domain adaptation: A survey, *Neurocomputing* **312**, 135–153 (2018).
6. Y. Ganin and V. Lempitsky. Unsupervised domain adaptation by backpropagation. In *International Conference on Machine Learning* (2015).
7. K. Zhou, Z. Liu, Y. Qiao, T. Xiang, and C. C. Loy, Domain generalization: A survey., *IEEE transactions on Pattern Analysis and Machine Intelligence* **45**(4), 4396–4415 (April, 2023).
8. J. Liang, D. Hu, and J. Feng. Do we really need to access the source data? Source hypothesis transfer for unsupervised domain adaptation. In *International Conference on Machine Learning* (2020).
9. J. N. Kundu, N. Venkat, R. V. Babu, et al. Universal source-free domain adaptation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020).
10. S. Yang, Y. Wang, J. van de Weijer, L. Herranz, and S. Jui. Generalized source-free domain adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).
11. H. Xia, H. Zhao, and Z. Ding. Adaptive adversarial network for source-free domain adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).
12. Y. Liu, P. Kothari, B. van Delft, B. Bellot-Gurlet, T. Mordan, and A. Alahi. Ttt++: When does self-supervised test-time training fail or thrive? In *Advances in Neural Information Processing Systems* (2021).
13. Y. Sun, X. Wang, Z. Liu, J. Miller, A. Efros, and M. Hardt. Test-time training with self-supervision for generalization under distribution shifts. In *International Conference on Machine Learning* (2020).
14. D. Wang, E. Shelhamer, S. Liu, B. Olshausen, and T. Darrell. Tent: Fully test-time adaptation by entropy minimization. In *International Conference on Learning Representations* (2021).
15. Y. Iwasawa and Y. Matsuo. Test-time classifier adjustment module for model-agnostic domain generalization. In *Advances in Neural Information Processing Systems* (2021).
16. H. Tang, K. Chen, and K. Jia. Unsupervised domain adaptation via structurally regularized deep clustering. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2020).
17. D.-H. Lee et al. Pseudo-label: The simple and efficient semi-supervised learning method for deep neural networks. In *Workshop on Challenges in Representation Learning, ICML* (2013).
18. K. Sohn, D. Berthelot, N. Carlini, Z. Zhang, H. Zhang, C. A. Raffel, E. D. Cubuk, A. Kurakin, and C.-L. Li, Fixmatch: Simplifying semi-supervised learning with consistency and confidence, *Advances in Neural Information Processing Systems* (2020).

19. E. Tzeng, J. Hoffman, N. Zhang, K. Saenko, and T. Darrell, Deep domain confusion: Maximizing for domain invariance, *arXiv preprint arXiv:1412.3474* (2014).
20. A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. J. Smola, A kernel two-sample test., *Journal of Machine Learning Research* **13**, 723–773 (2012).
21. B. Sun and K. Saenko. Deep coral: Correlation alignment for deep domain adaptation. In *European Conference on Computer Vision* (2016).
22. W. Zellinger, T. Grubinger, E. Lughofer, T. Natschläger, and S. Saminger-Platz. Central moment discrepancy (cmd) for domain-invariant representation learning. In *International Conference on Learning Representations* (2016).
23. J. Hoffman, E. Tzeng, T. Park, J.-Y. Zhu, P. Isola, K. Saenko, A. Efros, and T. Darrell. Cycada: Cycle-consistent adversarial domain adaptation. In *International Conference on Machine Learning* (2018).
24. J. Jiang and C. Zhai. Instance weighting for domain adaptation in nlp. In *ACL* (2007).
25. H. Yan, Y. Ding, P. Li, Q. Wang, Y. Xu, and W. Zuo. Mind the class weight bias: Weighted maximum mean discrepancy for unsupervised domain adaptation. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition* (2017).
26. Q. Wang, O. Fink, L. Van Gool, and D. Dai. Continual test-time domain adaptation. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2022).
27. J. R. Hershey and P. A. Olsen. Approximating the kullback leibler divergence between gaussian mixture models. In *IEEE International Conference on Acoustics, Speech and Signal Processing* (2007).
28. E. Arazo, D. Ortego, P. Albert, N. E. O’Connor, and K. McGuinness. Pseudo-labeling and confirmation bias in deep semi-supervised learning. In *International Joint Conference on Neural Networks* (2020).
29. B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama. Co-teaching: Robust training of deep neural networks with extremely noisy labels. In *Advances in Neural Information Processing Systems* (2018).
30. J. Li, Y. Wong, Q. Zhao, and M. S. Kankanhalli. Learning to learn from noisy labeled data. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2019).
31. D. Berthelot, N. Carlini, I. Goodfellow, N. Papernot, A. Oliver, and C. A. Raffel, Mixmatch: A holistic approach to semi-supervised learning, *Advances in Neural Information Processing Systems* (2019).

32. E. D. Cubuk, B. Zoph, J. Shlens, and Q. V. Le, Randaugment: Practical automated data augmentation with a reduced search space, *arXiv preprint arXiv:1909.13719* (2019).
33. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, Dynamic graph cnn for learning on point clouds, *Acm Transactions On Graphics (tog)* (2019).
34. S. Ioffe and C. Szegedy. Batch normalization: Accelerating deep network training by reducing internal covariate shift. In *International Conference on Machine Learning* (2015).
35. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning* (2020).
36. L. van der Maaten and G. Hinton, Visualizing data using t-SNE, *Journal of Machine Learning Research* **9**, 2579–2605 (2008).

Chapter 9

Algorithm-System-Hardware Co-design for Efficient 3D Deep Learning

**Zhijian Liu^{*,†}, Haotian Tang^{*,‡},
Yujun Lin^{*,§}, and Song Han[¶]**

*Massachusetts Institute of Technology (MIT),
Cambridge, MA, USA*

[†]*zhijian@mit.edu*

[‡]*kentang@mit.edu*

[§]*yujunlin@mit.edu*

[¶]*songhan@mit.edu*

3D point cloud deep learning has received significant attention due to its wide applications, such as augmented/mixed reality, autonomous vehicles/drones, and robots. Despite its remarkable accuracy, the computational cost and sparse nature of 3D point cloud deep learning models greatly hinder their use in real-world, latency-sensitive applications. In this chapter, we develop efficient algorithms, systems, and hardware for 3D deep learning to overcome the computational challenges (especially sparsity) of 3D point cloud data, making it more applicable in a broader range of real-world scenarios. From the algorithm

*Indicates equal contributions.

perspective, we introduce novel 3D building blocks (PVCNN and SPVNAS)^a that are tailor-made for point cloud data to reduce sparse and irregular overheads. On the system level, we develop a high-performance library (TorchSparse)^b that can handle sparse and irregular workloads on general-purpose hardware, which is typically optimised for dense and regular workloads. Furthermore, we develop a specialised hardware accelerator (PointAcc)^c that can support various types of point cloud deep learning primitives and mitigate memory bottlenecks for sparse point cloud computing.

1. Introduction

3D point clouds provide a more accurate portrayal of 3D environments than 2D images, as they capture objects' precise location and shape in a 3D space. In recent years, the availability of 3D sensors has seen a marked increase, with most L3+ autonomous vehicles equipped with LiDAR scanners and the latest smartphones featuring LiDAR or Time-of-Flight (ToF) sensors. This has made 3D data more widely accessible, leading to its growing popularity in real-world applications, such as augmented/mixed reality, autonomous vehicles/drones, and robots.

Like many other domains, neural networks have become the standard for processing and analysing 3D point cloud data.^{1,2} The rapid advancement of deep neural networks has led to promising results in this field. For instance, within the past five years, the 3D object detection accuracy has been improved dramatically, from a merely 30% mean average precision (mAP)³ to over 75% mAP⁴ on the nuScenes leaderboard.⁵ While this is an impressive achievement, it has a significant computational cost. For example, in 2019, the state-of-the-art 3D point cloud segmentation model (when using the state-of-the-art 3D inference library)⁶ could only run at two frames per second (FPS) on a desktop GPU. This greatly hinders the use of these models in practical applications, particularly on low-power edge devices, such as smartphones and drones.

Sparsity and irregularity are the main factors that contribute to the inefficiency of 3D deep learning models. 3D point clouds often

^a<https://pvnas.mit.edu>.

^b<https://torchsparse.mit.edu>.

^c<https://pointacc.mit.edu>.

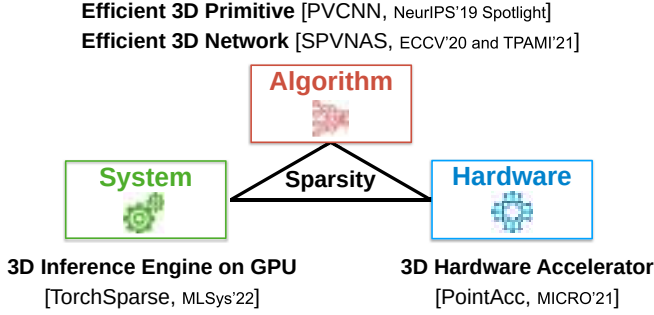


Fig. 1. We co-design algorithm, system, and hardware to address the computation and sparsity challenge in 3D point cloud deep learning.

exhibit an extremely high degree of spatial sparsity, with more than 99.9% of the data being empty or irrelevant. This makes conventional 2D sparse algorithms/systems/hardware inapplicable. Additionally, 3D point clouds have irregular neighbourhood structures, where the neighbours of a point do not lie contiguously in memory, resulting in expensive structuring overhead and random memory access. These unique characteristics of 3D point clouds present a significant challenge for developing efficient and accurate 3D point cloud deep learning models.

As in Figure 1, this chapter proposes efficient algorithms, systems, and hardware for 3D deep learning that overcome the computational challenges associated with 3D point cloud data. From an algorithmic perspective, we explore novel 3D building blocks (PVCNN and SPVNAS) specifically designed to handle point cloud data while minimising sparse and irregular overheads. We develop a high-performance GPU library (TorchSparse) at the system level to accelerate sparse and irregular workloads on general-purpose hardware. Moreover, we design a specialised hardware accelerator (PointAcc) that supports various types of point cloud deep learning primitives and alleviates memory bottlenecks in point cloud computing.

2. Background

3D data usually comes in the format of point clouds:

$$\mathbf{x} = \{\mathbf{x}_k\} = \{(\mathbf{p}_k, \mathbf{f}_k)\}, \quad (1)$$

where $\mathbf{p}_k$ is the coordinate of the k th point and $\mathbf{f}_k$ is the feature corresponding to $\mathbf{p}_k$. The voxelised representation can also be unified into this formulation, where $\mathbf{p}_k$ stands for the coordinate of the k th voxel grid. Based on this, point cloud convolution can be formulated as

$$\mathbf{y}_k = \sum_{\mathbf{x}_i \in \mathcal{N}(\mathbf{x}_k)} \mathcal{K}(\mathbf{x}_k, \mathbf{x}_i) \times \mathcal{F}(\mathbf{x}_i). \quad (2)$$

We iterate $\mathbf{x}_k$ over the entire input during the convolution. For each centre $\mathbf{x}_k$, we first index its neighbours $\mathbf{x}_i$ in $\mathcal{N}(\mathbf{x}_k)$, then convolve the neighbouring features $\mathcal{F}(\mathbf{x}_i)$ with the kernel $\mathcal{K}(\mathbf{x}_k, \mathbf{x}_i)$, and produce the corresponding output $\mathbf{y}_k$. Recent 3D deep learning methods can be categorised into voxel-based, point-based, and sparse voxel-based variants based on how Equation (2) is instantiated.

2.1. Voxel-Based Methods

Conventionally, researchers relied on the volumetric representation and applied the convolution to process 3D data.^{1,7-11} To name a few, Maturana *et al.*⁹ proposed the vanilla volumetric CNN, Qi *et al.*¹ extended 2D CNNs to 3D and systematically analysed the relationship between 3D CNNs and multi-view CNNs, and Wang *et al.*¹² incorporated the octree into the volumetric CNN to reduce the memory consumption. Recent studies suggest that the volumetric representation can be used in 3D shape segmentation^{10,13,14} and 3D object detection¹¹ as well. Due to the sparse nature of 3D data, the dense volumetric representation is inherently inefficient and also inevitably introduces information loss.

2.1.1. Bottleneck: Large memory footprint

Voxel-based representation is regular and has a good memory locality. However, it requires very high resolution to avoid losing much information. When the resolution is low, multiple points are bucketed into the same voxel grid, and these points will no longer be *distinguishable*. A point is kept only when it exclusively occupies one voxel grid. In Figure 2, we investigate the number of distinguishable points and the memory consumption (during training with a batch size of 16) with different resolutions. On a single GPU (with 12 GB of memory), the largest affordable resolution is 64, which will lead to

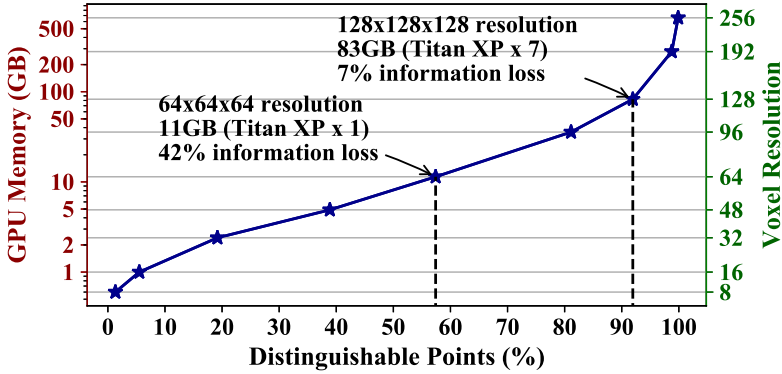


Fig. 2. Voxel-based models suffer from large information loss at acceptable GPU memory consumption.

42% of information loss. To keep more than 90% of the information, we then need to double the resolution to 128, consuming $7.2\times$ GPU memory (82.6 GB), which is prohibitive in constrained scenarios. Although the GPU memory increases cubically with the resolution, the number of distinguishable points has a diminishing return. Therefore, the voxel-based solution is not scalable.

2.2. Point-Based Methods

PointNet² takes advantage of the symmetric function to directly process the unordered point sets in 3D. Later research^{15–17} proposed to stack PointNets hierarchically to model neighbourhood information and increase model capacity. Instead of stacking PointNets as basic blocks, another type of method^{18–20} abstracts away the symmetric function using dynamically generated convolution kernels or learned neighbourhood permutation function. Some other research, such as SPLATNet,²¹ that naturally extends the 2D image SPLAT to 3D, and SOnet,²² which uses the self-organisation mechanism with the theoretical guarantee of invariance to point order, also shows great potential in general-purpose 3D modelling with point clouds as input. Apart from these general-purpose models, there are also some attempts for specific 3D tasks. SegCloud,²³ SGPN,²⁴ SPGraph,²⁵ ParamConv,²⁶ SSCN,²⁷ and RSNet²⁸ are specialised for 3D semantic and instance segmentation. As for 3D object detection, F-PointNet²⁹ is based on the RGB detector and point-based regional proposal

networks; PointRCNN³⁰ follows a similar idea while removing the RGB detector.

2.2.1. Bottleneck: Sparse data organisation

Point-based methods process the point cloud directly and thus require much lower GPU memory than voxel-based models thanks to the sparse representation. However, this will lead to an irregular memory access pattern and introduce the dynamic kernel computation overhead, which becomes the efficiency bottleneck.

Irregular memory access: Different from the voxel-based representation, neighbouring points $\mathbf{x}_i \in \mathcal{N}(\mathbf{x}_k)$ in the point-based representation will not be laid out contiguously in the memory. Besides, 3D points are scattered in $\mathbb{R}^3$; therefore, we need to explicitly identify who is in the neighbouring set $\mathcal{N}(\mathbf{x}_k)$, rather than by direct indexing. Point-based methods often define $\mathcal{N}(\mathbf{x}_k)$ as nearest neighbours in the coordinate space^{18,20} or the feature space.¹⁷ Either requires explicit and expensive KNN computation. After KNN, gathering all neighbours $\mathbf{x}_i$ in $\mathcal{N}(\mathbf{x}_k)$ will require a large number of random memory accesses, which is not cache friendly. Combining the cost of neighbour indexing and data movement, the state-of-the-art point-based models spend **36%**,¹⁸ **52%**,¹⁷ and **57%**²⁰ of the total runtime on structuring the irregular data and accessing the memory randomly (Figure 3).

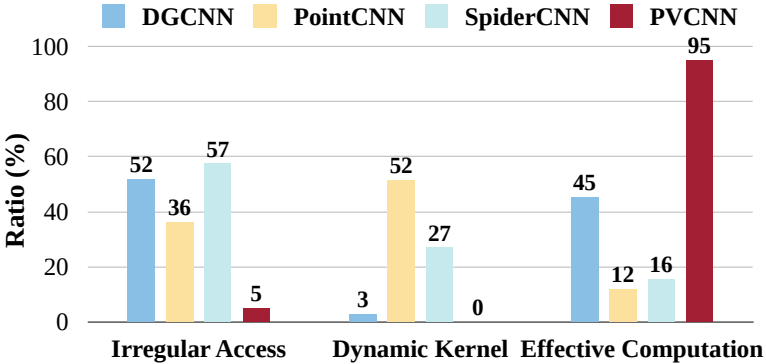


Fig. 3. Point-based model suffers from large irregular memory access and dynamic kernel computation overheads.

Dynamic kernel computation: For 3D volumetric convolutions, the kernel value $\mathcal{K}(\mathbf{x}_k, \mathbf{x}_i)$ can be directly indexed because the relative positions of the neighbour $\mathbf{x}_i$ are fixed for different centre $\mathbf{x}_k$: e.g., each axis of the offset $\mathbf{p}_i - \mathbf{p}_k$ can only be 0, ± 1 for the convolution with a size of 3. For point-based convolution, the points are scattered over the entire 3D space irregularly; therefore, the relative positions of neighbours become unpredictable. In this case, we will have to calculate the kernel $\mathcal{K}(\mathbf{x}_k, \mathbf{x}_i)$ for each neighbour $\mathbf{x}_i$ *on the fly*. For instance, SpiderCNN²⁰ leverages the third-order Taylor expansion as a continuous approximation of the kernel $\mathcal{K}(\mathbf{x}_k, \mathbf{x}_i)$; PointCNN¹⁸ permutes the neighbouring points into a canonical order with the feature transformer $\mathcal{F}(\mathbf{x}_i)$. Both will introduce additional matrix multiplications. Empirically, the overhead of dynamic kernel computation for PointCNN can be more than **50%** (Figure 3).

The combined overhead of irregular memory access and dynamic kernel computation ranges from **55%** (DGCNN) to **88%** (PointCNN). Thus, most computations are wasted on dealing with the irregularity of point-based representation.

2.3. Sparse Voxel-Based Methods

Recently, researchers also started to study 3D sparse convolution^{6,27,31} for point cloud processing. Sparse convolution operates on point clouds with quantised coordinates $\mathbf{p} = \lfloor \mathbf{p}_i^{\text{raw}} / \mathbf{v} \rfloor$, where $\mathbf{v}$ is the voxel size and $\mathbf{p}_i^{\text{raw}}$ is the original coordinates in Equation (1). For example, in CenterPoint,³² point clouds on Waymo³³ are quantised with $\mathbf{v} = [0.1 \text{ m}, 0.1 \text{ m}, 0.15 \text{ m}]$. This means that we will only keep one point within each $0.1 \text{ m} \times 0.1 \text{ m} \times 0.15 \text{ m}$ grid.

In sparse convolution (Figure 4), we reformulate the function $\mathcal{N}$ in Equation (2) by defining the D -dimensional neighbourhood with kernel size K as $\Delta^D(K)$ (e.g., $\Delta^2(5) = \{-2, -1, 0, 1, 2\}^2$ and $\Delta^3(3) = \{-1, 0, 1\}^3$). The forward form of sparse convolution on the k th output point could then be represented as

$$\mathbf{x}_k^{\text{out}} = \sum_{\delta \in \Delta_K^D} \sum_j 1[\mathbf{p}_j = s\mathbf{q}_k + \delta] (\mathbf{x}_j^{\text{in}} \cdot \mathbf{W}_\delta), \quad (3)$$

where $\mathbf{p}_j \in \mathbf{P}^{\text{in}}$, $\mathbf{q}_k \in \mathbf{P}^{\text{out}}$, $1[\cdot]$ is a binary indicator, s is the stride, and $\mathbf{W}_\delta \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$ corresponds to the weight matrix for kernel

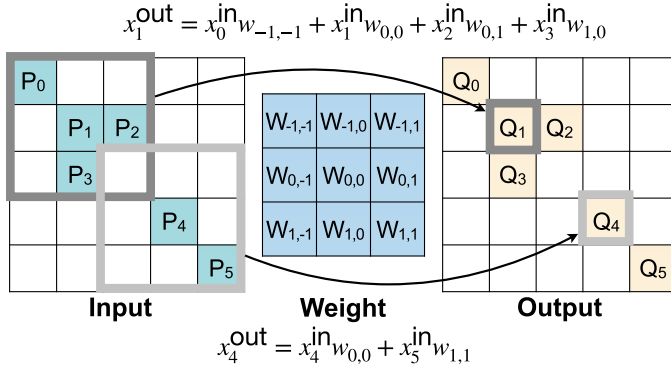


Fig. 4. Sparse convolution (Equation (3)) on $\Delta^2(3)$: computation is performed only on *non-zero* inputs.

offset $\delta \in \Delta^D(K)$. Alternatively, we collect all possible (j, k, δ) pairs in Equation (3) and define *maps* $\mathcal{M} = \{(p_j, q_k, \mathbf{W}_\delta)\}$. Sparse convolution iterates over all map entries and performs $\mathbf{x}_k^{\text{out}} = \mathbf{x}_k^{\text{out}} + \mathbf{x}_j^{\text{in}} \cdot \mathbf{W}_\delta$. Such formulation is closer to the GPU implementation described in Algorithm 1.

Algorithm 1 Gather-GEMM-Scatter for sparse convolution

Input: input features $\mathbf{X}^{\text{in}}$, weights $\mathbf{W}$, maps $\mathcal{M}$

Output: output features $\mathbf{X}^{\text{out}}$

$\mathbf{X}^{\text{out}} \leftarrow \mathbf{0}$

Separately perform gather-GEMM-scatter for each weight.

for δ in Δ_K^D **do**

$\mathbf{F} \leftarrow \emptyset$

Gather features for w_n .

for $m, (p_j, q_k, \mathbf{W}_\delta)$ in $\text{enumerate}(\mathcal{M}[\mathbf{W}_\delta])$ **do**

$\mathbf{F}[m] \leftarrow \mathbf{X}^{\text{in}}[j]$

end for

Matrix-matrix multiplication.

$\mathbf{F} \leftarrow \mathbf{F} \cdot \mathbf{W}_\delta$

Scatter partial sums to $\mathbf{X}^{\text{out}}$.

for $m, (p_j, q_k, \mathbf{W}_\delta)$ in $\text{enumerate}(\mathcal{M}[\mathbf{W}_\delta])$ **do**

$\mathbf{X}^{\text{out}}[q_k] \leftarrow \mathbf{X}^{\text{out}}[q_k] + \mathbf{F}[m]$

end for

end for

Sparse convolution has been widely adopted in various automotive applications. 3D semantic segmentation methods,^{34–36} 3D object

detection frameworks,^{31,32,37–42} 3D reconstruction methods,⁴³ multi-sensor fusion perception models,^{4,44,45} and end-to-end navigation methods⁴⁶ all used sparse convolutional backbones to extract features from LiDAR scans.

2.3.1. Bottleneck: Data movement and low-utilisation GEMM

We systematically profile the runtime of different components in two representative sparse CNNs: one for segmentation (Figure 5(a)) and one for detection (Figure 5(b)). Matrix multiplication is the core computation in sparse convolution and takes up a large proportion of total execution time (20–50%). This is because the computation workloads are very *non-uniform* due to the irregular nature of point clouds (on real-world LiDAR datasets, map sizes for different weights can differ by order of magnitude, and most map sizes are *small*). As a result, the matrix multiplication in MinkUNet ($0.5\times$ width) runs at 8.1 TFLOP/s on RTX 2080 Ti with FP16 quantisation, achieving only 30% device utilisation.

Furthermore, data movement is the largest bottleneck in sparse CNNs, which takes up 40–50% of total runtime on average. Recent GPU implementations of sparse convolution take a Gather-GEMM-Scatter dataflow (as described in Algorithm 1). Scatter-gather operations are bottlenecked by GPU memory bandwidth (*limited*) rather than computation resources (*abundant*). Worse still, the dataflow

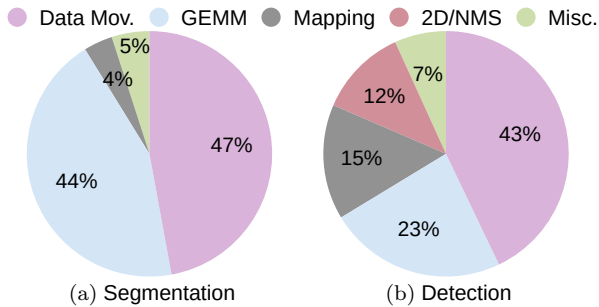


Fig. 5. Data movement and GEMM operations account for a large proportion of the runtime of sparse CNNs. For detection workloads, point clouds are denser, so the mapping overhead is larger.

Note: Models: MinkUNet⁶ (segmentation) and CenterPoint³² (detection). Hardware: NVIDIA RTX 3090.

in Algorithm 1 completely separates scatter-gather operations for different kernel offsets. This further ruins the possibility of any reuse in the data movement. It is also noteworthy that the large mapping latency in the CenterPoint detector (Figure 5(b)) also stems from memory overhead: hashmap construction and output coordinate calculation both require multiple DRAM accesses.

2.4. Summary

This section discusses various formulations for convolution on point clouds, which can be classified into voxel-based, point-based, and sparse voxel-based. However, running these methods efficiently on general-purpose hardware platforms is challenging due to the sparsity and irregularity of point clouds. Voxel-based methods, in particular, have a large memory footprint, while all other solutions suffer from irregular memory access and data structuring overhead. In the remaining part of this chapter, we present our attempts to accelerate 3D deep learning through algorithm, system, and hardware innovations.

3. Algorithm: PVCNN and SPVNAS

Designing efficient 3D neural networks needs to take the hardware into consideration. Compared with arithmetic operations, memory operations are much more expensive: i.e., they consume two orders of magnitude *higher* energy and have two orders of magnitude *lower* bandwidth (Figure 6). Another very important aspect is the memory access pattern: i.e., the random access will introduce memory bank conflicts and decrease the throughput (Figure 7). From the hardware perspective, conventional 3D models are inefficient due to large memory footprints and random memory access.

3.1. Designing Efficient 3D Primitives

We introduce a novel hardware-efficient 3D primitive for small 3D objects and indoor scenes: *Point-Voxel Convolution (PVConv)*. It combines the advantages of point-based methods (small memory footprint) and voxel-based methods (good data locality and regularity). For large outdoor scenes, we further propose *Sparse*

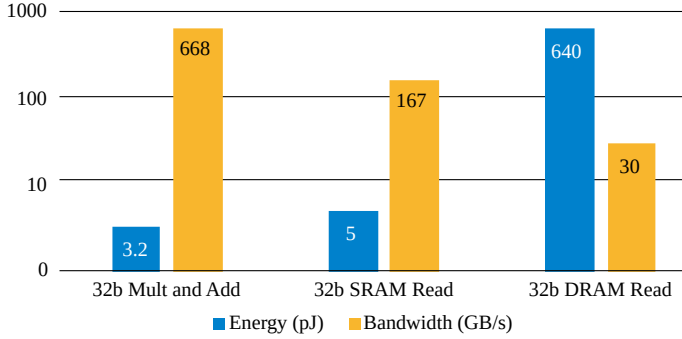


Fig. 6. Off-chip DRAM accesses take two orders of magnitude more energy than arithmetic operations (640 pJ vs. 3 pJ), whereas the bandwidth is two orders of magnitude lower (30 GB/s vs. 668 GB/s). Efficient 3D models should **reduce the memory footprint**, which is the bottleneck of *voxel-based* methods. All energy numbers are adopted from the EIE⁴⁷ paper.

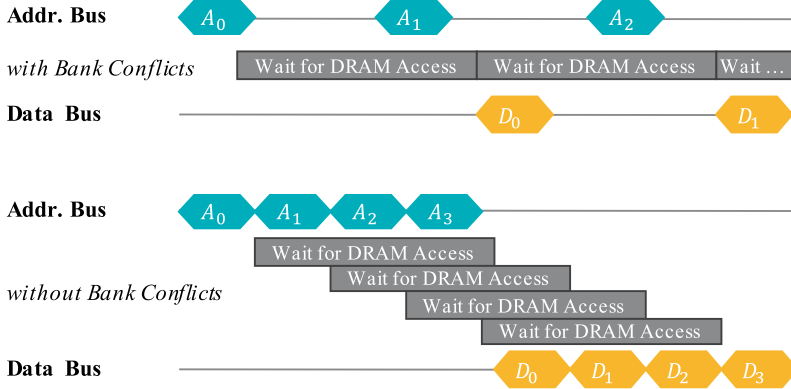


Fig. 7. Random memory access is inefficient since it cannot take advantage of the DRAM burst and will cause bank conflicts, whereas contiguous memory access does not suffer from the above issue. Efficient 3D models should **avoid random memory accesses**, which is the bottleneck of *point-based* methods.

Point-Voxel Convolution (SPVConv) that enhances PVConv with the sparse convolution to enable higher resolutions in the voxel-based branch.

3.1.1. Point-Voxel Convolution (PVConv)

PVConv separates the *fine-grained* feature transformation and the *coarse-grained* neighbour aggregation so that each branch can be

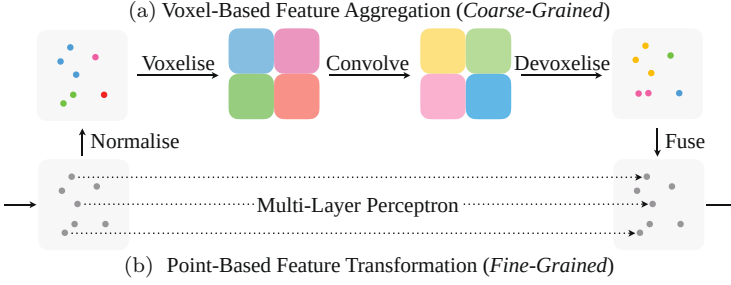


Fig. 8. Point-Voxel Convolution (PVConv) is composed of a *low-resolution* voxel-based branch and a *high-resolution* point-based branch. The voxel-based branch extracts *coarse-grained* neighbourhood information, which is supplemented by *fine-grained* individual point features extracted from the point-based branch.

implemented very efficiently and effectively. As in Figure 8, the upper voxel-based branch first transforms the points into *low-resolution* voxel grids and then aggregates the neighbouring points with voxel-based convolutions, followed by the devoxelisation to convert them back to points. Either voxelisation or devoxelisation requires a single scan on all points, making the memory cost low. The lower point-based branch extracts the features for each individual point. As it does not aggregate the neighbour’s information, it is able to afford a very *high resolution*.

Voxel-based feature aggregation: A key component of convolution is aggregating the neighbouring information to extract the local features. Due to its regularity, we choose to perform this feature aggregation in the volumetric domain.

Normalisation: The scale of different point clouds might be different. Thus, we normalise the coordinates $\{\mathbf{p}_k\}$ before converting the point cloud into the volumetric domain. First, we translate all points into the local coordinate system with the gravity centre as the origin. After that, we normalise the points into the unit sphere by dividing all coordinates by $\max\|\mathbf{p}_k\|_2$ and then scale and translate the points to $[0, 1]$. We denote the normalised coordinates as $\{\hat{\mathbf{p}}_k\}$.

Voxelisation: We transform the normalised point cloud $\{(\hat{\mathbf{p}}_k, \mathbf{f}_k)\}$ into the dense voxelised representation $\{\mathbf{V}_{u,v,w}\}$ by averaging all of the features $\mathbf{f}_k$ whose coordinate $\hat{\mathbf{p}}_k = (\hat{x}_k, \hat{y}_k, \hat{z}_k)$ falls into the voxel

grid (u, v, w) :

$$V_{u,v,w,c} = \sum_{k=1}^n \mathbb{I}[\lfloor \hat{\mathbf{x}}_k r \rfloor = u, \lfloor \hat{\mathbf{y}}_k r \rfloor = v, \lfloor \hat{\mathbf{z}}_k r \rfloor = w] \times \mathbf{f}_{k,c} / N_{u,v,w}, \quad (4)$$

where r denotes the voxel resolution, $\mathbb{I}[\cdot]$ is the binary indicator of whether the coordinate $\hat{\mathbf{p}}_k$ belongs to the voxel grid (u, v, w) , $\mathbf{f}_{k,c}$ denotes the c th channel feature corresponding to $\hat{\mathbf{p}}_k$, and $N_{u,v,w}$ is the normalisation factor (i.e., the number of points that fall in that voxel grid).

Feature aggregation: After converting the points into the voxel grids, we apply a stack of 3D volumetric convolutions to aggregate the features. Similar to conventional 3D models, we apply batch normalisation⁴⁸ and nonlinear activation function⁴⁹ after each 3D volumetric convolution.

Devoxelisation: As we need to fuse the information with the point-based feature transformation branch, we transform the voxel-based features back to the domain of the point cloud. A simple implementation of the voxel-to-point mapping is the nearest-neighbour interpolation (i.e., assign the feature of a grid to all points in it). However, this implementation will make the points in the same voxel grid always share the same feature values. Therefore, we instead leverage the trilinear interpolation to transform the voxel grids to points to ensure that the features mapped to each point are distinct.

Point-based feature transformation: The voxel-based feature aggregation branch fuses the neighbourhood information in a coarse granularity. However, in order to model finer-grained individual point features, low-resolution voxel-based methods alone might not be sufficient. Hence, we directly operate on each point to extract individual point features using an MLP. Though simple, the MLP outputs distinct and discriminative features for each point. Such high-resolution individual point information is critical to supplement the coarse-grained voxel-based information. Furthermore, with individual point features and aggregated neighbourhood information, we can fuse two branches efficiently with addition as they provide complementary information.

3.1.2. Analysis

PVConv is much better than previous voxel-based and point-based models in terms of both efficiency and effectiveness.

Better locality and regularity: Our PVConv is more efficient than conventional point-based convolutions due to its better data locality and regularity. Our voxelisation and devoxelisation both require only $\mathcal{O}(n)$ random memory accesses, where n is the number of points since we only need to iterate over all points once to scatter them to their corresponding grids. However, for conventional point-based methods, gathering neighbours for all points will require at least $\mathcal{O}(kn)$ random memory accesses, where k is the number of neighbours. Thus, our PVConv is $k\times$ more efficient from this viewpoint. As the typical value for k is 32/64 in PointNet++¹⁵ and 16 in PointCNN,¹⁸ PVConv empirically reduces the number of incongruous memory accesses by 16-64 $\times$, achieving better data locality. Besides, as our convolutions are done in the voxel domain, which is regular, our PVConv does not require KNN computation and dynamic kernel computation, which are usually quite expensive.

Higher resolution: As our point-based feature extraction branch is implemented as MLP, a natural advantage is that we can maintain the same number of points throughout the whole network while still being able to model neighbourhood information. Let us compare our PVConv and the set abstraction (SA) module in PointNet++.¹⁵ Suppose we have a batch of 2048 points with 64-channel features (with a batch size of 16), and we then aggregate information from 125 neighbours of each point and transform the aggregated feature to output the features with the same size. In this case, the SA module requires 75.2 ms of latency and 3.6 GB of memory, while our PVConv only requires 25.7 ms of measured latency and 1.0 GB of memory. The SA module will have to downsample to 685 points (i.e., around $3\times$ downsampling) to match up with the latency of our PVConv, while the memory consumption will still be $1.5\times$ higher. Therefore, with the same latency, our PVConv can model the full point cloud, while the SA module has to downsample the input aggressively, which will inevitably induce information loss.

3.1.3. Sparse Point-Voxel Convolution (SPVConv)

PVConv is very efficient and effective, especially for small 3D objects and indoor scenes, as their scales are relatively small; however, it is less suitable for large outdoor scenes due to the coarse voxelisation. The PVConv-based model can afford the resolution of at most 128 in its voxel-based branch on a single GPU (with 12 GB of memory). For a large outdoor scene (with a size of 100 m×100 m×10 m), each voxel grid will correspond to a large area (with a size of 0.8 m×0.8 m×0.1 m). In this case, small instances (e.g., pedestrians) will only occupy very few voxel grids. From a few points, PVConv can hardly learn any useful information from the voxel-based branch, leading to relatively low performance (see Table 1). Alternatively, we can process the large 3D scenes piece by piece so that each sliding window is smaller. To preserve the fine-grained information, we found empirically that the voxel size needs to be lower than 0.05 m. In this case, we have to run the model once for each of the 244 sliding windows, which will take 35 seconds to process a single scene. Such a significant latency is not affordable for most real-time applications (e.g., autonomous driving).

To this end, we introduce *Sparse Point-Voxel Convolution (SPVConv)* to effectively and efficiently process the large 3D scene. It follows a similar two-branch architectural design as PVConv except that the volumetric convolution in the voxel-based branch is replaced with the sparse convolution.⁶ As point clouds are intrinsically very sparse, this modification can significantly reduce memory consumption using a higher resolution in the voxel-based branch. Furthermore, SPVConv can also be considered by adding a

Table 1. Comparison between PVConv and SPVConv in large outdoor scenes. PVConv is not suitable for large scenes. If processing with sliding windows, its latency is not affordable for deployment. If taking the whole scene, its resolution is too coarse to capture useful information.

	Input	Voxel Size (m)	Latency (ms)	Mean IoU
PVConv	Sliding Window	0.05	35640	—
	Entire Scene	0.80	146	39.0
SPVConv	Entire Scene	0.05	85	58.8

high-resolution point-based branch to the vanilla sparse convolution to preserve the fine details (i.e., small instances).

Most operations in PVConv can be directly adapted to the sparse voxelised representation. However, the voxelisation and devoxelisation are not as trivial since we cannot easily index a given 3D coordinate in the sparse voxelised representation. A straightforward implementation based on the iterative coordinate comparison will require $\mathcal{O}(mn)$ of time, where m is the number of points in the point cloud representation and n is the number of activated points in the sparse voxelised representation. As m and n are typically at the order of 10^5 , this naive implementation is not practical for real-time applications. Instead, we propose to use the parallel hash tables on GPUs to accelerate the sparse voxelisation and devoxelisation. Note that existing implementations^{6,27} use CPU-based hash tables for kernel map construction in the sparse convolution, which is much slower. Concretely, we first construct a hash table for all activated points in the sparse voxelised representation, which can be finished in $\mathcal{O}(n)$ of time. After that, we iterate over all points, and for each point, we then query its coordinate in the hash table to obtain its corresponding index in the sparse voxelised representation. As the lookup over the hash table requires $\mathcal{O}(1)$ time in the worst case, this query step will, in total, take $\mathcal{O}(m)$ time. Therefore, the total time of coordinate indexing will be reduced from $\mathcal{O}(mn)$ to $\mathcal{O}(m + n)$.

3.2. Searching Efficient 3D Architectures

Even with the efficient 3D primitive, designing an efficient neural network model is still challenging. We need to carefully adjust the network architecture (e.g., channel numbers and kernel sizes of all layers) to meet the requirements for real-world applications (e.g., latency, energy, and accuracy). In this section, we propose *3D Neural Architecture Search (3D-NAS)* to automatically design efficient 3D models (Figure 9). We first carefully design a search space tailored for 3D and then introduce our training paradigm that supports a large number of neural networks within a single super network. Finally, we use the evolutionary search to explore the best candidate in the design space given a resource constraint.

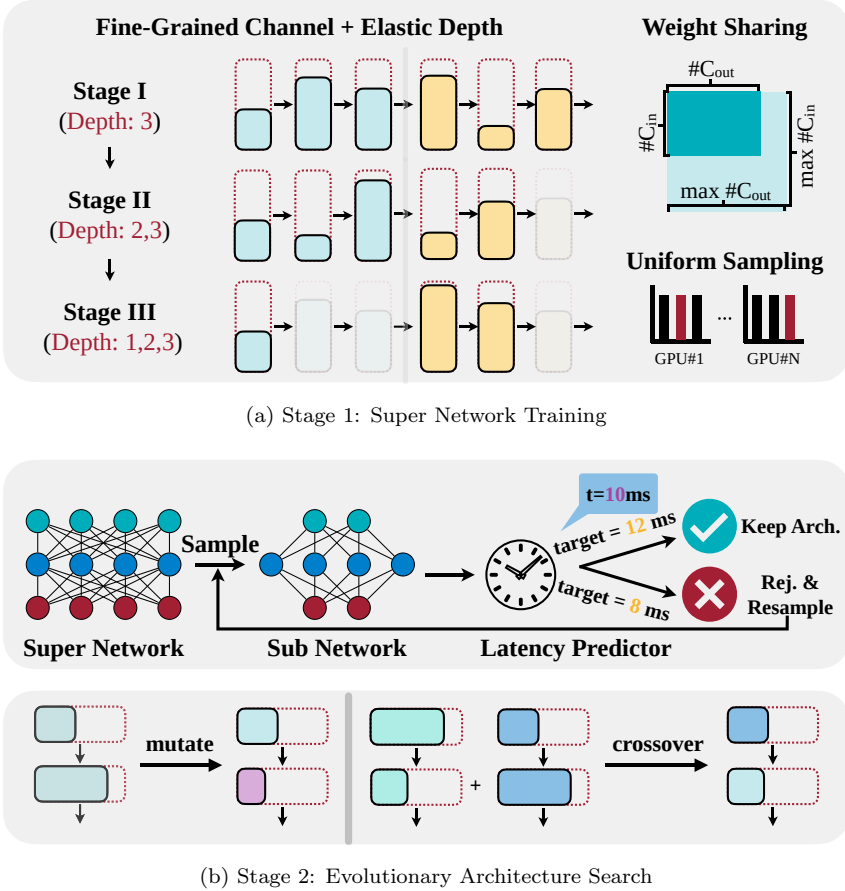


Fig. 9. We propose a two-stage 3D Neural Architecture Search (3D-NAS) framework to automatically design efficient 3D deep learning architectures. (a) In the first stage, we train a *super network* that supports all candidate networks within the design space. (b) In the second stage, we perform *evolutionary architecture search* to find the best candidate network given a specific resource constraint.

3.2.1. Design space

The design space quality impacts the performance of neural architecture search. In our search space, we use fine-grained channel numbers and elastic network depths; however, we do not support different kernel sizes.

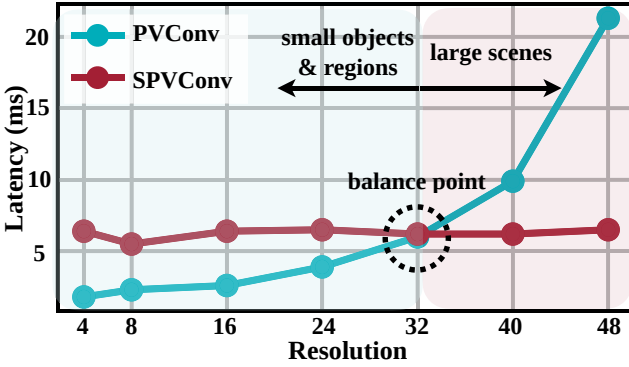


Fig. 10. PVConv is more efficient and effective at smaller resolutions while SPVConv is more efficient at larger resolutions.

Primitive selection: PVConv accesses the memory contiguously and has a relatively small memory footprint for small objects and indoor scans. However, it does not scale up very well to large-scale outdoor scenes (Table 1) as the resolution of its voxel-based branch is still constrained by the memory. SPVConv, on the other hand, can efficiently scale up to large scans but falls short in modelling small objects and regions due to the irregular overhead introduced by sparse operations. As in Figure 10, the latency of SPVConv almost stays constant when the voxel resolution increases from 4 to 48, but the latency of PVConv quickly grows when the resolution scales up. As a result, at smaller resolutions, PVConv can be up to $3.6\times$ faster than SPVConv, while SPVConv becomes $3.3\times$ faster than PVConv once the resolution is larger than 48. As the spatial range of single objects/indoor scans is smaller than $1.5\text{ m}\times 1.5\text{ m}\times 1.5\text{ m}$, it is sufficient to use voxel resolution smaller than 32, and therefore, PVConv is favoured over SPVConv. On the other hand, SPVConv is favoured when the spatial range scales far beyond $1.5\text{ m}\times 1.5\text{ m}\times 1.5\text{ m}$.

Fine-grained channel numbers: The computation cost increases quadratically with the number of channels; therefore, the channel number selection dramatically influences the network efficiency. Most existing neural architecture frameworks⁵⁰ only support the coarse-grained channel number selection: e.g., searching the expansion ratio of the ResNet/MobileNet blocks over a few (2–3) choices. In this case, only the intermediate channel numbers of the

blocks can be changed, while the input and output channel numbers will remain the same. Empirically, we observe that this limits the variety of the search space. To this end, we enlarge the search space by allowing all channel numbers to be selected from a large collection of choices (with a size of $O(n)$). This fine-grained channel number selection largely increases the number of candidates for each block: e.g., from 2–3 to $O(n^2)$ for a block with two convolutions.

Elastic network depth: For 3D CNNs, reducing the channel numbers alone cannot achieve significant measured speedup, which is different from normal 2D CNNs. For example, by shrinking all channel numbers in MinkowskiNet by $4\times$ and $8\times$, the number of MACs is reduced to 7.5 G and 1.9 G, respectively. However, although the number of MACs is drastically reduced, their measured latency on the GPU is very similar: 105 ms and 96 ms (on NVIDIA GTX 1080 Ti GPU). This suggests that scaling down the number of channels alone cannot offer us very efficient models, even though the number of MACs is minimal. This might be because 3D modules are usually more memory-bounded than 2D modules; MACs decrease quadratically with channel number, while memory decreases linearly. Thus, we incorporate the elastic network depth into our design space so that layers with very small computation (and considerable memory cost) can be removed and merged into their neighbouring layers.

Small kernel matters: Kernel sizes are usually included in the search space of 2D CNNs. This is because a single convolution with a larger kernel size can be more efficient than multiple convolutions with smaller kernel sizes on GPUs. However, it is not the case for 3D CNNs. From the perspective of computation cost, a single 2D convolution with a kernel size of 5 requires only $1.4\times$ more MACs than two 2D convolutions with kernel sizes of 3. In comparison, a single 3D convolution with a kernel size of 5 requires $2.3\times$ more MACs than two 3D convolutions with kernel sizes of 3 (if applied to dense voxel grids). This higher computation cost makes it less suitable to use large kernel sizes in 3D CNNs. Furthermore, the computation overhead of 3D modules is also related to the kernel sizes. For example, the sparse convolution requires $O(k^3n)$ time to build the kernel map, where k is the kernel size and n is the number of points, which indicates that its cost grows cubically with respect to the kernel size.

Based on these reasons, we decided to keep the kernel size of all convolutions to 3 and not allow the kernel size to change in our search space. Even with the small kernel size, we can still achieve a large receptive field by changing the network depth, achieving the same effect as changing the kernel size.

3.2.2. Training paradigm

Searching over a fine-grained design space is very challenging since it is impossible to train every sampled candidate network from scratch.⁵¹ Motivated by Guo *et al.*,⁵² we incorporate all candidate networks into a single super network. After training this super network once, each candidate network can be extracted with inherited weights. The total training cost during the neural architecture search can then be reduced from $\mathcal{O}(n)$ to $\mathcal{O}(1)$, where n is the number of candidate networks.

Uniform sampling: At each training iteration, we randomly sample a candidate network from the super network: i.e., randomly select the channel number for each layer and then randomly select the network depth (i.e., the number of blocks to be used) for each stage. The total number of candidate networks to be sampled during training is very limited; thus, we choose to sample different candidate networks on different GPUs and average their gradients at each step so that more candidate networks can be sampled. For 3D, this is more critical because the 3D datasets usually contain fewer samples than the 2D datasets: e.g., 20K on SemanticKITTI⁵³ vs. 1M on ImageNet.⁵⁴

Weight sharing: As the number of candidate networks is enormous, every candidate network will only be optimised for a small fraction of the total schedule. Therefore, uniform sampling alone is insufficient to train all candidate networks sufficiently (i.e., achieving the same level of performance as being trained from scratch). To tackle this, we adopt the weight-sharing technique so that every candidate network can be optimised at each iteration, even if it is not sampled. Specifically, given the input channel number C_{in} and output channel number C_{out} of each convolution layer, we simply index the first C_{in} and C_{out} channels from the weight tensor accordingly to perform the convolution.⁵² We similarly crop the first c channels

from the weight tensor for each batch normalisation layer based on the sampled channel number c . Finally, with the sampled depth d for each stage, we keep the first d layers instead of randomly sampling d of them. This ensures that each layer will always correspond to the same depth index within the stage.

Progressive depth shrinking: Suppose we have n stages, each with m different depth choices ranging from 1 to m . If we sample the depth d_k for each stage k randomly, the expected total depth of the network will be $\mathbb{E}[d] = \sum_{k=1}^n \mathbb{E}[d_k] = n(m+1)/2$, which is much smaller than the maximum depth nm . Furthermore, the probability of the largest candidate network (with the maximum depth) being sampled is extremely small: m^{-n} . Therefore, the largest candidate networks are poorly trained due to the small possibility of being sampled. To this end, we introduce progressively depth shrinking to alleviate this issue. We divide the training epochs into m segments for m different depth choices. During the k th training segment, we only allow the depth of each stage to be selected from $m-k+1$ to m . This is essentially designed to enlarge the search space gradually so that these large candidate networks can be sampled more frequently.

3.2.3. Search algorithm

After training the super network, we use the evolutionary search to find the best network architecture under a specific resource constraint. We support #MACs and measured latency as our resource constraint.

#MACs constraint: #MACs is an implementation- and hardware-agnostic efficiency metric. Unlike dense 2D CNNs, #MACs of sparse 3D CNNs cannot be determined by the input size and network architecture. This is because the sparse convolution only performs the computation over the active synapses; thus, its computation cost is also related to the size of the kernel map, which is determined by the input sparsity pattern. To address this, we estimate the average kernel map size over the entire dataset for each convolution layer and then sum them up to compute the average #MACs.

Latency constraint: We also support the measured latency on the target hardware as our resource constraint. We first encode each candidate network into a 1D architecture vector. We then randomly sample 50,000 candidate networks from the design space and measure their latency on the target hardware. With the collected pairs of architecture vectors and measured latency, we finally train an MLP regressor to estimate the latency based on the network architecture. The resulting predictor can accurately predict the latency of different candidate networks with a relative error of less than 2% on both edge and cloud GPUs.

Evolutionary search: We automate the architecture search with the evolutionary algorithm.⁵² First, we initialise the population with n randomly sampled candidate networks. At every iteration, we evaluate all candidate networks in the population and select the k models with the highest accuracy (i.e., the fittest individuals). The population for the next iteration is then generated with $(n/2)$ mutations and $(n/2)$ crossovers. For each mutation, we randomly select one among the top- k candidates and modify each of its architectural parameters (e.g., channel numbers and network depths) with a pre-defined probability; for each crossover, we select two from the top- k candidates and produce a new network by fusing them randomly. Finally, the best network architecture is obtained from the population of the last iteration. During the evolutionary search, we ensure that all candidate networks in the population always meet the given resource constraint (we will resample another candidate network until the resource constraint is satisfied).

3.3. Experiments

In this section, we evaluate our models on six 3D benchmark datasets:

- ShapeNet⁸ (coarse-grained object part segmentation),
- PartNet⁵⁵ (fine-grained object part segmentation),
- S3DIS^{56,57} (indoor scene segmentation),
- SemanticKITTI⁵³ (outdoor scene segmentation),
- nuScenes⁵ (outdoor scene segmentation),
- KITTI⁵⁸ (outdoor object detection).

On these datasets, we evaluate four variants of our method:

- **PVCNN** (manually designed model with PVConv),
- **PVNAS** (3D-NAS applied to PVCNN),
- **SPVCNN** (manually designed model with SPVConv),
- **SPVNAS** (3D-NAS applied to SPVCNN).

3.3.1. 3D object part segmentation on ShapeNet

We first evaluate our method on 3D part segmentation and conduct experiments on the large-scale 3D object dataset: ShapeNet.⁸ As 3D objects are very small, it is suitable to process them using PVConv. Therefore, we build our PVCNN by replacing the MLP layers in PointNet² with PVConv’s.

Baselines: We use PointNet,² RSNet,²⁸ PointNet++¹⁵ (with multi-scale grouping), DGCNN,¹⁷ SpiderCNN,²⁰ and PointCNN¹⁸ as point-based baselines. We reimplement 3D-UNet⁵⁹ as the voxel-based baseline. For a fair comparison, we follow the same evaluation protocol as in Li *et al.*¹⁸ and Graham *et al.*²⁷ The evaluation metric is mean intersection-over-union (mIoU): we first calculate the part-averaged IoU for each of the 2,874 test models and average the values as the final metrics. Besides, we report the measured GPU latency and GPU memory consumption on a single NVIDIA GTX 1080 Ti GPU. We ensure the input data has the same size with 2048 points and a batch size of 8.

Results: As in Table 2, PVCNN outperforms all previous models. It directly improves the accuracy of its backbone (PointNet) by 2.5% with an even smaller overhead compared with PointNet++. Besides, we also design narrower versions of our PVCNN by reducing the number of channels to 25% ($0.25 \times C$) and 50% ($0.5 \times C$). The resulting model requires only 46.4% latency of PointNet. It still outperforms several point-based methods with sophisticated neighbourhood aggregation, including RSNet, PointNet++, and DGCNN, which are almost an order of magnitude slower. PVCNN achieves a much better accuracy vs. latency trade-off compared with all point-based methods. With similar accuracy, PVCNN is **24** $\times$ faster than SpiderCNN and **3.3** $\times$ faster than PointCNN. Our PVCNN also achieves a significantly better accuracy vs. memory trade-off compared with the

Table 2. Results of object part segmentation on ShapeNet part. On average, PVCNN outperforms point-based models with $5.5\times$ speedup and $3\times$ memory reduction and outperforms the voxel-based baseline with $59\times$ measured speedup and $11\times$ memory reduction.

	#Par. (M)	#MACs (G)	Mem. (G)	Lat. (ms)	mIoU
PointNet ²	2.5	5.3	1.5	15.1	83.7
3D-UNet ⁵⁹	8.1	2996.9	8.8	682.1	84.6
RSNet ²⁸	6.9	1.4	0.8	74.6	84.9
PointNet++ ¹⁵	1.8	4.9	2.0	77.9	85.1
DGCNN ¹⁷	1.5	18.5	2.4	87.8	85.1
SPVCNN (0.25×C)	0.3	0.3	1.1	20.2	84.4
PVCNN (0.25×C)	0.3	1.0	0.8	8.3	85.2
SPVNAS	1.7	1.0	1.2	18.4	85.2
PVNAS-A	0.4	0.9	0.9	7.0	85.2
SpiderCNN ²⁰	2.6	10.6	6.5	170.7	85.3
SPVCNN (0.5×C)	1.1	1.3	1.4	24.7	85.1
PVCNN (0.5×C)	1.1	3.9	1.0	16.0	85.5
PVNAS-B	0.6	2.2	1.0	11.6	85.5
PVNAS-C	0.7	2.9	1.0	14.0	85.6
PointConv ⁶⁰	21.6	11.6	6.5	163.7	85.7
PointCNN ¹⁸	8.3	26.9	2.5	135.8	86.1
SPVCNN (1×C)	4.2	5.3	2.0	38.0	85.6
PVCNN (1×C)	4.2	15.3	1.6	41.4	86.2

voxel-based baseline. With better accuracy, PVCNN can save the GPU memory consumption by $10\times$ compared with 3D-UNet. We further apply 3D-NAS to PVCNN under different latency constraints to obtain a family of PVNAS models. With the same accuracy as PVCNN (0.25×C), PVNAS-A achieves $1.2\times$ faster inference speed. It also outperforms PointNet by 1.5 mIoU with $3\times$ measured speedup. Compared with PVCNN (0.5×C), PVNAS-B achieves $1.4\times$ speedup with no loss of accuracy, and PVNAS-C achieves $1.1\times$ speedup with better mIoU.

PV/SPVConv: On ShapeNet, PVCNN/PVNAS achieves far better efficiency-accuracy trade-offs than SPVCNN/SPVNAS. Specifically, PVCNN performs similar or better mIoU than SPVCNN with $2.4\times$ measured speedup. Similarly, PVNAS-A achieves the same accuracy as SPVNAS with $2.6\times$ lower latency. These results validate our claim that PVConv is more favourable for modelling small 3D objects.

3.3.2. 3D object part segmentation on PartNet

We also conduct experiments on the more challenging fine-grained 3D object part segmentation benchmark: PartNet.⁵⁵ Unlike ShapeNet, where each object is annotated with 2–6 parts, objects in PartNet can have as many as 50 parts. To perform fine-grained segmentation, the models usually take in a batch of 10,000 points instead of 2,048 points.

Baselines: We compare PVCNN with state-of-the-art point-based methods including PointNet,² PointNet++,¹⁵ Deep LPN,⁶¹ SpiderCNN,²⁰ PointCNN,¹⁸ and ResGCN.⁶² We also compare PVNAS with automatically designed point cloud segmentation networks, including SGAS⁶³ and LC-NAS.⁶⁴ To ensure fair comparisons, we follow the original experiment setting in Mo *et al.*⁵⁵ to train separate models for different object classes. The results of our method are averaged over at least three runs. For the other methods, we report the accuracy from their papers and measure #Params, #MACs, and latency with their open-source implementation.

Results: As in Table 3, PVCNN achieves superior results compared with all manually designed point-based methods. Specifically, PVCNN ($0.5\times C$) outperforms PointCNN with **27.7** $\times$ model size reduction, **31.3** $\times$ computation reduction, and **23.7** $\times$ measured speedup. This indicates that PVConv scales much better (w.r.t. the number of points) than other point-based primitives. Then, we apply 3D-NAS to PVCNN under 4.2/4.9 ms latency constraints on NVIDIA GTX 1080 Ti GPU. The resulting PVNAS outperforms PVCNN ($0.72\times C$) and PVCNN ($1\times C$) with 1.3 $\times$ and 1.4 $\times$ speedup, respectively. It also compares favourably with AutoML-based approaches. With more than **29** $\times$ speedup, PVNAS achieves a similar IoU compared with SGAS and LC-NAS.

PV/SPVConv: On PartNet, PVCNN ($0.5\times C$) is **3.8** $\times$ faster than SPVCNN ($1\times C$), while achieving **2.2%** higher accuracy. 3D-NAS improves the performance of SPVCNN significantly; however, SPVNAS is still **3.1** $\times$ slower than PVNAS-A. This again emphasises the importance of primitive selection.

Deployment: We deploy our PVCNN and PVNAS on edge devices. As in Figure 11(a), PVCNN runs in real time (20 FPS)

Table 3. Results of fine-grained object part segmentation on PartNet.

	#Par. (M)	#MACs (G)	Lat. (ms)	mIoU
PointNet ²	2.5	25.1	9.4	35.6
PointNet++ ¹⁵	1.8	5.4	26.0	42.5
Deep LPN ⁶¹	2.7	3.0	206.8	38.6
SpiderCNN ²⁰	2.6	52.2	2170.8	37.0
PointCNN ¹⁸	8.3	71.9	106.6	46.4
ResGCN ⁶²	3.8	55.9	771.3	45.1
SPVCNN (0.5×C)	0.3	1.6	13.5	43.6
PVCNN (0.5×C)	0.3	2.3	4.4	47.2
SPVCNN (0.72×C)	0.6	3.3	15.4	44.3
PVCNN (0.72×C)	0.6	4.7	5.5	47.3
SPVCNN (1×C)	1.1	6.4	16.9	45.0
PVCNN (1×C)	1.1	9.1	6.9	47.8
SGAS ⁶³	—	—	143*	48.3
LC-NAS-14 ⁶⁴	—	—	152*	48.6
LC-NAS-18 ⁶⁴	—	—	185*	46.6
SPVNAS	0.4	1.2	13.2	46.8
PVNAS-A	0.3	2.7	4.2	47.5
PVNAS-B	0.4	3.9	4.9	48.0

Notes: *: numbers are from Li *et al.*,⁶⁴ measured on an NVIDIA RTX 2080 GPU.

on NVIDIA Jetson Nano, which only consumes the power of a light bulb (5 W). Even when the number of input points increases by 5× on PartNet, PVNAS can still run at 8.3 FPS on NVIDIA Jetson Nano (Figure 11(b)). Thus, PVNAS can empower efficient 3D vision on low-power devices, which has become increasingly important with the rise of AR/VR applications.

3.3.3. 3D indoor scene segmentation

We then evaluate our method on 3D semantic segmentation and conduct experiments on the large-scale indoor scene dataset: S3DIS.^{56,57} Though indoor scenes are usually much larger (e.g., 6 m×6 m×3 m) than single 3D objects, it is still affordable to process them using sliding windows (e.g., 1.5 m×1.5 m×3 m). Note that the evaluation protocol is different from recent methods⁶ that take in the entire scene. Our setting is closer to the real-world scenario since depth cameras only capture a small region in a single pass. Similar

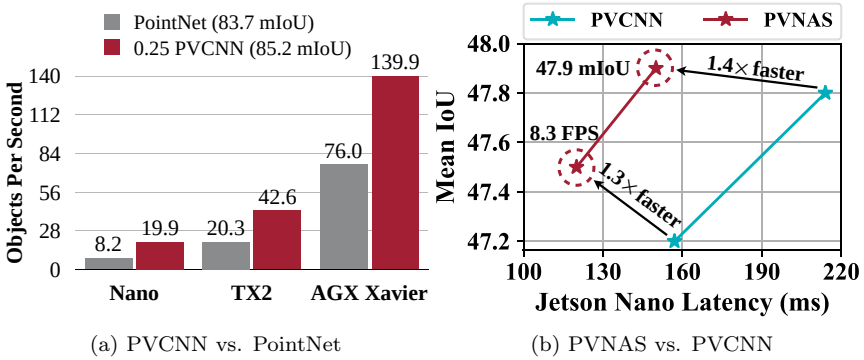


Fig. 11. PVCNN achieves **real-time** 3D object segmentation with 2,048 input points on edge devices. PVNAS achieves **8.3 FPS** with **10,000** input points on NVIDIA Jetson Nano.

to object part segmentation, we build PVCNN based on PointNet² and PVCNN++ based on PointNet++.¹⁵

Baselines: We compare our models with the state-of-the-art point-based models^{2,17,18,28} and the voxel-based baseline.⁵⁹ Following Tchapmi *et al.*²³ and Li *et al.*,¹⁸ we train the models on Area 1,2,3,4,6 and test them on Area 5 because it is the only one that does not overlap with any other area. Both data processing and evaluation protocol are the same as PointCNN¹⁸ for a fair comparison. We measure the latency and memory consumption with 32768 points per batch on a single NVIDIA GTX 1080 Ti GPU.

Results: As in Table 4, PVCNN improves its backbone (i.e., PointNet) by more than **13%** in mIoU and outperforms DGCNN (which involves sophisticated graph convolutions) by a large margin in both accuracy and latency. Remarkably, our PVCNN++ outperforms the state-of-the-art point-based model (PointCNN) by 1.7% in mIoU with **4×** lower latency and the voxel-based baseline (3D-UNet) by 4% in mIoU with more than **8×** lower latency and GPU memory consumption. Similar to object part segmentation, we also design compact models by reducing the number of channels in our PVCNN to 12.5%, 25%, and 50% and our PVCNN++ to 50%. Remarkably, the narrower version of our PVCNN outperforms DGCNN with **15×** measured speedup and RSNet with **9×** measured speedup. Furthermore, it achieves 4% improvement in mIoU over PointNet while still

Table 4. Results of indoor scene segmentation on S3DIS. On average, PVCNN and PVCNN++ outperform point-based models with $8\times$ speedup and $3\times$ memory reduction. They outperform the voxel-based baseline with $14\times$ speedup and $10\times$ memory reduction.

	#Par. (M)	#MACs (G)	Mem. (GB)	Lat. (ms)	mIoU
PointNet ²	1.2	3.6	1.0	12.1	43.0
PVCNN (0.125×C)	0.04	0.3	0.6	6.2	46.9
DGCNN ¹⁷	1.0	36.9	2.4	168.1	48.0
RSNet ²⁸	6.9	2.2	1.1	111.5	52.0
SPVCNN (0.25×C)	0.2	0.4	1.1	21.1	50.4
PVCNN (0.25×C)	0.2	0.9	0.7	9.0	52.3
3D-UNet ⁵⁹	14.0	349.6	6.8	574.7	55.0
SPVCNN (1×C)	2.7	6.6	1.8	41.0	54.9
PVCNN (1×C)	2.6	13.0	1.3	39.1	56.1
PVCNN++ (0.5×C)	3.4	6.6	0.7	40.9	57.6
PointCNN ¹⁸	11.5	17.5	4.6	282.3	57.3
PVCNN++ (1×C)	13.7	26.2	0.8	67.2	59.0

being $2.5\times$ faster than this highly efficient 3D model (which does not have any neighbourhood aggregation).

PV/SPVConv: With the same network structure, PVCNN is more effective than SPVCNN, achieving 1.2–2.1% higher mIoU (Table 4). Similar to our results on object segmentation, SPVCNN fails to achieve large speedups with small channel numbers: with $4\times$ channel reduction, SPVCNN achieves only $1.9\times$ speedup while PVCNN achieves $4.3\times$ speedup. Thus, PVCNN is a superior choice for indoor scene segmentation.

3.3.4. 3D outdoor scene segmentation on SemanticKITTI

We evaluate our method on 3D semantic segmentation and conduct experiments on the large-scale outdoor scene dataset: SemanticKITTI.⁵³ This dataset contains 23,201 LiDAR point clouds for training and 20,351 for testing, and it is annotated from all 22 sequences in the KITTI⁶⁷ Odometry benchmark. We train all models on the entire training set and report the mean intersection-over-union (mIoU) on the official test set under the single scan setting. We measure the latency on a single NVIDIA GTX 1080 Ti GPU with a batch size of 1.

Results: As in Table 5, SPVNAS outperforms the previous state-of-the-art MinkowskiNet⁶ by **3.3%** in mIoU with $1.7\times$ model size reduction, $1.5\times$ computation reduction, and $1.1\times$ measured speedup. Further, we downscale our SPVNAS by setting the resource constraint to 15G MACs. This offers us a much smaller model that outperforms MinkowskiNet with **8.3** $\times$ model size reduction, **7.6** $\times$ computation reduction, and **2.7** $\times$ measured speedup. In Figure 12, we provide qualitative comparisons between SPVNAS and MinkowskiNet: SPVNAS makes fewer errors for small instances.

Table 5. Results of outdoor scene segmentation on SemanticKITTI (3D). SPVNAS outperforms MinkowskiNet with **2.7** $\times$ speedup.

	#Params (M)	#MACs (G)	Latency (ms)	mIoU
PointNet ²	3.0*	—	500*	14.6
SPGraph ²⁵	0.3*	—	5200*	17.4
PointNet++ ¹⁵	6.0*	—	5900*	20.1
TangentConv ¹³	0.4*	—	3000*	40.9
RandLA-Net ⁶⁵	1.2	66.5	880 (256+624) [†]	53.9
KPConv ⁶⁶	18.3	207.3	—	58.8
MinkowskiNet ⁶	21.7	114.0	294	63.1
PVCNN	2.5	42.4	146	39.0
SPVCNN	21.8	118.6	317.1	65.3
SPVNAS-A	2.6	15.0	110	63.7
SPVNAS-B	12.5	73.8	259	66.4

Notes: [†]: Computation time + post-processing time. *: Results from Behley *et al.*⁵³

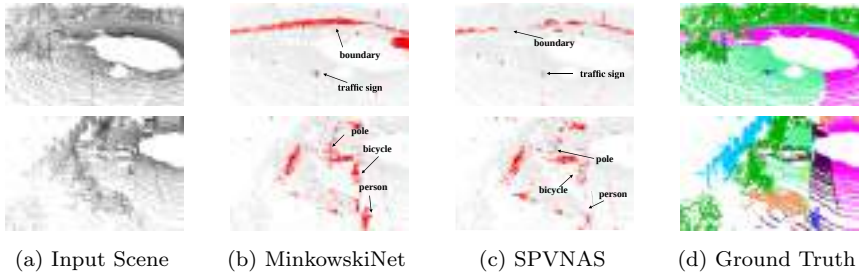


Fig. 12. MinkowskiNet usually has a higher error recognising small objects and region boundaries, while our SPVNAS recognises small objects better thanks to the high-resolution point-based branch.

Table 6. Results of outdoor scene segmentation on SemanticKITTI (2D). SPVNAS outperforms the 2D projection-based methods with at least $7.1\times$ computation reduction.

	#Params (M)	#MACs (G)	Latency (ms)	mIoU
DarkNet21Seg ⁵³	24.7	212.6	73 (49+24) [†]	47.4
DarkNet53Seg ⁵³	50.4	376.3	102 (78+24) [†]	49.9
SqueezeSegV3-21 ⁶⁸	9.4	187.5	97 (73+24) [†]	51.6
SqueezeSegV3-53 ⁶⁸	26.2	515.2	238 (214+24) [†]	55.9
3D-MiniNet ⁶⁹	4.0	—	—	55.8
PolarNet ⁷⁰	13.6	135.0	62	57.2
SalsaNext ⁷¹	6.7	62.8	71 (47+24) [†]	59.5
SPVNAS	1.1	8.9	89	60.3

Note: [†]: computation time + projection time.

In Table 6, we compare SPVNAS with 2D projection-based models. With a smaller backbone (by removing the decoder layers), SPVNAS outperforms DarkNets⁵³ by more than **10%** in mIoU with $1.2\times$ speedup even though 2D CNNs are much better optimised by modern deep learning libraries. Compared with other 2D methods, our SPVNAS achieves at least **8.5** $\times$ model size reduction and **15.2** $\times$ computation reduction while being much more accurate. Furthermore, SPVNAS achieves higher mIoU than KPConv,⁶⁶ which is the previous state-of-the-art point-based model, with **17** $\times$ model size reduction and **23** $\times$ computation reduction.

3.3.5. 3D outdoor scene segmentation on nuScenes

The distribution of point clouds varies significantly across different sensors. To verify the generalisability of our SPVCNN and SPVNAS, we also conduct experiments on a recent 3D segmentation benchmark: nuScenes.⁵ The dataset contains 34,149 LiDAR point clouds for training and 6,008 for testing and provides point-level semantic annotations for each point cloud. We follow the official instruction to split the training set into **train** split (consisting of 28,130 LiDAR point clouds) and **val** split (consisting of 6,019 LiDAR point clouds). We train all manually designed models on the **train** split and evaluate them on the **val** split. For our SPVNAS, we search the best model on a 20% holdout **minival** split from the **train** split and perform the evaluation on the **val** split. We report the official 31-class

Table 7. Results of outdoor scene segmentation on nuScenes. SPVNAS outperforms MinkowskiNet with $2.1\times$ computation reduction and $1.4\times$ measured speedup.

	#Params (M)	#MACs (G)	Latency (ms)	mIoU
PolarSeg	13.6	45.6	42.8	47.8
MinkowskiNet	21.7	22.2	81.0	53.7
SPVCNN	21.8	23.1	89.9	54.7
SPVNAS	9.6	10.6	59.1	54.7

mIoU as the evaluation metric. Like SemanticKITTI, we measure the latency on an NVIDIA GTX 1080 Ti GPU with a batch size of 1.

Results: As in Table 7, SPVNAS achieves a better accuracy vs. efficiency trade-off than both PolarSeg⁷⁰ and MinkowskiNet.⁶ It outperforms MinkowskiNet by 1.0% in mIoU with $2.3\times$ model size reduction, $2.1\times$ computation reduction, and $1.4\times$ speedup. It achieves 6.9% mIoU improvement over the projection-based PolarSeg. Both efficient 3D primitive design (SPVConv, $+1.0$ mIoU) and network architecture search (3D-NAS, $2.2\times$ computation reduction, $1.5\times$ measured speedup) contribute to the accuracy and efficiency boost of SPVNAS.

3.3.6. 3D outdoor object detection

We evaluate our method on 3D object detection and conduct experiments on the large-scale outdoor scene dataset: KITTI.⁶⁷ We follow the conventional training-validation split, where 3,712 samples are used for training and 3,769 samples are left for validation. We report the 3D average precision (with 40 recall positions) on the validation set under IoU thresholds of 0.7 for cars and 0.5 for cyclists and pedestrians.

Results: We first apply our method to F-PointNet,²⁹ a two-stage 3D object detection framework. It first leverages the off-the-shelf 2D object detector to produce frustum proposals. For each frustum, it then applies PointNets to classify the content. As the frustums are relatively small, it is suitable to process them using PVConv. Thus, we build our PVCNN based on F-PointNet by replacing the MLP layers within its instance segmentation network with PVConv

Table 8. Results of (two-stage) outdoor object detection on KITTI. PVCNN outperforms F-PointNet++ in all categories with $1.8\times$ measured speedup and $1.4\times$ memory reduction.

	Mem. (G)	Lat. (ms)	Car			Cyclist			Pedestrian		
			Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
F-PN	1.3	29.1	85.2	71.6	63.8	77.1	56.5	52.8	66.4	56.9	50.4
F-PN++	2.0	105.2	84.7	72.0	64.2	75.6	56.7	53.3	68.4	60.0	52.6
PVCNN	1.4	58.9	85.3	72.1	64.2	78.1	57.5	53.7	70.6	61.2	56.3

Table 9. Results of (one-stage) outdoor object detection on KITTI. SPVCNN outperforms SECOND in most categories, especially cyclists.

	Car			Cyclist			Pedestrian		
	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard
SECOND	89.8	80.9	78.4	82.5	62.8	58.9	68.3	60.8	55.3
SPVCNN	90.9	81.8	79.2	85.1	63.8	60.1	68.2	61.6	55.9

and keeping its box proposal and refinement networks unchanged. Next, we compare our PVCNN with F-PointNet (whose backbone is PointNet) and F-PointNet++ (whose backbone is PointNet++). In Table 8, even if our PVCNN does not aggregate neighbouring features in the box estimation network while F-PointNet++ does, PVCNN still outperforms it in all classes with $1.8\times$ lower latency. Remarkably, our model achieves 2.4% average mAP improvement in the most challenging pedestrian class. Moreover, compared with F-PointNet, our PVCNN obtains up to $4\text{--}5\%$ mAP improvement in pedestrians, indicating that our model is efficient and expressive.

Apart from the frustum-based framework, we apply our method to SECOND,³¹ a single-stage 3D object detection framework. It consists of a sparse encoder using 3D sparse convolutions and a region proposal network that performs 2D convolutions after projecting the encoded features into the bird’s-eye view (BEV). Unlike F-PointNet, SECOND is applied to the entire outdoor scenes, which are of large scale. Therefore, we choose to replace the sparse encoder with SPVCNN. As a fair comparison, we reimplement and retrain SECOND, which outperforms the results claimed in the original paper.³¹ As summarised in Table 9, SPVCNN consistently improves in all

categories, especially cyclists and pedestrians. This is because the high-resolution point-based branch carries more information for small objects.

3.4. Summary

In this section, we systematically analysed the bottlenecks of previous point-based and voxel-based models and proposed a novel hardware-efficient 3D primitive to combine the best from both. We further enhanced this primitive with the sparse convolution to make it more effective for large (outdoor) scenes. Finally, based on our designed 3D primitive, we introduced the 3D neural architecture search framework to automatically explore the best network architecture that satisfies a given resource constraint. Extensive experiments on multiple 3D benchmark datasets validate the efficiency and effectiveness of our proposed method.

4. System: TorchSparse

A key operator in SPVCNN is sparse convolution (defined in Equation (3)). It is not well optimised on GPUs due to the high cost of data movement and fragmented GEMM computation. Several pioneering implementations of sparse convolution have adopted different dataflows for this operator. For instance, SparseConvNet²⁷ and SpConv³¹ use the vanilla gather-GEMM-scatter dataflow, while MinkowskiEngine⁶ proposes the fetch-on-demand dataflow. TorchSparse v1⁷² optimises the gather-scatter paradigm by fusing memory operations and grouping computations adaptively into batches to improve device utilisation. Recently, SpConv v2^{31,73} has adapted the implicit GEMM dataflow for dense convolution to the sparse domain, achieving state-of-the-art performance on real-world workloads. However, all libraries assume a single dataflow across the execution of the model, which limits the design space for kernel optimisation, such as tiling parameters.

In this section, we present an in-depth analysis of existing dataflows. Our analysis identifies significant room for optimisation, even for well-engineered kernels tuned at the PTX assembly level in SpConv v2.^{31,73} For instance, although SpConv v2 attempts to reduce computation overhead in implicit GEMM using mask sorting, the control flow overhead for zero skipping outweighs the

benefits of smaller redundant computation. Furthermore, the fetch-on-demand dataflow implemented by MinkowskiEngine and SpConv could achieve significantly better device occupancy with a simple block fusion design. After applying our optimisations to these existing dataflows, we further design an autotuner that determines the optimal runtime dataflow for each layer at compile time.

We also extend the opset of existing inference engines to support all primitives for 3D semantic segmentation, object detection, and scene reconstruction. Our system, TorchSparse, has been evaluated on seven representative models across three benchmark datasets, achieving end-to-end speedups of **2.9** $\times$, **3.3** $\times$, **2.2** $\times$, and **1.8** $\times$ on an NVIDIA A100 GPU, outperforming state-of-the-art systems, such as MinkowskiEngine, SpConv 1.2, TorchSparse, and SpConv v2. Additionally, in the training phase, our system outperforms the best existing training system, SpConv v2, by up to **1.3** $\times$ under mixed-precision training scenarios.

4.1. *TorchSparse Library*

TorchSparse APIs: TorchSparse offers easy-to-use and PyTorch-like APIs, depicted in Figures 13 and 14. Both `module` and `functional` implementations are provided for all sparse point cloud operators. To construct a sparse convolutional network for point clouds, only a conversion from PyTorch’s `nn.Conv3d`, `nn.BatchNorm3d`, and `nn.ReLU` to our `spnn` counterpart is required, as shown in Figure 13. Unlike other libraries, users do not need to specify additional fields at module initialisation time in TorchSparse. For example, SpConv requires an `indice_key` field during convolution class initialisation to help the system exploit map reuse opportunities. However, this is done automatically in the TorchSparse frontend without user annotation. We also provide the same forward and backward implementation conventions as PyTorch. As in Figure 14, it is much more intuitive in TorchSparse to implement a sparse residual block compared with SpConv. Furthermore, when integrated into existing frameworks such as `mmdet3d`,⁷⁴ which provides existing implementations for the residual block (and many other `Conv2d`-based blocks) for images, one only needs to override its member modules with `spnn` equivalence and does not even need to write

<pre># Import libraries from torch import nn import torchsparse.nn as spnn from torchsparse import SparseTensor # Tensor construction tensor = SparseTensor(feats, coords).cuda() # Neural network definition net = nn.Sequential(spnn.Conv3d(IC, OC, kernel_size=3, stride=2, padding=1), spnn.BatchNorm(OC), spnn.ReLU(True), spnn.Conv3d(OC, OC, kernel_size=[1, 3, 1], stride=1, padding=[0, 1, 0])).cuda()</pre>	<pre># Inference out = net(tensor) # To PyTorch dense tensor out_dense = out.dense() # Training out_dense.backward(top_grad) # mmdet3d Integration from mmdcv.cnn.bricks.registry import * from mmdcv.cnn import build_conv_layer CONV_LAYERS._register_module(spnn.Conv3d, "SparseConv3d", force=True) conv_layer = build_conv_layer(dict(type="SparseConv3d"), IC, OC, kernel_size, stride=stride, padding=padding, bias=False) # Equivalent to spnn.Conv3d(IC, OC, ...)</pre>
---	---

Fig. 13. TorchSparse offers user-friendly, PyTorch-like interfaces that allow seamless support of both training and inference. It can be easily integrated into open-source frameworks such as *mmdet3d*⁷⁴ with <10 lines of code.

<pre>def forward(self, x): identity = x.features out = self.conv1(x) out = out.replace_feature(self.bn1(out.features)) out = out.replace_feature(self.relu(out.features)) out = self.conv2(out) out = out.replace_feature(self.bn2(out.features)) if self.downsample is not None: identity = self.downsample(x) out = out.replace_feature(out.features + identity) out = out.replace_feature(self.relu(out.features)) return out</pre>	<pre>def forward(self, x): identity = x out = self.relu(self.bn1(self.conv1(x))) out = self.relu(self.bn2(self.conv2(out))) if self.downsample is not None: identity = self.downsample(x) out = out + identity out = self.relu(out) return out</pre>
Left: <i>SpConv</i> implementation Right: <i>TorchSparse++</i> implementation	

Fig. 14. TorchSparse offers a more intuitive implementation for a residual block composed of sparse convolution layers compared with *SpConv*.^{31,73}

the forward function. Our API design dramatically simplifies the development of point cloud models.

Integration: It is easy to integrate TorchSparse into existing algorithm frameworks and open-source repositories. For example, as in

Figure 13, frameworks like `mmdet3d`⁷⁴ often provide *registries* to support building modules from a configuration dictionary. Registering operators in TorchSparse is as simple as registering a `Conv2d` layer to these frameworks.

4.2. Implementation

Most functions in TorchSparse are implemented straightforwardly. However, it is non-trivial to map the sparse convolution operator onto GPUs efficiently. We, therefore, elucidate the rationale behind our design choices and optimisations on this operator in this section.

4.2.1. Gather-GEMM-scatter dataflow

Overview: A gather-GEMM-scatter^{27,31} implementation of sparse convolution (Figure 16) will finish all computation for one weight $\mathbf{W}_\delta$ before moving on to the other. It has an outmost host loop over K^D kernel offsets. For each offset δ , we first calculate all pairs $\mathcal{M}_\delta = \{(\mathbf{p}_j, \mathbf{q}_k)\}$ such that $\mathbf{p}_j = \mathbf{s}\mathbf{q}_k + \delta$. As is shown in Figure 15(a), we then group all input features $\{\mathbf{x}_j^{\text{in}}\}$ together, resulting in a $|\mathcal{M}_\delta| \times C_{\text{in}}$ matrix in DRAM, and multiply it with weight $\mathbf{W}_\delta \in \mathbb{R}^{C_{\text{in}} \times C_{\text{out}}}$, finally we scatter the results back to output positions $\{\mathbf{x}_k^{\text{out}}\}$ according to $\mathcal{M}_\delta$. For example, since $\mathbf{p}_0 = 1 \times \mathbf{q}_1 + (-1, -1)$, $\mathbf{p}_4 = 1 \times \mathbf{q}_5 + (-1, -1)$, we group features $\mathbf{x}_0^{\text{in}}, \mathbf{x}_4^{\text{in}}$ together, multiply them by $\mathbf{W}_{-1,-1}$, and scatter back to outputs $\mathbf{x}_1^{\text{out}}, \mathbf{x}_5^{\text{out}}$.

Advantages: The gather-GEMM-scatter dataflow has small and controllable computation overhead, thus suitable for devices with limited computation capability. It is also easy to implement and maintain. After feature gathering, the main computation for each offset δ is simply dense matrix multiplication and can be delegated to existing vendor libraries, such as cuBLAS and cuDNN. As such, only the data movement operations (i.e., scatter and gather) need to be implemented and optimised in CUDA.

Challenges and improvements: We follow TorchSparse v1⁷² to improve the cache locality in gather and scatter operations by reordering the memory access. We also batch GEMMs for different δ s together to greatly improve device utilisation. However, such simplicity comes at the cost of performance. Even if TorchSparse v1⁷²

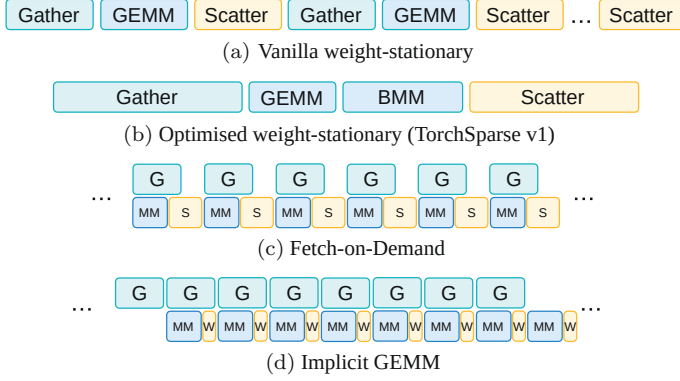


Fig. 15. Waterfall diagram for different dataflows for sparse convolution on GPU: weight-stationary dataflows (a, b) are easier to implement and maintain but do not overlap memory access with computation. Both fetch-on-demand and implicit GEMM dataflows require custom MMA routines but can hide the memory access time with pipelining.

$W_{-1,-1}$	$W_{-1,0}$	$W_{-1,1}$	$W_{0,-1}$	$W_{0,0}$	$W_{0,1}$	$W_{1,-1}$	$W_{1,0}$	$W_{1,1}$
x_0^{in}	x_1^{in}	x_2^{in}	x_1^{in}	$x_0^{\text{i/o}}$	x_2^{in}	x_3^{in}	x_3^{in}	x_1^{in}
x_4^{in}	$\downarrow$	$\downarrow$	$\downarrow$	$x_1^{\text{i/o}}$	$\downarrow$	$\downarrow$	$\downarrow$	x_5^{in}
$\downarrow$	x_3^{out}	x_3^{out}	x_2^{out}	$x_2^{\text{i/o}}$	x_1^{out}	x_2^{out}	x_1^{out}	$\downarrow$
x_1^{out}				$x_3^{\text{i/o}}$				x_0^{out}
x_5^{out}				$x_4^{\text{i/o}}$				x_4^{out}
				$x_5^{\text{i/o}}$				

Fig. 16. Illustration of the gather-GEMM-scatter dataflow for Figure 4 workload: we first gather input features according to $\mathcal{M}_\delta$ for each weight δ , then perform GEMM or batched GEMM, and finally scatter the results back to output locations given in $\mathcal{M}_\delta$.

exploits perfect data reuse, it is still inevitable to perform a huge DRAM access of $\sum_{\delta \in \Delta^D(K)} |\mathcal{M}_\delta| \times (C_{\text{in}} + C_{\text{out}})$ for writing the gather buffer and reading from the scatter buffer. This is 4-10 $\times$ as large as the input/output features in real workloads. Worse still, as in Figures 15(a) and 15(b), memory operations are non-overlapping with computation because the dense matrix multiplication kernel cannot be invoked until the gather buffer is filled. The black-box nature

of vendor libraries also makes it impossible for us to write back to DRAM in advance.

4.2.2. Fetch-on-demand dataflow

Overview: The gather-GEMM-scatter implementation requires three separate CUDA kernel calls in each host loop iteration over δ . An alternative fetch-on-demand dataflow⁶ merges the gather, matrix multiplication, and scatter kernel calls into a single CUDA kernel. Instead of materialising the $|\mathcal{M}_\delta| \times C_{\text{in}}$ gather buffer in DRAM, it *fetches* $\{\mathbf{x}_j^{\text{in}} | (\mathbf{p}_j, \mathbf{q}_k) \in \mathcal{M}_\delta\}$ *on demand* into the L1 shared memory, performs matrix multiplication in the on-chip storage, and directly scatters the partial sums (resided in the register file) to corresponding outputs $\{\mathbf{x}_k^{\text{out}} | (\mathbf{p}_j, \mathbf{q}_k) \in \mathcal{M}_\delta\}$ without first instantiating them in a DRAM scatter buffer.

Advantages: The fetch-on-demand dataflow enjoys the similar benefit of low redundant computation to gather-GEMM-scatter. Despite not being able to exploit perfect reuse opportunities in both gathering and scattering as [72], it overlaps the computation with memory access operations. It also saves DRAM writes to large gather/scatter buffers. Thus, it is faster than a vanilla gather-scatter dataflow (Figure 15(a)).

Challenges and improvements: Fetch-on-demand dataflow has been implemented as part of MinkowskiEngine⁶ and SpConv.^{31,73} Both implementations assume that computation for different weights is done separately. As studied in TorchSparse,⁷² the per-weight workload sizes could be small (~ 1000) on a dataset with sparser LiDAR scans (e.g., nuScenes). Thus, the GPU can be underutilised with separate computation. To tackle this challenge, we incorporate block fusion⁷⁵ in our implementation, which launches a group of thread blocks for each weight *in parallel*. This effectively increases the parallelism by $|\Delta^D(K)| \times$. We profile the impact of block fusion in Figure 17. We notice that block fusion significantly outperforms the separate computation variant implemented by MinkowskiEngine and SpConv in layers with *few input points* and *large channels*. In layers with a large number of inputs, the gain from block fusion is much smaller. This is because we must introduce atomic operations

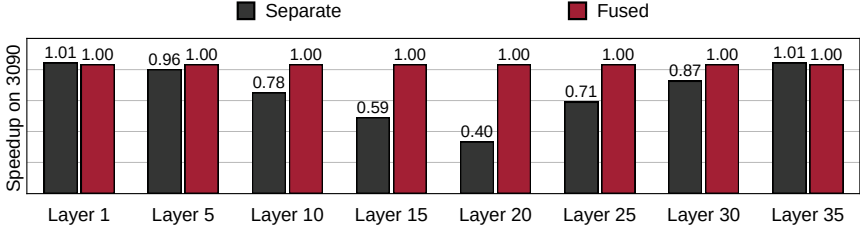


Fig. 17. Effectiveness of block fusion in the fetch-on-demand dataflow with MinkUNet⁶ architecture on the SemanticKITTI⁵³ dataset. Block fusion is more effective for middle layers with large channels and a small number of input points. Remarkably, it could be **2.5** $\times$ faster than the non-fusion baseline in Layer 20, which is a layer that implicit GEMM is not good at.

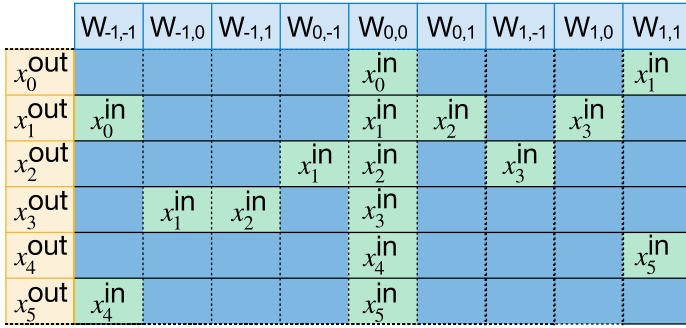


Fig. 18. Illustration of the vanilla implicit GEMM dataflow for Figure 4 workload: each green grid corresponds to a C_{in} -dimensional input feature and blue grids correspond to the redundant computation. The input feature matrix is **not** stored in DRAM.

to avoid race conditions when writing back the results. For example, both $W_{-1,0}$ and $W_{-1,1}$ in Figure 16 want to write back to x_3^{out} .

4.2.3. Implicit GEMM dataflow

Overview: A very recent advance in sparse convolution inference, SpConv v2,^{31,73} takes an alternative approach of implicit GEMM computation, which is originally designed for dense convolution on images in CUTLASS⁷⁶ and cuDNN⁷⁷ and has been implemented in PointAcc,⁷⁸ a specialised accelerator for point cloud workloads. As in Figure 18, the sparse convolution workload in Figure 4 is equivalent to a dense GEMM $C = A_{M \times K} \times B_{K \times N}$ with $M = N_{\text{out}}$ (number of output points), $N = C_{\text{out}}$, $K = |\Delta^D(K)|C_{\text{in}}$. We visualise

matrix $\mathbf{A}$ in Figure 18, where we have $N_{\text{out}} \times |\Delta^D(K)|$ grids and each grid corresponds to C_{in} channels. Grids in green correspond to input features. For example, output point $\mathbf{q}_1$ has four neighbours: $\mathbf{p}_0$ (with $\mathbf{w}_{-1,-1}$), $\mathbf{p}_1$ (with $\mathbf{w}_{0,0}$), $\mathbf{p}_2$ (with $\mathbf{w}_{0,1}$), and $\mathbf{p}_3$ (with $\mathbf{w}_{1,0}$) so the second row in Figure 18 has four green entries. *Implicit* GEMM means that instead of explicitly writing $\mathbf{A}$ into DRAM in an im2col⁷⁷ manner, we load tiles of $\mathbf{A}$ into the on-chip SRAM on the fly. Indices in $\mathbf{A}$ are mapped to locations in the input feature map according to neighbourhood relationships.

SpConv v2 — Sorted implicit GEMM: In Figure 18, we assume that each warp has three threads, and each thread performs calculations on one output point. A vanilla implicit GEMM implementation performs 0×0 computation for positions without corresponding neighbours. As such, there are 16 effective MACs and 38 wasted MACs in Figure 18. In real-world workloads, the amount of redundant computation ranges from 30% to 300% of effective MACs.

Therefore, Yan *et al.*^{31,73} proposed SpConv v2 to reduce the proportion of wasted computation. As in Figure 19(a), a bitmask is first calculated for each output point. It indicates whether a specific neighbour of each output point exists. The bitmask is then converted to a decimal number and sorted. As such, in Figure 19(b), the number of redundant MACs is reduced from 38 to 14. Experiments on real-world LiDAR scans show that sorting the bitmask can typically bring about a $1.3\times$ to $2.0\times$ reduction in redundant computation. To further reduce wasted MACs, Yan *et al.* split the bitmask into two trunks and sort these two bitmasks separately. A SplitK reduction⁷⁶ followed by output reordering is performed to get the final results.

Advantages: An implicit GEMM dataflow does not require scattering since its writeback stage is output-stationary. Input feature gathering and matrix multiplication-accumulation (MMA) operations can also be pipelined because they are implemented in a single CUDA kernel. Consequently, as in Figure 15(d), the ideal run-time of an output-stationary kernel is $\max(\text{time_gather}, \text{time_mma} + \text{time_wb})$, as opposed to $\text{time_gather} + \text{time_mma} + \text{time_scatter}$ in a gather-GEMM-scatter dataflow.

Challenges and improvements: Despite the improved performance resulting from overlapped computation and memory

	$W_{-1,-1}$	$W_{-1,0}$	$W_{-1,1}$	$W_{0,-1}$	$W_{0,0}$	$W_{0,1}$	$W_{1,-1}$	$W_{1,0}$	$W_{1,1}$		
x_0^{out}	0	0	0	0	1	0	0	0	1	17	1 st
x_1^{out}	1	0	0	0	1	1	0	1	0	282	6 th
x_2^{out}	0	0	0	1	1	0	1	0	0	52	3 rd
x_3^{out}	0	1	1	0	1	0	0	0	0	208	4 th
x_4^{out}	0	0	0	0	1	0	0	0	1	17	2 nd
x_5^{out}	1	0	0	0	1	0	0	0	0	272	5 th

(a) Bitmask in SpConv v2.

	$W_{-1,-1}$	$W_{-1,0}$	$W_{-1,1}$	$W_{0,-1}$	$W_{0,0}$	$W_{0,1}$	$W_{1,-1}$	$W_{1,0}$	$W_{1,1}$	
x_0^{out}					x_0^{in}				x_1^{in}	
x_4^{out}					x_4^{in}				x_5^{in}	
x_2^{out}				x_1^{in}	x_2^{in}		x_3^{in}			
x_3^{out}		x_1^{in}	x_2^{in}		x_3^{in}					
x_5^{out}	x_4^{in}				x_5^{in}					
x_1^{out}	x_0^{in}				x_1^{in}	x_2^{in}		x_3^{in}		

(b) Sorting reduces redundant computation in SpConv v2.

Fig. 19. SpConv v2 sorts the input bitmasks and reorders the computation accordingly. White grids correspond to skipped zero computation. As a result, redundant computation is reduced from 38 MACs (Figure 18) to 14 MACs for the example in Figure 4. For simplicity, we do not visualise mask splitting in SpConv v2. Each thread block has three threads.

operations, the SpConv v2 dataflow still has some shortcomings. First, the implicit GEMM dataflow cannot be implemented using existing matrix multiplication APIs provided by vendor libraries. Consequently, the authors of SpConv v2 had to re-implement the entire CUTLASS framework from scratch using a custom Python-based template metaprogrammer,⁷⁹ which comprises a staggering **40,000** lines of code. This makes SpConv v2 difficult to maintain and improve upon. To address this issue, we manually developed a code generator (with the help of TVM⁸⁰ and TensorIR⁸¹) that has the same coverage (Pascal to Hopper architectures, FP16/TF32/FP32 precisions) as SpConv v2. As a result, our code generator delivers

better performance than SpConv v2 with less than **2,000** lines of code, making it easier to extend and maintain.

Furthermore, it requires explicit bitmask calculation and sorting in Figure 19(a). Sorting is a memory-intensive operator that can only run on low-throughput CUDA cores. Modern GPUs tend to greatly increase the performance of tensor cores at the cost of a reduced number of CUDA cores. As a result, the tensor cores on A100 and H100 deliver $16\times$ better peak performance compared with CUDA cores on the same device. Worse still, even if sorting operators are free, we still need to implement zero-skipping control logic in the computation kernel. The control logic will inevitably introduce branch divergences, which could also limit performance. We further benchmark the effectiveness of sorting in the SpConv v2 dataflow on seven representative self-driving workloads in Figures 20 and 21. Despite reducing the computation overhead by $1.3\text{--}2\times$, the sorting overhead still outweighs such benefit on real-world detection workloads (Figure 21).

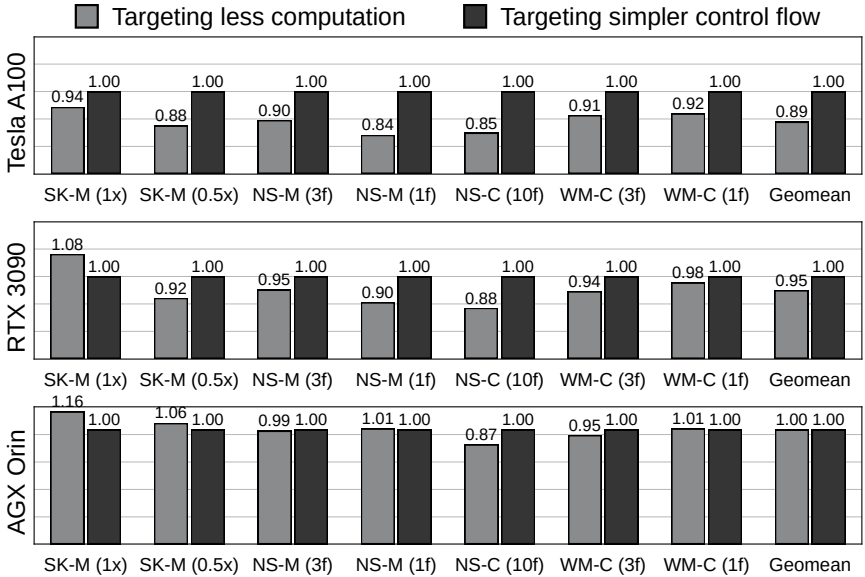


Fig. 20. Impact of sorting in TorchSparse using the implicit GEMM dataflow. Sorting is beneficial for large workloads (e.g., MinkUNet on SemanticKITTI, with >100 GFLOPs per scene) and computation-limited devices but does not show a clear advantage in average latency even on Orin.

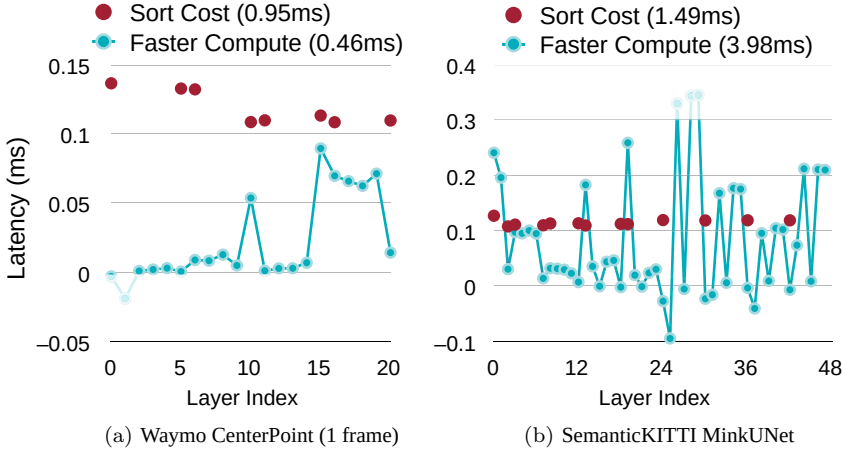


Fig. 21. Sorting is able to reduce the computation time, but its overhead outweighs the benefit on detection workloads.

4.2.4. Automatic dataflow selection

After implementing our optimisations, we discovered that different dataflows still have unique advantages and shortcomings. For instance, the fetch-on-demand dataflow enjoys the largest parallelism with block fusion, but the increased parallelism comes at the cost of many atomic operations. Furthermore, it still writes partial sums to the DRAM more than once, slower than the implicit GEMM dataflow that only writes back once for each output point. On the other hand, implicit GEMM is not perfect either. The implicit $\mathbf{A}$ matrix in Section 4.2.3 has $|\Delta^D(K)|C_{\text{in}}$ columns, corresponding to 3456 columns for a $3 \times 3 \times 3$ neighbourhood and 128 input channels. Since state-of-the-art MMA routines^{76,77} traverse input columns sequentially, which can significantly limit parallelism. To benefit from all the optimised dataflows in TorchSparse, we introduce an autotuner that selects the best kernel layerwise. This approach ensures that we leverage the strengths of each dataflow while mitigating their weaknesses.

To determine the optimal dataflow for different layers, we divide all layers into different groups (illustrated in Figure 22) following [75]. All layers within each group use the same input-output mappings (*maps*) and are forced to execute the same dataflow. This is because

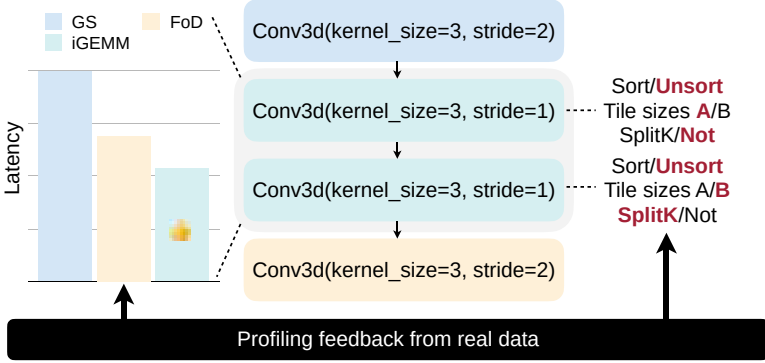


Fig. 22. Illustration of two-level compile-time autotuner in TorchSparse. Layers using the same kernel maps will be assigned to the same group. After group partition, we search for the dataflow within each group and then decide the dataflow-specific parameters for each layer (e.g., tile sizes and parallelism). Profiling feedback on a small subset of the interested workload guides the dataflow and parameter selection.

different dataflows require different map structures. Implementations such as gather-GEMM-scatter and fetch-on-demand require the maps to be stored in a weight-stationary order, represented as $\mathcal{M}_\delta = \{(\mathbf{p}_j, \mathbf{q}_k) | \mathbf{p}_j = \mathbf{s}\mathbf{q}_k + \delta, \mathbf{q}_k \in \mathbf{P}^{\text{out}}\}$, which makes it difficult to infer all the neighbours of an output point (required by implicit GEMM). On the other hand, the implicit GEMM implementation stores the maps in an output-stationary order, represented as $\mathcal{M}_k = \{\mathbf{p}_k^{(\delta)} | \mathbf{p}_k^{(\delta)} = \mathbf{s}\mathbf{q}_k + \delta, \delta \in \Delta^D(K)\}$, which makes it difficult to infer all the inputs that use the same weight (required by the other two dataflows). Moreover, generating maps for all dataflows will incur significant overhead ($\sim$ runtime for one sparse convolution layer within each group). Therefore, allowing intra-group heterogeneous dataflow selection is not desired.

After group partition, we apply a two-level exhaustive search on a subset of the target workload. We first traverse all groups and choose the dataflow for each group from [gather-GEMM-scatter, fetch-on-demand, implicit-GEMM, sorted-implicit-GEMM].^d We then iterate over all layers within each group and perform a grid

^dThis is because the implicit GEMM dataflow requires different map structures with or without sorting.

scan over all map-agnostic layer parameters (e.g., whether to use block fusion in fetch-on-demand, tile sizes for all dataflows, computation overhead tolerance in gather-scatter). Finally, we record the configuration for each layer that achieves the lowest average latency on the random subset.

4.3. Evaluation

4.3.1. Setup

We build our system based on TorchSparse v1⁷² and compare it with four libraries, including MinkowskiEngine 0.5.4,⁶ SpConv 1.2.1,³¹ TorchSparse v1,⁷² and SpConv 2.2.3.³¹ All systems are integrated into PyTorch 1.12.0 with CUDA 11.3/11.7 and cuDNN 8.4.1. We follow TorchSparse v1⁷² to evaluate all systems on seven workloads: MinkUNet⁶ ($0.5 \times / 1 \times$ width) on SemanticKITTI,⁵³ MinkUNet ($1/3$ frames) on nuScenes-LiDARSeg,⁵ CenterPoint³² (10 frames) on nuScenes detection, and CenterPoint ($1/3$ frames) on Waymo Open Dataset.³³ For detection workloads (CenterPoint), *we only evaluate the runtime of SparseConv layers.*

4.3.2. Inference

We compare our results with the baseline design MinkowskiEngine, SpConv 1.2.1, TorchSparse v1, and SpConv 2.2.3 in Figure 23. All evaluations are done in unit batch size. TorchSparse outperforms baseline systems consistently on both Tesla A100 and RTX 3090 GPUs and achieves **2.9–3.7 $\times$** , **3.2–3.3 $\times$** , **2.0–2.2 $\times$** , and **1.4–1.8 $\times$** measured end-to-end speedup over the state-of-the-art MinkowskiEngine, SpConv 1.2.1, TorchSparse v1, and SpConv 2.2.3, respectively. In Figure 24, we also compare TorchSparse with SpConv 2.2.3 on NVIDIA Jetson Orin, an edge GPU platform widely deployed on real-world autonomous vehicles. TorchSparse is **1.24 $\times$** faster than SpConv 2.2.3 while achieving up to **1.57 $\times$** speedup on nuScenes.

4.3.3. Training

We compare the training performance of TorchSparse and existing systems on a single A100 GPU in Figure 25. We run the forward and backward pass of all workloads with batch sizes of 1

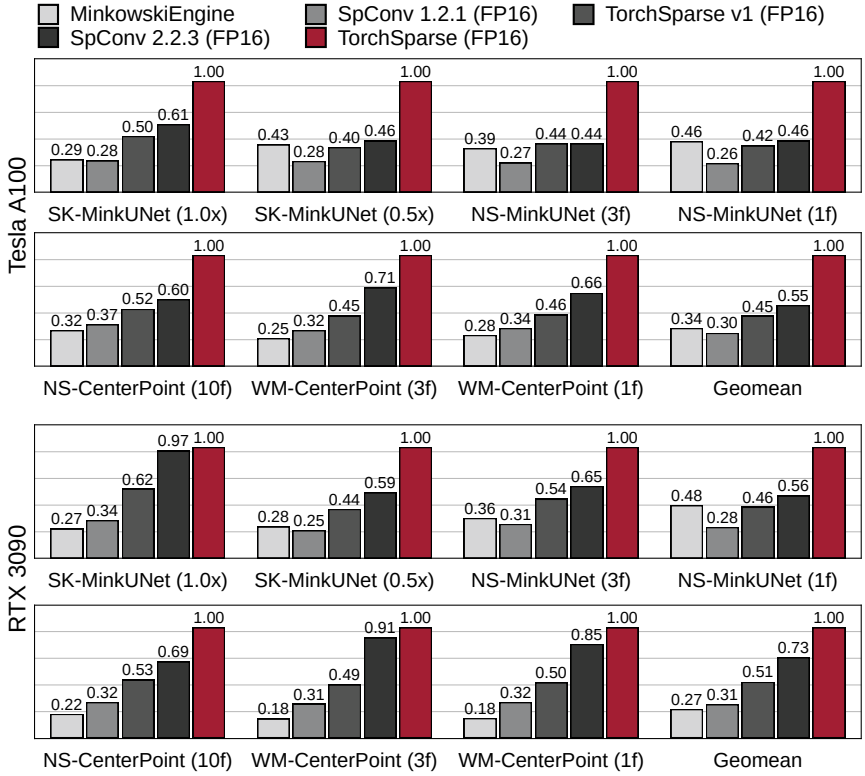


Fig. 23. TorchSparse significantly outperforms existing point cloud inference engines in 3D object detection and LiDAR segmentation benchmarks. It achieves $1.36\text{--}1.81\times$ geomean speedup over state-of-the-art SpConv 2.2.3 and is $1.96\text{--}2.20\times$ faster than TorchSparse v1.

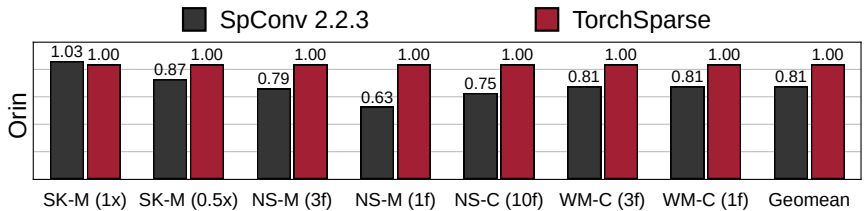


Fig. 24. TorchSparse is $1.24\times$ faster on average and up to $1.57\times$ more efficient than SpConv 2.2.3 on resource-constrained NVIDIA Jetson Orin. All workloads run at >18 FPS, significantly faster than the LiDAR frequency.

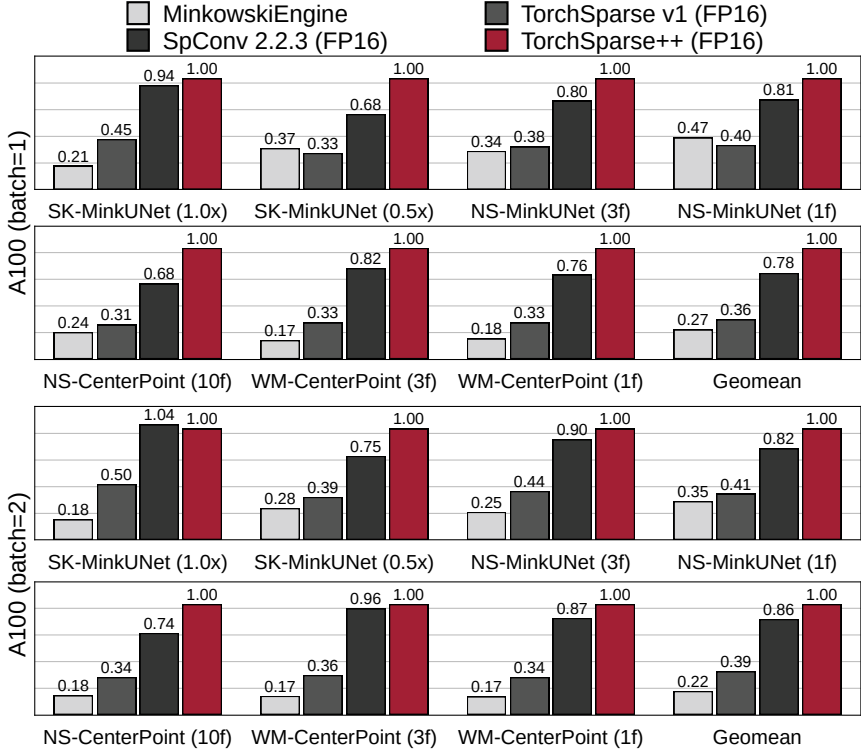


Fig. 25. TorchSparse achieves better mixed-precision training performance compared with MinkowskiEngine, TorchSparse v1, and SpConv 2.2.3. It is $1.29\times$ faster than SpConv 2.2.3 with a batch size of 1 and $1.16\times$ faster with a batch size of 2. It significantly outperforms MinkowskiEngine by up to $4.6\times$ across seven benchmarks.

and 2 in mixed precision training (i.e., all gradients are calculated in FP16) except for MinkowskiEngine, which does not support FP16. All workloads evaluated in Figure 25 can reach the same accuracy using TorchSparse compared with TorchSparse v1 (for segmentation workloads) and SpConv 2.2.3 (for detection workloads) with FP32 precision in both unit-batch and batch=2 training. Given the fact that A100 FP16 tensor core arithmetics has $16\times$ higher throughput compared with FP32 (non-tensor core) computation (312 TFLOPS vs. 19.5 TFLOPS), we do not perform FP32 evaluation. As a result, TorchSparse is $3.8\text{--}4.6\times$, $2.5\text{--}2.8\times$, and $1.2\text{--}1.3\times$ faster than MinkowskiEngine, TorchSparse v1, and SpConv 2.2.3, respectively. Therefore,

TorchSparse paves the way for rapid model iteration for real-world autonomous driving applications.

4.4. *Summary*

This section introduced TorchSparse, a high-performance GPU computation library designed for deep learning on point clouds. TorchSparse supports all primitives required for 3D semantic segmentation, object detection, and scene reconstruction workloads in autonomous driving. We conducted a holistic analysis of existing dataflows for point cloud convolution. We further improved each dataflow to increase parallelism for fetch-on-demand and reduce control flow overhead for implicit GEMM. Our highly optimised implementations enabled us to build an input-aware autotuner that selects the best dataflow for each layer. As a result, TorchSparse achieves impressive speedups, with $1.8\text{--}3.3\times$ faster inference and $1.2\text{--}3.7\times$ faster training compared to state-of-the-art MinkowskiEngine, SpConv v1/v2, and TorchSparse v1 on seven real-world perception workloads.

5. **Hardware: PointAcc**

In the previous section, we delved into the implementation of sparse convolution in point cloud deep learning using TorchSparse on GPUs, which offers an efficient solution for processing highly sparse point cloud inputs. However, it does not cover other point cloud deep learning primitives, such as point-based methods. Furthermore, while specialised point cloud inference libraries can certainly improve the speed and efficiency of point cloud processing on general-purpose GPUs, they are still constrained by the architecture of the GPU. Processing large amounts of irregularly structured data and performing complex calculations on them can lead to significant memory bottlenecks on traditional GPU architectures and high energy consumption due to heavy data movement between on-chip and off-chip memories. Moreover, as LiDARs become cheaper and more widely deployed, it becomes increasingly challenging to process 3D point clouds on edge devices with limited computational power compared to powerful GPUs. Some applications, such as autonomous driving and augmented reality, require high accuracy and low latency for real-time interactions with humans and low energy consumption

for longer battery life. To address these challenges, a specialised hardware accelerator for point cloud deep learning is a natural solution.

Numerous works have explored both FPGA⁸² and ASIC^{83–87} accelerators for efficient DNN inference. However, these specialised NN accelerators are ill-suited for point cloud deep learning for two primary reasons:

Bottleneck I — Mapping operations: The mapping operations are essential for point cloud deep learning. However, as shown in Figure 3, the sparse-conv-based networks spend up to 15% of total runtime on mapping operations., while the point-based networks can spend more than 50% of total runtime on general-purpose hardware platforms. Unfortunately, existing specialised NN accelerators lack support for these mapping operations, and accelerating GEMM only will worsen the performance. Consider TPUs⁸⁸ as an example, which are tailored for dense matrix multiplications. To perform mapping operations, all relevant data must first be moved to host memory, the CPU used to calculate the mapping operations, and the features gathered accordingly before sending the contiguous matrices back to the TPU. This round trip between heterogeneous memory can be extremely time-consuming, with data movement time often taking up 60% to 90% of total runtime.

Bottleneck II — Sparse computation: Point cloud deep learning processes highly sparse data differently. EIE,⁴⁷ Cambricon-X,⁸⁹ Cnvlutin,⁹⁰ and SCNN⁹¹ exploited the sparsity in DNNs and speeded up the inference by skipping unstructured zeros. However, these conventional sparse accelerators do not support modern point cloud networks. In traditional sparse CNNs, the non-zero inputs are multiplied with every non-zero weight. Thus, the non-zeros will dilate in the output, exploited in the prior sparse NN accelerators. However, in point cloud NNs, each non-zero input point is not always multiplied with all non-zero weights: the relationship among input points, weights, and output points is determined by mapping operations.

5.1. Architecture

We present the PointAcc architecture design, as shown in Figure 26. It consists of three parts: mapping unit, memory management unit

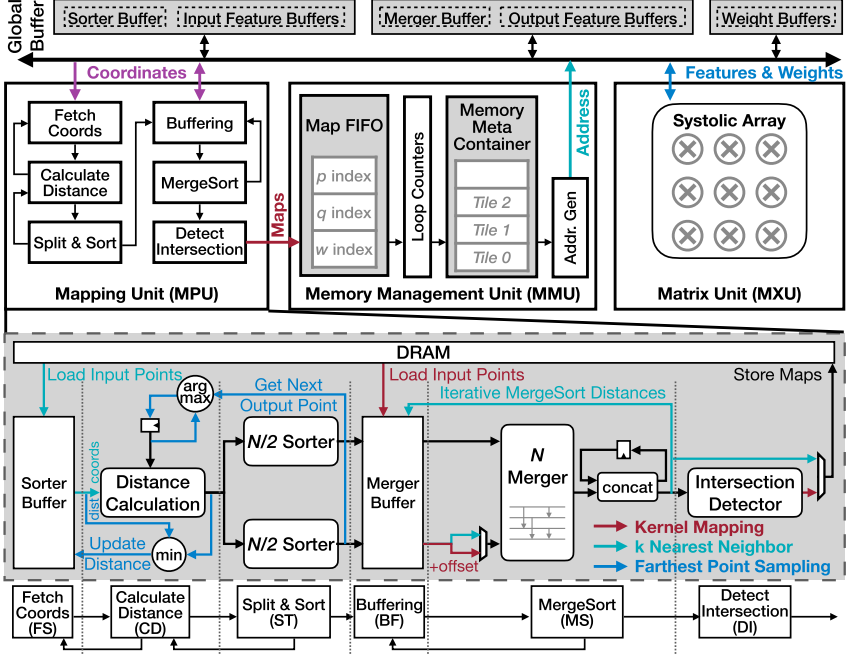


Fig. 26. Overview of PointAcc Architecture.

(MMU), and matrix unit. The MMU bridges the other two units by preparing the matrix unit data based on the mapping unit output. By configuring the data flow in each unit, PointAcc flexibly supports various point cloud networks efficiently.

5.1.1. Mapping unit

Conventional point cloud accelerators^{92–97} only focus on k -nearest neighbours which is only one of the mapping operations in the domain. On the contrary, our Mapping Unit (MPU) targets all diverse mapping operations, including kernel mapping, k -nearest neighbours, ball query, and farthest point sampling. Instead of designing specialised modules for each possible operation, we propose to unify these diverse mapping operations into one computation paradigm and convert them to point-cloud-agnostic operations, which can be further used to design one versatile specialised architecture.

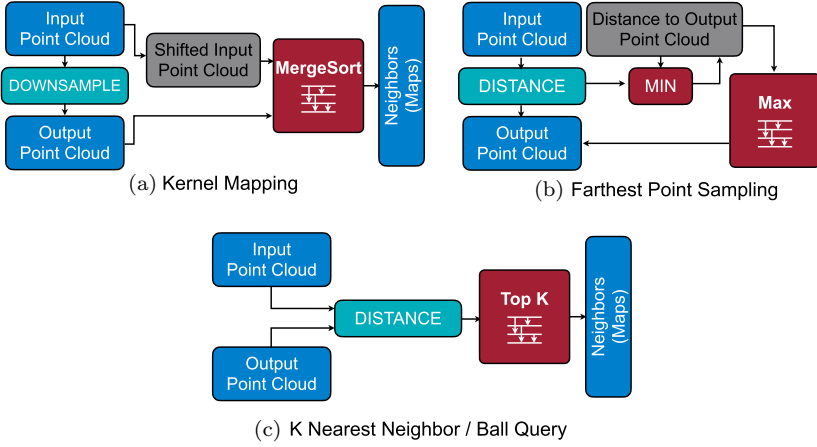


Fig. 27. Mapping operations introduced by point cloud are unified to a similar paradigm with ranking-based compute kernel.

The ultimate goal of mapping operations is to generate maps in the form of (input point index, output point index, weight index) tuple for further computation (e.g., convolution). We observe that no matter which algorithm is used, these maps are always constructed based on the *comparison* among distances. Thus we offer to convert these comparisons into ranking operations (Figure 27) and process them in parallel for different points in the point cloud (point-level parallelism).

Farthest point sampling obtains the point in the input point cloud with the largest distance to the current output point cloud. Then, we convert it to a **Max** operation on distances (Figure 27(b)).

K-nearest neighbour searches k points in the input point cloud with the smallest distances to the given output point, and **ball query** further requires these distances to be smaller than a pre-defined value. They can be implemented with **TopK** operation (Figure 27(c)).

Kernel mapping can be regarded as finding the input points with the exact distance in the specific direction to the output points. For example, for the maps associating with weight $w_{-1,0}$, the input points are all right above the output points with a distance of 1. Hence, the comparison is **Equal** operation on distances, done via the hash table in the state-of-the-art implementation.³⁴ The hash table records the input point cloud coordinates, and each output point will query its possible neighbour from the table. A query hit indicates a map is

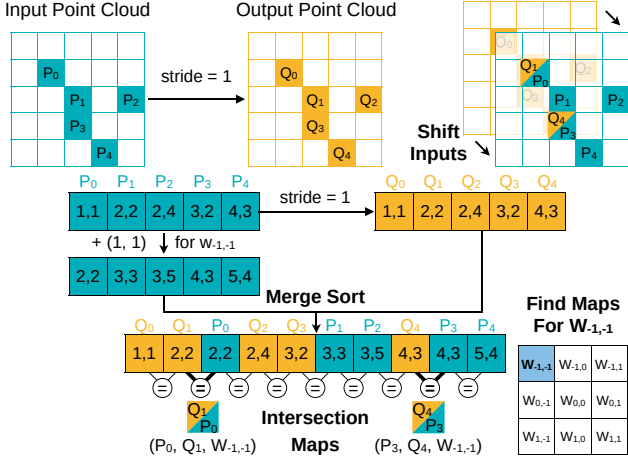


Fig. 28. Mergesort-based kernel mapping implementation.

founded. However, such a hash-table-based solution is inefficient in terms of circuit specialisation. On one hand, we cannot afford a large on-chip SRAM for the hash table, which could be as large as 160 MB considering the input point cloud size and the load factor. On the other hand, we cannot parallelise it efficiently, since a parallelised hash table requires random parallelised reads to the SRAM, typically requiring an N -by- N crossbar with space complexity of $O(N^2)$.

Instead, as shown in Figure 28, we model the kernel mapping as finding the *intersection* on point coordinates between output point cloud O and shifted input point cloud $I' = \{p - \delta | p \in I\}$. The parallelised `Equal` operation between two point clouds can be further converted to `MergeSort` of two point clouds (Figure 27(c)). The input point cloud I is first applied with the offset $-\delta$ and then merge-sorted with the output point cloud O , an optionally downsampled version of the input cloud. The intersection can be easily found by examining the adjacent elements, keeping those with the same coordinate and removing others. Experiments show that our mergesort-based solution could provide a $1.4\times$ speedup while saving up to $14\times$ area compared to the hash-table-based design with the same parallelism.

MergeSort of arbitrary length: An N -merger can only process a fixed-length array of N elements, typically less than 64, which is far away from the size of point clouds (10^3 – 10^5 points). As shown in

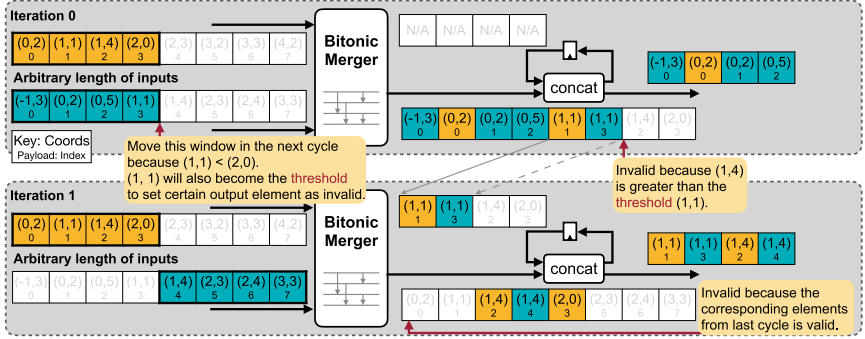


Fig. 29. Mapping unit handles MergeSort of arbitrary length of inputs with forwarding loop.

Figure 29, after the merger, we add a forwarding loop to handle such a large scale. In each cycle, the merger takes in two arrays of $N/2$ elements but only consumes one array. Only the first $N/2$ output elements are considered valid, and the rest of the $N/2$ elements are buffered for the next cycle.

Sort/TopK of arbitrary length: A pass of the first five stages in MPU without forwarding works as a typical bitonic sorter. However, similar to the challenge in MergeSort mentioned earlier, to handle the *arbitrary* length of inputs, we perform a classical merge sort algorithm by forwarding the outputs of stage MS to stage FM and iteratively merge-sorting two sorted subarrays, as shown in Figure 30(a). By truncating the intermediate subarrays to the length of k , MPU can directly support TopK with the same dataflow as running Sort operation, as illustrated in Figure 30(b).

Since the k of TopK in point cloud models is usually very small (e.g., 16/32/64) compared to the size of the input (e.g., 8192), the overhead of reusing sorter would be negligible. Experiments show that, on average, our design is $1.18\times$ faster than the quick-selection-based top-k engine proposed in SpAtten⁸⁷ with the same parallelism.

5.1.2. Memory management unit

As pointed out in Bottleneck II, explicit gather and scatter hinder the matrix computation. Therefore, we leverage the MMU to bridge the gap between computational resource needs and memory bound.

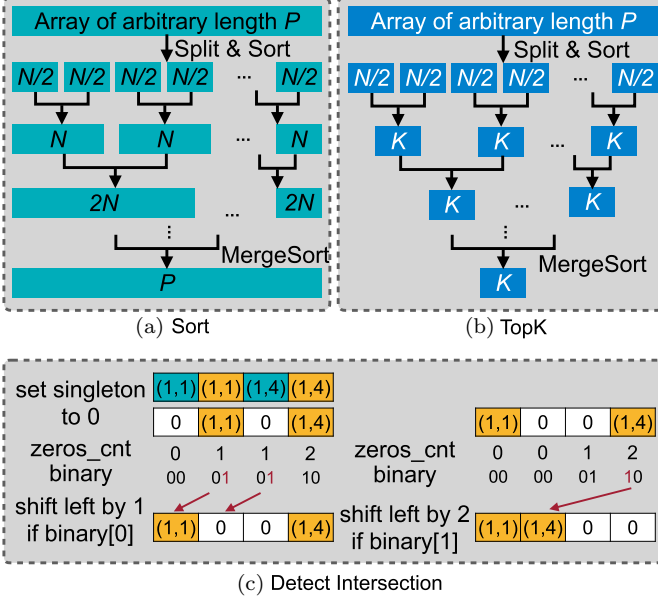


Fig. 30. (a) Mapping unit supports Sort of arbitrary length of inputs by iteratively MergeSort split and sorted subarrays in a tree structure; (b) mapping unit flexibly realises TopK on arbitrary length of inputs by truncating the intermediate merge-sorted subarrays to the length of k ; (c) the intersection detector taking N elements has $\log N$ stages.

5.1.3. Data orchestration

In point cloud networks, #layers of sparse computation (point cloud convolution) and dense computation (FC, convolution with kernel size of 1) is comparable. Among traditional memory idioms, workload-agnostic cache design is favoured for sparse computation, while workload-controlled scratchpad is popular for dense computation.⁹⁸ To better handle both types of computation, MMU hybridises two memory designs. MMU decouples the memory request initiator and response receiver (Figure 31(a)) and manages the on-chip buffers in the granularity of “tile” (Figure 31(b)). A memory tile contains the minimum memory space required for a computation tile of tiled matrix multiplication. The memory tile information, such as address range and starting offset, is exposed in the Memory Meta Info Register (MIR). Therefore, MMU can perform explicit and precise control over the memory space by manipulating the placement and

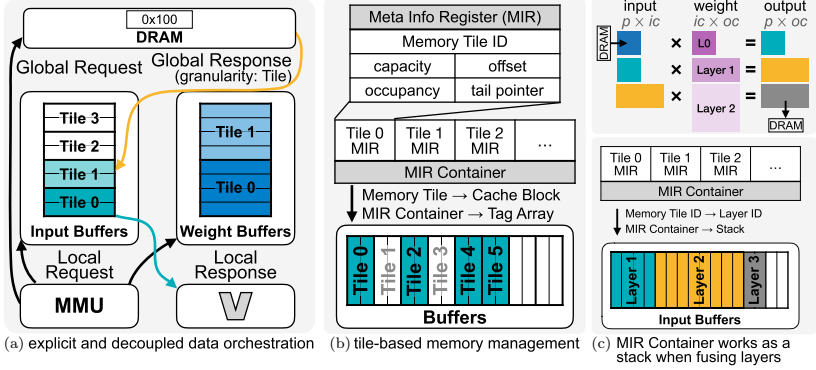


Fig. 31. MMU design overview. (a) MMU exploits explicit and decoupled data orchestration.⁹⁸ (b) MMU manages the on-chip memory in the granularity of “tile” (i.e., block). Its meta information (e.g., allocated capacity and starting address) is recorded in the Memory Tile Meta Info Register (MIR). (c) MIR Container is configured as a stack where different MIRs represent the data of different layers when PointAcc temporally fuses the consecutive layers.

replacement of MIRs in the MIR Container (Figure 31(b)): MMU will treat the MIR Container as a Tag Array when the cache is needed for sparse computation and as a FIFO or Stack when scratchpad is needed for dense computation (Figure 31(c)).

5.1.4. Data flow

By reordering the computation loops of matrix multiplication, one can easily realise different data flow to improve the data reuse for inputs/weights/outputs. Since $\#points$ ($10^3 \sim 10^5$) is much larger than the $\#channels$ ($10 \sim 10^3$), even by orders of magnitude, we opt *weight stationery* data flow for inner computation loop nests to reduce on-chip memory accesses for weights. MMU will not increment input/output channels or neighbour dimensions until it traverses all points in the on-chip buffers. Furthermore, we opt *output stationery* dataflow for outer loop nests to *eliminate* the off-chip scatter of partial sums. MMU will not swap out the output features before it traverses all neighbours and all input channels.

5.1.5. MMU for sparse computation

MatMul computation in point cloud convolution is sparse and guided by the maps generated by mapping unit. Thus, in addition to

computation loop counters, the address generator will also require information from the maps, as shown in Figure 26 (top).

Optimise the computation flow: Computation flow affects memory behaviour. The state-of-the-art GPU implementation will first gather all required input feature vectors, concatenate them as a contiguous matrix, and then apply MM to calculate partial sums, referred to as the Gather-GEMM-Scatter flow. Contrarily, PointAcc calculates Matrix-Vector multiplication immediately after fetching the input features, referred to as *Fetch-on-Demand flow*. As shown in Figure 32, the Fetch-on-Demand flow will save the DRAM access for input features by at least $3\times$ by reducing the repetitive reads for gather and eliminating the writes after gather and reads for MM in the Gather-GEMM-Scatter flow.

Configure input buffers as cache: In order to further reduce the repetitive reads of the same input point features in the Fetch-on-Demand flow, MMU configures the Input Buffers as a direct-mapped *cache* by reusing the MIR Container as a shared Tag Array recording the feature information.

Different from the traditional cache, contiguous entries in the input buffers are treated as a “cache block”, and thus the block size is software-controllable. The tag comprises the point and channel indexes of the *first* input point inside the cache block. A cache hit

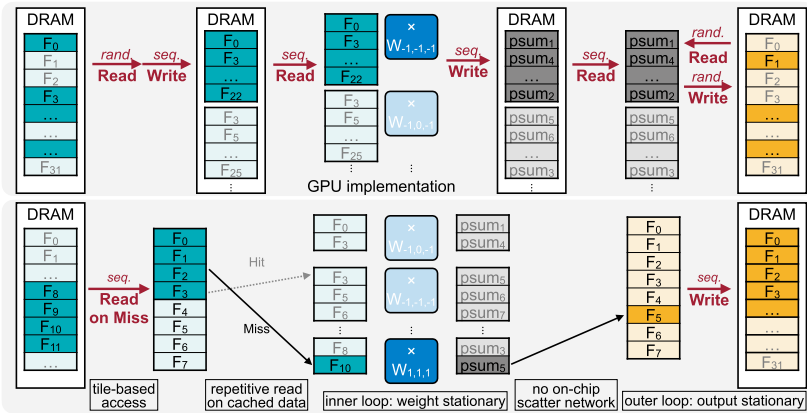


Fig. 32. To handle the sparsity of point cloud convolution, MMU configures the input buffers as “cache” *on demand* and streams the matrix computation without off-chip scattering.

occurs if both the input point index and channel index of requested point features lie in the cache block size. Otherwise, it is a cache miss, and MMU will load the data block (i.e., a memory tile) containing the requested point features from DRAM.

5.1.6. MMU for dense computation

The matrix computation is dense for FC layers and convolutions with a kernel size of 1, and the input features are already contiguous in the DRAM. Thus, MMU only queries the MIR Container at the very beginning of each computation loop tile, reading out the MIR of the current tile and trying to allocate the memory and prefetch the data for the next computation tile. A memory tile will only be evicted (i.e., release) when it conflicts with the prefetching tile. Hence, data will stay on-chip as long as possible and be reused as much as possible.

Layer fusion: PointAcc is able to fuse the computation of consecutive FCs in the point cloud convolution to further eliminate the DRAM accesses of the intermediate features. The conventional layer fusion⁹⁹ spatially pipelines the computation and thus fixes the number of fused layers and requires matching throughput in between. However, the number of consecutive layers varies among different models and even among different blocks in the same model. MMU thus *temporally* fuses these computations by simply configuring the MIR Container works as *Stack* and identifying the MIR by the layer index. The matrix unit always works with the top entry of the MIR Container. When switching back to the previous unfinished layer, MIR Container will pop the top entry.

For FCs in the point cloud models, the point dimension can be regarded as the batch size dimension in traditional CNN. Thus, MMU can simplify the layer fusion logic by tiling the point dimension without halo elements between tiles. The number of fused layers and their tilings are determined during the compilation to avoid memory overflow. For each set of consecutive FCs, we will try to fuse all unprocessed FCs. If the estimated memory of required intermediate data overflows for all possible tilings, we will discard the last layer and try to fuse the remaining ones. Such a process is repeated until all layers are processed.

Example: Figure 33 shows an example of PointAcc fusing 3 consecutive FC layers:

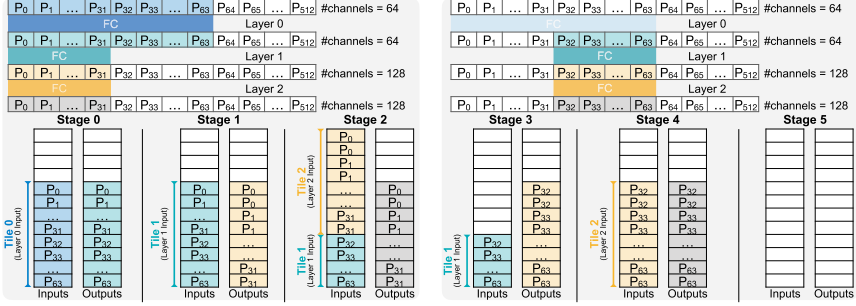


Fig. 33. PointAcc temporally fuses the consecutive FC layers: the data of the current layer being computed are always at the top of the stack (e.g., Layer 2 in Stage 2). A memory tile is released if all the data are used (e.g., Layer 0 in Stage 1). If there are unused data for the previous layers, MIR only releases the used part (e.g., Layer 1 tile in Stage 2 is halved compared to that in Stage 1).

- **Stage 0:** MMU loads features of p_0 to p_{63} of Layer 0 from DRAM.
- **Stage 1:** The computation in Stage 0 used up all loaded data, and thus Layer 0 tile is released. Switching to Layer 1, MMU pushes features of Layer 1 from Output Buffers.
- **Stage 2:** Since Layer 1 computation only uses half of the input features (p_0 to p_{31}), the Layer 1 tile capacity is halved. Switching to Layer 2, MMU pushes features of Layer 2 similar to Stage 1.
- **Stage 3:** Since Layer 2 is the last fused layer, we switch back to the previous layer (Layer 1) after MMU pops Layer 2 data. Since Layer 1 tile capacity is non-zero, we continue to compute Layer 1 for the rest features (p_{32} to p_{63}).
- **Stage 4:** Layer 1 tile is released since all data are used. Switching to Layer 2, MMU pushes features of Layer 2 similar to Stage 2.
- **Stage 5:** After finishing Layer 2 and switching back to Layer 1, we find that Layer 1 tile capacity is zero. Thus we continue switching back to Layer 0. Since Layer 0 tile capacity is also zero, we will update outer loop counters and then continue to work on Layer 0 for the next tile p_{64} to p_{127} .

5.1.7. Matrix unit

Matrix Unit (MXU) adopts the classic systolic array as the computing core since it has been proven to be efficient and effective for dense matrix multiplication.

In order to completely eliminate the on-chip scatter network, MXU parallelises the computation in input channels (ic) and output channels (oc) dimensions: each row of PEs computes in ic, and each column of PEs computes in oc dimension independently. Thus, MXU only accesses features of one output point in one cycle, no more spatially scattering different points at one time and hence no need for the scatter circuit.

5.2. Evaluation

5.2.1. Evaluation setup

Benchmarks: We pick eight diverse point cloud network benchmarks across various application domains: object classification, part segmentation, detection, and semantic segmentation. Table 10 shows that the networks include classical and state-of-the-art ones. The selected datasets also contain various sizes and modalities for input point clouds, from daily objects to indoor and spacious outdoor scenes. Such extensive benchmarks allow us to evaluate PointAcc thoroughly.

Hardware implementation: We implement PointAcc with Verilog and verify the design through RTL simulations. We synthesise PointAcc with Cadence Genus under TSMC 40 nm technology. Power of PointAcc is simulated with fully annotated switching activity generated with the selected benchmarks. We develop a cycle-accurate simulator to model the exact behaviour of the hardware and calculate

Table 10. Evaluation benchmarks.

Application	Dataset	Scene	Model	Notation
Classification	ModelNet40	Object	PointNet PointNet++ (SSG)	PointNet PointNet++(c)
Part Segmentation	ShapeNet	Object	PointNet++ (MSG) DGCNN	PointNet++(ps) DGCNN
Detection	KITTI	Outdoor	F-PointNet++	F-PointNet++
Segmentation	S3DIS	Indoor	PointNet++ (SSG) MinkUNet	PointNet++(s) MinkNet(i)
	SemanticKITTI	Outdoor	MinkUNet	MinkNet(o)

Table 11. Evaluated ASIC platforms.

Chip	Mesorasi	PointAcc	PointAccsmall
Cores	$16 \times 16 = 256$	$64 \times 64 = 4096$	$16 \times 16 = 256$
SRAM (KB)	1624	776	274
Area (mm ²)	–	15.7	3.9
Frequency	1 GHz	1 GHz	1 GHz
DRAM	LPDDR3-1600	HBM 2	DDR4-2133
Bandwidth	12.8 GB/s	256 GB/s	17 GB/s
Technology	16 nm	40 nm	40 nm
Peak performance	512 GOPS	8 TOPS	512 GOPS

the cycle counts and read/write of on-chip SRAMs. The simulator is also verified against the Verilog implementation. We integrate the simulator with Ramulator¹⁰⁰ to model the DRAM behaviours. We obtain SRAM energy with CACTI¹⁰¹ and DRAM energy using Ramulator-dumped DRAM commands trace.

Baselines: We adopt three hardware platforms as evaluation baselines: server-level products, edge devices, and a specialised ASIC design. For server-level products, we compare full-size PointAcc against Xeon[®] 6130 CPU, RTX 2080Ti GPU, and TPU-v3. For edge devices and specialised ASIC, we compare the edge configuration (PointAcc.Edge) against Jetson Xavier NX, Jetson Nano, Raspberry Pi 4B, and Mesorasi⁹⁷ with NPU of 16×16 systolic array.

ASIC design parameters are compared in Table 11.

We implement point cloud networks with PyTorch (matrix operations with Intel MKLDNN/cuDNN and mapping operations with C++ OpenMP/custom CUDA kernels). Our implementation achieves $2.7 \times$ speedup over the state-of-the-art implementation MinkowskiEngine.⁶

5.2.2. Evaluation results

Speedup and energy savings: Figure 34 presents the performance and energy benefits of PointAcc in comparison with GPU, TPU, and CPU products. On average, PointAcc offers $3.7 \times$, $53 \times$, $90 \times$ speedup and $22 \times$, $210 \times$, $176 \times$ energy savings, respectively. Figure 35 shows the speedup and energy savings of PointAcc.Edge

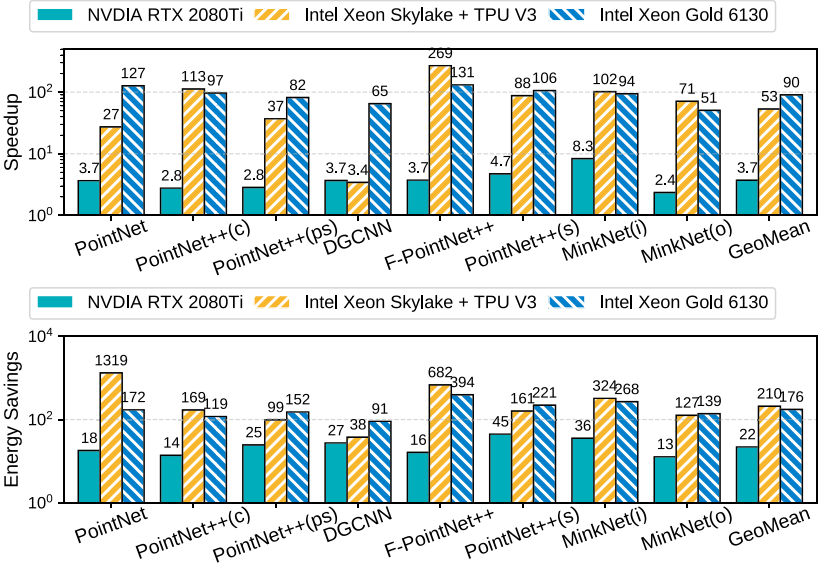


Fig. 34. Performance gain over the server products: PointAcc is $3.7\times$ faster than RTX 2080Ti on average.

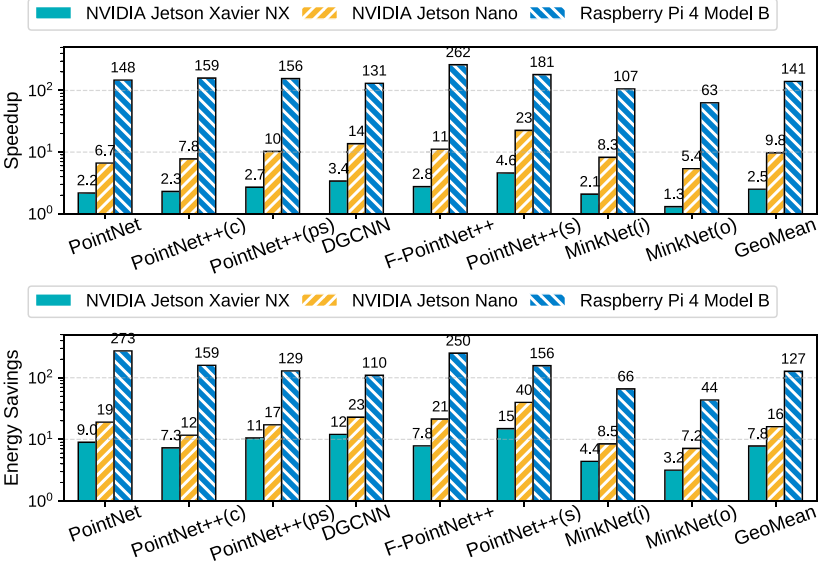


Fig. 35. Performance gain over the edge devices: PointAcc.Edge is $2.5\times$ faster than Jetson NX on average.

over Jetson Xavier NX, Jetson Nano, and Raspberry Pi devices. On average, PointAcc.Edge achieves $2.5\times$, $9.8\times$, $141\times$ speedup and $7.8\times$, $16\times$, $127\times$ energy savings, respectively. The improvements are *consistent* on different benchmarks. For TPU V3, the considerable gain mainly comes from supporting mapping operations with shared compute kernel design, thus significantly reducing the data movements to/from the host CPU.

Comparison with Mesorasi architecture: Figure 36 shows the runtime speedup and energy savings of PointAcc.Edge over Mesorasi designs on PointNet++-based benchmarks. PointAcc.Edge achieves $1.3\times$ speedup and $11\times$ energy savings over Mesorasi hardware design on average. Unlike our design, Mesorasi is limited since it does not support independent weights for different neighbours. It is crucial for many point cloud networks,^{6,18,34,60} especially for SparseConv-based models, which not only improve the accuracy but also are capable of processing large-scale point clouds. In Figure 37, we compare PointAcc.Edge with Mesorasi for the same segmentation task on the indoor scene dataset S3DIS. We scale down the SparseConv-based state-of-the-art model MinkUNet to a shallower, narrower version, denoted as Mini-MinkUNet. Co-designed with the neural network, PointAcc.Edge delivers over $100\times$ speedup and improves the mean Intersection-over-Unit (mIoU) accuracy by 9.1%.

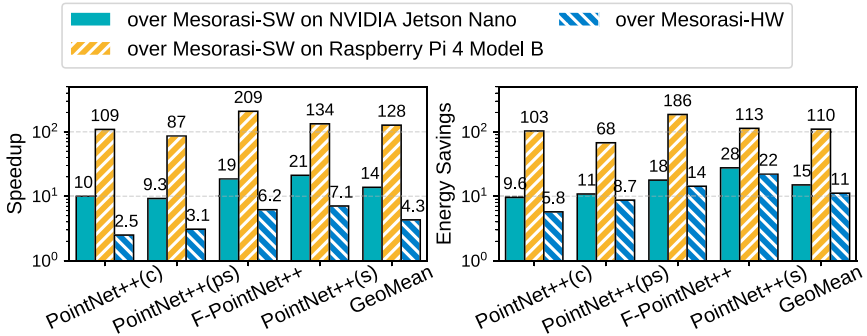


Fig. 36. Speedup and energy savings of PointAcc.Edge over Mesorasi. Mesorasi-SW runs Mesorasi networks without specialised architectural support; Mesorasi-HW executes Mesorasi networks with dedicated aggregation unit AU and neural processing unit NPU.

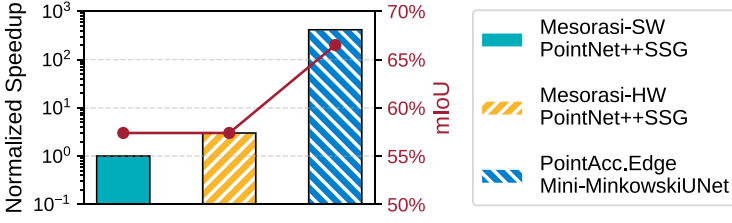


Fig. 37. Mesorasi does not support independent weights for different neighbours, which is crucial in some variants of PointNet++-based blocks¹⁵ or SparseConv-based blocks.⁶ When running the same segmentation task on the S3DIS dataset, PointAcc.Edge can execute networks with 10% higher accuracy and 100× lower latency.

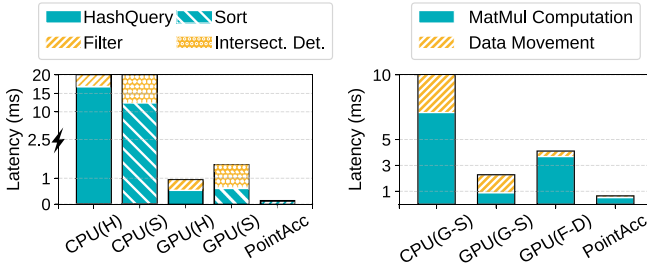


Fig. 38. Latency breakdown of SparseConv-based blocks in operation level. Left: Kernel Mapping. Mergesort-based solution (*S*) runs slower than a hash-table-based algorithm (*H*) on CPU/GPU but 1.4× faster with 14× smaller area after circuit specialisation (Section 5.1.1). Right: Convolution. Compared to the Gather-GEMM-Scatter flow (*G-S*), the reduction of data movement in Fetch-On-Demand flow (*F-D*) is dwarfed by the MV computation cost on GPU but benefits PointAcc (Section 5.1.5).

5.2.3. Source of performance gain

Ranking-based conversion of mapping operations: Figure 38 (left) breaks down the latency of the mapping operation (kernel mapping here) of the 1st downsampling SparseConv-based block on SemanticKITTI. The mergesort-based algorithm (Figure 28) even worsens the performance on CPU/GPU. But compared to CPU/GPU, PointAcc is over 10× faster. Our design provides enough parallelism for comparison-intensive merge sort and eliminates the repetitive access on intermediate results by spatially pipelining the stages of merge sort and intersection detection. On CPU/GPU, detecting intersection costs almost 2× runtime than filtering the query miss on the hash table because the length of inputs of

intersection detection is doubled due to the merge of input/output point clouds. Querying the hash table costs almost comparable time with merge sort on GPU. It is because the hash-table-based algorithm only needs one pass on the input, but most stages of bitonic merge require a scan of the input from GPU global memory.

Fetch-on-demand computation flow: Figure 38 (right) breaks down the latency of the matrix multiplication (convolution here) of the 1st layer of MinkUNet on SemanticKITTI. Fetch-On-Demand flow saves the memory footprint by $3\times$ but decomposes the matrix-matrix multiplication into fragmentary matrix-vector multiplications and thus significantly increases the overhead due to low utilisation of GPU. However, such overhead is removed in PointAcc because of the computation power of the systolic array. Therefore, PointAcc spends almost comparable time on the whole operation against the MatMul computation part only in the Gather-GEMM-Scatter flow.

Configurable caching: Figure 39 demonstrates the distribution (i.e., probability density) of the DRAM access size per layer in MinkUNet on the S3DIS and SemanticKITTI datasets. A wider region indicates a higher frequency of the given data size. The shape of the distribution is nearly the same with/without caching, which indicates that the caching works *consistently* on different layers and datasets. On average, the configurable cache reduces the layer DRAM access by $3.5\times$ to $6.3\times$, where each point feature is only fetched nearly once on average.

Temporal layer fusion: Figure 40 shows the reduction ratio of DRAM access when running PointNet++-based networks with Fusion Mode. Compared to running networks layer by layer independently, our simplified layer fusion logic helps cut the DRAM

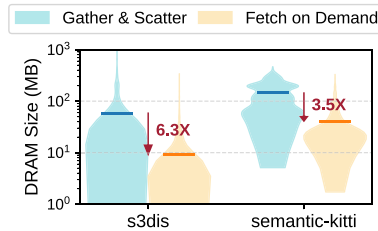


Fig. 39. MinkUNet layer DRAM access size distribution with and without caching. The bar denotes the average DRAM access size per layer.

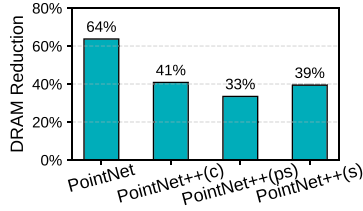
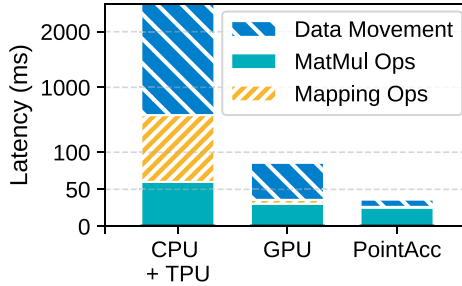
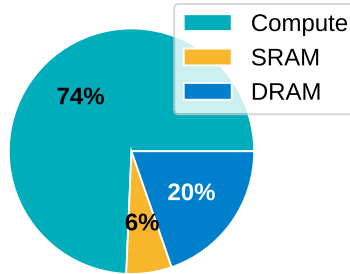


Fig. 40. PointNet++ DRAM access size reduction of fusion mode memory management compared to running layer by layer independently.



(a) Latency breakdown



(b) Energy breakdown

Fig. 41. Performance breakdown of PointAcc on MinkNet(o) benchmark: PointAcc reduces the latency and energy cost of data movement.

access from 33% to 64%. Since there are no downsampling layers in PointNet, we can fuse more layers than other PointNet++-based networks, leading to $1.5\times$ to $2\times$ more DRAM reduction.

Overall performance breakdown: Figure 41(b) breaks down the latency and energy consumption of PointAcc running MinkUNet on the SemanticKITTI dataset. The support of mapping operations eliminates the extra cost of communication between co-processors, as

in the CPU+TPU case. The specialised data orchestration in MMU helps reduce the DRAM access and makes the data movement overlap the matrix multiplication. Therefore, the MatMul operations dominate the overall latency. Moreover, the computation covers 69% of total energy while DRAM access costs 23% of total energy, which differs from the observation in Chen *et al.*⁸⁴ where DRAM accesses dominate the energy consumption of FC layers.

5.3. Summary

This section presents a specialised point cloud deep learning accelerator PointAcc. This new advancement could help bring powerful point cloud AI to real-world applications, from augmented reality on iPhones to autonomous driving of intelligent vehicles, supporting real-time interactions with humans. PointAcc supports the newly introduced mapping operations by unifying and mapping them onto a shared ranking-based compute kernel. At the same time, PointAcc addresses the massive memory footprint problem due to the sparsity of point clouds by streaming the sparse computation with on-demand caching and temporally fusing the consecutive layers of dense computation. Extensive evaluation experiments show that PointAcc delivered significant speedup and energy reduction over CPU, GPU, and TPU.

6. Conclusion

This chapter presents efficient algorithms, systems, and hardware for 3D point cloud deep learning, addressing the computation and sparsity challenges associated with 3D point cloud data. Novel 3D building blocks and network architecture (PVCNN and SPV-NAS) are explored from an algorithmic perspective to handle point cloud data while minimising sparse and irregular overheads. A high-performance library (TorchSparse) is developed at the system level to accelerate sparse and irregular workloads on general-purpose hardware. Furthermore, a specialised hardware accelerator (PointAcc) is designed to support various types of point cloud deep learning primitives and alleviate memory bottlenecks in sparse point cloud computing.

References

1. C. R. Qi, H. Su, M. Niessner, A. Dai, M. Yan, and L. J. Guibas. Volumetric and Multi-View CNNs for Object Classification on 3D Data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).
2. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2017).
3. A. H. Lang, S. Vora, H. Caesar, L. Zhou, and J. Yang. PointPillars: Fast Encoders for Object Detection from Point Clouds. In *CVPR* (2019).
4. Z. Liu, H. Tang, A. Amini, X. Yang, H. Mao, D. Rus, and S. Han. BEVFusion: Multi-Task Multi-Sensor Fusion with Unified Bird’s-Eye View Representation. In *IEEE International Conference on Robotics and Automation (ICRA)* (2023).
5. H. Caesar, V. Bankiti, A. H. Lang, S. Vora, V. E. Liong, Q. Xu, A. Krishnan, Y. Pan, G. Baldan, and O. Beijbom. nuScenes: A Multimodal Dataset for Autonomous Driving. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
6. C. Choy, J. Gwak, and S. Savarese. 4D Spatio-Temporal ConvNets: Minkowski Convolutional Neural Networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
7. Z. Wu, S. Song, A. Khosla, F. Yu, L. Zhang, X. Tang, and J. Xiao. 3D ShapeNets: A Deep Representation for Volumetric Shapes. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2015).
8. A. X. Chang, T. Funkhouser, L. Guibas, P. Hanrahan, Q. Huang, Z. Li, S. Savarese, M. Savva, S. Song, H. Su, J. Xiao, L. Yi, and F. Yu. ShapeNet: An Information-Rich 3D Model Repository, *arXiv:1512.03012* (2015).
9. D. Maturana and S. Scherer. VoxNet: A 3D Convolutional Neural Network for Real-Time Object Recognition. In *Proceedings of IEEE/RSJ International Conference of Intelligent Robotics System* (2015).
10. Z. Wang and F. Lu, Voxsegnet: Volumetric cnns for semantic part segmentation of 3D shapes, *IEEE Transactions on Visualization and Computer Graphics* **26**(9), 2919–2930 (2020).
11. Y. Zhou and O. Tuzel. VoxelNet: End-to-End Learning for Point Cloud Based 3D Object Detection. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).

12. P.-S. Wang, Y. Liu, Y.-X. Guo, C.-Y. Sun, and X. Tong, O-cnn: octree-based convolutional neural networks for 3D shape analysis, *ACM Transactions on Graphics* **36**(4), 72:1–72:11 (2017).
13. M. Tatarchenko, J. Park, V. Koltun, and Q.-Y. Zhou. Tangent Convolutions for Dense Prediction in 3D. In *Proc. IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
14. T. Le and Y. Duan. PointGrid: A Deep Network for 3D Shape Understanding. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
15. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep Hierarchical Feature Learning on Point Sets in a Metric Space. In *Advances in Neural Information Processing Systems (NeurIPS)* (2017).
16. R. Klokov and V. S. Lempitsky. Escape from Cells: Deep Kd-Networks for the Recognition of 3D Point Cloud Models. In *IEEE/CVF International Conference on Computer Vision (ICCV)* (2017).
17. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon, Dynamic graph cnn for learning on point clouds, *ACM Trans. Graph* **38**(5), 146:1–146:12 (2019).
18. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on $\mathcal{X}$ -Transformed Points. In *Advances in Neural Information Processing Systems (NeurIPS)* (2018).
19. S. Lan, R. Yu, G. Yu, and L. S. Davis. Modeling Local Geometric Structure of 3D Point Clouds using Geo-CNN. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
20. Y. Xu, T. Fan, M. Xu, L. Zeng, and Y. Qiao. SpiderCNN: Deep Learning on Point Sets with Parameterized Convolutional Filters. In *European Conference on Computer Vision (ECCV)* (2018).
21. H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz. SPLATNet: Sparse Lattice Networks for Point Cloud Processing. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
22. J. Li, B. M. Chen, and G. H. Lee. SO-Net: Self-Organizing Network for Point Cloud Analysis. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
23. L. P. Tchapmi, C. B. Choy, I. Armeni, J. Gwak, and S. Savarese. SEGCloud: Semantic Segmentation of 3D Point Clouds. In *Proceedings of International Conference on 3D Vision* (2017).
24. W. Wang, R. Yu, Q. Huang, and U. Neumann. SGPNet: Similarity Group Proposal Network for 3D Point Cloud Instance Segmentation. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).

25. L. Landrieu and M. Simonovsky. Large-Scale Point Cloud Semantic Segmentation With Superpoint Graphs. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
26. S. Wang, S. Suo, W.-C. Ma, A. Pokrovsky, and R. Urtasun. Deep Parametric Continuous Convolutional Neural Networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
27. B. Graham, M. Engelcke, and L. van der Maaten. 3D Semantic Segmentation With Submanifold Sparse Convolutional Networks. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
28. Q. Huang, W. Wang, and U. Neumann. Recurrent Slice Networks for 3D Segmentation on Point Clouds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
29. C. R. Qi, W. Liu, C. Wu, H. Su, and L. J. Guibas. Frustum Point-Nets for 3D Object Detection from RGB-D Data. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2018).
30. S. Shi, X. Wang, and H. Li. PointRCNN: 3D Object Proposal Generation and Detection From Point Cloud. In *CVPR* (2019).
31. Y. Yan, Y. Mao, and B. Li, Second: Sparsely embedded convolutional detection., *Sensors*. **18**(10), 3337 (2018).
32. T. Yin, X. Zhou, and P. Krähenbühl. Center-based 3D Object Detection and Tracking. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021).
33. P. Sun, H. Kretschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, V. Vasudevan, W. Han, J. Ngiam, H. Zhao, A. Timofeev, S. Ettinger, M. Krivokon, A. Gao, A. Joshi, Y. Zhang, J. Shlens, Z. Chen, and D. Anguelov. Scalability in Perception for Autonomous Driving: Waymo Open Dataset. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
34. H. Tang, Z. Liu, S. Zhao, Y. Lin, J. Lin, H. Wang, and S. Han. Searching Efficient 3D Architectures with Sparse Point-Voxel Convolution. In *European Conference on Computer Vision (ECCV)* (2020).
35. Z. Liu, H. Tang, S. Zhao, K. Shao, and S. Han, Pvnas: 3D neural architecture search with point-voxel convolution., *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(11), 8552–8568 (2022).
36. R. Cheng, R. Razani, E. Taghavi, E. Li, and B. Liu. (AF)²-S3Net: Attentive Feature Fusion with Adaptive Feature Selection for Sparse Semantic Segmentation Network. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2021).

37. Z. Zhou, X. Zhao, Y. Wang, P. Wang, and H. Foroosh. CenterFormer: Center-based Transformer for 3D Object Detection. In *ECCV* (2022).
38. X. Chen, S. Shi, B. Zhu, K. C. Cheung, H. Xu, and H. Li. MPPNet: Multi-Frame Feature Intertwining with Proxy Points for 3D Temporal Object Detection. In *ECCV* (2022).
39. D. Ye, W. Chen, Z. Zhou, Y. Xie, Y. Wang, P. Wang, and H. Foroosh. LidarMultiNet: Unifying LiDAR Semantic Segmentation, 3D Object Detection, and Panoptic Segmentation in a Single Multi-task Network, *arXiv* (2022).
40. Y. Chen, J. Liu, X. Qi, X. Zhang, J. Sun, and J. Jia. Scaling up Kernels in 3D CNNs. In *CVPR* (2023).
41. R. Ge, Z. Ding, Y. Hu, W. Shao, L. Huang, K. Li, and Q. Liu. 1st Place Solutions to the Real-time 3D Detection and the Most Efficient Model of the Waymo Open Dataset Challenge 2021. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW)* (2021).
42. X. Bai, Z. Hu, X. Zhu, Q. Huang, Y. Chen, H. Fu, and C.-L. Tai. TransFusion: Robust LiDAR-Camera Fusion for 3D Object Detection with Transformers. In *CVPR* (2022).
43. R. Cheng, C. Agia, Y. Ren, X. Li, and B. Liu. S3CNet: A Sparse Semantic Scene Completion Network for LiDAR Point Clouds. In *CoRL* (2020).
44. Y. Chen, Y. Li, X. Zhang, J. Sun, and J. Jia. Focal Sparse Convolutional Networks for 3D Object Detection. In *CVPR* (2022).
45. Y. Li, Y. Chen, X. Qi, Z. Li, J. Sun, and J. Jia. Unifying Voxel-based Representation with Transformer for 3D Object Detection. In *NeurIPS* (2022).
46. Z. Liu, A. Amini, S. Zhu, S. Karaman, S. Han, and D. Rus. Efficient and Robust LiDAR-Based End-to-End Navigation. In *IEEE International Conference on Robotics and Automation (ICRA)* (2021).
47. S. Han, X. Liu, H. Mao, J. Pu, A. Pedram, M. A. Horowitz, and W. J. Dally. EIE: efficient inference engine on compressed deep neural network, *ACM SIGARCH Computer Architecture News* **44**(3), 243–254 (2016).
48. S. Ioffe and C. Szegedy. Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift. In *Proceedings of International Conference on Machine Learning* (2015).
49. A. L. Maas, A. Y. Hannun, and A. Y. Ng. Rectifier Nonlinearities Improve Neural Network Acoustic Models. In *Proceedings of International Conference on Machine Learning* (2013).

50. H. Cai, L. Zhu, and S. Han. ProxylessNAS: Direct Neural Architecture Search on Target Task and Hardware. In *Proceedings of International Conference on Machine Learning* (2019).
51. M. Tan, B. Chen, R. Pang, V. Vasudevan, M. Sandler, A. Howard, and Q. V. Le. MnasNet: Platform-Aware Neural Architecture Search for Mobile. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
52. Z. Guo, X. Zhang, H. Mu, W. Heng, Z. Liu, Y. Wei, and J. Sun. Single Path One-Shot Neural Architecture Search with Uniform Sampling. In *Proceedings of European Conference on Computer Vision (ECCV)* (2020).
53. J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall. SemanticKITTI: A Dataset for Semantic Scene Understanding of LiDAR Sequences. In *IEEE/CVF International Conference on Computer Vision (ICCV)* (2019).
54. J. Deng, W. Dong, R. Socher, L.-J. Li, K. Li, and L. Fei-Fei. ImageNet: A Large-Scale Hierarchical Image Database. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2009).
55. K. Mo, S. Zhu, A. X. Chang, L. Yi, S. Tripathi, L. J. Guibas, and H. Su. PartNet: A Large-Scale Benchmark for Fine-Grained and Hierarchical Part-Level 3D Object Understanding. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
56. I. Armeni, O. Sener, A. R. Zamir, H. Jiang, I. Brilakis, M. Fischer, and S. Savarese. 3D Semantic Parsing of Large-Scale Indoor Spaces. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2016).
57. I. Armeni, A. Sax, A. R. Zamir, and S. Savarese. Joint 2D-3D-Semantic Data for Indoor Scene Understanding, *arXiv:1702.01105* (2017).
58. A. Geiger, P. Lenz, and R. Urtasun. Are we ready for Autonomous Driving? The KITTI Vision Benchmark Suite. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2012).
59. O. Cicek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3D U-Net: Learning Dense Volumetric Segmentation from Sparse Annotation. In *Proceedings of Medical Image Computing and Computer Assisted Intervention* (2016).
60. W. Wu, Z. Qi, and L. Fuxin. PointConv: Deep Convolutional Networks on 3D Point Clouds. In *IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).

61. E.-T. Le, I. Kokkinos, and N. J. Mitra. Going Deeper with Lean Point Networks. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
62. G. Li, M. Müller, G. Qian, I. C. Delgadillo, A. Abualshour, A. Thabet, and B. Ghanem. DeepGCNs: Can GCNs Go as Deep as CNNs? In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2019).
63. G. Li, G. Qian, I. C. Delgadillo, M. Müller, A. Thabet, and B. Ghanem. SGAS: Sequential Greedy Architecture Search. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
64. G. Li, M. Xu, S. Giancola, A. Thabet, and B. Ghanem, LC-NAS: Latency Constrained Neural Architecture Search for Point Cloud Networks, *arXiv:2008.10309* (2020).
65. Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham. RandLA-Net: Efficient Semantic Segmentation of Large-Scale Point Clouds. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
66. H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas. KPConv: Flexible and Deformable Convolution for Point Clouds. In *IEEE/CVF International Conference on Computer Vision (ICCV)* (2019).
67. A. Geiger, P. Lenz, C. Stiller, and R. Urtasun, Vision meets robotics: The KITTI dataset, *The International Journal of Robotics Research* **32**(11), 1231–1237 (2013).
68. C. Xu, B. Wu, Z. Wang, W. Zhan, P. Vajda, K. Keutzer, and M. Tomizuka. SqueezeSegV3: Spatially-Adaptive Convolution for Efficient Point-Cloud Segmentation. In *Proceedings of European Conference on Computer Vision (ECCV)* (2020).
69. I. Alonso, L. Riazuelo, L. Montesano, and A. C. Murillo. 3D-MiniNet: Learning a 2D Representation from Point Clouds for Fast and Efficient 3D LIDAR Semantic Segmentation. In *Proceedings of IEEE/RSJ International Conference of Intelligent Robotics System* (2020).
70. Y. Zhang, Z. Zhou, P. David, X. Yue, Z. Xi, B. Gong, and H. Foroosh. PolarNet: An Improved Grid Representation for Online LiDAR Point Clouds Semantic Segmentation. In *Proceedings of IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (2020).
71. T. Cortinhal, G. Tzelepis, and E. E. Aksoy, SalsaNext: Fast, Uncertainty-aware Semantic Segmentation of LiDAR Point Clouds for Autonomous Driving, *arXiv:2003.03653* (2020).

72. H. Tang, Z. Liu, X. Li, Y. Lin, and S. Han. TorchSparse: Efficient Point Cloud Inference Engine. In *Fifth Conference on Machine Learning and Systems (MLSys)* (2022).
73. Y. Yan. SpConv. <https://github.com/traveller59/spconv> (2022).
74. M. Contributors. MMDetection3D: OpenMMLab next-generation platform for general 3D object detection. <https://github.com/open-mmlab/mmdetection3d> (2020).
75. K. Hong, Z. Yu, G. Dai, X. Yang, Y. Lian, Z. Liu, N. Xu, and Y. Wang. Exploiting Hardware Utilization and Adaptive Dataflow for Efficient Sparse Convolution in 3D Point Clouds. In *Sixth Conference on Machine Learning and Systems (MLSys)* (2023).
76. A. Kerr, H. Wu, M. Gupta, D. Blasig, P. Ramini, *et al.* CUTLASS: CUDA Template Library for Linear Algebra Subroutines. <https://github.com/NVIDIA/CUTLASS> (2022).
77. S. Chetlur, C. Woolley, P. Vandermersch, J. Cohen, J. Tran, B. Catanzaro, and E. Shelhamer. cuDNN: Efficient Primitives for Deep Learning. In *Advances in Neural Information Systems (NeurIPS) Workshops* (2014).
78. Y. Lin, Z. Zhang, H. Tang, H. Wang, and S. Han. PointAcc: Efficient Point Cloud Accelerator. In *54th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)* (2021).
79. Y. Yan. CUMM: CUda Matrix Multiply library. <https://github.com/FindDefinition/cumm> (2022).
80. T. Chen, T. Moreau, Z. Jiang, L. Zheng, E. Yan, H. Shen, M. Cowan, L. Wang, Y. Hu, L. Ceze, *et al.* TVM: An Automated End-to-End Optimizing Compiler for Deep Learning. In *13th USENIX Symposium on Operating Systems Design and Implementation (OSDI)* (2018).
81. S. Feng, B. Hou, H. Jin, W. Lin, J. Shao, R. Lai, Z. Ye, L. Zheng, C. H. Yu, Y. Yu, and T. Chen. TensorIR: An Abstraction for Automatic Tensorized Program Optimization. In *ASPLOS* (2023).
82. Y. Umuroglu, N. J. Fraser, G. Gambardella, M. Blott, P. Leong, M. Jahre, and K. Vissers. FINN: A framework for fast, scalable binarized neural network inference. In *Proceedings of the 2017 ACM/SIGDA International Symposium on Field-Programmable Gate Arrays*, pp. 65–74 (2017).
83. Y. Chen, T. Luo, S. Liu, S. Zhang, L. He, J. Wang, L. Li, T. Chen, Z. Xu, N. Sun, *et al.* DaDianNao: A machine-learning supercomputer. In *2014 47th Annual IEEE/ACM International Symposium on Microarchitecture*, pp. 609–622 (2014).

84. Y.-H. Chen, T. Krishna, J. S. Emer, and V. Sze, Eyeriss: An energy-efficient reconfigurable accelerator for deep convolutional neural networks, *IEEE Journal of Solid-State Circuits*. **52**(1), 127–138 (2016).
85. A. Shafiee, A. Nag, N. Muralimanohar, R. Balasubramonian, J. P. Strachan, M. Hu, R. S. Williams, and V. Srikumar, ISAAC: A convolutional neural network accelerator with in-situ analog arithmetic in crossbars, *ACM SIGARCH Computer Architecture News* **44**(3), 14–26 (2016).
86. Z. Zhang*, H. Wang*, S. Han, and W. J. Dally. Sparch: Efficient architecture for sparse matrix multiplication. In *2020 IEEE International Symposium on High Performance Computer Architecture (HPCA)*, pp. 261–274 (2020).
87. H. Wang, Z. Zhang, and S. Han. Spatten: Efficient sparse attention architecture with cascade token and head pruning. In *2021 IEEE International Symposium on High Performance Computer Architecture (HPCA)* (2021).
88. N. P. Jouppi, C. Young, N. Patil, D. Patterson, G. Agrawal, R. Bajwa, S. Bates, S. Bhatia, N. Boden, A. Borchers, R. Boyle, P. Cantin, C. Chao, C. Clark, J. Coriell, M. Daley, M. Dau, J. Dean, B. Gelb, T. V. Ghaemmaghami, R. Gottipati, W. Gulland, R. Hagmann, C. R. Ho, D. Hogberg, J. Hu, R. Hundt, D. Hurt, J. Ibarz, A. Jaffey, A. Jaworski, A. Kaplan, H. Khaitan, D. Killebrew, A. Koch, N. Kumar, S. Lacy, J. Laudon, J. Law, D. Le, C. Leary, Z. Liu, K. Lucke, A. Lundin, G. MacKean, A. Maggiore, M. Mahony, K. Miller, R. Nagarajan, R. Narayanaswami, R. Ni, K. Nix, T. Norrie, M. Omernick, N. Penukonda, A. Phelps, J. Ross, M. Ross, A. Salek, E. Samadiani, C. Severn, G. Sizikov, M. Snellman, J. Souter, D. Steinberg, A. Swing, M. Tan, G. Thorson, B. Tian, H. Toma, E. Tuttle, V. Vasudevan, R. Walter, W. Wang, E. Wilcox, and D. H. Yoon. In-datacenter performance analysis of a tensor processing unit. In *International Symposium on Computer Architecture (ISCA)* (2017).
89. S. Zhang, Z. Du, L. Zhang, H. Lan, S. Liu, L. Li, Q. Guo, T. Chen, and Y. Chen. Cambricon-X: An accelerator for sparse neural networks. In *2016 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–12 (2016).
90. J. Albericio, P. Judd, T. Hetherington, T. Aamodt, N. E. Jerger, and A. Moshovos, Cnvlutin: Ineffectual-neuron-free deep neural network computing, *ACM SIGARCH Computer Architecture News* **44**(3), 1–13 (2016).
91. A. Parashar, M. Rhu, A. Mukkara, A. Puglielli, R. Venkatesan, B. Khailany, J. Emer, S. W. Keckler, and W. J. Dally, SCNN: An accelerator for compressed-sparse convolutional neural networks, *ACM SIGARCH Computer Architecture News* **45**(2), 27–40 (2017).

92. F. Gieseke, J. Heinermann, C. Oancea, and C. Igel. Buffer kd trees: processing massive nearest neighbor queries on gpus. In *International Conference on Machine Learning*, pp. 172–180 (2014).
93. D. Qiu, S. May, and A. Nüchter. GPU-accelerated nearest neighbor search for 3D registration. In *International Conference on Computer Vision Systems*, pp. 194–203 (2009).
94. S. Heinzle, G. Guennebaud, M. Botsch, and M. Gross. A hardware processing unit for point sets. In *Proceedings of the 23rd ACM SIGGRAPH/EUROGRAPHICS Symposium on Graphics Hardware*, pp. 21–31 (2008).
95. F. Winterstein, S. Bayliss, and G. A. Constantinides. FPGA-based K-means clustering using tree-based data structures. In *2013 23rd International Conference on Field programmable Logic and Applications*, pp. 1–6 (2013).
96. T. Xu, B. Tian, and Y. Zhu. Tigris: Architecture and algorithms for 3D perception in point clouds. In *MICRO*, pp. 629–642 (2019).
97. Y. Feng, B. Tian, T. Xu, P. Whatmough, and Y. Zhu. Mesorasi: Architecture support for point cloud analytics via delayed-aggregation. In *2020 53rd Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1037–1050 (2020).
98. M. Pellauer, Y. S. Shao, J. Clemons, N. Crago, K. Hegde, R. Venkatesan, S. W. Keckler, C. W. Fletcher, and J. Emer. Buffets: An Efficient and Composable Storage Idiom for Explicit Decoupled Data Orchestration. In *Proceedings of the Twenty-Fourth International Conference on Architectural Support for Programming Languages and Operating Systems*, pp. 137–151 (2019).
99. M. Alwani, H. Chen, M. Ferdman, and P. Milder. Fused-layer CNN accelerators. In *Proceedings of the 49th Annual IEEE/ACM International Symposium on Microarchitecture (MICRO)*, pp. 1–12 (2016).
100. Y. Kim, W. Yang, and O. Mutlu. Ramulator: A fast and extensible dram simulator, *IEEE Computer Architecture Letters* **15**(1), 45–49 (2015).
101. N. Muralimanohar, R. Balasubramonian, and N. Jouppi. Cacti 6.0: A tool to model large caches, *IEEE* (2015).

This page intentionally left blank

Chapter 10

Sampling Strategies for Efficient Segmentation and Object Detection of 3D Point Clouds

Qingyong Hu

*Department of Computer Science, University of Oxford, Oxford, UK
qingyong.hu@cs.ox.ac.uk*

A key aspect of efficient point cloud processing is sampling, which significantly reduces computational overhead, memory requirements, and redundancy. In this chapter, we first present a comprehensive review and in-depth analysis of existing representative heuristic-based and learning-based sampling methods, highlighting their merits and limitations. Subsequently, we introduce an efficient semantic segmentation method and an object detection method for large-scale 3D point clouds containing millions of points, with the point cloud sampling strategy serving as a crucial component. Through a detailed examination of the RandLA-Net and IA-SSD algorithms, we aim to provide valuable insights for future research and development in this domain. Our overarching goal is to advance the efficiency, accuracy, and scalability of point cloud processing techniques, ultimately benefiting a wide range of applications across various domains.

1. Introduction

Advancements in 3D sensor technology have led to significant improvements in resolution and an increase in the scale of point cloud data. For instance, a 64-line Velodyne LiDAR can generate over three billion points in a single hour when continuously scanning a specific scene.¹ The processing and analysis of large-scale, high-density 3D point clouds are computationally demanding, which poses considerable challenges to the efficient execution of deep neural networks.

Point cloud downsampling, a technique that reduces the number of points in a point cloud while retaining its essential features, is critical for various real-time applications. On the one hand, downsampling can significantly reduce the number of points in a point cloud, thereby lowering the computational resources required for processing and analysis. This reduction in complexity translates into faster processing speed and decreased memory consumption, which is essential for the processing of large-scale point clouds on systems with limited computational capabilities, expanding the applicability of point-cloud-based solutions to a wider range of devices and platforms. On the other hand, point cloud downsampling can effectively remove redundant or irrelevant points that do not contribute to the overall shape and structure of the point cloud. As a result, the accuracy and robustness of subsequent processing steps, such as object recognition, segmentation and registration, are likely to be enhanced. Additionally, downsampling can contribute to reducing noise and outliers within the data, further improving the performance and robustness of the algorithms.

Despite the numerous advantages of point cloud downsampling, there is currently no unified or consensus understanding of the ideal downsampling method to be employed across various tasks and applications. Different tasks and applications may have varying preferences for sampling strategies based on their individual requirements. For example, in object detection tasks where point cloud data is used to identify and classify objects, such as cars and pedestrians, the optimal downsampling strategy should minimise noise and outliers while preserving the foreground features as much as possible. In contrast, for robotic applications that rely on point cloud data for localisation and mapping, a downsampling strategy that maintains the overall structure of the environment while simultaneously reducing computational demands may be more suitable.

This chapter offers a comprehensive review and in-depth analysis of existing point cloud downsampling strategies, with the goal of providing valuable insights for researchers and practitioners in the field. In addition, we introduce two sampling methods specifically tailored for point cloud semantic segmentation and point cloud object detection. These methodologies aim to address the diverse requirements of various applications, ultimately enabling more effective and efficient use of point cloud data across a range of disciplines.

2. Point Cloud Sampling Strategies

Point cloud sampling strategies can be classified into two main categories: heuristic-based sampling and learning-based sampling. In the following sections, we discuss representative methods from each category in detail.

2.1. *Heuristic Sampling Strategies*

As the name suggests, a heuristic downsampling strategy is an approach to reducing the number of points in 3D point clouds based on heuristic rules or principles, which are derived from practical experience, intuition, or problem-specific knowledge. These strategies aim to simplify the point cloud data while preserving its essential features, often guided by specific characteristics, patterns, or relationships present in the data. As shown in Figure 1, representative heuristic downsampling strategies available for 3D point clouds include the following:

- **Random sampling (RS):** This is a simple downsampling method² that involves selecting points from the input point cloud uniformly and randomly, without considering any geometric or topological features. Specifically, each point in the point cloud has an equal probability of being selected, and the selection process iterates through the points once. The time complexity of RS is $\mathcal{O}(k)$, where k represents the number of randomly selected points in the point cloud. Random sampling is computationally and memory efficient, and easy to implement. However, it may not preserve important features or structures within the data, leading to a loss

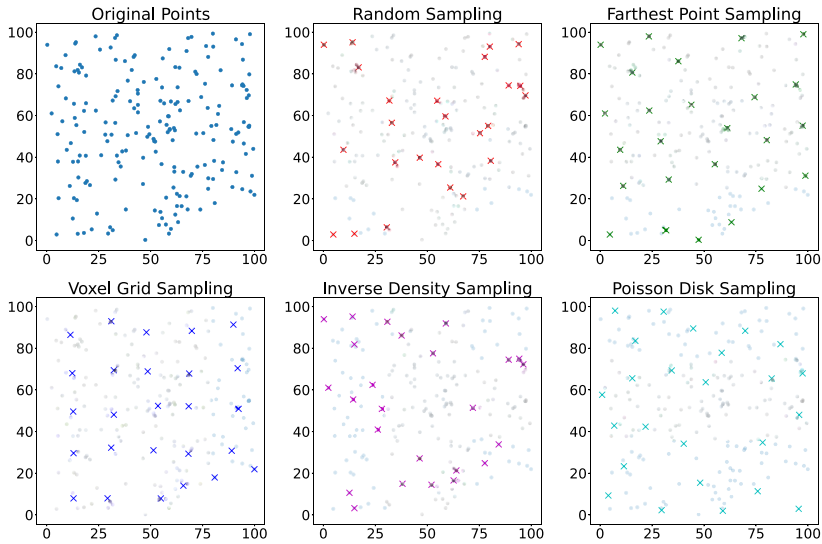


Fig. 1. Comparison of different heuristic-based sampling algorithms, the sampled points are represented by cross symbols of different colours.

of information that could potentially affect subsequent processing tasks.

- **Farthest point sampling (FPS):** This is a downsampling method³ that iteratively selects points that are farthest from the current set of chosen points, aiming to maximise the minimum distance between the sampled points and preserving the overall structure of the point cloud. The time complexity of FPS is $\mathcal{O}(nk^2)$, where n is the number of points in the point cloud and k is the number of points in the downsampled set. During each iteration, FPS identifies the point farthest from the current set of selected points, which requires distance comparisons for all remaining points. FPS is effective in preserving the overall structure of the point cloud and minimising the loss of critical features. However, it is less efficient and scalable than random sampling and may be computationally expensive for large-scale datasets.
- **Voxel grid sampling (VGS):** This is a downsampling technique that organises the input point cloud into a regular grid of voxels.⁴ Points within each voxel are replaced by a representative point, such as the centroid, effectively reducing the number of points while preserving the overall structure of the data. The algorithm

assigns each point to a voxel and calculates the representative point for each occupied voxel in a single pass, resulting in a complexity of $\mathcal{O}(n + (W/d)(H/d)(D/d))$, where n is the number of points in the point cloud, W , H , and D are the width, height, and depth of the grid structure, respectively, and d is the voxel size. Since the number of voxels in the 3D grid structure is much smaller than the number of points in the input point cloud, the time complexity is typically dominated by the voxelisation step, which is $\mathcal{O}(n)$. This makes VGS computationally efficient and effective in reducing the size of the point cloud while preserving the overall structure. However, VGS may also suffer from a loss of information due to the voxelisation process, which can lead to oversimplification or distortion of the underlying geometry.

- **Inverse density importance sampling (IDIS):** This is a down-sampling method⁵ that estimates the local point densities within the point cloud and selects points based on their inverse density values. The time complexity of inverse density importance sampling depends on the method used for estimating local point densities and selecting points based on their inverse density values. Density estimation typically involves finding neighbouring points within a specified radius or using a fixed number of nearest neighbours, which can have a complexity of $\mathcal{O}(n \log n)$ when using efficient data structures like k-d trees. Point selection can have varying complexities depending on the method employed (e.g., thresholding, ranking, or probabilistic sampling). Overall, the complexity of inverse density importance sampling can range from $\mathcal{O}(n \log n)$ to $\mathcal{O}(n^2)$, depending on the specific implementation. Inverse density importance sampling can effectively remove redundant points while adapting to the local point densities in the point cloud, leading to better preservation of features and structures while reducing data redundancy. However, it can be computationally intensive, especially when using inefficient methods for density estimation or point selection.
- **Poisson disk sampling (PDS):** This is a technique that generates a downsampled point cloud based on a probability density function derived from the local geometric properties of the data (e.g., surface normals).⁶ The time complexity of Poisson downsampling is $\mathcal{O}(n \log n)$, where n is the number of points in the point cloud. The algorithm typically uses data structures like octrees

or KD-trees for efficient spatial indexing and nearest-neighbour queries, which contribute to the $\mathcal{O}(n \log n)$ complexity. Poisson downsampling generates a more uniform distribution of points, resulting in better preservation of features and structures within the point cloud. It is particularly useful for preserving geometric details and producing visually pleasing results. However, this sampling strategy can be computationally more expensive than simpler methods like random sampling or voxel grid sampling. Additionally, selecting appropriate parameters for the Poisson disk sampling process can be challenging and may require tuning for specific datasets or applications.

In summary, heuristic downsampling strategies for 3D point clouds, such as random sampling, farthest point sampling, voxel grid sampling, inverse density importance sampling, and Poisson disk downsampling, provide various advantages and trade-offs in terms of computational complexity, feature preservation, and adaptability. The choice of the appropriate downsampling strategy depends on the specific requirements of the task or application, as well as the desired balance between computational efficiency and the preservation of critical features and structures within the data.

2.2. *Learning-Based Sampling Strategies*

A learning-based downsampling strategy is an approach to reducing the number of points in a 3D point cloud by leveraging data-driven machine learning techniques, particularly deep learning models, to optimise the point selection process. These strategies aim to minimise the loss of critical features and structures within the data while reducing its size, based on patterns and relationships learned from training data. Learning-based downsampling methods are especially useful when traditional or heuristic methods may not be sufficient or when an adaptive and specialised solution is required for specific tasks or applications. Representative learning-based downsampling strategies available for 3D point clouds include the following:

- **Generator-based sampling (GS):** Learn2Sample⁷ is a representative generator-based sampling strategy. The main idea is to learn to generate a small set of task-optimised points towards a task-specific optimisation objective. Unlike other sampling methods,

the generated point cloud may not necessarily be a subset of the original point cloud, providing more flexibility. The advantage of this method is that the learned sampling strategy is task-oriented and data-driven. However, the trained sampling strategy is susceptible to overfitting to a model rather than a task. Additionally, during the inference stage, farthest point sampling (FPS) is often used to match the generated subset with the original set, which incurs additional computation.

- **Continuous relaxation-based sampling (CRS):** CRS approaches^{8,9} use the reparameterisation trick to relax the discrete sampling operation with a differentiable function, which enables the use of gradient-based optimisation techniques for end-to-end training. Specifically, instead of directly sampling from a discrete distribution, this method involves expressing the sampling operation as a function of a continuous random variable, such as a Gaussian distribution. By doing so, the gradient of the sampling operation can be approximated by the gradient of the function of the continuous random variable, which can be computed using standard techniques. In particular, each sampled point is learnt based on a weighted sum over the full point clouds. It results in an unaffordable memory cost due to the large weight matrix required to sample all the new points simultaneously using a one-pass matrix multiplication approach. For example, it is estimated to take more than a 300 GB memory footprint to sample 10% of 10^6 points.
- **Policy gradient-based sampling (PGS):** PGS is a sampling strategy that formulates the downsampling process as a Markov decision process,¹⁰ breaking down the downsampling process into smaller, sequential steps. At each step, the algorithm learns a probability distribution that determines which points to sample next. By treating the sampling operation as a sequential decision-making problem, PGS learns a probability distribution that can be used to sample the points. However, the large exploration space associated with larger point clouds can lead to high variance in the learned probability distribution. For example, when attempting to sample 10% of 10^6 points, the exploration space is $C_{10^6}^{10^5}$, which is extremely large and makes it unlikely to learn an effective sampling policy. Furthermore, empirical studies have shown that PGS can be difficult to converge when used for large point clouds.

In addition to the downsampling strategies introduced earlier, there are other learning-based samplers that have been proposed in recent research, such as SampleNet¹¹ and Meta-Sampler.¹² Learning-based downsampling strategies have emerged as a promising approach for optimising point selection in 3D point clouds. By leveraging deep learning techniques, these strategies can adapt to specific tasks and handle complex patterns in the input data, resulting in improved performance and accuracy. The advantages of learning-based sampling strategies include advanced and adaptive solutions that can effectively balance data reduction and feature preservation, resulting in better downsamplings for a wide range of point cloud processing tasks. However, it is important to consider that these methods require a training phase, which can increase the overall computation time compared to traditional heuristic methods.

Next, this chapter introduces an efficient semantic segmentation method and an object detection method for large-scale 3D point clouds containing millions of points, in which the point cloud sampling strategy plays a crucial role. Specifically, two methods are introduced: RandLA-Net for large-scale 3D point cloud semantic segmentation and IA-SSD for object detection in large-scale 3D point clouds.

3. RandLA-Net: Random Sampling for Point Cloud Semantic Segmentation

3.1. Overview

For large-scale point clouds containing millions of points spanning across hundreds of meters, the objective of the semantic segmentation task is to generate dense point-wise semantic predictions. To accomplish this using a deep neural network, it is essential to progressively and efficiently downsample these data points. Given the considerable benefits of random sampling in terms of computational and memory costs, as well as scalability, this chapter presents a random sampling-based neural network architecture termed RandLA-Net.

As illustrated in Figure 2, the core concept of this method involves the systematic application of efficient random sampling and local feature aggregation. This approach minimises computational demands while preserving vital geometric information to the

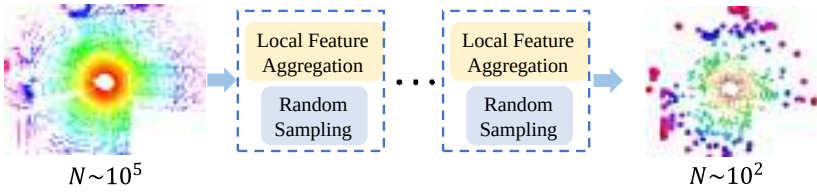


Fig. 2. In each layer of RandLA-Net, the large-scale point cloud is significantly downsampled yet is capable of retaining features necessary for accurate segmentation.

greatest extent possible. Consequently, RandLA-Net strikes a balance between efficiency and accuracy in the context of point cloud semantic segmentation.

3.2. The Quest for Efficient Sampling

In light of the computational challenges associated with large-scale point cloud processing, it is essential to identify an effective approach that balances efficiency and accuracy. Several existing methods, including FPS, IDIS, and GS, are highly computationally demanding, rendering them unsuitable for handling large-scale point clouds. Furthermore, CRS is burdened by an extensive memory footprint, while learning PPGS proves challenging. Although PDS also demonstrates efficient processing, it suffers from subpar performance. In comparison, random sampling offers two notable advantages: (1) remarkable computational efficiency due to its independence from the total number of input points and (2) minimal memory requirements, as additional memory for computation is unnecessary. Consequently, random sampling emerges as a more appropriate component for efficiently processing large-scale point clouds than alternative approaches. Nonetheless, random sampling may inadvertently exclude numerous valuable point features. To address this limitation, we introduce a robust local feature aggregation module, detailed in the subsequent section, to enhance the overall efficacy of random sampling in large-scale point cloud processing.

3.3. Local Feature Aggregation

As depicted in Figure 3, our local feature aggregation module operates in parallel for each 3D point and consists of three neural

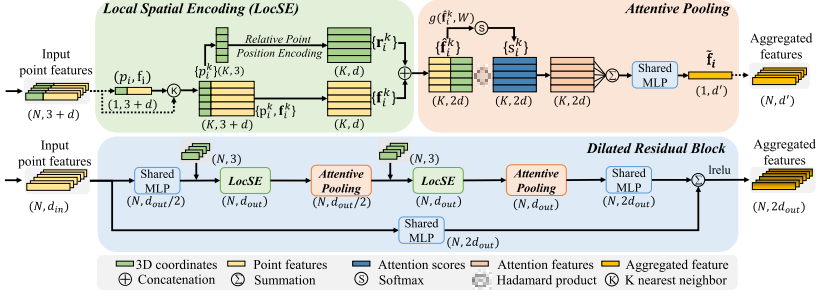


Fig. 3. The proposed local feature aggregation module. The top panel shows the local spatial encoding block that extracts features and the attentive pooling mechanism that weights the important neighbouring features, based on the local context and geometry. The bottom panel shows how two of these components are chained together, to increase the receptive field size, within a residual block.

units: (1) local spatial encoding (LocSE), (2) attentive pooling, and (3) dilated residual block.

(1) Local spatial encoding

Given a point cloud $\mathbf{P}$ accompanied by per-point features (e.g., raw RGB or intermediate learned features), the local spatial encoding unit explicitly embeds the x - y - z coordinates of all neighbouring points to ensure that the corresponding point features are consistently aware of their relative spatial locations. This enables the LocSE unit to explicitly capture local geometric patterns, ultimately benefiting the entire network in effectively learning complex local structures. Specifically, this unit comprises the following steps:

Identifying neighbouring points: Initially, the neighbouring points for the i th point are determined using the simple k -nearest neighbours (KNN) algorithm, which is based on point-wise Euclidean distances, to ensure efficiency.

Relative point position encoding: For each of the nearest K points $p_i^1 \dots p_i^k \dots p_i^K$ of the centre point p_i , the relative point position is encoded as follows:

$$\mathbf{r}_i^k = \text{MLP}(p_i \oplus p_i^k \oplus (p_i - p_i^k) \oplus \|p_i - p_i^k\|), \quad (1)$$

where p_i and p_i^k represent the absolute x - y - z positions of points, $\oplus$ denotes the concatenation operation, and $\|\cdot\|$ calculates the Euclidean distance between the neighbouring and centre points. Although it may appear that $\mathbf{r}_i^k$ is encoded from redundant point

positions, this approach interestingly aids the network in learning local features and achieves good performance in practice.

Point feature augmentation: For each neighbouring point p_i^k , the encoded relative point positions $\mathbf{r}_i^k$ are concatenated with their corresponding point features $\mathbf{f}_i^k$, resulting in an augmented feature vector $\hat{\mathbf{f}}_i^k$. For simplicity, we keep the feature vectors $\mathbf{r}_i^k$ and $\mathbf{f}_i^k$ with the same dimensions in our implementation, although they can be flexibly adjusted to have different dimensions.

Ultimately, the output of the LocSE unit is a new set of neighbouring features $\hat{\mathbf{F}}_i = \hat{\mathbf{f}}_i^1 \dots \hat{\mathbf{f}}_i^k \dots \hat{\mathbf{f}}_i^K$, which explicitly encode the local geometric structures for the centre point p_i . We note that recent work¹³ also utilises point positions to enhance semantic segmentation. However, in contrast to our LocSE, which explicitly encodes relative positions to augment neighbouring point features, this paper¹³ employs positions to learn point scores.

(2) Attentive pooling

This neural unit serves to aggregate the set of neighbouring point features $\hat{\mathbf{F}}_i$. Existing works^{3,14} typically employ max/mean pooling to rigidly integrate neighbouring features, leading to the loss of a significant amount of information. In contrast, we leverage the powerful attention mechanism to automatically learn important local features. Specifically, drawing inspiration from Ref. [15], our attentive pooling unit comprises the following steps:

Computing attention scores: Given the set of local features $\hat{\mathbf{F}}_i = \hat{\mathbf{f}}_i^1 \dots \hat{\mathbf{f}}_i^k \dots \hat{\mathbf{f}}_i^K$, we design a shared function $g(\cdot)$ to learn a unique attention score for each feature. In particular, the function $g(\cdot)$ consists of a shared MLP followed by a softmax function. It is formally defined as follows:

$$\mathbf{s}_i^k = g(\hat{\mathbf{f}}_i^k, \mathbf{W}), \quad (2)$$

where $\mathbf{W}$ is the learnable weights of a shared MLP.

Weighted summation: The learned attention scores can be viewed as a soft mask that automatically selects important features. Formally, these features are weighted and summed as follows:

$$\tilde{\mathbf{f}}_i = \sum_{k=1}^K (\hat{\mathbf{f}}_i^k \odot \mathbf{s}_i^k), \quad (3)$$

where $\odot$ denotes the element-wise product. In summary, given the input point cloud $\mathbf{P}$, for the i th point p_i , our LocSE and Attentive Pooling units learn to aggregate the geometric patterns and features of its K nearest points and finally generate an informative feature vector $\tilde{\mathbf{f}}_i$.

(3) Dilated residual block

As large point clouds will be significantly downsampled, it is advantageous to substantially increase the receptive field for each point, ensuring that the geometric details of input point clouds are more likely to be preserved, even if some points are omitted. As depicted in Figure 3, inspired by the successful ResNet¹⁶ and the effective dilated networks,¹⁷ we stack multiple LocSE and Attentive Pooling units to achieve the dilation of point receptive fields and introduce a skip connection to facilitate residual learning.

To further demonstrate the capability of our dilated residual block, Figure 4 illustrates that the red 3D point observes K neighbouring points after the first LocSE/Attentive Pooling operation and subsequently receives information from up to K^2 neighbouring points, i.e., its two-hop neighbourhood, after the second operation. This method provides an efficient way to dilate the receptive field and expand the effective neighbourhood through feature propagation. Theoretically, as more units are stacked, the block becomes increasingly powerful, as its sphere of reach grows larger. However,

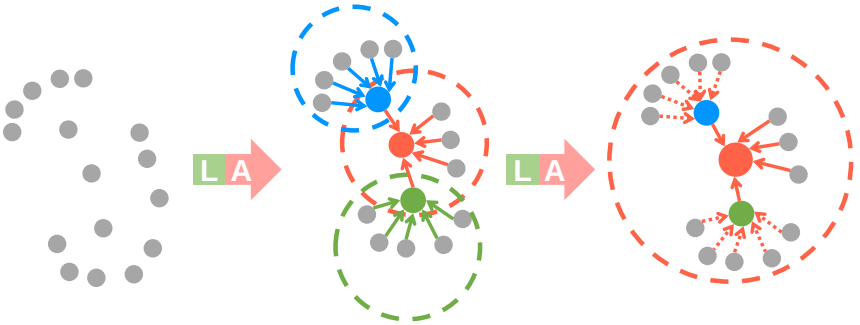


Fig. 4. Illustration of the dilated residual block which significantly increases the receptive field (dotted circle) of each point, coloured points represent the aggregated features.

Notes: L: Local spatial encoding and A: Attentive pooling.

additional units would inevitably compromise the overall computational efficiency. Moreover, the entire network is prone to overfitting. In our RandLA-Net, we simply stack two sets of LocSE and Attentive Pooling as the standard residual block, achieving a satisfactory balance between efficiency and effectiveness.

Overall, our local feature aggregation module is designed to effectively preserve complex local structures via explicitly considering neighbouring geometries and significantly increasing receptive fields. Moreover, this module only consists of feedforward MLPs, thus being computationally efficient.

3.4. Network Architecture

Figure 5 shows the detailed architecture of RandLA-Net, which stacks multiple local feature aggregation modules and random sampling layers. The network follows the widely used encoder–decoder architecture with skip connections. The input point cloud is first fed to a shared MLP layer to extract per-point features. Four encoding and decoding layers are then used to learn features for each point. At last, three fully connected layers and a dropout layer are used to predict the semantic label of each point. Note that, our RandLA-Net can be wider and deeper by altering the number of layers, feature channels, and adjusting the sampling rate. However, larger models require additional computation and may more easily lead to overfitting. The details of each component are as follows:

Network input: The input is a large-scale point cloud with a size of $\tilde{N} \times d_{\text{in}}$ (the batch dimension is dropped for simplicity), where $\tilde{N}$ is the total number of input points and d_{in} is the feature dimension of

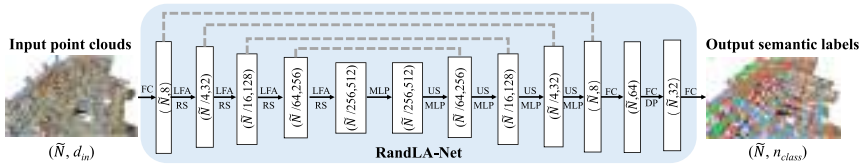


Fig. 5. The detailed architecture of our RandLA-Net. $(\tilde{N}, D)$ represents the number of points and feature dimension, respectively.

Notes: FC: Fully Connected layer, LFA: Local Feature Aggregation, RS: Random Sampling, MLP: shared Multi-Layer Perceptron, US: Up-sampling, DP: Dropout.

each input point. For both S3DIS¹⁸ and Semantic3D¹⁹ datasets, each point is represented by its 3D coordinates and colour information (i.e., x - y - z -R-G-B), while each point of the SemanticKITTI²⁰ dataset is only represented by 3D coordinates.

Encoding layers: Four encoding layers are used in our network to progressively reduce the size of the point clouds and increase the per-point feature dimensions. Each encoding layer consists of a local feature aggregation module and a random sampling operation. The point cloud is downsampled with a four-fold decimation ratio. In particular, only 25% of the point features are retained after each layer, i.e., $(\tilde{N} \rightarrow \frac{\tilde{N}}{4} \rightarrow \frac{\tilde{N}}{16} \rightarrow \frac{\tilde{N}}{64} \rightarrow \frac{\tilde{N}}{256})$. Meanwhile, the per-point feature dimension is gradually increased each layer to preserve more information, i.e., $(8 \rightarrow 32 \rightarrow 128 \rightarrow 256 \rightarrow 512)$.

Decoding layers: Four decoding layers are used after the encoding layers. For each layer in the decoder, we adopt the nearest-neighbour interpolation for efficiency and simplicity. In particular, the coordinates of all downsampled points in each encoding layer are temporally stored for reference. For each query point in the decoding layers, we use the KNN algorithm to find the nearest-neighbouring point from the points of the previous layer. The features of the nearest point are copied to the target point. Subsequently, the upsampled feature maps are concatenated with the intermediate feature maps produced by encoding layers through skip connections, after which a shared MLP is applied to the concatenated feature vectors.

Final semantic prediction: The final semantic label of each point is obtained through three shared fully connected layers $(\tilde{N}, 64) \rightarrow (\tilde{N}, 32) \rightarrow (\tilde{N}, n_{\text{class}})$ and a dropout layer. The dropout ratio is 0.5.

Network output: The output of RandLA-Net is the predicted semantics of all points, with a size of $\tilde{N} \times n_{\text{class}}$, where n_{class} is the number of classes.

3.5. Experiments

3.5.1. Efficiency of random sampling

In this section, we empirically evaluate the efficiency of existing sampling approaches including FPS, IDIS, PDS, RS, GS, CRS, and PGS,

which have been discussed in previous section. In particular, we conduct the following four groups of experiments:

- **Group 1:** Given a small-scale point cloud ($\sim 10^3$ points), we use each sampling approach to progressively downsample it. Specifically, the point cloud is downsampled by five steps with only 25% points being retained in each step on a single GPU, i.e., a four-fold decimation ratio. This means that there are only $\sim (1/4)^5 \times 10^3$ points left in the end. This downsampling strategy emulates the procedure used in PointNet++.³ For each sampling approach, we sum up its time and memory consumption for comparison.
- **Group 2/3/4:** The total number of points is increased towards large-scale, i.e., around 10^4 , 10^5 , and 10^6 points, respectively. We use the same five sampling steps as in Group 1.

Analysis: Figure 6 compares the total time and memory consumption of each sampling approach to process different scales of point clouds. The following can be seen: (1) For small-scale point clouds ($\sim 10^3$), all sampling approaches tend to have similar time and memory consumption and are unlikely to incur a heavy or limiting computation burden. (2) For large-scale point clouds ($\sim 10^6$), FPS/IDIS/PDS/GS/CRS/PGS are either extremely time-consuming or memory-costly for large-scale computation. By contrast, random sampling has superior time and memory efficiency overall. This result clearly demonstrates that most existing networks^{3,9,13,14,21,22} are only able to be optimised on small blocks of point clouds primarily because they rely on the expensive sampling approaches. Motivated by this, we use the efficient random sampling strategy in our RandLA-Net.

3.5.2. Semantic segmentation on public benchmarks

In this section, we evaluate the semantic segmentation of our RandLA-Net on multiple large-scale public datasets: the outdoor Semantic3D,¹⁹ SemanticKITTI,²⁰ Toronto-3D,²³ NPM3D,²⁴ and the indoor S3DIS.¹⁸ For a fair comparison, we follow KPConv⁴ to pre-process the whole input point clouds by using the grid-sampling strategy at the beginning. In fact, this is likely to reduce the point density and help increase the actual receptive field for each 3D point.

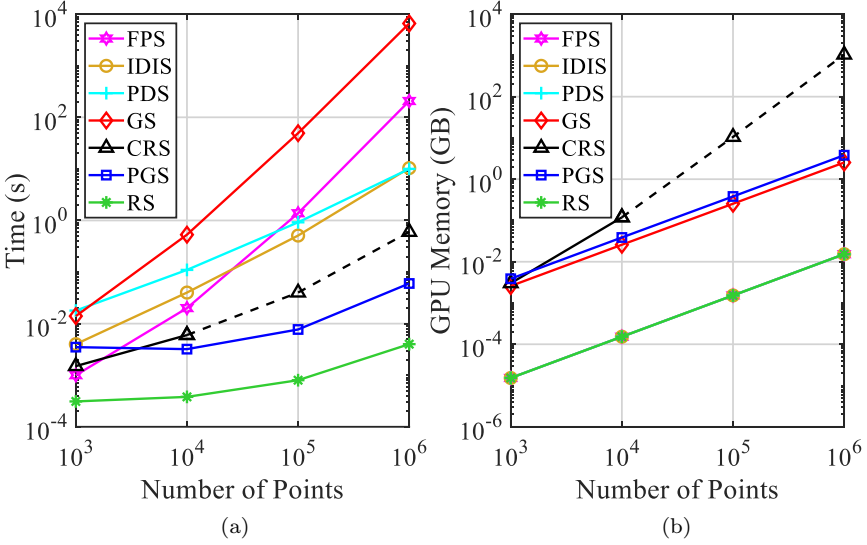


Fig. 6. Time and memory consumption of different sampling approaches. The dashed lines represent estimated values due to the limited GPU memory. Note that the curves of GPU memory consumption for RPS, PDS, FPS, and IDIS overlap together because these sampling methods mainly run on the CPU.

(1) Evaluation on Semantic3D

The Semantic3D dataset¹⁹ consists of 15 point clouds for training and 15 for online testing. Each point cloud has up to 10^8 points, covering up to $160 \times 240 \times 30$ m in real-world 3D space. The raw 3D points belong to eight classes and contain 3D coordinates, RGB information, and intensity. We only use the 3D coordinates and colour information to train and test our RandLA-Net. Overall Accuracy (OA) and mean Intersection-over-Union (mIoU) of all classes are used as the evaluation metrics. For a fair comparison, we only include the results of recently published strong baselines^{4,25–31} and the current state-of-the-art approach RGNet.³²

Table 1 presents the quantitative results of different approaches achieved on the *reduced-8* test set. RandLA-Net clearly outperforms all existing methods in terms of both mIoU and OA. Notably, RandLA-Net also achieves superior performance on six of the eight classes, except *low vegetation* and *scanning artefact*.

Table 1. Quantitative results of different approaches on Semantic3D (*reduced-8*).¹⁹ This test consists of 78,699,329 points. The scores are obtained from the recent publications. Accessed on 1 May 2020.

Methods	mIoU(%)	OA(%)	Man-made.	Natural.	High veg.	Low veg.	Buildings	Hard Scape	Scanning art.	Cars
SnapNet ₂₅	59.1	88.6	82.0	77.3	79.7	22.9	91.1	18.4	37.3	64.4
SEGCloud ³³	61.3	88.1	83.9	66.0	86.0	40.5	91.1	30.9	27.5	64.3
RF_MSSF ²⁷	62.7	90.3	87.6	80.3	81.8	36.4	92.2	24.1	42.6	56.6
MSDeepVoxNet ²⁸	65.3	88.4	83.0	67.2	83.8	36.7	92.4	31.3	50.0	78.2
ShellNet ²⁹	69.3	93.2	96.3	90.4	83.9	41.0	94.2	34.7	43.9	70.2
GACNet ³⁰	70.8	91.9	86.4	77.7	88.5	60.6	94.2	37.3	43.5	77.8
SPG ³¹	73.2	94.0	97.4	92.6	87.9	44.0	83.2	31.0	63.5	76.2
KPConv ⁴	74.6	92.9	90.9	82.2	84.2	47.9	94.9	40.0	77.3	79.7
RCNet ³²	74.7	94.5	97.5	93.0	88.1	48.1	94.6	36.2	72.0	68.0
RandLA-Net (Ours)	77.4	94.8	95.6	91.4	86.6	51.5	95.7	51.5	69.8	76.8

(2) Evaluation on SemanticKITTI

SemanticKITTI²⁰ consists of 43552 densely annotated LIDAR scans belonging to 22 sequences. Each scan is a large-scale point cloud with $\sim 10^5$ points and spanning up to $160 \times 160 \times 20$ m in 3D space. Officially, the sequences 00~07 and 09~10 (19130 scans) are used for training, the sequence 08 (4071 scans) for validation, and the sequences 11~21 (20351 scans) for online testing. The raw 3D points only have 3D coordinates without colour information. The mIoU score over 19 categories is used as the standard metric.

Table 2 shows a quantitative comparison of our RandLA-Net with two families of recent approaches, i.e., (1) point-based methods^{3,31,34–36} and (2) projection-based approaches,^{20,37–40} and Figure 7 shows some qualitative results of RandLA-Net on the validation split. It can be seen that our RandLA-Net surpasses all point-based approaches^{3,31,34–36} by large margins, with remarkable improvement of 15% compared with the second-best approach. We also outperform most projection-based methods^{20,37,38} but not significantly, primarily because SqueezeSegV3⁴¹ achieves much better results on the small object category, such as *motorcyclist*. However, our RandLA-Net is more lightweight with fewer parameters compared with the projection-based methods.

(3) Evaluation on S3DIS

The S3DIS dataset¹⁸ consists of 271 rooms belonging to 6 large areas. Each point cloud is a medium-sized single room ($\sim 20 \times 15 \times 5$ m) with dense 3D points. To evaluate the semantic segmentation of our RandLA-Net, we use the standard 6-fold cross-validation in our experiments. The mean IoU (mIoU), mean class Accuracy (mAcc), and Overall Accuracy (OA) of the total 13 classes are compared.

Table 3 quantitatively compares the performance of our RandLA-Net with existing baselines on this dataset. Our RandLA-Net achieves on-par or better performance than state-of-the-art methods. Note that most of these baselines^{3,14,22,29,47,48} tend to use sophisticated but expensive operations or samplings to optimise the networks on small blocks (e.g., 1×1 m) of point clouds, and the relatively small rooms act in their favours to be divided into tiny blocks. By contrast, RandLA-Net is able to take the entire rooms as input and efficiently infer per-point semantics in a single pass.

Table 2. Quantitative results of different approaches on SemanticKITTI.²⁰ The scores are obtained from the recent publications. Accessed on 1 May 2020.

Methods	Size	mIoU(%)	Params (M)	Road	Sidewalk	Parking	Other- ground	Building	Car	Truck	Bicycle
PointNet ³⁴	50K pts	14.6	3	61.6	35.7	15.8	1.4	41.4	46.3	0.1	1.3
SPG ³¹		17.4	0.25	45.0	28.5	0.6	0.6	64.3	49.3	0.1	0.2
SPLATNet ³⁵		18.4	0.8	64.6	39.1	0.4	0.0	58.3	58.2	0.0	0.0
PointNet++ ³		20.1	6	72.0	41.8	18.7	5.6	62.3	53.7	0.9	1.9
TangentConv ³⁶		40.9	0.4	83.9	63.9	33.4	15.4	83.4	90.8	15.2	2.7
SqueezeSeg ³⁷	64*2048 pixels	29.5	1	85.4	54.3	26.9	4.5	57.4	68.8	3.3	16.0
SqueezeSegV2 ³⁸		39.7	1	88.6	67.6	45.8	17.7	73.7	81.8	13.4	18.5
DarkNet21Seg ²⁰		47.4	25	91.4	74.0	57.0	26.4	81.9	85.4	18.6	26.2
DarkNet53Seg ²⁰		49.9	50	91.8	74.6	64.8	27.9	84.1	86.4	25.5	24.5
RangeNet53++ ³⁹		52.2	50	91.8	75.2	65.0	27.8	87.4	91.4	25.7	25.7
SalsaNext ⁴²		54.5	6.73	90.9	74.0	58.1	27.8	87.9	90.9	21.7	36.4
SqueezeSegV3 ⁴¹		55.9	26	91.7	74.8	63.4	26.4	89.0	92.5	29.6	38.7
LatticeNet ⁴⁰	—	52.2	—	88.8	73.8	64.6	25.6	86.9	88.6	43.3	12.0
PolarNet ⁴³	—	54.3	14	90.8	74.4	61.7	21.7	90.0	93.8	22.9	40.2
RandLA- Net (Ours)	50K pts	55.9	1.24	90.5	74.0	61.8	24.5	89.7	94.2	43.9	47.4

(Continued)

Table 2. (Continued)

Methods	Motorcycle	Other-vehicle	Vegetation	Trunk	Terrain	Person	Bicyclist	Motorcyclist	Fence	Pole	Traffic-sign
PointNet ³⁴	0.3	0.8	31.0	4.6	17.6	0.2	0.2	0.0	12.9	2.4	3.7
SPG ³¹	0.2	0.8	48.9	27.2	24.6	0.3	2.7	0.1	20.8	15.9	0.8
SPLATNet ³⁵	0.0	0.0	71.1	9.9	19.3	0.0	0.0	0.0	23.1	5.6	0.0
PointNet++ ³	0.2	0.2	46.5	13.8	30.0	0.9	1.0	0.0	16.9	6.0	8.9
TangentConv ³⁶	16.5	12.1	79.5	49.3	58.1	23.0	28.4	8.1	49.0	35.8	28.5
SqueezeSeg ³⁷	4.1	3.6	60.0	24.3	53.7	12.9	13.1	0.9	29.0	17.5	24.5
SqueezeSegV2 ³⁸	17.9	14.0	71.8	35.8	60.2	20.1	25.1	3.9	41.1	20.2	36.3
DarkNet21Seg ²⁰	26.5	15.6	77.6	48.4	63.6	31.8	33.6	4.0	52.3	36.0	50.0
DarkNet53Seg ²⁰	32.7	22.6	78.3	50.1	64.0	36.2	33.6	4.7	55.0	38.9	52.2
RangeNet53++ ³⁹	34.4	23.0	80.5	55.1	64.6	38.3	38.8	4.8	58.6	47.9	55.9
SalsaNext ⁴²	29.5	19.9	81.8	61.7	66.3	52.0	52.7	16.0	58.2	51.7	58.0
SqueezeSegV3 ⁴¹	36.5	33.0	82.0	58.7	65.4	45.6	46.2	20.1	59.4	49.6	58.9
LatticeNet ⁴⁰	20.8	24.8	76.4	57.9	54.7	34.2	39.9	60.9	55.2	41.5	42.7
PolarNet ⁴³	30.1	28.5	84.0	65.5	67.8	43.2	40.2	5.6	61.3	51.8	57.5
RandLA-Net (Ours)	32.2	39.1	83.8	63.6	68.6	48.4	47.4	9.4	60.4	51.0	50.7

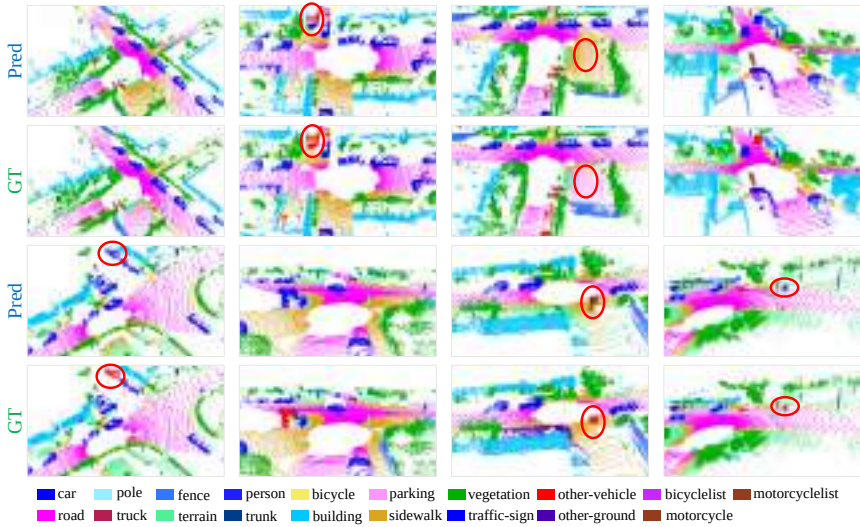


Fig. 7. Qualitative results of RandLA-Net on the validation set of Semantic-KITTI.²⁰ Red boxes show the failure cases.

4. IASSD: Instance-Aware Sampling for Point Cloud Object Detection

4.1. Overview

Distinct from dense prediction tasks such as 3D semantic segmentation, which necessitates point-wise prediction, **3D object detection naturally focuses on small, yet crucial foreground objects** (i.e., instances of interest including *car* and *pedestrian*), instead of treating all points equally. However, existing point-based detectors usually employ task-agnostic downsampling techniques, such as random sampling⁴⁹ or farthest point sampling^{3,50} within their frameworks. Although these methods effectively reduce memory and computational costs, they also diminish the most critical foreground points during progressive downsampling.

To address this limitation, this chapter introduces an efficient, single-stage detector incorporating a learnable instance-aware downsampling strategy. As depicted in Figure 8, the proposed IA-SSD utilises the lightweight encoder-only architecture employed in Ref. [50] for enhanced efficiency. The input LiDAR point clouds

Table 3. Quantitative results of different approaches on S3DIS¹⁸ (6-fold cross-validation). Overall Accuracy (OA, %), mean class Accuracy (mAcc, %), mean IoU (mIoU, %), and per-class IoU (%) are reported.

Methods	OA(%)	mAcc(%)	mIoU(%)	Ceil.	Floor	Wall	Beam	Col.	Wind.	Door	Table	Chair	Sofa	Book.	Board	Clut.
PointNet ³⁴	78.6	66.2	47.6	88.0	88.7	69.3	42.4	23.1	47.5	51.6	54.1	42.0	9.6	38.2	29.4	35.2
RSNet ⁴⁴	—	66.5	56.5	92.5	92.8	78.6	32.8	34.4	51.6	68.1	59.7	60.1	16.4	50.2	44.9	52.0
3P-RNN ⁴⁵	86.9	—	56.3	92.9	93.8	73.1	42.5	25.9	47.6	59.2	60.4	66.7	24.8	57.0	36.7	51.6
SPG ³¹	86.4	73.0	62.1	89.9	95.1	76.4	62.8	47.1	55.3	68.4	73.5	69.2	63.2	45.9	8.7	52.9
PointCNN ¹⁴	88.1	75.6	65.4	94.8	97.3	75.8	63.3	51.7	58.4	57.2	71.6	69.1	39.1	61.2	52.2	58.6
PointWeb ²²	87.3	76.2	66.7	93.5	94.2	80.8	52.4	41.3	64.9	68.1	71.4	67.1	50.3	62.7	62.2	58.5
ShellNet ²⁹	87.1	—	66.8	90.2	93.6	79.9	60.4	44.1	64.9	52.9	71.6	84.7	53.8	64.6	48.6	59.4
PointASNL ⁴⁶	88.8	79.0	68.7	95.3	97.9	81.9	47.0	48.0	67.3	70.5	71.3	77.8	50.7	60.4	63.0	62.8
KPConv _{rigid} ⁴	—	78.1	69.6	93.7	92.0	82.5	62.5	49.5	65.7	77.3	57.8	64.0	68.8	71.7	60.1	59.6
KPConv _{deform} ⁴	—	79.1	70.6	93.6	92.4	83.1	63.9	54.3	66.1	76.6	57.8	64.0	69.3	74.9	61.3	60.3
RandLA-Net (Ours)	88.0	82.0	70.0	93.1	96.1	80.6	62.4	48.0	64.4	69.4	69.4	76.4	60.0	64.2	65.9	60.1

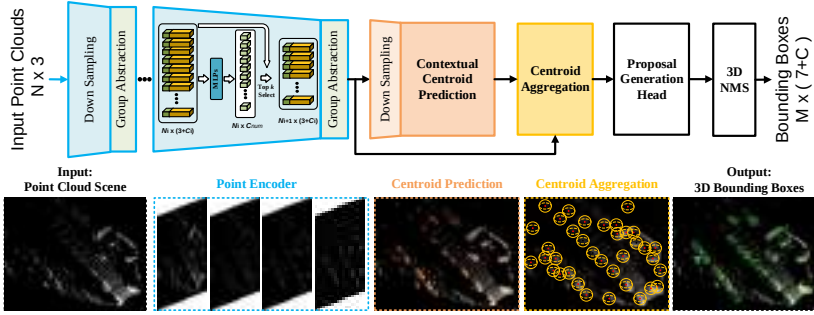


Fig. 8. Illustration of the proposed IA-SSD. The input point clouds are first fed into several Set Abstraction (SA) layers, followed by the instance-aware downsampling to progressively reduce the memory and computational cost. The preserved representative points are further fed into the contextual centroid perception module for instance centre prediction and proposal generation. Finally, the 3D bounding box and associated class labels are outputted.

are initially processed by the network to extract point-wise features, followed by the proposed instance-aware downsampling, which progressively mitigates computational costs while concurrently preserving informative foreground points. The learned latent features are further fed into the contextual centroid perception module to generate instance proposals and refine the final bounding boxes.

4.2. Motivation

Before delving into technical details, we first aim to conduct a quantitative experiment demonstrating that the existing heuristic downsampling strategies may not be well suited for 3D object detection tasks, since object detection tasks place greater emphasis on foreground points rather than all points. To this end, we perform an empirical study to quantitatively evaluate different sampling approaches. Specifically, we adopt the widely used encoding architecture (i.e., PointNet++³ with 4 encoding layers) and report the instance recall (i.e., the ratio of instances retained after sampling) at each layer in Table 4. In particular, random point sampling,² FPS-based on Euclidean distance (D-FPS),³ and feature distance (Feat-FPS)⁵⁰ are reported.

Analysis: The following can be seen: (1) The instance recall rate significantly declines after several random downsampling operations,

Table 4. Instance recall rate for foreground points (i.e., *car*, *pedestrian*, and *cyclist*) after several downsampling stages on the entire *validation* set (3,769 frames) of the KITTI benchmark. Note that the input point clouds with 16,384 points are progressively downsampled to 256 points through four downsampling layers. D-FPS is used in the first two layers for the proposed instance-aware downsampling strategies.

Sampling Strategies	4096 Points			1024 Points			512 Points			256 Points		
	Car	Ped.	Cyc.	Car	Ped.	Cyc.	Car	Ped.	Cyc.	Car	Ped.	Cyc.
Random ²	96.6%	99.1%	97.4%	87.5%	92.7%	84.1%	78.8%	84.9%	73.3%	67.4%	72.1%	57.3%
D-FPS ³	98.3%	100%	97.2%	97.9%	99.3%	97.2%	96.8%	90.6%	90.8%	91.4%	69.1%	71.6%
Feat-FPS ⁵⁰	98.3%	100%	97.2%	97.7%	98.0%	97.2%	96.3%	87.6%	94.5%	95.3%	80.1%	91.7%
Cls-aware (Ours)	98.3%	100%	97.2%	97.9%	99.3%	97.2%	97.9%	99.0%	95.4%	97.9%	97.4%	92.7%
Ctr-aware (Ours)	98.3%	100%	97.2%	97.9%	99.3%	97.2%	97.9%	99.0%	97.2%	97.9%	98.4%	97.2%

indicating that numerous foreground points have been discarded. (2) Both D-FPS and Feat-FPS yield a relatively better instance recall rate in the early stages, but they also fail to preserve a sufficient number of foreground points in the final encoding layer.

From these findings, we can infer that commonly employed heuristic-based point cloud downsampling strategies may not be particularly suitable for object detection tasks, as they do not inherently prioritise foreground objects. Furthermore, considering that foreground objects, such as *pedestrians* and *cyclists*, are small in scale and comprise a limited number of points, detectors based on heuristic point cloud downsampling strategies may struggle to identify these categories, since only an extremely limited number of foreground points remain after progressive downsampling.

4.3. Instance-Aware Downsampling Strategy

Motivated by the aforementioned experiments, we aim to propose a sampling strategy tailored for the 3D point cloud object detection task. Ideally, the proposed sampling should not only significantly reduce computational and memory costs but also retain as many foreground points as possible. The key to achieving this objective lies in accurately identifying which points in the point cloud belong to foreground points. Consequently, we have considered exploiting the latent semantics of each point to differentiate foreground points from the background, as the learned point features may contain richer semantic information with the hierarchical aggregation used in each layer. With this in mind, we introduce the following two task-oriented sampling methods, which incorporate foreground semantic priors into the network training pipeline:

Class-aware sampling: This sampling strategy aims to infer the semantic class of each point, further facilitating selective downsampling based on the semantic categories. To achieve this, we introduce additional branches aimed at exploiting the rich semantic information present in latent features. Specifically, two MLP layers are appended to the encoding layers to further estimate the semantic categories of each point. The point-wise one-hot semantic labels generated from the original bounding box annotations are utilised for supervision. For this purpose, we employ the conventional

cross-entropy loss:

$$L_{\text{cls-aware}} = - \sum_{c=1}^C (s_i \log(\hat{s}_i) + (1 - s_i) \log(1 - \hat{s}_i)), \quad (4)$$

where C denotes the number of categories, s_i is the one-hot labels, and $\hat{s}_i$ denotes the predicted logits. During inference, the points with the top k foreground scores are retained and regarded as the representative points that feed into the next encoding layers. As shown in Table 4, this strategy tends to preserve more foreground points, hence achieving a high ratio of instance recall.

Centroid-aware sampling: Given that estimating the instance centre is crucial for the final object detection, we further propose a centroid-aware downsampling strategy that assigns higher weights to points closer to the instance centroid. Specifically, we define the soft point mask for instance i as follows:

$$\text{Mask}_i = \sqrt[3]{\frac{\min(f^*, b^*)}{\max(f^*, b^*)} \times \frac{\min(l^*, r^*)}{\max(l^*, r^*)} \times \frac{\min(u^*, d^*)}{\max(u^*, d^*)}} \quad (5)$$

where $f^*, b^*, l^*, r^*, u^*, d^*$ represent the distances of a point to the six surfaces (front, back, left, right, up, and down) of the bounding box, respectively. In this case, a point closer to the centroid of the box is likely to have a higher mask score (with a maximum value of 1), while a point that lies on the surface will have a mask score of 0. During training, the soft point mask is used to assign different weights for points within a bounding box based on their spatial locations, thereby implicitly incorporating the geometry priors into the network training. Specifically, the weighted cross-entropy loss is calculated as follows:

$$L_{\text{ctr-aware}} = - \sum_{c=1}^C (\text{Mask}_i \cdot s_i \log(\hat{s}_i) + (1 - s_i) \log(1 - \hat{s}_i)). \quad (6)$$

The soft point mask is multiplied with the loss term of foreground points, in order to assign a higher probability to points near the centre. It is important to note that bounding boxes are not required during inference. We simply retain the top k points with the highest scores after downsampling, provided that the model is well trained.

4.4. Network Architectures

Contextual centroid prediction: Inspired by the success of context prediction in 2D images,^{51,52} we attempt to leverage the contextual cues surrounding the bounding box for instance centroid prediction. Specifically, following the approach in Ref. [53], we explicitly predict an offset $\Delta\hat{c}_{ij}$ to the instance centre. Furthermore, we introduce a regularisation term to minimise the uncertainty associated with centroid prediction. To achieve this, all votes per instance are aggregated, considering the surrounding interference, where $\bar{c}_i$ represents the mean destination of the i th instance. Consequently, the centroid prediction loss is formulated as follows:

$$L_{\text{cent}} = \frac{1}{|\mathcal{F}_+|} \frac{1}{|\mathcal{S}_+|} \sum_i \sum_j (|\Delta\hat{c}_{ij} - \Delta c_{ij}| + |\hat{c}_{ij} - \bar{c}_i|) \cdot \mathbf{I}_{\mathcal{S}}(p_{ij}),$$

where $\bar{c}_i = \frac{1}{|\mathcal{S}_+|} \sum_j \hat{c}_{ij}$, $\mathbf{I}_{\mathcal{S}} : \mathcal{P} \rightarrow \{0, 1\}$, (7)

where Δc_{ij} represents the ground-truth offset from point p_{ij} to the centre point. $\mathbf{I}_{\mathcal{S}}$ is an indicator function that determines whether a point is used for instance centre estimation. $|\mathcal{S}_+|$ denotes the number of points employed in predicting the instance centre. Notably, instead of solely utilising points or shifted points within the bounding box for instance centre prediction,^{50,53} our method exploits surrounding representative points from a broader context for centroid prediction. Specifically, we empirically investigate the impact of simple contextual cues on the final detection performance. In this regard, we manually expand the ground-truth bounding boxes or proportionally enlarge the boxes to cover more related context near the objects. The sampled points that fall within the expanded bounding box are employed for offset estimation and subsequent shifting.

Centroid-based instance aggregation: For the shifted representative (centroid) points, we further utilise a PointNet++ module to learn a latent representation for each instance. Specifically, we transform the neighbouring points to a local canonical coordinate system and then aggregate the point features through shared MLPs and symmetric functions.

Proposal generation head: The aggregated centroid point features are then fed into a proposal generation head for predicting bounding

boxes with associated classes. We encode the proposal as a multi-dimensional representation, including location, scale, and orientation. Finally, all proposals are filtered by 3D-NMS post-processing using a specific IoU threshold.

Our IA-SSD can be trained end-to-end. A multi-task loss is employed in our framework for joint optimisation. The total loss L_{total} consists of the downsampling strategy loss L_{sample} , centroid prediction loss L_{cent} , classification loss L_{cls} , and box generation loss L_{box} :

$$L_{\text{total}} = L_{\text{sample}} + L_{\text{cent}} + L_{\text{cls}} + L_{\text{box}}. \quad (8)$$

In particular, the box generation loss can be further decomposed into location, size, angle-bin, angle-res, and corner parts:

$$L_{\text{box}} = L_{\text{loc}} + L_{\text{size}} + L_{\text{angle-bin}} + L_{\text{angle-res}} + L_{\text{corner}}. \quad (9)$$

4.5. Experiments

Next, we introduce the results achieved by the proposed IA-SSD on several large-scale 3D point cloud datasets.

Evaluation on KITTI dataset: In the KITTI benchmark, objects are classified into three categories: *car*, *pedestrian*, and *cyclist*. These categories are further divided into three subsets (“Easy”, “Moderate”, and “Hard”) based on the levels of difficulty. The results on the “Moderate” subset are typically adopted as the primary indicator for final ranking. We report the results achieved by different methods (voxel, point, and point-voxel-based methods) on the test set of the KITTI dataset in Table 5. It is important to note that, since⁵⁰ does not provide a reproducible implementation or pre-trained models for *pedestrian* and *cyclist*, we present the results reported in their paper, the best-reproduced results, and the results achieved by OpenPCDet^a implementation for a fair comparison.

^a<https://github.com/open-mmlab/OpenPCDet>.

Table 5. Quantitative detection performance achieved by different methods on the KITTI *test* set. All results are evaluated by mean average precision with 40 recall positions via the official KITTI evaluation server. The results of our IA-SSD are shown in bold, and the best results are underlined.

				3D Car (IoU=0.7)			3D Ped. (IoU=0.5)			3D Cyc. (IoU=0.5)			
Method		Reference	Type	Easy	Mod.	Hard	Easy	Mod.	Hard	Easy	Mod.	Hard	Speed
Voxel-based	VoxelNet ⁵⁴	CVPR 2018	1-stage	77.47	65.11	57.73	39.48	33.69	31.5	61.22	48.36	44.37	4.5
	SECOND ⁵⁵	Sensors 2018	1-stage	84.65	75.96	68.71	45.31	35.52	33.14	75.83	60.82	53.67	20
	PointPillars ⁵⁶	CVPR 2019	1-stage	82.58	74.31	68.99	51.45	41.92	38.89	77.10	58.65	51.92	42.4
	3D IoU Loss ⁵⁷	3DV 2019	1-stage	86.16	76.50	71.39	—	—	—	—	—	—	12.5
	Associate-3Ddet ⁵⁸	CVPR 2020	1-stage	85.99	77.40	70.53	—	—	—	—	—	—	20
	SA-SSD ⁵⁹	CVPR 2020	1-stage	88.75	79.79	74.16	—	—	—	—	—	—	25
	CIA-SSD ⁶⁰	AAAI 2021	1-stage	89.59	80.28	72.87	—	—	—	—	—	—	32
	TANet ⁶¹	AAAI 2020	2-stage	84.39	75.94	68.82	53.72	<u>44.34</u>	40.49	75.70	59.44	52.53	28.5
	Part-A ²⁶²	TPAMI 2020	2-stage	87.81	78.49	73.51	53.10	43.35	40.06	79.17	63.52	56.93	12.5
Point-Voxel	Fast Point R-CNN ⁶³	ICCV 2019	2-stage	85.29	77.40	70.24	—	—	—	—	—	—	16.7
	STD ⁶⁴	ICCV 2019	2-stage	87.95	79.71	75.09	53.29	42.47	38.35	78.69	61.59	55.30	12.5
	PV-RCNN ⁶⁵	CVPR 2020	2-stage	<u>90.25</u>	<u>81.43</u>	<u>76.82</u>	52.17	43.29	40.29	78.60	63.71	<u>57.65</u>	12.5
	VIC-Net ⁶⁶	ICRA 2021	1-stage	88.25	80.61	75.83	43.82	37.18	35.35	78.29	63.65	57.27	17
	HVPR ⁶⁷	CVPR 2021	1-stage	86.38	77.92	73.04	53.47	43.96	<u>40.64</u>	—	—	—	36.1
Point-based	PointRCNN ⁶⁸	CVPR 2019	2-stage	86.96	75.64	70.70	47.98	39.37	36.01	74.96	58.82	52.53	10
	3D IoU-Net ⁶⁹	Arxiv 2020	2-stage	87.96	79.03	72.78	—	—	—	—	—	—	10
	Point-GNN ⁷⁰	CVPR 2020	1-stage	88.33	79.47	72.29	51.92	43.77	40.14	78.60	63.48	57.08	1.6
	3DSSD ⁵⁰	CVPR 2020	1-stage	88.36	79.57	74.55	<u>54.64</u>	44.27	40.23	<u>82.48</u>	64.10	56.90	25
	3DSSD [†] (Reproduced)	CVPR 2020	1-stage	87.73	78.58	72.01	35.03	27.76	26.08	66.69	59.00	55.62	23
	3DSSD [‡] (OpenPCDet)	CVPR 2020	1-stage	87.91	79.55	74.71	3.63	3.18	2.57	27.08	21.38	19.68	28
	IA-SSD (single-class)	—	1-stage	88.87	80.32	75.10	49.01	41.20	38.03	80.78	<u>66.01</u>	<u>58.12</u>	<u>85</u>
	IA-SSD (multi-class)	—	1-stage	88.34	80.13	75.04	46.51	39.03	35.60	78.35	61.94	55.70	<u>83</u>

Analysis: The following can be seen: (1) The proposed IA-SSD achieves the best *cyclist* detection performance, even outperforming several strong point-voxel and voxel detectors.^{62,65} This is mainly because the proposed instance aware sampling can effectively preserve foreground points, enabling accurate detection of small objects. (2) Our IA-SSD also achieves best *car* detection performance compared with other point-based detectors, outperforming PointRCNN⁶⁸ by (1.91%, 4.68%, 4.4%), and the SoTA method 3DSSD⁵⁰ by (0.51%, 0.75%, 0.55%) mAP. (3) Despite the competitive detection performance, the proposed IA-SSD also shows superior efficiency. It can detect with a speed of 85 FPS on a single NVIDIA RTX 2080Ti with Intel I9-10900X CPU@3.7GHz. (4) Thanks to the instance-aware sampling strategy and the contextual centroid perception module, our framework can be trained with multi-class together (i.e., training a single model for detecting multi-class objects), rather than training separate models for different objects.⁵⁰ In particular, the performance is still comparable with other state-of-the-art approaches. This allows our model much more efficient and flexible during inference. While the two-stage detector PV-RCNN does exhibit competitive performance in this dataset, it is essential to note that our 1-stage IA-SSD is much more efficient, due to the efficient instance-aware downsampling and a single-stage pipeline. Finally, we also show the qualitative results achieved by our IA-SSD in Figure 9.

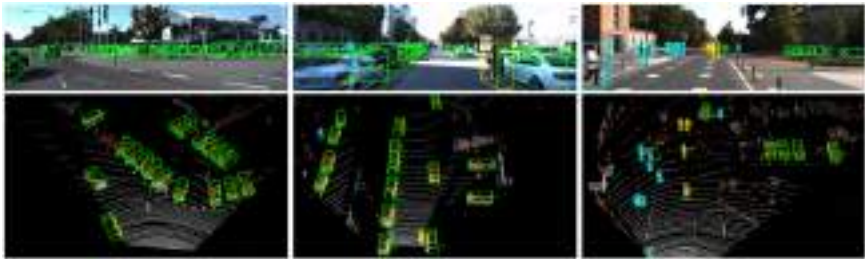


Fig. 9. Qualitative results achieved on the KITTI *test* set. Red points indicate centroid perception, while gold points denote the 256 representative points. Green boxes represent *car*, cyan for *pedestrian*, and yellow for *cyclist*. Best viewed in colour.

We can clearly see that the proposed IA-SSD is capable of detecting small and far-away instances, such as *pedestrian* and *cyclist*.

Evaluation on Waymo dataset: We further evaluate the performance of our method on the Waymo⁷¹ dataset. This dataset consists of nearly 160k 360° LiDAR samples in the *training* set and 40k in the *validation* set, with panoramic annotated objects. To ensure a fair comparison, we adapt our framework for the Waymo Dataset by only changing the number of input points from 16,384 to 65,536 and increasing the sampling scale by up to fourfold in each sampling layer while keeping the rest unchanged. Moreover, all baselines are implemented based on the OpenPCDet codebase for a rigorous comparison. As illustrated in Table 6, our method achieves significantly better detection performance for *pedestrian* and *cyclist* compared to other strong baselines, indicating that the proposed instance-aware sampling can indeed enhance the perception capacity for small objects. We also observe that our method exhibits slightly inferior detection performance for *vehicle* compared to other voxel-based methods. This may be due to the relatively complex distribution of the 3D sizes of such instances.

Efficiency of IA-SSD: We proceed to evaluate the computational and memory efficiency of the proposed IA-SSD. Given that performance may vary depending on hardware configurations, we re-implement several representative approaches and report memory usage and processing speed on the same platform for a fair comparison. It is important to note that we report memory consumption when feeding the same input point cloud with 16,384 points, following the OpenPCDet configuration. For speed evaluation, different models are tested with batch point clouds to fully utilise the same hardware. As demonstrated in Table 7, the proposed IA-SSD exhibits the lowest GPU memory consumption (up to 100 frames in parallel) and the highest inference speed (83 FPS) when compared to existing benchmark approaches.

Table 6. Quantitative detection performance achieved by different methods on the Waymo⁷¹ *validation* set. The results of our IA-SSD are shown in bold, and the best results are underlined.

Method	Type	Vehicle (LEVEL 1)		Vehicle (LEVEL 2)		Ped. (LEVEL 1)		Ped. (LEVEL 2)		Cyc. (LEVEL 1)		Cyc. (LEVEL 2)	
		mAP	mAPH	mAP	mAPH	mAP	mAPH	mAP	mAPH	mAP	mAPH	mAP	mAPH
PointPillars	Voxel-based	60.67	59.79	52.78	52.01	43.49	23.51	37.32	20.17	35.94	28.34	34.60	27.29
SECOND	Voxel-based	68.03	67.44	59.57	59.04	61.14	50.33	53.00	43.56	54.66	53.31	52.67	51.37
Part-A ²	Voxel-based	71.82	71.29	64.33	63.82	63.15	54.96	54.24	47.11	65.23	63.92	62.61	61.35
PV-RCNN	Point-Voxel	<u>74.06</u>	<u>73.38</u>	<u>64.99</u>	<u>64.38</u>	62.66	52.68	53.80	45.14	63.32	61.71	60.72	59.18
IA-SSD (Ours)	Point-based	70.53	69.67	61.55	60.80	<u>69.38</u>	<u>58.47</u>	<u>60.30</u>	<u>50.73</u>	<u>67.67</u>	<u>65.30</u>	<u>64.98</u>	<u>62.71</u>

Table 7. Efficiency comparison of different methods on the KITTI *validation* set. Here, “Mem.” and “Paral.” denote the GPU memory footprint per frame during inference and the maximum number of batches that can be parallelised on one RTX2080Ti (11 GB). “Speed[⊥]” and “Speed[⊤]” represent inference speed when processing one frame or fully loaded GPU memory, while[†] indicates dividing the scene into four parallel parts to speed up the first sampling layer. For a fair comparison, we also report each input scale of voxels/points according to their official setting.

Method	Mem.	Paral.	Speed [⊥]	Speed [⊤]	Input scale
PointPillars ⁵⁶	354 MB	28	48	58	2~9k
SECOND ⁵⁵	710 MB	14	30	40	11~17k
TANet ⁶¹	3000 MB	3	20	28	<12k
3DSSD ⁵⁰	502 MB	19	11	28	16384
PointRCNN ⁶⁸	560 MB	18	10	14	16384
Part-A ²⁶²	702 MB	13	12	19	11~17k
PV-RCNN ⁶⁵	1223 MB	8	8	10	11~17k
IA-SSD (Ours)	102 MB	100	23/48[†]	83	16384

5. Conclusion

In this chapter, we present a comprehensive overview of various heuristic and learning-based sampling methods for 3D point clouds. We also illustrate the applications of point cloud sampling in downstream tasks, particularly focusing on semantic segmentation and object detection. Specifically, we introduce two dedicated algorithms, RandLA-Net and IA-SSD, in which sampling strategies play a critical role in achieving optimal performance.

Overall, sampling is an indispensable step in efficient point cloud processing, which requires meticulous design and adaptation to meet the specific needs of the task at hand. Not only does the choice of the sampling strategy have profound implications for the overall efficiency and effectiveness of point cloud processing, but the ancillary modules should also be adapted to the chosen sampling method to achieve better results. These insights are supported by the outcomes from RandLA-Net and IA-SSD, in which well-crafted sampling strategies were instrumental in achieving their respective levels of performance.

Despite the progress made in this field, several potential avenues for future research could build upon our work:

- **Balancing universality and task orientation in sampling:** In an ideal scenario, a point cloud sampling strategy should be universal, i.e., applicable across various tasks, models, and datasets without modification. Nonetheless, different tasks inherently have distinct preferences for sampling strategies. Consequently, data-driven task-oriented sampling strategies are also desirable. A promising future research direction involves developing sampling algorithms that combine the merits of both universality and task orientation. For instance, the recently proposed Meta-Sampler¹² is a near-universal sampler that first trains on multiple tasks and datasets to learn a relatively general sampling strategy and then rapidly adapts to various tasks, models, and datasets using meta-learning techniques.
- **Efficient and scalable sampling strategies for large-scale 3D point clouds:** Real-time processing is essential for numerous applications, such as autonomous navigation or robotics. As sampling is a frequently employed operation in point cloud networks, developing a sampling strategy capable of processing large-scale point clouds in real time while maintaining high precision, efficiency, and scalability remains an open research question.
- **Robustness to noise and occlusions:** Point cloud data often contain noise and suffer from occlusions, which can negatively impact sampling methods. To date, limited research has been dedicated to developing sampling techniques that are robust to these challenges, ensuring consistent performance under varying conditions.
- **Explainable and interpretable sampling:** Developing explainable and interpretable sampling techniques is crucial for understanding and verifying the behaviour of point cloud processing algorithms. Future research should focus on designing methods that provide insights into the sampling process, allowing practitioners to better understand and trust the algorithms they employ.

In conclusion, this chapter has laid the groundwork for understanding the significance of point cloud sampling methods and their applications in semantic segmentation and object detection tasks.

By providing a detailed examination of the RandLA-Net and IA-SSD algorithms, we aim to inspire future research and development in this domain, ultimately leading to advance the efficiency, accuracy, and scalability of point cloud processing techniques in various domains.

References

1. S. Biswas, J. Liu, K. Wong, S. Wang, and R. Urtasun. Muscle: Multi sweep compression of lidar using deep entropy models, *Advances in Neural Information Processing Systems* **33**, 22170–22181 (2020).
2. Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham. RandLA-Net: Efficient semantic segmentation of large-scale point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11108–11117, IEEE (2020).
3. C. R. Qi, L. Yi, H. Su, and L. J. Guibas. PointNet++: Deep hierarchical feature learning on point sets in a metric space. In *Advances in Neural Information Processing Systems* (2017).
4. H. Thomas, C. R. Qi, J.-E. Deschaud, B. Marcotegui, F. Goulette, and L. J. Guibas. KPConv: Flexible and deformable convolution for point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 6411–6420, IEEE (2019).
5. F. Groh, P. Wieschollek, and H. P. A. Lensch. Flex-convolution (million-scale point-cloud learning beyond grid-worlds). In *Asia Conference on Computer Vision*, pp. 105–122 (2018).
6. R. L. Cook, Stochastic sampling in computer graphics, *ACM Transactions on Graphics (TOG)* **5**(1), 51–72 (1986).
7. O. Dovrat, I. Lang, and S. Avidan. Learning to sample. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2760–2769, IEEE (2019).
8. A. Abid, M. F. Balin, and J. Zou. Concrete autoencoders for differentiable feature selection and reconstruction. In *International Conference on Machine Learning* (2019).
9. J. Yang, Q. Zhang, B. Ni, L. Li, J. Liu, M. Zhou, and Q. Tian. Modeling point clouds with self-attention and gumbel subset sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3323–3332, IEEE (2019).
10. K. Xu, J. Ba, R. Kiros, K. Cho, A. Courville, R. Salakhudinov, R. Zemel, and Y. Bengio. Show, attend and tell: Neural image caption generation with visual attention. In *ICML* (2015).

11. I. Lang, A. Manor, and S. Avidan. Samplenet: Differentiable point cloud sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 7578–7588, IEEE (2020).
12. T.-Y. Cheng, Q. Hu, Q. Xie, N. Trigoni, and A. Markham. Meta-sampler: Almost-universal yet task-oriented sampling for point clouds. In *Computer Vision—ECCV 2022: 17th European Conference*, Tel Aviv, Israel, October 23–27, 2022, Proceedings, Part II, pp. 694–710, Springer (2022).
13. Y. Liu, B. Fan, S. Xiang, and C. Pan. Relation-shape convolutional neural network for point cloud analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 8895–8904, IEEE (2019).
14. Y. Li, R. Bu, M. Sun, W. Wu, X. Di, and B. Chen. PointCNN: Convolution on X-transformed points. In *Advances in Neural Information Processing Systems*, Vol. 31 (2018).
15. B. Yang, S. Wang, A. Markham, and N. Trigoni, Robust attentional aggregation of deep feature sets for multi-view 3D reconstruction, *International Journal of Computer Vision* **128**(1), 53–73 (2020).
16. K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 770–778, IEEE (2016).
17. F. Engelmann, T. Kontogianni, and B. Leibe. Dilated point convolutions: On the receptive field size of point convolutions on 3D point clouds. In *IEEE International Conference on Robotics and Automation (ICRA)*, pp. 9463–9469, IEEE (2020).
18. I. Armeni, S. Sax, A. R. Zamir, and S. Savarese. Joint 2D-3D-semantic data for indoor scene understanding. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE (2017).
19. T. Hackel, N. Savinov, L. Ladicky, J. Wegner, K. Schindler, and M. Pollefeys, Semantic3d. net: A new large-scale point cloud classification benchmark, *ISPRS Annals of the Photogrammetry, Remote Sensing and Spatial Information Sciences* (2017).
20. J. Behley, M. Garbade, A. Milioto, J. Quenzel, S. Behnke, C. Stachniss, and J. Gall. SemanticKITTI: A dataset for semantic scene understanding of lidar sequences. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9297–9307, IEEE (2019).
21. W. Wu, Z. Qi, and L. Fuxin. PointConv: Deep convolutional networks on 3D point clouds. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9621–9630, IEEE (2018).
22. H. Zhao, L. Jiang, C.-W. Fu, and J. Jia. PointWeb: Enhancing local neighborhood features for point cloud processing. In *Proceedings of the*

- IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5565–5573, IEEE (2019).
23. W. Tan, N. Qin, L. Ma, Y. Li, J. Du, G. Cai, K. Yang, and J. Li. Toronto-3D: A large-scale mobile lidar dataset for semantic segmentation of urban roadways. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops*, pp. 202–203, IEEE (2020).
 24. X. Roynard, J.-E. Deschaud, and F. Goulette, Paris-lille-3D: A large and high-quality ground-truth urban point cloud dataset for automatic segmentation and classification, *The International Journal of Robotics Research* **37**(6), 545–557 (2018).
 25. A. Boulch, B. Le Saux, and N. Audebert, Unstructured point cloud semantic labeling using deep segmentation networks., *3DOR@ Eurographics* **3** (2017).
 26. L. Tchapmi, C. Choy, I. Armeni, J. Gwak, and S. Savarese. SEG-Cloud: Semantic segmentation of 3D point clouds. In *International Conference on 3D Vision (3DV)*, pp. 537–547, IEEE (2017).
 27. H. Thomas, F. Goulette, J.-E. Deschaud, and B. Marcotegui. Semantic classification of 3D point clouds with multiscale spherical neighborhoods. In *International Conference on 3D Vision* (2018).
 28. X. Roynard, J.-E. Deschaud, and F. Goulette, Classification of point cloud scenes with multiscale voxel deep network, *arXiv preprint arXiv:1804.03583* (2018).
 29. Z. Zhang, B.-S. Hua, and S.-K. Yeung. ShellNet: Efficient point cloud convolutional neural networks using concentric shells statistics. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1607–1616, IEEE (2019).
 30. L. Wang, Y. Huang, Y. Hou, S. Zhang, and J. Shan. Graph attention convolution for point cloud semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 10296–10305, IEEE (2019).
 31. L. Landrieu and M. Simonovsky. Large-scale point cloud semantic segmentation with superpoint graphs. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 4558–4567, IEEE (2018).
 32. G. Truong, S. Z. Gilani, S. M. S. Islam, and D. Suter. Fast point cloud registration using semantic segmentation. In *2019 Digital Image Computing: Techniques and Applications (DICTA)*, pp. 1–8 (2019).
 33. L. Tchapmi, C. Choy, I. Armeni, J. Gwak, and S. Savarese. Segcloud: Semantic segmentation of 3D point clouds. In *International Conference on 3D Vision*, IEEE (2017).

34. C. R. Qi, H. Su, K. Mo, and L. J. Guibas. PointNet: Deep learning on point sets for 3D classification and segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 652–660, IEEE (2017).
35. H. Su, V. Jampani, D. Sun, S. Maji, E. Kalogerakis, M.-H. Yang, and J. Kautz. SPLATNet: Sparse lattice networks for point cloud processing. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2530–2539, IEEE (2018).
36. M. Tatarchenko, J. Park, V. Koltun, and Q.-Y. Zhou. Tangent convolutions for dense prediction in 3D. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 3887–3896, IEEE (2018).
37. B. Wu, A. Wan, X. Yue, and K. Keutzer. SqueezeSeg: Convolutional neural nets with recurrent crf for real-time road-object segmentation from 3D lidar point cloud. In *International Conference on Robotics and Automation (ICRA)*, pp. 1887–1893, IEEE (2018).
38. B. Wu, X. Zhou, S. Zhao, X. Yue, and K. Keutzer. SqueezeSegV2: Improved model structure and unsupervised domain adaptation for road-object segmentation from a lidar point cloud. In *International Conference on Robotics and Automation (ICRA)*, pp. 4376–4382 (2019).
39. A. Milioto, I. Vizzo, J. Behley, and C. Stachniss. RangeNet++: Fast and accurate lidar semantic segmentation. In *2019 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 4213–4220, IEEE (2019).
40. R. A. Rosu, P. Schütt, J. Quenzel, and S. Behnke, LatticeNet: Fast point cloud segmentation using permutohedral lattices, *arXiv preprint arXiv:1912.05905* (2019).
41. C. Xu, B. Wu, Z. Wang, W. Zhan, P. Vajda, K. Keutzer, and M. Tomizuka. SqueezeSegV3: Spatially-adaptive convolution for efficient point-cloud segmentation. In *European Conference on Computer Vision*, pp. 1–19, Springer (2020).
42. T. Cortinhal, G. Tzelepis, and E. E. Aksoy, SalsaNext: Fast semantic segmentation of lidar point clouds for autonomous driving, *arXiv preprint arXiv:2003.03653* (2020).
43. Y. Zhang, Z. Zhou, P. David, X. Yue, Z. Xi, B. Gong, and H. Foroosh. Polarnet: An improved grid representation for online lidar point clouds semantic segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 9601–9610, IEEE (2020).
44. Q. Huang, W. Wang, and U. Neumann. Recurrent slice networks for 3D segmentation of point clouds. In *Proceedings of the IEEE/CVF*

- Conference on Computer Vision and Pattern Recognition*, IEEE (2018).
45. X. Ye, J. Li, H. Huang, L. Du, and X. Zhang. 3D recurrent neural networks with context fusion for point cloud semantic segmentation. In *European Conference on Computer Vision*, pp. 403–417, Springer (2018).
 46. X. Yan, C. Zheng, Z. Li, S. Wang, and S. Cui. Pointasnl: Robust point clouds processing using nonlocal neural networks with adaptive sampling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 5589–5598, IEEE (2020).
 47. Y. Wang, Y. Sun, Z. Liu, S. E. Sarma, M. M. Bronstein, and J. M. Solomon. Dynamic graph CNN for learning on point clouds, *Acm Transactions On Graphics (TOG)* **38**(5), 1–12 (2019).
 48. L.-Z. Chen, X.-Y. Li, D.-P. Fan, M.-M. Cheng, K. Wang, and S.-P. Lu. LSA-Net: Feature learning on point sets by local spatial attention, *arXiv preprint arXiv:1905.05442* (2019).
 49. Q. Hu, B. Yang, L. Xie, S. Rosa, Y. Guo, Z. Wang, N. Trigoni, and A. Markham. Learning semantic segmentation of large-scale point clouds with random sampling, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **44**(11), 8338–8354 (2022).
 50. Z. Yang, Y. Sun, S. Liu, and J. Jia. 3dssd: Point-based 3D single stage object detector. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11040–11048, IEEE (2020).
 51. M. Yang, Y. Wu, and G. Hua. Context-aware visual tracking, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **31**(7), 1195–1209 (2009).
 52. S. K. Divvala, D. Hoiem, J. H. Hays, A. A. Efros, and M. Hebert. An empirical study of context in object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1271–1278 (2009).
 53. C. R. Qi, O. Litany, K. He, and L. J. Guibas. Deep hough voting for 3D object detection in point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9277–9286, IEEE (2019).
 54. Y. Zhou and O. Tuzel. VoxelNet: End-to-end learning for point cloud based 3D object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 4490–4499 (2018).
 55. Y. Yan, Y. Mao, and B. Li. Second: Sparsely embedded convolutional detection, *Sensors*, 3337 (2018).
 56. A. H. Lang, S. Vora, H. Caesar, L. Zhou, J. Yang, and O. Beijbom. Pointpillars: Fast encoders for object detection from point clouds. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 12697–12705, IEEE (2019).

57. D. Zhou, J. Fang, X. Song, C. Guan, J. Yin, Y. Dai, and R. Yang. Iou loss for 2D/3D object detection. In *International Conference on 3D Vision*, pp. 85–94, IEEE (2019).
58. L. Du, X. Ye, X. Tan, J. Feng, Z. Xu, E. Ding, and S. Wen. Associate-3Ddet: perceptual-to-conceptual association for 3D point cloud object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 13329–13338, IEEE (2020).
59. C. He, H. Zeng, J. Huang, X.-S. Hua, and L. Zhang. Structure aware single-stage 3D object detection from point cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 11873–11882, IEEE (2020).
60. W. Zheng, W. Tang, S. Chen, L. Jiang, and C.-W. Fu. Cia-ssd: Confident iou-aware single-stage object detector from point cloud. In *Proceedings of the AAAI Conference on Artificial Intelligence* (2021).
61. Z. Liu, X. Zhao, T. Huang, R. Hu, Y. Zhou, and X. Bai. Tanet: Robust 3D object detection from point clouds with triple attention. In *Proceedings of the AAAI Conference on Artificial Intelligence*, pp. 11677–11684 (2020).
62. S. Shi, Z. Wang, J. Shi, X. Wang, and H. Li, From points to parts: 3D object detection from point cloud with part-aware and part-aggregation network, *IEEE Transactions on Pattern Analysis and Machine Intelligence* **43**(8), 2647–2664 (2021).
63. Y. Chen, S. Liu, X. Shen, and J. Jia. Fast point r-cnn. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9775–9784, IEEE (2019).
64. Z. Yang, Y. Sun, S. Liu, X. Shen, and J. Jia. Std: Sparse-to-dense 3D object detector for point cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1951–1960, IEEE (2019).
65. S. Shi, C. Guo, L. Jiang, Z. Wang, J. Shi, X. Wang, and H. Li. Pv-rnn: Point-voxel feature set abstraction for 3D object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10529–10538, IEEE (2020).
66. T. Jiang, N. Song, H. Liu, R. Yin, Y. Gong, and J. Yao. Vic-net: Voxelization information compensation network for pointcloud 3D object detection. In *IEEE International Conference on Robotics and Automation* (2021).
67. J. Noh, S. Lee, and B. Ham. Hvpr: Hybrid voxel-point representation for single-stage 3D object detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, IEEE (2021).
68. S. Shi, X. Wang, and H. Li. Pointrcnn: 3D object proposal generation and detection from point cloud. In *Proceedings of the IEEE/CVF*

- International Conference on Computer Vision*, pp. 770–779, IEEE (2019).
69. J. Li, S. Luo, Z. Zhu, H. Dai, A. S. Krylov, Y. Ding, and L. Shao, 3D iou-net: Iou guided 3D object detector for point clouds, *arXiv preprint arXiv:2004.04962* (2020).
 70. W. Shi and R. Rajkumar. Point-gnn: Graph neural network for 3D object detection in a point cloud. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 1711–1719 (2020).
 71. P. Sun, H. Kretzschmar, X. Dotiwalla, A. Chouard, V. Patnaik, P. Tsui, J. Guo, Y. Zhou, Y. Chai, B. Caine, *et al.* Scalability in perception for autonomous driving: Waymo open dataset. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 2446–2454, IEEE (2020).

This page intentionally left blank

Chapter 11

Efficient 3D Representation Learning for Medical Image Analysis

Yucheng Tang^{*,§,¶}, Jie Liu^{†,||}, Zongwei Zhou^{‡,**},
Xin Yu^{§,††}, and Yuankai Huo^{§,‡‡}

^{*}*Nvidia, Santa Clara, USA*

[†]*City University of Hong Kong, Hong Kong, China*

[‡]*Johns Hopkins University, Baltimore, USA*

[§]*Vanderbilt University, Nashville, USA*

[¶]*yucheng.tang@vanderbilt.edu*

^{||}*jliu.ee@my.cityu.edu.hk*

^{**}*zzhou82@jh.edu*

^{††}*xin.yu@vanderbilt.edu*

^{‡‡}*yuankai.huo@vanderbilt.edu*

1. Introduction

Volumetric quantification of medical images plays a crucial role in the development, discovery, and assessment of anatomical mechanisms. Modern machine learning approaches¹ have become increasingly popular in medical image analysis and formed the foundation of understanding medical representations. Medical data, such as radiology images, computed tomography (CT), and magnetic resonance imaging (MRI), are typically produced in cross-sectional views of regions, which are then reconstructed into a 3D image. This non-invasive imaging technique generates detailed spatial context information

about internal structures. Accurately measuring spatial contexts can be crucial for making clinical decisions² and monitoring disease progression.

Medical image analysis involves fundamental tasks such as image segmentation, registration, detection, classification, synthesis, and enhancement which are essential quantification tools in enabling the characterisation of spatial context. For example, abdominal organ assessments require accurate 3D segmentation^{3,4} for prognosis and diagnosis. On the other hand, medical image registration facilitates the study of brain patterns through 3D correlations. These pioneering studies allow researchers to investigate imaging biomarkers of diseases,^{5–10} such as metabolic syndromes and brain malfunctions, and significantly advanced our understanding¹¹ of various medical conditions.

3D medical data takes many forms such as its original voxel-based images, surfaces, meshes, and point clouds. These formats have been widely used to represent anatomical structures or abnormalities.¹² Meshes are employed to model complex surfaces¹³ within human body, such as organs, vascular structures, or tumors. They consist of groups of vertices, edges, and faces that form geometric representations. In brain imaging, surfaces and meshes are widely utilised to present cortical tissues, and it's straightforward to register cortical surfaces for brain mapping studies. In addition, point clouds are one of the rising form of 3D medical images.¹⁴ They are efficient in representing large-scale unstructured volumetric data, such as CT and MRI. Point clouds are suitable for tasks, such as registration, when aligning multiple medical images. Point cloud-based methods have shown their promise in applications like object detection, lesion segmentation, and anatomical landmark localisation. To mitigate data scarcity, domain adaptation, semi-supervised learning approaches can play a crucial role in addressing the limitation.

Huo *et al.*¹⁵ proposed a self-ensembling adversarial domain adaptation method for segmenting CT and MRI. Their approach incorporated an adversarial learning module to align the feature distributions between the source and target domains while leveraging the generative adversarial models to improve model generalisation. Similarly, Dou *et al.*¹⁶ introduced a domain adaptation framework for cross-modality medical image segmentation, where a modality-specific adversarial loss was used to adapt the network to the target domain. Lee *et al.*¹⁷ proposed a semi-supervised learning

framework for medical image quality assurance, combining labelled and unlabelled data to boost model performance that effectively leveraged the unlabelled data. Similarly, Wang *et al.*¹⁸ introduced a semi-supervised framework to reduce dependence on labelled data by injecting multi-task learner and contrastive learning. These methods can well improve the models that demand large scale of annotated data.

Despite the significant progress in medical image analysis, understanding large spatial data remains a persistent challenge. Medical 3D images are typically high resolution, exhibit high anatomical variability, and are produced at a large scale. This makes it difficult to learn 3D representations robustly and efficiently from large-scale heterogeneous data. However, artificial intelligence (AI) presents an opportunity to overcome these challenges. AI models have the capability to model heterogeneity, variability, and long-range dependencies by rectifying inter- and intra-subject^{19,20} data. This enables researchers to better understand the underlying mechanisms of various medical conditions, leading to improved diagnosis, prognosis, and treatment outcomes.

Nevertheless, it is quite formidable to explore an efficient learning paradigm of 3D medical imaging data. Deep learning, a powerful tool for extracting features from spatial images, has been growing in recent decades. Initially, researchers use the 2D convolutional neural networks (CNNs) to solve volumetric radiographic scans in a slice-wise solution. However, 2D backbones^{21,22} usually neglect the spatial context in the sagittal and coronal views, yielding insufficient performance when delineating organs, vessels, or abnormalities. To capture additional contexts, the tri-planar architectures are proposed to exploit combinations of three views (axial, sagittal, and coronal) for each voxel. These approaches^{23,24} are dubbed as 2.5D methods that used sequences of 2D slices for learning spatial information. However, by constraining field of interest (ROI), 2.5D methods are vulnerable to long-range dependencies, especially for high-resolution CT/MRI scans. To leverage spatial context in medical images, 3D models^{25–27} are proposed to address basic medical image analysis tasks, including segmentation, registration, detection, and enhancement. Benefiting from the fast development of hardware (e.g., larger memory of GPU), 3D deep learning models can fit volumetric patches of CT/MRI scans. Due to the inherent variance of anatomies, scientists observe that significant complications in transferring 3D deep learning models^{25,28}

to the medical imaging domain can be interpreted by scales. Unlike natural images or video processing, features are common of a fixed scale, and modelling medical images requires dense prediction from multi-scale and hierarchical representations.

In the realm of medical image analysis, valuable insights are typically extracted from diverse imaging modalities, such as Computer Tomography (CT), Magnetic Resonance Imaging (MRI), Positron Emission Tomography (PET), and Single Photon Emission Computed Tomography (SPECT). The integration of these modalities has garnered significant interest for its potential to enhance diagnostic decision-making. By providing multi-modal imaging data, a comprehensive assessment of a patient's structural characteristics and soft tissue can be achieved. Each imaging modality exhibits unique advantages and can capture distinct characteristics that collectively contribute to a comprehensive understanding of medical conditions. Researchers devoted efforts to fusion strategies regarding different imaging modalities. The primary objective of image fusion is to leverage contrast and complementary information from multiple modalities, thereby overcoming individual modality limitations and improving the accuracy of diagnostic outcomes. Previous research works have delved into multi-modal medical image fusion techniques. Fatma *et al.*²⁹ introduced a classification of medical image registration, emphasising medical image fusion and its relevance to current disease-related research. Bikash *et al.*³⁰ conducted a comprehensive comparison of region-based image fusion techniques using various fusion quality metrics, encompassing multi-modal medical images, infrared rays, visible image fusion, multi-focus image fusion, and more. Jiao *et al.*¹¹⁶ provided a detailed explanation of the steps involved in multi-modal medical image fusion, including image fusion decomposition, reconstruction, and fusion rules, further contributing valuable insights into the fusion process. Raju *et al.*³¹ introduced a co-heterogeneous method with more than 5 multi-source and multi-phase CT imaging data for liver and lesion segmentation.

Modern, regarding long-range dependencies, motivated by natural language processing³² (NLP) and Vision Transformers^{33,34} (ViT) are proposed and exhibit potential in solving large-scale computer vision problems. The core principle of transformer-based networks is to create sequences of 3D tokens and employ the self-attention mechanism

for representation learning. The new perspective^{35,36} explores the effectiveness of vision transformers in analysing medical images. The homogeneity and heterogeneity of medical image representations are important factors to consider, and ViT-based methods provide scalable solutions to these challenges, such as variations in physical morphology, scanning protocols, or modalities.

Transformers demonstrate exceptional performance for fundamental representation learning and are effective in exploiting large-scale data for supervised³³ and self-supervised³⁷ pre-training. However, adapting transformer models from natural images to medical domains is a significant challenge due to the large context gap. Collecting expert annotations for medical images is expensive and time-consuming. Therefore, transfer learning, which reuses features from pre-trained weights on related tasks, can be employed to improve efficiency and robustness in medical image analysis. Self-supervised learning approaches, such as self-supervised pre-trained Swin UNETR³⁸ and masked autoencoder,³⁹ have been proposed to leverage performance without manual labels. In addition, the contrastive language-image pre-training (CLIP)⁴⁰ reveals the connection between organs and tumors, and its fixed-length property can improve the pre-trained model for open-vocabulary segmentation. This allows for addition of new classes without the risk of forgetting previously learned ones.

In this chapter, with respect to efficient 3D representation learning for medical image analysis, the core concepts are as follows:

- designing data-efficient models for training 3D medical images with less computational complexity,
- scaling 3D deep learning models for robust large-scale medical image pre-training,
- harnessing semantic relationships between anatomically spatial contexts, or
- prompting deep learning framework towards transferability, generalisability, and extensibility in 3D medical image analysis.

With a major focus on foundation models and medical segmentation, this chapter includes key components, such as 3D hierarchical transformers, self-supervised pre-training, transfer learning, and prompt tuning for 3D medical image analysis.

2. Transformers for 3D Medical Image Analysis

Applying transformer to 3D medical images shows unique challenges. One prominent limitation is the transformers' demand for large amounts of labelled medical data. Transformers, renowned for their exceptional performance in language and 2D vision tasks, have demonstrated their efficacy in handling vast datasets. However, in the field of medical imaging, acquiring annotated 3D labels is arduous and time-consuming. Medical image annotation requires extensive expert knowledge, involving specialised training and considerable effort to ensure accurate and reliable labels. Moreover, due to privacy and ethical concerns, access to large, comprehensive, and diverse 3D medical datasets is often restricted. Therefore, training transformer models for 3D medical image analysis becomes challenging, hindering the full realisation of their potential in medical imaging applications.

Another limitation arises from the intrinsic disparities between natural images and 3D medical images. As transformers were designed to process 2D input sequences. Unlike 2D images, which are smaller and comprise fixed-sized grids, 3D medical images encompass volumetric tokens characterised by irregular voxel spacing, varying resolutions, and intricate spatial relationships among anatomical structures. Original transformer receipt may struggle to capture the spatial contexts of medical images. Moreover, medical images often exhibit variant contrast, noise, and artefacts, posing a challenge to transformer-based models in extracting meaningful representations. Thus, it becomes crucial to design and adapt transformer architectures for 3D medical images. By addressing these limitations, the integration of transformer models into 3D medical image analysis has the potential to improve healthcare by enabling more accurate diagnosis, treatment planning, and disease monitoring.

2.1. *Multi-scale Representation Learning*

The merging of features from various layers can enhance representation learning by aggregating multi-level information. To model hierarchical features, multi-scale representations are employed, such as U-Net²⁵ and pyramid²⁸ networks. These networks combine spatial and semantic information using two structures: iterative deep layer

aggregation, which fuses multi-scale information, and hierarchical deep aggregation, which fuses representations across channels. In addition to using a single network, other methods, such as nested UNet++,⁴¹ nnUNets,⁴² coarse-to-fine,⁴³ and Random Patch,⁴⁴ use multi-stage pathways to progressively enrich different semantic levels of features with cascaded networks.

While CNN-based techniques are adept at representation learning, they have limitations in capturing long-range dependencies beyond their localised receptive fields. This can result in inadequate performance for structures with varying scales and shapes, such as abdominal organs, vessels, tumors, and brain tissues of different sizes. The emerging transformer-based architecture leverages the self-attention mechanism for volumetric medical image analysis. It involves reformulating 3D volumes as 1D sequence-to-sequence prediction problems and using transformer blocks as encoders to learn contextual information from embedded input patches. In particular, hierarchical transformers^{34,45} have achieved advancing performance by modelling long-range dependencies and visual features at varied scales. However, transformers lack some of the inductive biases inherent to CNNs, such as translation equivariance and locality, and therefore do not generalise well when trained on insufficient amounts of data. To optimise the model, especially for segmentation tasks, decoders are typically involved with a hybrid design. The learned representations from the transformer encoder are merged with the CNN-based architecture through skip connections, resulting in a more efficient and effective model.

2.2. Model Architecture

Self-attention-based transformer, as shown in Figure 1, is a category of deep learning techniques^{46–48} as expounded upon by language models. The self-attention mechanism, initially devised as a means of aligning various sequence positions to compute a hidden state, has become an aspect of sequence modelling methods. The transformer’s capacity to learn dependencies in long-range has led to its use in processing sequential signals, including in natural language processing (NLP)⁴⁸ and speech⁴⁹ processing. The transformer comprises an encoder blocks,⁴⁶ with each component consisting of a set of L blocks featuring the self-attention module. In this chapter,

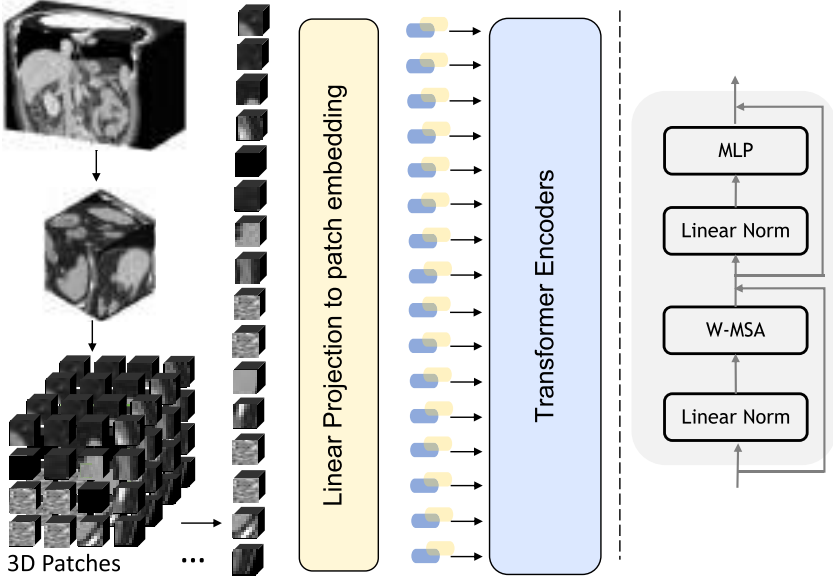


Fig. 1. Transformer encoder for 3D medical images. The 3D volume is partitioned to 3D patches and is directly encoded by patch embedding and position embedding layers, followed by several transformer blocks. The MLP denotes the multi-layer perceptron, and W-MSA is the windowed multi-head self-attention.

we describe transformer methodologies with the input of 3D medical images. Assuming the input signals are 3D volumes, where a 1D sequence of 3D tokens $\mathbf{x} \in \mathbb{R}^{H \times W \times D \times C}$. The resolution of the input sequence is given by (H, W, D) , with C input channels. The input sequence is divided into flattened and non-overlapping patches $\mathbf{x}_v \in \mathbb{R}^{N \times (P^3 \cdot C)}$, where (P, P, P) denotes the dimensions of each 3D patch, and $N = (H \times W \times D)/P^3$ is the length of the resulting sequence.

Subsequently, a linear layer can project the tokens into a K embedding space. In order to preserve the spatial information of the extracted patches, a 1D learnable positional embedding $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{N \times K}$ is added to the projected patch embedding features $\mathbf{E} \in \mathbb{R}^{(P^3 \cdot C) \times K}$ according to

$$\hat{\mathbf{x}} = \{X_1 \mathbf{E}, X_2 \mathbf{E}, \dots, X_N \mathbf{E}\}, \quad \mathbf{E} \in \mathbb{R}^{P^2 \times D}, \quad (1)$$

in the segmentation task or pre-training workflows, [class] token is usually not considered in the sequence of embeddings. Then a stack of transformer blocks^{32,33} (e.g., 12 blocks for base model) comprising of

multi-head self-attention (MSA) and multi-layer perceptron (MLP) with respect to

$$\mathbf{z}'_i = \text{MSA}(\text{Norm}(\mathbf{z}_{i-1})) + \mathbf{z}_{i-1}, \quad i = 1 \dots L, \quad (2)$$

$$\mathbf{z}_i = \text{MLP}(\text{Norm}(\mathbf{z}'_i)) + \mathbf{z}'_i, \quad i = 1 \dots L, \quad (3)$$

where MLP consists two linear layers with GELU activation, Norm represents layer normalisation, i denotes the intermediate block, and L is the total number of encoding layers. These settings are similar to the original vision transformer³³ receipt.

MSA are n designed in parallel to produce output vectors. The self-attention is derived by calculating the similarity between two vectors in $\mathbf{z}$. Then, the similarity and correlation between query Q and key K is normalised, attaining an attention distribution $A \in \mathbb{R}^{n \times n}$:

$$A(Q, K) = \text{Softmax} \left(\frac{Q \times K^\top}{\sqrt{d}} \right). \quad (4)$$

The final output is self-attention on the input matrix x , which is projected onto query, key, and value matrices (W_i^q , W_i^k , and W_i^v) for each of the h attention heads. The self-attention outputs for each head (Z_i) are concatenated and projected onto the output matrix (W^o) to produce the final output. The MSA operation applies the concatenation and projection to the query (Q), key (K), and value (V) matrices, enabling the computation of similarities among the context features. The projection matrix W^o has dimensions $\mathbb{R}^{hd \times c}$. Final MSA can be formulated as follows:

$$\begin{aligned} Z_i &= \text{SA}(X \times W_i^q, X \times W_i^k, X \times W_i^v), \\ \text{MSA}(Q, K, V) &= \text{Concat}[Z_1, \dots, Z_h] \times W^o. \end{aligned} \quad (5)$$

2.3. Case Study 1: Swin-Transformers for 3D Images

The Swin transformer (SwinT)³⁴ is one of the first transformer-based methods proposed to integrate hierarchical features. This approach utilises a shifted window mechanism on self-attention, where the feature dimensions K vary at different blocks and scales. In practice, learning 3D medical images requires global and local feature embeddings, but the vision transformer (ViT)³³ only partitions an image

into patches and encodes them into fixed dimensions of representations. In contrast, SwinT applies a shift of the window partition between consecutive self-attention blocks. This approach has been successfully applied in various fields, including object detection,⁵⁰ semantic segmentation,⁵¹ and video understanding,⁵² due to its outstanding performance.

Swin UNet TRansformers: The Swin UNet TRansformer³⁸ (Swin UNETR), designed for 3D medical image analysis, adopts an approach of utilising 3D patches and skip connections at multiple resolutions to connect a CNN-based decoder. The overall architecture of Swin UNETR is depicted in Figure 2, and the details of its encoder and decoder are described. Assuming that the input to the encoder is an image volume $\mathcal{X} \in \mathbb{R}^{H \times W \times D \times S}$, a 3D token with token resolution (H', W', D') has a dimension of $H' \times W' \times D' \times S$. The patch partitioning layer creates a sequence of 3D tokens with a size of $\frac{H}{H'} \times \frac{W}{W'} \times \frac{D}{D'}$, which is then projected into a C -dimensional space via an embedding block. To model token interactions efficiently in 3D, the input volumes are partitioned into non-overlapping windows, and local self-attention³⁴ is computed in each region. At block l , a window size $M \times M \times M$ is used to uniformly partition a 3D patch into $\left\lceil \frac{H'}{M} \right\rceil \times \left\lceil \frac{W'}{M} \right\rceil \times \left\lceil \frac{D'}{M} \right\rceil$ windows. In the consecutive layer $l + 1$, the partitioned windows are shifted by $(\lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor, \lfloor \frac{M}{2} \rfloor)$ pixels. The outputs of the encoder in layers l and $l + 1$ are shown as follows:

$$\begin{aligned}
 \hat{z}^l &= \text{W-MSA}(\text{LN}(z^{l-1})) + z^{l-1}, \\
 z^l &= \text{MLP}(\text{LN}(\hat{z}^l)) + \hat{z}^l, \\
 \hat{z}^{l+1} &= \text{SW-MSA}(\text{LN}(z^l)) + z^l, \\
 z^{l+1} &= \text{MLP}(\text{LN}(\hat{z}^{l+1})) + \hat{z}^{l+1},
 \end{aligned} \tag{6}$$

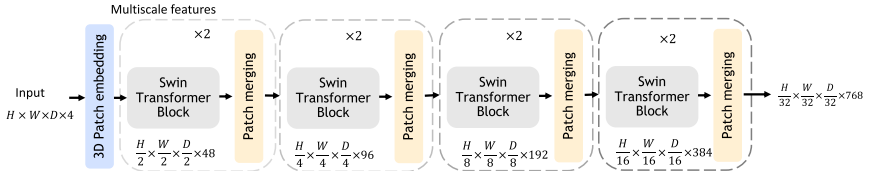


Fig. 2. Swin transformer blocks for 3D input data; the number of encoder blocks can be vary according to the embedding scales.

where W-MSA and SW-MSA are vanilla and window partitioning MSA modules, $\hat{z}^l$ and $\hat{z}^l$ are the embedding of W-MSA and SW-MSA; LN, Same to the original 3D vision transformer method, MLP are layer normalisation and MLP. The SW-MSA forms the basic 3D cyclic-shifting for hierarchical feature computation of shifted windowing. In the 3D medical segmentation model³⁸ SwinUNETR, SwinT encoder uses a patch size of $2 \times 2 \times 2$ with a feature dimension of $2 \times 2 \times 2 \times 1 = 8$ for single input channel CT images and a $C = 48$ feature space. The encoder architecture comprises 4 stages, with each stage consisting of 2 transformer blocks, resulting in a total of $L = 8$ layers. To reduce the resolution by a factor of 2 between each stage, a patch merging layer is performed. In Stage 1, transformer blocks and a linear embedding layer maintain the number of tokens at $\frac{H}{2} \times \frac{W}{2} \times \frac{D}{2}$. Additionally, a patch merging layer groups patches with a resolution of $2 \times 2 \times 2$ and concatenates them, resulting in a $4C$ -dimensional feature embedding. A linear layer is then employed to downsample the resolution by reducing the dimension to $2C$. The same approach is followed in Stages 2 through 4, where the resolutions are $\frac{H}{4} \times \frac{W}{4} \times \frac{D}{4}$, $\frac{H}{8} \times \frac{W}{8} \times \frac{D}{8}$, and $\frac{H}{16} \times \frac{W}{16} \times \frac{D}{16}$, respectively. These hierarchical representations of the encoder at different stages are useful in downstream applications, such as segmentation for multi-scale feature extraction. Note the design of Swin transformer encoder doesn't follow the original design which involves 12 blocks. To accommodate 3D medical volumes as input and computational efficiency, 4 down-sampling stages and a total of 8 Swin transformer layers are used.

SW-MSA and W-MSA: The transformer block used in Swin transformer replaces the standard multi-head self-attention (MSA) module used in ViT with a Window MSA (W-MSA) and a Shifted Window MSA (SW-MSA) module. They represent two variations of the self-attention mechanism integrated into transformer-based models for vision tasks. Their difference is mainly from their approaches to processing input sequences. WMSA operates by employing multiple attention heads over fixed non-overlapping windows within sequences, and it focus on local context within each window independently. Shifted window (Swin) MSA introduces a pivotal distinction by shifting the window positions for self-attention heads by the overlapped windows. This facilitates SW-MSA in capturing

a more global contextual and attention transitions between adjacent windows. SW-MSA stands out for its simplicity and computational efficiency of local information communication due to the use of non-overlapping windows. SW-MSA excels in effectively handling long-range dependencies and preserving superior context awareness, rendering it particularly advantageous for managing larger sequences or tasks that necessitate an extensive comprehension of the medical data, especially in 3D volumes.

2.4. Case Study 2: Hierarchical Transformers with Spatially Aggregated Blocks

One of the significant challenges when using hierarchical transformer models for 3D medical image analysis tasks is their computational efficiency. For example, Swin transformers use shifted window self-attention to process voxel pairs in 3D high-resolution images, which can be computationally expensive. To address this issue, NesT⁴⁵ proposes a method that promotes token interaction across patches by stacking canonical transformer blocks. The cross-block self-attention operates on non-overlapping image blocks within stages and merges representation hierarchically using an aggregation function. This approach improves computation efficiency by reducing the communication overhead among partitioned windows.

The UNesT model⁵³ is a novel approach to medical image segmentation that combines the strengths of transformer-based and convolutional neural network (CNN) architectures. It addresses the limitations of existing methods by integrating a hierarchical transformer-based encoder with a multi-scale CNN decoder. This approach allows UNesT to effectively learn local and global spatial features while also being computationally efficient. One of the key features of UNesT is the use of an adaptive local attention module, which enables the model to focus on important local features while suppressing noise and irrelevant information. This module is integrated into the transformer encoder, allowing UNesT to capture long-range dependencies and contextual information. UNesT has shown promising performance on several challenging medical image segmentation tasks, including whole-body CT⁵⁴ and MRI brain⁵⁵ segmentation. It outperformed state-of-the-art methods in terms of accuracy and computation efficiency. UNesT’s combination of strong

local spatial representation learning capability and efficient computation makes it a promising candidate for a wide range of medical image analysis applications.

Hierarchical 3D transformer encoder: Let $\mathcal{X} \in \mathbb{R}^{H \times W \times D \times C}$ be a 3D input image shown in Figure 3, and let the volumetric token have a patch size of $S_h \times S_w \times S_d \times C$. UNesT model projects the 3D tokens onto a size of $\frac{H}{S_h} \times \frac{W}{S_w} \times \frac{D}{S_d} \times C'$ in the patch projection stage, where C' is the embedded channels. To achieve efficient non-local communication, the nested transformer partition projected sequences of embeddings into blocks with a resolution of $\mathcal{X} \in \mathbb{R}^{b \times T \times n \times C'}$, where T is the number of blocks at the current hierarchy, b is the batch size, and n is the total length of sequences. The blockify mechanism is shown in Figure 4. The dimensions of the embeddings follow $T \times n = \frac{H}{S_h} \times \frac{W}{S_w} \times \frac{D}{S_d}$. Each block is separated into sequential transformer layers that consist of MSA, MLP, and layer normalisation (LN). Before the blocked transformers, learnable position embeddings are added to the sequences for capturing spatial relations. The final output of encoder layers $t - 1$ and t are derived as follows:

$$\begin{aligned} \hat{z}^t &= \text{MSA}_{\text{HRCHY}_1}(\text{LN}(z^{t-1})) + z^{t-1}, t = 1 \dots L, \\ z^t &= \text{MLP}(\text{LN}(\hat{z}^t)) + \hat{z}^t, t = 1 \dots L, \end{aligned} \quad (7)$$

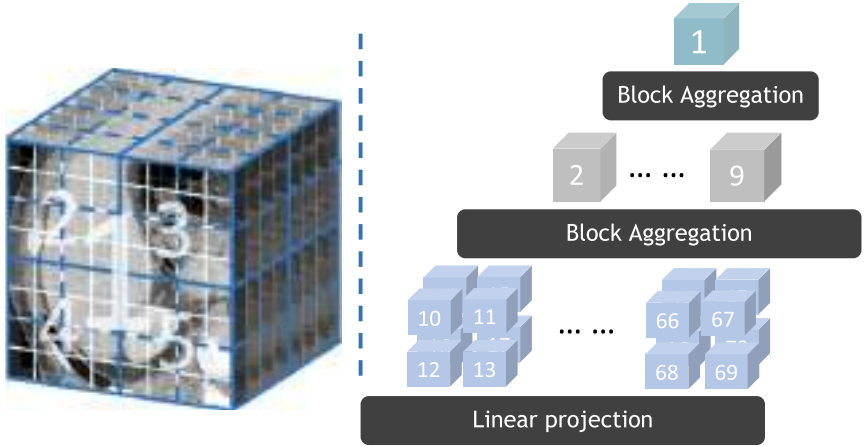


Fig. 3. Hierarchical transformer with aggregated blocks for medical data; three consecutive hierarchies are demonstrated.

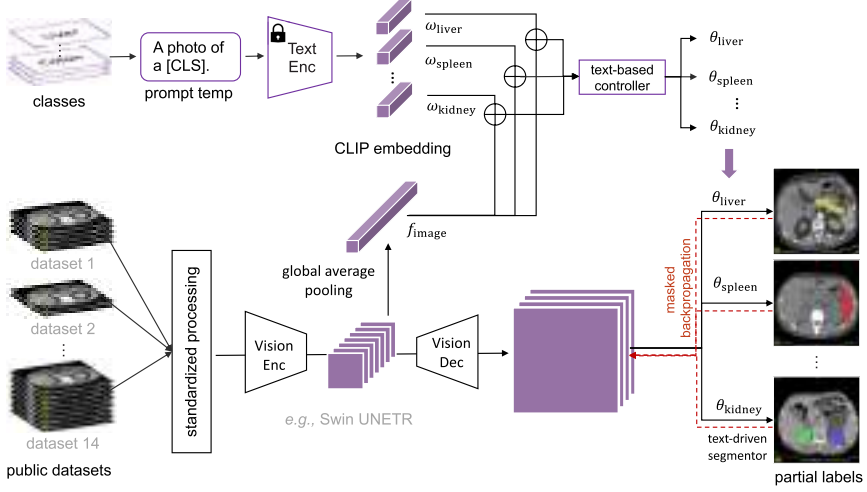


Fig. 4. The Universal Model proposed by Liu *et al.*,⁵⁶ consisting of a text branch and a vision branch.

where the MSA layer of hierarchy l is denoted as $\text{MSA}_{\text{HRCHY}_1}$, $\hat{z}^t$ and z^t represent the output representations of MSA and MLP, respectively, and L denotes the number of transformer layers. $\text{MSA}_{\text{HRCHY}_1}$ are in parallel to all partitioned blocks:

$$\begin{aligned} \text{MSA}_{\text{HRCHY}_1}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) &= \text{Stack}(\text{BLK}_1, \dots, \text{BLK}_T), \\ \text{BLK} &= \text{Attention}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \text{Softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{\sigma}}\right)\mathbf{V}, \end{aligned} \quad (8)$$

where σ is the size of each vector. The shared parameters of all blocks at each level of the hierarchy, given the input $\mathcal{X}$, result in hierarchical representations without increasing complexity. In the UNesT model, the spatial nesting operations are applied to 3D blocks, where each volume block is independently modelled and information across blocks is communicated through the aggregation module. The feature map initially has a size of $\frac{H}{S_h} \times \frac{W}{S_w} \times \frac{D}{S_d} \times C'$ at the first hierarchy and is downsampled by a factor of 2 for each dimension at the start of the next stage. Then, the feature maps are blockified into the prior size of $T \times n \times C'$. The feature size is reduced by $2 \times 2 \times 2$, the sequence length n remains the same, and the number of blocks T reduces by a factor of 8 at each hierarchy. After the transformer layer, the embeddings are unblocked to the input feature size using the block

aggregation operation. UNesT network contains 3 stages, resulting in a total of 64, 8, and 1 blocks in each hierarchy. In the volumetric self-attention, the encoded features are aggregated among adjacent block representations. The aggregation modules are designed and used in the 3D scenario to leverage local attention.

2.5. Medical Segmentation Applications

Medical image analysis is a diverse field that encompasses several tasks, including segmentation, recognition/classification/diagnosis, detection, registration, reconstruction, and enhancement. Among these tasks, medical image segmentation is one of the fundamental tasks since it provides a quantitative tool for volumetric assessment of biomarkers, abnormalities, and anatomical characterisations. With the advent of deep learning, modern medical segmentation algorithms have emerged and achieved outstanding performance. In recent years, transformer-based and data-efficient hierarchical approaches have achieved state-of-the-art segmentation results on several foundational benchmark datasets of 3D CT and MRI segmentation.

2.5.1. Multi-organ CT segmentation

Abdominal segmentation is an essential clinical investigation tool for internal body components, such as fat, bones, organs, blood vessels, and other soft tissues. Efficiently and accurately segmenting anatomies in CT scans can be significant. One of the prevailing benchmarking datasets, beyond the cranial vault (BTCV⁵⁴), is that the dataset consists of 100 contrast-enhanced⁵⁷ volumes with 13 labelled anatomical structures including the spleen, right kidney, left kidney, gallbladder, esophagus, liver, stomach, inferior vena cava (IVC), portal and splenic veins (PSV), pancreas, right adrenal gland, and left adrenal gland. Figure 5 shows the 3D transformer model (Swin UNETR³⁸) segmentation results on the multi-organ segmentation.

2.5.2. Tumor CT segmentation

Tumor segmentation is a very challenging task in medical image understanding; tumors are of varied shapes, morphological structures, and small volumes. The Medical Segmentation Decathlon

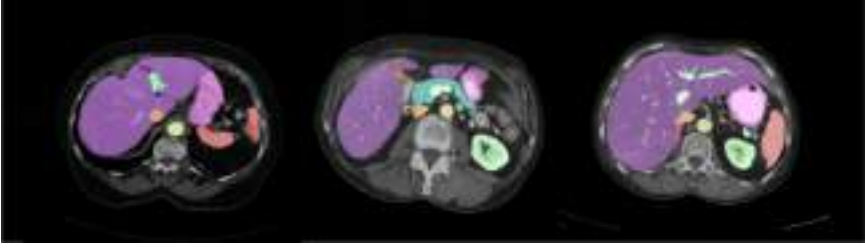


Fig. 5. Multi-organ segmentation results, qualitative segmentation masks predicted by Swin UNETR.³⁸

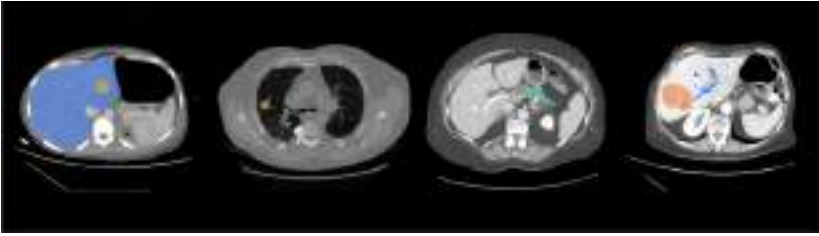


Fig. 6. Tumor segmentation visualisation including liver tumor, lung nodule, pancreas tumor, and hepatic vessel tumor. Images and segmentation masks from MSD challenge⁵⁸ and SwinUNETR.³⁸

(MSD) dataset⁵⁸ includes six tumor segmentation tasks related to various organs, such as liver and tumor, lung nodule, pancreas tumor, hepatic vessels tumor, and colon tumor. Qualitative results are shown in Figure 6. Medical image segmentation techniques can be effectively and adaptably evaluated by utilising the MSD challenge as a comprehensive standard.

2.5.3. Whole brain MRI segmentation

Whole brain segmentation is a fundamental tool in scientific and clinical research for quantitatively evaluating human brain structures. This technique offers a non-invasive means of assessing brain structure using MRI acquired in a clinical setting. Whole brain segmentation is a challenging task due to the dense prediction of a vast array of heterogeneous tissues (>130 structures) within a relatively small volume, as shown in Figure 7. The scarcity of manually labelled

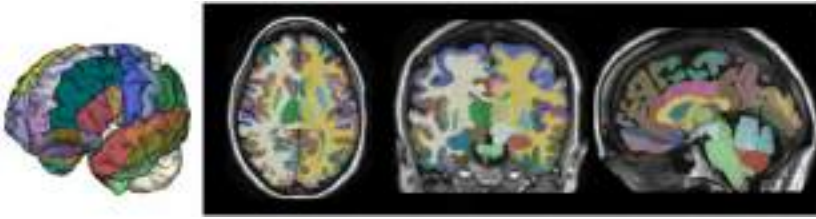


Fig. 7. Whole brain segmentation with 133 foreground structures predicted by single UNesT⁵³ model.

ground data further compounds this challenge, making accurate segmentation a difficult and time-consuming task.

2.6. *Scaling Medical Transformers*

Model scale, data scale, and resource scale are key factors in achieving better performance for deep learning models in medical image analysis. It is important to understand the complexity and scale of models when designing training and evaluation workflows. Larger models can be more robust to a larger scale of training data but require intensive computational resources. 3D deep learning models and transformers have shown good transferability, and their performance is associated with scaling laws. Model sizes can range from 5 million to 2 billion parameters, and datasets can vary from 10 to tens of thousands for pre-training and downstream tasks. Therefore, it is important to balance performance and resources when designing and understanding training protocols.

Observations of 3D transformers in medical image analysis include the fact that larger models typically achieve better performance for basic tasks, such as segmentation. Larger models can also benefit from additional scans, but representation learning capability can be limited by model size, as smaller models are easily saturated during training. For example, in the evaluation of the Swin UNETR model, a Swin UNETR-base network with 62M parameters achieved the best Dice score of 81.81 for the multi-organ segmentation challenge. Meanwhile, the smaller Swin UNETR-Small model, with 15.7M parameters, achieved the best Dice of 79.34, and the tiny Swin UNETR model, with only 4M parameters, attained a Dice score of 70.35.

3. Medical Image Pre-training with Self-Supervised Learning

Transformer-based models have two major benefits: First, tokenised embeddings have a larger capacity for learning long-range dependencies and can effectively capture global contextual information. Second, transformers can take advantage of large-scale pre-training. Furthermore, transformer models have demonstrated superior transferability³⁷ from pre-text tasks to downstream tasks. Studies on pre-training labelled data (supervised pre-training)^{59,60} demonstrate significant improvement. However, collecting large-scale annotated medical imaging data can be difficult, expensive, and time-consuming. Therefore, studies explore models that can learn representation efficiently through self-supervised learning. Empirical experiments have shown that transformer models excel at modelling large-scale data and preserving its representations for broader target tasks. This is particularly appealing in the medical domain, where obtaining unlabelled data is more feasible. Adapting self-supervised learning for medical image analysis is a challenging task. This is because there is a significant domain gap between medical image modalities, such as computed tomography (CT) and magnetic resonance imaging (MRI). Additionally, 3D cross-plane contextual information is usually insufficient when dealing with volumetric images. These unsolved problems drive medical image analysis to investigate self-supervised learning approaches. Along with 3D proxy tasks, pre-training learns the underlying patterns of human body structures and covers region of interest, such as head, neck, lung, abdomen, and pelvis.

3.1. *Self-Supervised Learning for 3D Medical Data*

Recent advances in self-supervised learning offer the promise of utilising unlabelled data. Self-supervised representation learning^{61–63} constructs feature embedding spaces by designing pre-text tasks. Another commonly used pre-text task is to memorise spatial context from medical images, which is motivated by image restoration. This idea is generalised to inpainting tasks^{64–66} to learn visual representations^{67–69} by predicting the original image patches. Similar efforts for reconstructing spatial context have been formulated as

solving Rubik’s cube⁷⁰ problem, random rotation prediction,^{71,72} and contrastive^{37,73} coding. Different from these efforts, more pre-training frameworks are simultaneously trained with a combination of pre-text tasks, tailored for 3D medical imaging data, and leverages a transformer-based encoder as a powerful feature extractor.

Deep neural networks have been used as the fundamental architecture^{74,75} to many target tasks, such as detection, classification, and segmentation. The models trained from expert annotated data show superior results. However, the performance of these models is common concerning the amount of training data, and different networks were designed with denser architectures, including AlexNet,⁷⁶ ResNet,⁷⁵ and DenseNet.⁷⁷ Collecting large-scale annotated datasets in the medical domain is resource-intensive. The BTCV⁵⁴ dataset, which consumes more than 10 expert’ efforts, forms 30 subjects’ multi-organ segmentation labels. To exploit the large-scale medical images and avoid extra expert efforts, self-supervised methods were proposed to learn visual and contextual features from unlabelled images.

In self-supervised learning, the models treat the learning as a supervised learning task with the generated pseudo label, which is called pre-text tasks. By modelling from pretext tasks, networks are trained by optimising designed objective functions. There are two key factors for designing pre-text tasks: (1) the augmentation/distortion can be automatically generated without manual efforts; (2) the visual features captured from the images are useful for representation learning.

Self-supervised learning and transfer learning⁷⁸ are explored in recent years and becoming the *de facto* framework⁷⁸ for medical image analysis. A pre-training workflow⁷⁹ focused on chest CT such as LUNA⁸⁰ due to its promising application of identifying lung nodules. Other approaches like denoising,⁸¹ image inpainting,⁶⁶ jigsaw puzzle,⁸² clustering,⁸³ patch shuffling,⁸⁴ and Rubik’s cube⁸⁵ are proposed, which achieved milestones in downstream tasks. More recently, the contrastive learning⁸⁶ brought insights and show its powerful capabilities in capturing visual representations. Contrastive learning is conducted between augmented data pairs, the model attempt to differentiate positive or negative pairs by minimising contrastive loss. Self-supervised pre-training workflow with transformers is shown in Figure 8.

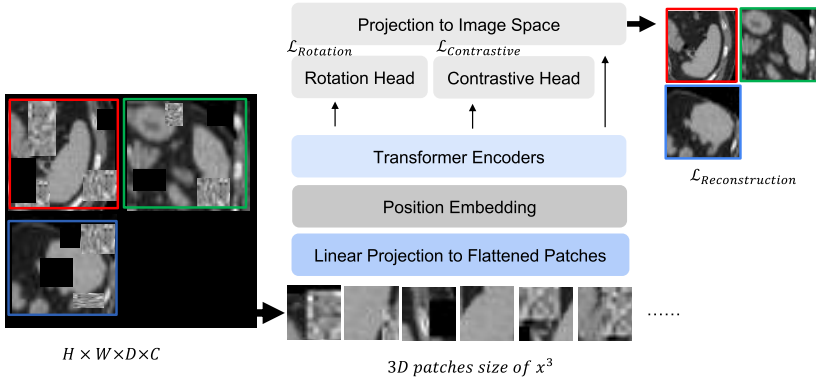


Fig. 8. A 3D medical image pre-training framework utilised by self-supervised SwinUNETR;³⁸ spatial distortion and augmentation are performed to construct multiple self-supervised learning tasks.

3.2. 3D Proxy Tasks

In this section, we describe several generic proxy tasks that are commonly used for pre-training 3D medical images.

Contrastive coding: The contrastive coding approach is a self-supervised learning strategy^{86,87} for visual representation learning. This is achieved by utilising a batch of augmented 3D volumes and maximising the mutual information between positive pairs (data-augmented samples from the same volume) while minimising the mutual information between negative pairs (volumes from the different subjects). To obtain the contrastive embedding, a linear head is attached to the encoder, which maps each augmented volume to a latent representation, denoted as v . The encoded representations are compared using cosine⁸⁶ similarities, to measure the distance between them. The 3D contrastive coding loss between a pair v_i and v_j is formally defined as follows:

$$\mathcal{L}_{\text{contrast}} = -\log \frac{\exp(\text{sim}(v_i, v_j)/t)}{\sum_k^{2N} 1_{k \neq i} \exp(\text{sim}(v_i, v_k)/t)}, \quad (9)$$

where 1 is the identifier function respect to 1 iff $k \neq i$. t denotes the normalised temperature. sim is the dot product between embeddings.

Masked autoencoder: Masked autoencoder (MAE) is a prominent proxy task designed for vision transformers. It divide a 3D image into non-overlapping patches. Subsequently, MAE randomly samples patches and then removed them, following a uniform distribution. A ratio is fixed as masked patches and the remaining ones are controlled to balance the redundancy. Similar to ViT, MAE encoder utilises linear projection and positional embeddings to embed patches, followed by multiple transformer blocks. However, MAE encoder only operates on a small subset, usually around 25% of the entire number of patches, and masked patches are removed without using any mask tokens. A side advantage is that MAE trains large amounts of data while significantly reducing the required computation resources.

Patch shuffling: Context restoration⁶⁷ is a group of self-supervised learning techniques. Given a medical image x_i , it randomly selects two isolated subvolumes and swaps their context. Then, repeat the shuffling process several times T . Similar to the inpainting task, the distorted patches are used as input, and the model takes the augmented images and restores the context back to its original pattern.

Low-level proxy tasks: Low-level vision tasks are typically referred to the processing of raw image data at a pixel level. Basic low-level tasks including denoising, filtering, enhancement, and feature extraction are exceptional visual learners for volumetric medical images. Noise and low resolution are common issues^{88,89} in medical imaging acquisition pipelines.

Masked volume inpainting: In the cutout data augmentation, regions of interest (ROIs) in a subvolume $\mathcal{X} \in \mathbb{R}^{H \times W \times D \times C}$ are randomly masked with a ratio of s . To reconstruct the original subvolume, a transpose convolution (reconstruction) head is attached to the encoder, which produces an output $\hat{\mathcal{X}}^{\mathcal{M}}$. The reconstruction objective is defined by computing the $L1$ loss between $\mathcal{X}$ and $\hat{\mathcal{X}}^{\mathcal{M}}$:

$$\mathcal{L}_{\text{inpaint}} = \|\mathcal{X} - \hat{\mathcal{X}}^{\mathcal{M}}\|_1. \quad (10)$$

Models genesis⁶⁴ and self-supervised transformer⁶⁶ adopt similar inner and outer cutout functions as reconstruction tasks are strong learners for representation learning.

Denoising: In medical image acquisitions, noise variation is common according to the scanning protocols. To utilise denoising as a pre-training task, the proxy task introduces noise to the original input and then uses the network to restore the original image from the noisy input. The most commonly encountered noise type is additive Gaussian noise, and we define the perturbed input $\hat{x}$ are as follows:

$$\hat{x} = x + N(\mu, \sigma), \quad (11)$$

where x is the input image and N denotes the normal distribution. For each subvolume, data augmentation process samples attain noise from a normal distribution. The mean and variance are controlled hyperparameters for adjusting noise levels.

Super-resolution: Image super-resolution is a technique that involves enhancing a low-resolution (LR) image to a high-resolution (HR). LR images are often characterised by zigzag edges and blurred artefacts, which makes it challenging to extract local features. In medical imaging, super-resolution is particularly useful as capturing high-resolution context can be challenging. To utilise super-resolution as a self-supervised pre-training approach, transformations are conducted to reduce the resolution of an input volume x , where an LR image $\hat{x}$ is obtained. The downsampling process is defined as follows:

$$\hat{x} = D(x, \epsilon), \quad (12)$$

where D denotes the function and ϵ is the ratio. In practice, linear interpolation is usually used for resampling. Subsequently, the training framework will restore the low-resolution image to its original high resolution.

The efficacy of self-supervise training tasks are widely discussed. Empirical studies show that pre-training with different combinations of proxy tasks can be beneficial. Apart from the multi-task learner, experiments show that the generative tasks such as masked volume inpainting and masked autoencoder are very effective at learning representations of 3D medical image, and its pre-trained weights are of most effective at performance gain in downstream tasks. Furthermore, low-level tasks of learning intrinsic context of image can be useful for foundation downstream tasks, such as medical segmentation. Contrastive learning is good at learning generic image embeddings that are of good generalisability for medical image analysis.

4. Language-Image Pre-training for Medicine

4.1. *Language-Image Pre-training*

Language-image pre-training techniques have been widely adopted in computer vision,^{90,91} as well as in medical imaging.⁹² These techniques involve training on large-scale multi-modal datasets to create joint representations of language and visual information, resulting in remarkable performance in various downstream tasks.^{93–96} The acquired representations can then be fine-tuned on specific tasks with smaller labelled datasets, making them particularly useful in 3D medical imaging representation learning. Several language-image pre-training models have been proposed in recent years, such as LXMERT,⁹⁷ CLIP,⁴⁰ and ConVIRT.⁹² These models are typically based on the transformer architecture³² and are trained on tasks that require an understanding of both textual and visual information, such as image captioning and contrastive objectives.^{98,99}

In 3D deep learning and medical imaging, language-image pre-training techniques have significant potential.^{92,100–102} They can provide a useful starting point for training models, reducing the need for large amounts of labelled data, especially for scarce 3D medical segmentation data. In addition, they enable the model to leverage implicit knowledge about the visual world and language understanding for improved performance.

4.2. *Partially Labelled Problem*

A major challenge to fine-tuning language-image pre-training models for downstream 3D segmentation tasks is the scarcity of fully labelled data.^{103–105} One reason for this challenge is the high cost of detailed per-voxel 3D segmentation annotations, which can take nearly one hour per organ for an expert annotator.

Since each institute has time, monetary, and clinical constraints, the number of CT scans in each dataset is limited, and the types of annotated organs vary significantly from institute to institute,^{58,106,107} which causes the partially labelled problem. There are four major issues: (i) Index inconsistency. The same organ can be labelled as different indexes. For example, the stomach is labelled 7' in BTCV but 5' in WORD. (ii) Name inconsistency. Naming can

be confusing if multiple labels refer to the same anatomical structure. For example, “postcava” in AMOS22 and “inferior vena cava” in BTCV. (iii) Background inconsistency. For example, when combining Pancreas-CT and MSD-Spleen, the pancreas is marked as the background in MSD-Spleen, but it should have been marked as the foreground. (iv) Organ overlapping. There is overlap between various organs. For example, “Hepatic Vessel” is part of the “Liver” and “Kidney Tumor” is a subvolume of the “Kidney”. This partially labelled problem imposes significant limitations on the performance of models trained on existing public datasets, ultimately hindering their effectiveness for many medical image analysis tasks.

4.3. Case Study: CLIP-Driven Universal Model

CLIP-driven *Universal Model*⁵⁶ incorporates the pre-trained language-image text embedding and solves the partially labelled problem. This framework is developed on an assembly of 14 public datasets of 3,410 CT scans with 25 organs and 6 types of tumors in total.

4.3.1. Background

Problem definition: Let M and N be the total number of datasets to combine and data points in the combination of the datasets, respectively. Given a dataset $\mathcal{D} = \{(\mathbf{X}_1, \mathbf{Y}_1), (\mathbf{X}_2, \mathbf{Y}_2), \dots, (\mathbf{X}_N, \mathbf{Y}_N)\}$, there are a total of K unique classes. For $\forall n \in [1, N]$, if the presence of $\forall k \in [1, K]$ classes in $\mathbf{X}_i$ is annotated in $\mathbf{Y}_i$, $\mathcal{D}$ is a *fully labelled* dataset; otherwise, $\mathcal{D}$ is a *partially labelled* dataset.

According to DoDNet¹⁰⁸ approach, most partially supervised methods ignore the semantic and anatomical relationship between organs and tumors. Second, they are inappropriate for segmenting various subtypes of tumors. To address these limitations, CLIP universal model⁵⁶ introduces prompt embedding.

4.3.2. Architecture

CLIP-driven *Universal Model* framework maintains a revised label taxonomy derived from a collection of public datasets and generates a binary segmentation mask for each class during image pre-processing.

For architecture design, Universal Model (in Figure 4) has a text branch and a vision branch. The text branch first generates the CLIP embedding for each organ and tumor using an appropriate medical prompting. This CLIP-based label encoding enhances the anatomical structure of universal model feature embedding. Then, the vision branch takes both CT scans and CLIP embedding to predict the segmentation mask.

Text branch: Let $\mathbf{w}_k$ be the CLIP embedding of the k -th class, produced by the pre-trained text encoder in CLIP and a medical prompt (e.g., “a computerised tomography of a [CLS]”, where [CLS] is a concrete class name). Universal Model first concatenates the CLIP embedding ($\mathbf{w}_k$) and the global image feature ($\mathbf{f}$) and then inputs it to a multi-layer perceptron (MLP), namely *text-based controller*,¹⁰⁹ to generate parameters ($\boldsymbol{\theta}_k$), i.e.,

$$\boldsymbol{\theta}_k = MLP(\mathbf{w}_k \oplus \mathbf{f}), \quad (13)$$

where $\oplus$ is the concatenation.

Vision branch: The vision branch pre-processes CT scans using isotropic spacing and uniformed intensity scale to reduce the domain gap among various datasets. A standardised and normalised CT pre-processing is important when combining multiple datasets. Substantial differences in CT scans can occur in image quality and technical display, originating from different acquisition parameters, reconstruction kernels, contrast enhancements, intensity variation, and so on.^{110–112} The standardised and normalised CT scans are then processed by the vision encoder. Let $\mathbf{F}$ be the image features extracted by the vision encoder. To process $\mathbf{F}$, Universal Model uses three sequential convolutional layers with $1 \times 1 \times 1$ kernels, namely *text-driven segmentor*. The first two layers have 8 channels, and the last one has 1 channel, corresponding to the class of $[\text{CLS}]_k$. The prediction for the class $[\text{CLS}]_k$ is computed as

$$\mathbf{P}_k = \text{Sigmoid}(((\mathbf{F} * \boldsymbol{\theta}_{k_1}) * \boldsymbol{\theta}_{k_2}) * \boldsymbol{\theta}_{k_3}), \quad (14)$$

where $\boldsymbol{\theta}_{k_1}, \boldsymbol{\theta}_{k_2}, \boldsymbol{\theta}_{k_3}$ are computed by Equation (13) and $*$ represents the convolution. For each class $[\text{CLS}]_k$, Universal Model generates the prediction using *one vs. all* manner (i.e., Sigmoid instead of Softmax).

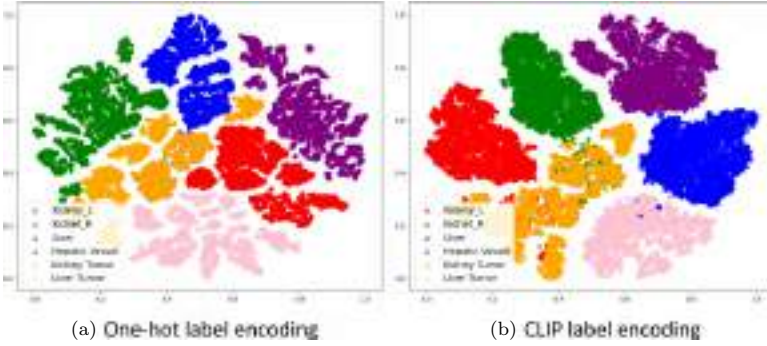


Fig. 9. t-SNE visualisation of embedding space in Universal Model.⁵⁶

4.3.3. Analysis

When applying the medical AI model in real-world scenarios, it is critical to test the generalisability of new data across many hospitals. Universal Model is capable of transferring segmentation tasks (t-SNE plots in Figure 9), which could serve as an effective foundation model for another segmentation task. Through pre-training by assembly dataset directly and fine-tuning to other datasets, the Universal Model achieves the promising DSC compared with other pre-training methods^{38,59,113,114} with 86.49%, 89.57%, 94.43%, and 88.95% for four tasks in TotalSegmentator dataset. This demonstrates the potential for improving the generalisation of medical imaging model embedding through directly capturing the fine-grained information for segmentation.

CLIP label encoding achieves a better feature cluster and shows anatomically structured semantics. Similar concepts are mapped close to each other in the embedding space. For example, “Liver” is closer to “Liver Tumor” and “Hepatic Vessel” (the hepatic vessel returns low-oxygen blood from the liver to the heart, which has a high anatomical relationship with the liver); “Left Kidney” is closer to “Right Kidney”.

Manual annotations have inter-rater and intra-rater variances,¹¹⁵ particularly in segmentation tasks, because some of the organs’ boundaries are blurry and ambiguous. The quality of pseudo labels predicted by Universal Model and manual annotation performed by human experts. 17 CT scans in BTCV have been annotated by two independent groups of radiologists from different institutes. Figure 10 presents their mutual DSC scores, i.e., $AI \leftrightarrow Dr1$, $AI \leftrightarrow Dr2$,

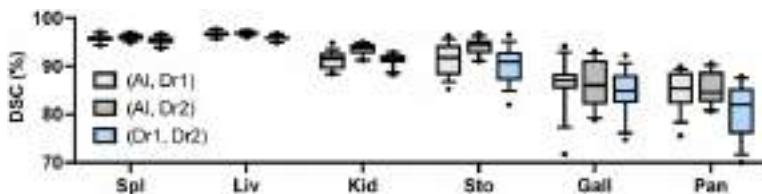


Fig. 10. DSC scores between manual annotation from two human experts and pseudo labels predicted by Universal Model.⁵⁶ Spleen (Spl), liver (Liv), kidneys (Kid), stomach (Sto), gallbladder (Gall), and pancreas (Pan).

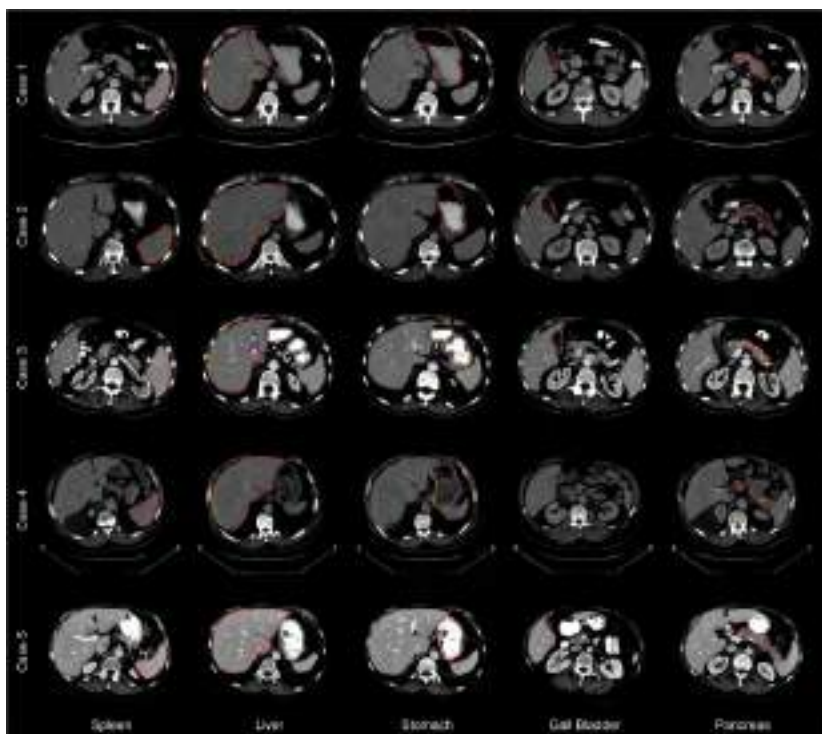


Fig. 11. Contour line comparison among two human experts and pseudo labels from Universal Model.⁵⁶ The red, green, and blue lines represent the annotation from Doctor 1, Doctor 2, and Universal Model separately.

and $Dr1 \leftrightarrow Dr2$. Similar performance is obtained between pseudo labels generated by the Universal Model (AI) and annotations performed by two human experts (Dr1, 2) on six organs. Examples of pseudo labels and human annotations are provided in Figure 11. The prediction results of these organs generated by the medical model are

comparable with human experts. This demonstrates that these six organs can be annotated by AI with a similar intra-observer variability to humans.

5. Conclusion

While medical imaging plays a crucial role in diagnosing, monitoring, and treating various diseases and conditions, the variation and heterogeneity of medical images pose significant challenges. In the field of abdominal studies and neuroscience, for example, anatomical structures vary significantly in terms of size, shape, and location, and such variations are often linked to specific medical conditions and functions. To address these challenges, 3D deep learning approaches have shown great potential in recent years. Deep learning models can learn complex features and patterns from large amounts of medical image data and use these patterns to accurately predict clinical outcomes, diagnose diseases, and detect abnormalities. Moreover, the use of 3D models provides a more comprehensive and accurate representation of anatomical structures as it allows for the analysis of volumetric data. Overall, there has never been a more exciting time to study fundamental medical image problems using modern deep learning. To date, vision foundation models show remarkable zero-shot performance across various medical image modalities. Segment Anything Model (SAM) achieved significant milestone on segmentation tasks of natural images, which is also a prioritised task for medical domain. Encompassing organ, abnormality, tumor, and other delineations, SAM show potential capability at drafting masks with its zero-show learning ability. However, there are many challenges due to domain gaps and modality discrepancies in medical imaging. As new models continue to emerge, computer hardware becomes more powerful, and advanced imaging machines are invented successively, the potential for 3D deep learning in medical imaging is vast.

References

1. S. K. Zhou, D. Rueckert, and G. Fichtinger. *Handbook of Medical Image Computing and Computer Assisted Intervention*. Cambridge, Massachusetts: Academic Press (2019).

2. S. B. Heymsfield, T. Fulenwider, B. Nordlinger, R. Barlow, P. Sones, and M. Kutner. Accurate measurement of liver, kidney, and spleen volume and mass by computerized axial tomography, *Annals of Internal Medicine* **90**(2), 185–187 (1979).
3. J. Czernin, M. R. Benz, and M. S. Allen-Auerbach. PET/CT imaging: The incremental value of assessing the glucose metabolic phenotype and the structure of cancers in a single examination, *European Journal of Radiology* **73**(3), 470–480 (2010).
4. E. Kulama. Scanning protocols for multislice CT scanners, *The British Journal of Radiology* **77**(Suppl.1), S2–S9 (2004).
5. Y. Tang, R. Gao, H. H. Lee, Q. S. Wells, A. Spann, J. G. Terry, J. J. Carr, Y. Huo, S. Bao, and B. A. Landman. Prediction of type ii diabetes onset with computed tomography and electronic medical records. In *Multimodal Learning for Clinical Decision Support and Clinical Image-Based Procedures: 10th International Workshop, ML-CDS 2020, and 9th International Workshop, CLIP 2020, Held in Conjunction with MICCAI 2020*, Lima, Peru, October 4–8, 2020, Proceedings 9, pp. 13–23 (2020).
6. H. R. Roth, A. Farag, E. Turkbey, L. Lu, J. Liu, and R. M. Summers. Data from pancreas-CT. The cancer imaging archive, *IEEE Transactions on Medical Imaging* **35**(5), 1170–1181 (2016). doi: 10.1109/TMI.2015.2482920.
7. A. Muhi, T. Ichikawa, U. Motosugi, H. Sou, H. Nakajima, K. Sano, M. Sano, S. Kato, T. Kitamura, Z. Fatima *et al.* Diagnosis of colorectal hepatic metastases: Comparison of contrast-enhanced CT, contrast-enhanced us, superparamagnetic iron oxide-enhanced MRI, and gadoteric acid-enhanced MRI, *Journal of Magnetic Resonance Imaging* **34**(2), 326–335 (2011).
8. G. Tognini, F. Ferrozzi, D. Bova, P. Bini, and M. Zompatori. Diabetes mellitus: CT findings of unusual complications related to the disease: A pictorial essay, *Clinical Imaging* **27**(5), 325–329 (2003).
9. B. Dussol, J. Moussi-Frances, S. Morange, C. Somma-Delpero, O. Mundler, and Y. Berland. A randomized trial of furosemide vs hydrochlorothiazide in patients with chronic renal failure and hypertension, *Nephrology Dialysis Transplantation* **20**(2), 349–353 (2005).
10. A. B. Chapman, J. E. Bost, V. E. Torres, L. Guay-Woodford, K. T. Bae, D. Landsittel, J. Li, B. F. King, D. Martin, L. H. Wetzell *et al.* Kidney volume and functional outcomes in autosomal dominant polycystic kidney disease, *Clinical Journal of the American Society of Nephrology* **7**(3), 479–486 (2012).

11. Z. Zhou, M. B. Gotway, and J. Liang. Interpreting medical images. In *Intelligent Systems in Medicine and Health*, pp. 343–371. Springer, Springer Cham (2022).
12. I. Lyu, H. Kang, N. D. Woodward, M. A. Styner, and B. A. Landman, Hierarchical spherical deformation for cortical surface registration, *Medical Image Analysis* **57**, 72–88 (2019).
13. L. Hao, S. Bao, Y. Tang, R. Gao, P. Parvathaneni, J. A. Miller, W. Voorhies, J. Yao, S. A. Bunge, K. S. Weiner, et al. Automatic labeling of cortical sulci using spherical convolutional neural networks in a developmental cohort. In *2020 IEEE 17th International Symposium on Biomedical Imaging (ISBI)*, pp. 412–415 (2020).
14. J. Yu, C. Zhang, H. Wang, D. Zhang, Y. Song, T. Xiang, D. Liu, and W. Cai, 3D medical point transformer: Introducing convolution to attention networks for medical point cloud analysis, *arXiv preprint arXiv:2112.04863* (2021).
15. Y. Huo, Z. Xu, S. Bao, A. Assad, R. G. Abramson, and B. A. Landman. Adversarial synthesis learning enables segmentation without target modality ground truth. In *2018 IEEE 15th International Symposium on Biomedical Imaging (ISBI 2018)*, pp. 1217–1220 (2018).
16. Q. Dou, C. Ouyang, C. Chen, H. Chen, and P.-A. Heng, Unsupervised cross-modality domain adaptation of convnets for biomedical image segmentations with adversarial loss, *arXiv preprint arXiv:1804.10916* (2018).
17. H. H. Lee, Y. Tang, O. Tang, Y. Xu, Y. Chen, D. Gao, S. Han, R. Gao, M. R. Savona, R. G. Abramson, et al. Semi-supervised multi-organ segmentation through quality assurance supervision. In *Medical Imaging 2020: Image Processing*, Vol. 11313, pp. 363–369 (2020).
18. K. Wang, B. Zhan, C. Zu, X. Wu, J. Zhou, L. Zhou, and Y. Wang, Semi-supervised medical image segmentation via a tripled-uncertainty guided mean teacher model with contrastive learning, *Medical Image Analysis* **79**, 102447 (2022).
19. Y. Tang, R. Gao, S. Han, Y. Chen, D. Gao, V. Nath, C. Bermudez, M. R. Savona, S. Bao, I. Lyu et al. Body part regression with self-supervision, *IEEE Transactions on Medical Imaging* **40**(5), 1499–1507 (2021).
20. Y. Huo, Y. Tang, Y. Chen, D. Gao, S. Han, S. Bao, S. De, J. G. Terry, J. J. Carr, R. G. Abramson et al. Stochastic tissue window normalization of deep learning on computed tomography, *Journal of Medical Imaging* **6**(4), 044005–044005 (2019).
21. O. Ronneberger, P. Fischer, and T. Brox. U-Net: Convolutional networks for biomedical image segmentation. In *Medical Image Computing and Computer-Assisted Intervention — MICCAI 2015: 18th*

- International Conference*, Munich, Germany, October 5–9, 2015, Proceedings, Part III 18, pp. 234–241 (2015).
22. L.-C. Chen, Y. Zhu, G. Papandreou, F. Schroff, and H. Adam. Encoder-decoder with atrous separable convolution for semantic image segmentation. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 801–818 (2018).
 23. X. Li, H. Chen, X. Qi, Q. Dou, C.-W. Fu, and P.-A. Heng. H-DenseUNet: Hybrid densely connected unet for liver and tumor segmentation from CT volumes, *IEEE Transactions on Medical Imaging* **37**(12), 2663–2674 (2018).
 24. S. Liu, D. Xu, S. K. Zhou, O. Pauly, S. Grbic, T. Mertelmeier, J. Wicklein, A. Jerebko, W. Cai, and D. Comaniciu. 3D anisotropic hybrid network: Transferring convolutional features from 2d images to 3D anisotropic volumes. In *Medical Image Computing and Computer Assisted Intervention–MICCAI 2018: 21st International Conference*, Granada, Spain, September 16–20, 2018, Proceedings, Part II 11, pp. 851–858 (2018).
 25. Ö. Çiçek, A. Abdulkadir, S. S. Lienkamp, T. Brox, and O. Ronneberger. 3D U-Net: Learning dense volumetric segmentation from sparse annotation. In *Medical Image Computing and Computer-Assisted Intervention–MICCAI 2016: 19th International Conference*, Athens, Greece, October 17–21, 2016, Proceedings, Part II 19, pp. 424–432 (2016).
 26. H. R. Roth, H. Oda, X. Zhou, N. Shimizu, Y. Yang, Y. Hayashi, M. Oda, M. Fujiwara, K. Misawa, and K. Mori. An application of cascaded 3D fully convolutional networks for medical image segmentation, *Computerized Medical Imaging and Graphics* **66**, 90–99 (2018).
 27. F. Milletari, N. Navab, and S.-A. Ahmadi. V-Net: Fully convolutional neural networks for volumetric medical image segmentation. In *2016 4th International Conference on 3D Vision (3DV)*, pp. 565–571 (2016).
 28. H. R. Roth, C. Shen, H. Oda, T. Sugino, M. Oda, Y. Hayashi, K. Misawa, and K. Mori. A multi-scale pyramid of 3D fully convolutional networks for abdominal multi-organ segmentation. In *Medical Image Computing and Computer Assisted Intervention — MICCAI 2018: 21st International Conference*, Granada, Spain, September 16–20, 2018, Proceedings, Part IV 11, pp. 417–425 (2018).
 29. F. E.-Z. A. El-Gamal, M. Elmogy, and A. Atwan, Current trends in medical image registration and fusion, *Egyptian Informatics Journal* **17**(1), 99–124 (2016).

30. B. Meher, S. Agrawal, R. Panda, and A. Abraham, A survey on region based image fusion methods, *Information Fusion* **48**, 119–132 (2019).
31. A. Raju, C.-T. Cheng, Y. Huo, J. Cai, J. Huang, J. Xiao, L. Lu, C. Liao, and A. P. Harrison. Co-heterogeneous and adaptive segmentation from multi-source and multi-phase ct imaging data: A study on pathological liver and lesion segmentation. In *European Conference on Computer Vision*, pp. 448–465 (2020).
32. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, L. Kaiser, and I. Polosukhin. Attention is all you need, *Advances in Neural Information Processing Systems* **30** (2017).
33. A. Dosovitskiy, L. Beyer, A. Kolesnikov, D. Weissenborn, X. Zhai, T. Unterthiner, M. Dehghani, M. Minderer, G. Heigold, S. Gelly et al. An image is worth 16x16 words: Transformers for image recognition at scale. *arXiv preprint* arXiv:2010.11929 (2020).
34. Z. Liu, Y. Lin, Y. Cao, H. Hu, Y. Wei, Z. Zhang, S. Lin, and B. Guo. Swin transformer: Hierarchical vision transformer using shifted windows. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 10012–10022 (2021).
35. J. Chen, Y. Lu, Q. Yu, X. Luo, E. Adeli, Y. Wang, L. Lu, A. L. Yuille, and Y. Zhou. TransUNet: Transformers make strong encoders for medical image segmentation. *arXiv preprint* arXiv:2102.04306 (2021).
36. A. Hatamizadeh, Y. Tang, V. Nath, D. Yang, A. Myronenko, B. Landman, H. R. Roth, and D. Xu. UNETR: Transformers for 3D medical image segmentation. In *Proceedings of the IEEE/CVF Winter Conference on Applications of Computer Vision*, pp. 574–584 (2022).
37. X. Chen, S. Xie, and K. He. An empirical study of training self-supervised vision transformers. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pp. 9640–9649 (2021).
38. Y. Tang, D. Yang, W. Li, H. R. Roth, B. Landman, D. Xu, V. Nath, and A. Hatamizadeh. Self-supervised pre-training of swin transformers for 3D medical image analysis. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 20730–20740 (2022).
39. K. He, X. Chen, S. Xie, Y. Li, P. Dollár, and R. Girshick. Masked autoencoders are scalable vision learners. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 16000–16009 (2022).

40. A. Radford, J. W. Kim, C. Hallacy, A. Ramesh, G. Goh, S. Agarwal, G. Sastry, A. Askell, P. Mishkin, J. Clark *et al.* Learning transferable visual models from natural language supervision. In *International Conference on Machine Learning*, pp. 8748–8763 (2021).
41. Z. Zhou, M. M. R. Siddiquee, N. Tajbakhsh, and J. Liang. UNet++: A nested U-Net architecture for medical image segmentation. In *Deep Learning in Medical Image Analysis and Multimodal Learning for Clinical Decision Support*, pp. 3–11. Springer (2018).
42. F. Isensee, P. F. Jaeger, S. A. Kohl, J. Petersen, and K. H. Maier-Hein. nnU-Net: A self-configuring method for deep learning-based biomedical image segmentation, *Nature Methods* **18**(2), 203–211 (2021).
43. Z. Zhu, Y. Xia, W. Shen, E. Fishman, and A. Yuille. A 3D coarse-to-fine framework for volumetric medical image segmentation. In *2018 International Conference on 3D Vision (3DV)*, pp. 682–690 (2018).
44. Y. Tang, R. Gao, H. H. Lee, S. Han, Y. Chen, D. Gao, V. Nath, C. Bermudez, M. R. Savona, R. G. Abramson *et al.* High-resolution 3D abdominal segmentation with random patch network fusion, *Medical Image Analysis* **69**, 101894 (2021).
45. Z. Zhang, H. Zhang, L. Zhao, T. Chen, S. Ö. Arik, and T. Pfister. Nested hierarchical transformer: Towards accurate, data-efficient and interpretable visual understanding. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 36, pp. 3417–3425 (2022).
46. A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, u. Kaiser, and I. Polosukhin. Attention is all you need. In *Advances in Neural Information Processing Systems*, pp. 5998–6008 (2017).
47. J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova. BERT: Pre-training of deep bidirectional transformers for language understanding. *arXiv preprint* arXiv:1810.04805 (2018).
48. T. Brown, B. Mann, N. Ryder, M. Subbiah, J. D. Kaplan, P. Dhariwal, A. Neelakantan, P. Shyam, G. Sastry, A. Askell *et al.* Language models are few-shot learners. In *Advances in Neural Information Processing Systems*, Vol. 33, pp. 1877–1901 (2020).
49. Y.-S. Chuang, C.-L. Liu, and H.-Y. Lee. SpeechBERT: Cross-modal pre-trained language model for end-to-end spoken question answering. *arXiv preprint* arXiv:1910.11559 (2019).
50. N. Carion, F. Massa, G. Synnaeve, N. Usunier, A. Kirillov, and S. Zagoruyko. End-to-end object detection with transformers. In *European Conference on Computer Vision* (2020).

51. S. Zheng, J. Lu, H. Zhao, X. Zhu, Z. Luo, Y. Wang, Y. Fu, J. Feng, T. Xiang, P. H. Torr *et al.* Rethinking semantic segmentation from a sequence-to-sequence perspective with transformers. *arXiv preprint arXiv:2012.15840* (2020).
52. L. Zhou, Y. Zhou, J. J. Corso, R. Socher, and C. Xiong. End-to-end dense video captioning with masked transformer. In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, pp. 8739–8748 (2018).
53. X. Yu, Q. Yang, Y. Zhou, L. Y. Cai, R. Gao, H. H. Lee, T. Li, S. Bao, Z. Xu, T. A. Lasko *et al.* UNesT: Local spatial representation learning with hierarchical transformer for efficient medical segmentation. *arXiv preprint arXiv:2209.14378* (2022).
54. B. Landman, Z. Xu, J. Igelsias, M. Styner, T. Langerak, and A. Klein. Miccai multi-atlas labeling beyond the cranial vault—workshop and challenge. In *Proceedings of MICCAI Multi-Atlas Labeling Beyond Cranial Vault — Workshop Challenge* (2015).
55. Y. Huo, Z. Xu, Y. Xiong, K. Aboud, P. Parvathaneni, S. Bao, C. Bermudez, S. M. Resnick, L. E. Cutting, and B. A. Landman. 3D whole brain segmentation using spatially localized atlas network tiles, *NeuroImage* **194**, 105–119 (2019).
56. J. Liu, Y. Zhang, J.-N. Chen, J. Xiao, Y. Lu, B. A. Landman, Y. Yuan, A. Yuille, Y. Tang, and Z. Zhou. CLIP-driven universal model for organ segmentation and tumor detection. *arXiv preprint arXiv:2301.00785* (2023).
57. Y. Tang, H. H. Lee, Y. Xu, O. Tang, Y. Chen, D. Gao, S. Han, R. Gao, C. Bermudez, M. R. Savona *et al.* Contrast phase classification with a generative adversarial network. In *Medical Imaging 2020: Image Processing*, Vol. 11313, pp. 220–227 (2020).
58. M. Antonelli, A. Reinke, S. Bakas, K. Farahani, B. A. Landman, G. Litjens, B. Menze, O. Ronneberger, R. M. Summers, B. van Ginneken *et al.* The medical segmentation decathlon. *arXiv preprint arXiv:2106.05735* (2021).
59. S. Chen, K. Ma, and Y. Zheng. Med3D: Transfer learning for 3D medical image analysis. *arXiv preprint arXiv:1904.00625* (2019).
60. M. Raghu, C. Zhang, J. Kleinberg, and S. Bengio. Transfusion: Understanding transfer learning for medical imaging. In *Advances in Neural Information Processing Systems* (2019).
61. S. Atito, M. Awais, and J. Kittler. SiT: Self-supervised vision transformer. *arXiv preprint arXiv:2104.03602* (2021).
62. Z. Dai, B. Cai, Y. Lin, and J. Chen. UP-DETR: Unsupervised pre-training for object detection with transformers. In *Proceedings of the*

- IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2021).
63. J. Liang, J. Cao, G. Sun, K. Zhang, L. Van Gool, and R. Timofte. SwinIR: Image restoration using swin transformer. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).
 64. Z. Zhou, V. Sodha, J. Pang, M. B. Gotway, and J. Liang. Models genesis, *Medical Image Analysis* **67**, 101840 (2021).
 65. F. Haghighi, M. R. H. Taher, Z. Zhou, M. B. Gotway, and J. Liang. Transferable visual words: Exploiting the semantics of anatomical patterns for self-supervised learning, *IEEE Transactions on Medical Imaging* (2021).
 66. D. Pathak, P. Krahenbuhl, J. Donahue, T. Darrell, and A. A. Efros. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2016).
 67. L. Chen, P. Bentley, K. Mori, K. Misawa, M. Fujiwara, and D. Rueckert. Self-supervised learning for medical image analysis using image context restoration, *Medical Image Analysis* **58**, 101539 (2019).
 68. X. Wang, Z. Xu, L. Tam, D. Yang, and D. Xu. Self-supervised image-text pre-training with mixed data in chest x-rays. *arXiv preprint arXiv:2103.16022* (2021).
 69. S. Azizi, B. Mustafa, F. Ryan, Z. Beaver, J. Freyberg, J. Deaton, A. Loh, A. Karthikesalingam, S. Kornblith, T. Chen *et al.* Big self-supervised models advance medical image classification. In *Proceedings of the IEEE/CVF International Conference on Computer Vision* (2021).
 70. J. Zhu, Y. Li, Y. Hu, K. Ma, S. K. Zhou, and Y. Zheng. Rubik's cube+: A self-supervised feature learning framework for 3D medical image analysis, *Medical Image Analysis* **64**, 101746 (2020).
 71. S. Gidaris, P. Singh, and N. Komodakis. Unsupervised representation learning by predicting image rotations. In *International Conference on Learning Representations* (2018).
 72. A. Taleb, W. Loetzsch, N. Danz, J. Severin, T. Gaertner, B. Bergner, and C. Lippert. 3D self-supervised methods for medical imaging. In *Advances in Neural Information Processing Systems* (2020).
 73. A. v. d. Oord, Y. Li, and O. Vinyals. Representation learning with contrastive predictive coding. *arXiv preprint arXiv:1807.03748* (2018).
 74. Y. Lecun, Y. Bengio, and G. Hinton. Deep learning (5, 2015). ISSN 14764687. <http://colah.github.io/>.

75. K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *IEEE Computer Society Conference on Computer Vision and Pattern Recognition*, Vol. 2016-Decem, pp. 770–778 (2016). ISBN 9781467388504. <http://image-net.org/challenges/LSVRC/2015/>.
76. A. Krizhevsky, I. Sutskever, and G. E. Hinton. ImageNet classification with deep convolutional neural networks. In *NIPS* (2012).
77. G. Huang, Z. Liu, L. Van Der Maaten, and K. Q. Weinberger. Densely connected convolutional networks. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition* (2017).
78. S. J. Pan and Q. Yang. A survey on transfer learning, *IEEE Transactions on Knowledge and Data Engineering* **22**(10), 1345–1359 (2009). ISSN 1041-4347.
79. Z. Zhou, V. Sodha, J. Pang, M. B. Gotway, and J. Liang. Models genesis, *Medical Image Analysis* **67**, 101840 (2021). ISSN 1361-8415.
80. A. A. A. Setio, A. Traverso, T. De Bel, M. S. Berens, C. Van Den Bogaard, P. Cerello, H. Chen, Q. Dou, M. E. Fantacci, and B. Geurts. Validation, comparison, and combination of algorithms for automatic detection of pulmonary nodules in computed tomography images: The luna16 challenge, *Medical Image Analysis* **42**, 1–13 (2017). ISSN 1361-8415.
81. P. Vincent, H. Larochelle, I. Lajoie, Y. Bengio, P.-A. Manzagol, and L. Bottou. Stacked denoising autoencoders: Learning useful representations in a deep network with a local denoising criterion, *Journal of Machine Learning Research* **11**(12) (2010). ISSN 1532-4435.
82. M. Noroozi and P. Favaro. Unsupervised learning of visual representations by solving jigsaw puzzles. In *European Conference on Computer Vision* (2016).
83. M. Caron, P. Bojanowski, A. Joulin, and M. Douze. Deep clustering for unsupervised learning of visual features. In *Proceedings of the European Conference on Computer Vision (ECCV)*, pp. 132–149 (2018).
84. L. Chen, P. Bentley, K. Mori, K. Misawa, M. Fujiwara, and D. Rueckert. Self-supervised learning for medical image analysis using image context restoration, *Medical Image Analysis* **58**, 101539 (2019). ISSN 1361-8415.
85. X. Zhuang, Y. Li, Y. Hu, K. Ma, Y. Yang, and Y. Zheng. Self-supervised feature learning for 3D medical images by playing a Rubik’s cube. In *International Conference on Medical Image Computing and Computer-Assisted Intervention* (2019).

86. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton. A simple framework for contrastive learning of visual representations. In *International Conference on Machine Learning* (2020).
87. T. Park, A. A. Efros, R. Zhang, and J.-Y. Zhu. Contrastive learning for unpaired image-to-image translation. In *European Conference on Computer Vision* (2020).
88. C. Zhao, M. Shao, A. Carass, H. Li, B. E. Dewey, L. M. Ellingsen, J. Woo, M. A. Guttman, A. M. Blitz, M. Stone *et al.* Applications of a deep learning method for anti-aliasing and super-resolution in MRI, *Magnetic Resonance Imaging* **64**, 132–141 (2019).
89. D.-H. Trinh, M. Luong, F. Dibos, J.-M. Rocchisani, C.-D. Pham, and T. Q. Nguyen. Novel example-based method for super-resolution and denoising of medical images, *IEEE Transactions on Image Processing* **23**(4), 1882–1895 (2014).
90. A. Frome, G. S. Corrado, J. Shlens, S. Bengio, J. Dean, M. Ranzato, and T. Mikolov, Devise: A deep visual-semantic embedding model. In *Advances in Neural Information Processing Systems*, **26** (2013).
91. A. Conneau and G. Lample. Cross-lingual language model pretraining. In *Advances in Neural Information Processing Systems*, Vol. 32 (2019).
92. Y. Zhang, H. Jiang, Y. Miura, C. D. Manning, and C. P. Langlotz. Contrastive learning of medical visual representations from paired images and text. In *Machine Learning for Healthcare Conference*, pp. 2–25 (2022).
93. Y. Rao, W. Zhao, G. Chen, Y. Tang, Z. Zhu, G. Huang, J. Zhou, and J. Lu. DenseCLIP: Language-guided dense prediction with context-aware prompting. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 18082–18091 (2022).
94. Z. Wang, Y. Lu, Q. Li, X. Tao, Y. Guo, M. Gong, and T. Liu. CRIS: CLIP-driven referring image segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 11686–11695 (2022).
95. P. Chambon, C. Bluethgen, C. P. Langlotz, and A. Chaudhari. Adapting pretrained vision-language foundational models to medical imaging domains. *arXiv preprint* arXiv:2210.04133 (2022).
96. K. Park, S. Woo, S. W. Oh, I. S. Kweon, and J.-Y. Lee. Per-CLIP video object segmentation. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1352–1361 (2022).
97. H. Tan and M. Bansal. LXMERT: Learning cross-modality encoder representations from transformers. *arXiv preprint* arXiv:1908.07490 (2019).

98. X. Hu, Z. Gan, J. Wang, Z. Yang, Z. Liu, Y. Lu, and L. Wang. Scaling up vision-language pre-training for image captioning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 17980–17989 (2022).
99. L. Zhou, H. Palangi, L. Zhang, H. Hu, J. Corso, and J. Gao. Unified vision-language pre-training for image captioning and vqa. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 34, pp. 13041–13049 (2020).
100. S. Eslami, G. de Melo, and C. Meinel. Does CLIP benefit visual question answering in the medical domain as much as it does in the general domain? *arXiv preprint arXiv:2112.13906* (2021).
101. Z. Wang, Z. Wu, D. Agarwal, and J. Sun. MedCLIP: Contrastive learning from unpaired medical images and text. *arXiv preprint arXiv:2210.10163* (2022).
102. Z. Qin, H. Yi, Q. Lao, and K. Li. Medical image understanding with pretrained vision language models: A comprehensive study. *arXiv preprint arXiv:2209.15517* (2022).
103. Z. Zhou. Towards Annotation-Efficient Deep Learning for Computer-Aided Diagnosis. PhD Thesis, Arizona State University (2021).
104. S. Wang, C. Li, R. Wang, Z. Liu, M. Wang, H. Tan, Y. Wu, X. Liu, H. Sun, R. Yang et al. Annotation-efficient deep learning for automatic medical image segmentation, *Nature Communications* **12**(1), 1–13 (2021).
105. F. Piccialli, V. Di Somma, F. Giampaolo, S. Cuomo, and G. Fortino. A survey on deep learning in medicine: Why, how and when? *Information Fusion* **66**, 111–137 (2021).
106. P. Bilic, P. F. Christ, E. Vorontsov, G. Chlebus, H. Chen, Q. Dou, C.-W. Fu, X. Han, P.-A. Heng, J. Hesser et al. The liver tumor segmentation benchmark (LiTS). *arXiv preprint arXiv:1901.04056* (2019).
107. N. Heller, S. McSweeney, M. T. Peterson, S. Peterson, J. Rickman, B. Stai, R. Tejapaul, M. Oestreich, P. Blake, J. Rosenberg et al. An international challenge to use artificial intelligence to define the state-of-the-art in kidney and kidney tumor segmentation in CT imaging. *Journal of Clinical Oncology*, **38**, 626 (2020).
108. J. Zhang, Y. Xie, Y. Xia, and C. Shen. DoDNet: Learning to segment multi-organ and tumors from multiple partially labeled datasets. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 1195–1204 (2021).
109. Z. Tian, C. Shen, and H. Chen. Conditional convolutions for instance segmentation. In *European Conference on Computer Vision*, pp. 282–298 (2020).

110. M. Orbes-Arteaga, T. Varsavsky, C. H. Sudre, Z. Eaton-Rosen, L. J. Haddow, L. Sørensen, M. Nielsen, A. Pai, S. Ourselin, M. Modat *et al.* Multi-domain adaptation in brain mri through paired consistency and adversarial learning. In *Domain Adaptation and Representation Transfer and Medical Image Learning with Less Labels and Imperfect Data*, pp. 54–62. Springer, Springer Cham (2019).
111. W. Yan, L. Huang, L. Xia, S. Gu, F. Yan, Y. Wang, and Q. Tao. MRI manufacturer shift and adaptation: Increasing the generalizability of deep learning segmentation for MR images acquired with different scanners, *Radiology: Artificial Intelligence* **2**(4), e190195 (2020).
112. P. Guo, P. Wang, J. Zhou, S. Jiang, and V. M. Patel. Multi-institutional collaborations for improving deep learning-based magnetic resonance image reconstruction using federated learning. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 2423–2432 (2021).
113. Z. Zhou, V. Sodha, M. M. R. Siddiquee, R. Feng, N. Tajbakhsh, M. B. Gotway, and J. Liang. Models genesis: Generic autodidactic models for 3D medical image analysis. In *International Conference on Medical Image Computing and Computer-Assisted Intervention*, pp. 384–393 (2019).
114. Y. Xie, J. Zhang, Y. Xia, and Q. Wu. UniMiSS: Universal medical self-supervised learning via breaking dimensionality barrier. In *European Conference on Computer Vision*, pp. 558–575 (2022).
115. W. Ji, S. Yu, J. Wu, K. Ma, C. Bian, Q. Bi, J. Li, H. Liu, L. Cheng, and Y. Zheng. Learning calibrated medical image segmentation via multi-rater agreement modeling. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pp. 12341–12351 (2021).
116. Jiao Du, Weisheng Li, Ke Lu, and Bin Xiao. An overview of multi-modal medical image fusion, *Neurocomputing* **215**, 3–20 (2016). ISSN: 0925-2312. <https://doi.org/10.1016/j.neucom.2015.07.160>.

This page intentionally left blank

Chapter 12

AI-Based 3D Metrology and Defect Detection of HBMs in XRM Scans

Ramanpreet Singh Pahwa^{*}, Richard Chang[†], and Wang Jie[‡]

*Institute for Infocomm Research (I²R),
Agency for Science, Technology and Research (A*STAR),
Connexis South Tower, Singapore,
Republic of Singapore*

^{}ramanpreet_pahwa@i2r.a-star.edu.sg*

[†]richard_chang@i2r.a-star.edu.sg

[‡]wang_jie@i2r.a-star.edu.sg

In this study, we employ the latest developments in 3D semi-supervised learning to create cutting-edge models for 3D object detection and segmentation of buried structures in high-resolution X-ray semiconductors scans. We illustrate our approach to locating the region of interest of HBM structures and their individual components and identifying various defects. We showcase how semi-supervised learning is utilised to capitalise on the vast amounts of available unlabelled data to enhance both detection and segmentation performance. Additionally, we explore the benefit of contrastive learning in the data pre-selection step for our detection model and multi-scale Mean Teacher training paradigm in 3D semantic segmentation to achieve better performance compared with the state of the art. We also provide an objective comparison for metrology-based defect detection with a 3D classification network. Our extensive experiments have shown that our approach outperforms state

of the art by up to 16% on object detection and 7.8% on semantic segmentation. Additionally, our automated metrology package shows a mean error of less than 2um for key features such as bond line thickness and provides better defect detection performance than the direct 3D classification approach. Overall, our method achieves state-of-the-art performance and can be used to improve the accuracy and efficiency of a wide range of failure analysis applications in semiconductor manufacturing.

1. Introduction

Quality control and evaluation play a critical role in the semiconductor packaging domain. It is essential to ensure that fabricated wafers and semiconductor packages have been manufactured as expected and do not contain any major defects. Hidden defects in miniaturised interconnects within 2.5D–3D High Bandwidth Memory (HBM) packages are a major source of low yield. Identifying these defects is both challenging and time-consuming. Additionally, C4 bump size has reduced drastically in the last 40 years. An average C4 bump size and density have reduced from 150 um in 1980s to approximately 20 um currently. This has also led to the increase in density of these bumps from 50 IO/mm² in 1980s to 2,500 IO/mm². Overall, this has led to significantly reduced yield and requires fast, non-destructive testing to address any yield issues as soon as possible.

The traditional failure analysis approach involves a destructive process of cross-sectioning a semiconductor package. This is followed by optical inspection to detect embedded process defects, such as solder extrusions, pad misalignment, embedded voids, and solder shorts. Typically, this failure analysis step is performed manually, aided by standard image processing techniques. However, this method is time-consuming, labour-intensive, requires domain expertise, and uses costly tools. Additionally, it only provides information on a single 2D plane, which necessitates repeating the process if more information is needed in adjacent regions.

Modern 3D X-ray machines can offer a satisfactory resolution for examining and analysing concealed features, such as Through Silicon Vias (TSVs), micro-bumps, and other metallic structures. This method offers great promise to serve as an exceptional non-destructive failure analysis technique. However, at present, scanning

takes 2–8 hours per sample, and some areas are affected by glaring artefacts, such as beam hardening.¹ As a result, these tools may not be practical for real-world deployment. However, like any technology, future advancements will help reduce acquisition time and improve the quality of these scans.

Artificial intelligence (AI) has had a major impact on many fields, including visual surveillance,² predictive maintenance,³ object detection,^{4–7} and image segmentation.⁸ The scientific community has made significant progress in the field of 3D deep learning due to advances in available compute and efficient resource management. Recently, deep learning has been used for 2D–3D detection and segmentation tasks in buried packages.^{9,10}

Traditionally, deep learning models require a large amount of labelled data to train. This can be time-consuming and expensive, especially in a fast-paced industry like semiconductor manufacturing, where chips may be revised multiple times per year. Our previous work developed fully supervised learning (FSL)-based 2D object detection and segmentation models,^{11–13} followed by a 2D and 3D semi-supervised segmentation learning (SSL)-based approach.^{14–16} In this chapter, we introduce a novel hierarchical consistency regularised Mean Teacher framework for performing 3D object detection and segmentation on 3D X-ray scans of dense High Bandwidth Memory (HBM) packages with only a limited amount of 3D labelled data. Our method uses an efficient and advanced AI-based automated attribute measurement technique to deliver crucial information about the HBMs, such as bond line thickness (BLT), solder extrusion, void-to-solder ratio, and pad misalignment. Since labelled semiconductor data is often difficult and expensive to obtain, our semi-supervised approach achieves better detection and segmentation accuracy with less labelled data.

In this work, we propose an innovative approach for 3D metrology and defect detection of HBMs. We use state-of-the-art techniques involving multi-view object detection, semi-supervised and contrastive learning, machine vision, and complex augmentations to identify and locate HBMs and perform accurate 3D segmentation and metrology on 2.5D–3D bumps. First, we locate the bumps in both the sagittal and transversal planes. We combine the information obtained from these two views to estimate the 3D boundaries of each individual bump. After locating these bumps in 3D X-ray Machine (XRM) data, we isolate them into individual Regions

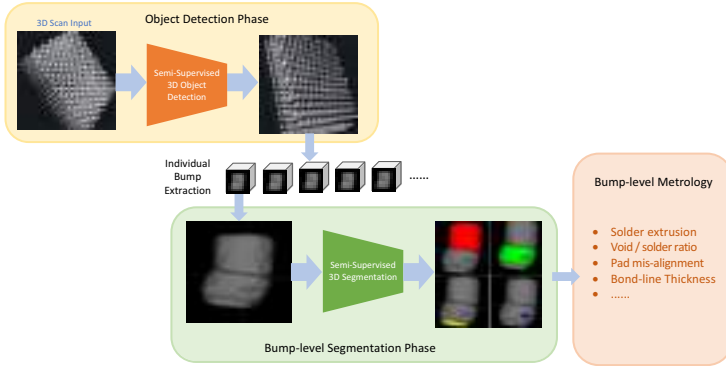


Fig. 1. Our proposed method for accurate 3D metrology on 2.5D-3D bumps.¹⁶ We first locate each bump individually in the 3D scan using our multi-view semi-supervised object detection model along with contrastive learning. Second, we use our 3D semi-supervised segmentation model to identify each component of each bump such as the copper pad and solder. Finally, we use our custom 3D metrology toolbox to measure and identify various defects in the bumps. This toolbox can measure the pad misalignment, the void-to-solder ratio, the bond line thickness, and solder extrusion.

of Interest (RoI). Second, these RoIs are processed by a 3D semi-supervised segmentation model using a VNet backbone to identify all the major components, such as Copper Pillars (CuPillar), Copper Pads (CuPad), Solders, and voids. Third, we use the segmentation output for post-processing, data cleaning, and 3D metrology using our custom metrology package. Finally, we use the metrology output to not only provide raw measurements to process engineers but also identify each individual bump as normal or defective and highlight the defect among the ones we cover — pad misalignment, void-to-solder ratio, and solder extrusion. Figure 1 shows our overall approach.

We present the related works in Section 2. We introduce our proposed method in Section 3. A detailed description of our multi-view semi-supervised object detection approach is in Section 3.1, 3D semi-supervised image segmentation methodology is in Section 3.2, and 3D metrology is in Section 3.3. Section 4 investigates our object detection and 3D segmentation approach, showcasing our capabilities by displaying end results. In particular, Section 4.4 demonstrates our result on 3D metrology. Lastly, we conclude this work in Section 5 and discuss the gaps in the current approach and potential future directions.

2. Related Work

2.1. Object Detection

Object detection methods have tremendously improved their performance thanks to deep learning architectures, huge public datasets, and higher computational capabilities. It has been applied now in many domains, such as robotics,¹⁷ medical imaging¹⁸ or semiconductors inspection.¹² Two architectures have been proposed in the literature. The first architecture performs one-stage detection, such as YOLO.¹⁹ They are faster and require less computational power. The second one performs a two-stage detection, such as Faster-RCNN.²⁰ A region proposal network (RPN) is included in the architecture and provides better detection accuracy compared to YOLO. However, they are slower and require more computational power. Several deep learning strategies have been used for object detection tasks depending on the availability and type of labelled data. Semi-supervised learning^{21,22} is used when there is only a limited amount of labelled data and this method can leverage unlabelled data to improve its detection accuracy. Data augmentations, mutual learning frameworks, pseudo labels, and specific loss functions are usually included in the architectures to compensate for the lack of labelled data and exploit unlabelled data efficiently. Unbiased Teacher²³ and Mean Teacher²⁴ have been introduced and demonstrated better performance compared to fully supervised methods. Unsupervised learning approaches have also been used for object detection. Since there is no labelled data, the architecture has to be able to extract and learn relevant visual features from the training data. SimCLR²⁵ and MoCo²⁶ show promising results in extracting meaningful image representations. Different loss strategies have been applied to unsupervised learning. Constructive losses²⁷ focus on the similarity between sampling pairs, whereas adversarial losses²⁸ are computed from probability distributions.

2.2. Semantic Segmentation

The field of semantic segmentation using deep learning methods has been widely studied and applied to various domains, including medical imaging²⁹ and semiconductor materials.¹ Many scenarios require dense mask predictions to reveal and identify the internal structures present in 3D regions.

The advent of deep learning techniques, particularly convolutional neural networks (CNNs), has led to significant advancements in semantic segmentation tasks. Early works in the field employed fully convolutional networks (FCNs) to perform end-to-end pixel-wise classification of input images.³⁰ Following the success of FCNs, various network architectures have been proposed to improve the performance of semantic segmentation. Some notable examples include the U-Net,³¹ which introduced skip connections between the encoding and decoding paths to improve the localisation of segmented objects, and the V-Net,³² a 3D extension of the U-Net architecture specifically designed for volumetric data. Annotation difficulty, data complexity, and class imbalance³³ are some of the major challenges in 3D segmentation. Some recent improvements have introduced strong data augmentations for better generalisation ability³⁴ and adopted various loss functions.^{35,36}

The scarcity of labelled data and the expensive annotation process in many application domains have motivated the exploration of semi-supervised learning techniques for semantic segmentation. Semi-supervised learning aims to improve model performance by leveraging a large amount of unlabelled data alongside a smaller labelled dataset. Various approaches have been proposed for semi-supervised semantic segmentation, such as adversarial training³⁷ and self-training.³⁸ These methods share the common goal of leveraging the information in the unlabelled data to enhance the learning process and improve model performance. The recent success of semi-supervised learning emerges under various tasks involving the teacher–student training paradigm. Several self-ensembling methods, such as Mean Teacher,³⁹ are introduced as a consistency regularisation method to counter different perturbations between the student and the teacher model.

Following the spirit of Mean Teacher, many achievements have been made to further improve teacher–student training, such as enhanced shape-awareness.⁴⁰ In addition, various consistency-based methods are proposed to improve the semi-supervised performance, including uncertainty-aware consistency,⁴¹ transformation consistency,⁴² multi-task consistency,⁴³ and multi-scale consistency.⁴⁴ Recent studies indicate that multi-scale consistency is a straightforward yet effective approach for enforcing consistency between different networks at various scales, achieving great success

in many tasks.^{45,46} Moreover, the feature maps of hidden layers in networks can be extracted to produce multi-scale predictions for deep supervision, improving the discrimination capability.

In this work, we build upon the existing literature by employing a semi-supervised Mean Teacher method with multi-scale V-Net pyramid architecture for the semantic segmentation of 3D semiconductor memory and logic bump data. Our approach aims to leverage the strengths of deep learning-based semantic segmentation, the Mean Teacher paradigm, and semi-supervised learning techniques to address the challenges associated with the limited availability of labelled data in this domain.

3. Our Approach

Each 3D XRM scan consists of 1,000 2D scans or slices of resolution $1,000 \times 1,000$. Due to computing restraints, it is infeasible and inefficient to process such a huge volume of data when we are interested in the regions of memory and logic bumps only. Therefore, a multi-step framework is used to first detect and extract the regions of interest, followed by segmenting the bumps to identify the defects and the core components like copper pillars and copper pads (Figure 1). Each slice is individually re-scaled to an input size of 640×640 during our slice-and-fuse approach for object detection.

3.1. Object Detection

In our approach, object detection is the first step used to locate individual memory and logic bumps in the XRM scans. Due to the large time-consuming in data labelling, we use a semi-supervised learning approach to use as little labelled data as possible to reach good detection performance. In selecting labelled data, a contrastive learning method is introduced to find the most informative data that will contribute the most to the model training. The final output of our model is a 3D bounding box, $[x, y, x, w, h, d]$, corresponding to the location and dimensions of the bumps for each scan. It is built by combining results from 2D slices in different views, as shown in Figure 2.

The training process of our semi-supervised method is shown in Figure 3. Our baseline semi-supervised learning framework follows

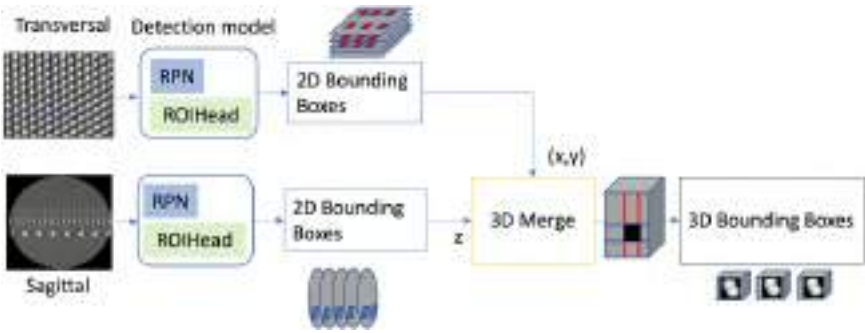


Fig. 2. 3D-slice-and-fuse approach with the detection model.¹⁵ Both detectors will run on transversal and sagittal views and output 3D bounding boxes corresponding to the bumps in the scans.

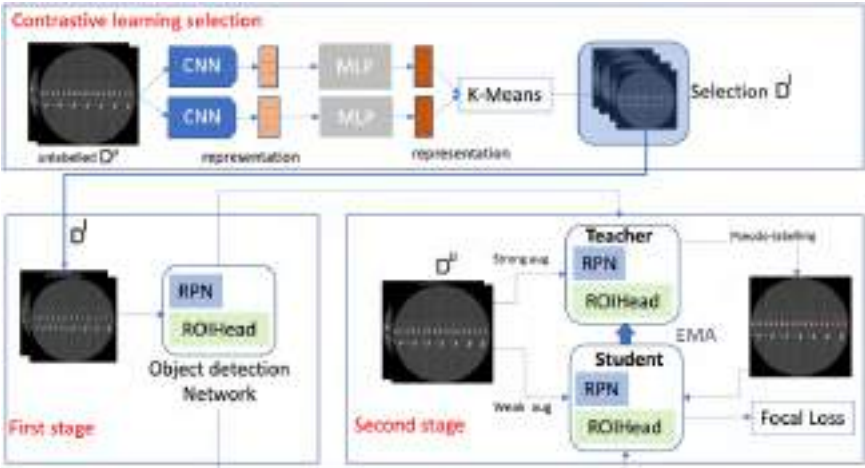


Fig. 3. Semi-supervised object detection training with contrastive learning selection.¹⁶ The most informative images are first selected using a simCLR model with k -means clustering. Semi-supervised learning object detection includes a first stage (burn-in) with labelled data and a second stage with unlabelled data and a student-teacher mutual learning framework.

the Unbiased Teacher,²³ and we select Detectron2⁴⁷ as our backbone detector as it demonstrated the best accuracy over other detectors^{48,49} on PASCAL VOC⁵⁰ and MS-COCO⁵¹ datasets. We propose a contrastive learning selection method to find the most representative 2D slices in order to make our model learn as much

information as possible in the limited labelled data. Figure 3 also shows our data pre-selection strategy. The simCLR model²⁵ is used to extract features in 2D images $I_i^{s,t}$ without the need for any prior information about the dataset. During the training process of simCLR model, different augmentation methods are first applied to input images, and the augmented images are then given to ResNet50⁴⁹ for feature representation. To further refine the feature representation, a projection head is added after ResNet50 during the training phase, which is a learnable network including an MLP with a hidden layer. For all pairs of feature representations, we calculate a contrastive loss to minimise the distance between images containing the same object and maximise the distance between images that include different objects. After the training, the MLP projection head is discarded, and we directly use the outputs of the encoder (ResNet50) as feature vectors of images. To find the most informative images, a k -means clustering method is applied to the feature vectors $h(i)$ and divided into n clusters $C = C_1, C_2, \dots, C_n$. The objective of clustering is to minimise intra-cluster variance and maximise inter-cluster variance. It is formulated as follows:

$$\arg \min \sum_{j=1}^k \sum_{h(i)} \| (i) - \mu_j \|^2 = \arg \min \sum_{j=1}^k |C_j| \text{Var}(C_j), \quad (1)$$

where μ_j is the mean of feature points in C_j . Each cluster of feature vectors represents a group of similar images. We select one image in each cluster for annotation, and these images form our labelled dataset D^l . The remaining images are all included in the unlabelled dataset D^u .

The training process of our object detection model includes two stages. In the first stage, the model is trained in a fully supervised way using only D^l . This trained model is then used to initialise the weight of student and teacher models in the second stage. In Stage 2, the model follows the mutual-learning framework to learn from the unlabelled dataset D^u . The teacher model generates its detection results, and the student model uses these results as pseudo labels to update its parameters. The teacher model's weights are, in turn, updated by the student model through the Exponential Moving Average (EMA). The focal loss is also integrated as it outperforms the original cross-entropy loss due to biased data. To fuse the 2D

detection results into 3D bounding boxes, we previously introduced a slice-and-fuse approach for object detection.¹⁴ The main idea is to run a 2D detector on each sagittal slice I_i^s which outputs 2D bounding boxes $[x, y]_n$ for each bump n and then concatenate the results into 3D bounding boxes $[x, y, x, w, h, d]_n$. However, a common issue is that when defective bumps appear in the 2D slices, it is hard to separate them from nearby bumps, which will further lead to ambiguity in the final detected 3D bounding boxes of bumps. To solve this problem, we propose a 3D slice-and-fuse approach, as shown in Figure 2. For each 3D sample of the dataset, we select their slices from both sagittal and transversal views and conduct detection in both views. Combining both sagittal and transversal views makes it more robust to eliminate deviation of detection results in 2D slices. While the sagittal view decides the x and y dimensions of 3D bounding, the transversal view decides the z direction.

3.2. Semantic Segmentation

The output of object detection is the individual bumps of memory and logic. These structures are then fed into a semantic segmentation model that segments the volumetric structure and identifies the defects. The memory and logic bumps have four foreground classes: Copper Pillar (Cu-Pillar), solder, Copper Pad (Cu-Pad), and void. The resolution of each detection of the bump is $100 \times 100 \times 100$ approximately. The ideal case should have the same size for all bumps and thus the same resolution. But due to the internal defects and the fabrication methods used, the bumps are not identical in size. Further study on 3D metrology can be facilitated by ensuring a similar dimension for all bumps.

The Mean Teacher paradigm, introduced by Tarvainen and Valpola,²⁴ is a consistency regularised semi-supervised learning method. The training method consists of two models that are trained simultaneously: a student model and a teacher model. The student model is trained on the labelled dataset to generate predictions on the unlabelled dataset that can be used as pseudo labels for the teacher model in some tasks. The network topology of the Mean Teacher model neglects the relationship between the semantic components. To better use the wider contextual information, we, following our

prior work,^{44–46} build on the Mean Teacher architecture by introducing additional prediction layers to supervise the quality of hierarchical hidden representations. These additional layers serve deep supervision insights and are also involved in minimising the multi-level segmentation loss. The additional optimisation technique helps regularise the hierarchical consistency, in turn, maximising the information learned from the unlabelled data.

We use 3D V-Net³² as the backbone model that is trained from scratch. To learn the additional hidden representations at every level, auxiliary layers are added after each decoding block to give the network a form of a hierarchical feature group. V-Net includes a downsampling encoder and an upsampling decoder each consisting of multiple stages. The latent feature information is preserved through these stages during the early levels of upsampling. Using these characteristics of the network architecture, we can conglomerate the hidden features learned at the additional auxiliary layers systematically. The structure of the multi-scale Mean Teacher, as derived from prior work,⁴⁴ contains an upsampling layer following every auxiliary layer, a $1 \times 1 \times 1$ channel convolution, and lastly a softmax layer. Figure 4 illustrates the architecture of our solution.

We performed several augmentations on the student and teacher model, both together and separately. We experimented with Gaussian blur, random contrast, and random sharpening as augmentations on the student and teacher models. These augmentations are applied

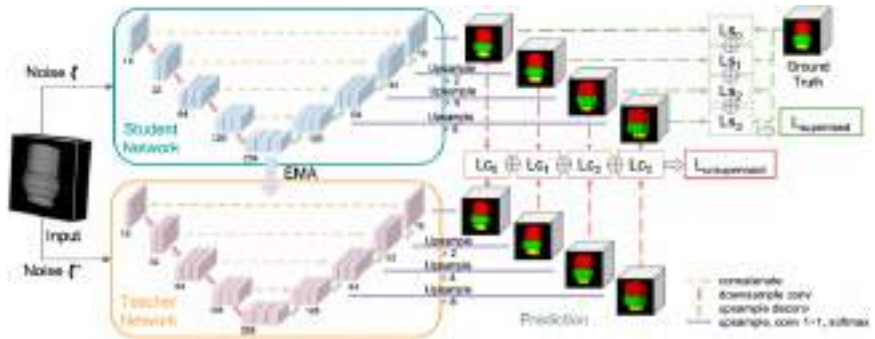


Fig. 4. Our multi-scale mean teacher (MMT) architecture uses V-Net as a backbone for 3D multi-class segmentation.¹⁶

separately to student and teacher models to better gauge how the models are reacting to the different perturbations.

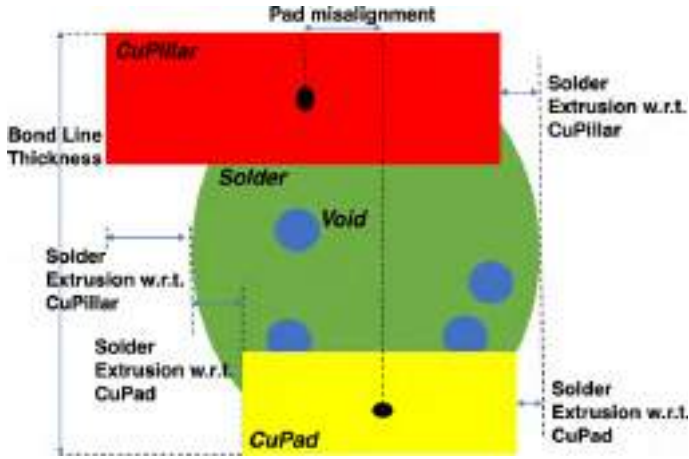
We perform deep supervised learning by accumulating the loss between the predictions for all levels and the ground truth for the labelled sample. We use these multi-scale predictions both for deep supervision of labelled samples and consistency regularisation for the unlabelled samples. We thus have tighter control of the training process by exploiting these predictions. The approach has shown promising results in experiments on various datasets and tasks.^{44–46}

3.3. 3D Metrology

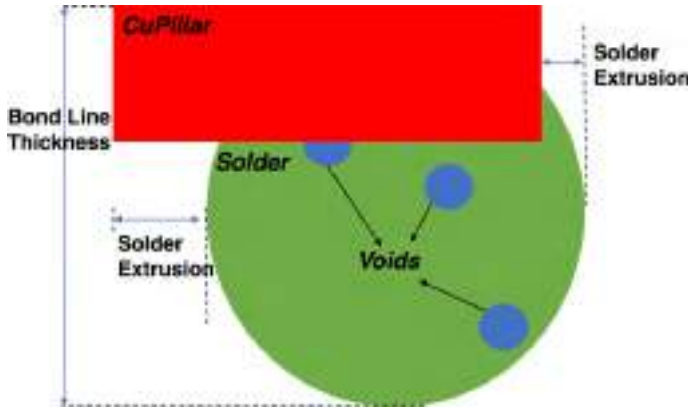
Critical features that are important for failure analysis of 3D HBM bumps are difficult to determine only from the segmentation results. So, a custom 3D metrology module is developed to improve the quality of defect detection practice. 3D metrology package measures four features, namely Bond Line Thickness (BLT), solder-to-void ratio, pad misalignment, and solder extrusion. These features are shown in Figure 5.

We perform a variety of post-processing procedures on the predicted output of our multi-scale Mean Teacher 3D segmentation model. We first verify that the bumps are aligned vertically so as to make certain that they are upright in a shared view. Following this as a second step, we apply morphological functions such as dilation and erosion to correctly re-label any pixels within the solder, copper pillar, or copper pad components that were misclassified as background. Third, the predicted voids are superimposed on top of the newly refined predictions. Some neighbouring components may be detected in the cropped individual 3D bumps. As a final step, we eliminate the predictions for such redundant neighbouring bumps. To do so, we maintain the forecasted voxels for each category, especially those voxels falling within a specific threshold to the centre of mass (CoM) of each category.

We establish the features needed to conduct our metrology, mainly the CoM, top, left, right, and bottom-most areas for each specific bump following the post-processing stage. These specific features as demonstrated in Figure 5. These are computed for each bump and later shared with domain experts to make important decisions regarding HBM failure analysis.



(a) Memory Bump



(b) Logic Bump

Fig. 5. Our 3D metrology features for HBM bumps.¹⁵ The features are computed in 3D using cross-sectional results. We show a 2D slice illustration for ease of understanding the metrology approach. Bond line thickness for logic only includes the vertical height of the solder and copper pillar.

3.4. Defect Detection

End-to-end deep learning approach has gained significant traction in the realm of defect identification and classification. Statistically, with sufficient training data, the model is capable of accurately discerning latent groups and delineating cluster boundaries, thereby

facilitating outlier detection. In our work, our dataset encompasses multiple labelled defects or anomalies. To evaluate the feasibility and robustness of deep learning in defect detection, we trained a series of binary classifiers, each dedicated to identifying a specific defect, namely void, extrusion, or pad misalignment.

We adopt 3D ResNet architecture⁵² for this task. ResNet⁵³ is the most prominent architecture in recent years. It has been extensively used as backbone network for various downstream tasks including detection and segmentation. For each given defect, we frame the problem as one binary classification task. The model is assigned to discern whether the given sample contains the defect feature or not, thereby classifying it as normal or abnormal. The 3D ResNet model, in particular, is adept at handling spatio-temporal features, making it a versatile tool for tasks, such as action recognition.

SVM or Support Vector Machine⁵⁴ is a supervised machine learning algorithm for tasks like classification and regression analysis. SVMs are one of the most robust algorithms that can handle both linear and nonlinear classification using the kernel trick. SVM is also called a maximum margin classifier as it tries to maximise the width of the gap between the two classes.

RBF or Radial Basis Function⁵⁵ kernel is the most generalised form of kernel function that is used in many kernelised machine learning algorithms, the most popular one being support vector machine or SVM classification. The RBF kernel function for two points X_1 and X_2 , represented as feature vectors in some input space, computes the similarity between the points in terms of the distance between them. This kernel is represented as follows:

$$K(X_1, X_2) = \exp - \frac{\|X_1 - X_2\|^2}{2\sigma^2},$$

where σ is the variance and $\|X_1 - X_2\|$ is the Euclidean (L_2 -norm) distance between two points.

We experimented with Linear and RBF Kernel SVM for the classification of defects. RBF Kernel SVM outperformed Linear SVM by more than 20% improvement in accuracy. Therefore, we decided to use RBF Kernel SVM for further hyperparameter tuning and analysis on our data.

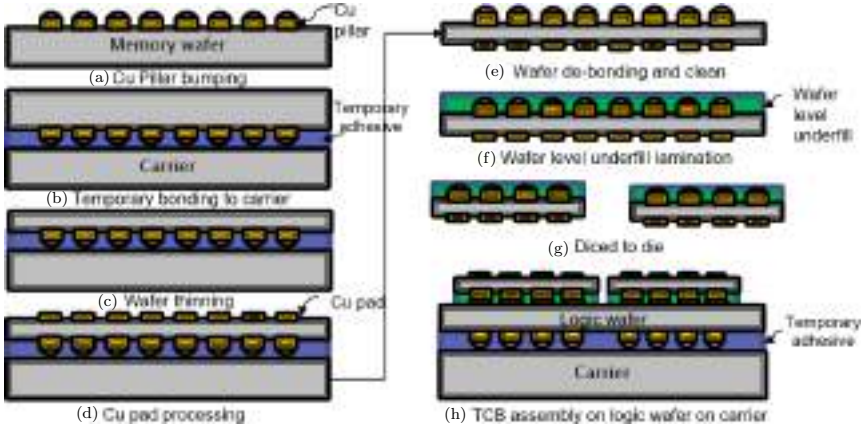


Fig. 6. Fabrication steps for HBM bumps.¹

4. Experiments

4.1. Data Fabrication

Our dataset includes fabricated 2.5D test vehicles that are similar to contemporary High-Performance Computing packages, in particular, logic and memory bumps. Standard 2.5D manufacturing processes were used to fabricate test vehicles on 300 mm silicon wafers. Daisy-chain silicon chips are produced to represent the DRAM and Logic dies which are assembled on top of each other to form High Bandwidth Memory (HBM) cubes, as shown in Figure 6. This process requires that both dies are first bonded to a carrier after the CuPillar bumping process and then reduced in size. Memory wafers are then debonded from the carrier after Cupad processing. The memory wafer is then diced to be stacked on the logic wafer using a thermo-compression bonding (TCB) process. In order to increase the amount of available data and its diversity, we purposely use suboptimal parameters during the packaging phase to induce defects. For more details on memory and logic bumps fabrication, the readers can refer to Pahwa *et al.*¹⁰

Following the fabrication process, our 3D scans are generated utilising a 3D X-ray microscopy (XRM) scanner.¹² Employing 3D XRM allows us to inspect the fabricated data in a non-destructive manner. We can then conduct various quality assessments in an efficient way.

Test vehicles are mounted on sample holders and placed on the XRM autoloader. Subsequently, they are rotated incrementally from -3° to 183° to acquire raw 2D X-ray scans. 3D X-ray scans are then generated from the 2D scans. The test vehicle can then be represented in high-resolution 3D. The resolution of each scan is approximately $1,000 \times 1,000 \times 1,000$ in size, i.e., 1 billion voxels. One 3D scan represents one test vehicle from a dedicated set of fabrication parameters. The different components of each memory and logic bumps are then labelled manually as copper pillar, copper pad, solder, and void. This labelling step has been done by our annotation team together with the semiconductors fabrication experts that helped us correctly annotate blurry boundaries and ambiguous cases.

4.2. *Object Detection*

For object detection, 3D scans are then divided into 2D slices for training, validation, and testing. Only slices that contain the logic and memory bumps have been selected. We use 17,335 and 4,593 slices for training and testing logic bumps and 18,009 and 4,467 slices for training and testing memory bumps, respectively. To train and test our model, our workstation has an Intel i9-10900X CPU processor with an NVIDIA TITAN RTX 24GB GPU containing 4,608 cuda cores.

We split our dataset into different amounts of labelled data from 1% to 10% for our experiments. Two models are trained for logic and memory, respectively. Weak data augmentation techniques on the student model are random horizontal flips and strong augmentation techniques on the teacher model are adding colour jittering, greyscale, Gaussian blur, and cutout patches. Our evaluation metric mAP consists of Intersection-over-Union (IoU) calculation estimating the accuracy of the predicted bounding box compared to the ground-truth data.¹ Different overlapping thresholds from 0.5 to 0.95 are considered. The recall rates of the method are also reported to evaluate the escapes or False Negatives. In our approach, we first use the simCLR²⁵ network with pre-trained weights on MS COCO⁵¹ to select the most relevant images in each data split. The idea is to use this network to extract discriminative visual features in the slices and classify the images into clusters according to their feature similarity. Figure 7 shows the images per cluster for a split of 1% which

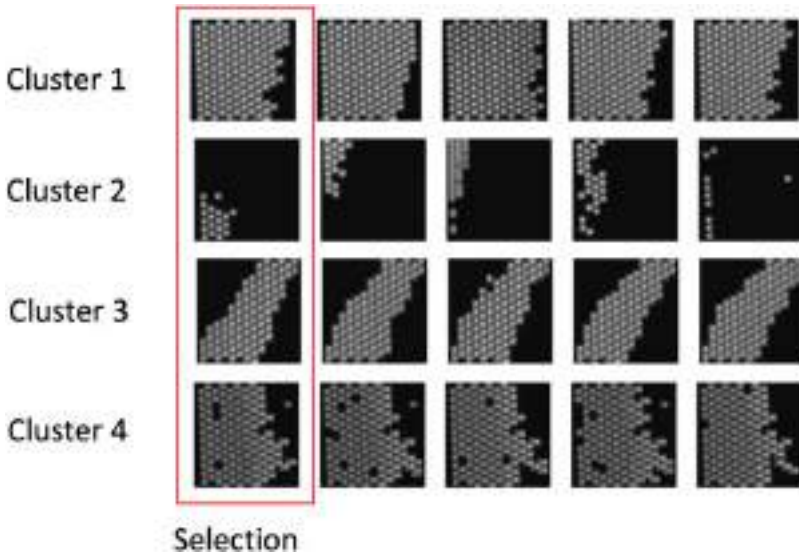


Fig. 7. Results of image pre-selection from the contrastive learning method.¹⁶ The first image on the left shows the selection and the images in the same row show similar images assigned to the same cluster. We notice that the selected images are all different and represent different data features in the dataset.

corresponds to a selection of 4 images for training. The following training parameters are used during the training process. The total number of steps is 10,000, the learning rate is 0.01, and the initial number of steps (burn-in stage) is 2,000. The EMA rate is set to 0.996.

To evaluate the data selection strategy, we compared the performance of the simCLR network on a dedicated object detection dataset¹⁷ and against other generic methods and different splits of labelled data. Results are shown in Table 1. We notice that the best detection results on the mAP¹ metric are obtained with our simCLR approach.

We evaluate the efficacy of the 2D semi-supervised object detection on the individual slices on both sagittal and transversal views with our contrastive learning selection. We set our baselines as Unbiased Teacher²³ for the semi-supervised approach and the fully Detectron2⁴⁷ for the fully supervised approach. The baselines include the first and second stages of the detection framework with a random data selection strategy. The first stage or “burn-in” stage uses the

Table 1. Comparison of data selection strategy using the mean teacher and our improved semi-supervised approach.

Accuracy on data selection strategies (mAP)			
Labelled dataset	1%	5%	10%
LeastConfidence ⁵⁶	61.87	78.9	84.34
MarginSampling ⁵⁷	62.67	79.12	84.53
EntropySampling ⁵⁸	61.34	79.32	84.92
simCLR ²⁵	63.07	79.54	86.56

Table 2. Object detection accuracy (Precision) for logic and memory dies. Our SSL approach provides more accurate results than Detectron2⁴⁷ (FSL baseline) and unbiased teacher²³ (SSL baseline) approaches on both sagittal and transversal views by up to 10% and 18% mAP with IOU = 0.5:0.95 on logic and memory bumps, respectively.

		1%	2%	5%	10%
IOU = 0.5:0.95		Precision	Precision	Precision	Precision
Memory					
Sagittal	Det2	0.607	0.631	0.634	0.648
	UBT	0.769	0.764	0.788	0.798
	Ours	0.786	0.791	0.824	0.821
Transver.	Det2	0.723	0.76	0.784	0.803
	UBT	0.764	0.781	0.798	0.824
	Ours	0.843	0.854	0.874	0.886
Logic					
Sagittal	Det2	0.776	0.781	0.782	0.809
	UBT	0.795	0.801	0.81	0.814
	Ours	0.848	0.889	0.906	0.917
Transver.	Det2	0.714	0.659	0.679	0.701
	UBT	0.788	0.80	0.824	0.843
	Ours	0.824	0.862	0.894	0.903

labelled data to train a detector model with Detectron2. Second, this model is then duplicated into student and teacher models and further trained with the remaining unlabelled data.

Tables 2 and 3 show the accuracy of the logic and memory bumps for the sagittal and transversal views. We can notice that our proposed model demonstrates a better accuracy compared to the FSL

Table 3. Object detection accuracy (Recall) for logic and memory dies. Our SSL approach has better recall rates compared to Detectron2⁴⁷ (FSL baseline) and unbiased teacher²³ (SSL baseline) approaches showing the detector had fewer misses. The results are consistent across both sagittal and transversal views and logic and memory bumps.

		1%	2%	5%	10%
IOU = 0.5:0.95		Recall	Recall	Recall	Recall
Memory					
Sagittal	Det2	0.662	0.68	0.685	0.685
	UBT	0.801	0.81	0.827	0.832
	Ours	0.81	0.826	0.831	0.845
Transver.	Det2	0.784	0.79	0.824	0.846
	UBT	0.812	0.812	0.834	0.853
	Ours	0.873	0.879	0.892	0.916
Logic					
Sagittal	Det2	0.806	0.814	0.817	0.848
	UBT	0.836	0.841	0.845	0.846b
	Ours	0.873	0.917	0.927	0.943
Transver.	Det2	0.753	0.703	0.725	0.739
	UBT	0.824	0.821	0.859	0.873
	Ours	0.859	0.893	0.923	0.931

model by up to 10% mAP for logic bumps and up to 16% for memory bumps. We also note that the performance improves when for higher splits when more labelled data is used which is expected since more labelled data should lead to better performance. The results also show that our data selection strategy is able to significantly improve the overall detection accuracy. The accuracy of the model improves compared to the baseline in both precision and recall metrics. This shows that selected images are representative of the overall dataset and the data distribution is well sampled with the selected images. The model has received helpful and informative images which led to better overall accuracy. On the other hand, the baseline model has a lower accuracy due to fewer informative images used in the first stage. The images are too similar to each other and do not reflect the data distribution of the entire dataset. Therefore, the training data

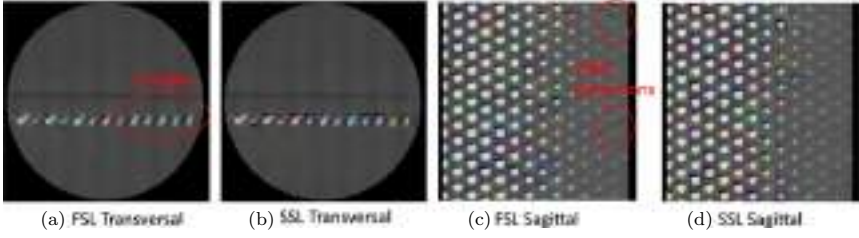


Fig. 8. (a,c) Detection results with fully supervised learning using Detectron2 backbone FSL. (b,d) Detection results with semi-supervised unbiased mean teacher models on logic bumps (2% split). Our approach shows better accuracy with a significantly lower escape rate and false detections.

is then biased and the model is not able to achieve high accuracy on the dataset.

Given the low distribution of the Semicon data, we notice that our SSL network performs well even with a very small amount of labelled data (1%). This demonstrates the reduced requirement for labelled data with our SSL framework. We also show the detection results on some slices for both sagittal and transversal views to highlight these differences in Figure 8. More false detections appear on the FSL model compared to ours. There are more escapes in the FSL model results meaning that some bumps have been missed by the model. The full implementation of the method includes three phases: the first phase detects the bumps on the individual slices, the second estimates the 3D bounding boxes in the scan, and finally, the third phase extracts the bump into individual files. Given the structure of the data and our processing scripts, all phases can be parallelised to reduce the processing time. Furthermore, our method has a better accuracy with only 1% labelled data compared other methods with 10% labelled data. This has been observed for both sagittal and transversal views for memory and logic bumps except for the 10% transversal split for the Unbiased Teacher.

Finally, our results show that a semi-supervised learning approach with data selection is able to provide better detection accuracy with less labelled data available. As semiconductor data generation is time-consuming and resource-intensive, our approach can significantly reduce the amount of time required for visual inspection and increase the flexibility and adaptability of the model on more diverse data. It is able to leverage limited labelled data and use unlabelled

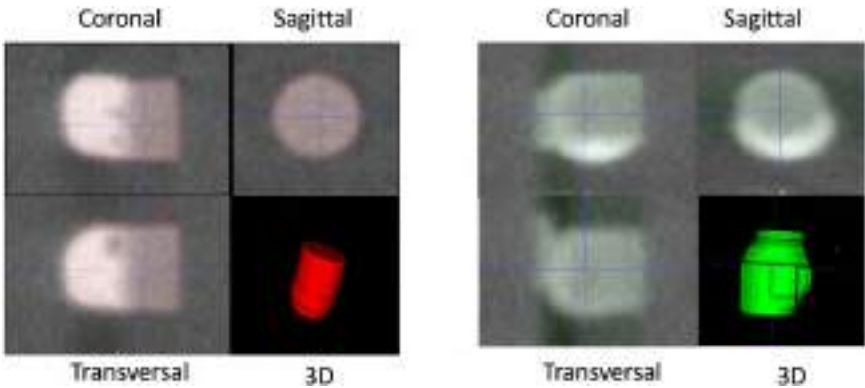


Fig. 9. Examples of logic (left) and memory (right) bumps extracted from the 3D scans.¹⁶ Three views (transversal, sagittal, and coronal) are shown with the 3D representation.

data to perform the bump extraction task. Our method was able to outperform the baseline and fully supervised models on all splits from 1% to 10% of labelled data. Our contrastive learning selection demonstrated an improvement of up to 9% on the mAP accuracy over the baseline for both memory and logic bumps and up to 16% over the fully supervised method. Our 3D slice-and-fuse approach shows that processing 2D slices is able to successfully leverage the good accuracy of 2D detectors and process high-resolution 3D scans. Figure 9 shows some examples of the detected logic and memory bumps from the 3D scans in each view and their 3D rendering.

4.3. Semantic Segmentation

For our segmentation experiment, we have in total $\{76, 36\}$ bump-level training 3D and $\{13, 7\}$ testing scans for memory and logic data. Subsets of 2.5%, 5%, 10%, 50%, and 100% labelled data are employed for training. The hardware used is identical to Section 4.2.

We establish our comparison between three different setups: fully supervised run using V-Net backbone, naive Mean Teacher semi-supervised run using the same V-Net structure, and the proposed semi-supervised multi-scale Mean Teacher method using V-Net with auxiliary layers. By comparing each training mode under various data percentages, we are able to concretely evaluate the effectiveness of the proposed method.

Similar to our object detection experiments, we adopt different optimisation metrics for supervised and unsupervised parts training. The supervised loss is computed using the multi-class Dice loss function, which measures the similarity between the predicted segmentation maps and the ground-truth labels. The consistency loss is calculated as the Mean Squared Error (MSE) between the student and teacher models' predictions on the unlabelled dataset. The overall loss function is defined as the weighted sum of the supervised loss and the consistency loss. Both the supervised loss on the labelled dataset and the consistency loss on the unlabelled dataset from the V-Net pyramid consist of multiple auxiliary losses, respectively. We empirically assign scale-wise components with weights 0.5, 0.2, 0.2, 0.1 for memory runs and 0.6, 0.25, 0.1, 0.05 for logic runs.

The V-Net model with auxiliary layers is trained with an initial learning rate of 0.01 and step-down decay interval of every 5,000 iteration at the scale of 0.1. We adopt a linear learning rate warm-up of 300 iterations and train the backbone network from scratch. The decay parameter of the exponential moving average (EMA) update rate is $\alpha = 0.999$ and the consistency weight is set to $\gamma = 0.01$. When the training initiates, the model experiences a linear consistency ramping-up stage sustained for 40 epochs until full scale. For all modes of experiments in our work, the training lasts for 10,000 iterations using SGD optimiser. Specifically for semi-supervised runs, we preserve the initial 2,000 iterations supervised, i.e., our semi-supervised runs consist of 2,000 burn-in iterations and 8,000 semi-supervised iterations.

We evaluate our model performance using multiple quantitative metrics: multi-class Dice coefficient and Jaccard coefficient (IoU). Table 4 shows the Dice and IoU performance between FSL training, Mean Teacher SSL training, and multi-scale Mean Teacher SSL training under various percentages of selected labelled data. We observe that the overall performance increases along with the addition of labelled data. Qualitatively, our method produces less misclassification and better conserves the overall shape of the material structure. Specifically, we achieve nearly 8% improvement on logic bump data. Figures 10 and 11 visualise some inferred test samples through colour-coded images. Although multi-scale runs fail to provide better results on fewer training samples, the performance surpasses its counterparts at a higher percentage of data. Empirically, we observe

Table 4. We report the V-Net FSL, Mean Teacher SSL, and our MMT SSL 3D semantic segmentation results for Memory and Logic die. The SSL approach is generally able to identify the segments more accurately. Our proposed multi-scale Mean Teacher (MMT) is showing many advantages at higher percentage data, especially for Logic die.

	V-Net		MT		Ours (MMT)	
	Dice	IoU	Dice	IoU	Dice	IoU
Memory (%)						
2.5	79.89	64.86	80.63	72.38	75.32	64.93
5	85.14	77.16	85.50	70.25	84.83	76.70
10	81.49	73.57	86.69	71.75	86.46	78.21
50	83.26	75.63	86.10	82.03	87.03	79.21
100	87.89	80.19	88.67	82.81	89.49	82.25
Logic (%)						
2.5	81.82	75.07	84.22	78.54	57.27	48.18
5	84.51	78.74	84.33	78.58	91.13	84.86
10	84.51	78.85	84.80	79.66	92.29	86.86
50	85.26	79.85	85.65	80.54	92.58	87.41
100	84.34	78.55	83.79	78.27	91.59	86.06

a similar trend in experiments with too strong regularisation, leading models to perform less effectively.

We have performed multiple augmentations on the data for both the student and teacher models. Augmentations used are Gaussian blur, random contrast, and sharpening. We also tuned the hyperparameters for the sigma (blurring kernel) and alpha (sharpening factor). Similar tuning has been done for the contrasting factor too. With these optimal hyperparameter values, we applied the augmentations on the student and teacher models using four different strategies, namely applying only Gaussian blur to both the student and teacher, applying only random contrast and sharpening to both the student and teacher, applying Gaussian blur only to the student model and contrast and sharpening to the teacher, and lastly the reverse of the previous combination. The motivation for applying different augmentation strategies to the student and teacher model is to investigate how each combination individually works on the models. From our extensive experiments, we observe that the student model

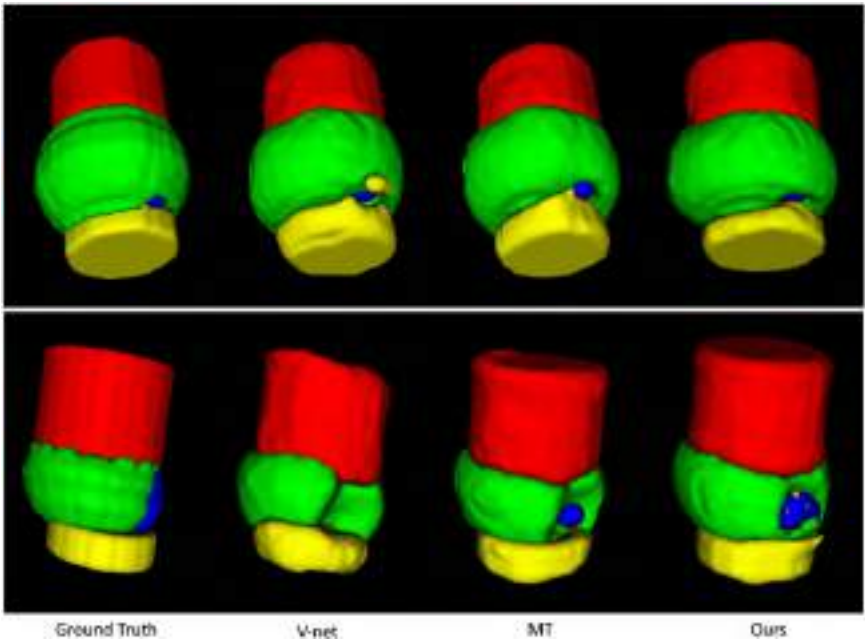


Fig. 10. We show one inferred test sample from ground-truth annotations, V-Net, mean teacher (MT), and our multi-scale mean teacher output.¹⁶ Our approach provides visually more consistent and less erroneous results than our baseline.

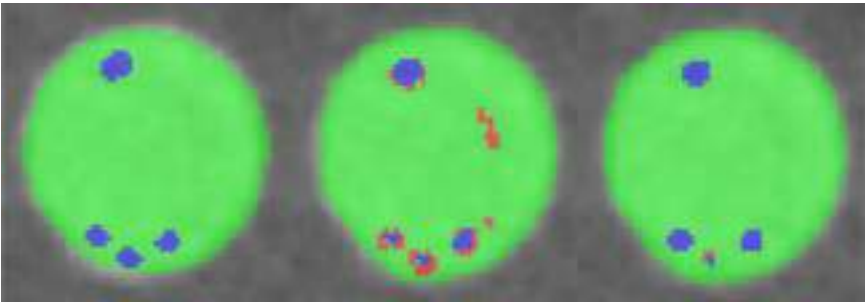


Fig. 11. A top-down perspective comparison between ground truth, baseline mean teacher, and multi-scale mean teacher inference results.¹⁶ Our regularised method provides better control over the training process and suppresses overall misclassification.

is not capable of handling relatively stronger augmentations (contrast and sharpening), whereas no such limitation is there for the teacher model which is capable of handling both strong and weak augmentations. For the overall model performance, we do not notice significant improvement upon applying these augmentations. A wider and more varied set of augmentations are needed to be experimented with before concluding the beneficial impact on the overall model performance.

4.4. 3D Metrology

We perform 3D metrology measurements as described in Section 3.3. We also perform a post-processing step to clean the inference using computer vision techniques as discussed in the previous section. We report our 3D metrology findings in Table 5. The results reflected in the table are averaged across all splits ranging from 1–100% labelled data and remaining data used as unlabelled data in a supervised-learning setting. We observe that aligning, cleaning, and performing neighbourhood clustering drastically improve the results when the inference has some serious flaws, such as situations when most of the cropped predicted bumps include false positives for copper pillars,

Table 5. We display the mean error for metrology features, such as bond line thickness, solder extrusion, Pad misalignment (in μm) between ground truth, V-Net, MT, ours, and post-processed predictions. We observe that our multi-scale mean teacher approach segments the die more accurately and our metrology package further improves the results significantly.

Metrology error	V-Net	MT	Ours	Post-processed
Memory Die				
Bond line thickness	5.654	2.19	1.41	1.41
Solder extrusion	3.924	3.30	3.27	2.53
Pad misalignment	2.286	2.12	0.91	0.91
Void-to-solder ratio	0.069	0.046	0.046	0.045
Logic Die				
Bond line thickness	4.55	3.57	1.63	1.45
Solder extrusion	0.54	1.36	1.00	0.68
Void-to-solder ratio	0.0029	1.20	0.0028	0.0029

pads, and solder in addition to neighbourhood components at the edges. Our final metrology results show a mean error of less than 1.41 μm for bond line thickness, 2.53 μm for solder extrusion, and 0.91 μm for pad misalignment when compared to the ground-truth labelled data.

4.5. 3D Defect Detection

We employ a ResNet architecture with a depth of 18 layers and utilise a sigmoid activation function to transform the model's output to binary prediction. Each defect-specific model underwent a training process for 20 epochs. Due to the limited quantity of labelled data, we implement an early stopping strategy to mitigate the risk of over-fitting. The model is trained using Adam optimiser, with a learning rate set at 0.0003. The batch size during this process is set to 16. To evaluate the model's performance, we establish benchmarks for all defects in memory and logic data. The assumption is that, due to the scarcity of input data and the absence of a quantitative prompt, the model might generate predictions with a significant bias. This bias was expected to be evident specifically on False Negatives and False Positives in the confusion matrix.

We also use the 3D metrology output from Section 4.4 to train the RBF kernel-based ML model to classify each individual bump as defective or non-defective for all individual defects — solder extrusion and void-to-solder ratio for logic and memory bumps, and pad misalignment for memory bumps only.

Once the output from metrology is received for each defect type, we first transform the training data to the standardised values by calculating the mean and standard deviation of the training samples. Using this same mean and standard deviation, we change the test data into standardised values as well. Finally, we tune the RBF kernel-based SVM model's hyperparameters using the training data. We performed a grid-based search over the possible values of the hyperparameters to find the most optimal value to be used for training the ML model.

We report our classification performances using generic 3D CNN and our metrology-based ML model in Table 6. We observe that our metrology-based approach provides very accurate results for defect detection compared to a generic 3D classification approach. This is

Table 6. We display the confusion matrix for defect detection based on a 3D CNN-based classification network and our metrology-based ML model. We display the numbers in percentages for ease of understanding. We observe that metrology-based detection is more accurate than classification-based detection. Additionally, we observe fewer escapes (false negatives) that are very expensive in the semicon industry.

Confusion matrix	3D CNN		Ours	
Memory Die				
Pad misalignment	15.3	0	46.2	0
	TP	FP	TP	FP
	38.5	46.2	7.6	46.2
	FN	TN	FN	TN
Solder extrusion	30.8	0	61.5	15.4
	30.8	38.4	0	23.1
Void-to-solder ratio	38.5	7.7	69.2	0
	30.8	23.0	0	30.8
Logic Die				
Solder extrusion	0	0	14.3	14.3
	57.1	42.9	0	71.6
Void-to-solder ratio	0	0	57.4	0
	14.3	85.7	0	42.6

because we are able to distill the relevant information of metrology to the ML model. In contrast, the CNN model needs to learn this information on its own and thus may require exponentially more data to reach the same level of accuracy.

4.6. Approach Robustness

Since our approach consists of multiple independent phases — object detection, cropping, segmentation, and 3D metrology, there is a distinct possibility of our multi-stage framework leading to the propagation of errors during the training of individual stages. We address this concern by ensuring that each module is as robust as possible to unexpected errors.

In particular, our object detection module accumulates the statistics of each detected 3D bump. If a particular bump exceeds the mean size by a big margin, for example, twice the size, we flag these

detections as erroneous and do not save these cropped regions for further processing.

We process each individual bump independently and do not assume that only one bump is present in each segment. In fact, often the neighbourhood bumps might be partially visible, especially when the solder bump collapses due to suboptimal packaging conditions. Our post-processing module ensures that any partially seen bumps on the edges of each crop are removed before performing 3D metrology to ensure the accuracy of the measurements.

All these steps ensure that any inaccuracies or unexpected outputs from one module are either properly addressed in the post-processing step of the current module or in the pre-processing step of the following module making our approach robust to such scenarios.

4.7. End-to-End Pipeline

Currently, our approach is based on offline processing. It involves processing the full 3D X-ray data slice by slice to detect the 3D boundaries of the individual bumps. For ease of use, we store these 3D individual bumps in a separate folder along with the locations in the original 3D scan. Thereafter, we process these bumps sequentially to perform 3D segmentation and metrology for further analysis.

In the future, for practical deployment, individual bump segmentation and metrology can be parallelised. We will also explore processing multiple bumps in a single 3D crop for overall faster processing of 3D segmentation and metrology. This way we aim to build an end-to-end pipeline for 3D defect detection and metrology for 3D X-ray scans.

This will enable end users to input the full 3D scan and be able to see the final 3D segmentation overlaid on the same scan rather than analysing individual bumps one by one.

5. Conclusion

In this study, we introduce a new framework for 3D metrology that uses cutting-edge 3D semi-supervised deep learning techniques for both object detection and semantic segmentation. We first describe how we detect objects from multiple views and merge the results

to improve bump detection performance. Next, we use 3D semi-supervised semantic segmentation to identify different components within individual structures, such as copper pillars, copper pads, voids, and solder regions. We also improve our semi-supervised semantic segmentation by introducing deep supervision and hierarchical consistent regularisation. We show that our contrastive learning coupled with the multi-scale Mean Teacher approach can achieve better results for HBM segmentation. Thereafter, our custom 3D metrology package uses sophisticated computer vision techniques to improve the segmentations further before performing 3D measurements of features, such as bond line thickness and pad misalignment. We also develop a machine learning model to use these measurements to automatically identify what defects may be present in each HBM.

In the future, we plan to incorporate active and contrastive learning methods for better data selection for the object detection step. This would allow us to focus on the most important features for object detection, rather than just visually informative features. We also plan to explore additional augmentations that can improve our semi-supervised segmentation method and incorporate balanced regularisation. We also explore employing black-box optimisation algorithms such as Bayesian optimisation to speed up the model's hyperparameter tuning process.⁵⁹ This approach could potentially enhance the efficiency of the model's training phase, bypassing the need for time-consuming manual tuning.

Our approach has the potential to revolutionise the way that semiconductor packages are inspected and analysed. By automating the process of defect detection, our approach can help improve the yield of HBM packages and reduce the cost of manufacturing. We hope that this paper serves as a foundation for future research on 3D deep learning for semiconductor packaging quality control.

Acknowledgements

This research is supported by Career Development Fund (Grant no. C210812046) which is supported and administered by the Agency for Science, Technology and Research (A*STAR). We would like to thank the Institute of Microelectronics (IME) for their invaluable support and domain expertise in designing, fabricating, and scanning

the HBMs. We are also grateful to Namrata Thakur and Li Yurui for their valuable contributions to training deep learning models and data management, results reporting, and their overall help on this chapter.

References

1. R. S. Pahwa, M. T. Lay Nwe, R. Chang, O. Z. Min, W. Jie, S. Gopalakrishnan, D. H. Soon Wee, R. Qin, V. S. Rao, H. Dai, J. Neumann, R. Pichumani, and T. Gregorich. Automated Attribute Measurements of Buried Package Features in 3D X-ray Images using Deep Learning. In *IEEE 71st Electronic Components and Technology Conference (ECTC)*, pp. 2196–2204 (2021). doi: 10.1109/ECTC32696.2021.00345.
2. R. S. Pahwa, K. Y. Chan, J. Bai, V. B. Saputra, M. N. Do, and S. Foong. Dense 3D Reconstruction for Visual Tunnel Inspection using Unmanned Aerial vehicle. In *IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*, pp. 7025–7032 (Nov, 2019). doi: 10.1109/IROS40897.2019.8967577.
3. R. S. Pahwa, J. Chao, J. Paul, Y. Li, M. T. Lay Nwe, S. Xie, A. James, A. Ambikapathi, Z. Zeng, and V. R. Chandrasekhar. Fault-Net: Faulty Rail-Valves Detection using Deep Learning and Computer Vision. In *IEEE Intelligent Transportation Systems Conference (ITSC)*, pp. 559–566 (2019). doi: 10.1109/ITSC.2019.8917062.
4. R. S. Pahwa, T. T. Ng, and M. N. Do. Tracking objects using 3D object proposals. In *Asia-Pacific Signal and Information Processing Association Annual Summit and Conference (APSIPA ASC)*, pp. 1657–1660 (Dec, 2017). doi: 10.1109/APSIPA.2017.8282298.
5. Z. Zhao, M. Xu, P. Qian, R. S. Pahwa, and R. Chang. DA-CIL: Towards domain adaptive class-incremental 3D object detection. In *British Machine Vision Conference (BMVC)* (2022). <https://bmvc2022.mpi-inf.mpg.de/0916.pdf>.
6. R. S. Pahwa, J. Lu, N. Jiang, T. T. Ng, and M. N. Do, Locating 3D Object Proposals: A Depth-Based Online Approach, *IEEE Transactions on Circuits and Systems for Video Technology* **28**(3), 626–639 (2018). doi: 10.1109/TCSVT.2016.2616143.
7. R. S. Pahwa, R. Chang, W. Jie, S. Satini, C. Viswanathan, D. Yiming, V. Jain, C. T. Pang, and W. K. Wah. A survey on object detection performance with different data distributions. In *International Conference on Social Robotics (ICSR)*, pp. 553–563, Springer International Publishing (2021). ISBN 978-3-030-90525-5.

8. T. L. Nwe, O. Z. Min, S. Gopalakrishnan, D. Lin, S. Prasad, S. Dong, Y. Li, and R. S. Pahwa. Improving 3D Brain Tumor Segmentation With Predict-Refine Mechanism Using Saliency And Feature Maps. In *IEEE International Conference on Image Processing (ICIP)*, pp. 2671–2675 (2020). doi: 10.1109/ICIP40778.2020.9190806.
9. W. Jie, R. Chang, X. Xun, C. Lile, C. S. Foo, and R. S. Pahwa. Improved bump detection and defect identification for hbms using refined machine learning approach. In *IEEE 24th Electronics Packaging Technology Conference (EPTC)*, pp. 848–853 (2022). doi: 10.1109/EPTC56328.2022.10013164.
10. R. S. Pahwa, S. W. Ho, R. Qin, R. Chang, O. Z. Min, W. Jie, V. S. Rao, T. L. Nwe, Y. Yang, J. T. Neumann, R. Pichumani, and T. Gregorich. Machine-Learning Based Methodologies for 3D X-Ray Measurement, Characterization and Optimization for Buried Structures in Advanced IC Packages. In *International Wafer Level Packaging Conference (IWLPC)*, pp. 01–07 (2020). doi: 10.23919/IWLPC52010.2020.9375903.
11. M. T. Lay Nwe, O. Z. Min, R. Chang, D. Lin, S. Prasad, S. Dong, Y. Li, and R. S. Pahwa. On the use of component structural characteristics for voxel segmentation in semicon 3D images. In *IEEE International Conference on Acoustics, Speech, and Signal Processing (ICASSP)* (2022).
12. R. S. Pahwa, T. L. Nwe, R. Chang, W. Jie, O. Z. Min, S. W. Ho, R. Qin, V. S. Rao, Y. Yang, J. T. Neumann, R. Pichumani, and T. Gregorich. Deep Learning Analysis of 3D X-ray Images for Automated Object Detection and Attribute Measurement of Buried Package Features. In *IEEE 22nd Electronics Packaging Technology Conference (EPTC)*, pp. 221–227 (2020). doi: 10.1109/EPTC50525.2020.9315043.
13. R. S. Pahwa, S. Gopalakrishnan, H. Su, O. E. Ping, H. Dai, D. H. S. Wee, R. Qin, and V. S. Rao. Automated Void Detection in TSVs from 2D X-Ray Scans using Supervised Learning with 3D X-Ray Scans. In *IEEE 71st Electronic Components and Technology Conference (ECTC)*, pp. 842–849 (2021). doi: 10.1109/ECTC32696.2021.00143.
14. R. S. Pahwa, R. Chang, W. Jie, X. Xun, O. Zaw Min, F. C. Sheng, C. Ser Choong, and V. S. Rao. Automated detection and segmentation of hbms in 3D x-ray images using semi-supervised deep learning. In *IEEE 72nd Electronic Components and Technology Conference (ECTC)*, pp. 1890–1897 (2022). doi: 10.1109/ECTC51906.2022.00297.
15. R. S. Pahwa, R. Chang, W. Jie, Z. Ziyuan, C. Lile, X. Xun, F. C. Sheng, C. Ser Choong, and V. S. Rao. 3D defect detection and metrology of hbms using semi-supervised deep learning. In *IEEE 73rd Electronic Components and Technology Conference (ECTC)* (2023).

16. J. Wang, R. Chang, Z. Zhao, and R. S. Pahwa, Robust detection, segmentation, and metrology of high bandwidth memory 3D scans using an improved semi-supervised deep learning approach, *Sensors* **23**(12), (2023). ISSN 1424-8220. doi: 10.3390/s23125470. <https://www.mdpi.com/1424-8220/23/12/5470>.
17. R. Chang, R. S. Pahwa, J. Wang, L. Chen, S. Satini, K. W. Wan, and D. Hsu. Creating semi-supervised learning-based adaptable object detection models for autonomous service robot. In *12th Conference on Learning Factories (CLF)* (2022). <http://dx.doi.org/10.2139/ssrn.4075994>.
18. H. Rahman, T. F. N. Bukht, A. Imran, J. Tariq, S. Tu, and A. Alzahrani, A deep learning approach for liver and tumor segmentation in ct images using resunet, *Bioengineering* **9**(8) (2022). ISSN 2306-5354. doi: 10.3390/bioengineering9080368.
19. C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao. Yolov7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors. In *2023 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 7464–7475 (2023). doi: 10.1109/CVPR52729.2023.00721.
20. S. Ren, K. He, R. Girshick, and J. Sun. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. In *Advances in Neural Information Processing Systems*, Vol. 28, pp. 91–99, Curran Associates, Inc. (2015).
21. J. Gao, J. Wang, S. Dai, L. Li, and R. Nevatia, NOTE-RCNN: noise tolerant ensemble RCNN for semi-supervised object detection, *CoRR*. abs/1812.00124 (2018). <http://arxiv.org/abs/1812.00124>.
22. J. Hoffman, S. Guadarrama, E. Tzeng, J. Donahue, R. B. Girshick, T. Darrell, and K. Saenko, LSDA: Large scale detection through adaptation, *CoRR*. abs/1407.5035 (2014). <http://arxiv.org/abs/1407.5035>.
23. Y. Liu, C. Ma, Z. He, C. Kuo, K. Chen, P. Zhang, B. Wu, Z. Kira, and P. Vajda, Unbiased teacher for semi-supervised object detection, *CoRR*. abs/2102.09480 (2021). <https://arxiv.org/abs/2102.09480>.
24. A. Tarvainen and H. Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *NIPS*, pp. 1195–1204 (2017).
25. T. Chen, S. Kornblith, M. Norouzi, and G. Hinton, A simple framework for contrastive learning of visual representations (2020).
26. K. He, H. Fan, Y. Wu, S. Xie, and R. Girshick, Momentum contrast for unsupervised visual representation learning (2020).
27. R. Hadsell, S. Chopra, and Y. LeCun. Dimensionality reduction by learning an invariant mapping. In *2006 IEEE Computer Society Conference on Computer Vision and Pattern Recognition (CVPR'06)*, Vol. 2, pp. 1735–1742 (2006). doi: 10.1109/CVPR.2006.100.

28. J. Donahue and K. Simonyan, Large scale adversarial representation learning, *CoRR*. abs/1907.02544 (2019). <http://arxiv.org/abs/1907.02544>.
29. W. Bai, O. Oktay, M. Sinclair, H. Suzuki, M. Rajchl, G. Tarroni, B. Glocker, A. King, P. M. Matthews, and D. Rueckert. Semi-supervised learning for network-based cardiac mr image segmentation. In *Medical Image Computing and Computer-Assisted Intervention (MICCAI)*, pp. 253–260, Springer International Publishing (2017). ISBN 978-3-319-66185-8.
30. J. Long, E. Shelhamer, and T. Darrell. Fully convolutional networks for semantic segmentation. In *2015 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 3431–3440 (2015). doi: 10.1109/CVPR.2015.7298965.
31. O. Ronneberger, P. Fischer, and T. Brox, U-net: Convolutional networks for biomedical image segmentation (2015).
32. F. Milletari, N. Navab, and S. Ahmadi. V-net: Fully convolutional neural networks for volumetric medical image segmentation. In *International Conference on 3D Vision (3DV)*, pp. 565–571 (2016). doi: 10.1109/3DV.2016.79.
33. Z. Li, K. Kamnitsas, and B. Glocker, Analyzing overfitting under class imbalance in neural networks for image segmentation, *IEEE Transactions on Medical Imaging* **40**, 1065–1077 (2021). doi: 10.1109/TMI.2020.3046692.
34. G. French, S. Laine, T. Aila, M. Mackiewicz, and G. Finlayson, Semi-supervised semantic segmentation needs strong, varied perturbations (2019). doi: 10.48550/ARXIV.1906.01916. <https://arxiv.org/abs/1906.01916>.
35. T.-Y. Lin, P. Goyal, R. Girshick, K. He, and P. Dollár, Focal loss for dense object detection (2017). doi: 10.48550/ARXIV.1708.02002. <https://arxiv.org/abs/1708.02002>.
36. W. Liu, Y. Wen, Z. Yu, and M. Yang, Large-margin softmax loss for convolutional neural networks (2016). doi: 10.48550/ARXIV.1612.02295. <https://arxiv.org/abs/1612.02295>.
37. W.-C. Hung, Y.-H. Tsai, Y.-T. Liou, Y.-Y. Lin, and M.-H. Yang. Adversarial learning for semi-supervised semantic segmentation. In *British Machine Vision Conference* (2018).
38. L. Yang, W. Zhuo, L. Qi, Y. Shi, and Y. Gao, St++: Make self-training work better for semi-supervised semantic segmentation (2022).
39. A. Tarvainen and H. Valpola. Mean teachers are better role models: Weight-averaged consistency targets improve semi-supervised deep learning results. In *Advances in Neural Information Processing Systems* (2017).

40. S. Li, C. Zhang, and X. He. Shape-aware semi-supervised 3D semantic segmentation for medical images. In *Medical Image Computing and Computer Assisted Intervention — MICCAI 2020*, pp. 552–561. Springer International Publishing (2020). doi: 10.1007/978-3-030-59710-8_54. https://doi.org/10.1007%2F978-3-030-59710-8_54.
41. L. Yu, S. Wang, X. Li, C.-W. Fu, and P.-A. Heng. Uncertainty-aware self-ensembling model for semi-supervised 3D left atrium segmentation. In *Medical Image Computing and Computer Assisted Intervention — MICCAI 2019: 22nd International Conference*, Shenzhen, China, October 13–17, 2019, Proceedings, Part II 22, pp. 605–613 (2019).
42. X. Li, L. Yu, H. Chen, C.-W. Fu, L. Xing, and P.-A. Heng, Transformation-consistent self-ensembling model for semisupervised medical image segmentation, *IEEE Transactions on Neural Networks and Learning Systems* **32**(2), 523–534 (2020).
43. X. Luo, J. Chen, T. Song, and G. Wang, Semi-supervised medical image segmentation through dual-task consistency (2020). doi: 10.48550/ARXIV.2009.04448. <https://arxiv.org/abs/2009.04448>.
44. S. Li, Z. Zhao, K. Xu, Z. Zeng, and C. Guan. Hierarchical consistency regularized mean teacher for semi-supervised 3D left atrium segmentation. In *2021 43rd Annual International Conference of the IEEE Engineering in Medicine & Biology Society (EMBC)*, pp. 3395–3398 (2021).
45. Z. Zhao, K. Xu, H. Z. Yeo, X. Yang, and C. Guan, Ms-mt: Multi-scale mean teacher with contrastive unpaired translation for cross-modality vestibular schwannoma and cochlea segmentation, *arXiv preprint arXiv:2303.15826* (2023).
46. Z. Zhao, Z. Zeng, K. Xu, C. Chen, and C. Guan, Dsal: Deeply supervised active learning from strong and weak labelers for biomedical image segmentation, *IEEE Journal of Biomedical and Health Informatics* **25**(10), 3744–3751 (2021).
47. Y. Wu, A. Kirillov, F. Massa, W.-Y. Lo, and R. Girshick. Detectron2. <https://github.com/facebookresearch/detectron2> (2019).
48. T. Lin, P. Dollár, R. B. Girshick, K. He, B. Hariharan, and S. Belongie, Feature pyramid networks for object detection, *CoRR*. abs/1612.03144 (2016). <http://arxiv.org/abs/1612.03144>.
49. K. He, X. Zhang, S. Ren, and J. Sun, Deep residual learning for image recognition, *CoRR*. abs/1512.03385 (2015). <http://arxiv.org/abs/1512.03385>.
50. M. Everingham, S. M. A. Eslami, L. Van Gool, C. K. I. Williams, J. Winn, and A. Zisserman, The pascal visual object classes challenge:

- A retrospective, *International Journal of Computer Vision* **111**(1), 98–136 (Jan, 2015).
51. T. Lin, M. Maire, S. Belongie, L. D. Bourdev, R. B. Girshick, J. Hays, P. Perona, D. Ramanan, P. Dollár, and C. L. Zitnick, Microsoft COCO: common objects in context, *CoRR*. abs/1405.0312 (2014). URL <http://arxiv.org/abs/1405.0312>.
 52. K. Hara, H. Kataoka, and Y. Satoh. Can spatiotemporal 3D cnns retrace the history of 2d cnns and imagenet? In *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 6546–6555 (2018).
 53. K. He, X. Zhang, S. Ren, and J. Sun. Deep residual learning for image recognition. In *The IEEE Conference on Computer Vision and Pattern Recognition (CVPR)* (June, 2016).
 54. C. Cortes and V. Vapnik, Support-vector networks **20**(3), 273–297 ISSN 1573-0565. doi: 10.1007/BF00994018.
 55. Y.-W. Chang, C.-J. Hsieh, K.-W. Chang, M. Ringgaard, and C.-J. Lin, Training and testing low-degree polynomial data mappings via linear SVM **11**(48), 1471–1490. <http://jmlr.org/papers/v11/chang10a.html>.
 56. D. D. Lewis and W. A. Gale. A sequential algorithm for training text classifiers. In *Proc. International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR)*, pp. 3–12, Springer-Verlag New York, Inc. (1994). ISBN 0-387-19889-X.
 57. T. Scheffer, C. Decomain, and S. Wrobel. Active hidden markov models for information extraction. In *Advances in Intelligent Data Analysis*, Springer Berlin Heidelberg (2001).
 58. B. Settles. Active learning literature survey. Computer Sciences Technical Report 1648, University of Wisconsin–Madison (2009).
 59. J. Wang, J. Xu, and X. Wang, Combination of hyperband and Bayesian optimization for hyperparameter optimization in deep learning (2018).

This page intentionally left blank

Index

A

adversarial attacks, 227–229, 244, 247
adversarial attacks on point clouds, 232
adversarial defence on point clouds, 233
adversarial defenders, 227, 229
anchored clustering, 261–262
asymmetric encoder–decoder design, 37
attentive pooling, 367
AugMono3D, 207, 213
average precision (AP), 214
axis-aligned bounding box (AABB), 210

B

benchmark 3D datasets, 6
birds-eye-view localisation (BEV), 214
BoxBath, 124

C

centroid-aware sampling, 382
Chamfer and Hausdorff distances, 184
Chamfer distance, 181
class-aware sampling, 381

CLIP-driven universal model, 422
CNN-based networks, 13
common types of 3D data, 3
computation and sparsity challenge, 283
continuous relaxation-based sampling (CRS), 363
contrastive coding, 418

D

2D CNNs, 240
3D CAD models, 186
3D CAD objects, 240
3D CNNs, 13, 232, 284, 299
3D CT and MRI segmentation, 413
3D HBM, 450
3D ResNet, 452
3D U-Net, 67, 70, 76
3D V-Net, 449
3D X-ray data, 466
3D X-ray machines, 440
3D X-ray microscopy (XRM) scanner, 453
3D XRM scan, 445
3D benchmark datasets, 5, 302, 313
3D bounding boxes, 445, 448, 458
3D building blocks, 346

- 3D building blocks (PVCNN and SPVNAS), 282–283
 - 3D bump, 465, 450
 - 3D contrastive coding, 418
 - 3D data representation, 4
 - 3D deep learning, 1, 10, 281–282, 421, 426, 441, 467
 - 3D deep learning methods, 284
 - 3D deep learning models, 401, 403, 415
 - 3D defect detection, 464, 466
 - 3D geometrical information, 67
 - 3D indoor scene segmentation, 306
 - 3D instance segmentation, 62, 79
 - 3D medical data, 400
 - 3D medical image analysis, 403–404, 408, 410
 - 3D medical images, 400, 403–404
 - 3D medical segmentation, 421
 - 3D metrology, 442, 448, 450, 463–464, 466–467
 - 3D neural architecture search (3D-NAS), 296, 313
 - 3D neural fields, 93
 - 3D object detection, 45, 202, 204–205, 228, 379
 - 3D object detection and segmentation, 439, 441
 - 3D object keypoints, 93
 - 3D object part segmentation, 305
 - 3D outdoor object detection, 311
 - 3D outdoor scene segmentation, 308, 310
 - 3D part segmentation, 303
 - 3D particle dynamics model, 119
 - 3D point cloud, 27–28, 58, 160, 209, 228, 230
 - 3D point cloud annotation, 60
 - 3D point cloud object detection, 381
 - 3D representation, 91, 149
 - 3D scene representations, 94
 - 3D scene understanding, 57
 - 3D segmentation, 442, 444, 466
 - 3D semantic segmentation, 46, 58, 61, 73
 - 3D semi-supervised deep learning, 466
 - 3D semi-supervised learning, 439
 - 3D semi-supervised semantic segmentation, 467
 - 3D slice-and-fuse, 448, 459
 - 3D super-voxels, 66
 - 3D transformers, 16, 415
 - 3D volumetric convolutions, 293
 - 3D-aware scene representations, 132, 134, 136, 144, 147, 149
 - 3D-structured neural field representations, 133
 - data movement and GEMM operations, 289
 - declarative defender, 234
 - deep neural networks, 417
 - degrees of freedom (DoF), 131
 - depth estimation, 204–205, 212
 - depth-aware monocular 3D object detector, 201
 - DiGS, 165, 182
 - dilated residual block, 368
 - Dirichlet energy, 176
 - discrete Variational AutoEncoder, 29
 - divergence loss, 165, 177, 194
 - divergence-guided shape, 160, 196
 - diverse 3D representations, 93
 - domain adaptation (DA), 258, 277
 - dynamic 3D modelling, 100
 - dynamic 3D particles, 116
 - dynamic 3D representations, 92
 - dynamic 3D scenes, 91, 93, 132, 150
 - dynamic interaction graphs, 118
 - dynamic kernel computation, 287
 - dynamic particle interaction networks (DPI-Nets), 117–118, 126, 129
 - dynamics model learning, 95–96
- E**
- edge devices, 340
 - efficient 3D neural networks, 290

efficient 3D representation learning, 403
 efficient point cloud processing, 357
 elastic network depth, 299
 emerging 3D deep learning applications, 21
 emerging applications of 3D data, 8
 explicit 3D structures, 101
 exponential moving average (EMA), 447, 460

F

farthest point sampling (FPS), 36, 331, 360
 fetch-on-demand dataflow, 318, 323, 328
 few-shot learning, 43
 fine-grained design space, 300
 FluidFall, 124
 FluidPour, 140
 FluidShake, 124, 130, 140
 fully supervised approaches, 73
 future directions of 3D deep learning, 18

G

gather-GEMM-scatter dataflow, 316, 320, 336
 generator-based sampling (GS), 362
 geometric constraints, 203
 geometric features, 203
 geometric information, 6
 geometric priors, 205
 geometrically motivated initialisation, 166, 196
 graph-based networks, 14, 232

H

hardware accelerator (PointAcc), 282–283, 346
 hardware-efficient 3D primitive, 290, 313
 hash table, 296
 Hausdorff distance, 181
 heatmap loss, 113

heuristic downsampling strategies, 362
 heuristic-based sampling, 359
 high bandwidth memory (HBM), 440–441, 453
 high-dimensional and complex, 7
 high-performance GPU library (TorchSparse), 282–283, 346

I

IA-SSD, 377, 384, 389, 391
 image-based networks, 231
 implicit GEMM dataflow, 323, 328
 implicit gradients, 231, 235, 244, 247–248
 implicit neural representations (INRs), 159–160, 162, 196
 implicit representations, 98
 instance-aware downsampling strategy, 377
 interaction networks (INs), 119, 127
 intersection over Union (IoU), 181
 inverse density importance sampling (IDIS), 361
 irregular memory access, 287

J

joint representations, 421

K

K -nearest neighbourhood, 36, 331
 kernel mapping, 331
 keypoint representations, 115
 KITTI, 311, 384
 KITTI 3D/BEV detection benchmark, 221
 KITTI dataset, 213, 228
 KL-divergence, 259, 262, 265

L

label-propagation, 60, 64
 language-image pre-training models, 421
 large-scale 3D point clouds, 357, 364

latent state, 98, 102–103
 lattice point classifier (LPC), 231, 239
 learning-based downsampling
 strategies, 362, 364
 learning-based sampling, 359
 LiDAR-based detection, 202
 local feature aggregation, 365
 local spatial encoding, 366

M

masked autoencoder (MAE), 419–420
 masked AutoEncoders, 29, 31
 masked volume inpainting, 419–420
 matrix multiplication, 289
 Mean Teacher, 444, 448
 Mean Teacher SSL training, 460
 median-squared Chamfer distance,
 187
 medical image analysis, 400, 403
 memory management unit, 333
 MFGI, 182
 MinkowskiEngine, 325, 328
 model predictive control (MPC), 102,
 104, 123, 138
 model predictive path integral
 (MPPI), 105, 139
 model-based optimisation, 97
 modelling of dynamic scenes, 99
 ModelNet40, 32, 41, 240
 ModelNet40-C, 269
 monocular 3D object detection,
 201–202, 205, 221
 multi-frequency geometric
 initialisation (MFGI), 170, 193
 multi-head self-attention (MSA), 407
 multi-layer perceptrons (MLPs), 136,
 161, 407
 multi-pass, 269
 multi-scale Mean Teacher, 450
 multi-scale V-Net, 445
 multi-view CNNs, 284
 multi-view data, 4
 multi-view time contrastive loss
 (TCN), 137

N

network architecture (PVCNN and
 SPVNAS), 346
 neural implicit representations, 99
 neural radiance fields (NeRF), 132,
 135
 novel-view synthesis, 207
 nuScenes, 310

O

object classification, 39
 one-pass inference, 269
 one-sided Chamfer distances, 190

P

part segmentation, 43
 particle dynamics model, 131
 particle-based simulator, 116
 particles, 93
 PartNet, 305
 permutohedral lattice, 233, 248
 photo-realistic images, 205
 pixel-level supervision, 212
 point cloud classification, 227
 point cloud downsampling, 358
 point cloud object detection,
 359
 point cloud sampling strategies, 359,
 390
 point cloud semantic segmentation,
 359
 point clouds, 3
 point-based feature extraction, 294
 point-based methods, 285
 point-based networks, 12, 61, 231
 point-based representation, 286
 point-BERT, 34–35, 42, 50
 point-clustering, 69
 point-MAE, 31, 35, 42, 50
 point-voxel convolution (PVConv),
 290, 292
 PointAcc, 329, 336, 346
 poisson disk sampling (PDS), 361
 policy gradient-based sampling
 (PGS), 363

position embedding, 37
 progressively depth shrinking, 301
 propagation networks, 120
 pseudo labels, 70, 264, 266
 pseudo LiDAR representations, 206

R

RandLA-Net, 364, 369, 371, 389, 391
 random sampling (RS), 359
 real-world applications, 17
 relation network, 60, 65–66, 82
 representation efficient 3D deep learning, 20
 resource efficient 3D deep learning, 21
 RGB-D scans, 5, 241
 RiceGrip, 124, 130
 RigidDrop, 140
 RigidStack, 140
 robust 3D deep learning, 21
 robust 3D point cloud classification, 248
 robust structured declarative classifiers, 248

S

S3DIS, 46, 71, 79, 82, 306, 374
 sample efficient 3D deep learning, 20
 sample-efficient learning, 101
 ScanNet, 45, 240
 ScanNet-v2, 71–72, 78, 82
 ScanObjectNN, 32, 40
 scene geometry, 205, 221
 scene reconstruction, 189
 scene representation, 119
 SDFs, 172, 174
 search space, 299
 second-order information, 173, 176
 second-order supervision, 175, 193
 segment anything model (SAM), 426
 self-supervised 3D keypoints, 102, 106
 self-supervised keypoints, 100
 self-supervised learning, 27–28, 33, 416–417
 self-training (ST), 60, 62, 69, 266–267
 Semantic3D, 372

SemanticKITTI, 308, 374
 semi-supervised learning, 458
 semi-supervised mean teacher, 445
 sequential test-time training (sTTT), 259, 261
 server-level products, 340
 shape space learning, 179
 ShapeNet, 39, 182, 186, 303
 signed distance function (SDF), 160
 SIRENs, 161, 165–166, 182
 source-free domain adaptation (SFDA), 258, 260
 sparse 3D CNNs, 301
 sparse annotations, 59, 63
 sparse coding, 235, 248
 sparse convolution, 287, 289, 295–296, 313, 319
 sparse point cloud computing, 346
 sparse point-voxel convolution (SPVConv), 291, 295
 sparse voxel-based methods, 287
 sparse voxelised representation, 296
 sparsity and irregularity, 282, 290
 spatial descriptor set, 104
 spatial expectation loss, 113
 SpConv, 325, 328
 specialised ASIC design, 340
 squared Chamfer, 181
 SRB dataset, 182, 192, 194
 stochastic gradient descent (SGD), 123, 138
 structured sparse coding, 231
 sTTT protocol, 268, 274
 super-voxel partition, 63, 72
 super-voxels, 68
 surface reconstruction benchmark (SRB), 161, 179, 182
 Swin UNet transformers, 408
 synthetic virtual-depth images, 217

T

task-agnostic downsampling, 377
 temporal consistency (TC), 264
 test-time adaptation (TTA), 258

test-time anchored clustering
(TTAC), 257, 259
test-time training (TTT), 257–258,
261
TorchSparse, 314, 316, 325, 328
transformer-based networks, 16
TTT protocols, 260, 269, 277
typical attackers for point clouds, 232
typical defenders for point clouds, 233

U

U-Net architecture, 211
uniform sampling, 300
unique characteristics of 3D point
clouds, 283
universal model, 423–424, 426
unordered and irregular, 7
unsupervised domain adaptation,
260, 262

V

viewpoint-invariant representations,
132–133, 135, 137, 149

virtual depths, 201, 204, 207, 221
virtual-depth images, 214
vision foundation models, 426
volumetric data, 4
volumetric quantification, 399
volumetric representation, 284
voxel grid sampling (VGS), 360
voxel-based feature aggregation,
293
voxel-based methods, 284
voxel-based networks, 61, 231
voxel-based representation, 284, 286
voxelisation and devoxelisation, 296

W

Waymo, 387
weakly and semi-supervised
approaches, 73
weakly supervised 3D scene
understanding, 63
weakly supervised 3D semantic
segmentation, 81
weight-sharing technique, 300